



Arm[®] Neoverse[™] N3 Core

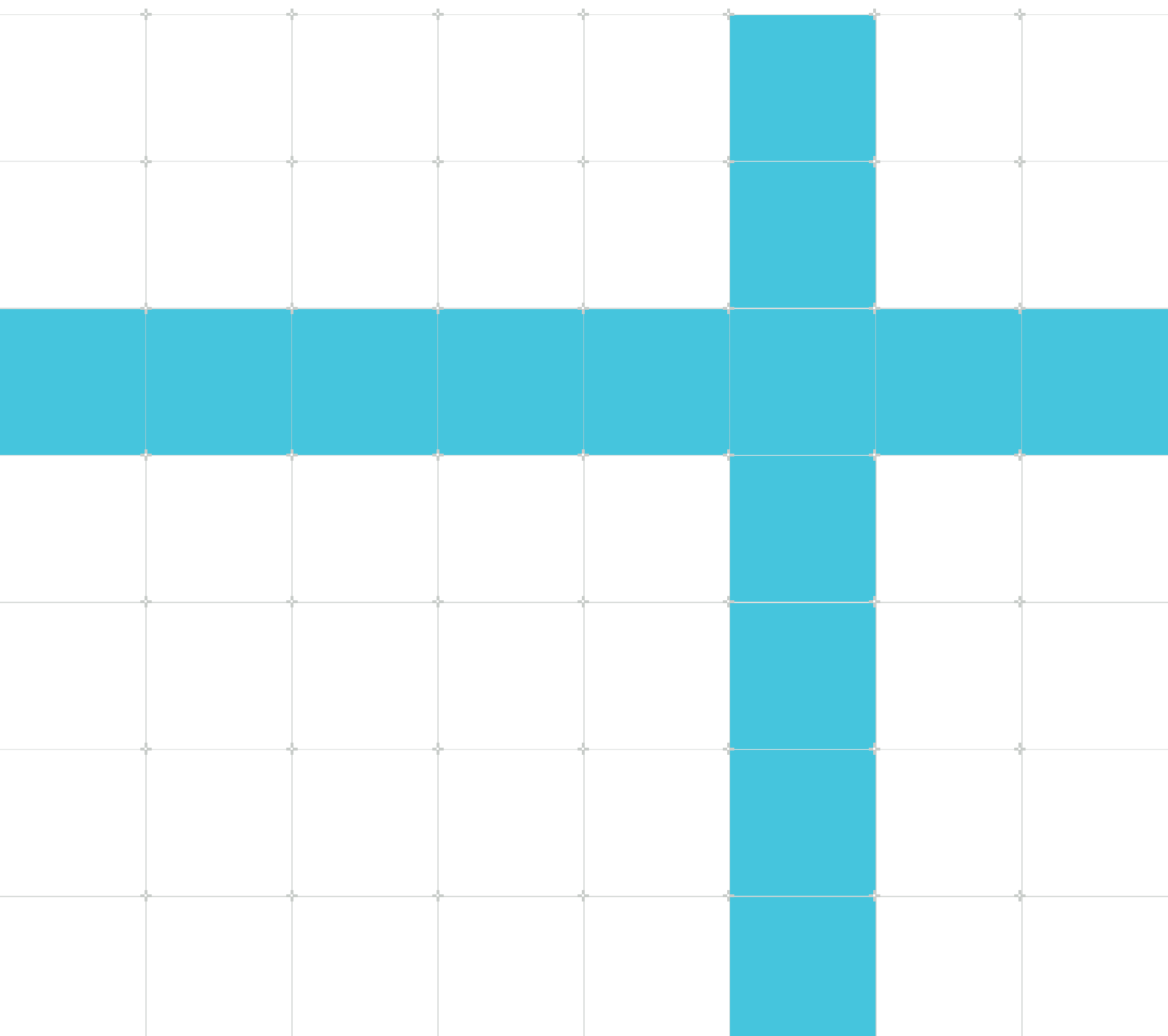
Revision r0p1

Technical Reference Manual

Non-Confidential

Issue 03

Copyright © 2023–2025 Arm Limited (or its affiliates). 107997_0001_03_en
All rights reserved.



Arm® Neoverse™ N3 Core Technical Reference Manual

This document is Non-Confidential.

Copyright © 2023–2025 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.

Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary Notice](#) found at the end of this document.

This document (107997_0001_03_en) was issued on 2025-02-25. There might be a later issue at <https://developer.arm.com/documentation/107997>

The product revision is r0p1.

See also: [Proprietary Notice](#) | [Product and document information](#) | [Useful resources](#)

Start Reading

If you prefer, you can skip to [the start of the content](#).

Intended audience

This manual is for system designers, system integrators, and programmers who are designing or programming a *System on Chip* (SoC) that uses an Arm core.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

This document includes language that can be offensive. We will replace this language in a future issue of this document.

To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on <https://support.developer.arm.com>.

To provide feedback on the document, fill the following survey: <https://developer.arm.com/documentation-feedback-survey>.

Contents

1. The Neoverse™ N3 core.....	17
1.1 Neoverse™ N3 core features.....	17
1.2 Neoverse™ N3 core configuration options.....	19
1.3 DSU-120 dependent features.....	20
1.4 Supported standards and specifications.....	21
1.5 Test features.....	28
1.6 Design tasks.....	28
1.7 Product revisions.....	29
2. Technical overview.....	30
2.1 Core components.....	30
2.2 Interfaces.....	34
2.3 Programmer's model.....	35
3. Clocks and resets.....	36
4. Power management.....	37
4.1 Voltage and power domains.....	37
4.2 Architectural clock gating modes.....	38
4.2.1 Wait for Interrupt and Wait for Event.....	39
4.2.2 Low-power state behavior considerations.....	39
4.3 Power control.....	40
4.4 Core power modes.....	40
4.4.1 On mode.....	42
4.4.2 Off mode.....	42
4.4.3 Emulated off mode.....	43
4.4.4 Full retention mode.....	43
4.4.5 Debug recovery mode.....	44
4.4.6 Warm reset mode.....	45
4.5 Performance and power management.....	45
4.5.1 Maximum Power Mitigation Mechanism.....	46
4.5.2 Performance Defined Power.....	46
4.5.3 Dispatch block.....	47

4.6 Neoverse™ N3 core powerup and powerdown sequence.....	47
4.6.1 Managing RAS fault and error interrupts during the core powerdown procedure.....	48
4.7 Debug over powerdown.....	48
5. Memory management.....	49
5.1 Memory Management Unit components.....	49
5.2 Translation Lookaside Buffer entry content.....	51
5.3 Translation Lookaside Buffer match process.....	51
5.4 Translation table walks.....	52
5.5 Hardware management of the Access flag and dirty state.....	53
5.6 Responses.....	53
5.7 Memory behavior and supported memory types.....	55
5.8 Page-based hardware attributes.....	56
6. L1 instruction memory system.....	58
6.1 L1 instruction cache behavior.....	58
6.2 L1 instruction cache Speculative memory accesses.....	59
6.3 Program flow prediction.....	59
6.4 Instruction cache hardware coherency.....	61
7. L1 data memory system.....	62
7.1 L1 data cache behavior.....	62
7.2 Write streaming mode.....	63
7.3 Atomic instruction implementation in the L1 data memory system.....	64
7.4 Internal exclusive monitor.....	64
7.5 Data prefetching.....	65
8. L2 memory system.....	67
8.1 L2 cache.....	67
8.2 Support for memory types.....	68
8.3 Transaction capabilities.....	68
9. Direct access to internal memory.....	70
9.1 L1 cache encodings.....	71
9.1.1 L1 RAM returned data.....	72
9.2 L2 cache encodings.....	73
9.2.1 L2 RAM returned data.....	74
9.3 L2 TLB encodings.....	74

9.3.1 L2 TLB RAM returned data.....	74
10. RAS Extension support.....	76
10.1 Cache protection behavior.....	76
10.2 Error containment.....	77
10.3 Fault detection and reporting.....	78
10.4 Error detection and reporting.....	78
10.4.1 Error reporting and performance monitoring.....	79
10.5 Error injection.....	79
10.6 AArch64 RAS registers.....	80
11. Utility bus.....	81
11.1 Base addresses for system components.....	81
12. GIC CPU interface.....	83
12.1 Disable the GIC CPU interface.....	83
12.2 AArch64 GIC system registers.....	84
13. Advanced SIMD and floating-point support.....	87
14. Scalable Vector Extensions support.....	88
15. System control.....	89
15.1 AArch64 Generic System Control registers.....	89
16. Random number generator support.....	97
16.1 AArch64 Random Number Control registers summary.....	98
17. Debug.....	99
17.1 Supported debug methods.....	101
17.2 Debug register interfaces.....	102
17.2.1 Core interfaces.....	102
17.2.2 Effects of resets on debug registers.....	103
17.2.3 Breakpoints and watchpoints.....	103
17.3 Debug events.....	103
17.4 Debug memory map and debug signals.....	104
17.5 ROM table.....	104
17.6 CoreSight component identification.....	105
17.7 CTI register identification values.....	105

17.8 AArch64 Debug registers.....	106
17.9 External Debug registers.....	107
17.10 External ROM table registers.....	109
18. Performance Monitors Extension support.....	110
18.1 Performance monitors events.....	110
18.2 Performance monitors interrupts.....	128
18.3 External register access permissions.....	128
18.4 AArch64 Performance Monitors registers.....	129
18.5 External PMU registers.....	132
19. Embedded Trace Extension support.....	136
19.1 Trace unit resources.....	137
19.2 Trace unit generation options.....	137
19.3 Reset the trace unit.....	138
19.4 Program and read the trace unit registers.....	139
19.5 Trace unit register interfaces.....	141
19.6 Interaction with the Performance Monitoring Unit and Debug.....	141
19.7 Embedded Trace Extension events.....	142
19.8 AArch64 Trace unit registers.....	142
19.9 External ETE registers.....	146
20. Trace Buffer Extension support.....	150
20.1 Program and read the trace buffer registers.....	150
20.2 Trace buffer register interface.....	150
21. Activity Monitors Extension support.....	151
21.1 Activity monitors access.....	151
21.2 Activity monitors counters.....	152
21.3 Activity monitors events.....	152
21.4 AArch64 Activity Monitors registers.....	153
21.5 External AMU registers.....	154
22. Statistical Profiling Extension support.....	157
22.1 Statistical Profiling Extension events packet.....	158
22.2 Statistical Profiling Extension data source packet.....	159
A. AArch64 registers.....	160

A.1 AArch64 Generic System Control registers summary.....	160
A.1.1 ACTLR_EL1, Auxiliary Control Register (EL1).....	167
A.1.2 AFSR0_EL1, Auxiliary Fault Status Register 0 (EL1).....	169
A.1.3 AFSR1_EL1, Auxiliary Fault Status Register 1 (EL1).....	171
A.1.4 AMAIR_EL1, Auxiliary Memory Attribute Indirection Register (EL1).....	174
A.1.5 LORID_EL1, LORegionID (EL1).....	177
A.1.6 IMP_CPUCFR_EL1, CPU Configuration Register.....	179
A.1.7 IMP_CPUACTLR_EL1, CPU Auxiliary Control Register.....	180
A.1.8 IMP_CPUACTLR2_EL1, CPU Auxiliary Control Register.....	182
A.1.9 IMP_CPUACTLR3_EL1, CPU Auxiliary Control Register.....	184
A.1.10 IMP_CPUACTLR4_EL1, CPU Auxiliary Control Register.....	186
A.1.11 IMP_CPUECTLR_EL1, CPU Extended Control Register.....	188
A.1.12 IMP_CPUECTLR2_EL1, CPU Extended Control Register.....	198
A.1.13 IMP_CPUBUSQOS_EL1, Quality of Service Bus Control Register, EL1.....	209
A.1.14 IMP_CPUPWRCTLR_EL1, CPU Power Control Register.....	211
A.1.15 IMP_CLUSTERACTLR_EL1, Cluster Auxiliary Control Register.....	214
A.1.16 IMP_ATCR_EL1, CPU Auxiliary Translation Control Register.....	216
A.1.17 IMP_CPUACTLR5_EL1, CPU Auxiliary Control Register.....	219
A.1.18 IMP_CPUACTLR6_EL1, CPU Auxiliary Control Register.....	221
A.1.19 IMP_CPUACTLR7_EL1, CPU Auxiliary Control Register.....	223
A.1.20 IMP_CPUACTLR8_EL1, CPU Auxiliary Control Register.....	225
A.1.21 AIDR_EL1, Auxiliary ID Register.....	226
A.1.22 RNDR, Random Number.....	228
A.1.23 RNDRRS, Reseeded Random Number.....	230
A.1.24 FPCR, Floating-point Control Register.....	231
A.1.25 ACTLR_EL2, Auxiliary Control Register (EL2).....	237
A.1.26 HACR_EL2, Hypervisor Auxiliary Control Register.....	240
A.1.27 AFSR0_EL2, Auxiliary Fault Status Register 0 (EL2).....	242
A.1.28 AFSR1_EL2, Auxiliary Fault Status Register 1 (EL2).....	244
A.1.29 AMAIR_EL2, Auxiliary Memory Attribute Indirection Register (EL2).....	247
A.1.30 IMP_ATCR_EL2, CPU Auxiliary Translation Control Register.....	249
A.1.31 IMP_AVTCR_EL2, CPU Auxiliary Virtualization Translation Control Register.....	253
A.1.32 ACTLR_EL3, Auxiliary Control Register (EL3).....	256
A.1.33 AFSR0_EL3, Auxiliary Fault Status Register 0 (EL3).....	259
A.1.34 AFSR1_EL3, Auxiliary Fault Status Register 1 (EL3).....	260
A.1.35 AMAIR_EL3, Auxiliary Memory Attribute Indirection Register (EL3).....	262

A.1.36 RMR_EL3, Reset Management Register (EL3).....	263
A.1.37 IMP_ISIDE_DATA0_EL3, RAMINDEX Instruction Data register 0.....	265
A.1.38 IMP_ISIDE_DATA1_EL3, RAMINDEX Instruction Data register 1.....	267
A.1.39 IMP_ISIDE_DATA2_EL3, RAMINDEX Instruction Data register 2.....	268
A.1.40 IMP_MMU_DATA0_EL3, RAMINDEX TLB Data register 0.....	269
A.1.41 IMP_MMU_DATA1_EL3, RAMINDEX TLB Data register 1.....	273
A.1.42 IMP_MMU_DATA2_EL3, RAMINDEX TLB Data register 2.....	279
A.1.43 IMP_DSIDE_DATA0_EL3, RAMINDEX L1D Data register 0.....	283
A.1.44 IMP_DSIDE_DATA1_EL3, RAMINDEX L1D Data register 1.....	285
A.1.45 IMP_DSIDE_DATA2_EL3, RAMINDEX L1D Data register 2.....	286
A.1.46 IMP_L2_DATA0_EL3, RAMINDEX L2 Data register 0.....	288
A.1.47 IMP_L2_DATA2_EL3, RAMINDEX L2 Data register 2.....	294
A.1.48 IMP_L2_DATA1_EL3, RAMINDEX L2 Data register 1.....	296
A.1.49 IMP_CPURNDBR_EL3, Random Number Base Register, EL3.....	297
A.1.50 IMP_CPURNDPEID_EL3, Random Number Packet Identification Register, EL3.....	299
A.1.51 IMP_CLUSTERCDBG_EL3, Cluster Cache Debug Register.....	301
A.1.52 IMP_ATCR_EL3, CPU Auxiliary Translation Control Register.....	303
A.1.53 IMP_CPUPSELR_EL3, Selected Instruction Private Control Register.....	305
A.1.54 IMP_CPUPCR_EL3, Selected Instruction Patch Control Register.....	307
A.1.55 IMP_CPUPOR_EL3, Selected Instruction Patch Opcode Register.....	309
A.1.56 IMP_CPUPMR_EL3, Selected Instruction Patch Mask Register.....	310
A.1.57 IMP_CPUPOR2_EL3, Selected Instruction Patch Opcode Register 2.....	312
A.1.58 IMP_CPUPMR2_EL3, Selected Instruction Patch Mask Register 2.....	314
A.1.59 IMP_CPUPFR_EL3, Selected Instruction Private Flag Register.....	315
A.2 AArch64 Special-purpose registers summary.....	317
A.2.1 IMP_CPUPPMPDPCR_EL1, Global PPMPPDP Configuration Register.....	318
A.2.2 IMP_CPUSYNCCASSISTCR_EL1, Synchronization Assist Configuration Register.....	321
A.2.3 IMP_CPUPPMCR_EL3, Global PPM Configuration Register.....	323
A.2.4 IMP_CPUPMCMCR_EL3, Global MPMM Configuration Register.....	325
A.3 AArch64 System instructions summary.....	327
A.3.1 RAMINDEX, RAMINDEX system instruction.....	327
A.4 AArch64 Performance Monitors registers summary.....	336
A.4.1 PMMIR_EL1, Performance Monitors Machine Identification Register.....	339
A.4.2 PMCR_ELO, Performance Monitors Control Register.....	341
A.4.3 PMCEID0_ELO, Performance Monitors Common Event Identification register 0.....	348
A.4.4 PMCEID1_ELO, Performance Monitors Common Event Identification register 1.....	356

A.5 AArch64 Other system control registers summary.....	363
A.6 AArch64 GIC system registers summary.....	364
A.6.1 ICC_AP0R0_EL1, Interrupt Controller Active Priorities Group 0 Registers.....	367
A.6.2 ICV_AP0R0_EL1, Interrupt Controller Virtual Active Priorities Group 0 Registers.....	370
A.6.3 ICC_AP1R0_EL1, Interrupt Controller Active Priorities Group 1 Registers.....	373
A.6.4 ICV_AP1R0_EL1, Interrupt Controller Virtual Active Priorities Group 1 Registers.....	377
A.6.5 ICC_CTLR_EL1, Interrupt Controller Control Register (EL1).....	381
A.6.6 ICV_CTLR_EL1, Interrupt Controller Virtual Control Register.....	386
A.6.7 ICH_VTR_EL2, Interrupt Controller VGIC Type Register.....	389
A.6.8 ICC_CTLR_EL3, Interrupt Controller Control Register (EL3).....	392
A.7 AArch64 Identification registers summary.....	396
A.7.1 MIDR_EL1, Main ID Register.....	398
A.7.2 MPIDR_EL1, Multiprocessor Affinity Register.....	400
A.7.3 REVIDR_EL1, Revision ID Register.....	402
A.7.4 MVFR0_EL1, AArch32 Media and VFP Feature Register 0.....	403
A.7.5 MVFR1_EL1, AArch32 Media and VFP Feature Register 1.....	405
A.7.6 MVFR2_EL1, AArch32 Media and VFP Feature Register 2.....	406
A.7.7 ID_AA64PFR0_EL1, AArch64 Processor Feature Register 0.....	408
A.7.8 ID_AA64PFR1_EL1, AArch64 Processor Feature Register 1.....	411
A.7.9 ID_AA64ZFR0_EL1, SVE Feature ID register 0.....	413
A.7.10 ID_AA64DFR0_EL1, AArch64 Debug Feature Register 0.....	416
A.7.11 ID_AA64DFR1_EL1, AArch64 Debug Feature Register 1.....	419
A.7.12 ID_AA64AFR0_EL1, AArch64 Auxiliary Feature Register 0.....	420
A.7.13 ID_AA64AFR1_EL1, AArch64 Auxiliary Feature Register 1.....	421
A.7.14 ID_AA64ISAR0_EL1, AArch64 Instruction Set Attribute Register 0.....	423
A.7.15 ID_AA64ISAR1_EL1, AArch64 Instruction Set Attribute Register 1.....	427
A.7.16 ID_AA64ISAR2_EL1, AArch64 Instruction Set Attribute Register 2.....	430
A.7.17 ID_AA64MMFR0_EL1, AArch64 Memory Model Feature Register 0.....	432
A.7.18 ID_AA64MMFR1_EL1, AArch64 Memory Model Feature Register 1.....	434
A.7.19 ID_AA64MMFR2_EL1, AArch64 Memory Model Feature Register 2.....	437
A.7.20 ID_AA64MMFR3_EL1, AArch64 Memory Model Feature Register 2.....	440
A.7.21 MPAMIDR_EL1, MPAM ID Register (EL1).....	442
A.7.22 CCSIDR_EL1, Current Cache Size ID Register.....	445
A.7.23 CLIDR_EL1, Cache Level ID Register.....	447
A.7.24 GMID_EL1, Multiple tag transfer ID register.....	451
A.7.25 CSSELR_EL1, Cache Size Selection Register.....	452

A.7.26 CTR_EL0, Cache Type Register.....	454
A.7.27 DCZID_EL0, Data Cache Zero ID register.....	457
A.8 AArch64 Generic Timer registers summary.....	459
A.9 AArch64 Debug registers summary.....	460
A.10 AArch64 Activity Monitors registers summary.....	462
A.10.1 AMCFGR_EL0, Activity Monitors Configuration Register.....	463
A.10.2 AMCGCR_EL0, Activity Monitors Counter Group Configuration Register.....	465
A.10.3 AMEVTYPER00_EL0, Activity Monitors Event Type Registers 0.....	467
A.10.4 AMEVTYPER01_EL0, Activity Monitors Event Type Registers 0.....	470
A.10.5 AMEVTYPER02_EL0, Activity Monitors Event Type Registers 0.....	472
A.10.6 AMEVTYPER03_EL0, Activity Monitors Event Type Registers 0.....	474
A.10.7 AMEVTYPER10_EL0, Activity Monitors Event Type Registers 1.....	477
A.10.8 AMEVTYPER11_EL0, Activity Monitors Event Type Registers 1.....	479
A.10.9 AMEVTYPER12_EL0, Activity Monitors Event Type Registers 1.....	481
A.11 AArch64 Trace unit registers summary.....	484
A.11.1 TRCIDR8, ID Register 8.....	488
A.11.2 TRCIMSPECO, IMP DEF Register 0.....	490
A.11.3 TRCIDR9, ID Register 9.....	492
A.11.4 TRCIDR10, ID Register 10.....	494
A.11.5 TRCIDR11, ID Register 11.....	495
A.11.6 TRCIDR12, ID Register 12.....	497
A.11.7 TRCIDR13, ID Register 13.....	499
A.11.8 TRCAUXCTLR, Auxiliary Control Register.....	500
A.11.9 TRCIDR0, ID Register 0.....	503
A.11.10 TRCIDR1, ID Register 1.....	506
A.11.11 TRCIDR2, ID Register 2.....	508
A.11.12 TRCIDR3, ID Register 3.....	510
A.11.13 TRCIDR4, ID Register 4.....	513
A.11.14 TRCIDR5, ID Register 5.....	516
A.11.15 TRCIDR6, ID Register 6.....	518
A.11.16 TRCIDR7, ID Register 7.....	520
A.11.17 TRCDEVID, Device Configuration Register.....	522
A.11.18 TRCCLAIMSET, Claim Tag Set Register.....	523
A.11.19 TRCCLAIMCLR, Claim Tag Clear Register.....	527
A.11.20 TRCDEVARCH, Device Architecture Register.....	530
A.12 AArch64 Memory Partitioning and Monitoring registers summary.....	532

A.12.1 MPAMVPMV_EL2, MPAM Virtual Partition Mapping Valid Register.....	533
A.12.2 MPAMVPM0_EL2, MPAM Virtual PARTID Mapping Register 0.....	535
A.12.3 MPAMVPM1_EL2, MPAM Virtual PARTID Mapping Register 1.....	537
A.13 AArch64 RAS registers summary.....	540
A.13.1 ERRIDR_EL1, Error Record ID Register.....	541
A.13.2 ERRSELR_EL1, Error Record Select Register.....	542
A.13.3 ERXFR_EL1, Selected Error Record Feature Register.....	545
A.13.4 ERXCTLR_EL1, Selected Error Record Control Register.....	549
A.13.5 ERXSTATUS_EL1, Selected Error Record Primary Status Register.....	555
A.13.6 ERXADDR_EL1, Selected Error Record Address Register.....	563
A.13.7 ERXPFGF_EL1, Selected Pseudo-fault Generation Feature register.....	566
A.13.8 ERXPFGCTL_EL1, Selected Pseudo-fault Generation Control register.....	571
A.13.9 ERXPFGCDN_EL1, Selected Pseudo-fault Generation Countdown register.....	576
A.13.10 ERXMISCO_EL1, Selected Error Record Miscellaneous Register 0.....	579
A.13.11 ERXMISC1_EL1, Selected Error Record Miscellaneous Register 1.....	586
A.13.12 ERXMISC2_EL1, Selected Error Record Miscellaneous Register 2.....	589
A.13.13 ERXMISC3_EL1, Selected Error Record Miscellaneous Register 3.....	591
A.14 AArch64 Statistical Profiling Extension registers summary.....	594
A.14.1 PMSNEVFR_EL1, Sampling Inverted Event Filter Register.....	595
A.14.2 PMSEVFR_EL1, Sampling Event Filter Register.....	598
A.14.3 PMSIDR_EL1, Sampling Profiling ID Register.....	601
A.14.4 PMBIDR_EL1, Profiling Buffer ID Register.....	604
A.15 AArch64 Trace Buffer Extension registers summary.....	606
A.15.1 TRBIDR_EL1, Trace Buffer ID Register.....	606
B. External registers.....	609
B.1 External MPMM registers summary.....	609
B.1.1 CPUPPMCR, Global PPM Configuration Register.....	609
B.1.2 CPUMPMPCR, Global MPMM Configuration Register.....	611
B.1.3 CPUPMPDPCR, Global PPMPDP Configuration Register.....	613
B.2 External CTI registers summary.....	615
B.3 External Debug registers summary.....	616
B.3.1 EDRCR, External Debug Reserve Control Register.....	618
B.3.2 EDACR, External Debug Auxiliary Control Register.....	620
B.3.3 EDPRCR, External Debug Power/Reset Control Register.....	621
B.3.4 MIDR_EL1, Main ID Register.....	623

B.3.5 EDPFR, External Debug Processor Feature Register.....	624
B.3.6 EDDFR, External Debug Feature Register.....	627
B.3.7 EDDEVARCH, External Debug Device Architecture register.....	629
B.3.8 EDDEVID2, External Debug Device ID register 2.....	631
B.3.9 EDDEVID1, External Debug Device ID register 1.....	632
B.3.10 EDDEVID, External Debug Device ID register 0.....	634
B.3.11 EDDEVTYPE, External Debug Device Type register.....	635
B.3.12 EDPIDR4, External Debug Peripheral Identification Register 4.....	636
B.3.13 EDPIDR0, External Debug Peripheral Identification Register 0.....	638
B.3.14 EDPIDR1, External Debug Peripheral Identification Register 1.....	639
B.3.15 EDPIDR2, External Debug Peripheral Identification Register 2.....	641
B.3.16 EDPIDR3, External Debug Peripheral Identification Register 3.....	642
B.3.17 EDCIDR0, External Debug Component Identification Register 0.....	644
B.3.18 EDCIDR1, External Debug Component Identification Register 1.....	645
B.3.19 EDCIDR2, External Debug Component Identification Register 2.....	646
B.3.20 EDCIDR3, External Debug Component Identification Register 3.....	648
B.4 External PMU registers summary.....	649
B.4.1 PMPCSSR, Snapshot Program Counter Sample Register.....	652
B.4.2 PMCIDSSR, Snapshot CONTEXTIDR_EL1 Sample Register.....	654
B.4.3 PMCID2SSR, Snapshot CONTEXTIDR_EL2 Sample Register.....	655
B.4.4 PMSSSR, PMU Snapshot Status Register.....	656
B.4.5 PMCCNTSR, PMU Cycle Counter Snapshot Register.....	657
B.4.6 PMEVCNTSR0, PMU Event Counter Snapshot Register.....	659
B.4.7 PMEVCNTSR1, PMU Event Counter Snapshot Register.....	660
B.4.8 PMEVCNTSR2, PMU Event Counter Snapshot Register.....	661
B.4.9 PMEVCNTSR3, PMU Event Counter Snapshot Register.....	662
B.4.10 PMEVCNTSR4, PMU Event Counter Snapshot Register.....	663
B.4.11 PMEVCNTSR5, PMU Event Counter Snapshot Register.....	665
B.4.12 PMEVCNTSR6, PMU Event Counter Snapshot Register.....	666
B.4.13 PMEVCNTSR7, PMU Event Counter Snapshot Register.....	667
B.4.14 PMEVCNTSR8, PMU Event Counter Snapshot Register.....	668
B.4.15 PMEVCNTSR9, PMU Event Counter Snapshot Register.....	669
B.4.16 PMEVCNTSR10, PMU Event Counter Snapshot Register.....	671
B.4.17 PMEVCNTSR11, PMU Event Counter Snapshot Register.....	672
B.4.18 PMEVCNTSR12, PMU Event Counter Snapshot Register.....	673
B.4.19 PMEVCNTSR13, PMU Event Counter Snapshot Register.....	674

B.4.20 PMEVCNTR14, PMU Event Counter Snapshot Register.....	675
B.4.21 PMEVCNTR15, PMU Event Counter Snapshot Register.....	677
B.4.22 PMEVCNTR16, PMU Event Counter Snapshot Register.....	678
B.4.23 PMEVCNTR17, PMU Event Counter Snapshot Register.....	679
B.4.24 PMEVCNTR18, PMU Event Counter Snapshot Register.....	680
B.4.25 PMEVCNTR19, PMU Event Counter Snapshot Register.....	681
B.4.26 PMCFGR, Performance Monitors Configuration Register.....	683
B.4.27 PMCR_ELO, Performance Monitors Control Register.....	685
B.4.28 PMCEID0, Performance Monitors Common Event Identification register 0.....	689
B.4.29 PMCEID1, Performance Monitors Common Event Identification register 1.....	693
B.4.30 PMCEID2, Performance Monitors Common Event Identification register 2.....	697
B.4.31 PMCEID3, Performance Monitors Common Event Identification register 3.....	701
B.4.32 PMSSCR, PMU Snapshot Capture Register.....	705
B.4.33 PMMIR, Performance Monitors Machine Identification Register.....	706
B.4.34 PMDEVARCH, Performance Monitors Device Architecture register.....	708
B.4.35 PMDEVID, Performance Monitors Device ID register.....	710
B.4.36 PMDEVTYPE, Performance Monitors Device Type register.....	711
B.4.37 PMPIDR4, Performance Monitors Peripheral Identification Register 4.....	712
B.4.38 PMPIDR0, Performance Monitors Peripheral Identification Register 0.....	714
B.4.39 PMPIDR1, Performance Monitors Peripheral Identification Register 1.....	715
B.4.40 PMPIDR2, Performance Monitors Peripheral Identification Register 2.....	717
B.4.41 PMPIDR3, Performance Monitors Peripheral Identification Register 3.....	718
B.4.42 PMCIDR0, Performance Monitors Component Identification Register 0.....	720
B.4.43 PMCIDR1, Performance Monitors Component Identification Register 1.....	721
B.4.44 PMCIDR2, Performance Monitors Component Identification Register 2.....	723
B.4.45 PMCIDR3, Performance Monitors Component Identification Register 3.....	724
B.5 External AMU registers summary.....	725
B.5.1 AMEVTYPE00, Activity Monitors Event Type Registers 0.....	727
B.5.2 AMEVTYPE01, Activity Monitors Event Type Registers 0.....	729
B.5.3 AMEVTYPE02, Activity Monitors Event Type Registers 0.....	730
B.5.4 AMEVTYPE03, Activity Monitors Event Type Registers 0.....	732
B.5.5 AMEVTYPE10, Activity Monitors Event Type Registers 1.....	734
B.5.6 AMEVTYPE11, Activity Monitors Event Type Registers 1.....	736
B.5.7 AMEVTYPE12, Activity Monitors Event Type Registers 1.....	737
B.5.8 AMCGCR, Activity Monitors Counter Group Configuration Register.....	739
B.5.9 AMCFGR, Activity Monitors Configuration Register.....	741

B.5.10 AMIIDR, Activity Monitors Implementation Identification Register.....	742
B.5.11 AMDEVARCH, Activity Monitors Device Architecture Register.....	744
B.5.12 AMDEVTYPE, Activity Monitors Device Type Register.....	745
B.5.13 AMPIDR4, Activity Monitors Peripheral Identification Register 4.....	746
B.5.14 AMPIDR0, Activity Monitors Peripheral Identification Register 0.....	748
B.5.15 AMPIDR1, Activity Monitors Peripheral Identification Register 1.....	749
B.5.16 AMPIDR2, Activity Monitors Peripheral Identification Register 2.....	750
B.5.17 AMPIDR3, Activity Monitors Peripheral Identification Register 3.....	752
B.5.18 AMCIDR0, Activity Monitors Component Identification Register 0.....	753
B.5.19 AMCIDR1, Activity Monitors Component Identification Register 1.....	754
B.5.20 AMCIDR2, Activity Monitors Component Identification Register 2.....	756
B.5.21 AMCIDR3, Activity Monitors Component Identification Register 3.....	757
B.6 External ETE registers summary.....	758
B.6.1 TRCAUXCTLR, Auxiliary Control Register.....	761
B.6.2 TRCIDR8, ID Register 8.....	762
B.6.3 TRCIDR9, ID Register 9.....	764
B.6.4 TRCIDR10, ID Register 10.....	765
B.6.5 TRCIDR11, ID Register 11.....	766
B.6.6 TRCIDR12, ID Register 12.....	767
B.6.7 TRCIDR13, ID Register 13.....	768
B.6.8 TRCIMSPEC0, IMP DEF Register 0.....	770
B.6.9 TRCIDR0, ID Register 0.....	771
B.6.10 TRCIDR1, ID Register 1.....	773
B.6.11 TRCIDR2, ID Register 2.....	775
B.6.12 TRCIDR3, ID Register 3.....	777
B.6.13 TRCIDR4, ID Register 4.....	779
B.6.14 TRCIDR5, ID Register 5.....	781
B.6.15 TRCIDR6, ID Register 6.....	783
B.6.16 TRCIDR7, ID Register 7.....	784
B.6.17 TRCITCTRL, Integration Mode Control Register.....	785
B.6.18 TRCCLAIMSET, Claim Tag Set Register.....	787
B.6.19 TRCCLAIMCLR, Claim Tag Clear Register.....	789
B.6.20 TRCDEVARCH, Device Architecture Register.....	791
B.6.21 TRCDEVID2, Device Configuration Register 2.....	793
B.6.22 TRCDEVID1, Device Configuration Register 1.....	794
B.6.23 TRCDEVID, Device Configuration Register.....	795

B.6.24 TRCDEVTYPE, Device Type Register.....	796
B.6.25 TRCPIDR4, Peripheral Identification Register 4.....	798
B.6.26 TRCPIDR5, Peripheral Identification Register 5.....	800
B.6.27 TRCPIDR6, Peripheral Identification Register 6.....	801
B.6.28 TRCPIDR7, Peripheral Identification Register 7.....	802
B.6.29 TRCPIDR0, Peripheral Identification Register 0.....	803
B.6.30 TRCPIDR1, Peripheral Identification Register 1.....	805
B.6.31 TRCPIDR2, Peripheral Identification Register 2.....	806
B.6.32 TRCPIDR3, Peripheral Identification Register 3.....	808
B.6.33 TRCCIDR0, Component Identification Register 0.....	809
B.6.34 TRCCIDR1, Component Identification Register 1.....	811
B.6.35 TRCCIDR2, Component Identification Register 2.....	812
B.6.36 TRCCIDR3, Component Identification Register 3.....	813
B.7 External RAS registers summary.....	815
B.7.1 ERR0FR, Error Record Feature Register.....	816
B.7.2 ERR0CTLR, Error Record Control Register.....	820
B.7.3 ERR0STATUS, Error Record Primary Status Register.....	822
B.7.4 ERR0ADDR, Error Record Address Register.....	832
B.7.5 ERR0MISC0, Error Record Miscellaneous Register 0.....	833
B.7.6 ERR0MISC1, Error Record Miscellaneous Register 1.....	839
B.7.7 ERR0MISC2, Error Record Miscellaneous Register 2.....	842
B.7.8 ERR0MISC3, Error Record Miscellaneous Register 3.....	844
B.7.9 ERR0PFGF, Pseudo-fault Generation Feature Register.....	846
B.7.10 ERR0PFGCTL, Pseudo-fault Generation Control Register.....	849
B.7.11 ERR0PFGCDN, Pseudo-fault Generation Countdown Register.....	852
B.7.12 ERRGSR, Error Group Status Register.....	853
B.7.13 ERRIIDR, Implementation Identification Register.....	854
B.7.14 ERRDEVAFF, Device Affinity Register.....	856
B.7.15 ERRDEVARCH, Device Architecture Register.....	858
B.7.16 ERRDEVID, Device Configuration Register.....	859
B.7.17 ERRPIDR4, Peripheral Identification Register 4.....	861
B.7.18 ERRPIDR0, Peripheral Identification Register 0.....	862
B.7.19 ERRPIDR1, Peripheral Identification Register 1.....	864
B.7.20 ERRPIDR2, Peripheral Identification Register 2.....	865
B.7.21 ERRPIDR3, Peripheral Identification Register 3.....	867
B.7.22 ERRCIDR0, Component Identification Register 0.....	868

B.7.23 ERRCIDR1, Component Identification Register 1.....	869
B.7.24 ERRCIDR2, Component Identification Register 2.....	871
B.7.25 ERRCIDR3, Component Identification Register 3.....	872
B.8 External ROM table registers summary.....	873
B.8.1 ROMENTRY0, Class 0x9 ROM Table Entries.....	874
B.8.2 ROMENTRY1, Class 0x9 ROM Table Entries.....	876
B.8.3 ROMENTRY2, Class 0x9 ROM Table Entries.....	878
B.8.4 ROMENTRY3, Class 0x9 ROM Table Entries.....	880
B.8.5 DEVARCH, Device Architecture Register.....	883
B.8.6 PIDR4, Peripheral Identification Register 4.....	884
B.8.7 PIDR0, Peripheral Identification Register 0.....	885
B.8.8 PIDR1, Peripheral Identification Register 1.....	886
B.8.9 PIDR2, Peripheral Identification Register 2.....	887
B.8.10 PIDR3, Peripheral Identification Register 3.....	889
B.8.11 CIDR0, Component Identification Register 0.....	890
B.8.12 CIDR1, Component Identification Register 1.....	891
B.8.13 CIDR2, Component Identification Register 2.....	892
B.8.14 CIDR3, Component Identification Register 3.....	893
Proprietary Notice.....	895
Product and document information.....	897
Product status.....	897
Revision history.....	897
Conventions.....	899
Useful resources.....	902

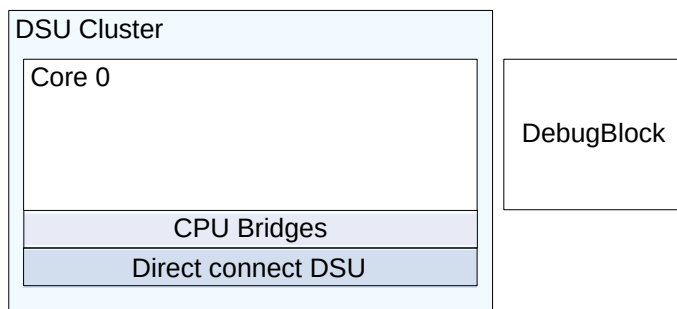
1. The Neoverse™ N3 core

The Neoverse™ N3 core is a balanced-performance, low-power, and constrained area product that implements the Arm®v9.2-A architecture. The Arm®v9.2-A architecture extends the architecture defined in the Arm®v8-A architectures up to Arm®v8.7-A.

The Neoverse™ N3 core is implemented inside a DSU-120 cluster and is always connected to the DSU-120. The Neoverse™ N3 core supports Direct connect only. For more information on the DSU Direct connect, see the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).

The following figure shows an example configuration with one Neoverse™ N3 core that is implemented as a single core in a DSU cluster which is configured for Direct connect, without the L3 cache, snoop filter, or *Snoop Control Unit* (SCU) logic present.

Figure 1-1: Neoverse™ N3 example configuration



Note

- This manual applies to the Neoverse™ N3 core only. Read this manual together with the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for detailed information about the DSU-120.
- This manual does not provide a complete list of registers. Read this manual together with the [Arm® Architecture Reference Manual for A-profile architecture](#).

1.1 Neoverse™ N3 core features

The Neoverse™ N3 core is used in a standalone DSU configuration, which is configured in Direct connect mode.

Regardless of the cluster configuration, the Neoverse™ N3 core always has the same features as described in the following lists.

Core features

- Implementation of the Arm®v9.2-A A64 instruction set
- AArch64 Execution state at all Exception levels, ELO to EL3
- *Memory Management Unit* (MMU)
- 48-bit *Physical Address* (PA) and 48-bit *Virtual Address* (VA)
- *Generic Interrupt Controller* (GIC) CPU interface to connect to an external interrupt Distributor
- Generic Timers interface that supports 64-bit count input from an external system counter
- Implementation of the *Reliability, Availability, and Serviceability* (RAS) Extension
- Implementation of the *Scalable Vector Extension* (SVE) with a 128-bit vector length and *Scalable Vector Extension 2* (SVE2)
- Integrated execution unit with *Advanced Single Instruction Multiple Data* (SIMD) and floating-point support
- *Activity Monitoring Unit* (AMU)
- Support for *Random Number Generator* (RNG)
- Support for the optional *Cryptographic Extension*



The Cryptographic Extension is licensed separately.

Cache features

- Separate L1 data and instruction caches
- Private, unified data and instruction L2 cache
- Optional error protection with parity or *Error Correcting Code* (ECC) allowing:
 - *Single Error Correction and Double Error Detection* (SECCDED) on L1 data cache and L2 cache
 - *Single Error Detection* (SED) on L1 instruction cache and L2 *Translation Lookaside Buffer* (TLB)
- Support for *Memory System Resource Partitioning and Monitoring* (MPAM)
- Support for instruction cache hardware coherency

Debug features

- Arm®v9.2-A debug logic
- *Performance Monitoring Unit* (PMU)
- *Embedded Trace Extension* (ETE)
- *TRace Buffer Extension* (TRBE)
- *Statistical Profiling Extension* (SPE)
- Optional *Embedded Logic Analyzer* (ELA), ELA-600



The ELA-600 is licensed separately.

Related information

[2. Technical overview](#) on page 30

1.2 Neoverse™ N3 core configuration options

You can choose the options that fit your implementation needs at build-time configuration.

You can configure your Neoverse™ N3 core implementation using the following options:

PMU event counters

You can configure the number of PMU event counters to be 6 or 20.

Cryptographic Extension

You can configure your implementation with or without the Cryptographic Extension. The Cryptographic Extension is licensed separately.

L1 data cache size

You can configure the L1 data cache to be 32KB or 64KB.

L1 instruction cache size

You can configure the L1 instruction cache to be 32KB or 64KB.

L2 Data RAM ECC granule

You can configure the L2 Data RAM ECC granule to be 128 bits or 256 bits.

L2 cache size

You can configure the L2 cache to be 128KB, 256KB, 512KB, 1024KB, or 2048KB.

CoreSight™ *Embedded Logic Analyzer (ELA)*

Support for the CoreSight™ ELA-600 is optional and available as a separately licensable product.

ELA

You can configure the ELA to be TRUE or FALSE.

ELA ATB FIFO

You can configure the ELA ATB FIFO depth when ELA-600 is implemented. For

`ELA_ATB_FIFO_DEPTH = 4, 8, 16, 32, 64.`

Timing closure

You can configure the L2 data cache RAMs timing behavior.

Reduced area configuration

You can configure the core to select the use of a variant of the product that has reduced logic and RAM structures.

See *RTL configuration process* in the *Arm® Neoverse™ N3 Core Configuration and Integration Manual* for detailed configuration options and guidelines.

1.3 DSU-120 dependent features

Some DSU-120 features and behaviors depend on whether your licensed core supports a particular feature.

The following table describes which DSU-120 dependent features are supported in your Neoverse™ N3 core.

Table 1-1: Neoverse™ N3 core features that have a dependency on the DSU-120

Feature	Supported in the Neoverse™ N3 core	Dependency on the DSU-120
Direct connect	Only supports Direct connect	Direct connect support at the DSU-120 cluster level only applies when your licensed core also supports Direct connect.
Core included in a complex	No	Affects the DSU-120 cluster configuration and external signals.
Cryptographic Extension	Yes, as an option	Affects the external signals of the DSU-120. For more information on MPMM, PDP, and the dispatch block signal, see 4.5 Performance and power management on page 45.
Maximum Power Mitigation Mechanism (MPMM)	Yes	
Performance Defined Power (PDP) feature	Yes	
DISPBLKy	Yes	
Dispatch block signal		
Statistical Profiling Extension (SPE) architecture	Yes	
Physical Address (PA) width	48-bit	Affects the CHI master and AXI master port bus widths. For more details, see the following chapters of the <i>Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual</i> : • <i>CHI master interface</i> • <i>AXI master interface</i>



- The Cryptographic Extension is supplied under a separate license.

1.4 Supported standards and specifications

The Neoverse™ N3 core complies with the Arm®v9.2-A architecture. The Arm®v9.2-A architecture extends the architecture defined in the Arm®v8-A architectures up to Arm®v8.7-A. The Neoverse™ N3 core also complies with specific Arm®v8-A architecture extensions and supports interconnect, interrupt, timer, debug, and trace architectures.

The Neoverse™ N3 core supports AArch64 at Exception levels EL0 to EL3.

Not all architectural features are implemented in the Neoverse™ N3 core. The following tables show the implementation status of Arm®v8-A and Arm®v9-A features supported by the Neoverse™ N3 core. There is a separate table for each version of the Arm®v8-A and Arm®v9-A architectures.



- Not all Arm®v8-A and Arm®v9-A architectural features are listed in the following tables. For more information on all the architectural features, see the [Arm® Architecture Reference Manual for A-profile architecture](#).
- The Neoverse™ N3 core is compatible with the architecture for the DSU-120. See the *Supported standards and specifications* section in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for a list of specific architectural versions and features supported by the DSU-120.

Table 1-2: Implementation status of the Arm®v8.0-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_PCSRv8	No	PC Sample-based Profiling Extension
Cryptographic Extension	Yes Configurable	For more information and additional cryptographic register descriptions, see the Arm® Neoverse™ N3 Core Cryptographic Extension Technical Reference Manual . This extension is licensed separately and access to the documentation is restricted by contract with Arm.
FEAT_SHA1	Yes	Advanced SIMD SHA1 instructions
FEAT_SHA256	Configurable	Advanced SIMD SHA256 instructions
FEAT_AES		Advanced SIMD AES instructions
FEAT_PMULL	Supported as part of the Arm®v8-A Cryptographic Extension	Advanced SIMD PMULL instructions
FEAT_DoubleLock	No	Double Lock
FEAT_CP15SDISABLE2	No	CP15DISABLE2
FEAT_FP	Yes	Floating point extension
FEAT_AdvSIMD	Yes	Advanced SIMD Extension For more information and register descriptions, see 13. Advanced SIMD and floating-point support on page 87.
FEAT_CRC32	Yes	CRC32 instructions

Feature	Implemented	Description
FEAT_PMUv3	Yes	PMU extension version 3
FEAT_nTLBPA	Yes	No intermediate caching by output address in TLB
FEAT_SB	Yes	Speculation barrier
FEAT_SSBS	Yes	Speculative Store Bypass Safe Instruction
FEAT_SSBS2	Yes	MRS and MSR instructions for SSBS version 2
FEAT_CSV2	Yes	Cache Speculation Variant 2
FEAT_CSV2_1p1	No	Cache Speculation Variant 2 version 1.1
FEAT_CSV2_1p2	No	Cache Speculation Variant 2 version 1.2
FEAT_CSV2_2	Yes	Cache Speculation Variant 2 version 2
FEAT_CSV3	Yes	Cache Speculation Variant 3
FEAT_SPECRES	Yes	Speculation restriction instructions
FEAT_DGH	Yes	Data Gathering Hint
FEAT_ETS	Yes	Enhanced Translation Synchronization
FEAT_ECBHB	Yes	<i>Exploitative Control using Branch History Buffer</i> information between exception levels The branch history information created in a context before an exception to a higher exception level, using AArch64, cannot be used by code before that exception. This prevents exploitative control of the execution of any indirect branches in code in a different context after the exception.

Table 1-3: Implementation status of the Arm®v8.1-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_LSE	Yes	Large System Extensions
FEAT_RDM	Yes	Rounding double multiply accumulate
FEAT_HPDS	Yes	Hierarchical permission disables in translation tables
FEAT_VHE	Yes	Virtualization Host Extensions
FEAT_PAN	Yes	Privileged access-never
FEAT_LOR	Yes	Limited ordering regions
FEAT_HAFDBS	Yes	Hardware updates to access flag and dirty state in translation tables
FEAT_VMID16	Yes	16-bit VMID
FEAT_PMUv3p1	Yes	PMU extensions version 3.1
FEAT_Debugv8p1	Yes	Debug with VHE
FEAT_PAN3	Yes	Support for SCTLR_ELx.EPAN

Table 1-4: Implementation status of the Arm®v8.2-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_TTCNP	Yes	Common not private translations
FEAT_XNX	Yes	Execute-never control distinction by Exception level at stage 2
FEAT_UAO	Yes	Unprivileged Access Override control
FEAT_PAN2	Yes	AT S1E1R and AT S1E1W instruction variants for PAN
FEAT_DPB	Yes	DC CVAP instruction
FEAT_Debugv8p2	Yes	Arm®v8.2-A Debug

Feature	Implemented	Description
FEAT_IESB	Yes	Implicit Error synchronization event
FEAT_AA32HPD	No	AArch32 Hierarchical permission disables
FEAT_HPDS2	Yes	Hierarchical permission disables in translation tables 2
FEAT_LSMAOC	No	Load/Store instruction multiple atomicity and ordering controls
FEAT_FP16	Yes	Half-precision floating-point data processing
FEAT_LVA	No	Large VA support
FEAT_LPA	No	Large PA and IPA support
FEAT_VPIPT	No	VMID-aware PIPT instruction cache
FEAT_PCSRv8p2	Yes	PC Sample-based profiling version 8.2
FEAT_RAS	Yes	<i>Reliability, Availability, and Serviceability</i> (RAS) Extension version 1.1
FEAT_SPE	Yes	<i>Statistical Profiling Extension</i> (SPE) For more information, see 22. Statistical Profiling Extension support on page 157.
FEAT_SVE	Yes	<i>Scalable Vector Extension</i> (SVE)
FEAT_SHA512	Yes	Advanced SIMD SHA512 instructions
FEAT_SHA3	Configurable	Advanced SIMD EOR3, RAX1, XAR, and BCAX instructions
FEAT_SM3		Advanced SIMD SM3 instructions
FEAT_SM4		Advanced SIMD SM4 instructions
FEAT_DotProd	Yes	Advanced SIMD Int8 dot product instructions
FEAT_FHM	Yes	Half-precision floating-point FMLAL instructions
FEAT_EVT	Yes	Enhanced Virtualization Traps
FEAT_DPB2	Yes	DC CVADP instruction
FEAT_BF16	Yes	AArch64 BFloat16 instructions
FEAT_AA32BF16	No	AArch32 BFloat16 instructions
FEAT_I8MM	Yes	Int8 Matrix Multiplication
FEAT_AA32I8MM	No	AArch32 Int8 Matrix Multiplication
FEAT_F32MM	No	SVE single-precision floating-point matrix multiply instruction
FEAT_F64MM	No	SVE double-precision floating-point matrix multiply instruction

Table 1-5: Implementation status of the Arm®v8.3-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_PAuth	Yes	Pointer authentication
FEAT_EPAC	No	Enhanced Pointer authentication
FEAT_PACIMP	No	Pointer authentication - IMPLEMENTATION DEFINED algorithm
FEAT_PACQARMA5	No	Pointer authentication - QARMA5 algorithm
FEAT_PACQARMA3	Yes	Pointer authentication - QARMA3 algorithm
FEAT_CONSTPACFIELD	Yes	PAC Algorithm enhancement
FEAT_JSCVT	Yes	JavaScript FJCVTS conversion instruction
FEAT_NV	No	Nested virtualization
FEAT_LRCP	Yes	Load-acquire RCpc instructions

Feature	Implemented	Description
FEAT_FCMA	Yes	Floating-point FCMLA and FCADD instructions
FEAT_CCIDX	Yes	Extended cache index
FEAT_SPEv1p1	Yes	Statistical Profiling Extensions version 1.1
FEAT_DoPD	Yes	Debug over Powerdown
FEAT_PAuth2	Yes	Enhancements to pointer authentication
FEAT_FPAC	Yes	Faulting on pointer authentication instructions <i>Faulting Pointer Authentication Code (FPAC)</i>
FEAT_FPACCOMBINE	Yes	Faulting on combined pointer authentication instructions

Table 1-6: Implementation status of the Arm®v8.4-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_SEL2	Yes	Secure EL2
FEAT_NV2	No	Enhanced support for nested virtualization
FEAT_S2FWB	Yes	Stage 2 forced write-back
FEAT_DIT	Yes	Data Independent Timing instructions
FEAT_IDST	Yes	ID space trap handling
FEAT_FlagM	Yes	Condition flag manipulation
FEAT_LSE2	Yes	Large System Extensions version 2
FEAT_LRCPC2	Yes	Load-acquire RCpc instructions version 2
FEAT_TLBIOS	Yes	TLB invalidate outer-shared instructions
FEAT_TLBIRANGE	Yes	TLB range invalidate range instructions
FEAT_TTL	Yes	Translation Table Level
FEAT_BBM	Yes	Translation table break before make levels
FEAT_RASv1p1	Yes	<i>Reliability, Availability, and Serviceability (RAS) Extension</i> version 1.1 All extensions up to Arm®v9.0-A at full containment capability with <i>Error Correcting Code (ECC)</i> configured. See 10. RAS Extension support on page 76 for more information on the implementation of this extension in the core.
FEAT_DoubleFault	Yes	Double Fault Extension
FEAT_Debugv8p4	Yes	Debug relaxations and extensions version 8.4
FEAT_PMUv3p4	Yes	PMU extension version 3.4
FEAT_TRF	Yes	Self hosted Trace Extensions
FEAT_TTST	Yes	Small translation tables
FEAT_AMUv1	Yes	Activity Monitors Extension
FEAT_MPAM	Yes	<i>Memory Partitioning and Monitoring (MPAM)</i> For more information on the <i>Memory System Resource Partitioning and Monitoring (MPAM)</i> Extension, see the Arm® Architecture Reference Manual for A-profile architecture .

Table 1-7: Implementation status of the Arm®v8.5-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_FlagM2	Yes	Condition flag manipulation version 2
FEAT_FRINTTS	Yes	FRINT32Z, FRINT32X, FRINT64Z, and FRINT64X instructions
FEAT_ExS	No	Disabling context synchronizing exception entry and exit
FEAT_GTG	Yes	Guest translation granule size
FEAT_BTI	Yes	<i>Branch Target Identification</i> (BTI)
FEAT_EOPD	Yes	Preventing ELO access to halves of address maps
FEAT_RNG	Yes	Random number generator
FEAT_RNG_TRAP	Yes	Trapping support for RNDR and RNDRRS
FEAT_MTE	Yes	Instruction-only Memory Tagging Extension The Neoverse™ N3 core always implements the <i>Memory Tagging Extension</i> (MTE) and therefore is compliant with the CHI.E protocol. For information on CHI.E commands inferred by MTE, see the <i>CHI master interface</i> chapter in the Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual.
FEAT_MTE2	Yes	Full Memory Tagging Extension
FEAT_MTE3	Configurable	MTE Asymmetric Fault Handling
	These features are enabled by setting the BROADCASTMTE pin to 1.	
FEAT_PMUv3p5	Yes	PMU Extension version 3.5

Table 1-8: Implementation status of the Arm®v8.6-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_ECV	Yes	Enhanced counter virtualization
FEAT_FGT	Yes	Fine Grain Traps
FEAT_TWED	No	Delayed trapping of WFE
FEAT_AMUv1p1	No	Activity Monitors Extension version 1.1
FEAT_MPAMv0p1	No	Memory Partitioning and Monitoring version 0.1
FEAT_MPAMv1p1	Yes	Memory Partitioning and Monitoring version 1.1
FEAT_MTPMU	No	Multi-threaded PMU Extensions

Table 1-9: Implementation status of the Arm®v8.7-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_AFP	Yes	Alternate floating-point behavior
FEAT_HCX	Yes	Support for the HCRX_EL2 register
FEAT_LPA2	No	Larger physical address for 4KB and 16KB translation granules
FEAT_LS64	No	Support for 64 byte loads/stores without return
FEAT_LS64_V	No	Support for 64-byte stores with return
FEAT_LS64_ACCDATA	No	Support for 64-byte ELO stores with return

Feature	Implemented	Description
FEAT_PMUv3p7	Yes	Arm®v8.7-A PMU Extensions See 18.1 Performance monitors events on page 110.
FEAT_RPRES	No	Increased precision of Reciprocal Estimate and Reciprocal Square Root Estimate
FEAT_SPEv1p2	Yes	Arm®v8.7-A SPE See 22. Statistical Profiling Extension support on page 157.
FEAT_WFxT	Yes	WFE and WFI instructions with timeout See 4.2.1 Wait for Interrupt and Wait for Event on page 38.
FEAT_XS	Yes	XS attribute

Table 1-10: Implementation status of the Arm®v8.8-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_CMOW	No	Control for cache maintenance permission
FEAT_Debugv8p8	No	Debug v8.8
FEAT_HBC	No	Hinted conditional branch
FEAT_HPMN0	Yes	Setting of MDCR_EL2.HPMN to zero
FEAT_MOPS	No	Standardization of memory operations
FEAT_NMI	No	Non-maskable Interrupts
FEAT_PMUv3p8	No	Arm®v8.8-A PMU Extensions
FEAT_PMUv3_TH	No	Event counting threshold
FEAT_SPEv1p3	No	Arm®v8.8-A Statistical Profiling Extensions
FEAT_TIDCP1	No	EL0 use of IMPLEMENTATION DEFINED functionality

Table 1-11: Implementation status of the Arm®v9.0-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_ETE	Yes	<i>Embedded Trace Extension (ETE)</i> See 19. Embedded Trace Extension support on page 136.
FEAT_SVE2	Yes	<i>Scalable Vector Extension (SVE) version 2</i> See 14. Scalable Vector Extensions support on page 88.
FEAT_SVE_AES	Configurable	SVE AES instructions
FEAT_SVE_PMULL128		SVE PMULL instructions
FEAT_SVE_SHA3		SVE SHA-3 instructions
FEAT_SVE_SM4		SVE SM4 instructions
FEAT_SVE_Sm4Perm	Supported as part of the Arm®v8-A Cryptographic Extension	
FEAT_SVE_BitPerm	Yes	SVE Bit Permute
FEAT_TME	No	<i>Transactional Memory Extension (TME)</i>

Feature	Implemented	Description
FEAT_TRBE	Yes	Trace Buffer Extension (TRBE) See 20. Trace Buffer Extension support on page 150.

Table 1-12: Implementation status of the Arm®v9.1-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_ETEv1p1	Yes	Embedded Trace Extension, version 1.1

Table 1-13: Implementation status of the Arm®v9.2-A features in the Neoverse™ N3 core

Feature	Implemented	Description
FEAT_BRBE	No	Branch Record Buffer Extensions
FEAT_RME	No	Realm Management Extension
FEAT_ETEv1p2	No	Embedded Trace Extension, version 1.2
FEAT_SME	No	Scalable Matrix Extension
FEAT_SME_FA64	No	Full A64 support in Streaming mode
FEAT_SME_F64F64	No	Double-precision floating-point outer product instructions
FEAT_SME_I16I64	No	16-bit to 64-bit integer widening outer product instructions
FEAT_EBF16	No	Enhanced BFloat16

The following table shows the other standards and specifications that the Neoverse™ N3 core supports.

Table 1-14: Other standards and specifications supported in the Neoverse™ N3 core

Standard or specification	Version	Description
FEAT_GICv4p1	GICv4.1	Generic Interrupt Controller (GIC) See the Arm® Generic Interrupt Controller Architecture Specification, GIC architecture version 3 and version 4 for more information.
Debug	-	Arm®v9.2-A architecture implemented with Arm®v8.4-A Debug architecture support and Arm®v8.3-A Debug over powerdown support. See the Arm® Architecture Reference Manual for A-profile architecture for information on this architecture.
CoreSight™ architecture	v3.0	For more information on CoreSight™ architecture, see the Arm® CoreSight™ Architecture Specification v3.0 .

Related information

[2.1 Core components](#) on page 30

1.5 Test features

The Neoverse™ N3 core provides test signals that enable the use of both *Automatic Test Pattern Generation* (ATPG) and *Memory Built-In Self Test* (MBIST) to test the core logic and memory arrays.

The Neoverse™ N3 core includes an ATPG test interface that provides signals to control the *Design for Test* (DFT) features of the core. To prevent problems with DFT implementation, you must carefully consider how you use these signals.

Arm also provides MBIST interfaces that enable you to test the RAMs at operational frequency. You can add your own MBIST controllers to automatically generate test patterns and perform result comparisons. Optionally, you can use your EDA tool to test the physical RAMs directly instead of using the supplied Arm interfaces.

For the list of test signals and information on their usage, see the *Design for Test integration guidelines* chapter in the *Arm® Neoverse™ N3 Core Configuration and Integration Manual*.

For the list of external scan control signals, see the *Design for Test integration guidelines* chapter in the *Arm® DynamIQ™ Shared Unit-120 Configuration and Integration Manual*.



The *Arm® Neoverse™ N3 Core Configuration and Integration Manual* and *Arm® DynamIQ™ Shared Unit-120 Configuration and Integration Manual* are confidential documents that are available with the appropriate product licenses.

1.6 Design tasks

The Neoverse™ N3 core is delivered as a synthesizable RTL description in SystemVerilog. Before you can use the Neoverse™ N3 core, you must implement, integrate, and program it.

A different party can perform each of the following tasks:

Implementation

The implementer configures the RTL, adds vendor cells/RAMs, and takes the design through the synthesis and Place and Route (P&R) steps to produce a hard macrocell.

The implementer chooses the options that affect how the RTL source files are rendered. These options can affect the area, maximum frequency, power, and features of the resulting macrocell.

Other components such as *Design for Test* (DFT) structures and, if necessary, power switches can be added to the implementation flow.

Integration

The integrator connects the macrocell into a *System on Chip* (SoC). This task includes connecting it to a memory system and peripherals.

The integrator configures some features of the core by tying inputs to specific values. These configuration settings affect the start-up behavior before any software configuration is made and can also limit the options available to the software.

Software programming

The system programmer develops the software to configure and initialize the core and tests the application software.

The programmer configures the core by programming values into registers. The programmed values affect the behavior of the core.

The operation of the final device depends on the build configuration, the configuration inputs, and the software configuration.

See *Functional integration* in the *Arm® DynamIQ™ Shared Unit-120 Configuration and Integration Manual* for signal descriptions.

See *RTL configuration process* in the *Arm® Neoverse™ N3 Core Configuration and Integration Manual* and in the *Arm® DynamIQ™ Shared Unit-120 Configuration and Integration Manual* for implementation options.

1.7 Product revisions

The following table indicates the main differences in functionality between product revisions.

Revision	Notes
r0p0	First release
r0p1	Errata fixes

Changes in functionality that have an impact on the documentation also appear in [Revision history](#) on page 897.

2. Technical overview

The components in the Neoverse™ N3 core are designed to make it a balanced-performance core.

The main blocks include:

- L1 instruction and L1 data memory systems
- L2 memory system
- Register rename
- Instruction decode
- Instruction issue
- Execution pipeline
- *Memory Management Unit* (MMU)
- Trace unit and trace buffer
- *Performance Monitoring Unit* (PMU)
- *Activity Monitoring Unit* (AMU)
- *Generic Interrupt Controller* (GIC) CPU interface
- Branch prediction

The Neoverse™ N3 core interfaces with the DSU-120 through the CPU bridge.

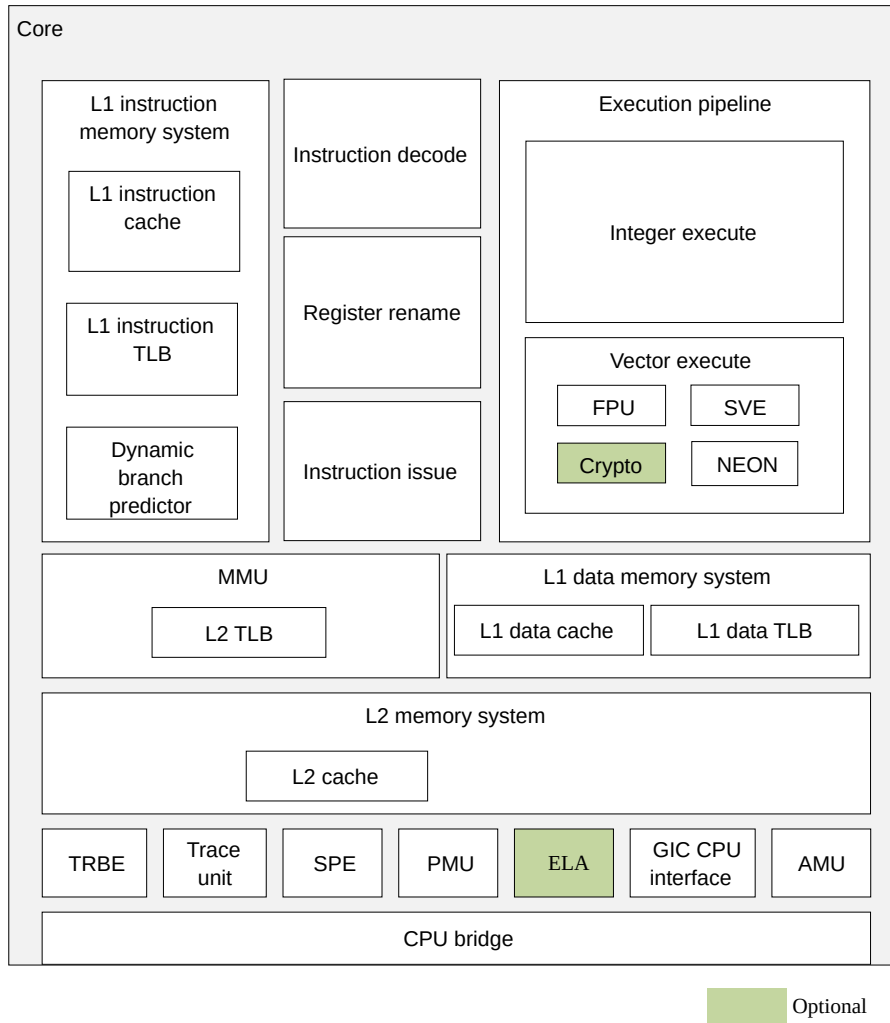
The Neoverse™ N3 core implements the Arm®v9.2-A architecture. The Arm®v9.2-A architecture extends the architecture defined in the Arm®v8-A architectures up to Arm®v8.7-A.

The programmer's model and the architecture features implemented, such as the Generic Timer, are compliant with the standards in [1.4 Supported standards and specifications](#) on page 20.

2.1 Core components

The Neoverse™ N3 core includes components designed to make it a balanced-performance, low-power, and constrained area product. The Neoverse™ N3 core includes a CPU bridge that connects the core to the DSU-120. The DSU-120 connects the core to an external memory system and the rest of the *System on Chip* (SoC).

The following figure shows the Neoverse™ N3 core components.

Figure 2-1: Neoverse™ N3 core components

L1 instruction memory system

The L1 instruction memory system fetches instructions from the instruction cache and delivers the instruction stream to the instruction decode unit.

The L1 instruction memory system includes:

- A 32KB or 64KB, 4-way set associative L1 instruction cache with 64-byte cache lines
- A fully associative L1 instruction *Translation Lookaside Buffer* (TLB) with native support for 4KB, 16KB, 64KB, and 2MB page sizes
- A dynamic branch predictor

Instruction decode

The instruction decode unit decodes instructions from AArch64 into an internal format, which it then passes to the execution pipeline.

Register rename

The register rename unit performs register renaming to facilitate out-of-order execution and dispatches decoded instructions to various issue queues.

Instruction issue

The instruction issue unit controls when the decoded instructions are dispatched to the execution pipelines. It includes issue queues for storing instructions pending dispatch to execution pipelines.

Integer execute

The execution pipeline includes the integer execute unit that performs arithmetic and logical data processing operations.

Vector execute

The vector execute unit is part of the execution pipeline and performs Advanced SIMD and floating-point operations. The vector execute unit executes the *Scalable Vector Extension* (SVE) and *Scalable Vector Extension 2* (SVE2) instructions, and optionally executes the cryptographic instructions.

Advanced SIMD and floating-point support

Advanced SIMD is a media and signal processing architecture that adds instructions primarily for audio, video, 3D graphics, image, and speech processing. The floating-point architecture provides support for single-precision and double-precision floating-point operations.



The Advanced SIMD architecture, its associated implementations, and supporting software, are also referred to as NEON™ technology.

Cryptographic Extension

The Cryptographic Extension is optional in the Neoverse™ N3 core. The Cryptographic Extension adds new instructions to the Advanced SIMD and the *Scalable Vector Extension* (SVE) instruction sets that accelerate:

- *Advanced Encryption Standard* (AES) encryption and decryption.
- The *Secure Hash Algorithm* (SHA) functions, SHA-1, SHA-2, and SHA-3.
- The SVE2 versions of the SHA-3 instructions EOR3, XAR, and BCAX are supported even when the Cryptographic Extension is not configured.
- Armv8.2-SM SM3 hash function and SM4 encryption and decryption instructions.
- Finite field arithmetic that is used in algorithms such as Galois/Counter Mode and Elliptic Curve Cryptography.



The optional Cryptographic Extension is not included in the base product. Arm supplies the Cryptographic Extension only under an additional license to the Neoverse™ N3 core license.

Scalable Vector Extension

The *Scalable Vector Extension* (SVE) and *Scalable Vector Extension 2* (SVE2) are extensions to the Armv8-A architecture. SVE and SVE2 are intended to complement, not replace, AArch64 Advanced SIMD and floating-point functionality.

L1 data memory system

The L1 data memory system executes load and store instructions. It also services memory coherency requests.

The L1 data memory system includes:

- A 32KB or 64KB, 4-way set associative cache with 64-byte cache lines.
- A fully associative L1 data TLB with native support for 4KB, 16KB, 64KB, and 2MB page sizes.

Memory Management Unit

The *Memory Management Unit* (MMU) provides fine-grained memory system control through a set of virtual-to-physical address mappings and memory attributes that are held in translation tables.

These address mapping and memory attributes are saved into the TLB when an address is translated. The TLB entries include global and *Address Space IDentifiers* (ASIDs) to prevent context switch TLB invalidations. They also include *Virtual Machine IDentifiers* (VMIDs) to prevent TLB invalidations on virtual machine switches by the hypervisor.

L2 memory system

The L2 memory system includes the L2 cache. The L2 cache is private to the core and is 8-way set associative. You can configure its RAM size to be 128KB, 256KB, 512KB, 1024KB, or 2048KB. The L2 memory system is connected to the DSU-120 through a CPU bridge.

Trace unit and Trace Buffer Extension

The Neoverse™ N3 core supports a range of debug, test, and trace options including an instruction-trace-only trace unit and *TRace Buffer Extension* (TRBE).

The Neoverse™ N3 core also includes a ROM table that contains a list of components in the system. Debuggers can use the ROM table to determine which CoreSight™ components are implemented.

All the debug and trace components of the Neoverse™ N3 core are described in this manual. For more information about the *Embedded Logic Analyzer* (ELA), see the [Arm® CoreSight™ ELA-600 Embedded Logic Analyzer Technical Reference Manual](#).

Statistical Profiling Extension

The Neoverse™ N3 core implements the *Statistical Profiling Extension* (SPE) to the Arm®v8.7-A architecture. The SPE provides a statistical view of the performance characteristics of executed instructions that software writers can use to optimize their code for better performance.

Performance Monitoring Unit

The *Performance Monitoring Unit* (PMU) provides either 6 or 20 performance monitors that can be configured to gather statistics on the operation of each core and the memory system. The information can be used for debug and code profiling.

Activity Monitoring Unit

The Neoverse™ N3 core implements the Activity Monitors Extension to the Arm®v8.4-A architecture. Activity monitors in the *Activity Monitoring Unit* (AMU) provide useful information for system power management and persistent monitoring.

GIC CPU interface

The *Generic Interrupt Controller* (GIC) CPU interface, when integrated with an external Distributor component, is a resource for supporting and managing interrupts in a cluster system.

CPU bridge

The CPU bridge controls buffering and synchronization between the core and the DSU-120.

The CPU bridge is asynchronous to allow different frequency, power, and area implementation points for each core.

Related information

- 5. [Memory management](#) on page 49
- 6. [L1 instruction memory system](#) on page 58
- 7. [L1 data memory system](#) on page 62
- 8. [L2 memory system](#) on page 67
- 12. [GIC CPU interface](#) on page 83
- 18. [Performance Monitors Extension support](#) on page 110
- 19. [Embedded Trace Extension support](#) on page 136
- 20. [Trace Buffer Extension support](#) on page 150
- 21. [Activity Monitors Extension support](#) on page 151
- 22. [Statistical Profiling Extension support](#) on page 157

2.2 Interfaces

The DSU-120 manages all Neoverse™ N3 core external interfaces to the *System on Chip* (SoC).

See the *Technical overview* chapter in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for detailed information on these interfaces.

2.3 Programmer's model

The Neoverse™ N3 core implements the Arm®v9.2-A architecture. The Arm®v9.2-A architecture extends the architecture defined in the Arm®v8-A architectures up to Arm®v8.7-A. The Neoverse™ N3 core supports the AArch64 Execution state at all Exception levels, EL0 to EL3.

For more information about the programmer's model, see [Arm® Architecture Reference Manual for A-profile architecture](#).

Related information

[1.4 Supported standards and specifications](#) on page 20

3. Clocks and resets

To provide dynamic power savings, the Neoverse™ N3 core supports hierarchical clock gating. It also supports Warm and Cold resets.

The Neoverse™ N3 core has a single clock domain and receives a single clock input. This clock input is gated by an architectural clock gate in the CPU bridge.

In addition, the Neoverse™ N3 core implements extensive clock gating that includes:

- Regional clock gates to various blocks that can gate off portions of the clock tree
- Local clock gates that can gate off individual registers or banks of registers

The Neoverse™ N3 core receives the following reset signals from the DSU-120 side of the CPU bridge:

- A Warm reset for all registers in the core except for:
 - Some parts of the Debug logic
 - Some parts of the trace unit logic
 - *Reliability, Availability, and Serviceability* (RAS) logic
- A Cold reset for the logic in the core, including the debug logic, trace logic, and RAS logic.

For a complete description of the clock gating and reset scheme of the core, see the following sections in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#):

- *Clocks and resets*
- *Power and reset control with Power Policy Units*

4. Power management

The Neoverse™ N3 core provides mechanisms to control both dynamic and static power dissipation.

The dynamic power management includes the following features:

- Hierarchical clock gating
- Per-core *Dynamic Voltage and Frequency Scaling* (DVFS)

The static power management includes the following features:

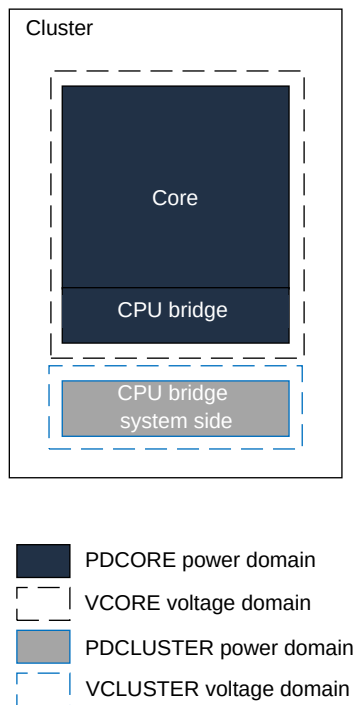
- Powerdown
- Dynamic retention, a low-power mode that retains the register and RAM state

4.1 Voltage and power domains

The DSU-120 *Power Policy Units* (PPUs) control power management for the Neoverse™ N3 core. The core supports one power domain, PDCORE, and one system power domain, PDCLUSTER. Similarly, it supports one core voltage domain, VCORE, and one cluster system voltage domain, VCLUSTER. The power domains and voltage domains have the same boundaries.

The PDCORE power domain contains all Neoverse™ N3 core logic and part of the core asynchronous bridge that belongs to the VCORE domain. The PDCLUSTER power domain contains the part of the CPU bridge that belongs to the VCLUSTER domain.

The following figure shows the Neoverse™ N3 core power domain and voltage domain. It also shows the cluster power domain and voltage domain that cover the system side of the CPU bridge.

Figure 4-1: Neoverse™ N3 core voltage domains and power domains

You can tie the VCORE and VCLUSTER voltage domains to the same supply if either:

- The core is configured to run synchronously with the DSU-120 sharing the same clock.
- The core is not required to support *Dynamic Voltage and Frequency Scaling* (DVFS).

Clamping cells between power domains are inferred through power intent files (UPF) rather than instantiated in the RTL. See *Power management* in the *Arm® Neoverse™ N3 Core Configuration and Integration Manual* for more information.

For detailed information on the DSU-120 cluster power domains and voltage domains, see *Power management* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).

4.2 Architectural clock gating modes

The `WFI`, `WFE`, `WFIIT`, and `WFET` instructions put the core into a low-power mode. These instructions architecturally disable the clock at the top of the clock tree. The core remains fully powered and retains the state.

4.2.1 Wait for Interrupt and Wait for Event

Wait for Interrupt (WFI) and *Wait for Event* (WFE) are features that put the core in a low-power state by disabling most of the core clocks, while keeping the core powered up. When the core is in WFI or WFE state, the input clock is gated externally to the core at the CPU bridge.

The logic uses a small amount of dynamic power to wake up the core from WFI or WFE low-power state. Other than this power use, the drawn power is reduced to static leakage current only.

When the core executes the `WFI`, `WFE`, `WFIT`, or `WFET` instruction, it waits for all instructions in the core, including explicit memory accesses, to retire before it enters a low-power state. The `WFI`, `WFE`, `WFIT`, and `WFET` instructions also ensure that store instructions have updated the cache or have been issued to the system.



Executing the `WFE` and `WFET` instructions when the event register is set does not cause entry into low-power state, but clears the event register.

The core exits the WFI or WFE state when one of the following events occurs:

- The core detects a reset.
- The core detects one of the architecturally defined WFI or WFE wakeup events.

WFI and WFE wakeup events can include physical and virtual interrupts.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about entering low-power state and wakeup events.

4.2.2 Low-power state behavior considerations

You must consider how certain events affect the *Wait for Interrupt* (WFI) and *Wait for Event* (WFE) low-power state behavior of the Neoverse™ N3 core.

While the core is in WFI or WFE state, the clocks in the core are temporarily enabled when any of the following events are detected:

- An access on the utility bus interface
- A *Generic Interrupt Controller* (GIC) CPU access
- A debug access through the *Advanced Peripheral Bus* (APB) interface
- A system snoop request that must be serviced by the core L1 data cache or the L2 cache
- A cache or *Translation Lookaside Buffer* (TLB) maintenance operation that must be serviced by the core L1 instruction cache, L1 data cache, L2 cache, or TLB



The core does not exit WFI or WFE state when the clocks are temporarily enabled.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about WFI and WFE.

4.3 Power control

The DSU-120 *Power Policy Units* (PPUs) control all core and cluster power mode transitions.

The core has its own PPU to control its own core power domain.

In addition, there is a PPU for the cluster.

The PPU's decide and request any change in power mode. The Neoverse™ N3 core then performs any actions necessary to reach the requested power mode. For example, the core might gate clocks, clean caches, or disable coherency before it accepts the request.

For more information about the PPU's for the cluster and the core, see the following sections in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#):

- *Power management*
- *Power and reset control with Power Policy Units*

Related information

[A.2.3 IMP_CPUPPMCR_EL3, Global PPM Configuration Register](#) on page 322

[A.2.4 IMP_CPUMPMCR_EL3, Global MPMM Configuration Register](#) on page 325

[B.1.1 CPUPPMCR, Global PPM Configuration Register](#) on page 609

[B.1.2 CPUMPMCR, Global MPMM Configuration Register](#) on page 611

4.4 Core power modes

The Neoverse™ N3 core power domain has a defined set of power modes and corresponding legal transitions between these modes.

The *Power Policy Unit* (PPU) of a core manages at the cluster level the transitions between the power modes for that core. See *Power management* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for more information.

The following table shows the supported Neoverse™ N3 core power modes.

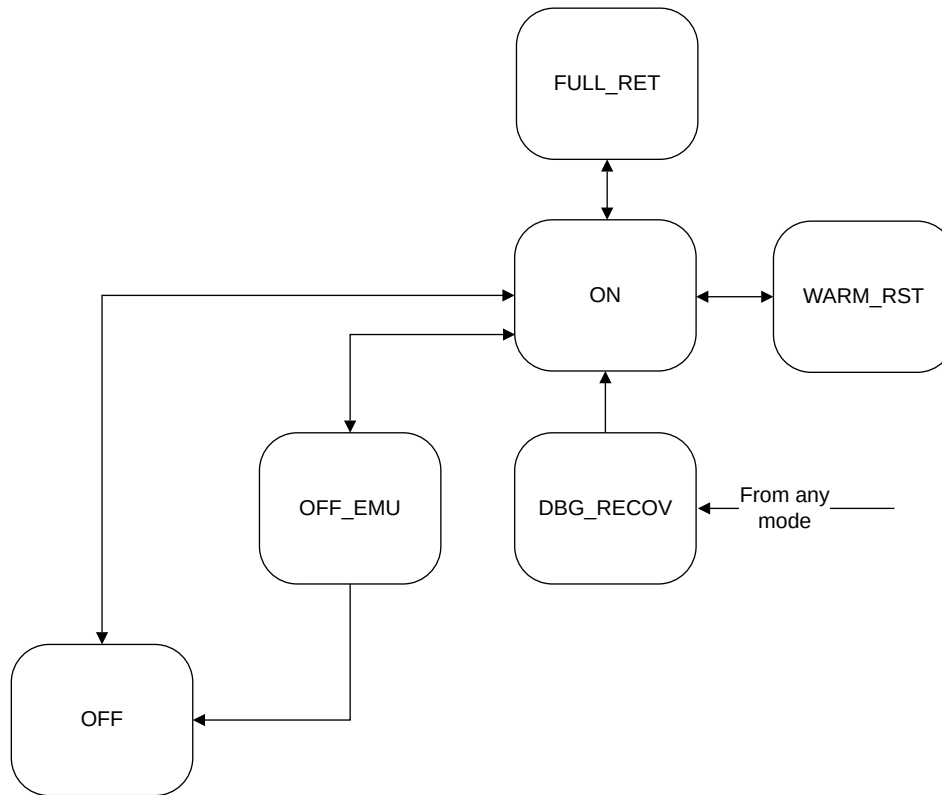


Power modes that are not shown in the following table are not supported and must not occur. Deviating from the legal power modes can lead to **UNPREDICTABLE** results. You must comply with the dynamic power management, and powerup and powerdown sequences described in [4.6 Neoverse N3 core powerup and powerdown sequence](#) on page 47.

Table 4-1: Neoverse™ N3 core power modes

Power mode	Short name	Power state
On	ON	The core is powered up and active.
Full retention	FULL_RET	The core is in retention mode. In this mode, only power that is required to retain register and RAM state is available. The core is not operational. A core must be in <i>Wait for Interrupt</i> (WFI) or <i>Wait for Event</i> (WFE) low-power state before it enters this mode.
Off	OFF	The core is powered down.
Emulated Off	OFF_EMU	Emulated off mode permits you to debug the powerup and powerdown cycle without changing the software. In this mode, the core proceeds through all the powerdown steps, except: - The clock is not gated and power is not removed when the core is powered down. - Only a Warm reset is asserted. The debug logic is preserved in the core and remains accessible by the debugger.
Debug recovery	DBG_RECOV	The RAM and logic are powered up. This mode is for applying a Warm reset to the DSU-120 cluster, while preserving memory and <i>Reliability, Availability, and Serviceability</i> (RAS) registers for debug purposes. Both cache and RAS state are preserved when transitioning from DBG_RECOV to ON. Caution: This mode must not be used during normal system operation.
Warm reset	WARM_RST	A Warm reset resets all state except for the trace logic and the debug and RAS registers.

The following figure shows the supported modes for the Neoverse™ N3 core power domain and the legal transitions between them.

Figure 4-2: Neoverse™ N3 core power mode transitions**Related information**

[4.2 Architectural clock gating modes](#) on page 38

[4.2.1 Wait for Interrupt and Wait for Event](#) on page 38

[4.4.4 Full retention mode](#) on page 43

4.4.1 On mode

In the On power mode, the Neoverse™ N3 core is on and fully operational.

The core can be initialized into the On mode. When a transition to the On mode is completed, all caches are accessible and coherent. Other than the normal architectural steps to enable caches, no additional software configuration is required.

4.4.2 Off mode

In the Off power mode, power is removed completely from the core and no state is retained.

In Off mode, all core logic and RAMs are off. The domain is inoperable and all core state is lost. The L1 and L2 caches are disabled, cleaned, and invalidated. Also, the core is removed from coherency automatically on transition to Off mode.

A Cold reset can reset the core in this mode.

An attempted external debug access to core debug registers or a utility bus access when the core domain is off returns an error response on the internal debug interface. The error indicates that the core is not available.



The core-specific debug registers in the DebugBlock for *External Debug Over PowerDown* (EDOPD) feature can be accessed while the core is in Off mode.

4.4.3 Emulated off mode

In Emulated off mode, all core domain logic and RAMs are kept on. All Debug registers must retain their state and be accessible from the external debug interface. All other functional interfaces behave as if the core were in Off mode.

4.4.4 Full retention mode

Full retention mode is a dynamic retention mode that is controlled using the *Power Policy Unit* (PPU). On wakeup, full power to the core can be restored and execution can continue.

In Full retention mode, only power that is required to retain register and RAM state is available. The core is in retention state and is non-operational.

The core enters Full retention mode when all of the following conditions are met:

- The retention timer has expired. For more information on setting the retention timer, see [A.1.14 IMP_CPUPWRCTLR_EL1, CPU Power Control Register](#) on page 211.
- The core is in *Wait for Interrupt* (WFI) or *Wait for Event* (WFE) low-power state.
- The core clock is temporarily disabled for any of the following reasons:
 - L1 snoops or L2 snoops
 - Cache or *Translation Lookaside Buffer* (TLB) maintenance operations
 - Debug or *Generic Interrupt Controller* (GIC) access

The core exits Full retention mode when it detects any of the following events:

- A WFI or WFE wakeup event, as defined in the [Arm® Architecture Reference Manual for A-profile architecture](#).
- An event that requires the core clock to be temporarily enabled without exiting the WFI or WFE low-power state. For example:
 - L1 snoops or L2 snoops
 - Cache or TLB maintenance operations
 - Debug access from the DebugBlock of the *DynamlQ™ Shared Unit* (DSU)

- GIC access

Related information

[4.2.1 Wait for Interrupt and Wait for Event](#) on page 38

4.4.5 Debug recovery mode

Debug recovery mode supports debug of external watchdog-triggered reset events, such as watchdog timeout.

By default, the core invalidates its caches when it transitions from Off to On mode. Using Debug recovery mode allows the L1 cache and L2 cache contents that were present before the reset to be observable after the reset. In this mode, the contents of the caches are retained and are not altered on the transition back to the On mode.

In addition to preserving the cache contents, Debug recovery mode supports preserving the *Reliability, Availability, and Serviceability* (RAS) state. A transition to Debug recovery mode is made from any state, which puts the core into a Warm reset state. There is no external mechanism to apply a Warm reset mode other than programming the DSU-120 *Power Policy Units* (PPUs).

For more information on the DSU-120 *Power Policy Units* (PPUs), see *The Power Policy Unit* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).



Debug recovery is strictly for debug purposes. It must not be used for functional purposes because correct operation of the caches is not guaranteed when entering this mode.

Debug recovery mode can occur at any time with no guarantee of the state of the core. A request of this type is accepted immediately, therefore its effects on the core, the DSU-120 cluster, or the wider system are **UNPREDICTABLE**, and a wider system reset might be required. In particular, any outstanding memory system transactions at the time of the reset might complete after the reset. The core is not expecting these transactions to complete after a reset, and might cause a system deadlock.

If the system sends a snoop to the DSU-120 cluster during Debug recovery mode, depending on the cluster state:

- The snoop might get a response and disturb the contents of the caches.
- The snoop might not get a response and cause a system deadlock.

4.4.6 Warm reset mode

A Warm reset resets all state except for the trace logic, debug registers, and *Reliability, Availability, and Serviceability* (RAS) registers.



WARM_RST mode is strictly for debug purposes.

A Warm reset is applied to the Neoverse™ N3 core when the core *Power Policy Unit* (PPU) in the DSU-120 is programmed for WARM_RST mode.

WARM_RST mode is only expected to be used for resets triggered by a system level issue, such as a watchdog timeout.

WARM_RST mode can occur at any time with no guarantee of the state of the core. A request to transition to WARM_RST mode is accepted immediately. Therefore, its effects on the core, the DynamIQ™ cluster, or the wider system are **UNPREDICTABLE**, and a wider system reset might be required. For example, if there were any outstanding memory transactions at the time of the reset, these transactions might complete after the reset. Unless the system interconnect is also reset, the cluster will not expect these transactions to complete after the reset, and a system deadlock might occur.



An alternative method for placing the core into Warm reset is to use the Arm®v8-A Reset Management Register, RMR_EL3. When the core runs in EL3, it requests a Warm reset of the core if you set the RMR_EL3.RR bit to 1. If RMR_EL3.RR is set to 1 before a WFI instruction is executed, then the core will request a Warm reset. The RMR_EL3.RR controlled Warm reset of the core is independent of the PPU WARM_RST power mode.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about RMR_EL3.

4.5 Performance and power management

The Neoverse™ N3 core implements *Performance and Power Management* (PPM) features that can be used to limit high activity events within the core or trade off efficiency versus peak performance.

The PPM features are:

- *Maximum Power Mitigation Mechanism* (MPMM)
- *Performance Defined Power* (PDP)

4.5.1 Maximum Power Mitigation Mechanism

Maximum Power Mitigation Mechanism (MPMM) is a power management feature that detects and limits high activity events, specifically high-power load-store events, and vector unit instructions.

If the count of high-activity events exceeds a pre-defined threshold during an evaluation period, MPMM temporarily limits the rate of instruction execution and memory system transactions.

MPMM provides three gears that enable it to limit certain classes of workloads. Each MPMM gear limits workloads at a different level of aggressiveness, where gear 0 produces the most aggressive throttling and gear 2 the least aggressive. The *Activity Monitoring Unit* (AMU) provides metrics for each gear. An external power controller can use these metrics to budget SoC power in the following ways:

- By limiting the number of cores that can execute higher activity workloads
- By switching to a different *Dynamic Voltage and Frequency Scaling* (DVFS) operating point

MPMM is not intended to limit workloads that operate close to typical power levels. The MPMM event detection and limiting are targeted to limit workloads that operate at significantly higher power levels than typical integer workloads.



MPMM must not be relied on as the only electrical safety mechanism. It is essentially a localized assistance mechanism that operates at core level. MPMM is not a substitute for a coarse-grained emergency power reduction scheme, but it does minimize the likelihood of such a scheme being engaged. It is a first line of defense rather than a complete solution.

Related information

[A.2.3 IMP_CPUPPMCR_EL3, Global PPM Configuration Register](#) on page 322

[A.2.4 IMP_CPUMPMCR_EL3, Global MPMM Configuration Register](#) on page 325

[B.1.1 CPUPPMCR, Global PPM Configuration Register](#) on page 609

[B.1.2 CPUMPMCR, Global MPMM Configuration Register](#) on page 611

4.5.2 Performance Defined Power

Performance Defined Power (PDP) is a power management feature that trades off-peak performance for a reduced power envelope on general workloads.

The PDP is configured using a level of aggressiveness among three possible values. When the level of aggressiveness is increased, the average workload power is reduced but it causes more performance loss, which varies by workload.

The PDP has an impact on:

- Core power reduction. The core power is reduced and the efficiency is increased.

- External memory system power reduction. Memory request bandwidth is modulated to reduce power in the memory system.

4.5.3 Dispatch block

In extreme core thermal or power conditions, you can temporarily halt forward progress of the core without stopping the clock.

A pin is provided on the DSU-120 boundary that can directly be used to force the core to stall for the duration that the pin is asserted. When the core is stalled, the dispatch of new instructions is stopped. However, instructions that have already been dispatched will continue to execute and complete as normal.

4.6 Neoverse™ N3 core powerup and powerdown sequence

There is no specific sequence to power up the Neoverse™ N3 core. To power down the core, you must follow a specific sequence. There are no software steps required to bring a core into coherence after reset.

To power down the Neoverse™ N3 core:

1. If required, save the state of the core to system memory to allow for retrieval of the core state during powerup.
2. Disable interrupts to the core.
 - a. Disable the interrupt enable bits in the ICC_IGRPEN0_EL1 and ICC_IGPREN1_EL1 registers.
 - b. Set the GIC distributor wake-up request for the core using the GICR_WAKER register.
 - c. Read the GICR_WAKER register to confirm that the ChildrenAsleep bit indicates that the interface is inactive.
3. Optionally, disable the interrupt outputs from the RAS registers. For more information, see [Managing RAS fault and error interrupts during the core powerdown procedure](#).
4. Set the IMP_CPUPWRCTLR_EL1.CORE_PWRDN_EN bit to 1 to indicate to the power controller that a powerdown is requested.
5. Execute an `ISB` instruction.
6. Execute a `WFI` instruction. Once the `WFI` instruction is executed, the powerdown sequence cannot be interrupted.

After you have executed the `WFI` and subsequently received a powerdown request from the power controller, the hardware:

- Disables and cleans the core caches
- Removes the core from system coherency

When the IMP_CPUPWRCTLR_EL1.CORE_PWRDN_EN bit is set, executing a `WFI` instruction automatically masks all interrupts and wakeup events in the core. As a result, applying a reset is the only way to wake up the core from the *Wait for Interrupt* (WFI) state.

4.6.1 Managing RAS fault and error interrupts during the core powerdown procedure

After the `WFI` instruction is executed, the power management architecture does not permit interrupting the core software.

The WFI instruction is normally the point of no return for powering down the core. However, if a RAS fault or error interrupt is signaled from the core during the power down procedure, this will cause the core to deny the power down request and cause the core to wake up from WFI.

Therefore, if the RAS fault and error interrupt outputs remain active during the power down procedure, the software must be designed so that if it wakes up from the power down WFI, it can analyze the RAS fault or error and clear the interrupt output before re-executing the WFI.

Alternatively, the software could disable the RAS fault and error interrupt outputs before executing the powerdown WFI so that the WFI is the point of no return for powering down the core. However, this would mean that any detected faults or errors encountered during the powerdown procedure would not be reported and the records of the fault or error would be lost.

4.7 Debug over powerdown

The Neoverse™ N3 core supports debug over powerdown, which allows a debugger to retain its connection with the core even when powered down. This behavior enables debug to continue through powerdown scenarios rather than having to re-establish a connection each time the core is powered up.

The debug over powerdown logic is part of the DebugBlock in the DSU-120. The DebugBlock is external to the DSU-120 cluster and must remain powered on during the debug over powerdown process.

See *Debug* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for more information.

5. Memory management

The *Memory Management Unit* (MMU) translates an input address to an output address.

This translation is based on address mapping and memory attribute information that is available in the Neoverse™ N3 core internal registers and translation tables. The MMU also controls memory access permissions, memory ordering, and cache policies for each region of memory.

An address translation from an input address to an output address is described as a stage of address translation. The Neoverse™ N3 core can perform:

- Stage 1 translations that translate an input *Virtual Address* (VA) to an output *Physical Address* (PA) or *Intermediate Physical Address* (IPA).
- Stage 2 translations that translate an input IPA to an output PA.
- Combined stage 1 and stage 2 translations that translate an input VA to an IPA, and then translate that IPA to an output PA. The Neoverse™ N3 core performs translation table walks for each stage of the translation.

In addition to translating an input address to an output address, a stage of address translation also defines the memory attributes of the output address. With a two-stage translation, the stage 2 translation can modify the attributes that the stage 1 translation defines. A stage of address translation can be disabled or bypassed, and core can define memory attributes for disabled and bypassed stages of translation.

Each stage of address translation uses address translations and associated memory properties that are held in memory-mapped translation tables. Translation table entries can be cached into a *Translation Lookaside Buffer* (TLB). The translation table entries enable the MMU to provide fine-grained memory system control and to control the table walk hardware.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

5.1 Memory Management Unit components

The Neoverse™ N3 core *Memory Management Unit* (MMU) includes several *Translation Lookaside Buffers* (TLBs), an L2 TLB, and a translation table prefetcher.

A TLB is a cache of recently executed page translations within the MMU. The Neoverse™ N3 core implements a two-level TLB structure. The L2 TLB stores all page sizes and is responsible for breaking down these pages into smaller pages when required for the L1 data TLB or L1 instruction TLB.

The following table describes the MMU components.

Table 5-1: MMU components

Component	Description
L1 instruction TLB	- Located in the L1 instruction block - Caches entries at the 4KB, 16KB, 64KB, or 2MB granularity of <i>Virtual Address (VA)</i> to <i>Physical Address (PA)</i> mapping only - Fully associative 32 entries
L1 data TLB	- Located in the L1 data block - Caches entries at the 4KB, 16KB, 64KB, or 2MB granularity of VA to PA mappings only - Fully associative 48 entries
L1 <i>Statistical Profiling Extension (SPE)</i> TLB	- Located in the SPE block - VA to PA translations of any page and block size 1 entry
L1 <i>TRace Buffer Extension (TRBE)</i> TLB	- Located in the TRBE block - VA to PA translations of any page and block size 1 entry
L2 TLB	- Located in the MMU block - Includes a walk cache functionality - Made of two translation caches dedicated to specific translation levels: Small page TLB - Stores the results of level 3 translations for pages of size 4KB, 16KB, or 64KB 6-way set associative without reduced area configured, 1536 entries 4-way set associative with reduced area configured, 1024 entries Medium page TLB - Stores the results of level 2 translations for blocks of size 2MB, 32MB, or 512MB 4-way set associative 256 entries
Translation table prefetcher	- Detects access to contiguous translation tables and prefetches the next one - Can be disabled in the ECTLR register

TLB entries contain:

- A global indicator and an *Address Space Identifier (ASID)* to allow context switches without requiring the TLB to be invalidated
- A *Virtual Machine Identifier (VMID)* to allow virtual machine switches by the hypervisor without requiring the TLB to be invalidated

A hit in the L1 instruction TLB returns the PA to the instruction cache for comparison. It also checks the access permissions to signal an Instruction Abort.

A hit in the L1 data TLB returns the PA to the data cache for comparison. It also checks the access permissions to signal a Data Abort.

A miss in the L1 data TLB that hits in the L2 TLB has a penalty compared to a hit in the L1 data TLB. This penalty can be increased depending on the arbitration of pending requests.

5.2 Translation Lookaside Buffer entry content

Translation Lookaside Buffer (TLB) entries store the context information required to facilitate a match and avoid the need for a TLB clean on a context or virtual machine switch.

Each TLB entry contains:

- A *Virtual Address* (VA)
- A *Physical Address* (PA)
- A set of memory properties that includes type and access permissions

Each TLB entry is associated with either:

- A particular *Address Space Identifier* (ASID)
- A global indicator

Each TLB entry also contains a field to store the *Virtual Machine Identifier* (VMID) in the entry applicable to accesses from EL0 and EL1. The VMID permits hypervisor virtual machine switches without requiring the TLB to be invalidated.

Related information

[5.4 Translation table walks](#) on page 52

5.3 Translation Lookaside Buffer match process

The Armv8-A architecture supports multiple *Virtual Address* (VA) spaces that are translated differently.

Each *Translation Lookaside Buffer* (TLB) entry is associated with a particular translation regime:

- Secure EL3
- Secure EL2
- Secure EL2 and EL0
- Non-secure EL2
- Non-secure EL2 and EL0
- Secure EL1 and EL0
- Non-secure EL1 and EL0

A TLB match entry occurs when the following conditions are met:

- Its VA[48:N], where N is $\log_2$ of the block size for that translation that is stored in the TLB entry, matches the requested address.
- Entry translation regime matches the current translation regime.
- The *Address Space Identifier* (ASID) matches the current ASID held in the TTBR0_ELx or TTBR1_ELx register associated with the target translation regime, or the entry is marked global.
- The *Virtual Machine Identifier* (VMID) matches the current VMID held in the VTTBR_EL2 register.

The ASID information is used for the purpose of TLB matching for entries using:

- The Secure EL1 and ELO and Non-secure EL1 and ELO translation regime
- The Secure EL2 and ELO and Non-secure EL2 and ELO translation regime

The VMID information is used for the purpose of TLB matching for entries using:

- The Secure EL1 and ELO and Non-secure EL1 and ELO translation regime, when EL2 is enabled.

5.4 Translation table walks

When the Neoverse™ N3 core generates a memory access, the *Memory Management Unit* (MMU) searches for the requested *Virtual Address* (VA) in the *Translation Lookaside Buffers* (TLBs). If it is not present, then it is a miss and the MMU proceeds by looking up the translation table during a translation table walk.

When the Neoverse™ N3 core generates a memory access, the MMU:

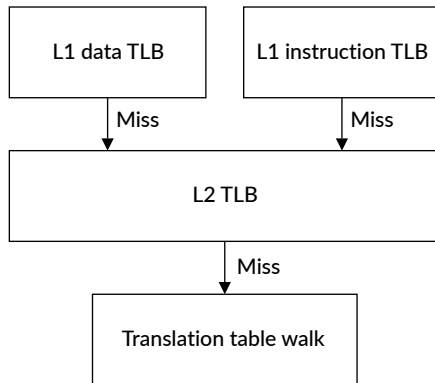
1. Performs a lookup for the requested VA, current *Address Space Identifier* (ASID), current *Virtual Machine Identifier* (VMID), and current translation regime in the relevant instruction or data L1 TLB.
2. If there is a miss in the relevant L1 TLB, then the MMU performs a lookup in the L2 TLB for the requested VA, current ASID, current VMID, and translation regime.
3. If there is a miss in the L2 TLB, then the MMU performs a hardware translation table walk.

Address translation is performed only when the MMU is enabled. It can also be disabled for a particular translation base register, in which case the MMU returns a Translation fault.

You can program the MMU to make the accesses that are generated by translation table walks cacheable. This means that translation table entries can be cached in the L2 cache, subsequent levels of cache, and external caches.

During a lookup or translation table walk, the access permission bits in the matching translation table entry determine whether the access is permitted. If the permission checks are violated, then the MMU returns a Permission fault. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

The following figure shows the TLB lookup process.

Figure 5-1: Translation table walks

In translation table walks, the descriptor is fetched from the L2 or external memory system.

Related information

- 6. [L1 instruction memory system](#) on page 58
- 7. [L1 data memory system](#) on page 62
- 8. [L2 memory system](#) on page 67

5.5 Hardware management of the Access flag and dirty state

The Neoverse™ N3 core includes the option to perform hardware updates to the translation tables.

This feature is enabled in TCR_ELx (where x is 1-3) and VTCR_EL2. To support hardware management of dirty state, translation table descriptors include the *Dirty Bit Modifier* (DBM) field.

The Neoverse™ N3 core supports hardware updates to the Access flag and to dirty state only when the translation tables are held in Inner Write-Back and Outer Write-Back Normal memory regions. If software requests a hardware update in a region that is not Inner Write-Back or Outer Write-Back Normal memory, then the Neoverse™ N3 core returns an abort with the following encoding:

- ESR_ELx.DFSC = 0b110001 for Data Aborts
- ESR_ELx.IFSC = 0b110001 for Instruction Aborts

5.6 Responses

Certain faults and aborts can cause an exception to be taken because of a memory access.

MMU responses

When one of the following operations is completed, the *Memory Management Unit* (MMU) generates a translation response to the requester:

- An L1 instruction or data *Translation Lookaside Buffer* (TLB) hit
- An L2 TLB hit
- A translation table walk

The responses from the MMU contain the following information:

- The *Physical Address* (PA) that corresponds to the translation
- A set of permissions
- Secure or Non-secure state information
- All the information that is required to report aborts

MMU aborts

The MMU can detect faults that are related to address translation and can cause exceptions to be taken to the core. Faults can include address size faults, translation faults, access flag faults, and permission faults.

External aborts

External aborts occur in the memory system and are different from aborts that the MMU detects. Normally, external memory aborts are rare. External aborts are caused by errors that are flagged by the external memory interfaces or are generated because of an uncorrected *Error Correcting Code* (ECC) error in the L1 data cache or L2 cache arrays.

External aborts are reported synchronously when they occur during:

- Translation table walks for instruction fetches, loads, and stores
- Load operations to Inner Write-Back, Outer Write-Back Normal Cacheable memory



The address captured in the *Fault Address Register* (FAR) is the target address of the instruction that generated the synchronous external abort.

External aborts are reported asynchronously when they occur during:

- Load operations to all memory locations other than Inner Write-Back, Outer Write-Back Normal memory when the access is not caused by a translation table walk
- Store operations to any memory type
- Cache maintenance, TLB invalidate, and instruction cache invalidate operations
- Atomic operations including `AtomicLd`, `AtomicSt`, `AtomicCAS`, and `AtomicSwap`

Misprogramming contiguous hints

When there is a descriptor that contains a set CH bit, the input *Virtual Address* (VA) address space must include all contiguous VAs contained in this block.

The VA address space is defined by:

- TCR_ELx.TxSZ for stage 1 translations
- VTCR_EL2.T0SZ for stage 2 translations

The Neoverse™ N3 core treats such a block as not causing a translation fault and disregards the value of the contiguous bit.

Conflict aborts

The Neoverse™ N3 core does not generate conflict abort exceptions.

When a TLB conflict is detected in the L1 TLB or L2 TLB, hardware automatically handles the conflict by invalidating the conflict entries.

5.7 Memory behavior and supported memory types

The Neoverse™ N3 core supports memory types defined in the Armv8-A architecture.

Device memory types have the following attributes:

G – Gathering

The capability to gather and merge requests together into a single transaction

R – Reordering

The capability to reorder transactions

E – Early Write Acknowledgement

The capability to accept early acknowledgement of write transactions from the interconnect



In the following table, the n prefix means the capability is not allowed.

The following table shows how memory types are supported in the Neoverse™ N3 core.

Table 5-2: Supported memory types

Memory type	Shareability	Inner Cacheability	Outer Cacheability	Notes
Device nGnRnE	Outer Shareable	-	-	Treated as Device nGnRnE
Device nGnRE	Outer Shareable ¹	-	-	Treated as Device nGnRE
Device nGRE	Outer Shareable ¹	-	-	Treated as Device nGRE

¹ Non-cacheable and Device are treated as Outer Shareable. Combinations of Non-cacheable and Write-Through are treated as Non-cacheable, and therefore are Outer Shareable.

Memory type	Shareability	Inner Cacheability	Outer Cacheability	Notes
Device GRE	Outer Shareable ¹	-	-	Treated as Device GRE
Normal	Outer Shareable ¹	Non-cacheable	Any	Treated as Non-cacheable
Normal	Outer Shareable ¹	Write-Through Cacheable	Any	Treated as Non-cacheable
Normal	Outer Shareable ¹	Write-Back Cacheable	Non-cacheable	Treated as Non-cacheable
Normal	Outer Shareable ¹	Write-Back Cacheable	Write-Through Cacheable	Treated as Non-cacheable
Normal	See Table 5-3: Shareability for Normal memory on page 56.	Write-Back Cacheable (any allocation hint)	Write-Back Cacheable No Allocate	Treated as Write-Back Read and Write Allocate but the outer cacheability propagated to the CHI interconnect is 0 (No Allocate)
Normal	See Table 5-3: Shareability for Normal memory on page 56.	Write-Back Cacheable (any allocation hint)	Write-Back Read or Write Allocate	Treated as Write-Back Read and Write Allocate but the outer cacheability propagated to the CHI interconnect is 1, therefore upgraded to Write and Read Allocate

The following table shows how the shareability is treated for certain Normal memory.

Table 5-3: Shareability for Normal memory

Shareability	Treated as
Non-shareable	Non-cacheable
Outer Shareable	Outer Shareable
Inner Shareable	Outer Shareable

5.8 Page-based hardware attributes

The architecture defines *Page-Based Hardware Attributes* (PBHA) as an optional **IMPLEMENTATION DEFINED** feature. This section describes how the Neoverse™ N3 core implements PBHA.

It allows software to set up to four bits in the translation tables, which are then propagated through the memory system with transactions and can be used in the system to control system components. The meaning of the bits is specific to the system design.

For information on how to set and enable the PBHA bits in the translation tables, see the [Arm® Architecture Reference Manual for A-profile architecture](#). When disabled, the PBHA value that is propagated on the bus is 0.

For memory accesses caused by a translation table walk, the IMP_ATCR_ELx and IMP_AVTCR_EL2 registers control the PBHA values.

PBHA combination between stage 1 and stage 2 on memory accesses

PBHA should always be considered as an attribute of the physical address.

When stage 1 and stage 2 are enabled:

- If both stage 1 PBHA and stage 2 PBHA are enabled, the final PBHA is stage 2 PBHA.
- If stage 1 PBHA is enabled and stage 2 PBHA is disabled, the final PBHA is stage 1 PBHA.
- If stage 1 PBHA is disabled and stage 2 PBHA is enabled, the final PBHA is stage 2 PBHA.
- If both stage 1 PBHA and stage 2 PBHA are disabled, the final PBHA is defined to 0.

Enable of PBHA has a granularity of 1 bit, so this property is applied independently on each PBHA bit.

Mismatched aliases

If the same physical address is accessed through more than one virtual address mapping, and the PBHA bits are different in the mappings, then the results are **UNPREDICTABLE**. The PBHA value sent on the bus could be for either mapping.

6. L1 instruction memory system

The Neoverse™ N3 core L1 instruction memory system fetches instructions and predicts branches. It includes the L1 instruction cache, the L1 instruction *Translation Lookaside Buffer* (TLB), and the branch prediction unit.

The L1 instruction memory system provides an instruction stream to the decoder. To increase overall performance and reduce power consumption, the L1 instruction memory system uses dynamic branch prediction and instruction caching.

The following table shows the L1 instruction memory system features.

Table 6-1: L1 instruction memory system features

Feature	Description
L1 instruction cache	32KB or 64KB 4-way set associative <i>Physically Indexed, Physically Tagged</i> (PIPT) - Optionally protected with parity
Cache line length	64 bytes
Cache policy	<i>Pseudo-Least Recently Used</i> (PLRU) cache replacement policy
Interface with L2 memory system	32 bytes per cycle interface



Note

The L1 instruction TLB also resides in the L1 instruction memory system. However, it is part of the *Memory Management Unit* (MMU) and is described in [5. Memory management](#) on page 49.

6.1 L1 instruction cache behavior

The L1 instruction cache is invalidated automatically at reset unless the core power mode is initialized to Debug Recovery.

In Debug recovery mode, the L1 instruction cache is not functional.

L1 instruction cache disabled behavior

Disabling the L1 instruction cache has no effect on the operation of the L1 instruction cache. Instructions can be cached into, and fetched from, the L1 instruction cache even when it is disabled. The software must take into account Non-cacheable accesses to ensure correct behavior. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

If the L1 instruction cache is disabled, then all memory accesses caused by instruction fetches are performed using the Non-cacheable memory attribute. It means that instruction fetches might not

be coherent with caches in the same core or other cores. The software must take this into account by performing the appropriate cache maintenance operations.

L1 instruction cache maintenance

The cache maintenance operation can happen at any time, regardless of L1 status (disabled or enabled).

Related information

[4.4.5 Debug recovery mode](#) on page 44

6.2 L1 instruction cache Speculative memory accesses

Instruction fetches are Speculative and there can be several unresolved branches in the pipeline. A branch instruction or exception in the code stream can cause a pipeline flush, discarding the currently fetched instructions.

On instruction fetches, pages with Device memory type attributes are treated as Non-Cacheable Normal Memory. To prevent instruction fetches, device memory pages must be marked with the translation table descriptor attribute bit *eXecute Never* (XN). The device and code address spaces must be separated in the physical memory map. This separation prevents Speculative fetches to read-sensitive devices when address translation is disabled.

If the L1 instruction cache is enabled and if the instruction fetches miss in the L1 instruction cache, then they can still look up in the L1 data cache.

However, the lookup never causes an L1 data cache refill, regardless of the data cache enable status. The line is only allocated in the L2 cache, provided that the L1 instruction cache is enabled. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

6.3 Program flow prediction

The Neoverse™ N3 core contains program flow prediction hardware, also known as branch prediction. Branch prediction increases overall performance and enhances power efficiency.

Program flow prediction is enabled when the *Memory Management Unit* (MMU) is enabled for the current Exception level. If program flow prediction is disabled, then all taken branches incur a penalty that is associated with cleaning the pipeline. If program flow prediction is enabled, then it predicts whether a conditional or unconditional branch is to be taken, as follows:

- For conditional branches, it predicts whether the branch is to be taken and the address that the branch goes to, known as the branch target address.
- For unconditional branches, it only predicts the branch target address.

Program flow prediction hardware contains the following functionality:

- A *Branch Target Buffer* (BTB) holding the branch target address of previously taken branches

- A *Branch Prediction* (BP) predictor that uses the previous branch history
- The return stack, including nested subroutine return addresses
- A static branch predictor
- An indirect branch predictor

Predicted and non-predicted instructions

Program flow prediction hardware predicts all branch instructions, and includes:

- Conditional branches
- Unconditional branches
- Return instructions
- Indirect branches

The following instructions are not predicted:

- Exception return instructions (including `ERET`, `ERETAA`, `ERETAB`)
- Supervisor call instructions
- Hypervisor call instructions
- Secure Monitor call instructions

Return stack

The return stack stores the return address of procedure call instructions. This address should be equal to the value written in the Link Register (X30) by these instructions.

Any of the following instructions causes a return stack push:

- `BL`
- `BLR`
- `BLRAA`
- `BLRAAZ`
- `BLRAB`
- `BLRABZ`

Any of the following instructions cause a return stack pop:

- `RET`
- `RETAA`
- `RETAB`

6.4 Instruction cache hardware coherency

With instruction cache hardware coherency, the following behaviors in the core are affected:

- L1 instruction cache and L2 cache become strictly inclusive. Any cache line present in the L1 instruction cache is also present in the L2 cache.
- Instruction cache invalidate instructions are treated as no-ops and do not cause instruction cache invalidation or DVM message broadcasts to other cores.
- L2 cache monitors all store and cache invalidation coherency traffic and ensures that the L1 instruction cache invalidates any entry that is written to, or invalidated from, the L2 cache.
- CTR_EL0[29] reads as 1. Using this register, software can discover that the core implements instruction cache hardware coherency and can optimize functions to not issue instruction cache instructions.

The support for instruction cache coherency is always included. The functionality can be disabled by setting BROADCASTICINVAL pin to 1.

The following restrictions and recommendations apply to instruction cache hardware coherency in the core:

- The coherency domain containing a core configured with instruction cache hardware coherency must not contain any coherent agents that require software instruction cache maintenance.
- Arm recommends that systems using instruction cache hardware coherency should be configured with an L2 cache size of at least 1MB. An L2 cache size of 512KB is also acceptable, but will see approximately a 1-2% reduction in performance due to the overhead of a strictly inclusive L1 instruction cache and L2 cache.

7. L1 data memory system

The Neoverse™ N3 core L1 data memory system executes load and store instructions. It services memory coherency requests and specific instructions such as atomics, cache maintenance operations, and memory tagging instructions. The L1 data memory system includes the L1 data cache and the L1 data *Translation Lookaside Buffer* (TLB).

The following table shows the L1 data memory system features.

Table 7-1: L1 data memory system features

Feature	Description
L1 data cache	32KB or 64KB 4-way set associative, 16 banks <i>Virtually Indexed, Physically Tagged</i> (VIPT) behaving as <i>Physically Indexed, Physically Tagged</i> (PIPT) - Optionally protected with <i>Error Correcting Code</i> (ECC)
Cache line length	64 bytes
Cache policy	Pseudo- <i>Least Recently Used</i> (LRU) cache replacement policy
Interface with integer execute pipeline and vector execute	3×64-bit read paths and 4×64-bit write paths for the integer execute pipeline 2×128-bit write paths and 3×128-bit read paths for the vector execute



Note

The L1 data TLB also resides in the L1 data memory system. However, it is part of the *Memory Management Unit* (MMU) and is described in [5. Memory management](#) on page 49.

7.1 L1 data cache behavior

The L1 data cache is invalidated automatically at reset unless the core power mode is initialized to Debug recovery mode.

In Debug recovery mode, the caches are not guaranteed to be functional and should not be enabled.

There is no operation to invalidate the entire data cache. If software requires this function, then it must be constructed by iterating over the cache geometry and executing a series of individual invalidates by set/way instructions. The `dc c1sw` instruction performs both a clean and invalidate of the target set/way. The values of HCR_EL2.SWIO have no effect. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about `dc c1sw` and HCR_EL2.

Data Cacheability disabled behavior

If the data Cacheability is disabled, then:

- A new line is not allocated in the L2 cache as a result of a load instruction.
- All load and store instructions to cacheable memory are treated as Non-cacheable.
- Data cache maintenance operations continue to execute normally.

The L1 data and L2 caches cannot be disabled independently. When a core disables the L1 data cache, cacheable memory accesses issued by that core are no longer cached in the L1 or L2 cache.

To maintain data coherency between multiple cores, the Neoverse™ N3 core uses the *Modified Exclusive Shared Invalid* (MESI) protocol.



The way that cache indices are determined means that there is no direct relationship between the *Physical Address* (PA) and set number. You cannot use targeted operations that assume a relationship between the PA and set number. To flush the entire cache, you must perform set and way maintenance operations over the number of sets and ways described in CCSIDR_EL1 for that cache.

Related information

[4.4.5 Debug recovery mode](#) on page 44

7.2 Write streaming mode

The Neoverse™ N3 core supports write streaming mode, sometimes referred to as read allocate mode, both for the L1 and the L2 cache.

A cache line is allocated to the L1 or L2 cache on either a read miss or a write miss. However, writing large blocks of data can pollute the cache with unnecessary data. It can also waste power and performance when a linefill is performed only to discard the linefill data, because the entire line gets overwritten by subsequent writes (for example using `memset()` or `memcpy()`). In some situations, cache line allocation on writes is not required. For example, when executing the C standard library `memset()` function to clear a large block of memory to a known value.

To prevent unnecessary cache line allocation, the memory system detects when the core has written a sequence of full cache lines. If this situation is detected on a configurable number of consecutive linefills, then it switches into write streaming mode.

When in write streaming mode, load operations behave as normal, and can still cause linefills. Writes still look up in the cache, but if they miss, then they write out to the L2 or system rather than starting a linefill.



More than the specified number of linefills might be observed on the CHI master or AXI master interface before the memory system switches to write streaming mode.

The write streaming mode remains enabled until either:

- It detects a cacheable write burst that is not a full cache line.
- There is a subsequent load operation that targets the same line as an outstanding write stream.

When a Neoverse™ N3 core has switched to write streaming mode, the memory system continues to monitor the bus traffic. It signals to the L2 cache to go into write streaming mode when it observes a further number of full cache line writes.

The write streaming threshold defines the number of consecutive cache lines that are fully written without being read before store operations stop causing cache allocations. You can configure the write streaming threshold for each cache (L1 and L2) by writing the register [A.1.11 IMP_CPUECTLR_EL1, CPU Extended Control Register](#) on page 188.

7.3 Atomic instruction implementation in the L1 data memory system

The Neoverse™ N3 core supports the atomic instructions added in the Arm®v8.1-A architecture.

Atomic instructions to Cacheable memory can be performed as either near atomics or far atomics, the Neoverse™ N3 core performs these instructions as near atomics by default.

Alternatively, IMP_CPUECTLR2_EL1 can be programmed so that depending on the system behavior, some atomic instructions attempt to execute as far atomics.

When *Memory Tagging Extension* (MTE) is enabled with precise checking, all checked atomics are performed near.

The Neoverse™ N3 core supports atomics to Device or Non-cacheable memory, however this relies on the interconnect also supporting atomics. If such an atomic instruction is executed when the interconnect does not support them, then it results in an abort.

7.4 Internal exclusive monitor

The Neoverse™ N3 core includes an internal exclusive monitor with a 2-state, open and exclusive state machine that manages Load-Exclusive and Store-Exclusive accesses and Clear-Exclusive (CLREX) instructions.

You can use these instructions to construct semaphores, ensuring synchronization between different processes running on the core, and also between different cores that are using the same coherent memory locations for the semaphore. A Load-Exclusive instruction tags a small block of memory for exclusive access. CTR_ELO defines the size of the tagged blocks as 16 words, one cache line.

**Note**

A Load-Exclusive or Store-Exclusive instruction is an instruction that has a mnemonic starting with `LDX`, `LDAX`, `STX`, or `STLX`.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information on these instructions.

See [A.7.26 CTR_EL0, Cache Type Register](#) on page 454 for more technical reference and register information.

7.5 Data prefetching

Data prefetching can boost execution performance by fetching data before it is needed.

Preload instructions

For cases that cannot be handled efficiently by data prefetchers, the Neoverse™ N3 core supports the AArch64 prefetch memory instructions, `PRFM`.

These instructions signal to the memory system that memory accesses from a specified address are likely to occur soon. The memory system takes actions that aim to reduce the latency of memory accesses when they occur.

`PRFM` instructions perform a lookup in the cache. If they miss and the memory accesses are to a cacheable address, then a linefill starts. However, a `PRFM` instruction retires when its linefill is started, and it does not wait until the linefill is complete.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information on prefetch memory and preloading caches.

Hardware data prefetcher

The load/store unit includes hardware prefetcher engines that are responsible for generating prefetches targeting L1 and L2 caches. Specifically, the prefetch engine in the L1 memory subsystem targets the L1 and L2 caches. The prefetch engine in the L2 memory subsystem targets the L2 cache. The load side prefetcher uses the *Virtual Address* (VA) and the *Program Counter* (PC). The store side prefetcher uses the *Virtual Address* (VA) only.

The CPUECTLR registers allow control over some aspects of the prefetcher behavior. For more information, see:

- [A.1.11 IMP_CPUECTLR_EL1, CPU Extended Control Register](#) on page 188
- [A.1.12 IMP_CPUECTLR2_EL1, CPU Extended Control Register](#) on page 198

Data cache zero

In the Neoverse™ N3 core, the *Data Cache Zero by Virtual Address* (`DC ZVA`) instruction sets a 64-byte block of memory, which is aligned to 64 bytes, to zero.

For more information, see the [Arm® Architecture Reference Manual for A-profile architecture](#).

8. L2 memory system

The Neoverse™ N3 core L2 memory system connects the core with the DSU-120 through the CPU bridge. It includes the private L2 cache.

The L2 cache is unified and private to each Neoverse™ N3 core in a cluster.

The L2 memory system includes data prefetcher engines that use the *Virtual Address* (VA) and the *Program Counter* (PC). The different engines are able to prefetch data in the L2 cache.

The following table shows the L2 memory system features.

Table 8-1: L2 memory system features

Feature	Description
L2 cache	128KB, 256KB, 512KB, 1024KB, or 2048KB 8-way set associative, 2 banks <i>Physically Indexed, Physically Tagged</i> (PIPT) - Optionally protected with <i>Error Correcting Code</i> (ECC)
Cache line length	64 bytes
Cache policy	Dynamic biased cache replacement policy
Interface with the DSU-120	One CHI Issue E compliant interfaces with 256-bit read and write channel widths

8.1 L2 cache

The integrated L2 cache handles both instruction and data requests from the instruction and data side, as well as translation table walk requests.

- When BROADCASTICINVAL input is 1: The L1 instruction cache and L2 cache are weakly inclusive. Instruction fetches that miss in the L1 instruction cache and L2 cache allocate both caches, but the invalidation of the L2 cache does not cause back-invalidates of the L1 instruction cache.
- When BROADCASTICINVAL input is 0: The L1 instruction cache and L2 cache are strictly inclusive. Any data contained in the L1 instruction cache is also present in the L2 cache. Victimization from the L2 cache can cause invalidations of the L1 instruction cache.

The L1 data cache and L2 cache are strictly exclusive. Any data contained in the L1 data cache is never present in the L2 cache.

The L2 cache is invalidated automatically at reset unless the core power mode is initialized to Debug recovery mode.

**Note**

The way that cache indices are determined means that there is no direct relationship between the *Physical Address* (PA) and set number. You cannot use targeted operations that assume a relationship between the PA and set number. To flush the entire cache, you must perform set and way maintenance operations over the number of sets and ways described in CCSIDR_EL1 for that cache. This operation is compliant with the Armv8-A architecture.

Related information

4.4.5 [Debug recovery mode](#) on page 44

8.2 Support for memory types

The Neoverse™ N3 core simplifies coherency logic by downgrading some memory types.

Memory that is marked as both Inner Write-Back Cacheable and Outer Write-Back Cacheable is cached in the L1 data cache and the L2 cache.

Memory that is marked as Inner Write-Through is downgraded to Non-cacheable.

Memory that is marked Outer Write-Through or Outer Non-cacheable is downgraded to Non-cacheable, even if the inner attributes are Write-Back Cacheable.

The additional attribute hints are used as follows:

Allocation hint

Allocation hints help to determine the rules of allocation of newly fetched lines in the system.

Transient hint

Lines that are marked as transient are not allocated in downstream caches when they are evicted from the L1 cache, and they behave the same as non-transient lines in all other situations.

8.3 Transaction capabilities

The interface between the Neoverse™ N3 core L2 memory system and the DSU-120 provides transaction capabilities for the core.

The following table shows the maximum possible values for read, write, *Distributed Virtual Memory* (DVM) issuing, and snoop capabilities of the Neoverse™ N3 core L2 cache.

Table 8-2: Neoverse™ N3 core transaction capabilities

Attribute	Maximum value	Description
Write issuing capability	124	This is the maximum number of outstanding write transactions for memory that is cacheable, Non-cacheable, and Device GRE/nGRE.
	62	This is the maximum number of outstanding write transactions for memory that is Device nGnRE and nGnRnE.
Read issuing capability	62	This is the maximum number of outstanding read transactions for memory that is cacheable, Non-cacheable, and Device GRE/nGRE.
	31	This is the maximum number of outstanding read transactions for memory that is Device nGnRE and nGnRnE.
Snoop acceptance capability	64	This is the maximum number of outstanding snoops accepted.
DVM issuing capability	31	This is the maximum number of outstanding DVM operation transactions.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for information on the different memory types.

9. Direct access to internal memory

The Neoverse™ N3 core provides a mechanism to read the internal memory that the L1 caches, L2 cache, and *Translation Lookaside Buffer* (TLB) structures use through **IMPLEMENTATION DEFINED** System registers. When the coherency between the cache data and the system memory data is broken, you can use this mechanism to investigate any issues.

Direct access to internal memory is available only in EL3. In all other exception levels, executing these instructions results in an Undefined Instruction exception.

You can access the contents of the internal memory using the twelve read-only (RO) System registers in [Table 9-1: System registers used to access internal memory](#) on page 70. The internal memory is selected by programing the **IMPLEMENTATION DEFINED** RAMINDEX register using the following sys instruction:

```
SYS #6, C15, C0, #0, <Xt>
```

For more information on the RAMINDEX register, see [A.3.1 RAMINDEX, RAMINDEX system instruction](#) on page 327. The data is read from the read-only System registers as shown in the following table.



Note

- All the System registers are read-only (RO) and 64-bits wide
- For the register reset value, see the individual bit resets
- Any access to the data registers returns data
- Click the register name for details on the returned data format

Table 9-1: System registers used to access internal memory

Register name	Description	Access encoding
IMP_ISIDE_DATA0_EL3	Instruction Data register 0	MRS <Xt>, S3_6_C15_C0_0
IMP_ISIDE_DATA1_EL3	Instruction Data register 1	MRS <Xt>, S3_6_C15_C0_1
IMP_ISIDE_DATA2_EL3	Instruction Data register 2	MRS <Xt>, S3_6_C15_C0_2
IMP_DSIDE_DATA0_EL3	L1D Data register 0	MRS <Xt>, S3_6_C15_C1_0
IMP_DSIDE_DATA1_EL3	L1D Data register 1	MRS <Xt>, S3_6_C15_C1_1
IMP_DSIDE_DATA2_EL3	L1D Data register 2	MRS <Xt>, S3_6_C15_C1_2
IMP_L2_DATA0_EL3	L2 Data register 0	MRS <Xt>, S3_6_C15_C1_3
IMP_L2_DATA1_EL3	L2 Data register 1	MRS <Xt>, S3_6_C15_C1_5
IMP_L2_DATA2_EL3	L2 Data register 2	MRS <Xt>, S3_6_C15_C1_4
IMP_MMU_DATA0_EL3	TLB Data register 0	MRS <Xt>, S3_6_C15_C0_3
IMP_MMU_DATA1_EL3	TLB Data register 1	MRS <Xt>, S3_6_C15_C0_4
IMP_MMU_DATA2_EL3	TLB Data register 2	MRS <Xt>, S3_6_C15_C0_5

9.1 L1 cache encodings

Both the L1 data and instruction caches are 4-way set associative.

The size of the configured cache determines the number of sets in each way. The encoding that is used to locate the cache data entry for tag and data memory is set in x_n in the appropriate `sys` instruction. It is similar for both the tag and data RAM access.

The following tables show the encodings required for locating and selecting a given cache line.

Table 9-2: Neoverse™ N3 L1 instruction cache tag location encoding for 64KB

Bit field of X_n	Description
[31:24]	RAMID = 0x00
[23:20]	Reserved
[19:18]	Way
[17:14]	Reserved
[13:6]	Virtual Address bits[13:6]
[5:0]	Reserved

Table 9-3: Neoverse™ N3 L1 instruction cache tag location encoding for 32KB

Bit field of X_n	Description
[31:24]	RAMID = 0x00
[23:20]	Reserved
[19:18]	Way
[17:13]	Reserved
[12:6]	Virtual Address bits[12:6]
[5:0]	Reserved

Table 9-4: Neoverse™ N3 L1 instruction cache data location encoding for 64KB

Bit field of X_n	Description
[31:24]	RAMID = 0x01
[23:20]	Reserved
[19:18]	Way
17	Reserved
[16:14]	Virtual Address bits[5:3]
[13:6]	Virtual Address bits[13:6]
[5:0]	Reserved

Table 9-5: Neoverse™ N3 L1 instruction cache data location encoding for 32KB

Bit field of X_n	Description
[31:24]	RAMID = 0x01
[23:20]	Reserved
[19:18]	Way

Bit field of Xn	Description
17	Reserved
[16:14]	Virtual Address bits[5:3]
13	Reserved
[12:6]	Virtual Address bits[12:6]
[5:0]	Reserved

Table 9-6: Neoverse™ N3 L1 data cache tag location encoding for 64KB (<n> = 13), and 32KB (<n> = 12)

Bit field of Xn	Description
[31:24]	RAMID = 0x08
[23:20]	Reserved
[19:18]	Way
[17:16]	Bank selection 0b00 Tag RAM 0 0b01 Tag RAM 1 0b10 Tag RAM 2
[15:<n+1>]	Unused
[<n>:6]	Virtual Address bits[<n>:6]
[5:0]	Reserved

Table 9-7: Neoverse™ N3 L1 data cache data location encoding for 64KB (<n> = 13), and 32KB (<n> = 12)

Bit field of Xn	Description
[31:24]	RAMID = 0x09
[23:20]	Reserved
[19:18]	Way
[17:16]	Virtual Address bits[5:4]
[15:<n+1>]	Unused
[<n>:6]	Virtual Address bits[<n>:6]
[5:0]	Reserved

9.1.1 L1 RAM returned data

For each register, any access to the L1 RAM returns data.

Click the register name in the following table for details on the returned data format.

Table 9-8: Generic system control register summary

Name	Op0	CRn	Op1	CRm	Op2	Reset	Width	Description
IMP_ISIDE_DATA0_EL3	3	C15	6	C0	0	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 0

Name	Op0	CRn	Op1	CRm	Op2	Reset	Width	Description
IMP_ISIDE_DATA1_EL3	3	C15	6	C0	1	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 1
IMP_ISIDE_DATA2_EL3	3	C15	6	C0	2	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 2
IMP_DSIDE_DATA0_EL3	3	C15	6	C1	0	See individual bit resets.	64-bit	RAMINDEX L1D Data register 0
IMP_DSIDE_DATA1_EL3	3	C15	6	C1	1	See individual bit resets.	64-bit	RAMINDEX L1D Data register 1
IMP_DSIDE_DATA2_EL3	3	C15	6	C1	2	See individual bit resets.	64-bit	RAMINDEX L1D Data register 2

9.2 L2 cache encodings

The L2 cache is 8-way set associative.

The size of the configured cache determines the number of sets in each way. The encoding that is used to locate the cache data entry for tag and data memory is set in x_n in the appropriate sxs instruction. It is similar for both the tag and data RAM access.

The following tables show the encodings required for locating and selecting a given cache line.

Table 9-9: Neoverse™ N3 L2 cache tag location encoding for 2048KB (<n> = 17), 1024KB (<n> = 16), 512KB (<n> = 15), 256KB (<n> = 14), and 128KB (<n> = 13)

Bit field of X_n	Description
[31:24]	RAMID = 0x10
[23:21]	Reserved
[20:18]	Way
[17:7]	Physical Address bits[17:7] For L2 ≤ 1024KB (n ≤ 16), bits[17:<n>+1] are RES0
[6]	Superbank - Physical Address bit[6]
[5:0]	Reserved

Table 9-10: Neoverse™ N3 L2 cache data location encoding for 2048KB (<n> = 17), 1024KB (<n> = 16), 512KB (<n> = 15), 256KB (<n> = 14), and 128KB (<n> = 13)

Bit field of X_n	Description
[31:24]	RAMID = 0x11
[23:21]	Reserved
[20:18]	Way
[17:7]	Physical Address bits[17:7] For L2 ≤ 1024KB (n ≤ 16), bits[17:<n>+1] are RES0
[6]	Superbank - Physical Address bit[6]
[5:4]	16B granule inside the line
[3:0]	Reserved

9.2.1 L2 RAM returned data

For each register, any access to the L2 RAM returns data.

Click the register name in the following table for details on the returned data format.

Table 9-11: Generic system control register summary

Name	Op0	CRn	Op1	CRm	Op2	Reset	Width	Description
IMP_L2_DATA0_EL3	3	C15	6	C1	3	See individual bit resets.	64-bit	RAMINDEX L2 Data register 0
IMP_L2_DATA2_EL3	3	C15	6	C1	4	See individual bit resets.	64-bit	RAMINDEX L2 Data register 2
IMP_L2_DATA1_EL3	3	C15	6	C1	5	See individual bit resets.	64-bit	RAMINDEX L2 Data register 1

9.3 L2 TLB encodings

The L2 TLB RAM for small pages (TCSP) is 6-way set associative, and the L2 TLB RAM for medium pages (TCMP) is 4-way set associative.

The following tables show the encodings required for locating and selecting a given cache line.

Table 9-12: Neoverse™ N3 L2 TLB location encoding

Bit field of Xn	Description
[31:24]	RAMID = 0x18
[23:20]	Reserved
[19:16]	- TCSP: Way (0-5) - TCMP: Way (0-3)
[15:13]	Reserved
12	Array 0b0 TCSP 0b1 TCMP
[11:8]	Reserved
[7:0]	- TCSP: Index (0-255) - TCMP: Index (0-63)

9.3.1 L2 TLB RAM returned data

For each register, any access to the L2 TLB RAM returns data.

Click the register name in the following table for details on the returned data format.

Table 9-13: Generic system control register summary

Name	Op0	CRn	Op1	CRm	Op2	Reset	Width	Description
IMP_MMU_DATA0_EL3	3	C15	6	C0	3	See individual bit resets.	64-bit	RAMINDEX TLB Data register 0
IMP_MMU_DATA1_EL3	3	C15	6	C0	4	See individual bit resets.	64-bit	RAMINDEX TLB Data register 1
IMP_MMU_DATA2_EL3	3	C15	6	C0	5	See individual bit resets.	64-bit	RAMINDEX TLB Data register 2

10. RAS Extension support

The Neoverse™ N3 core supports the *Reliability, Availability, and Serviceability* (RAS) Extension, including all extensions up to Arm®v9.2-A.

In particular, the Neoverse™ N3 core supports these RAS Extension features:

- *Fault Handling Interrupts* (FHIs)
- *Error Recovery Interrupts* (ERIs)
- Poison attribute on bus transfers
- Cache protection with *Single Error Detect* (SED) parity on the functional RAMs that only contain clean data. This includes the L1 instruction cache tag, L1 instruction cache data, and the *Memory Management Unit* (MMU) RAMs.
- Cache protection with *Single Error Correct, Double Error Detect* (SECCDED), *Error Correcting Code* (ECC) on the functional RAMs that contain dirty data. This includes the L1 data cache tag, L1 data cache data, L2 cache tag, L2 cache data, and the L2 *Transaction Queue* (TQ) RAMs.
- Error Data Record registers to help software perform recovery actions
- Error injection capabilities to facilitate software and system debug
- The *Error Synchronization Barrier* (ESB) instruction to synchronize unrecoverable errors. When an `esb` instruction is executed, the core ensures that all SError interrupts that are generated by instructions before the `esb` are either taken or deferred. If the core cannot take the interrupt, it records the interrupt in the Deferred Interrupt Status Register DISR_EL1. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information on DISR_EL1.

Fault detection features are included in groups within the DSU-120 cluster and the Neoverse™ N3 core. Each group of fault detection features is referred to as a node. You can access each node by using either the System registers or the utility bus. The following nodes are implemented in the Neoverse™ N3 core and the DSU-120 cluster:

- Node 0 includes the private L1 memory system, L2 memory system, and the MMU/TLB in the core.

For more information on the architectural RAS Extension and the definition of a node, see the [Arm® Architecture Reference Manual for A-profile architecture](#).

10.1 Cache protection behavior

The configuration of the *Reliability, Availability, and Serviceability* (RAS) Extension that is implemented in the Neoverse™ N3 core includes cache protection. In this case, the Neoverse™ N3 core protects against errors that result in a RAM bitcell holding the incorrect value.

The RAMs in the Neoverse™ N3 core have the following capabilities:

SED parity

Single Error Detect (SED). One bit of parity is applicable to the protected data. The data size is specific for each RAM and depends on the protection granule.

SECDED ECC

Single Error Correct, Double Error Detect (SECDED), Error Correcting Code (ECC). The data size is specific for each RAM and depends on the protection granule.

The following table indicates which protection type is applied to each RAM in the Neoverse™ N3 core. The core can progress and remain functionally correct when there is a single-bit error in any RAM.

Table 10-1: RAM cache protection

RAM	Parity or ECC support
L1 instruction cache tag	SED parity
L1 instruction cache data	
<i>Translation Lookaside Buffer (TLB)</i>	
L1 data cache tag	SECDED ECC
L1 data cache auxiliary tag	
L1 data cache data	
L2 cache tag	
L2 cache data	
L2 <i>Transaction Queue (TQ)</i>	

If there are multiple single-bit errors in different RAMs, or within different protection granules within the same RAM, then the core also remains functionally correct.

If there is a double-bit error in a single RAM within the same protection granule, the behavior depends on the RAM:

- For RAMs with SECDED capability, the core detects, and either reports or defers the error. If the error is in a cache line containing dirty data, then that data might be lost.
- For RAMs with only SED, the core does not detect a double-bit error, which might cause data corruption.

If there are errors that are three or more bits within the same protection granule, the core might or might not detect the errors. Whether the core detects the errors or not depends on the RAM and the position of the errors within the RAM.

The cache protection feature of the core has a minimal performance impact when no errors are present.

10.2 Error containment

The Neoverse™ N3 core supports error containment for data errors. This means that detected data errors are not silently propagated. Data errors are deferred using data poisoning to ensure that

you are aware of the error. Uncorrectable L1 data cache tag errors and L2 cache tag errors are not containable.

Error containment also implies support for poisoning if there is a double error on an eviction. This ensures that the error of the associated data is reported when it is consumed.

Support for the *Error Synchronization Barrier* (ESB) instruction in the core also allows further isolation of imprecise exceptions that are reported when poisoned data is consumed.

10.3 Fault detection and reporting

When the Neoverse™ N3 core detects a fault, it raises a *Fault Handling Interrupt* (FHI) exception or an *Error Recovery Interrupt* (ERI) exception through the fault or the error signals. FHIs and ERIs are reflected in the *Reliability, Availability, and Serviceability* (RAS) registers, which are updated in the node that detects the errors.

Fault handling interrupts

When `ERRnCTLR.FI` is set, all detected Deferred errors, Uncorrected errors, and overflows of the corrected error counters generate an FHI. When `ERRnCTLR.CFI` is set, all detected Corrected errors also generate an FHI.

FHIs from core *n* are signaled using `nCOREFAULTIRQ[n]`.

Error recovery interrupts

When `ERRnCTLR.UI` is set, all detected Uncorrected errors that are not deferred generate an ERI.

ERIs from core *n* are signaled using `nCOREERRIRQ[n]`.

10.4 Error detection and reporting

When the Neoverse™ N3 core consumes an error, it raises different exceptions depending on the error type.

The Neoverse™ N3 core might raise:

- A *Synchronous External Abort* (SEA)
- An *Asynchronous External Abort* (AEA)
- An *Error Recovery Interrupt* (ERI)

10.4.1 Error reporting and performance monitoring

All memory errors detected by *Error Correcting Code* (ECC) or parity errors trigger the MEMORY_ERROR event.

The *Performance Monitoring Unit* (PMU) counters count the MEMORY_ERROR event if it is selected and the counter is enabled.

In Secure state, the MEMORY_ERROR event is counted only if MDCR_EL3.SPME is asserted. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for a description of MDCR_EL3.

Related information

[18.1 Performance monitors events](#) on page 110

10.5 Error injection

Error injection consists of inserting an error in the error detection logic to verify the error handling software.

Error injection uses the error detection and reporting registers to insert errors. The Neoverse™ N3 core can inject the following error types:

Corrected errors

A *Corrected Error* (CE) is generated for a single-bit *Error Correcting Code* (ECC) error on an L1 data cache access.

Deferred errors

A *Deferred Error* (DE) is generated for a double-bit ECC error on eviction of a cache line from the L1 cache to the L2 cache or as a result of a snoop on the L1 cache.

Uncontainable errors

An *Uncontainable Error* (UC) is generated for a double-bit ECC error on the L1 and L2 tag RAM following an eviction.

An error can be injected immediately or when a 32-bit counter reaches zero. You can control the value of the counter through the Error Pseudo-fault Generation Countdown Register, ER1PFGCDN_EL1. The value of the counter decrements on a per clock cycle basis. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about ER1PFGCDN_EL1.



Error injection is a separate source of error within the system and does not create hardware faults.

10.6 AArch64 RAS registers

The summary table provides an overview of all RAS registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 10-2: RAS registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ERRIDR_EL1	3	0	C5	C3	0	See individual bit resets.	64-bit	Error Record ID Register
ERRSELR_EL1	3	0	C5	C3	1	See individual bit resets.	64-bit	Error Record Select Register
ERXFR_EL1	3	0	C5	C4	0	See individual bit resets.	64-bit	Selected Error Record Feature Register
ERXCTLR_EL1	3	0	C5	C4	1	See individual bit resets.	64-bit	Selected Error Record Control Register
ERXSTATUS_EL1	3	0	C5	C4	2	See individual bit resets.	64-bit	Selected Error Record Primary Status Register
ERXADDR_EL1	3	0	C5	C4	3	See individual bit resets.	64-bit	Selected Error Record Address Register
ERXPFGF_EL1	3	0	C5	C4	4	See individual bit resets.	64-bit	Selected Pseudo-fault Generation Feature register
ERXPFGCTL_EL1	3	0	C5	C4	5	See individual bit resets.	64-bit	Selected Pseudo-fault Generation Control register
ERXPFGCDN_EL1	3	0	C5	C4	6	See individual bit resets.	64-bit	Selected Pseudo-fault Generation Countdown register
ERXMISCO_EL1	3	0	C5	C5	0	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 0
ERXMISC1_EL1	3	0	C5	C5	1	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 1
ERXMISC2_EL1	3	0	C5	C5	2	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 2
ERXMISC3_EL1	3	0	C5	C5	3	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 3
DISR_EL1	3	0	C12	C1	1	See individual bit resets.	64-bit	Deferred Interrupt Status Register
VSESR_EL2	3	4	C5	C2	3	See individual bit resets.	64-bit	Virtual SError Exception Syndrome Register
VDISR_EL2	3	4	C12	C1	1	See individual bit resets.	64-bit	Virtual Deferred Interrupt Status Register

11. Utility bus

The utility bus provides access to control registers for various system components in the DSU-120 and the core within the DSU-120 cluster. The utility bus is implemented as a 64-bit AMBA AXI5 slave port, and the control registers are memory-mapped onto the utility bus.

The utility bus provides access to the following system functions in the Neoverse™ N3 core:

- *Reliability, Availability, and Serviceability* (RAS) registers for the core
- *Activity Monitor Unit* (AMU) registers in the core
- *Maximum Power Mitigation Mechanism* (MPMM) registers in the core



Information about the *Power Policy Unit* (PPU) registers for the core in the cluster is provided in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#). For all other registers accessed by the utility bus, see *Utility bus* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).

11.1 Base addresses for system components

Each set of System registers is grouped on separate 64KB page boundaries allowing access to be enforced by a *Memory Management Unit* (MMU).

The following table shows the base addresses for each set of system component registers and what Security state they should be accessed from.



- The base address for each set of registers for the core RAS, AMU, and MPMM registers depend on the core instance number <n>, from 0 to the total number of cores in the cluster minus one.
- In the following table, any address space that is not documented is treated as **RAZ/WI**.
- The base addresses in the following table are the addresses accessed on the utility bus interface. The system interconnect typically maps these addresses into a particular address range based on the system address map. Therefore, software has to add the base address listed here onto the system address range base to get the absolute physical address of a register.

Table 11-1: Utility bus base addresses for system component registers

Base address, n is core instance number	Registers	Security state	Memory map
0x<n>9_0000	Core <n> AMU	Both	B.5 External AMU registers summary on page 725
0x<n>A_0000	Core <n> RAS	Secure	B.7 External RAS registers summary on page 815
0x<n>B_0000	Core <n> MPMM	Secure	B.1 External MPMM registers summary on page 609

Base address, n is core instance number	Registers	Security state	Memory map
0x<n>D_0000 - 0x<n>F_0000	Reserved	-	-



For more information on utility bus base addresses for system component registers, see the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).

12. GIC CPU interface

The *Generic Interrupt Controller* (GIC) supports and controls interrupts. The GIC Distributor connects to the Neoverse™ N3 core through a GIC CPU interface. The GIC CPU interface includes registers to mask, identify, and control the state of interrupts that are forwarded to the core.

Each core in a DSU-120 cluster has a GIC CPU interface, which connects to a common external Distributor component.

The GICv4.1 architecture implemented in the Neoverse™ N3 core supports:

- Two Security states
- Secure virtualization
- *Software-Generated Interrupts* (SGIs)
- Message-based interrupts
- System register access for the CPU interface
- Interrupt masking and prioritization
- Cluster environments, including systems that contain more than eight cores
- Wakeup events in power management environments

The GIC includes interrupt grouping functionality that supports:

- Configuring each interrupt to belong to either Group 0 or Group 1, where Group 0 interrupts are always Secure
- Signaling Group 1 interrupts to the target core using either the IRQ or the FIQ exception request. Group 1 interrupts can be Secure or Non-secure
- Signaling Group 0 interrupts to the target core using the FIQ exception request only
- A unified scheme for handling the priority of Group 0 and Group 1 interrupts

See the [Arm® Generic Interrupt Controller Architecture Specification, GIC architecture version 3 and version 4](#) for more information about interrupt groups.

12.1 Disable the GIC CPU interface

The Neoverse™ N3 core always includes the *Generic Interrupt Controller* (GIC) CPU interface. However, you can disable it to meet your requirements.

To disable the GIC CPU interface, assert the GICCDISABLE signal HIGH at reset. If you disable it this way, then you can use an external GIC IP to drive the interrupt signals (nFIQ, nIRQ). If the Neoverse™ N3 core is not integrated with an external GIC interrupt Distributor component (minimum GICv3 architecture) in the system, then you must disable the GIC CPU interface.

If you disable the GIC CPU interface, then:

- The virtual input signals nVIRQ and nVFIQ and the input signals nIRQ and nFIQ can be driven by an external GIC in the SoC.
- GIC system register access generates **UNDEFINED** instruction exceptions.

**Note**

If you enable the GIC CPU interface, then you must tie off nVIRQ and nVFIQ to HIGH. This is because the GIC CPU interface generates the virtual interrupt signals to the core. The nIRQ and nFIQ signals are controlled by software, therefore there is no requirement to tie them HIGH.

See *Functional integration* in the *Arm® DynamIQ™ Shared Unit-120 Configuration and Integration Manual* for more information on these signals.

12.2 AArch64 GIC system registers

The summary table provides an overview of all GIC system registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 12-1: GIC system registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ICC_PMR_EL1	3	0	C4	C6	0	See individual bit resets.	64-bit	Interrupt Controller Interrupt Priority Mask Register
ICV_PMR_EL1	3	0	C4	C6	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Priority Mask Register
ICC_IAR0_EL1	3	0	C12	C8	0	See individual bit resets.	64-bit	Interrupt Controller Interrupt Acknowledge Register 0
ICV_IAR0_EL1	3	0	C12	C8	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Acknowledge Register 0
ICC_EOIR0_EL1	3	0	C12	C8	1	See individual bit resets.	64-bit	Interrupt Controller End Of Interrupt Register 0
ICV_EOIR0_EL1	3	0	C12	C8	1	See individual bit resets.	64-bit	Interrupt Controller Virtual End Of Interrupt Register 0
ICC_HPPIRO_EL1	3	0	C12	C8	2	See individual bit resets.	64-bit	Interrupt Controller Highest Priority Pending Interrupt Register 0
ICV_HPPIRO_EL1	3	0	C12	C8	2	See individual bit resets.	64-bit	Interrupt Controller Virtual Highest Priority Pending Interrupt Register 0
ICC_BPR0_EL1	3	0	C12	C8	3	See individual bit resets.	64-bit	Interrupt Controller Binary Point Register 0
ICV_BPR0_EL1	3	0	C12	C8	3	See individual bit resets.	64-bit	Interrupt Controller Virtual Binary Point Register 0

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ICC_AP0R0_EL1	3	0	C12	C8	4	See individual bit resets.	64-bit	Interrupt Controller Active Priorities Group 0 Registers
ICV_AP0R0_EL1	3	0	C12	C8	4	See individual bit resets.	64-bit	Interrupt Controller Virtual Active Priorities Group 0 Registers
ICC_AP1R0_EL1	3	0	C12	C9	0	See individual bit resets.	64-bit	Interrupt Controller Active Priorities Group 1 Registers
ICV_AP1R0_EL1	3	0	C12	C9	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Active Priorities Group 1 Registers
ICC_DIR_EL1	3	0	C12	C11	1	See individual bit resets.	64-bit	Interrupt Controller Deactivate Interrupt Register
ICV_DIR_EL1	3	0	C12	C11	1	See individual bit resets.	64-bit	Interrupt Controller Deactivate Virtual Interrupt Register
ICC_RPR_EL1	3	0	C12	C11	3	See individual bit resets.	64-bit	Interrupt Controller Running Priority Register
ICV_RPR_EL1	3	0	C12	C11	3	See individual bit resets.	64-bit	Interrupt Controller Virtual Running Priority Register
ICC_SGI1R_EL1	3	0	C12	C11	5	See individual bit resets.	64-bit	Interrupt Controller Software Generated Interrupt Group 1 Register
ICC_ASIG1R_EL1	3	0	C12	C11	6	See individual bit resets.	64-bit	Interrupt Controller Alias Software Generated Interrupt Group 1 Register
ICC_SGI0R_EL1	3	0	C12	C11	7	See individual bit resets.	64-bit	Interrupt Controller Software Generated Interrupt Group 0 Register
ICC_IAR1_EL1	3	0	C12	C12	0	See individual bit resets.	64-bit	Interrupt Controller Interrupt Acknowledge Register 1
ICV_IAR1_EL1	3	0	C12	C12	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Acknowledge Register 1
ICC_EOIR1_EL1	3	0	C12	C12	1	See individual bit resets.	64-bit	Interrupt Controller End Of Interrupt Register 1
ICV_EOIR1_EL1	3	0	C12	C12	1	See individual bit resets.	64-bit	Interrupt Controller Virtual End Of Interrupt Register 1
ICC_HPIR1_EL1	3	0	C12	C12	2	See individual bit resets.	64-bit	Interrupt Controller Highest Priority Pending Interrupt Register 1
ICV_HPIR1_EL1	3	0	C12	C12	2	See individual bit resets.	64-bit	Interrupt Controller Virtual Highest Priority Pending Interrupt Register 1
ICC_BPR1_EL1	3	0	C12	C12	3	See individual bit resets.	64-bit	Interrupt Controller Binary Point Register 1
ICV_BPR1_EL1	3	0	C12	C12	3	See individual bit resets.	64-bit	Interrupt Controller Virtual Binary Point Register 1
ICC_CTLR_EL1	3	0	C12	C12	4	See individual bit resets.	64-bit	Interrupt Controller Control Register (EL1)
ICV_CTLR_EL1	3	0	C12	C12	4	See individual bit resets.	64-bit	Interrupt Controller Virtual Control Register
ICC_SRE_EL1	3	0	C12	C12	5	See individual bit resets.	64-bit	Interrupt Controller System Register Enable register (EL1)
ICC_IGRPEN0_EL1	3	0	C12	C12	6	See individual bit resets.	64-bit	Interrupt Controller Interrupt Group 0 Enable register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ICV_IGRPEN0_EL1	3	0	C12	C12	6	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Group 0 Enable register
ICC_IGRPEN1_EL1	3	0	C12	C12	7	See individual bit resets.	64-bit	Interrupt Controller Interrupt Group 1 Enable register
ICV_IGRPEN1_EL1	3	0	C12	C12	7	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Group 1 Enable register
ICH_AP0R0_EL2	3	4	C12	C8	0	See individual bit resets.	64-bit	Interrupt Controller Hyp Active Priorities Group 0 Registers
ICH_AP1R0_EL2	3	4	C12	C9	0	See individual bit resets.	64-bit	Interrupt Controller Hyp Active Priorities Group 1 Registers
ICC_SRE_EL2	3	4	C12	C9	5	See individual bit resets.	64-bit	Interrupt Controller System Register Enable register (EL2)
ICH_HCR_EL2	3	4	C12	C11	0	See individual bit resets.	64-bit	Interrupt Controller Hyp Control Register
ICH_VTR_EL2	3	4	C12	C11	1	See individual bit resets.	64-bit	Interrupt Controller VGIC Type Register
ICH_MISR_EL2	3	4	C12	C11	2	See individual bit resets.	64-bit	Interrupt Controller Maintenance Interrupt State Register
ICH_EISR_EL2	3	4	C12	C11	3	See individual bit resets.	64-bit	Interrupt Controller End of Interrupt Status Register
ICH_ELRSR_EL2	3	4	C12	C11	5	See individual bit resets.	64-bit	Interrupt Controller Empty List Register Status Register
ICH_VMCR_EL2	3	4	C12	C11	7	See individual bit resets.	64-bit	Interrupt Controller Virtual Machine Control Register
ICH_LR0_EL2	3	4	C12	C12	0	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICH_LR1_EL2	3	4	C12	C12	1	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICH_LR2_EL2	3	4	C12	C12	2	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICH_LR3_EL2	3	4	C12	C12	3	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICC_CTLR_EL3	3	6	C12	C12	4	See individual bit resets.	64-bit	Interrupt Controller Control Register (EL3)
ICC_SRE_EL3	3	6	C12	C12	5	See individual bit resets.	64-bit	Interrupt Controller System Register Enable register (EL3)
ICC_IGRPEN1_EL3	3	6	C12	C12	7	See individual bit resets.	64-bit	Interrupt Controller Interrupt Group 1 Enable register (EL3)

13. Advanced SIMD and floating-point support

The Neoverse™ N3 core supports the Advanced *Single Instruction Multiple Data* (SIMD) and scalar floating-point instructions in the A64 instruction set without floating-point exception trapping.

The Neoverse™ N3 core floating-point implementation includes features up to Arm®v9.2-A. BFloat16 floating-point and Int8 matrix multiplication are part of these supported features.

The Neoverse™ N3 core implements all operations in hardware with support for all combinations of:

- Rounding modes
- Flush-to-zero
- Default *Not a Number* (NaN) modes

The Neoverse™ N3 core supports *Alternate Floating Point* behavior (FEAT_AFP), as part of Arm®v8.7-A and Arm®v9.2-A.

14. Scalable Vector Extensions support

The Neoverse™ N3 core supports the *Scalable Vector Extension* (SVE) and the *Scalable Vector Extension 2* (SVE2). SVE and SVE2 are intended to complement, not replace, AArch64 Advanced *Single Instruction Multiple Data* (SIMD) and floating-point functionality.

SVE is an optional extension introduced by the Armv8.2 architecture. The key features that SVE provides are:

- Predication
- Gather-load and scatter-store
- Software-managed speculative vectorization

The Neoverse™ N3 core implements a scalable vector length of 128 bits.

All the features and additions that SVE and SVE2 introduce are described in the [Arm® Architecture Reference Manual for A-profile architecture](#).

15. System control

The system registers control and provide status information for the functions that the core implements.

The main functions of the system registers are:

- System performance monitoring
- Cache configuration and management
- Overall system control and configuration
- *Memory Management Unit* (MMU) configuration and management
- *Generic Interrupt Controller* (GIC) configuration and management

The system registers are accessible in AArch64 Execution state at EL0 to EL3. Some of the system registers are accessible through the external debug interface or utility bus interface.

15.1 AArch64 Generic System Control registers

The summary table provides an overview of all Generic System Control registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 15-1: Generic System Control registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ACTLR_EL1	3	0	C1	C0	1	See individual bit resets.	64-bit	Auxiliary Control Register (EL1)
RGSR_EL1	3	0	C1	C0	5	See individual bit resets.	64-bit	Random Allocation Tag Seed Register.
GCR_EL1	3	0	C1	C0	6	See individual bit resets.	64-bit	Tag Control Register.
TTBR0_EL1	3	0	C2	C0	0	See individual bit resets.	64-bit	Translation Table Base Register 0 (EL1)
TTBR1_EL1	3	0	C2	C0	1	See individual bit resets.	64-bit	Translation Table Base Register 1 (EL1)
TCR_EL1	3	0	C2	C0	2	See individual bit resets.	64-bit	Translation Control Register (EL1)
APIAKeyLo_EL1	3	0	C2	C1	0	See individual bit resets.	64-bit	Pointer Authentication Key A for Instruction (bits[63:0])

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
APIAKeyHi_EL1	3	0	C2	C1	1	See individual bit resets.	64-bit	Pointer Authentication Key A for Instruction (bits[127:64])
APIBKeyLo_EL1	3	0	C2	C1	2	See individual bit resets.	64-bit	Pointer Authentication Key B for Instruction (bits[63:0])
APIBKeyHi_EL1	3	0	C2	C1	3	See individual bit resets.	64-bit	Pointer Authentication Key B for Instruction (bits[127:64])
APDAKeyLo_EL1	3	0	C2	C2	0	See individual bit resets.	64-bit	Pointer Authentication Key A for Data (bits[63:0])
APDAKeyHi_EL1	3	0	C2	C2	1	See individual bit resets.	64-bit	Pointer Authentication Key A for Data (bits[127:64])
APDBKeyLo_EL1	3	0	C2	C2	2	See individual bit resets.	64-bit	Pointer Authentication Key B for Data (bits[63:0])
APDBKeyHi_EL1	3	0	C2	C2	3	See individual bit resets.	64-bit	Pointer Authentication Key B for Data (bits[127:64])
APGAKeyLo_EL1	3	0	C2	C3	0	See individual bit resets.	64-bit	Pointer Authentication Key A for Code (bits[63:0])
APGAKeyHi_EL1	3	0	C2	C3	1	See individual bit resets.	64-bit	Pointer Authentication Key A for Code (bits[127:64])
SPSel	3	0	C4	C2	0	See individual bit resets.	64-bit	Stack Pointer Select
CurrentEL	3	0	C4	C2	2	See individual bit resets.	64-bit	Current Exception Level
PAN	3	0	C4	C2	3	See individual bit resets.	64-bit	Privileged Access Never
UAO	3	0	C4	C2	4	See individual bit resets.	64-bit	User Access Override
AFSRO_EL1	3	0	C5	C1	0	See individual bit resets.	64-bit	Auxiliary Fault Status Register 0 (EL1)
AFSR1_EL1	3	0	C5	C1	1	See individual bit resets.	64-bit	Auxiliary Fault Status Register 1 (EL1)
ESR_EL1	3	0	C5	C2	0	See individual bit resets.	64-bit	Exception Syndrome Register (EL1)
TFSR_EL1	3	0	C5	C6	0	See individual bit resets.	64-bit	Tag Fault Status Register (EL1)
TFSREO_EL1	3	0	C5	C6	1	See individual bit resets.	64-bit	Tag Fault Status Register (EL0).
FAR_EL1	3	0	C6	C0	0	See individual bit resets.	64-bit	Fault Address Register (EL1)
PAR_EL1	3	0	C7	C4	0	See individual bit resets.	64-bit	Physical Address Register
MAIR_EL1	3	0	C10	C2	0	See individual bit resets.	64-bit	Memory Attribute Indirection Register (EL1)
AMAIR_EL1	3	0	C10	C3	0	See individual bit resets.	64-bit	Auxiliary Memory Attribute Indirection Register (EL1)
LORSA_EL1	3	0	C10	C4	0	See individual bit resets.	64-bit	LORegion Start Address (EL1)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
LOREA_EL1	3	0	C10	C4	1	See individual bit resets.	64-bit	LORegion End Address (EL1)
LORN_EL1	3	0	C10	C4	2	See individual bit resets.	64-bit	LORegion Number (EL1)
LORC_EL1	3	0	C10	C4	3	See individual bit resets.	64-bit	LORegion Control (EL1)
LORID_EL1	3	0	C10	C4	7	See individual bit resets.	64-bit	LORegionID (EL1)
VBAR_EL1	3	0	C12	C0	0	See individual bit resets.	64-bit	Vector Base Address Register (EL1)
ISR_EL1	3	0	C12	C1	0	See individual bit resets.	64-bit	Interrupt Status Register
CONTEXTIDR_EL1	3	0	C13	C0	1	See individual bit resets.	64-bit	Context ID Register (EL1)
TPIDR_EL1	3	0	C13	C0	4	See individual bit resets.	64-bit	EL1 Software Thread ID Register
SCXTNUM_EL1	3	0	C13	C0	7	See individual bit resets.	64-bit	EL1 Read/Write Software Context Number
IMP_CPUCFR_EL1	3	0	C15	C0	0	See individual bit resets.	64-bit	CPU Configuration Register
IMP_CPUACTLR_EL1	3	0	C15	C1	0	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR2_EL1	3	0	C15	C1	1	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR3_EL1	3	0	C15	C1	2	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR4_EL1	3	0	C15	C1	3	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUECTLR_EL1	3	0	C15	C1	4	See individual bit resets.	64-bit	CPU Extended Control Register
IMP_CPUECTLR2_EL1	3	0	C15	C1	5	See individual bit resets.	64-bit	CPU Extended Control Register
IMP_CPUBUSQOS_EL1	3	0	C15	C1	7	See individual bit resets.	64-bit	Quality of Service Bus Control Register, EL1
IMP_CPUPWRCTLR_EL1	3	0	C15	C2	7	See individual bit resets.	64-bit	CPU Power Control Register
IMP_CLUSTERECFR_EL1	3	0	C15	C3	0	See individual bit resets.	64-bit	Cluster Configuration Register
IMP_CLUSTERIDR_EL1	3	0	C15	C3	1	See individual bit resets.	64-bit	Cluster Main Revision Register
IMP_CLUSTERREVIDR_EL1	3	0	C15	C3	2	See individual bit resets.	64-bit	Cluster ECO ID Register
IMP_CLUSTERACTLR_EL1	3	0	C15	C3	3	See individual bit resets.	64-bit	Cluster Auxiliary Control Register
IMP_CLUSTERECTLR_EL1	3	0	C15	C3	4	See individual bit resets.	64-bit	Cluster Extended Control Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CLUSTERPWRCTLR_EL1	3	0	C15	C3	5	See individual bit resets.	64-bit	Cluster Power Control Register
IMP_CLUSTERPWDRN_EL1	3	0	C15	C3	6	See individual bit resets.	64-bit	Cluster Power Down Register
IMP_CLUSTERPWRSTAT_EL1	3	0	C15	C3	7	See individual bit resets.	64-bit	Cluster Power Status Register
IMP_CLUSTERL3DNTH0_EL1	3	0	C15	C4	0	See individual bit resets.	64-bit	Cluster L3 Downsize Threshold0 Register
IMP_CLUSTERL3DNTH1_EL1	3	0	C15	C4	1	See individual bit resets.	64-bit	Cluster L3 Downsize Threshold1 Register
IMP_CLUSTERL3UPTH0_EL1	3	0	C15	C4	2	See individual bit resets.	64-bit	Cluster L3 Upsize Threshold0 Register
IMP_CLUSTERL3UPTH1_EL1	3	0	C15	C4	3	See individual bit resets.	64-bit	Cluster L3 Upsize Threshold1 Register
IMP_CLUSTERBUSQOS_EL1	3	0	C15	C4	4	See individual bit resets.	64-bit	Cluster Bus QoS Control Register
IMP_CLUSTERL3HIT_EL1	3	0	C15	C4	5	See individual bit resets.	64-bit	Cluster L3 Hit Counter Register
IMP_CLUSTERL3MISS_EL1	3	0	C15	C4	6	See individual bit resets.	64-bit	Cluster L3 Miss Counter Register
IMP_ATCR_EL1	3	0	C15	C7	0	See individual bit resets.	64-bit	CPU Auxiliary Translation Control Register
IMP_CPUACTLR5_EL1	3	0	C15	C8	0	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR6_EL1	3	0	C15	C8	1	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR7_EL1	3	0	C15	C8	2	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR8_EL1	3	0	C15	C8	3	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CLUSTERPPSTART_EL1	3	0	C15	C9	0	See individual bit resets.	64-bit	Cluster Peripheral Port Start Address Register
IMP_CLUSTERPPEND_EL1	3	0	C15	C9	1	See individual bit resets.	64-bit	Cluster Peripheral Port End Address Register
IMP_CLUSTERRCFR2_EL1	3	0	C15	C9	2	See individual bit resets.	64-bit	Cluster Configuration Register 2
IMP_CLUSTERL3UPTH2_EL1	3	0	C15	C9	3	See individual bit resets.	64-bit	Cluster L3 Upsize Threshold2 Register
IMP_CLUSTERRSVD_9_4_EL1	3	0	C15	C9	4	See individual bit resets.	64-bit	RESERVED
IMP_CLUSTERRSVD_9_5_EL1	3	0	C15	C9	5	See individual bit resets.	64-bit	RESERVED
IMP_CLUSTERRSVD_9_6_EL1	3	0	C15	C9	6	See individual bit resets.	64-bit	RESERVED
IMP_CLUSTERRSVD_9_7_EL1	3	0	C15	C9	7	See individual bit resets.	64-bit	RESERVED

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AIDR_EL1	3	1	C0	C0	7	See individual bit resets.	64-bit	Auxiliary ID Register
RNDR	3	3	C2	C4	0	See individual bit resets.	64-bit	Random Number
RNDRRS	3	3	C2	C4	1	See individual bit resets.	64-bit	Reseeded Random Number
NZCV	3	3	C4	C2	0	See individual bit resets.	64-bit	Condition Flags
DAIF	3	3	C4	C2	1	See individual bit resets.	64-bit	Interrupt Mask Bits
DIT	3	3	C4	C2	5	See individual bit resets.	64-bit	Data Independent Timing
SSBS	3	3	C4	C2	6	See individual bit resets.	64-bit	Speculative Store Bypass Safe
TCO	3	3	C4	C2	7	See individual bit resets.	64-bit	Tag Check Override
FPCR	3	3	C4	C4	0	See individual bit resets.	64-bit	Floating-point Control Register
FPSR	3	3	C4	C4	1	See individual bit resets.	64-bit	Floating-point Status Register
TPIDR_ELO	3	3	C13	C0	2	See individual bit resets.	64-bit	ELO Read/Write Software Thread ID Register
TPIDRRO_ELO	3	3	C13	C0	3	See individual bit resets.	64-bit	ELO Read-Only Software Thread ID Register
SCXTNUM_ELO	3	3	C13	C0	7	See individual bit resets.	64-bit	ELO Read/Write Software Context Number
ACTLR_EL2	3	4	C1	C0	1	See individual bit resets.	64-bit	Auxiliary Control Register (EL2)
HACR_EL2	3	4	C1	C1	7	See individual bit resets.	64-bit	Hypervisor Auxiliary Control Register
TTBR0_EL2	3	4	C2	C0	0	See individual bit resets.	64-bit	Translation Table Base Register 0 (EL2)
TTBR1_EL2	3	4	C2	C0	1	See individual bit resets.	64-bit	Translation Table Base Register 1 (EL2)
TCR_EL2	3	4	C2	C0	2	See individual bit resets.	64-bit	Translation Control Register (EL2)
VTTBR_EL2	3	4	C2	C1	0	See individual bit resets.	64-bit	Virtualization Translation Table Base Register
VTCR_EL2	3	4	C2	C1	2	See individual bit resets.	64-bit	Virtualization Translation Control Register
VSTTBR_EL2	3	4	C2	C6	0	See individual bit resets.	64-bit	Virtualization Secure Translation Table Base Register
VSTCR_EL2	3	4	C2	C6	2	See individual bit resets.	64-bit	Virtualization Secure Translation Control Register
AFSRO_EL2	3	4	C5	C1	0	See individual bit resets.	64-bit	Auxiliary Fault Status Register 0 (EL2)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AFSR1_EL2	3	4	C5	C1	1	See individual bit resets.	64-bit	Auxiliary Fault Status Register 1 (EL2)
ESR_EL2	3	4	C5	C2	0	See individual bit resets.	64-bit	Exception Syndrome Register (EL2)
TFSR_EL2	3	4	C5	C6	0	See individual bit resets.	64-bit	Tag Fault Status Register (EL2)
FAR_EL2	3	4	C6	C0	0	See individual bit resets.	64-bit	Fault Address Register (EL2)
HPFAR_EL2	3	4	C6	C0	4	See individual bit resets.	64-bit	Hypervisor IPA Fault Address Register
MAIR_EL2	3	4	C10	C2	0	See individual bit resets.	64-bit	Memory Attribute Indirection Register (EL2)
AMAIR_EL2	3	4	C10	C3	0	See individual bit resets.	64-bit	Auxiliary Memory Attribute Indirection Register (EL2)
VBAR_EL2	3	4	C12	C0	0	See individual bit resets.	64-bit	Vector Base Address Register (EL2)
CONTEXTIDR_EL2	3	4	C13	C0	1	See individual bit resets.	64-bit	Context ID Register (EL2)
TPIDR_EL2	3	4	C13	C0	2	See individual bit resets.	64-bit	EL2 Software Thread ID Register
SCXTNUM_EL2	3	4	C13	C0	7	See individual bit resets.	64-bit	EL2 Read/Write Software Context Number
IMP_ATCR_EL2	3	4	C15	C7	0	See individual bit resets.	64-bit	CPU Auxiliary Translation Control Register
IMP_AVTCR_EL2	3	4	C15	C7	1	See individual bit resets.	64-bit	CPU Auxiliary Virtualization Translation Control Register
ACTLR_EL3	3	6	C1	C0	1	See individual bit resets.	64-bit	Auxiliary Control Register (EL3)
SCR_EL3	3	6	C1	C1	0	See individual bit resets.	64-bit	Secure Configuration Register
CPTR_EL3	3	6	C1	C1	2	See individual bit resets.	64-bit	Architectural Feature Trap Register (EL3)
MDCR_EL3	3	6	C1	C3	1	See individual bit resets.	64-bit	Monitor Debug Configuration Register (EL3)
TTBRO_EL3	3	6	C2	C0	0	See individual bit resets.	64-bit	Translation Table Base Register 0 (EL3)
TCR_EL3	3	6	C2	C0	2	See individual bit resets.	64-bit	Translation Control Register (EL3)
AFSR0_EL3	3	6	C5	C1	0	See individual bit resets.	64-bit	Auxiliary Fault Status Register 0 (EL3)
AFSR1_EL3	3	6	C5	C1	1	See individual bit resets.	64-bit	Auxiliary Fault Status Register 1 (EL3)
ESR_EL3	3	6	C5	C2	0	See individual bit resets.	64-bit	Exception Syndrome Register (EL3)
TFSR_EL3	3	6	C5	C6	0	See individual bit resets.	64-bit	Tag Fault Status Register (EL3)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
FAR_EL3	3	6	C6	C0	0	See individual bit resets.	64-bit	Fault Address Register (EL3)
MAIR_EL3	3	6	C10	C2	0	See individual bit resets.	64-bit	Memory Attribute Indirection Register (EL3)
AMAIR_EL3	3	6	C10	C3	0	See individual bit resets.	64-bit	Auxiliary Memory Attribute Indirection Register (EL3)
VBAR_EL3	3	6	C12	C0	0	See individual bit resets.	64-bit	Vector Base Address Register (EL3)
RVBAR_EL3	3	6	C12	C0	1	See individual bit resets.	64-bit	Reset Vector Base Address Register (if EL3 implemented)
RMR_EL3	3	6	C12	C0	2	See individual bit resets.	64-bit	Reset Management Register (EL3)
TPIDR_EL3	3	6	C13	C0	2	See individual bit resets.	64-bit	EL3 Software Thread ID Register
SCXTNUM_EL3	3	6	C13	C0	7	See individual bit resets.	64-bit	EL3 Read/Write Software Context Number
IMP_ISIDE_DATA0_EL3	3	6	C15	C0	0	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 0
IMP_ISIDE_DATA1_EL3	3	6	C15	C0	1	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 1
IMP_ISIDE_DATA2_EL3	3	6	C15	C0	2	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 2
IMP_MMU_DATA0_EL3	3	6	C15	C0	3	See individual bit resets.	64-bit	RAMINDEX TLB Data register 0
IMP_MMU_DATA1_EL3	3	6	C15	C0	4	See individual bit resets.	64-bit	RAMINDEX TLB Data register 1
IMP_MMU_DATA2_EL3	3	6	C15	C0	5	See individual bit resets.	64-bit	RAMINDEX TLB Data register 2
IMP_DSIDE_DATA0_EL3	3	6	C15	C1	0	See individual bit resets.	64-bit	RAMINDEX L1D Data register 0
IMP_DSIDE_DATA1_EL3	3	6	C15	C1	1	See individual bit resets.	64-bit	RAMINDEX L1D Data register 1
IMP_DSIDE_DATA2_EL3	3	6	C15	C1	2	See individual bit resets.	64-bit	RAMINDEX L1D Data register 2
IMP_L2_DATA0_EL3	3	6	C15	C1	3	See individual bit resets.	64-bit	RAMINDEX L2 Data register 0
IMP_L2_DATA2_EL3	3	6	C15	C1	4	See individual bit resets.	64-bit	RAMINDEX L2 Data register 2
IMP_L2_DATA1_EL3	3	6	C15	C1	5	See individual bit resets.	64-bit	RAMINDEX L2 Data register 1
IMP_CPURNDBR_EL3	3	6	C15	C3	0	See individual bit resets.	64-bit	Random Number Base Register, EL3
IMP_CPURNDPEID_EL3	3	6	C15	C3	1	See individual bit resets.	64-bit	Random Number Packet Identification Register, EL3
IMP_CLUSTERDBG_EL3	3	6	C15	C4	7	See individual bit resets.	64-bit	Cluster Cache Debug Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CLUSTERPMMDCR_EL3	3	6	C15	C6	3	See individual bit resets.	64-bit	Monitor Debug Configuration Register (EL3)
IMP_ATCR_EL3	3	6	C15	C7	0	See individual bit resets.	64-bit	CPU Auxiliary Translation Control Register
IMP_CPUPSELR_EL3	3	6	C15	C8	0	See individual bit resets.	64-bit	Selected Instruction Private Control Register
IMP_CPUPCR_EL3	3	6	C15	C8	1	See individual bit resets.	64-bit	Selected Instruction Patch Control Register
IMP_CPUPOR_EL3	3	6	C15	C8	2	See individual bit resets.	64-bit	Selected Instruction Patch Opcode Register
IMP_CPUPMR_EL3	3	6	C15	C8	3	See individual bit resets.	64-bit	Selected Instruction Patch Mask Register
IMP_CPUPOR2_EL3	3	6	C15	C8	4	See individual bit resets.	64-bit	Selected Instruction Patch Opcode Register 2
IMP_CPUPMR2_EL3	3	6	C15	C8	5	See individual bit resets.	64-bit	Selected Instruction Patch Mask Register 2
IMP_CPUPFR_EL3	3	6	C15	C8	6	See individual bit resets.	64-bit	Selected Instruction Private Flag Register

16. Random number generator support

The Neoverse™ N3 core supports two random number instructions introduced in the Arm®v8.5-A extension.

The Neoverse™ N3 core always supports the two following features:

- FEAT_RNG: The instructions rely on the availability of *True Random Number Generator* (TRNG) and *Deterministic Random Bit Generator* (DRBG), as described in the following section.
- FEAT_RNG_TRAP: The instructions trap to EL3. This should be used if the system does not contain a TRNG and DRBG as described in the following section.

The following instructions return a 64-bit random number into a general purpose register:

- MRS Xn, RNDR
- MRS Xn, RNDRRS

The Neoverse™ N3 core expects the *True Random Number Generator* (TRNG) and the *Deterministic Random Bit Generator* (DRBG) to be available as a memory-mapped peripherals and must be capable of the following requirements.

- Design the TRNG and DRBG as architecturally stipulated.
- Provide as many copies of TRNG and DRBG as is necessary to meet the overall bandwidth and latency requirements of the system.
- Reseed the DRBG from the TRNG when a RNDRRS instruction is received, as defined by the address encoding described in the Neoverse™ N3 core microarchitecture.
- Provide *Quality of Service* (QOS) managed access to DRBG bandwidth as architecturally defined.
- Provide access to each TRNG and DRBG block through a memory-mapped Dev-nGnRnE read (LDP Xreg).
- The address used by the LDP Xreg for RNDR and RNDRRS instructions is a physical-address defined as follows:
 - A combination of base register of 64K page (CPURNDBR_EL3[47:16]), PE-specific identifier (CPURNDPEID_EL3[10:0]), instruction-type.
 - RNDR address: {CPURNDBR_EL3[47:16], CPURNDPEID_EL3[10:0], 1'b0, 4'b0}
 - RNDRRS address: {CPURNDBR_EL3[47:16], CPURNDPEID_EL3[10:0], 1'b1, 4'b0}
- Set CPURNDBR_EL3[47:16] in each Neoverse™ N3 core to match the peripheral base of the TRNG and DRBG block corresponding to the core. The association of the core to TRNG and DRBG block is defined by the system integrator.
- The TRNG and DRBG block must correctly decode the read-address, using [15:5] as the unique core identifier for QOS guarantees.
- The TRNG and DRBG block must correctly decode bit [4] of the read-address, 0 specifying an RNDR instruction and 1 specifying an RNDRRS instruction.
- Upon receiving a RNDR or RNDRRS request, the TRNG and DRBG block must return a 64-bit random number in the first 64 bits and 1 in the second 64 bits. In the event that the DRBG

block is unable to provide a random number within a system integrator defined timeframe, it will return 0 in the first and second 64 bits.

- In the event of a bus error, a `RNDR` or `RNDRRS` request will fail and the core will set the `PSTATE.Z` flag and assert a SEI.



Note

The random number generator can be tested by running the *National institute of Standards and Technology* (NIST) tests available as part of the *SBSA Architecture Compliance Suite* (ACS). The SBSA ACS is available at <https://github.com/ARM-software/sbsa-acs>. The NIST tests are available at https://github.com/ARM-software/sbsa-acs/tree/master/test_pool/nist_sts

16.1 AArch64 Random Number Control registers summary

The summary table provides an overview of all Random Number Control registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the *Arm® Architecture Reference Manual for A-profile architecture*.

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the *Arm® Architecture Reference Manual for A-profile architecture*.

Table 16-1: Random Number Control registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CPURNDBR_EL3	3	6	C15	C3	0	See individual bit resets.	64-bit	CPU Random Number Base Register
IMP_CPURNDPEID_EL3	3	6	C15	C3	1	See individual bit resets.	64-bit	CPU Random Number Packet Identification Register
RNDR	3	3	C2	C4	0	See individual bit resets.	64-bit	Random Number
RNDRRS	3	3	C2	C4	1	See individual bit resets.	64-bit	Reseeded Random Number

17. Debug

The DSU-120 cluster provides a debug system that supports both self-hosted and external debug. It has an external DebugBlock component and integrates various CoreSight debug related components.

The CoreSight debug related components are split into two groups, with some components in the DSU-120 cluster, and others in the separate DebugBlock.

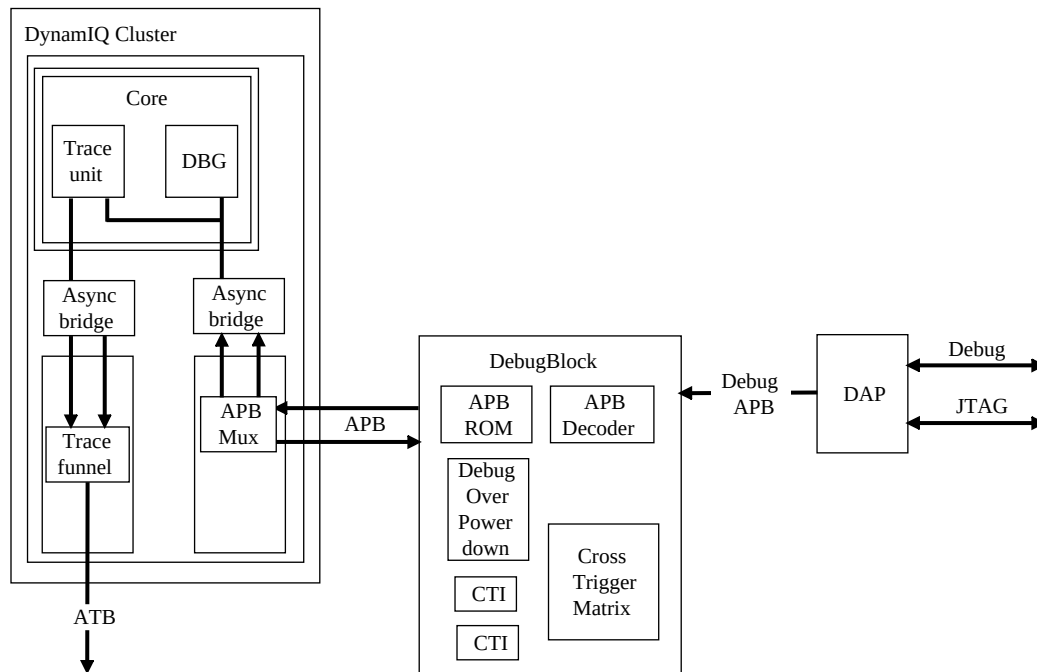
The DebugBlock is a dedicated debug component in the DSU-120, separate from the cluster. The DebugBlock operates within a separate power domain, which enables connection to a debugger to be maintained when the core and the DSU-120 cluster are both powered down.

The connection between the cluster and the DebugBlock consists of a pair of *Advanced Peripheral Bus* (APB) interfaces, one in each direction. All debug traffic, except the authentication interface, takes place over this interface as read or write APB transactions. This debug traffic includes register reads, register writes, and *Cross Trigger Interface* (CTI) triggers.

The debug system implements the following CoreSight debug components:

- Per-core trace unit, integrated into the CoreSight subsystem
- Per-core CTI, contained in the DebugBlock
- *Cross Trigger Matrix* (CTM)
- Debug control provided by AMBA® APB interface to the DebugBlock

The following figure shows how the debug system is implemented with the DSU-120 cluster.

Figure 17-1: DSU-120 cluster debug components

The primary debug APB interface on the DebugBlock controls the debug components. The APB decoder decodes the requests on this bus before they are sent to the appropriate component in the DebugBlock or in the DSU-120 cluster. The per-core CTIs are connected to a CTM.

The core contains a debug component that the debug APB bus accesses. The core supports debug over powerdown using modules in the DebugBlock that mirror key core information. These modules allow access to debug over powerdown CoreSight™ registers while the core is powered down.

The trace unit in the core outputs trace, which is funneled in the DSU-120 cluster down to a single AMBA® 4 ATBv1.1 interface.

See *Debug* in the [Arm® DynamiQ™ Shared Unit-120 Technical Reference Manual](#) for more information about the DSU-120 cluster debug components.

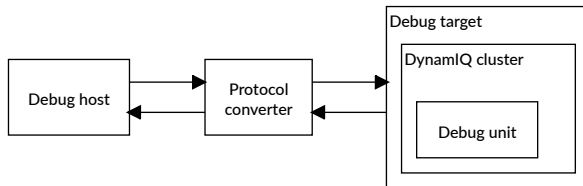
The Neoverse™ N3 core also supports direct access to internal memory, that is, cache debug. Direct access to internal memory allows software to read the internal memory that the L1 and L2 cache and *Translation Lookaside Buffer* (TLB) structures use. See [9. Direct access to internal memory](#) on page 70 for more information.

17.1 Supported debug methods

The DSU-120 cluster along with its associated core is part of a debug system that supports both self-hosted and external debug.

The following figure shows a typical external debug system.

Figure 17-2: External debug system



Debug host

A computer, for example a personal computer, that is running a software debugger such as the Arm® Debugger. You can use the debug host to issue high-level commands. For example, you can set a breakpoint at a certain location or examine the contents of a memory address.

Protocol converter

The debug host sends messages to the debug target using an interface such as Ethernet. However, the debug target typically implements a different interface protocol. A device such as DSTREAM is required to convert between the two protocols.

Debug target

The lowest level of the system implements system support for the protocol converter to access the debug unit. For DSU-120 based devices, the mechanism used to access the debug unit is based on the CoreSight architecture. The DSU-120 DebugBlock is accessed using an *Advanced Peripheral Bus* (APB) interface and the debug accesses are then directed to the Neoverse™ N3 core inside the DSU-120 cluster. An example of a debug target is a development system with a test chip or a silicon part with a Neoverse™ N3 core.

Debug unit

Helps debugging software that is running on the core:

- DSU-120 and external hardware based around the core
- Operating systems
- Application software

With the debug unit, you can:

- Stop program execution.
- Examine and alter process and coprocessor state.
- Examine and alter memory and the state of the input or output peripherals.
- Restart the *Processing Element* (PE).

For self-hosted debug, the debug target runs debug monitor software that runs on the core in the DSU-120 cluster. This way, it does not require expensive interface hardware to connect a second host computer.

17.2 Debug register interfaces

The Neoverse™ N3 core implements the Arm®v9.2-A Debug architecture. It also supports the Arm®v8.4-A Debug architecture and Arm®v8.3-A Debug over powerdown.

The Debug architecture defines a set of Debug registers. The Debug register interfaces provide access to these registers either from software running on the core or from an external debugger. See *Debug* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for more information.

Related information

[4.7 Debug over powerdown](#) on page 48

17.2.1 Core interfaces

System register access allows the Neoverse™ N3 core to access certain Debug registers directly. The Debug register interfaces provide access to these registers either from software running on the core or from an external debugger.

Access to the Debug registers is partitioned as follows:

Debug

This function is both system register based and memory-mapped. You can access the Debug register map using the *Advanced Peripheral Bus* (APB) slave port that connects into the DebugBlock of the DSU-120.

Performance monitoring

This function is system register based and memory-mapped. You can access the performance monitor registers using the APB slave port that connects into the DebugBlock of the DSU-120.

Trace

This function is system register based and memory-mapped. You can access the trace unit registers using the APB slave port that connects into the DebugBlock of the DSU-120.

Statistical profiling

This function is system register based.

ELA registers

You can access the *Embedded Logic Analyzer* (ELA) registers using the APB slave port that connects into the DebugBlock of the DSU-120.

The ELA-600 is licensed separately.



This function is memory-mapped and is not accessible using System registers.

For information on APB slave port interface, see the *Debug* chapter or the *Interfaces* section in the *Technical overview* chapter of the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).

17.2.2 Effects of resets on debug registers

The `complexporeset_n` and `complexreset_n` signals of the core affect the debug registers.

`complexporeset_n` maps to a Cold reset that covers reset of the core logic and the integrated debug functionality. This signal initializes the core logic, including the trace unit, breakpoint, watchpoint logic, performance monitor, and debug logic.

`complexreset_n` maps to a Warm reset that covers reset of the core logic. This signal resets some of the debug and performance monitor logic.

17.2.3 Breakpoints and watchpoints

The Neoverse™ N3 core supports six breakpoints, four watchpoints, and a standard *Debug Communications Channel* (DCC).

A breakpoint consists of a breakpoint control register and a breakpoint value register. These two registers are referred to as a *Breakpoint Register Pair* (BRP). Four of the breakpoints (BRP 0-3) match only to the *Virtual Address* (VA) and the other two (BRP 4 and 5) match against either the VA or context ID, or the *Virtual Machine ID* (VMID).

You can use watchpoints to stop your target when a specific memory address is accessed by your program. All the watchpoints can be linked to two breakpoints (BRP 4 and 5) to enable a memory request to be trapped in a given process context.

17.3 Debug events

A debug event can be either a software debug event or a Halting debug event.

The Neoverse™ N3 core responds to a debug event in one of the following ways:

- It ignores the debug event
- It takes a debug exception
- It enters debug state

In the Neoverse™ N3 core, watchpoint debug events are always synchronous. Memory hint instructions and cache clean operations, except `dc zva`, and `dc ivac`, do not generate watchpoint

debug events. Store exclusive instructions generate a watchpoint debug event even when the check for the control of exclusive monitor fails. Atomic `cas` instructions generate a watchpoint debug event even when the compare operation fails.

A Cold reset sets the Debug OS Lock. For the debug events and debug register accesses to operate normally, the Debug OS Lock must be cleared.

17.4 Debug memory map and debug signals

The debug memory map and debug signals are handled at the DSU-120 cluster level.

See *Debug* and *ROM tables* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#).

17.5 ROM table

The Neoverse™ N3 core includes a ROM table that contains a list of components in the system. Debuggers must use the ROM table to determine which CoreSight components are implemented.

The ROM table is a CoreSight debug related component that aids system debug along with CoreSight SoC and is for the Neoverse™ N3 core. The ROM tables comply with the [Arm® CoreSight™ Architecture Specification v3.0](#).

The DSU-120 has its own ROM tables, one for the cluster and one for the DebugBlock, and has entry points in the cluster ROM table for the ROM tables belonging to each core. See *ROM tables* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for more information.

The Neoverse™ N3 core ROM table includes the following entries:

Table 17-1: Core ROM table

Offset	Name	Description
0x0000	ROMENTRY0	Core debug
0x0004	ROMENTRY1	Core PMU
0x0008	ROMENTRY2	Core trace unit
0x000C	ROMENTRY3	Optional ELA

Related information

[B.8 External ROM table registers summary](#) on page 873

17.6 CoreSight component identification

Each component associated with the Neoverse™ N3 core has a unique set of CoreSight™ ID values. The following table shows these values.

Table 17-2: Neoverse™ N3 core CoreSight™ component identification

Component	Peripheral ID	Component ID	DevType	DevArch	Core revision
Debug	0x04100BBD8E	0xB105900D	0x15	0x47709A15	rOp1
PMU			0x16	0x47702A16	
Trace unit			0x13	0x47715A13	
ROM table			0x00	0x47700AF7	

17.7 CTI register identification values

The Neoverse™ N3 core *Cross Trigger Interface* (CTI) registers are located in the DebugBlock of the DSU-120.

For the cluster and core CTI register names and descriptions, see *External CTI registers* in the *Debug* chapter of the *Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual*. Only the core CTI register peripheral ID values will differ from the cluster CTI register peripheral ID values.

The core CTI register peripheral ID values are listed in the following table.

Table 17-3: Core CTI register peripheral ID values

Register	Bitfield position	Bitfield name	Value
CTIPIDR4	[7:4]	SIZE	0b0000
	[3:0]	DES_2	0b0100
CTIPIDR3	[7:4]	REVAND	0x0
	[3:0]	CMOD	0b0000
CTIPIDR2	[7:4]	REVISION	0x1
	[3]	JEDEC	0b1
	[2:0]	DES_1	0b011
CTIPIDR1	[7:4]	DES_0	0b1011
	[3:0]	PART_1	0b1101
CTIPIDR0	[7:0]	PART_0	0x8E

17.8 AArch64 Debug registers

The summary table provides an overview of all Debug registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 17-4: Debug registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
OSDTRRX_EL1	2	0	C0	C0	2	See individual bit resets.	64-bit	OS Lock Data Transfer Register, Receive
DBGBVR0_EL1	2	0	C0	C0	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR0_EL1	2	0	C0	C0	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR0_EL1	2	0	C0	C0	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR0_EL1	2	0	C0	C0	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
DBGBVR1_EL1	2	0	C0	C1	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR1_EL1	2	0	C0	C1	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR1_EL1	2	0	C0	C1	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR1_EL1	2	0	C0	C1	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
MDCCINT_EL1	2	0	C0	C2	0	See individual bit resets.	64-bit	Monitor DCC Interrupt Enable Register
MDSCR_EL1	2	0	C0	C2	2	See individual bit resets.	64-bit	Monitor Debug System Control Register
DBGBVR2_EL1	2	0	C0	C2	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR2_EL1	2	0	C0	C2	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR2_EL1	2	0	C0	C2	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR2_EL1	2	0	C0	C2	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
OSDTRTX_EL1	2	0	C0	C3	2	See individual bit resets.	64-bit	OS Lock Data Transfer Register, Transmit
DBGBVR3_EL1	2	0	C0	C3	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR3_EL1	2	0	C0	C3	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR3_EL1	2	0	C0	C3	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR3_EL1	2	0	C0	C3	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
DBGBVR4_EL1	2	0	C0	C4	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR4_EL1	2	0	C0	C4	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGBVR5_EL1	2	0	C0	C5	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR5_EL1	2	0	C0	C5	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
OSECCR_EL1	2	0	C0	C6	2	See individual bit resets.	64-bit	OS Lock Exception Catch Control Register
MDRAR_EL1	2	0	C1	C0	0	See individual bit resets.	64-bit	Monitor Debug ROM Address Register
OSLAR_EL1	2	0	C1	C0	4	See individual bit resets.	64-bit	OS Lock Access Register
OSLSR_EL1	2	0	C1	C1	4	See individual bit resets.	64-bit	OS Lock Status Register
OSDLR_EL1	2	0	C1	C3	4	See individual bit resets.	64-bit	OS Double Lock Register
DBGPRCR_EL1	2	0	C1	C4	4	See individual bit resets.	64-bit	Debug Power Control Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
DBGCLAIMSET_EL1	2	0	C7	C8	6	See individual bit resets.	64-bit	Debug CLAIM Tag Set register
DBGCLAIMCLR_EL1	2	0	C7	C9	6	See individual bit resets.	64-bit	Debug CLAIM Tag Clear register
DBGAUTHSTATUS_EL1	2	0	C7	C14	6	See individual bit resets.	64-bit	Debug Authentication Status register
MDCCSR_ELO	2	3	C0	C1	0	See individual bit resets.	64-bit	Monitor DCC Status Register
DBGDTR_ELO	2	3	C0	C4	0	See individual bit resets.	64-bit	Debug Data Transfer Register, half-duplex
DBGDTRRX_ELO	2	3	C0	C5	0	See individual bit resets.	64-bit	Debug Data Transfer Register, Receive
DBGDTRTX_ELO	2	3	C0	C5	0	See individual bit resets.	64-bit	Debug Data Transfer Register, Transmit
TRFCR_EL1	3	0	C1	C2	1	See individual bit resets.	64-bit	Trace Filter Control Register (EL1)
MDCR_EL2	3	4	C1	C1	1	See individual bit resets.	64-bit	Monitor Debug Configuration Register (EL2)
TRFCR_EL2	3	4	C1	C2	1	See individual bit resets.	64-bit	Trace Filter Control Register (EL2)

17.9 External Debug registers

The summary table provides an overview of all memory-mapped Debug registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 17-5: Debug registers summary

Offset	Name	Reset	Width	Description
0x020	EDESR	See individual bit resets.	32-bit	External Debug Event Status Register
0x024	EDECR	See individual bit resets.	32-bit	External Debug Execution Control Register
0x030	EDWAR [31:0]	See individual bit resets.	32-bit	External Debug Watchpoint Address Register
0x034	EDWAR [63:32]	See individual bit resets.	32-bit	External Debug Watchpoint Address Register
0x080	DBGDTRRX_ELO	See individual bit resets.	32-bit	Debug Data Transfer Register, Receive
0x084	EDITR	See individual bit resets.	32-bit	External Debug Instruction Transfer Register
0x088	EDSCR	See individual bit resets.	32-bit	External Debug Status and Control Register
0x08C	DBGDTRTX_ELO	See individual bit resets.	32-bit	Debug Data Transfer Register, Transmit
0x090	EDRCR	See individual bit resets.	32-bit	External Debug Reserve Control Register
0x094	EDACR	See individual bit resets.	32-bit	External Debug Auxiliary Control Register
0x098	EDECCR	See individual bit resets.	32-bit	External Debug Exception Catch Control Register
0x300	OSLAR_EL1	See individual bit resets.	32-bit	OS Lock Access Register
0x310	EDPRCR	See individual bit resets.	32-bit	External Debug Power/Reset Control Register
0x314	EDPRSR	See individual bit resets.	32-bit	External Debug Processor Status Register
0x400	DBGBVR0_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x408	DBGBCRO_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x410	DBGBVR1_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers

Offset	Name	Reset	Width	Description
0x418	DBGBCR1_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x420	DBGBVR2_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x428	DBGBCR2_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x430	DBGBVR3_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x438	DBGBCR3_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x440	DBGBVR4_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x448	DBGBCR4_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x450	DBGBVR5_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x458	DBGBCR5_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x800	DBGWVR0_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x808	DBGWCR0_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0x810	DBGWVR1_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x818	DBGWCR1_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0x820	DBGWVR2_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x828	DBGWCR2_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0x830	DBGWVR3_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x838	DBGWCR3_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0xD00	MIDR_EL1	See individual bit resets.	32-bit	Main ID Register
0xD20	EDPFR [31:0]	See individual bit resets.	32-bit	External Debug Processor Feature Register
0xD24	EDPFR [63:32]	See individual bit resets.	32-bit	External Debug Processor Feature Register
0xD28	EDDFR [31:0]	See individual bit resets.	32-bit	External Debug Feature Register
0xD2C	EDDFR [63:32]	See individual bit resets.	32-bit	External Debug Feature Register
0xD60	EDAA32PFR	See individual bit resets.	64-bit	External Debug Auxiliary Processor Feature Register
0xFA0	DBGCLAIMSET_EL1	See individual bit resets.	32-bit	Debug CLAIM Tag Set register
0xFA4	DBGCLAIMCLR_EL1	See individual bit resets.	32-bit	Debug CLAIM Tag Clear register
0xFA8	EDDEVAFF0	See individual bit resets.	32-bit	External Debug Device Affinity register 0
0xFAC	EDDEVAFF1	See individual bit resets.	32-bit	External Debug Device Affinity register 1
0xFB0	EDLAR	See individual bit resets.	32-bit	External Debug Lock Access Register
0xFB4	EDLSR	See individual bit resets.	32-bit	External Debug Lock Status Register
0xFB8	DBGAUTHSTATUS_EL1	See individual bit resets.	32-bit	Debug Authentication Status register
0xFBC	EDDEVARCH	See individual bit resets.	32-bit	External Debug Device Architecture register
0xFC0	EDDEVID2	See individual bit resets.	32-bit	External Debug Device ID register 2
0xFC4	EDDEVID1	See individual bit resets.	32-bit	External Debug Device ID register 1
0xFC8	EDDEVID	See individual bit resets.	32-bit	External Debug Device ID register 0
0xFCC	EDDEVTYPE	See individual bit resets.	32-bit	External Debug Device Type register
0xFD0	EDPIDR4	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 4
0xFE0	EDPIDR0	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 0
0xFE4	EDPIDR1	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 1
0xFE8	EDPIDR2	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 2
0xFEC	EDPIDR3	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 3

Offset	Name	Reset	Width	Description
0xFF0	EDCIDR0	See individual bit resets.	32-bit	External Debug Component Identification Register 0
0xFF4	EDCIDR1	See individual bit resets.	32-bit	External Debug Component Identification Register 1
0xFF8	EDCIDR2	See individual bit resets.	32-bit	External Debug Component Identification Register 2
0xFFC	EDCIDR3	See individual bit resets.	32-bit	External Debug Component Identification Register 3

17.10 External ROM table registers

The summary table provides an overview of all memory-mapped ROM table registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 17-6: ROM table registers summary

Offset	Name	Reset	Width	Description
0x0	ROMENTRY0	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0x4	ROMENTRY1	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0x8	ROMENTRY2	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0xC	ROMENTRY3	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0xFB8	AUTHSTATUS	See individual bit resets.	32-bit	Authentication Status Register
0xFBC	DEVARCH	See individual bit resets.	32-bit	Device Architecture Register
0xFD0	PIDR4	See individual bit resets.	32-bit	Peripheral Identification Register 4
0xFE0	PIDR0	See individual bit resets.	32-bit	Peripheral Identification Register 0
0xFE4	PIDR1	See individual bit resets.	32-bit	Peripheral Identification Register 1
0xFE8	PIDR2	See individual bit resets.	32-bit	Peripheral Identification Register 2
0xFEC	PIDR3	See individual bit resets.	32-bit	Peripheral Identification Register 3
0xFF0	CIDR0	See individual bit resets.	32-bit	Component Identification Register 0
0xFF4	CIDR1	See individual bit resets.	32-bit	Component Identification Register 1
0xFF8	CIDR2	See individual bit resets.	32-bit	Component Identification Register 2
0xFFC	CIDR3	See individual bit resets.	32-bit	Component Identification Register 3

18. Performance Monitors Extension support

The Neoverse™ N3 core implements the Performance Monitors Extension, including Arm®v8.4-A, Arm®v8.5-A and Arm®v8.7-A performance monitoring features.

The Neoverse™ N3 core *Performance Monitoring Unit* (PMU):

- Collects events through an event interface from other units in the design. These events are used as triggers for event counters.
- Supports cycle counters through the Performance Monitors Control Register.
- Implements PMU snapshots for context samples.
- Provides six or 20 PMU 64-bit counters that count any of the events available in the core. The absolute counts that are recorded might vary because of pipeline effects. This variation has negligible effect except in cases where the counters are enabled for a very short time.

You can program the PMU using either the System registers or the external Debug APB interface.

18.1 Performance monitors events

The Neoverse™ N3 core *Performance Monitoring Unit* (PMU) collects events from other units in the design and uses numbers to reference these events.

Common event PMU events

The following table shows the Neoverse™ N3 core performance monitors events that are generated and the numbers that the PMU uses to reference the events. The table also shows the bit position of each event on the event bus. Event numbers that are not listed are reserved.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about these PMU events.



Note

Unless otherwise indicated, each of these events can be exported to the trace unit and selected in accordance with the *Arm® Embedded Trace Extension*.

Table 18-1: Common event PMU events

Event number	Mnemonic	Description
0x0000	SW_INCR	Instruction architecturally executed, Condition code check pass, software increment This event counts any instruction architecturally executed (condition code check pass).

Event number	Mnemonic	Description
0x0001	L1I_CACHE_REFILL	Level 1 instruction cache refill This event counts any instruction fetch which misses in the cache. The following instructions are not counted: - Cache maintenance instructions - Non-cacheable accesses
0x0002	L1I_TLB_REFILL	Level 1 instruction TLB refill This event counts any refill of the L1 instruction TLB from the MMU Translation Cache (MMUTC). This includes refills that result in a translation fault. The following instructions are not counted: TLB maintenance instructions This event counts regardless of whether the MMU is enabled.
0x0003	L1D_CACHE_REFILL	Level 1 data cache refill This event counts any load or store operation or translation table walk that causes data to be read from outside the L1 cache, including accesses which do not allocate into L1. The following instructions are not counted: - Cache maintenance instructions and prefetches. - Stores of an entire cache line, even if they make a coherency request outside the L1. - Partial cache line writes which do not allocate into the L1 cache. - Non-cacheable accesses. This event counts the sum of L1D_CACHE_REFILL_RD and L1D_CACHE_REFILL_WR.
0x0004	L1D_CACHE	Level 1 data cache access This event counts any load or store operation or translation table walk that looks up in the L1 data cache. In particular, any access that could count the L1D_CACHE_REFILL event causes this event to count. The following instructions are not counted: - Cache maintenance instructions and prefetches. - Non-cacheable accesses. This event counts the sum of L1D_CACHE_RD and L1D_CACHE_WR.
0x0005	L1D_TLB_REFILL	Level 1 data TLB refill This event counts any refill of the data L1 TLB from the L2 TLB. This includes refills which result in a translation fault. The following instructions are not counted: TLB maintenance instructions. This event counts regardless of whether the MMU is enabled.

Event number	Mnemonic	Description
0x0008	INST_RETIRED	Instruction architecturally executed This event counts all retired instructions, including ones that fail their condition check.
0x0009	EXC_TAKEN	Exception taken The counter counts each exception taken.
0x000A	EXC_RETURN	Instruction architecturally executed, Condition code check pass, exception return
0x000B	CID_WRITE_RETIRED	Instruction architecturally executed, Condition code check pass, write to CONTEXTIDR This event only counts writes using the CONTEXTIDR_EL1 mnemonic. Writes to CONTEXTIDR_EL12 and CONTEXTIDR_EL2 are not counted.
0x000C	PC_WRITE_RETIRED	Instruction architecturally executed, Condition code check pass, software change of the PC This event counts all branches taken and popped from the branch monitor. This excludes some exception entries (HVC/SVC/SMC/ISB and exception return) and BRKPT but excludes debug entries, and CCFAIL branches.
0x000D	BR_IMMED_RETIRED	Branch Instruction architecturally executed, immediate This event counts all branches decoded as immediate branches, taken or not, and popped from the branch monitor. This excludes exception entries, debug entries, and CCFAIL branches.
0x000E	BR_RETURN_RETIRED	Branch instruction architecturally executed, procedure return, taken Instruction architecturally executed, Condition code check pass, procedure return
0x0010	BR_MIS_PRED	Branch instruction speculatively executed, mispredicted, or not predicted This event counts any predictable branch instruction which is mispredicted either due to dynamic misprediction or because the MMU is off and the branches are statically predicted not taken.
0x0011	CPU_CYCLES	Cycle
0x0012	BR_PRED	Predictable branch instruction speculatively executed This event counts all predictable branches.
0x0013	MEM_ACCESS	Data memory access This event counts memory accesses due to load or store instructions. The following instructions are not counted: • Instruction fetches. • Cache maintenance instructions. • Translation table walks or prefetches. This event counts the sum of MEM_ACCESS_RD and MEM_ACCESS_WR.

Event number	Mnemonic	Description
0x0014	L1I_CACHE	Level 1 instruction cache access This event counts any instruction fetch which accesses the L1 instruction cache. The following instructions are not counted: - Cache maintenance instructions. - Non-cacheable accesses.
0x0015	L1D_CACHE_WB	Level 1 data cache write-back This event counts any write-back of data from the L1 data cache to lower level of caches. The event counts both victim line evictions and snoops. The following instructions are not counted: - Evictions from L1 caused by cache maintenance operations. - Invalidations which do not result in data being transferred out of the L1. - Full-line writes which write to L2 without writing L1, such as write-streaming mode.
0x0016	L2D_CACHE	Level 2 data cache access - If the core is configured with a per-core L2 cache, this event counts any transaction from L1 which looks up in the L2 cache, and any writeback from the L1 to the L2. Snoops from outside the core and cache maintenance operations are not counted. - If the core is not configured with a per-core L2 cache, this event counts the cluster cache event, as defined by L3D_CACHE. - If neither a per-core cache nor a cluster cache are configured, then this event is not implemented.
0x0017	L2D_CACHE_REFILL	Level 2 data cache refill - If the core is configured with a per-core L2 cache, this event counts any Cacheable transaction from L1 which causes data to be read from outside the core. L2 refills caused by stashes into L2 are not meant to be counted. - If the core is not configured with a per-core L2 cache, this event counts the cluster cache event, as defined by L3D_CACHE_REFILL. - If neither a per-core cache nor a cluster cache are configured, then this event is not implemented.
0x0018	L2D_CACHE_WB	Level 2 data cache write-back If the core is configured with a per-core L2 cache, this event counts any write-back of data from the L2 cache to a location outside the core. The event includes snoops to the L2 which return data, regardless of whether they cause an invalidation. Invalidation from the L2 which do not write data outside of the core and snoops which return data from the L1 are not counted. If the core is not configured with a per-core L2 cache, this event is not implemented.
0x0019	BUS_ACCESS	Bus access This event counts for every beat of data that is transferred over the data channels between the core and the SCU. If both read and write data beats are transferred on a given cycle, this event is counted twice on that cycle. This event counts the sum of BUS_ACCESS_RD and BUS_ACCESS_WR.

Event number	Mnemonic	Description
0x001B	INST_SPEC	Operation speculatively executed Counts operations that have been speculatively executed.
0x001C	TTBR_WRITE_RETIRED	Instruction architecturally executed, condition code check pass, write to TTBR This event only counts writes to TTBR0/TTBR1 in AArch32 and TTBR0_EL1/TTBR1_EL1 in AArch64. The following instructions are not counted: • Accesses to TTBR0_EL12/TTBR1_EL12 or TTBR0_EL2/TTBR1_EL2.
0x001D	BUS_CYCLES	Bus cycle This event duplicates CPU_CYCLES.
0x001E	CHAIN	Chain a pair of event counters For odd-numbered counters, this event increments the count by one for each overflow of the preceding even-numbered counter. For even numbered counters, there is no increment.
0x0020	L2D_CACHE_ALLOCATE	Level 2 data cache allocation without refill This event counts any full cache line write into the L2 cache which does not cause a linefill, including write-backs from L1 to L2 and full-line writes which do not allocate into L1.
0x0021	BR_RETIRED	Branch instruction architecturally executed This event counts all branches, taken or not, popped from the branch monitor. This excludes exception entries, debug entries, and CCFail branches in the ** core, an ISB is a branch and even microarchitectural ISBs are counted.
0x0022	BR_MIS_PRED_RETIRED	Branch instruction architecturally executed, mispredicted This event counts any branch that is counted by BR_RETIRED which is not correctly predicted and causes pipeline clears
0x0023	STALL_FRONTEND	No operation has been sent for execution, due to the frontend No operation has been issued, because of the frontend The counter counts on any cycle when no operations are issued due to the instruction queue being empty.
0x0024	STALL_BACKEND	No operation has been sent for execution due to the backend. No operation has been issued, because of the backend. The counter counts on any cycle when no operations are issued due to a pipeline stall.
0x0025	L1D_TLB	Level 1 data TLB access This event counts any load or store operation which accesses the data L1 TLB. If both a load and a store are executed on a cycle, this event counts twice. This event counts regardless of whether the MMU is enabled.
0x0026	L1I_TLB	Level 1 instruction TLB access This event counts any instruction fetch which accesses the instruction L1 TLB. This event counts regardless of whether the MMU is enabled.

Event number	Mnemonic	Description
0x0027	L2I_CACHE	Level 2 instruction cache access The counter counts each instruction memory access to at least the L2 instruction or unified cache. Each instruction memory access to other L2 instruction or unified memory structures, such as refill buffers, is also counted.
0x0028	L2I_CACHE_REFILL	Level 2 instruction cache refill The counter counts each access counted by L2I_CACHE that causes a refill of the L2 instruction or unified cache, or any L1 data, instruction, or unified cache of this PE, from outside of those caches.
0x0029	L3D_CACHE_ALLOCATE	Level 3 data cache allocation without refill This event counts any full cache line write into the L3 cache which does not cause a linefill, including write-backs from L2 to L3 cache and full-line writes which do not allocate into L2. If IMP_CPUCFR_EL1.no_scu is set to 1, this event does not count.
0x002A	L3D_CACHE_REFILL	Level 3 data cache refill This event counts for any cacheable read transaction returning data from the SCU for which the data source was outside the cluster. If IMP_CPUCFR_EL1.no_scu is set to 1, this event does not count.
0x002B	L3D_CACHE	Level 3 data cache access This event counts for any cacheable read, write or write-back transaction sent to the SCU. If IMP_CPUCFR_EL1.no_scu is set to 1, this event does not count.
0x002D	L2D_TLB_REFILL	Level 2 data TLB refill This event counts on any refill of the L2 TLB, caused by either an instruction or data access. This event does not count if the MMU is disabled.
0x002F	L2D_TLB	Level 2 data TLB access Attributable level 2 unified TLB access. This event counts on any access to the L2 TLB (caused by a refill of any of the L1 TLBs). This event does not count if the MMU is disabled.
0x0031	REMOTE_ACCESS	Access to another socket in a multi-socket system This event counts any transactions returning data from another socket in a multi-socket system.
0x0032	LL_CACHE	Last level cache access The counter counts each memory-read operation or memory-write operation that causes a cache access to at least the last level cache.

Event number	Mnemonic	Description
0x0033	LL_CACHE_MISS	Last level cache miss The counter counts each access counted by LL_CACHE that is not completed by the last level cache.
0x0034	DTLB_WALK	Data TLB access with at least one translation table walk Access to data TLB that caused a translation table walk. This event counts on any data access which causes L2D_TLB_REFILL to count.
0x0035	ITLB_WALK	Instruction TLB access with at least one translation table walk Access to instruction TLB that caused a translation table walk. This event counts on any instruction access which causes L2D_TLB_REFILL to count.
0x0036	LL_CACHE_RD	Last level cache access, read This event counts any cacheable read transaction which returns a data source of 'interconnect cache', 'DRAM', 'remote' or 'inter-cluster peer'.
0x0037	LL_CACHE_MISS_RD	Last Level cache miss read This event counts any cacheable read transaction which returns a data source of 'DRAM', 'remote' or 'inter-cluster peer'.
0x0039	L1D_CACHE_LMISS_RD	Level 1 data cache long-latency read miss Level 1 data cache access, read. This event counts any load operation or translation table walk access which looks up in the L1 data cache. In particular, any access which could count the L1D_CACHE_REFILL_RD event causes this event to count. The following instructions are not counted: - Cache maintenance instructions and prefetches. - Non-cacheable accesses.
0x003A	OP_RETIRED	This event counts each operation counted by OP_SPEC that would be executed in a Simple sequential execution of the program.
0x003B	OP_SPEC	Micro-operation speculatively executed This event counts the number of operations executed by the core, including those that are executed speculatively and would not be executed in a simple sequential execution of the program.
0x003C	STALL	No operation sent for execution This event counts every Attributable cycle on which no Attributable instruction or operation was sent for execution on this core.

Event number	Mnemonic	Description
0x003D	STALL_SLOT_BACKEND	No operation sent for execution on a Slot due to the backend Counts each Slot counted by STALL_SLOT where no Attributable instruction or operation was for execution because the backend is unable to accept one of: - The instruction operation available for the PE on the slot. - Any operation on the slot.
0x003E	STALL_SLOT_FRONTEND	No operation sent for execution due to the frontend. Counts each slot counted by STALL_SLOT where no Attributable instruction or operation was sent for execution because there was no Attributable instruction or operation available to issue from the PE from the frontend for the Slot.
0x003F	STALL_SLOT	No operation sent for execution on a Slot The counter counts on each Attributable cycle the number of instruction or operation Slots that were not occupied by an instruction or operation Attributable to the PE.
0x0040	L1D_CACHE_RD	Level 1 data cache access, read Counts any load operation or translation table walk access which looks up in the L1 data cache. In particular, any access which could count the L1D_CACHE_REFILL_RD event causes this event to count. Cache maintenance instructions and prefetches are not counted. Non-cacheable accesses are not counted.
0x0041	L1D_CACHE_WR	Level 1 data cache access, write Counts any store operation which looks up in the L1 data cache. In particular, any access which could count the L1D_CACHE_REFILL event causes this event to count. Cache maintenance instructions and prefetches are not counted. Non-cacheable accesses are not counted.
0x0044	L1D_CACHE_REFILL_INNER	Level 1 data cache refill, inner This event counts any L1 data cache linefill (as counted by L1D_CACHE_REFILL) which hits in lower level of caches, or another core in the cluster.
0x0045	L1D_CACHE_REFILL_OUTER	Level 1 data cache refill, outer This event counts any L1 data cache linefill (as counted by L1D_CACHE_REFILL) which does not hit in lower level of caches, or another core in the cluster, and instead obtains data from outside the cluster.
0x0048	L1D_CACHE_INVAL	Level 1 data cache invalidate The counter counts each invalidation of a cache line in the Level 1 data or unified cache. The counter does not count events if a cache refill of the Level 1 data or unified cache invalidates a line in the Level 1 data or unified cache.
0x0050	L2D_CACHE_RD	Level 2 data cache access, read This event counts any transaction issued from L1 caches which looks up in the L2 cache, including requests for instructions fetches and MMU table walks. The transaction is counted regardless of the source that generated it in the L1, being a load, store or prefetch request.

Event number	Mnemonic	Description
0x0051	L2D_CACHE_WR	Level 2 data cache access, write This event counts any full cache line write into the L2 cache which does not cause a linefill, including write-backs from L1 to L2, full-line writes which do not allocate into L1 and MMU descriptor hardware updates performed in L2.
0x0052	L2D_CACHE_REFILL_RD	Level 2 data cache refill, read This event counts any Cacheable transaction generated by a read operation which causes data to be read from outside the L2.
0x0053	L2D_CACHE_REFILL_WR	Level 2 data cache refill, write This event counts any Cacheable transaction generated by a store operation which causes data to be read from outside the L2.
0x0056	L2D_CACHE_WB_VICTIM	Level 2 data cache write-back, victim This event counts any datafull write-back operation caused by allocations.
0x0057	L2D_CACHE_WB_CLEAN	Level 2 data cache write-back, cleaning, and coherency This event counts any datafull write-back operation caused by cache maintenance operations or external coherency requests.
0x0058	L2D_CACHE_INVALID	Level 2 data cache invalidate This event counts any cache maintenance operation which causes the invalidation of a line present in the L2 cache.
0x0060	BUS_ACCESS_RD	Bus access, read This event counts for every beat of data that is transferred over the read data channel between the core and the SCU.
0x0061	BUS_ACCESS_WR	Bus access, write This event counts for every beat of data that is transferred over the write data channel between the core and the SCU.
0x0066	MEM_ACCESS_RD	Data memory access, read This event counts memory accesses due to load instructions. The following instructions are not counted: • Instruction fetches • Cache maintenance instructions • Translation table walks • Prefetches.

Event number	Mnemonic	Description
0x0067	MEM_ACCESS_WR	Data memory access, write This event counts memory accesses due to store instructions. The following instructions are not counted: • Instruction fetches. • Cache maintenance instructions. • Translation table walks. • Prefetches.
0x006E	STREX_FAIL_SPEC	Exclusive operation speculatively executed, Store-Exclusive fail. Exclusive operation speculatively executed, STREX or STX fail.
0x006F	STREX_SPEC	Exclusive operation speculatively executed, Store-Exclusive. Exclusive operation speculatively executed, STREX or STX.
0x0070	LD_SPEC	Operation speculatively executed, load
0x0071	ST_SPEC	Operation speculatively executed, store
0x0073	DP_SPEC	Operation speculatively executed, integer data processing This event counts retired integer data-processing instructions.
0x0074	ASE_SPEC	Operation speculatively executed, Advanced SIMD This event counts retired Advanced SIMD instructions.
0x0075	VFP_SPEC	Operation speculatively executed, floating-point This event counts retired floating-point instructions.
0x0076	PC_WRITE_SPEC	Operation speculatively executed, software change of the PC This event counts at Decoder step each instruction changing the PC: all branches, some exceptions (HVC/SVC/SMC/ISB and exception return).
0x0077	CRYPTO_SPEC	Operation speculatively executed, Cryptographic instruction This event counts retired Cryptographic instructions.
0x007C	ISB_SPEC	Barrier speculatively executed, ISB
0x007D	DSB_SPEC	Barrier speculatively executed, DSB
0x007E	DMB_SPEC	Barrier speculatively executed, DMB
0x0081	EXC_UNDEF	Exception taken, other synchronous Counts the number of undefined exceptions taken locally
0x0082	EXC_SVC	Exception taken, Supervisor Call Exception taken locally, Supervisor Call
0x0083	EXC_PABORT	Exception taken, Instruction Abort Exception taken locally, Instruction Abort

Event number	Mnemonic	Description
0x0084	EXC_DABORT	Exception taken, Data Abort or SError Exception taken locally, Data Abort and SError
0x0086	EXC_IRQ	Exception taken, IRQ Exception taken locally, IRQ
0x0087	EXC_FIQ	Exception taken, FIQ Exception taken locally, FIQ
0x0088	EXC_SMC	Exception taken, Secure Monitor Call Exception taken locally, Secure Monitor Call
0x008A	EXC_HVC	Exception taken, Hypervisor Call Exception taken locally, Hypervisor Call
0x008B	EXC_TRAP_PABORT	Exception taken, Instruction Abort not Taken locally
0x008C	EXC_TRAP_DABORT	Exception taken, Data Abort or SError not Taken locally
0x008D	EXC_TRAP_OTHER	Exception taken, other traps not Taken locally
0x008E	EXC_TRAP_IRQ	Exception taken, IRQ not Taken locally
0x008F	EXC_TRAP_FIQ	Exception taken, FIQ not Taken locally
0x0090	RC_LD_SPEC	Release consistency operation speculatively executed, Load-Acquire
0x0091	RC_ST_SPEC	Release consistency operation speculatively executed, Store-Release
0x00A0	L3D_CACHE_RD	Level 3 data cache access, read This event counts for any cacheable read transaction sent to the SCU. If IMP_CPUCFR_EL1.no_scu is set to 1, this event does not count.
0x4000	SAMPLE_POP	Statistical Profiling sample population The counter increments for each operation that might be sampled, whether or not the operation was sampled. Operations that are executed at an Exception level or Security state in which the Statistical Profiling Extension is disabled are not counted.
0x4001	SAMPLE_FEED	Statistical Profiling sample taken The counter increments each time the sample interval counter reaches zero and is reloaded, and the sample does not collide with the previous sample. Samples that are removed by filtering, or discarded, and not written to the Profiling Buffer are counted.
0x4002	SAMPLE_FILTRATE	Statistical Profiling sample taken and not removed by filtering The counter increments each time that a completed sample record is checked against the filters and not removed. Sample records that are not removed by filtering, but are discarded before being written to the Profiling Buffer because of a Profiling Buffer management event, are counted.
0x4003	SAMPLE_COLLISION	Statistical Profiling sample collided with previous sample The counter increments for each sample record that is taken when the previous sampled operation has not completed generating its sample record.

Event number	Mnemonic	Description
0x4004	CNT_CYCLES	Constant frequency cycles The counter increments at a constant frequency equal to the rate of increment of the System Counter, CNTPCT_EL0.
0x4005	STALL_BACKEND_MEM	Memory stall cycles The counter counts each cycle counted by STALL_BACKEND_MEMBOUND where there is a demand data miss in the last level of data or unified cache within the PE clock domain or a non-cacheable data access in progress.
0x4006	L1I_CACHE_LMISS	Level 1 instruction cache long-latency miss The counter counts each access counted by L1I_CACHE that incurs additional latency because it returns instructions from outside the L1 instruction cache.
0x4009	L2D_CACHE_LMISS_RD	Level 2 data cache long-latency read miss The counter counts each memory read access counted by L2D_CACHE that incurs additional latency because it returns data from outside the L2 data or unified cache of this PE.
0x400A	L2I_CACHE_LMISS	Level 2 instruction cache long-latency miss The counter counts each memory read access counted by L2I_CACHE that incurs additional latency because it returns data from outside the L2 instruction or unified cache of this PE.
0x400B	L3D_CACHE_LMISS_RD	Level 3 data cache long-latency read miss The counter counts each memory read access counted by L3D_CACHE that incurs additional latency because it returns data from outside the L3 data or unified cache of this PE. If IMP_CPUCFR_EL1.no_scu is set to 1, this event does not count.
0x400C	TRB_WRAP	Trace buffer current write pointer wrapped
0x400D	PMU_OVFS	PMU overflow, counters accessible to EL1 and EL0 Note: This event is exported to the trace unit, but cannot be counted in the PMU.
0x400E	TRB_TRIG	Trace buffer Trigger Event Note: This event is only exported to the trace unit and is not visible to the PMU.
0x400F	PMU_HOVFS	PMU overflow, counters reserved for use by EL2 Note: This event is only exported to the trace unit and is not visible to the PMU.

Event number	Mnemonic	Description
0x4010	TRCEXTOUT0	Trace unit external output 0 PE Trace Unit external output 0 Note: This event is not exported to the trace unit.
0x4011	TRCEXTOUT1	Trace unit external output 1 PE Trace Unit external output 1 Note: This event is not exported to the trace unit.
0x4012	TRCEXTOUT2	Trace unit external output 2 PE Trace Unit external output 2 Note: This event is not exported to the trace unit.
0x4013	TRCEXTOUT3	Trace unit external output 3 PE Trace Unit external output 3 Note: This event is not exported to the trace unit.
0x4018	CTI_TRIGOUT4	Cross Trigger Interface output trigger 4
0x4019	CTI_TRIGOUT5	Cross Trigger Interface output trigger 5
0x401A	CTI_TRIGOUT6	Cross Trigger Interface output trigger 6
0x401B	CTI_TRIGOUT7	Cross Trigger Interface output trigger 7
0x4020	LDST_ALIGN_LAT	Access with additional latency from alignment The counter counts each access counted by MEM_ACCESS that, due to the alignment of the address and size of data being accessed, incurred additional latency.
0x4021	LD_ALIGN_LAT	Load with additional latency from alignment The counter counts each memory-read access counted by LDST_ALIGN_LAT.
0x4022	ST_ALIGN_LAT	Store with additional latency from alignment The counter counts each memory-write access counted by LDST_ALIGN_LAT
0x4024	MEM_ACCESS_CHECKED	Checked data memory access
0x4025	MEM_ACCESS_CHECKED_RD	Checked data memory access, read
0x4026	MEM_ACCESS_CHECKED_WR	Checked data memory access, write
0x8005	ASE_INST_SPEC	Advanced SIMD operations speculatively executed

Event number	Mnemonic	Description
0x8006	SVE_INST_SPEC	SVE operation, including load/store The counter counts speculatively executed operations due to SVE instructions. It is IMPLEMENTATION DEFINED whether this event counts operations due to non-SIMD SVE instructions.
0x8014	FP_HP_SPEC	Half-precision floating-point operation speculatively executed
0x8018	FP_SP_SPEC	Single-precision floating-point operation speculatively executed
0x801C	FP_DP_SPEC	Double-precision floating-point operation speculatively executed
0x8074	SVE_PRED_SPEC	SVE predicated operations speculatively executed
0x8075	SVE_PRED_EMPTY_SPEC	SVE predicated operations with no active predicates speculatively executed
0x8076	SVE_PRED_FULL_SPEC	SVE predicated operations with all active predicates speculatively executed
0x8077	SVE_PRED_PARTIAL_SPEC	SVE predicated operations with partially active predicates speculatively executed
0x8079	SVE_PRED_NOT_FULL_SPEC	SVE predicated operations with no or partially active predicates speculatively executed
0x80BC	SVE_LDFF_SPEC	SVE First-fault load operations speculatively executed
0x80BD	SVE_LDFF_FAULT_SPEC	SVE First-fault load operations speculatively executed which set FFR bit to 0
0x80C0	FP_SCALE_OPS_SPEC	Scalable floating-point element operations speculatively executed
0x80C1	FP_FIXED_OPS_SPEC	Non-scalable floating-point element operations speculatively executed
0x80E3	ASE_SVE_INT8_SPEC	Advanced SIMD and SVE 8-bit integer operation speculatively executed
0x80E7	ASE_SVE_INT16_SPEC	Advanced SIMD and SVE 16-bit integer operation speculatively executed
0x80EB	ASE_SVE_INT32_SPEC	Advanced SIMD and SVE 32-bit integer operation speculatively executed
0x80EF	ASE_SVE_INT64_SPEC	Advanced SIMD and SVE 64-bit integer operation speculatively executed
0x8108	BR_IMMED_TAKEN_RETIRED	Instruction architecturally executed, immediate branch taken
0x810C	BR_INDNR_TAKEN_RETIRED	Instruction architecturally executed, indirect branch excluding procedure return retired
0x8110	BR_IMMED_PRED_RETIRED	Branch instruction architecturally executed, predicted immediate The counter counts the instructions on the architecturally executed path counted by both BR_IMMED_RETIRED and BR_PRED_RETIRED. These are all immediate branch instructions where the branch was correctly predicted"
0x8111	BR_IMMED_MIS_PRED_RETIRED	Branch instruction architecturally executed, mispredicted immediate The counter counts the instructions on the architecturally executed path, counted by both BR_IMMED_RETIRED and BR_MIS_PRED_RETIRED. These are all immediate branch instructions where the branch was mispredicted.
0x8112	BR_IND_PRED_RETIRED	Branch instruction architecturally executed, predicted indirect The counter counts the instructions on the architecturally executed path counted by both BR_IND_RETIRED and BR_PRED_RETIRED. These are branch instructions where the branch was correctly predicted, but does not include immediate instructions.
0x8113	BR_IND_MIS_PRED_RETIRED	Branch instruction architecturally executed, mispredicted indirect The counter counts the instructions on the architecturally executed path counted by both BR_IND_RETIRED and BR_MIS_PRED_RETIRED. These are branch instructions where the branch was mispredicted, but does not include immediate instructions.

Event number	Mnemonic	Description
0x8114	BR_RETURN_PRED_RETIRE	Branch instruction architecturally executed, predicted procedure return The counter counts the instructions on the architecturally executed path counted by BR_IND_PRED_RETIRE where, if taken, the branch would be counted by BR_RETURN_RETIRE. These are branch return instructions, where the branch was correctly predicted.
0x8115	BR_RETURN_MIS_PRED_RETIRE	Branch instruction architecturally executed, mispredicted procedure return The counter counts the instructions on the architecturally executed path counted by BR_IND_MIS_PRED_RETIRE where, if taken, the branch would also be counted by BR_RETURN_RETIRE. These are branch return instructions where the branch was mispredicted.
0x8116	BR_INDNR_PRED_RETIRE	Branch instruction architecturally executed, predicted indirect excluding procedure return The counter counts the instructions on the architecturally executed path counted by BR_IND_PRED_RETIRE where, if taken, the branch would not be counted by BR_RETURN_RETIRE. These are branch instructions where the branch was correctly predicted, but does not include immediate or return instructions
0x8117	BR_INDNR_MIS_PRED_RETIRE	Branch instruction architecturally executed, mispredicted indirect excluding procedure return The counter counts the instructions on the architecturally executed path counted by BR_IND_MIS_PRED_RETIRE where, if taken, the branch would not be counted by BR_RETURN_RETIRE. These are branch instructions where the branch was mispredicted, but does not include immediate or return instructions
0x811C	BR_PRED_RETIRE	Branch instruction architecturally executed, predicted branch The counter counts the instructions on the architecturally executed path counted by BR_RETIRE that are not counted by BR_MIS_PRED_RETIRE. These are branch instructions, where the branch was correctly predicted.
0x811D	BR_IND_RETIRE	Instruction architecturally executed, indirect branch
0x8120	INST_FETCH_PERCYC	Event in progress, INST_FETCH The counter counts by the number of INST_FETCH events in progress on each Processor cycle
0x8121	MEM_ACCESS_RD_PERCYC	Event in progress, MEM_ACCESS_RD The counter counts by the number of MEM_ACCESS_RD events in progress on each Processor Cycle
0x8124	INST_FETCH	Instruction memory access The counter counts each Instruction memory access that the PE makes
0x8128	DTLB_WALK_PERCYC	Total cycles, DTLB_WALK The counter counts by the number of data TLB walk events in progress on each processor cycle.
0x8129	ITLB_WALK_PERCYC	Total cycles, ITLB_WALK The counter counts by the number of instruction TLB walk events in progress on each processor cycle.

Event number	Mnemonic	Description
0x812A	SAMPLE_FEED_BR	Statistical Profiling sample taken, branch The counter counts each sample counted by SAMPLE_FEED that are branch operations.
0x812B	SAMPLE_FEED_LD	Statistical Profiling sample taken, load The counter counts each sample counted by SAMPLE_FEED that are load or load atomic operations.
0x812C	SAMPLE_FEED_ST	Statistical Profiling sample taken, store The counter counts each sample counted by SAMPLE_FEED that are store or atomic operations, including load atomic operations.
0x812D	SAMPLE_FEED_OP	Statistical Profiling sample taken, matching operation type The counter counts each sample counted by SAMPLE_FEED that meets the operation type filter constraints.
0x812E	SAMPLE_FEED_EVENT	Statistical Profiling sample taken, matching events The counter counts each sample counted by SAMPLE_FEED that meets the Events packet filter constraints.
0x812F	SAMPLE_FEED_LAT	Statistical Profiling sample taken, exceeding minimum latency The counter counts each sample counted by SAMPLE_FEED that meets the operation latency filter constraints.
0x8134	DTLB_HWUPD	Data TLB hardware update of translation table The counter counts each access counted by L1D_TLB that causes a hardware update of a translation table entry.
0x8135	ITLB_HWUPD	Instruction TLB hardware update of translation table The counter counts each access counted by L1I_TLB that causes a hardware update of a translation table entry.
0x8136	DTLB_STEP	Data TLB translation table walk, step The counter counts each translation table walk access made by a refill of the data or unified TLB.
0x8137	ITLB_STEP	Instruction TLB translation table walk, step The counter counts each translation table walk access made by a refill of the instruction TLB.
0x8138	DTLB_WALK_LARGE	Data TLB large page translation table walk The counter counts each translation table walk counted by DTLB_WALK where the result of the walk yields a large page size.
0x8139	ITLB_WALK_LARGE	Instruction TLB large page translation table walk The counter counts each translation table walk counted by ITLB_WALK where the result of the walk yields a large page size.

Event number	Mnemonic	Description
0x813A	DTLB_WALK_SMALL	Data TLB small page translation table walk The counter counts each translation table walk counted by DTLB_WALK where the result of the walk yields a small page size.
0x813B	ITLB_WALK_SMALL	Instruction TLB small page translation table walk The counter counts each translation table walk counted by ITLB_WALK where the result of the walk yields a small page size.
0x8140	L1D_CACHE_RW	Level 1 data cache demand access The counter counts each access counted by L1D_CACHE that is due to a demand read or demand write access.
0x8148	L2D_CACHE_RW	Level 2 data cache demand access The counter counts each access counted by L2D_CACHE that is due to a demand Memory-read operation or demand Memory-write operation.
0x8149	L2I_CACHE_RD	Level 2 instruction cache demand fetch The counter counts each access counted by L2I_CACHE that is due to a demand instruction memory access
0x8152	L3D_CACHE_MISS	Level 3 data cache demand access miss The counter counts each access counted by L3D_CACHE_RW that misses in the L1 to L3 data or unified caches, causing an access to outside of the L1 to L3 caches of this PE. If IMP_CPUCFR_EL1.no_scu is set to 1, this event does not count.
0x8154	L1D_CACHE_HWPRF	Level 1 data cache hardware prefetch The counter counts each fetch triggered by L1 prefetchers
0x8158	STALL_FRONTEND_MEMBOUND	Frontend stall cycles, memory bound The counter counts each cycle counted by STALL_FRONTEND when no instructions are delivered from the memory system.
0x8159	STALL_FRONTEND_L1I	Frontend stall cycles, level 1 instruction cache The counter counts each cycle counted by STALL_FRONTEND_MEMBOUND when there is a demand miss in the first level instruction cache.
0x815B	STALL_FRONTEND_MEM	Frontend stall cycles, last level PE cache or memory The counter counts each cycle counted by STALL_FRONTEND_MEMBOUND when there is a demand instruction miss in the last level of instruction or unified cache within the PE clock domain or a non-cacheable instruction fetch in progress.
0x815C	STALL_FRONTEND_TLB	Frontend stall cycles, TLB The counter counts each cycle counted by STALL_FRONTEND_MEMBOUND when there is an instruction or unified TLB demand miss.
0x8160	STALL_FRONTEND_CPUBOUND	Frontend stall cycles, processor bound The counter counts each cycle counted by STALL_FRONTEND when the frontend is stalled on a frontend processor resource, not including memory.

Event number	Mnemonic	Description
0x8162	STALL_FRONTEND_FLUSH	Frontend stall cycles, flush recovery. No operation sent for execution due to the frontend flush recovery.
0x8164	STALL_BACKEND_MEMBOUND	Backend stall cycles, memory bound The counter counts each cycle counted by STALL_BACKEND when the backend is waiting for a memory access to complete.
0x8165	STALL_BACKEND_L1D	Backend stall cycles, level 1 data cache The counter counts each cycle counted by STALL_BACKEND_MEMBOUND where there is a demand data miss in the L1 of data or unified cache.
0x8167	STALL_BACKEND_TLB	Backend stall cycles, TLB The counter counts each cycle counted by STALL_BACKEND_MEMBOUND where there is a demand data miss in the data or unified TLB.
0x8168	STALL_BACKEND_ST	Back stall cycles store The counter counts each cycle counted by STALL_BACKEND_MEMBOUND when the backend is stalled waiting for a store.
0x816A	STALL_BACKEND_CPUBOUND	Backend stall cycles, processor bound The counter counts each cycle counted by STALL_BACKEND when the backend is stalled on a processor resource, not including memory.
0x816B	STALL_BACKEND_BUSY	Backend stall cycles, backend busy The counter counts each cycle by STALL_BACKEND when operations are available from the frontend but the backend is not able to accept an operation because an execution unit is busy.
0x816D	STALL_BACKEND_RENAME	Backend stall cycles, rename full The counter counts each cycle counted by STALL_BACKEND_CPUBOUND when operation are available from the frontend but at least one is not ready to be sent to the backend because no rename register is available.
0x8171	CAS_NEAR_PASS	Atomic memory Operation speculatively executed, Compare and Swap pass The counter counts each Compare and Swap operation counted by CAS_NEAR_SPEC that updates the location accessed.
0x8172	CAS_NEAR_SPEC	Atomic memory Operation speculatively executed, Compare and Swap near The counter counts each Compare and Swap operation that executes locally to the PE.
0x8173	CAS_FAR_SPEC	Atomic memory Operation speculatively executed, Compare and Swap far The counter counts each Compare and Swap operation that does not execute locally to the PE.
0x81BC	L1D_CACHE_REFILL_HWPRF	Level 1 data cache refill, hardware prefetch The counter counts each refill triggered by L1 prefetchers

Event number	Mnemonic	Description
0x8285	L2D_CACHE_PRF	Level 2 data cache, preload or prefetch hit The counter counts each fetch counted by either L2D_CACHE_HWPRF or L2D_CACHE_PRFM
0x828D	L2D_CACHE_REFILL_PRF	Level 2 data cache refill, preload or prefetch hit The counter counts each refill counted by either L2D_CACHE_REFILL_HWPRF or L2D_CACHE_REFILL_PRFM.
0x829A	LL_CACHE_REFILL	Last level cache refill The counter counts each access counted by LL_CACHE that causes a refill of the Last level cache, or any other data, instruction, or unified cache of this PE, from outside of those caches.

18.2 Performance monitors interrupts

The *Performance Monitoring Unit* (PMU) can be configured to generate an interrupt when one or more of the counters overflow.

When the PMU generates an interrupt, the nPMUIRQ[n] output is driven LOW.

See *Performance Monitors Extension support* in the [Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual](#) for more information.

18.3 External register access permissions

The Neoverse™ N3 core supports access to the *Performance Monitoring Unit* (PMU) registers from the system register interface and a memory-mapped interface.

Access to a register depends on:

- Whether the core is powered up
- The state of the OS Lock
- The state of External Performance Monitors Access Disable

The behavior is specific to each register and is not described in this manual. For a detailed description of these features and their effects on the registers, see the [Arm® Architecture Reference Manual for A-profile architecture](#). The register descriptions provided in this manual describe whether each register is read/write or read-only.

18.4 AArch64 Performance Monitors registers

The summary table provides an overview of all Performance Monitors registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 18-2: Performance Monitors registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMINTENSET_EL1	3	0	C9	C14	1	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Set register
PMINTENCLR_EL1	3	0	C9	C14	2	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Clear register
PMMIR_EL1	3	0	C9	C14	6	See individual bit resets.	64-bit	Performance Monitors Machine Identification Register
IMP_CLUSTERPMCR_EL1	3	0	C15	C5	0	See individual bit resets.	64-bit	Performance Monitors Control Register
IMP_CLUSTERPMCNTENSET_EL1	3	0	C15	C5	1	See individual bit resets.	64-bit	Performance Monitors Count Enable Set Register
IMP_CLUSTERPMCNTENCLR_EL1	3	0	C15	C5	2	See individual bit resets.	64-bit	Performance Monitors Count Enable Clear Register
IMP_CLUSTERPMOVSSET_EL1	3	0	C15	C5	3	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Set Register
IMP_CLUSTERPMOVSCCLR_EL1	3	0	C15	C5	4	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Clear Register
IMP_CLUSTERPMSELR_EL1	3	0	C15	C5	5	See individual bit resets.	64-bit	Performance Monitors Event Counter Selection Register
IMP_CLUSTERPMINTENSET_EL1	3	0	C15	C5	6	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Set Register
IMP_CLUSTERPMINTENCLR_EL1	3	0	C15	C5	7	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Clear Register
IMP_CLUSTERPMCCNTR_EL1	3	0	C15	C6	0	See individual bit resets.	64-bit	Performance Monitors Cycle Count Register
IMP_CLUSTERPMXEVTYPER_EL1	3	0	C15	C6	1	See individual bit resets.	64-bit	Performance Monitors Selected Event Type Register
IMP_CLUSTERPMXVCNTR_EL1	3	0	C15	C6	2	See individual bit resets.	64-bit	Performance Monitors Selected Event Count Register
IMP_CLUSTERPMCEIDO_EL1	3	0	C15	C6	4	See individual bit resets.	64-bit	Performance Monitors Common Event Identification Register 0
IMP_CLUSTERPMCEID1_EL1	3	0	C15	C6	5	See individual bit resets.	64-bit	Performance Monitors Common Event Identification Register 1
IMP_CLUSTERRSVD_6_6_EL1	3	0	C15	C6	6	See individual bit resets.	64-bit	Reserved

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CLUSTERRSVD_6_7_EL1	3	0	C15	C6	7	See individual bit resets.	64-bit	Reserved
PMCR_ELO	3	3	C9	C12	0	See individual bit resets.	64-bit	Performance Monitors Control Register
PMCNTENSET_ELO	3	3	C9	C12	1	See individual bit resets.	64-bit	Performance Monitors Count Enable Set register
PMCNTENCLR_ELO	3	3	C9	C12	2	See individual bit resets.	64-bit	Performance Monitors Count Enable Clear register
PMOVSLR_ELO	3	3	C9	C12	3	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Clear Register
PMSWINC_ELO	3	3	C9	C12	4	See individual bit resets.	64-bit	Performance Monitors Software Increment register
PMSELR_ELO	3	3	C9	C12	5	See individual bit resets.	64-bit	Performance Monitors Event Counter Selection Register
PMCEID0_ELO	3	3	C9	C12	6	See individual bit resets.	64-bit	Performance Monitors Common Event Identification register 0
PMCEID1_ELO	3	3	C9	C12	7	See individual bit resets.	64-bit	Performance Monitors Common Event Identification register 1
PMCCNTR_ELO	3	3	C9	C13	0	See individual bit resets.	64-bit	Performance Monitors Cycle Count Register
PMXEVTYPER_ELO	3	3	C9	C13	1	See individual bit resets.	64-bit	Performance Monitors Selected Event Type Register
PMXVCNTR_ELO	3	3	C9	C13	2	See individual bit resets.	64-bit	Performance Monitors Selected Event Count Register
PMUSERENR_ELO	3	3	C9	C14	0	See individual bit resets.	64-bit	Performance Monitors User Enable Register
PMOVSSET_ELO	3	3	C9	C14	3	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Set register
PMEVCNTR0_ELO	3	3	C14	C8	0	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR1_ELO	3	3	C14	C8	1	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR2_ELO	3	3	C14	C8	2	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR3_ELO	3	3	C14	C8	3	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR4_ELO	3	3	C14	C8	4	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR5_ELO	3	3	C14	C8	5	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR6_ELO	3	3	C14	C8	6	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR7_ELO	3	3	C14	C8	7	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR8_ELO	3	3	C14	C9	0	See individual bit resets.	64-bit	Performance Monitors Event Count Registers

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMEVCNTR9_ELO	3	3	C14	C9	1	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR10_ELO	3	3	C14	C9	2	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR11_ELO	3	3	C14	C9	3	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR12_ELO	3	3	C14	C9	4	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR13_ELO	3	3	C14	C9	5	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR14_ELO	3	3	C14	C9	6	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR15_ELO	3	3	C14	C9	7	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR16_ELO	3	3	C14	C10	0	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR17_ELO	3	3	C14	C10	1	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR18_ELO	3	3	C14	C10	2	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR19_ELO	3	3	C14	C10	3	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVTYPER0_ELO	3	3	C14	C12	0	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER1_ELO	3	3	C14	C12	1	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER2_ELO	3	3	C14	C12	2	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER3_ELO	3	3	C14	C12	3	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER4_ELO	3	3	C14	C12	4	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER5_ELO	3	3	C14	C12	5	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER6_ELO	3	3	C14	C12	6	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER7_ELO	3	3	C14	C12	7	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER8_ELO	3	3	C14	C13	0	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER9_ELO	3	3	C14	C13	1	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER10_ELO	3	3	C14	C13	2	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER11_ELO	3	3	C14	C13	3	See individual bit resets.	64-bit	Performance Monitors Event Type Registers

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMEVTYPEPER12_ELO	3	3	C14	C13	4	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER13_ELO	3	3	C14	C13	5	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER14_ELO	3	3	C14	C13	6	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER15_ELO	3	3	C14	C13	7	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER16_ELO	3	3	C14	C14	0	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER17_ELO	3	3	C14	C14	1	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER18_ELO	3	3	C14	C14	2	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPEPER19_ELO	3	3	C14	C14	3	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMCCFILTR_ELO	3	3	C14	C15	7	See individual bit resets.	64-bit	Performance Monitors Cycle Count Filter Register

18.5 External PMU registers

The summary table provides an overview of all memory-mapped PMU registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 18-3: PMU registers summary

Offset	Name	Reset	Width	Description
0x0	PMEVCNTR0_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x8	PMEVCNTR1_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x10	PMEVCNTR2_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x18	PMEVCNTR3_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x20	PMEVCNTR4_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x28	PMEVCNTR5_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x30	PMEVCNTR6_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x38	PMEVCNTR7_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x40	PMEVCNTR8_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x48	PMEVCNTR9_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x50	PMEVCNTR10_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers

Offset	Name	Reset	Width	Description
0x58	PMEVCNTR11_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x60	PMEVCNTR12_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x68	PMEVCNTR13_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x70	PMEVCNTR14_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x78	PMEVCNTR15_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x80	PMEVCNTR16_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x88	PMEVCNTR17_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x90	PMEVCNTR18_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x98	PMEVCNTR19_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x0F8	PMCCNTR_ELO [31:0]	See individual bit resets.	32-bit	Performance Monitors Cycle Counter
0x0FC	PMCCNTR_ELO [63:32]	See individual bit resets.	32-bit	Performance Monitors Cycle Counter
0x200	PMPCSR [31:0]	See individual bit resets.	32-bit	Program Counter Sample Register
0x204	PMPCSR [63:32]	See individual bit resets.	32-bit	Program Counter Sample Register
0x220	PMPCSR [31:0]	See individual bit resets.	32-bit	Program Counter Sample Register
0x224	PMPCSR [63:32]	See individual bit resets.	32-bit	Program Counter Sample Register
0x208	PMCID1SR	See individual bit resets.	32-bit	CONTEXTIDR_EL1 Sample Register
0x228	PMCID1SR	See individual bit resets.	32-bit	CONTEXTIDR_EL1 Sample Register
0x20C	PMVIDSR	See individual bit resets.	32-bit	VMID Sample Register
0x22C	PMCID2SR	See individual bit resets.	32-bit	CONTEXTIDR_EL2 Sample Register
0x400	PMEVTYPER0_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x404	PMEVTYPER1_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x408	PMEVTYPER2_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x40C	PMEVTYPER3_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x410	PMEVTYPER4_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x414	PMEVTYPER5_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x418	PMEVTYPER6_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x41C	PMEVTYPER7_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x420	PMEVTYPER8_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x424	PMEVTYPER9_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x428	PMEVTYPER10_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x42C	PMEVTYPER11_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x430	PMEVTYPER12_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x434	PMEVTYPER13_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x438	PMEVTYPER14_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x43C	PMEVTYPER15_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x440	PMEVTYPER16_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x444	PMEVTYPER17_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x448	PMEVTYPER18_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x44C	PMEVTYPER19_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x47C	PMCCFILTR_ELO	See individual bit resets.	32-bit	Performance Monitors Cycle Counter Filter Register

Offset	Name	Reset	Width	Description
0x600	PMPCSSR	See individual bit resets.	64-bit	Snapshot Program Counter Sample Register
0x608	PMCIDSSR	See individual bit resets.	32-bit	Snapshot CONTEXTIDR_EL1 Sample Register
0x60C	PMCID2SSR	See individual bit resets.	32-bit	Snapshot CONTEXTIDR_EL2 Sample Register
0x610	PMSSSR	See individual bit resets.	32-bit	PMU Snapshot Status Register
0x618	PMCCNTSR	See individual bit resets.	64-bit	PMU Cycle Counter Snapshot Register
0x620	PMEVCNTSR0	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x628	PMEVCNTSR1	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x630	PMEVCNTSR2	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x638	PMEVCNTSR3	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x640	PMEVCNTSR4	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x648	PMEVCNTSR5	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x650	PMEVCNTSR6	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x658	PMEVCNTSR7	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x660	PMEVCNTSR8	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x668	PMEVCNTSR9	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x670	PMEVCNTSR10	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x678	PMEVCNTSR11	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x680	PMEVCNTSR12	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x688	PMEVCNTSR13	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x690	PMEVCNTSR14	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x698	PMEVCNTSR15	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6A0	PMEVCNTSR16	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6A8	PMEVCNTSR17	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6B0	PMEVCNTSR18	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6B8	PMEVCNTSR19	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0xC00	PMCNTENSET_ELO	See individual bit resets.	32-bit	Performance Monitors Count Enable Set register
0xC20	PMCNTENCLR_ELO	See individual bit resets.	32-bit	Performance Monitors Count Enable Clear register
0xC40	PMINTENSET_EL1	See individual bit resets.	32-bit	Performance Monitors Interrupt Enable Set register
0xC60	PMINTENCLR_EL1	See individual bit resets.	32-bit	Performance Monitors Interrupt Enable Clear register
0xC80	PMOVSLR_ELO	See individual bit resets.	32-bit	Performance Monitors Overflow Flag Status Clear register
0xCA0	PMSWINC_ELO	See individual bit resets.	32-bit	Performance Monitors Software Increment register
0xCC0	PMOVSET_ELO	See individual bit resets.	32-bit	Performance Monitors Overflow Flag Status Set register
0xE00	PMCFGR	See individual bit resets.	32-bit	Performance Monitors Configuration Register
0xE04	PMCR_ELO	See individual bit resets.	32-bit	Performance Monitors Control Register
0xE20	PMCEID0	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 0
0xE24	PMCEID1	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 1
0xE28	PMCEID2	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 2
0xE2C	PMCEID3	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 3
0xE30	PMSSCR	See individual bit resets.	32-bit	PMU Snapshot Capture Register
0xE40	PMMIR	See individual bit resets.	32-bit	Performance Monitors Machine Identification Register

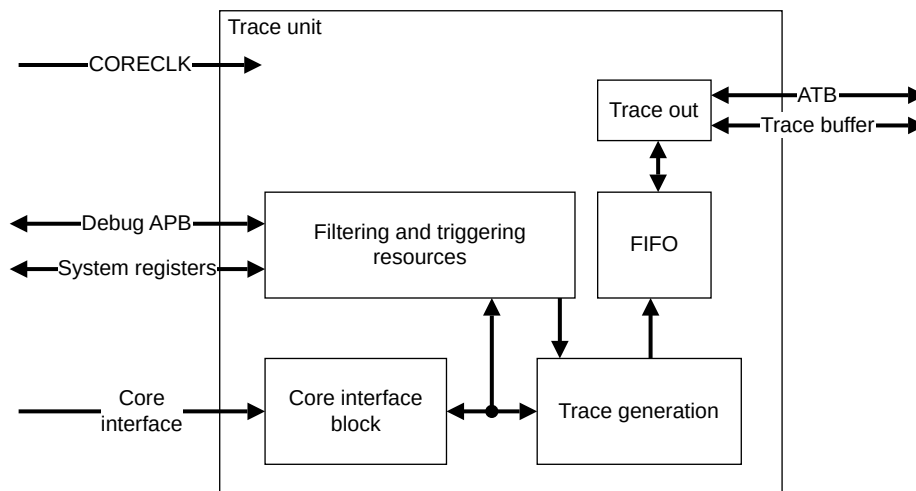
Offset	Name	Reset	Width	Description
0xFA8	PMDEVAFF0	See individual bit resets.	32-bit	Performance Monitors Device Affinity register 0
0xFAC	PMDEVAFF1	See individual bit resets.	32-bit	Performance Monitors Device Affinity register 1
0xFB0	PMLAR	See individual bit resets.	32-bit	Performance Monitors Lock Access Register
0xFB4	PMLSR	See individual bit resets.	32-bit	Performance Monitors Lock Status Register
0xFB8	PMAUTHSTATUS	See individual bit resets.	32-bit	Performance Monitors Authentication Status register
0xFBC	PMDEVARCH	See individual bit resets.	32-bit	Performance Monitors Device Architecture register
0xFC8	PMDEVID	See individual bit resets.	32-bit	Performance Monitors Device ID register
0xFCC	PMDEVTYPE	See individual bit resets.	32-bit	Performance Monitors Device Type register
0xFD0	PMPIDR4	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 4
0xFE0	PMPIDR0	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 0
0xFE4	PMPIDR1	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 1
0xFE8	PMPIDR2	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 2
0xFEC	PMPIDR3	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 3
0xFF0	PMCIDR0	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 0
0xFF4	PMCIDR1	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 1
0xFF8	PMCIDR2	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 2
0xFFC	PMCIDR3	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 3

19. Embedded Trace Extension support

The Neoverse™ N3 core implements the *Embedded Trace Extension* (ETE). The trace unit performs real-time instruction flow tracing based on the ETE. The trace unit is a CoreSight component and is an integral part of the Arm real-time debug solution.

The following figure shows the main components of the trace unit:

Figure 19-1: Trace unit components



Core interface

The core interface monitors and generates P0 elements that are essentially executed branches and exceptions traced in program order.

Trace generation

The trace generation logic generates various trace packets based on P0 elements.

Filtering and triggering resources

You can limit the amount of trace data that the trace unit generates by filtering. For example, you can limit trace generation to a certain address range. The trace unit supports other logic analyzer style filtering options. The trace unit can also generate a trigger that is a signal to the Trace Capture Device to stop capturing trace.

FIFO

The trace unit generates trace in a highly compressed form. The *First In First Out* (FIFO) enables trace bursts to be flattened out. When the FIFO is full, the FIFO signals an overflow. The trace generation logic does not generate any new trace until the FIFO is emptied. This behavior causes a gap in the trace when viewed in the debugger.

Trace out

Trace from the FIFO is output on the AMBA ATB interface or to the trace buffer.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

19.1 Trace unit resources

Trace resources include counters, external input and output signals, and comparators.

The following table shows the trace unit resources, and indicates which of these resources the Neoverse™ N3 core implements.

Table 19-1: Trace unit resources implemented

Description	Configuration
Number of resource selection pairs implemented	8
Number of external input selectors implemented	4
Number of <i>Embedded Trace Extension</i> (ETE) events	4
Number of counters implemented	2
Reduced function counter implemented	Not implemented
Number of sequencer states implemented	4
Number of Virtual Machine ID comparators implemented	1
Number of Context ID comparators implemented	1
Number of address comparator pairs implemented	4
Number of single-shot comparator controls	1
Number of core comparator inputs implemented	0
Data address comparisons implemented	Not implemented
Number of data value comparators implemented	0

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

19.2 Trace unit generation options

The Neoverse™ N3 core trace unit implements a set of generation options.

The following table shows the trace generation options that are implemented in the Neoverse™ N3 core trace unit.

Table 19-2: Trace unit generation options implemented

Description	Configuration
Instruction address size in bytes	8

Description	Configuration
Data address size in bytes	0, because the <i>Embedded Trace Extension</i> (ETE) does not implement data tracing
Data value size in bytes	0, because the ETE does not implement data tracing
Virtual Machine ID size in bytes	4
Context ID size in bytes	4
Support for conditional instruction tracing	Not implemented
Support for tracing of data	Not implemented
Support for tracing of load and store instructions because PO elements	Not implemented
Support for cycle counting in the instruction trace	Implemented
Support for branch broadcast tracing	Implemented
Number of events that are supported in the trace	4
Return stack support	Implemented
Tracing of SError exception support	Implemented
Instruction trace cycle counting minimum threshold	4
Size of Trace ID	7 bits
Synchronization period support	Read/write
Global timestamp size	64 bits
Number of cores available for tracing	1
ATB trigger support	Implemented
Low-power behavior override	Not implemented
Stall control support	Not implemented
Support for overflow avoidance	Not implemented
Support for using CONTEXTIDR_EL2 in <i>Virtual Machine Identifier</i> (VMID) comparator	Implemented

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

19.3 Reset the trace unit

The reset for the trace buffer is the same as a Cold reset for the core. When using the *TRace Buffer Extension* (TRBE), a Warm reset disables the trace buffer and therefore it is not possible to use the trace buffer to capture trace for a Warm reset.

If the trace unit is reset, then tracing stops until the trace unit is reprogrammed and re-enabled. However, if the core is reset using Warm reset, the last few instructions provided by the core before the reset might not be traced.

19.4 Program and read the trace unit registers

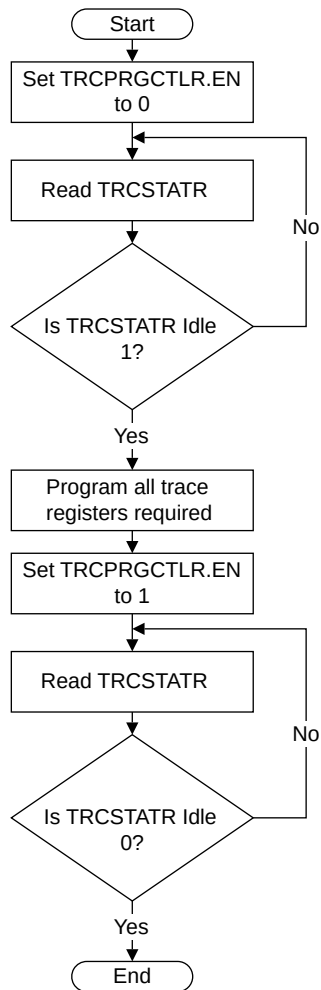
You must program and read the trace unit registers using either the Debug *Advanced Peripheral Bus* (APB) interface or the System register interface.

The core does not have to be in debug state when you program the trace unit registers. When you program the trace unit registers, you must enable all the changes at the same time. Otherwise, if you program the counter, it might start to count based on incorrect events before the correct setup is in place for the trigger condition. To disable the trace unit, use the TRCPRGCTLR.EN bit.

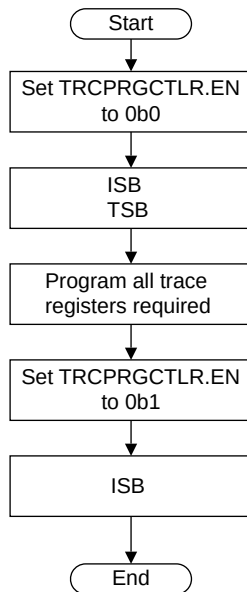
See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about the following trace unit registers:

- Programming Control Register, TRCPRGCTLR
- Trace Status Register, TRCSTATR

The following figure shows the flow for programming trace unit registers using the Debug APB interface:

Figure 19-2: Programming trace unit registers using the Debug APB interface

The following figure shows the flow for programming trace unit registers using the System register interface:

Figure 19-3: Programming trace registers using the System register interface

19.5 Trace unit register interfaces

The Neoverse™ N3 core supports an *Advanced Peripheral Bus* (APB) memory-mapped interface and a system register interface to trace unit registers.

Register accesses differ depending on the trace unit state. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for information on the behaviors and access mechanisms.

19.6 Interaction with the Performance Monitoring Unit and Debug

The trace unit interacts with the *Performance Monitoring Unit* (PMU) and it can access the PMU events.

Interaction with the PMU

The Neoverse™ N3 core includes a PMU that enables events, such as cache misses and executed instructions, to be counted over time.

The PMU and trace unit function together.

Use of PMU events by the trace unit

The PMU architectural events are available to the trace unit through the extended input facility. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information about PMU events.

The trace unit uses four extended external input selectors to access the PMU events. Each selector can independently select one of the PMU events, that are then active for the cycles where the relevant events occur. These selected events can then be accessed by any of the event registers within the trace unit.

Related information

[18. Performance Monitors Extension support](#) on page 110

[18.1 Performance monitors events](#) on page 110

19.7 Embedded Trace Extension events

The Neoverse™ N3 core trace unit collects events from other units in the design and uses numbers to reference these events.

Other than the events mentioned in [18.1 Performance monitors events](#) on page 110, the events listed in the following table are also exported.

Table 19-3: ETE events

Event number	Event mnemonic	Description
0x400D	PMU_OVFS	PMU overflow, counters accessible to EL1 and EL0
0x400E	TRB_TRIG	Trace buffer Trigger Event
0x400F	PMU_HOVS	PMU overflow, counters reserved for use by EL2

19.8 AArch64 Trace unit registers

The summary table provides an overview of all Trace unit registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 19-4: Trace unit registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCTRACEIDR	2	1	C0	C0	1	See individual bit resets.	64-bit	Trace ID Register
TRCVICTLR	2	1	C0	C0	2	See individual bit resets.	64-bit	ViewInst Main Control Register
TRCSEQEVRO	2	1	C0	C0	4	See individual bit resets.	64-bit	Sequencer State Transition Control Register <n>
TRCCNTRLDVRO	2	1	C0	C0	5	See individual bit resets.	64-bit	Counter Reload Value Register <n>

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCIDR8	2	1	C0	C0	6	See individual bit resets.	64-bit	ID Register 8
TRCIMSPECO	2	1	C0	C0	7	See individual bit resets.	64-bit	IMP DEF Register 0
TRCPRGCTLR	2	1	C0	C1	0	See individual bit resets.	64-bit	Programming Control Register
TRCVIIECTLR	2	1	C0	C1	2	See individual bit resets.	64-bit	ViewInst Include/Exclude Control Register
TRCSEQEVR1	2	1	C0	C1	4	See individual bit resets.	64-bit	Sequencer State Transition Control Register <n>
TRCCNTRLDVR1	2	1	C0	C1	5	See individual bit resets.	64-bit	Counter Reload Value Register <n>
TRCIDR9	2	1	C0	C1	6	See individual bit resets.	64-bit	ID Register 9
TRCVISSCTLR	2	1	C0	C2	2	See individual bit resets.	64-bit	ViewInst Start/Stop Control Register
TRCSEQEVR2	2	1	C0	C2	4	See individual bit resets.	64-bit	Sequencer State Transition Control Register <n>
TRCIDR10	2	1	C0	C2	6	See individual bit resets.	64-bit	ID Register 10
TRCSTATR	2	1	C0	C3	0	See individual bit resets.	64-bit	Trace Status Register
TRCIDR11	2	1	C0	C3	6	See individual bit resets.	64-bit	ID Register 11
TRCCONFIGR	2	1	C0	C4	0	See individual bit resets.	64-bit	Trace Configuration Register
TRCCNTCTLR0	2	1	C0	C4	5	See individual bit resets.	64-bit	Counter Control Register <n>
TRCIDR12	2	1	C0	C4	6	See individual bit resets.	64-bit	ID Register 12
TRCCNTCTLR1	2	1	C0	C5	5	See individual bit resets.	64-bit	Counter Control Register <n>
TRCIDR13	2	1	C0	C5	6	See individual bit resets.	64-bit	ID Register 13
TRCAUXCTLR	2	1	C0	C6	0	See individual bit resets.	64-bit	Auxiliary Control Register
TRCSEQRSTEV	2	1	C0	C6	4	See individual bit resets.	64-bit	Sequencer Reset Control Register
TRCSEQSTR	2	1	C0	C7	4	See individual bit resets.	64-bit	Sequencer State Register
TRCEVENTCTLR	2	1	C0	C8	0	See individual bit resets.	64-bit	Event Control 0 Register
TRCEXTINSELRO	2	1	C0	C8	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCCNTVR0	2	1	C0	C8	5	See individual bit resets.	64-bit	Counter Value Register <n>

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCIDR0	2	1	C0	C8	7	See individual bit resets.	64-bit	ID Register 0
TRCEVENTCTL1R	2	1	C0	C9	0	See individual bit resets.	64-bit	Event Control 1 Register
TRCEXTINSELR1	2	1	C0	C9	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCCNTVR1	2	1	C0	C9	5	See individual bit resets.	64-bit	Counter Value Register <n>
TRCIDR1	2	1	C0	C9	7	See individual bit resets.	64-bit	ID Register 1
TRCRSR	2	1	C0	C10	0	See individual bit resets.	64-bit	Resources Status Register
TRCEXTINSELR2	2	1	C0	C10	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCIDR2	2	1	C0	C10	7	See individual bit resets.	64-bit	ID Register 2
TRCEXTINSELR3	2	1	C0	C11	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCIDR3	2	1	C0	C11	7	See individual bit resets.	64-bit	ID Register 3
TRCTSCTLR	2	1	C0	C12	0	See individual bit resets.	64-bit	Timestamp Control Register
TRCIDR4	2	1	C0	C12	7	See individual bit resets.	64-bit	ID Register 4
TRCSYNCPR	2	1	C0	C13	0	See individual bit resets.	64-bit	Synchronization Period Register
TRCIDR5	2	1	C0	C13	7	See individual bit resets.	64-bit	ID Register 5
TRCCCCTLR	2	1	C0	C14	0	See individual bit resets.	64-bit	Cycle Count Control Register
TRCIDR6	2	1	C0	C14	7	See individual bit resets.	64-bit	ID Register 6
TRCBBCTLR	2	1	C0	C15	0	See individual bit resets.	64-bit	Branch Broadcast Control Register
TRCIDR7	2	1	C0	C15	7	See individual bit resets.	64-bit	ID Register 7
TRCSSCCR0	2	1	C1	C0	2	See individual bit resets.	64-bit	Single-shot Comparator Control Register <n>
TRCOSLSR	2	1	C1	C1	4	See individual bit resets.	64-bit	Trace OS Lock Status Register
TRCRSCTLR2	2	1	C1	C2	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR3	2	1	C1	C3	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR4	2	1	C1	C4	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCRSCTLR5	2	1	C1	C5	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR6	2	1	C1	C6	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR7	2	1	C1	C7	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR8	2	1	C1	C8	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCSSCSR0	2	1	C1	C8	2	See individual bit resets.	64-bit	Single-shot Comparator Control Status Register <n>
TRCRSCTLR9	2	1	C1	C9	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR10	2	1	C1	C10	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR11	2	1	C1	C11	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR12	2	1	C1	C12	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR13	2	1	C1	C13	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR14	2	1	C1	C14	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR15	2	1	C1	C15	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCACVR0	2	1	C2	C0	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR0	2	1	C2	C0	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR1	2	1	C2	C2	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR1	2	1	C2	C2	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR2	2	1	C2	C4	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR2	2	1	C2	C4	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR3	2	1	C2	C6	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR3	2	1	C2	C6	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR4	2	1	C2	C8	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR4	2	1	C2	C8	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR5	2	1	C2	C10	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCACATR5	2	1	C2	C10	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR6	2	1	C2	C12	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR6	2	1	C2	C12	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR7	2	1	C2	C14	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR7	2	1	C2	C14	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCCIDCVR0	2	1	C3	C0	0	See individual bit resets.	64-bit	Context Identifier Comparator Value Registers <n>
TRCVMIDCVR0	2	1	C3	C0	1	See individual bit resets.	64-bit	Virtual Context Identifier Comparator Value Register <n>
TRCCIDCCTLR0	2	1	C3	C0	2	See individual bit resets.	64-bit	Context Identifier Comparator Control Register 0
TRCVMIDCCTLR0	2	1	C3	C2	2	See individual bit resets.	64-bit	Virtual Context Identifier Comparator Control Register 0
TRCDEVID	2	1	C7	C2	7	See individual bit resets.	64-bit	Device Configuration Register
TRCCCLAIMSET	2	1	C7	C8	6	See individual bit resets.	64-bit	Claim Tag Set Register
TRCCCLAIMCLR	2	1	C7	C9	6	See individual bit resets.	64-bit	Claim Tag Clear Register
TRCAUTHSTATUS	2	1	C7	C14	6	See individual bit resets.	64-bit	Authentication Status Register
TRCDEVARCH	2	1	C7	C15	6	See individual bit resets.	64-bit	Device Architecture Register

19.9 External ETE registers

The summary table provides an overview of all memory-mapped ETE registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 19-5: ETE registers summary

Offset	Name	Reset	Width	Description
0x004	TRCPRGCTLR	See individual bit resets.	32-bit	Programming Control Register
0x00C	TRCSTATR	See individual bit resets.	32-bit	Trace Status Register
0x010	TRCCONFIGR	See individual bit resets.	32-bit	Trace Configuration Register

Offset	Name	Reset	Width	Description
0x018	TRCAUXCTLR	See individual bit resets.	32-bit	Auxiliary Control Register
0x020	TRCEVENTCTLOR	See individual bit resets.	32-bit	Event Control 0 Register
0x024	TRCEVENTCTL1R	See individual bit resets.	32-bit	Event Control 1 Register
0x028	TRCRSR	See individual bit resets.	32-bit	Resources Status Register
0x030	TRCTSCTLR	See individual bit resets.	32-bit	Timestamp Control Register
0x034	TRCSYNCPR	See individual bit resets.	32-bit	Synchronization Period Register
0x038	TRCCCTLR	See individual bit resets.	32-bit	Cycle Count Control Register
0x03C	TRCBCTLR	See individual bit resets.	32-bit	Branch Broadcast Control Register
0x040	TRCTRACEIDR	See individual bit resets.	32-bit	Trace ID Register
0x044	TRCQCTLR	See individual bit resets.	32-bit	Q Element Control Register
0x080	TRCVICTLR	See individual bit resets.	32-bit	ViewInst Main Control Register
0x084	TRCVIICTLR	See individual bit resets.	32-bit	ViewInst Include/Exclude Control Register
0x088	TRCVISSCTLR	See individual bit resets.	32-bit	ViewInst Start/Stop Control Register
0x100	TRCSEQEVR0	See individual bit resets.	32-bit	Sequencer State Transition Control Register <n>
0x104	TRCSEQEVR1	See individual bit resets.	32-bit	Sequencer State Transition Control Register <n>
0x108	TRCSEQEVR2	See individual bit resets.	32-bit	Sequencer State Transition Control Register <n>
0x118	TRCSEQRSTEVR	See individual bit resets.	32-bit	Sequencer Reset Control Register
0x11C	TRCSEQSTR	See individual bit resets.	32-bit	Sequencer State Register
0x120	TRCEXTINSELRO	See individual bit resets.	32-bit	External Input Select Register <n>
0x124	TRCEXTINSELR1	See individual bit resets.	32-bit	External Input Select Register <n>
0x128	TRCEXTINSELR2	See individual bit resets.	32-bit	External Input Select Register <n>
0x12C	TRCEXTINSELR3	See individual bit resets.	32-bit	External Input Select Register <n>
0x140	TRCCNTRLDVR0	See individual bit resets.	32-bit	Counter Reload Value Register <n>
0x144	TRCCNTRLDVR1	See individual bit resets.	32-bit	Counter Reload Value Register <n>
0x150	TRCCNTCTLR0	See individual bit resets.	32-bit	Counter Control Register <n>
0x154	TRCCNTCTLR1	See individual bit resets.	32-bit	Counter Control Register <n>
0x160	TRCCNTVR0	See individual bit resets.	32-bit	Counter Value Register <n>
0x164	TRCCNTVR1	See individual bit resets.	32-bit	Counter Value Register <n>
0x180	TRCIDR8	See individual bit resets.	32-bit	ID Register 8
0x184	TRCIDR9	See individual bit resets.	32-bit	ID Register 9
0x188	TRCIDR10	See individual bit resets.	32-bit	ID Register 10
0x18C	TRCIDR11	See individual bit resets.	32-bit	ID Register 11
0x190	TRCIDR12	See individual bit resets.	32-bit	ID Register 12
0x194	TRCIDR13	See individual bit resets.	32-bit	ID Register 13
0x1C0	TRCIMSPECO	See individual bit resets.	32-bit	IMP DEF Register 0
0x1E0	TRCIDR0	See individual bit resets.	32-bit	ID Register 0
0x1E4	TRCIDR1	See individual bit resets.	32-bit	ID Register 1
0x1E8	TRCIDR2	See individual bit resets.	32-bit	ID Register 2
0x1EC	TRCIDR3	See individual bit resets.	32-bit	ID Register 3
0x1F0	TRCIDR4	See individual bit resets.	32-bit	ID Register 4

Offset	Name	Reset	Width	Description
0x1F4	TRCIDR5	See individual bit resets.	32-bit	ID Register 5
0x1F8	TRCIDR6	See individual bit resets.	32-bit	ID Register 6
0x1FC	TRCIDR7	See individual bit resets.	32-bit	ID Register 7
0x208	TRCRSCTLR2	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x20C	TRCRSCTLR3	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x210	TRCRSCTLR4	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x214	TRCRSCTLR5	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x218	TRCRSCTLR6	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x21C	TRCRSCTLR7	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x220	TRCRSCTLR8	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x224	TRCRSCTLR9	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x228	TRCRSCTLR10	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x22C	TRCRSCTLR11	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x230	TRCRSCTLR12	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x234	TRCRSCTLR13	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x238	TRCRSCTLR14	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x23C	TRCRSCTLR15	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x280	TRCSSCCR0	See individual bit resets.	32-bit	Single-shot Comparator Control Register <n>
0x2A0	TRCSSCSR0	See individual bit resets.	32-bit	Single-shot Comparator Control Status Register <n>
0x304	TRCOSLSR	See individual bit resets.	32-bit	Trace OS Lock Status Register
0x310	TRCPDCR	See individual bit resets.	32-bit	PowerDown Control Register
0x314	TRCPDSR	See individual bit resets.	32-bit	PowerDown Status Register
0x400	TRCACVR0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x408	TRCACVR1	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x410	TRCACVR2	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x418	TRCACVR3	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x420	TRCACVR4	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x428	TRCACVR5	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x430	TRCACVR6	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x438	TRCACVR7	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x480	TRCACATR0	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x488	TRCACATR1	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x490	TRCACATR2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x498	TRCACATR3	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4A0	TRCACATR4	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4A8	TRCACATR5	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4B0	TRCACATR6	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4B8	TRCACATR7	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x600	TRCCIDCVR0	See individual bit resets.	64-bit	Context Identifier Comparator Value Registers <n>
0x640	TRCVMIDCVR0	See individual bit resets.	64-bit	Virtual Context Identifier Comparator Value Register <n>

Offset	Name	Reset	Width	Description
0x680	TRCCIDCCTLRO	See individual bit resets.	32-bit	Context Identifier Comparator Control Register 0
0x688	TRCVMIDCCTLRO	See individual bit resets.	32-bit	Virtual Context Identifier Comparator Control Register 0
0xF00	TRCITCTRL	See individual bit resets.	32-bit	Integration Mode Control Register
0xFA0	TRCCCLAIMSET	See individual bit resets.	32-bit	Claim Tag Set Register
0xFA4	TRCCCLAIMCLR	See individual bit resets.	32-bit	Claim Tag Clear Register
0xFA8	TRCDEVAFF	See individual bit resets.	64-bit	Device Affinity Register
0xFB0	TRCLAR	See individual bit resets.	32-bit	Lock Access Register
0xFB4	TRCLSR	See individual bit resets.	32-bit	Lock Status Register
0xFB8	TRCAUTHSTATUS	See individual bit resets.	32-bit	Authentication Status Register
0xFBC	TRCDEVARCH	See individual bit resets.	32-bit	Device Architecture Register
0xFC0	TRCDEVID2	See individual bit resets.	32-bit	Device Configuration Register 2
0xFC4	TRCDEVID1	See individual bit resets.	32-bit	Device Configuration Register 1
0xFC8	TRCDEVID	See individual bit resets.	32-bit	Device Configuration Register
0xFCC	TRCDEVTYPE	See individual bit resets.	32-bit	Device Type Register
0xFD0	TRCPIDR4	See individual bit resets.	32-bit	Peripheral Identification Register 4
0xFD4	TRCPIDR5	See individual bit resets.	32-bit	Peripheral Identification Register 5
0xFD8	TRCPIDR6	See individual bit resets.	32-bit	Peripheral Identification Register 6
0xFDC	TRCPIDR7	See individual bit resets.	32-bit	Peripheral Identification Register 7
0xFE0	TRCPIDR0	See individual bit resets.	32-bit	Peripheral Identification Register 0
0xFE4	TRCPIDR1	See individual bit resets.	32-bit	Peripheral Identification Register 1
0xFE8	TRCPIDR2	See individual bit resets.	32-bit	Peripheral Identification Register 2
0xFEC	TRCPIDR3	See individual bit resets.	32-bit	Peripheral Identification Register 3
0xFF0	TRCCIDR0	See individual bit resets.	32-bit	Component Identification Register 0
0xFF4	TRCCIDR1	See individual bit resets.	32-bit	Component Identification Register 1
0xFF8	TRCCIDR2	See individual bit resets.	32-bit	Component Identification Register 2
0xFFC	TRCCIDR3	See individual bit resets.	32-bit	Component Identification Register 3

20. Trace Buffer Extension support

The Neoverse™ N3 core implements the *TRace Buffer Extension* (TRBE). The TRBE writes the program flow trace generated by the trace unit directly to memory. The TRBE is programmed through System registers.

When enabled, the TRBE can:

- Accept trace data from the trace unit and write it to L2 memory.
- Discard trace data from the trace unit. In this case, the data is lost.
- Reject trace data from the trace unit. In this case, the trace unit retains data until the TRBE accepts it.

When disabled, the TRBE ignores trace data and the trace unit sends trace data to the AMBA® *Trace Bus* (ATB) interface.

20.1 Program and read the trace buffer registers

You can program and read the *TRace Buffer Extension* (TRBE) registers using the System register interface.

The core does not have to be in debug state when you program the TRBE registers. When you program the TRBE registers, you must enable all the changes at the same time. Otherwise, if you program the counter, it might start to count based on incorrect events before the correct setup is in place for the trigger condition. To disable the TRBE, use the TRBLIMITR_EL1.E bit.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for information on the TRBE register behaviors and access mechanisms.

20.2 Trace buffer register interface

The Neoverse™ N3 core supports a System register interface to *TRace Buffer Extension* (TRBE) registers.

Register accesses differ depending on the TRBE state. See the [Arm® Architecture Reference Manual for A-profile architecture](#) for information on the behaviors and access mechanisms.

21. Activity Monitors Extension support

The Neoverse™ N3 core implements the Activity Monitors Extension to the Arm®v8.4-A architecture. Activity monitoring has features similar to performance monitoring features, but it is intended for system management use whereas performance monitoring is aimed at user and debug applications.

The activity monitors provide useful information for system power management and persistent monitoring. The activity monitors are read-only in operation and their configuration is limited to the highest Exception level implemented.

The Neoverse™ N3 core implements seven counters in two groups, each of which is a 64-bit counter that counts a fixed event. Group 0 has four counters 0-3, and Group 1 has three counters 10-12.

21.1 Activity monitors access

The Neoverse™ N3 core supports access to activity monitors from the System register interface and supports read-only memory-mapped access using the utility bus interface.

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for information on the memory mapping of these registers.

Access enable bit

There are multiple access enable bits associated with the Activity Monitors System registers:

- AMUSERENR_ELO.EN controls access from EL0 to the Activity Monitors System registers
- CPTR_EL2.TAM controls access from EL0 and EL1 to the Activity Monitors System registers
- CPTR_EL3.TAM controls access from EL0, EL1, and EL2 to the Activity Monitors Extension System registers



AMUSERENR_ELO.EN is configurable at EL1, EL2, and EL3. All other controls, as well as the value of the counters, are configurable only at the highest implemented Exception level.

For a detailed description of access controls for the registers, see the [Arm® Architecture Reference Manual for A-profile architecture](#).

System register access

The activity monitors are accessible using the `MRS` and `MSR` instructions.

External memory-mapped access

Activity monitors can be memory-mapped accessed from the utility bus interface. In this case, the Activity Monitors registers only provide read access to the Activity Monitor Event Counter registers.

The base address for *Activity Monitoring Unit* (AMU) registers on the utility bus interface is $0x\langle n \rangle 90000$, where n is the Neoverse™ N3 core instance number in the DSU-120 cluster.

These registers are treated as RAZ/WI if either:

- The register is marked as Reserved.
- The register is accessed in the wrong Security state.

21.2 Activity monitors counters

The Neoverse™ N3 core implements four activity monitors counters, 0-3, and three auxiliary counters, 10-12 that map to specific *Activity Monitoring Unit* (AMU) events.

Each counter has the following characteristics:

- All events are counted in 64-bit wrapping counters that overflow when they wrap. There is no support for overflow status indication or interrupts.
- Any change in clock frequency can affect any counter. For example, when a *WFI*, *WFE*, *WFI*, or *WFE* instruction stops the clock.
- Events 0-3 and auxiliary events 10-12 are fixed, and the *AMEVTYPER0* $\langle n \rangle$ _ELO and *AMEVTYPER1* $\langle n \rangle$ _ELO evtCount bits are read-only.
- The activity monitor counters are reset to zero on a Cold reset of the power domain of the core. When the core is not in reset, activity monitoring is available.

21.3 Activity monitors events

Activity monitors events in the Neoverse™ N3 core are either fixed or programmable, and they map to the activity monitors counters.

The following table shows the mapping of counters to fixed events.

Table 21-1: Mapping of counters to fixed events

Activity monitor counter $\langle n \rangle$	Event	Event number	Description
AMEVCNTR00	CPU_CYCLES	0x0011	Core frequency cycles
AMEVCNTR01	CNT_CYCLES	0x4004	Constant frequency cycles

Activity monitor counter <n>	Event	Event number	Description
AMEVCNTR02	INSTR_RETIRED	0x0008	Instruction architecturally executed This counter increments for every instruction that is executed architecturally, including instructions that fail their condition code check.
AMEVCNTR03	STALL_BACKEND_MEM	0x4005	Memory stall cycles This counter counts cycles in which the core is unable to dispatch instructions from the front end to the back end due to a back end stall caused by a miss in the last level of cache within the core clock domain.
AMEVCNTR10	MPMM_THRESHOLD_GEAR0	0x0300	<i>Maximum Power Mitigation System (MPMM) Gear 0 activity period threshold exceeded</i>
AMEVCNTR11	MPMM_THRESHOLD_GEAR1	0x0301	<i>Maximum Power Mitigation System (MPMM) Gear 1 activity period threshold exceeded</i>
AMEVCNTR12	MPMM_THRESHOLD_GEAR2	0x0302	<i>Maximum Power Mitigation System (MPMM) Gear 2 activity period threshold exceeded</i>

Related information

[4.5.1 Maximum Power Mitigation Mechanism](#) on page 45

21.4 AArch64 Activity Monitors registers

The summary table provides an overview of all Activity Monitors registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 21-2: Activity Monitors registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AMCR_ELO	3	3	C13	C2	0	See individual bit resets.	64-bit	Activity Monitors Control Register
AMCFGR_ELO	3	3	C13	C2	1	See individual bit resets.	64-bit	Activity Monitors Configuration Register
AMCGCR_ELO	3	3	C13	C2	2	See individual bit resets.	64-bit	Activity Monitors Counter Group Configuration Register
AMUSERENR_ELO	3	3	C13	C2	3	See individual bit resets.	64-bit	Activity Monitors User Enable Register
AMCNTENCLR0_ELO	3	3	C13	C2	4	See individual bit resets.	64-bit	Activity Monitors Count Enable Clear Register 0
AMCNTENSET0_ELO	3	3	C13	C2	5	See individual bit resets.	64-bit	Activity Monitors Count Enable Set Register 0

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AMCNTENCLR1_ELO	3	3	C13	C3	0	See individual bit resets.	64-bit	Activity Monitors Count Enable Clear Register 1
AMCNTENSET1_ELO	3	3	C13	C3	1	See individual bit resets.	64-bit	Activity Monitors Count Enable Set Register 1
AMEVCNTR00_ELO	3	3	C13	C4	0	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVCNTR01_ELO	3	3	C13	C4	1	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVCNTR02_ELO	3	3	C13	C4	2	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVCNTR03_ELO	3	3	C13	C4	3	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVTYPER00_ELO	3	3	C13	C6	0	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVTYPER01_ELO	3	3	C13	C6	1	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVTYPER02_ELO	3	3	C13	C6	2	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVTYPER03_ELO	3	3	C13	C6	3	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVCNTR10_ELO	3	3	C13	C12	0	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 1
AMEVCNTR11_ELO	3	3	C13	C12	1	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 1
AMEVCNTR12_ELO	3	3	C13	C12	2	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 1
AMEVTYPER10_ELO	3	3	C13	C14	0	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 1
AMEVTYPER11_ELO	3	3	C13	C14	1	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 1
AMEVTYPER12_ELO	3	3	C13	C14	2	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 1

21.5 External AMU registers

The summary table provides an overview of all memory-mapped AMU registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table 21-3: AMU registers summary

Offset	Name	Reset	Width	Description
0x0	AMEVCNTR00 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x4	AMEVCNTR00 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x8	AMEVCNTR01 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0xC	AMEVCNTR01 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x10	AMEVCNTR02 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x14	AMEVCNTR02 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x18	AMEVCNTR03 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x1C	AMEVCNTR03 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x100	AMEVCNTR10 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x104	AMEVCNTR10 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x108	AMEVCNTR11 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x10C	AMEVCNTR11 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x110	AMEVCNTR12 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x114	AMEVCNTR12 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x400	AMEVTYPER00	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x404	AMEVTYPER01	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x408	AMEVTYPER02	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x40C	AMEVTYPER03	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x480	AMEVTYPER10	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 1
0x484	AMEVTYPER11	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 1
0x488	AMEVTYPER12	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 1
0xC00	AMCNTENSET0	See individual bit resets.	32-bit	Activity Monitors Count Enable Set Register 0
0xC04	AMCNTENSET1	See individual bit resets.	32-bit	Activity Monitors Count Enable Set Register 1
0xC20	AMCNTENCLR0	See individual bit resets.	32-bit	Activity Monitors Count Enable Clear Register 0
0xC24	AMCNTENCLR1	See individual bit resets.	32-bit	Activity Monitors Count Enable Clear Register 1
0xCE0	AMCGCR	See individual bit resets.	32-bit	Activity Monitors Counter Group Configuration Register
0xE00	AMCFGR	See individual bit resets.	32-bit	Activity Monitors Configuration Register
0xE04	AMCR	See individual bit resets.	32-bit	Activity Monitors Control Register
0xE08	AMIIDR	See individual bit resets.	32-bit	Activity Monitors Implementation Identification Register
0xFA8	AMDEVAFF0	See individual bit resets.	32-bit	Activity Monitors Device Affinity Register 0
0xFAC	AMDEVAFF1	See individual bit resets.	32-bit	Activity Monitors Device Affinity Register 1
0xFBC	AMDEVARCH	See individual bit resets.	32-bit	Activity Monitors Device Architecture Register
0xFCC	AMDEVTYPE	See individual bit resets.	32-bit	Activity Monitors Device Type Register
0xFD0	AMPIDR4	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 4
0xFE0	AMPIDR0	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 0
0xFE4	AMPIDR1	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 1
0xFE8	AMPIDR2	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 2
0xFEC	AMPIDR3	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 3
0xFF0	AMCIDR0	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 0

Offset	Name	Reset	Width	Description
0xFF4	AMCIDR1	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 1
0xFF8	AMCIDR2	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 2
0xFFC	AMCIDR3	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 3

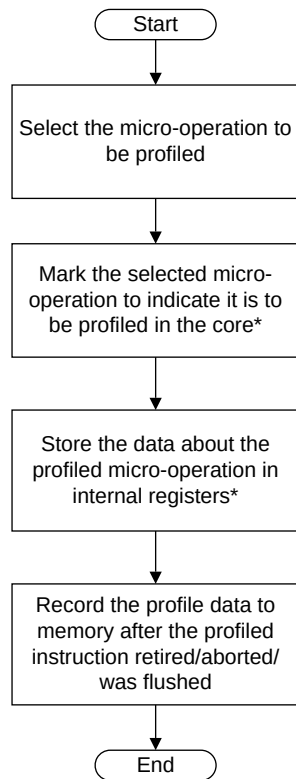
22. Statistical Profiling Extension support

The Neoverse™ N3 core implements the *Statistical Profiling Extension* (SPE) to the Arm®v8.7-A architecture. The SPE provides a statistical view of the performance characteristics of executed instructions that software writers can use to optimize their code for better performance.

The Neoverse™ N3 core profiles micro-operations to minimize the amount of logic necessary to support the SPE.

The following figure shows the SPE behavior in the Neoverse™ N3 core.

Figure 22-1: SPE behavior



* Throughout the lifetime of the micro-operation in the core

Profiles are collected periodically and a down-counter drives the selection of the micro-operations to be profiled. This counter counts the number of speculative micro-operations that are dispatched, decremented once for each micro-operation. When the counter reaches zero, a micro-operation is identified as being sampled and is profiled throughout its lifetime in the core.

SPE profiles are written to memory using a *Virtual Address* (VA), which means that writes of profiles must have access to the *Memory Management Unit* (MMU) to translate a VA to a *Physical Address* (PA), and must have a means to be written to memory.



Profiling is expected to be largely non-intrusive to the performance of the core. The performance of the core is not meaningfully perturbed while profiling is taking place. The rate of occurrence depends on the sampling rate. You can specify a sampling rate that is meaningfully intrusive to the performance of the core. Arm recommends that the minimum sampling interval is once per 1024 micro-operations. This value is communicated to software through PMSIDR_EL1.Interval, bits[11:8].

See the [Arm® Architecture Reference Manual for A-profile architecture](#) for more information.

22.1 Statistical Profiling Extension events packet

The events packet indicates the **IMPLEMENTATION DEFINED** events that the sampled operation generated.

The following table shows the events defined in the 32-bit events packet implemented in the Neoverse™ N3 core.

Table 22-1: SPE events packet

Bits	Definition
[31:19]	Reserved
[18]	Empty predicate
[17]	Partial predicate
[16:13]	Reserved
[12]	Late prefetch
[11]	Data alignment flag
[10]	Remote access
[9]	Last level cache miss
[8]	Last level cache access
[7]	Branch mispredicted
[6]	Not taken
[5]	L1 data cache <i>Translation Lookaside Buffer</i> (TLB)
[4]	TLB access
[3]	L1 data cache refill
[2]	L1 data cache access
[1]	Architecturally retired
[0]	Generated exception

22.2 Statistical Profiling Extension data source packet

The data source packet indicates where the data returned for a load or store operation was sourced.

The following table shows the data source defined in the 8-bit data source packet implemented in the Neoverse™ N3 core.

Table 22-2: SPE data source packet

Value	Name
0b0000	L1 data cache
0b1000	L2 cache
0b1001	Peer core
0b1010	Local cluster
0b1011	System cache
0b1100	Peer cluster
0b1101	Remote
0b1110	<i>Dynamic Random Access Memory (DRAM)</i>

Appendix A AArch64 registers

This appendix contains the descriptions for the Neoverse™ N3 AArch64 registers.

This manual does not provide a complete list of registers. Read this manual together with the [Arm® Architecture Reference Manual for A-profile architecture](#).

A.1 AArch64 Generic System Control registers summary

The summary table provides an overview of all Generic System Control registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-1: Generic System Control registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ACTLR_EL1	3	0	C1	C0	1	See individual bit resets.	64-bit	Auxiliary Control Register (EL1)
RGSR_EL1	3	0	C1	C0	5	See individual bit resets.	64-bit	Random Allocation Tag Seed Register.
GCR_EL1	3	0	C1	C0	6	See individual bit resets.	64-bit	Tag Control Register.
TTBR0_EL1	3	0	C2	C0	0	See individual bit resets.	64-bit	Translation Table Base Register 0 (EL1)
TTBR1_EL1	3	0	C2	C0	1	See individual bit resets.	64-bit	Translation Table Base Register 1 (EL1)
TCR_EL1	3	0	C2	C0	2	See individual bit resets.	64-bit	Translation Control Register (EL1)
APIAKeyLo_EL1	3	0	C2	C1	0	See individual bit resets.	64-bit	Pointer Authentication Key A for Instruction (bits[63:0])
APIAKeyHi_EL1	3	0	C2	C1	1	See individual bit resets.	64-bit	Pointer Authentication Key A for Instruction (bits[127:64])
APIBKeyLo_EL1	3	0	C2	C1	2	See individual bit resets.	64-bit	Pointer Authentication Key B for Instruction (bits[63:0])
APIBKeyHi_EL1	3	0	C2	C1	3	See individual bit resets.	64-bit	Pointer Authentication Key B for Instruction (bits[127:64])
APDAKeyLo_EL1	3	0	C2	C2	0	See individual bit resets.	64-bit	Pointer Authentication Key A for Data (bits[63:0])
APDAKeyHi_EL1	3	0	C2	C2	1	See individual bit resets.	64-bit	Pointer Authentication Key A for Data (bits[127:64])
APDBKeyLo_EL1	3	0	C2	C2	2	See individual bit resets.	64-bit	Pointer Authentication Key B for Data (bits[63:0])

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
APDBKeyHi_EL1	3	0	C2	C2	3	See individual bit resets.	64-bit	Pointer Authentication Key B for Data (bits[127:64])
APGAKeyLo_EL1	3	0	C2	C3	0	See individual bit resets.	64-bit	Pointer Authentication Key A for Code (bits[63:0])
APGAKeyHi_EL1	3	0	C2	C3	1	See individual bit resets.	64-bit	Pointer Authentication Key A for Code (bits[127:64])
SPSel	3	0	C4	C2	0	See individual bit resets.	64-bit	Stack Pointer Select
CurrentEL	3	0	C4	C2	2	See individual bit resets.	64-bit	Current Exception Level
PAN	3	0	C4	C2	3	See individual bit resets.	64-bit	Privileged Access Never
UAO	3	0	C4	C2	4	See individual bit resets.	64-bit	User Access Override
AFSR0_EL1	3	0	C5	C1	0	See individual bit resets.	64-bit	Auxiliary Fault Status Register 0 (EL1)
AFSR1_EL1	3	0	C5	C1	1	See individual bit resets.	64-bit	Auxiliary Fault Status Register 1 (EL1)
ESR_EL1	3	0	C5	C2	0	See individual bit resets.	64-bit	Exception Syndrome Register (EL1)
TFSR_EL1	3	0	C5	C6	0	See individual bit resets.	64-bit	Tag Fault Status Register (EL1)
TFSREO_EL1	3	0	C5	C6	1	See individual bit resets.	64-bit	Tag Fault Status Register (EL0).
FAR_EL1	3	0	C6	C0	0	See individual bit resets.	64-bit	Fault Address Register (EL1)
PAR_EL1	3	0	C7	C4	0	See individual bit resets.	64-bit	Physical Address Register
MAIR_EL1	3	0	C10	C2	0	See individual bit resets.	64-bit	Memory Attribute Indirection Register (EL1)
AMAIR_EL1	3	0	C10	C3	0	See individual bit resets.	64-bit	Auxiliary Memory Attribute Indirection Register (EL1)
LORSA_EL1	3	0	C10	C4	0	See individual bit resets.	64-bit	LORegion Start Address (EL1)
LOREA_EL1	3	0	C10	C4	1	See individual bit resets.	64-bit	LORegion End Address (EL1)
LORN_EL1	3	0	C10	C4	2	See individual bit resets.	64-bit	LORegion Number (EL1)
LORC_EL1	3	0	C10	C4	3	See individual bit resets.	64-bit	LORegion Control (EL1)
LORID_EL1	3	0	C10	C4	7	See individual bit resets.	64-bit	LORegionID (EL1)
VBAR_EL1	3	0	C12	C0	0	See individual bit resets.	64-bit	Vector Base Address Register (EL1)
ISR_EL1	3	0	C12	C1	0	See individual bit resets.	64-bit	Interrupt Status Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
CONTEXTIDR_EL1	3	0	C13	C0	1	See individual bit resets.	64-bit	Context ID Register (EL1)
TPIDR_EL1	3	0	C13	C0	4	See individual bit resets.	64-bit	EL1 Software Thread ID Register
SCXTNUM_EL1	3	0	C13	C0	7	See individual bit resets.	64-bit	EL1 Read/Write Software Context Number
IMP_CPUCFR_EL1	3	0	C15	C0	0	See individual bit resets.	64-bit	CPU Configuration Register
IMP_CPUACTLR_EL1	3	0	C15	C1	0	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR2_EL1	3	0	C15	C1	1	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR3_EL1	3	0	C15	C1	2	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR4_EL1	3	0	C15	C1	3	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR_EL1	3	0	C15	C1	4	See individual bit resets.	64-bit	CPU Extended Control Register
IMP_CPUACTLR2_EL1	3	0	C15	C1	5	See individual bit resets.	64-bit	CPU Extended Control Register
IMP_CPUBUSQOS_EL1	3	0	C15	C1	7	See individual bit resets.	64-bit	Quality of Service Bus Control Register, EL1
IMP_CUPWRCTLR_EL1	3	0	C15	C2	7	See individual bit resets.	64-bit	CPU Power Control Register
IMP_CLUSTERCFR_EL1	3	0	C15	C3	0	See individual bit resets.	64-bit	Cluster Configuration Register
IMP_CLUSTERIDR_EL1	3	0	C15	C3	1	See individual bit resets.	64-bit	Cluster Main Revision Register
IMP_CLUSTERREVIDR_EL1	3	0	C15	C3	2	See individual bit resets.	64-bit	Cluster ECO ID Register
IMP_CLUSTERACTLR_EL1	3	0	C15	C3	3	See individual bit resets.	64-bit	Cluster Auxiliary Control Register
IMP_CLUSTERECTLR_EL1	3	0	C15	C3	4	See individual bit resets.	64-bit	Cluster Extended Control Register
IMP_CLUSTERPWRCTLR_EL1	3	0	C15	C3	5	See individual bit resets.	64-bit	Cluster Power Control Register
IMP_CLUSTERPWRDN_EL1	3	0	C15	C3	6	See individual bit resets.	64-bit	Cluster Power Down Register
IMP_CLUSTERPWRSTAT_EL1	3	0	C15	C3	7	See individual bit resets.	64-bit	Cluster Power Status Register
IMP_CLUSTERL3DNTH0_EL1	3	0	C15	C4	0	See individual bit resets.	64-bit	Cluster L3 Downsize Threshold0 Register
IMP_CLUSTERL3DNTH1_EL1	3	0	C15	C4	1	See individual bit resets.	64-bit	Cluster L3 Downsize Threshold1 Register
IMP_CLUSTERL3UPTH0_EL1	3	0	C15	C4	2	See individual bit resets.	64-bit	Cluster L3 Upsize Threshold0 Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CLUSTERL3UPTH1_EL1	3	0	C15	C4	3	See individual bit resets.	64-bit	Cluster L3 Upsize Threshold1 Register
IMP_CLUSTERBUSQOS_EL1	3	0	C15	C4	4	See individual bit resets.	64-bit	Cluster Bus QoS Control Register
IMP_CLUSTERL3HIT_EL1	3	0	C15	C4	5	See individual bit resets.	64-bit	Cluster L3 Hit Counter Register
IMP_CLUSTERL3MISS_EL1	3	0	C15	C4	6	See individual bit resets.	64-bit	Cluster L3 Miss Counter Register
IMP_ATCR_EL1	3	0	C15	C7	0	See individual bit resets.	64-bit	CPU Auxiliary Translation Control Register
IMP_CPUACTLR5_EL1	3	0	C15	C8	0	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR6_EL1	3	0	C15	C8	1	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR7_EL1	3	0	C15	C8	2	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CPUACTLR8_EL1	3	0	C15	C8	3	See individual bit resets.	64-bit	CPU Auxiliary Control Register
IMP_CLUSTERPPSTART_EL1	3	0	C15	C9	0	See individual bit resets.	64-bit	Cluster Peripheral Port Start Address Register
IMP_CLUSTERPPEND_EL1	3	0	C15	C9	1	See individual bit resets.	64-bit	Cluster Peripheral Port End Address Register
IMP_CLUSTERRCFR2_EL1	3	0	C15	C9	2	See individual bit resets.	64-bit	Cluster Configuration Register 2
IMP_CLUSTERL3UPTH2_EL1	3	0	C15	C9	3	See individual bit resets.	64-bit	Cluster L3 Upsize Threshold2 Register
IMP_CLUSTERRSVD_9_4_EL1	3	0	C15	C9	4	See individual bit resets.	64-bit	RESERVED
IMP_CLUSTERRSVD_9_5_EL1	3	0	C15	C9	5	See individual bit resets.	64-bit	RESERVED
IMP_CLUSTERRSVD_9_6_EL1	3	0	C15	C9	6	See individual bit resets.	64-bit	RESERVED
IMP_CLUSTERRSVD_9_7_EL1	3	0	C15	C9	7	See individual bit resets.	64-bit	RESERVED
AIDR_EL1	3	1	C0	C0	7	See individual bit resets.	64-bit	Auxiliary ID Register
RNDR	3	3	C2	C4	0	See individual bit resets.	64-bit	Random Number
RNDRRS	3	3	C2	C4	1	See individual bit resets.	64-bit	Reseeded Random Number
NZCV	3	3	C4	C2	0	See individual bit resets.	64-bit	Condition Flags
DAIF	3	3	C4	C2	1	See individual bit resets.	64-bit	Interrupt Mask Bits
DIT	3	3	C4	C2	5	See individual bit resets.	64-bit	Data Independent Timing

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
SSBS	3	3	C4	C2	6	See individual bit resets.	64-bit	Speculative Store Bypass Safe
TCO	3	3	C4	C2	7	See individual bit resets.	64-bit	Tag Check Override
FPCR	3	3	C4	C4	0	See individual bit resets.	64-bit	Floating-point Control Register
FPSR	3	3	C4	C4	1	See individual bit resets.	64-bit	Floating-point Status Register
TPIDR_ELO	3	3	C13	C0	2	See individual bit resets.	64-bit	ELO Read/Write Software Thread ID Register
TPIDRRO_ELO	3	3	C13	C0	3	See individual bit resets.	64-bit	ELO Read-Only Software Thread ID Register
SCXTNUM_ELO	3	3	C13	C0	7	See individual bit resets.	64-bit	ELO Read/Write Software Context Number
ACTLR_EL2	3	4	C1	C0	1	See individual bit resets.	64-bit	Auxiliary Control Register (EL2)
HACR_EL2	3	4	C1	C1	7	See individual bit resets.	64-bit	Hypervisor Auxiliary Control Register
TTBR0_EL2	3	4	C2	C0	0	See individual bit resets.	64-bit	Translation Table Base Register 0 (EL2)
TTBR1_EL2	3	4	C2	C0	1	See individual bit resets.	64-bit	Translation Table Base Register 1 (EL2)
TCR_EL2	3	4	C2	C0	2	See individual bit resets.	64-bit	Translation Control Register (EL2)
VTTBR_EL2	3	4	C2	C1	0	See individual bit resets.	64-bit	Virtualization Translation Table Base Register
VTCR_EL2	3	4	C2	C1	2	See individual bit resets.	64-bit	Virtualization Translation Control Register
VSTTBR_EL2	3	4	C2	C6	0	See individual bit resets.	64-bit	Virtualization Secure Translation Table Base Register
VSTCR_EL2	3	4	C2	C6	2	See individual bit resets.	64-bit	Virtualization Secure Translation Control Register
AFSR0_EL2	3	4	C5	C1	0	See individual bit resets.	64-bit	Auxiliary Fault Status Register 0 (EL2)
AFSR1_EL2	3	4	C5	C1	1	See individual bit resets.	64-bit	Auxiliary Fault Status Register 1 (EL2)
ESR_EL2	3	4	C5	C2	0	See individual bit resets.	64-bit	Exception Syndrome Register (EL2)
TFSR_EL2	3	4	C5	C6	0	See individual bit resets.	64-bit	Tag Fault Status Register (EL2)
FAR_EL2	3	4	C6	C0	0	See individual bit resets.	64-bit	Fault Address Register (EL2)
HPFAR_EL2	3	4	C6	C0	4	See individual bit resets.	64-bit	Hypervisor IPA Fault Address Register
MAIR_EL2	3	4	C10	C2	0	See individual bit resets.	64-bit	Memory Attribute Indirection Register (EL2)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AMAIR_EL2	3	4	C10	C3	0	See individual bit resets.	64-bit	Auxiliary Memory Attribute Indirection Register (EL2)
VBAR_EL2	3	4	C12	C0	0	See individual bit resets.	64-bit	Vector Base Address Register (EL2)
CONTEXTIDR_EL2	3	4	C13	C0	1	See individual bit resets.	64-bit	Context ID Register (EL2)
TPIDR_EL2	3	4	C13	C0	2	See individual bit resets.	64-bit	EL2 Software Thread ID Register
SCXTNUM_EL2	3	4	C13	C0	7	See individual bit resets.	64-bit	EL2 Read/Write Software Context Number
IMP_ATCR_EL2	3	4	C15	C7	0	See individual bit resets.	64-bit	CPU Auxiliary Translation Control Register
IMP_AVTCR_EL2	3	4	C15	C7	1	See individual bit resets.	64-bit	CPU Auxiliary Virtualization Translation Control Register
ACTLR_EL3	3	6	C1	C0	1	See individual bit resets.	64-bit	Auxiliary Control Register (EL3)
SCR_EL3	3	6	C1	C1	0	See individual bit resets.	64-bit	Secure Configuration Register
CPTR_EL3	3	6	C1	C1	2	See individual bit resets.	64-bit	Architectural Feature Trap Register (EL3)
MDCR_EL3	3	6	C1	C3	1	See individual bit resets.	64-bit	Monitor Debug Configuration Register (EL3)
TTBR0_EL3	3	6	C2	C0	0	See individual bit resets.	64-bit	Translation Table Base Register 0 (EL3)
TCR_EL3	3	6	C2	C0	2	See individual bit resets.	64-bit	Translation Control Register (EL3)
AFSR0_EL3	3	6	C5	C1	0	See individual bit resets.	64-bit	Auxiliary Fault Status Register 0 (EL3)
AFSR1_EL3	3	6	C5	C1	1	See individual bit resets.	64-bit	Auxiliary Fault Status Register 1 (EL3)
ESR_EL3	3	6	C5	C2	0	See individual bit resets.	64-bit	Exception Syndrome Register (EL3)
TFSR_EL3	3	6	C5	C6	0	See individual bit resets.	64-bit	Tag Fault Status Register (EL3)
FAR_EL3	3	6	C6	C0	0	See individual bit resets.	64-bit	Fault Address Register (EL3)
MAIR_EL3	3	6	C10	C2	0	See individual bit resets.	64-bit	Memory Attribute Indirection Register (EL3)
AMAIR_EL3	3	6	C10	C3	0	See individual bit resets.	64-bit	Auxiliary Memory Attribute Indirection Register (EL3)
VBAR_EL3	3	6	C12	C0	0	See individual bit resets.	64-bit	Vector Base Address Register (EL3)
RVBAR_EL3	3	6	C12	C0	1	See individual bit resets.	64-bit	Reset Vector Base Address Register (if EL3 implemented)
RMR_EL3	3	6	C12	C0	2	See individual bit resets.	64-bit	Reset Management Register (EL3)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TPIDR_EL3	3	6	C13	C0	2	See individual bit resets.	64-bit	EL3 Software Thread ID Register
SCXTNUM_EL3	3	6	C13	C0	7	See individual bit resets.	64-bit	EL3 Read/Write Software Context Number
IMP_ISIDE_DATA0_EL3	3	6	C15	C0	0	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 0
IMP_ISIDE_DATA1_EL3	3	6	C15	C0	1	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 1
IMP_ISIDE_DATA2_EL3	3	6	C15	C0	2	See individual bit resets.	64-bit	RAMINDEX Instruction Data register 2
IMP_MMU_DATA0_EL3	3	6	C15	C0	3	See individual bit resets.	64-bit	RAMINDEX TLB Data register 0
IMP_MMU_DATA1_EL3	3	6	C15	C0	4	See individual bit resets.	64-bit	RAMINDEX TLB Data register 1
IMP_MMU_DATA2_EL3	3	6	C15	C0	5	See individual bit resets.	64-bit	RAMINDEX TLB Data register 2
IMP_DSIDE_DATA0_EL3	3	6	C15	C1	0	See individual bit resets.	64-bit	RAMINDEX L1D Data register 0
IMP_DSIDE_DATA1_EL3	3	6	C15	C1	1	See individual bit resets.	64-bit	RAMINDEX L1D Data register 1
IMP_DSIDE_DATA2_EL3	3	6	C15	C1	2	See individual bit resets.	64-bit	RAMINDEX L1D Data register 2
IMP_L2_DATA0_EL3	3	6	C15	C1	3	See individual bit resets.	64-bit	RAMINDEX L2 Data register 0
IMP_L2_DATA2_EL3	3	6	C15	C1	4	See individual bit resets.	64-bit	RAMINDEX L2 Data register 2
IMP_L2_DATA1_EL3	3	6	C15	C1	5	See individual bit resets.	64-bit	RAMINDEX L2 Data register 1
IMP_CPURNDBR_EL3	3	6	C15	C3	0	See individual bit resets.	64-bit	Random Number Base Register, EL3
IMP_CPURNDPEID_EL3	3	6	C15	C3	1	See individual bit resets.	64-bit	Random Number Packet Identification Register, EL3
IMP_CLUSTERCDBG_EL3	3	6	C15	C4	7	See individual bit resets.	64-bit	Cluster Cache Debug Register
IMP_CLUSTERPMMDCR_EL3	3	6	C15	C6	3	See individual bit resets.	64-bit	Monitor Debug Configuration Register (EL3)
IMP_ATCR_EL3	3	6	C15	C7	0	See individual bit resets.	64-bit	CPU Auxiliary Translation Control Register
IMP_CPUPSELR_EL3	3	6	C15	C8	0	See individual bit resets.	64-bit	Selected Instruction Private Control Register
IMP_CPUPCR_EL3	3	6	C15	C8	1	See individual bit resets.	64-bit	Selected Instruction Patch Control Register
IMP_CPUPOR_EL3	3	6	C15	C8	2	See individual bit resets.	64-bit	Selected Instruction Patch Opcode Register
IMP_CPUPMR_EL3	3	6	C15	C8	3	See individual bit resets.	64-bit	Selected Instruction Patch Mask Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CPUPOR2_EL3	3	6	C15	C8	4	See individual bit resets.	64-bit	Selected Instruction Patch Opcode Register 2
IMP_CPUPMR2_EL3	3	6	C15	C8	5	See individual bit resets.	64-bit	Selected Instruction Patch Mask Register 2
IMP_CPUPFR_EL3	3	6	C15	C8	6	See individual bit resets.	64-bit	Selected Instruction Private Flag Register

A.1.1 ACTLR_EL1, Auxiliary Control Register (EL1)

Provides **IMPLEMENTATION DEFINED** configuration and control options for execution at EL1 and EL0.



Note

Arm recommends the contents of this register have no effect on the PE when AArch64-HCR_EL2.{E2H, TGE} is {1, 1}, and instead the configuration and control fields are provided by the AArch64-ACTLR_EL2 register. This avoids the need for software to manage the contents of these register when switching between a Guest OS and a Host OS.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-1: AArch64_actlr_el1 bit assignments

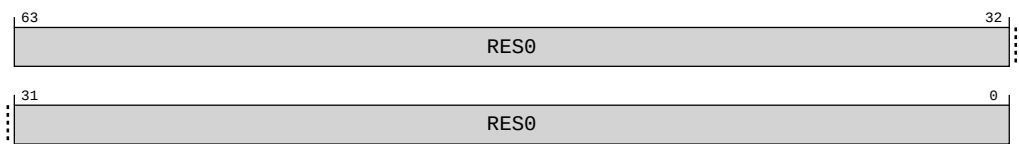


Table A-2: ACTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, ACTLR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0001	0b0000	0b001

MSR ACTLR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0001	0b0000	0b001

Accessibility

MRS <Xt>, ACTLR_EL1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TACR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ACTLR_EL1;
elseif PSTATE.EL == EL2 then
    return ACTLR_EL1;
elseif PSTATE.EL == EL3 then
    return ACTLR_EL1;
```

MSR ACTLR_EL1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TACR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        ACTLR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    ACTLR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
```



```
ACTLR_EL1 = X[t];
```

A.1.2 AFSR0_EL1, Auxiliary Fault Status Register 0 (EL1)

Provides additional **IMPLEMENTATION DEFINED** fault status information for exceptions taken to EL1.

Configurations

This register is available in all configurations.

Attributes

Width

64

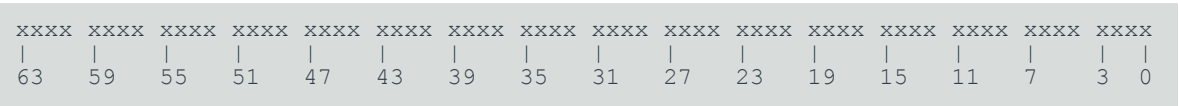
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-2: AArch64_afsr0_el1 bit assignments

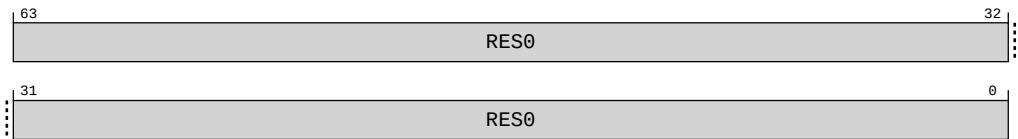


Table A-5: AFSR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic AFSRO_EL1 or AFSRO_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSRO_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b000

MSR AFSRO_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b000

MRS <Xt>, AFSRO_EL12

op0	op1	CRn	CRm	op2
0b11	0b101	0b0101	0b0001	0b000

MSR AFSRO_EL12, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b101	0b0101	0b0001	0b000

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic AFSRO_EL1 or AFSRO_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSRO_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGRTR_EL2.AFSRO_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return AFSRO_EL1;
elsif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AFSRO_EL2;
    else
        return AFSRO_EL1;
elsif PSTATE.EL == EL3 then
    return AFSRO_EL1;

```

MSR AFSRO_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;

```

```

elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.AFSR0_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AFSR0_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AFSR0_EL2 = X[t];
    else
        AFSR0_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    AFSR0_EL1 = X[t];

```

MRS <Xt>, AFSR0_EL12

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AFSR0_EL1;
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        return AFSR0_EL1;
    else
        UNDEFINED;

```

MSR AFSR0_EL12, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AFSR0_EL1 = X[t];
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        AFSR0_EL1 = X[t];
    else
        UNDEFINED;

```

A.1.3 AFSR1_EL1, Auxiliary Fault Status Register 1 (EL1)

Provides additional **IMPLEMENTATION DEFINED** fault status information for exceptions taken to EL1.

Configurations

This register is available in all configurations.

Attributes

Width

64

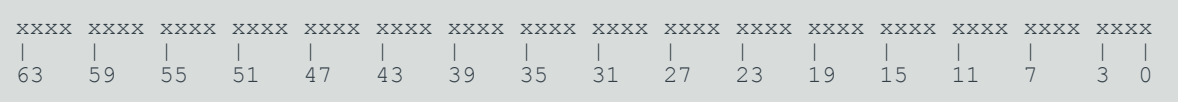
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-3: AArch64_afsr1_el1 bit assignments

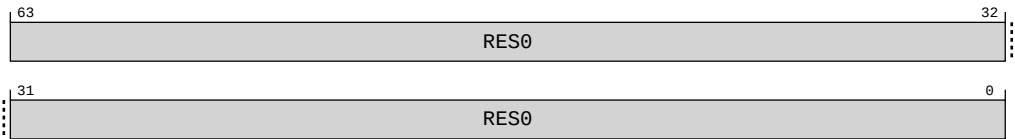


Table A-10: AFSR1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic AFSR1_EL1 or AFSR1_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b001

MSR AFSR1_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b001

MRS <Xt>, AFSR1_EL12

op0	op1	CRn	CRm	op2
0b11	0b101	0b0101	0b0001	0b001

MSR AFSR1_EL12, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b101	0b0101	0b0001	0b001

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic AFSR1_EL1 or AFSR1_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSR1_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.AFSR1_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return AFSR1_EL1;
elsif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AFSR1_EL2;
    else
        return AFSR1_EL1;
elsif PSTATE.EL == EL3 then
    return AFSR1_EL1;

```

MSR AFSR1_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.AFSR1_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AFSR1_EL1 = X[t];
elsif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AFSR1_EL2 = X[t];
    else
        AFSR1_EL1 = X[t];
elsif PSTATE.EL == EL3 then
        AFSR1_EL1 = X[t];

```

MRS <Xt>, AFSR1_EL12

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AFSR1_EL1;
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        return AFSR1_EL1;
    else
        UNDEFINED;
```

MSR AFSR1_EL12, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AFSR1_EL1 = X[t];
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        AFSR1_EL1 = X[t];
    else
        UNDEFINED;
```

A.1.4 AMAIR_EL1, Auxiliary Memory Attribute Indirection Register (EL1)

Provides **IMPLEMENTATION DEFINED** memory attributes for the memory regions specified by AArch64-MAIR_EL1.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

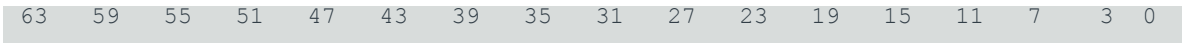
Generic System Control

Access type

See bit descriptions

Reset value





Where the reset reads xxxx, see individual bits.

Bit descriptions

AMAIR_EL1 is permitted to be cached in a TLB.

Figure A-4: AArch64_amair_el1 bit assignments

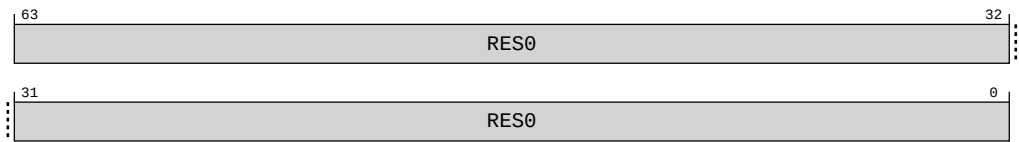


Table A-15: AMAIR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic AMAIR_EL1 or AMAIR_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AMAIR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1010	0b0011	0b000

MSR AMAIR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1010	0b0011	0b000

MRS <Xt>, AMAIR_EL12

op0	op1	CRn	CRm	op2
0b11	0b101	0b1010	0b0011	0b000

MSR AMAIR_EL12, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b101	0b1010	0b0011	0b000

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic AMAIR_EL1 or AMAIR_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AMAIR_EL1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.AMAIR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return AMAIR_EL1;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AMAIR_EL2;
    else
        return AMAIR_EL1;
elseif PSTATE.EL == EL3 then
    return AMAIR_EL1;
```

MSR AMAIR_EL1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.AMAIR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AMAIR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AMAIR_EL2 = X[t];
    else
        AMAIR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    AMAIR_EL1 = X[t];
```

MRS <Xt>, AMAIR_EL12

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AMAIR_EL1;
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        return AMAIR_EL1;
    else
        UNDEFINED;
```


MSR AMAIR_EL12, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AMAIR_EL1 = X[t];
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        AMAIR_EL1 = X[t];
    else
        UNDEFINED;
```

A.1.5 LORID_EL1, LORegionID (EL1)

Indicates the number of LORegions and LORegion descriptors supported by the PE.

Configurations

If no LORegion descriptors are implemented, then the registers AArch64-LORC_EL1, AArch64-LORN_EL1, AArch64-LOREA_EL1, and AArch64-LORSA_EL1 are RES0.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0100	xxxx	xxxx	0000	0100
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-5: AArch64_lorid_el1 bit assignments

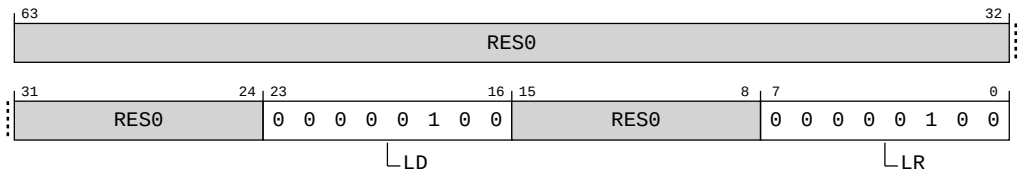


Table A-20: LORID_EL1 bit descriptions

Bits	Name	Description	Reset
[63:24]	RES0	Reserved	RES0
[23:16]	LD	Number of LORegion descriptors supported by the PE. This is an 8-bit binary number. 0b00000100 Four LOR descriptors are supported	0x04
[15:8]	RES0	Reserved	RES0
[7:0]	LR	Number of LORegions supported by the PE. This is an 8-bit binary number. Note: If LORID_EL1 indicates that no LORegions are implemented, then LoadLOAcquire and StoreLORelease will behave as LoadAcquire and StoreRelease. 0b00000100 Four LORegions are supported	0x04

Access

MRS <Xt>, LORID_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1010	0b0100	0b111

Accessibility

MRS <Xt>, LORID_EL1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TLOR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TLOR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.LORID_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TLOR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return LORID_EL1;
```

```
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TLOR == '1' then
        UNDEFINED;
    elseif SCR_EL3.TLOR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return LORID EL1;
    elseif PSTATE.EL == EL3 then
        return LORID_EL1;
```

A.1.6 IMP_CPUCFR_EL1, CPU Configuration Register

This register provides configuration information for the core.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x1x1
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-6: AArch64_imp_cpucfr_el1 bit assignments

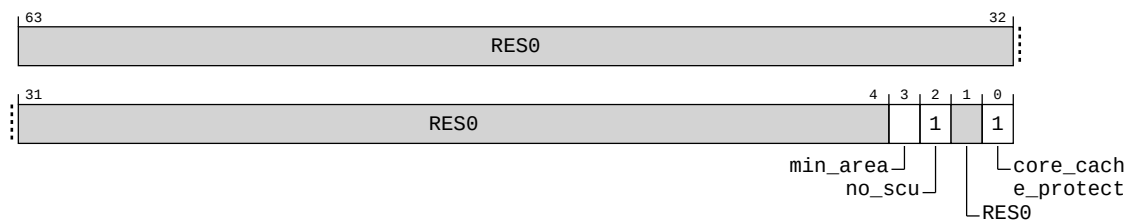


Table A-22: IMP_CPUCFR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:4]	RES0	Reserved	RES0
[3]	min_area	Indicates whether the processor is built using the area optimized configuration 0b0 not min_area. 0b1 min_area.	The reset values can be the following: 0b0, 0b1, respective to the value.
[2]	no_scu	Indicates whether the SCU is present or not. Possible values of this bit are: 0b1 The SCU is not present.	0b1
[1]	RES0	Reserved	RES0
[0]	core_cache_protect	Indicates whether ECC is present or not. Possible values of this field are: 0b1 ECC is present.	0b1

Access

MRS <Xt>, S3_0_C15_C0_0

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0000	0b000

Accessibility

MRS <Xt>, S3_0_C15_C0_0

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUCFR_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUCFR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUCFR_EL1;

```

A.1.7 IMP_CPUACTLR_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-7: AArch64_imp_cpuactlr_el1 bit assignments

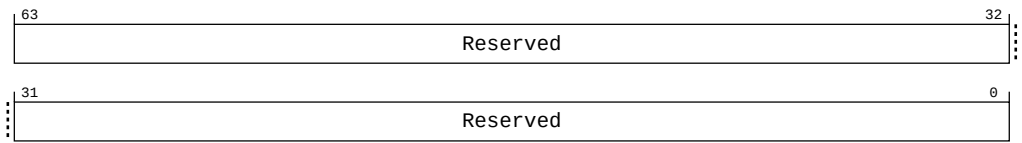


Table A-24: IMP_CPUACTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MSR S3_0_C15_C1_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b000

MRS <Xt>, S3_0_C15_C1_0

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b000

Accessibility

MSR S3_0_C15_C1_0, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elsif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUACTLR_EL1 = X[t];
    elsif PSTATE.EL == EL3 then
        IMP_CPUACTLR_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C1_0

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR_EL1;
elsif PSTATE.EL == EL2 then
    return IMP_CPUACTLR_EL1;
elsif PSTATE.EL == EL3 then
    return IMP_CPUACTLR_EL1;

```

A.1.8 IMP_CPUACTLR2_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-8: AArch64_imp_cpuactlr2_el1 bit assignments

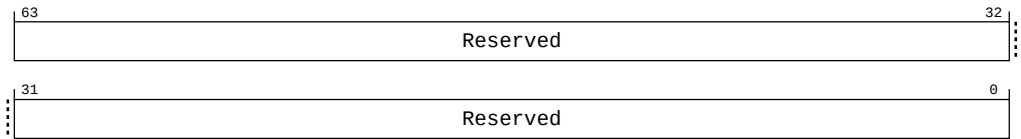


Table A-27: IMP_CPUACTLR2_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MSR S3_0_C15_C1_1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b001

MRS <Xt>, S3_0_C15_C1_1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b001

Accessibility

MSR S3_0_C15_C1_1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
```

```

        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR2_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUACTLR2_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR2_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C1_1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR2_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUACTLR2_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUACTLR2_EL1;

```

A.1.9 IMP_CPUACTLR3_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-9: AArch64_imp_cpuactlr3_el1 bit assignments

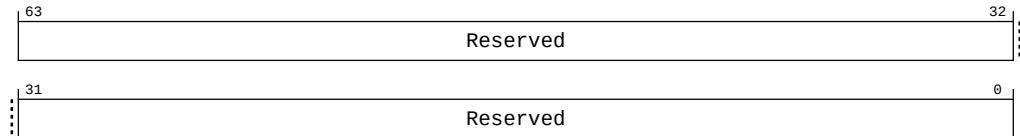


Table A-30: IMP_CPUACTLR3_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MSR S3_0_C15_C1_2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b010

MRS <Xt>, S3_0_C15_C1_2

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b010

Accessibility

MSR S3_0_C15_C1_2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR3_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then

```

```
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        IMP_CPUACTLR3_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_CPUACTLR3_EL1 = X[t];
```

MRS <Xt>, S3_0_C15_C1_2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR3_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUACTLR3_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUACTLR3_EL1;
```

A.1.10 IMP_CPUACTLR4_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-10: AArch64_imp_cpuctlr4_el1 bit assignments

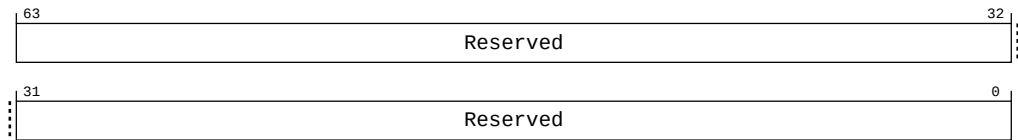


Table A-33: IMP_CPUACTLR4_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MSR S3_0_C15_C1_3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b011

MRS <Xt>, S3_0_C15_C1_3

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b011

Accessibility

MSR S3_0_C15_C1_3, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR4_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR4_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR4_EL1 = X[t];
  
```

MRS <Xt>, S3_0_C15_C1_3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR4_EL1;
elsif PSTATE.EL == EL2 then
    return IMP_CPUACTLR4_EL1;
elsif PSTATE.EL == EL3 then
    return IMP_CPUACTLR4_EL1;
```

A.1.11 IMP_CPUACTLR4_EL1, CPU Extended Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

0010	0100	0010	0000	0000	0000	0111	0010	0000	0110	0010	0000	1011	xxx0	0001	0110
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-11: AArch64_imp_cpuctlr_el1 bit assignments

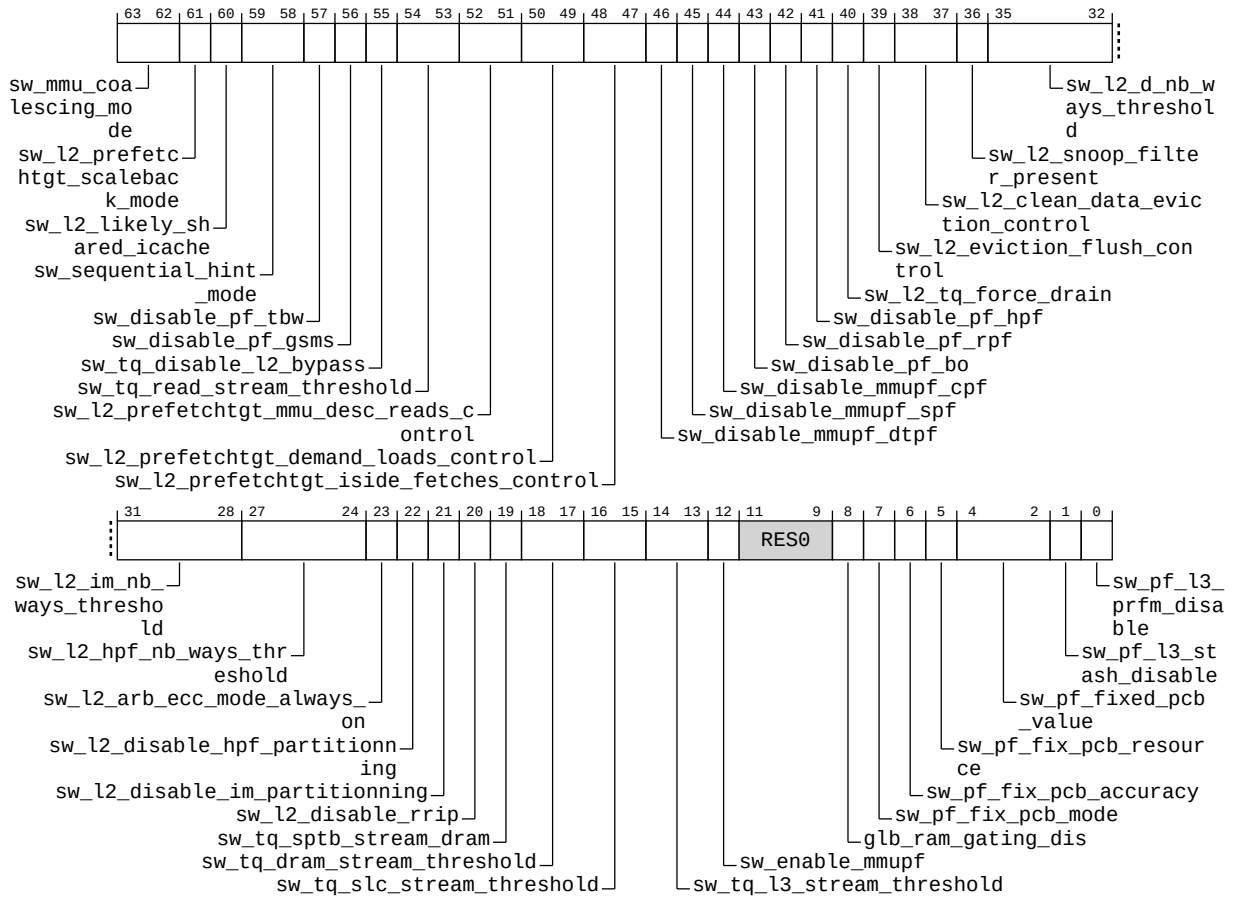


Table A-36: IMP_CPUECTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:62]	sw_mmu_coalescing_mode	Configure MMU coalescing mode 0b00 Coalescing mode to be used is 8x32 0b01 Coalescing mode to be used is 4x2048 0b10 Reserved for future use 0b11 Coalescing is disabled: All TLB entries contain a single VA-PA translation.	0b00

Bits	Name	Description	Reset
[61]	sw_l2_prefetchtgt_scaleback_mode	Control PrefetchTgt scaleback mode 0b0 Scaleback mode is conservative 0b1 Scaleback mode is aggressive	0b1
[60]	sw_l2_likely_shared_icache	Instruction fetch shared state control 0b0 Instruction fetch requests do not assert TXREQ LikelyShared 0b1 Instruction fetch requests do assert TXREQ LikelyShared and request a SharedClean copy of data	0b0
[59:58]	sw_sequential_hint_mode	Configure sequential hint working mode 0b00 Sequential Hint is enabled in coverage mode (Best-Offset and Generation Table) 0b01 Sequential Hint is enabled in compromise mode (only Best-Offset) 0b10 Sequential Hint is enabled in high accuracy mode (only Best-Offset) 0b11 Sequential Hint is disabled	0b01
[57]	sw_disable_pf_tbw	Disable Table Walk Prefetcher 0b0 Enabled 0b1 Disabled	0b0
[56]	sw_disable_pf_gsms	Disable Global Spatial Memory Streaming Prefetcher 0b0 Enabled 0b1 Disabled	0b0
[55]	sw_tq_disable_l2_bypass	Disable the heuristics that sends L1 evictions directly to L3, bypassing the L2 0b0 Enabled 0b1 Disabled	0b0

Bits	Name	Description	Reset
[54:53]	sw_tq_read_stream_threshold	Configure thresholds for transforming WriteEvictOrEvict into Evict 0b00 1024 KB 0b01 2048 KB 0b10 4096 KB 0b11 Disable	0b01
[52:51]	sw_l2_prefetchtgt_mmu_desc_reads_control	Control activation of the PrefetchTgt for MMU descriptors reads 0b00 PrefetchTgt are not sent for MMU descriptors reads 0b01 Conservatively generate PrefetchTgt for cacheable requests from the MMU, always generate for noncacheable 0b10 Aggressively generate PrefetchTgt for cacheable requests from the MMU, always generate for noncacheable 0b11 Always generate PrefetchTgt for cacheable requests from the MMU, always generate for noncacheable	0b00
[50:49]	sw_l2_prefetchtgt_demand_loads_control	Control activation of the PrefetchTgt for the Demand Loads/Stores 0b00 PrefetchTgt are not sent for Demand Loads/Stores 0b01 Conservatively generate PrefetchTgt for cacheable requests for Demand Loads/Stores, always generate for noncacheable 0b10 Aggressively generate PrefetchTgt for cacheable requests for Demand Loads/Stores, always generate for noncacheable 0b11 Always generate PrefetchTgt for cacheable requests for Demand Loads/Stores, always generate for noncacheable	0b00

Bits	Name	Description	Reset
[48:47]	sw_l2_prefetchtgt_iside_fetches_control	Control activation of the PrefetchTgt for the Iside fetches 0b00 PrefetchTgt are not sent for Iside fetches 0b01 Conservatively generate PrefetchTgt for cacheable requests from the Iside, always generate for noncacheable 0b10 Aggressively generate PrefetchTgt for cacheable requests from the Iside, always generate for noncacheable 0b11 Always generate PrefetchTgt for cacheable requests from the Iside, always generate for noncacheable	0b00
[46]	sw_disable_mmupf_dtpf	Control activation of MMU debug & trace prefetcher (DTPF) 0b0 MMU D&T prefetcher is enabled allowing the prefetching of next/previous page for each D&T request (requires feature bit ENABLE_MMUPF in CPUECTLR to be set as well) 0b1 MMU D&T prefetcher is disabled	0b0
[45]	sw_disable_mmupf_spf	Control activation of MMU stream prefetcher (SPF) 0b0 MMU stream prefetcher is enabled allowing the prefetching of next/previous page when a stream is detected (requires feature bit ENABLE_MMUPF in CPUECTLR to be set as well) 0b1 MMU stream prefetcher is disabled	0b0
[44]	sw_disable_mmupf_cpf	Control activation of MMU coalescing prefetcher (CPF) 0b0 MMU coalescing prefetcher is enabled allowing the prefetching of next/previous clusters in case of coalesced entries (requires feature bit ENABLE_MMUPF in CPUECTLR to be set as well) 0b1 MMU coalescing prefetcher is disabled	0b0
[43]	sw_disable_pf_bo	Disable Best Offset Prefetcher 0b0 Enabled 0b1 Disabled	0b0
[42]	sw_disable_pf_rpf	Disable Region Prefetcher 0b0 Enabled 0b1 Disabled	0b0

Bits	Name	Description	Reset
[41]	sw_disable_pf_hpf	Disable History Prefetcher 0b0 Enabled 0b1 Disabled	0b0
[40]	sw_l2_tq_force_drain	Force all evictions (L1/L2) to be drained out of TQ 0b0 Evictions are drained normally 0b1 Evictions are forced to drain ASAP	0b0
[39]	sw_l2_eviction_flush_control	Controls whether hardware cache flushes and DC CISC instructions send data when evicting clean cache-lines on the CHI interface 0b0 Disables sending data when hardware cache flushes or DC CISC instructions evict a clean cache-line. Sending of Evict transactions is controlled by Downstream Snoop Filter Present. This is the reset value 0b1 Sending of data when hardware cache flushes or DC CISC instructions evict clean cache-lines is controlled by Downstream Cache Control. Sending of Evict transactions is controlled by Downstream Snoop Filter Present	0b0
[38:37]	sw_l2_clean_data_eviction_control	Downstream Cache Control 0b00 Disables sending data when clean cache-lines are evicted 0b01 Enables sending WriteEvictFull transactions when Unique Clean cache-lines are evicted. Shared Clean cache-line evictions do not send data 0b10 Enables sending WriteEvictOrEvict transactions when Unique Clean cache-lines are evicted. Shared Clean cache-line evictions do not send data 0b11 Enables sending WriteEvictOrEvict transactions when Unique Clean or Shared Clean cache-lines are evicted.	0b11
[36]	sw_l2_snoop_filter_present	Downstream Snoop Filter Present 0b0 Disables sending Evict transactions when clean cache-lines are evicted without data. 0b1 Enables sending Evict transactions when clean cache-lines are evicted without data.	0b1

Bits	Name	Description	Reset
[35:32]	sw_l2_d_nb_ways_threshold	D-side ways partition in L2 cache. The value specified indicates the number of ways dedicated to requests from L1 data cache. It is expected within the range 0b0000-0b1000, an higher value will be ignored and set to 0b1000. The specified way partitioning is not guaranteed if the sum with sw_l2_im_nb_ways_threshold and sw_l2_hpf_nb_ways_threshold is not equal to 8, the number of L2 cache ways. If the sum is lower than 8, the remaining ways will be used for D and I/MMU requests. Reset value is 0b0010 and is the recommended value for this product	0b0010
[31:28]	sw_l2_im_nb_ways_threshold	I-side/MMU ways partition in L2 cache. The value specified indicates the number of ways dedicated to requests from L1 I-cache or MMU. It is expected within the range 0b0000-0b1000, an higher value will be ignored and set to 0b1000. The specified way partitioning is not guaranteed if the sum with sw_l2_d_nb_ways_threshold and sw_l2_hpf_nb_ways_threshold is not equal to 8, the number of L2 cache ways. If the sum is lower than 8, the remaining ways will be used for D and I/MMU requests. Reset value is 0b0000 and is the recommended value for this product	0b0000
[27:24]	sw_l2_hpf_nb_ways_threshold	HPF ways partition in L2 cache. The value specified indicates the number of ways dedicated to requests from HPF. It is expected within the range 0b0000-0b1000, an higher value will be ignored and set to 0b1000. The specified way partitioning is not guaranteed if the sum with sw_l2_d_nb_ways_threshold and sw_l2_im_nb_ways_threshold is not equal to 8, the number of L2 cache ways. If the sum is lower than 8, the remaining ways will be used for D and I/MMU requests. Reset value is 0b0110 and is the recommended value for this product	0b0110
[23]	sw_l2_arb_ecc_mode_always_on	Force one more cycle in the L2 pipeline to allow for inline ECC error corrections on the Tag RAM 0b0 Default 0b1 Force	0b0
[22]	sw_l2_disable_hpf_partitionning	Disable L2 cache replacement policy partitioning for History Prefetcher 0b0 Default 0b1 Disable	0b0
[21]	sw_l2_disable_im_partitionning	Disable L2 cache replacement policy partitioning for L1I and MMU 0b0 Default 0b1 Disable	0b1
[20]	sw_l2_disable_rrip	Disable RRIP replacement policy in L2 cache, falling back to random policy 0b0 Default 0b1 Disable	0b0

Bits	Name	Description	Reset
[19]	sw_tq_sptb_stream_dram	Force all writes for Trace and Statistical Profiling buffers to external memory 0b0 Default 0b1 Disable	0b0
[18:17]	sw_tq_dram_stream_threshold	Configure thresholds for streaming writes to external memory 0b00 1024 KB 0b01 2048 KB 0b10 4096 KB 0b11 Disable	0b00
[16:15]	sw_tq_slc_stream_threshold	Configure thresholds for streaming writes to System Level Cache 0b00 256 KB 0b01 512 KB 0b10 1024 KB 0b11 Disable	0b01
[14:13]	sw_tq_l3_stream_threshold	Configure thresholds for streaming writes to L3 cache 0b00 16 KB 0b01 32 KB 0b10 64 KB 0b11 Disable	0b01
[12]	sw_enable_mmupf	Control activation of MMU prefetchers 0b0 MMU prefetchers are disabled 0b1 MMU prefetchers are enabled and can send translation requests to MMU pipeline	0b1
[11:9]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[8]	glb_ram_gating_dis	Global RAM clock gating disable 0b0 RAMs clock gating active 0b1 RAMs clock gating disabled	0b0
[7]	sw_pf_fix_pcb_mode	Disable L2 PF dynamic modes and fix to a static value given by sw_pf_fixed_pcb_value 0b0 Dynamic behaviour is enabled 0b1 Modes are fixed to a static value	0b0
[6]	sw_pf_fix_pcb_accuracy	Disable L2 PF dynamic accuracy feedback and fix to a static value given by sw_pf_fixed_pcb_value 0b0 Dynamic behaviour is enabled 0b1 Accuracy feedback is fixed to a static value	0b0
[5]	sw_pf_fix_pcb_resource	Disable L2 PF dynamic resource feedback and fix to a static value given by sw_pf_fixed_pcb_value 0b0 Dynamic behaviour is enabled 0b1 Resource feedback is fixed to a static value	0b0

Bits	Name	Description	Reset
[4:2]	sw_pf_fixed_pcb_value	Static value for L2 PF mode, accuracy feedback or resource feedback if dynamic behaviour is disabled 0b000 Mode = 0 (turn PF off), accuracy_feedback = VERY_LOW, resource_feedback = FULLY_SATURATED 0b001 Mode = 1, accuracy_feedback = LOW, resource_feedback = PARTLY_SATURATED 0b010 Mode = 2, accuracy_feedback = MEDIUM, resource_feedback = NORMAL 0b011 Mode = 3, accuracy_feedback = HIGH, resource_feedback = LOW 0b100 Mode = 4, accuracy_feedback = VERY_HIGH, resource_feedback = EMPTY 0b101 Mode = 5 (most aggressive), invalid for accuracy and resource feedback 0b110 Invalid 0b111 Invalid	0b101
[1]	sw_pf_l3_stash_disable	Disables usage of the StashOnce CHI request 0b0 Enabled 0b1 Disabled	0b1
[0]	sw_pf_l3_prfm_disable	Prevents PRFM to send stash requests on CHI 0b0 PRFM that target L3 are normally sent on CHI 0b1 PRFM that target L3 are not sent on CHI	0b0

Access

MSR S3_0_C15_C1_4, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b100

MRS <Xt>, S3_0_C15_C1_4

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b100

Accessibility

MSR S3_0_C15_C1_4, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ECTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ECTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ECTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUECTLR_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ECTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUECTLR_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUECTLR_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C1_4

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUECTLR_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUECTLR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUECTLR_EL1;

```

A.1.12 IMP_CPUECTLR2_EL1, CPU Extended Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

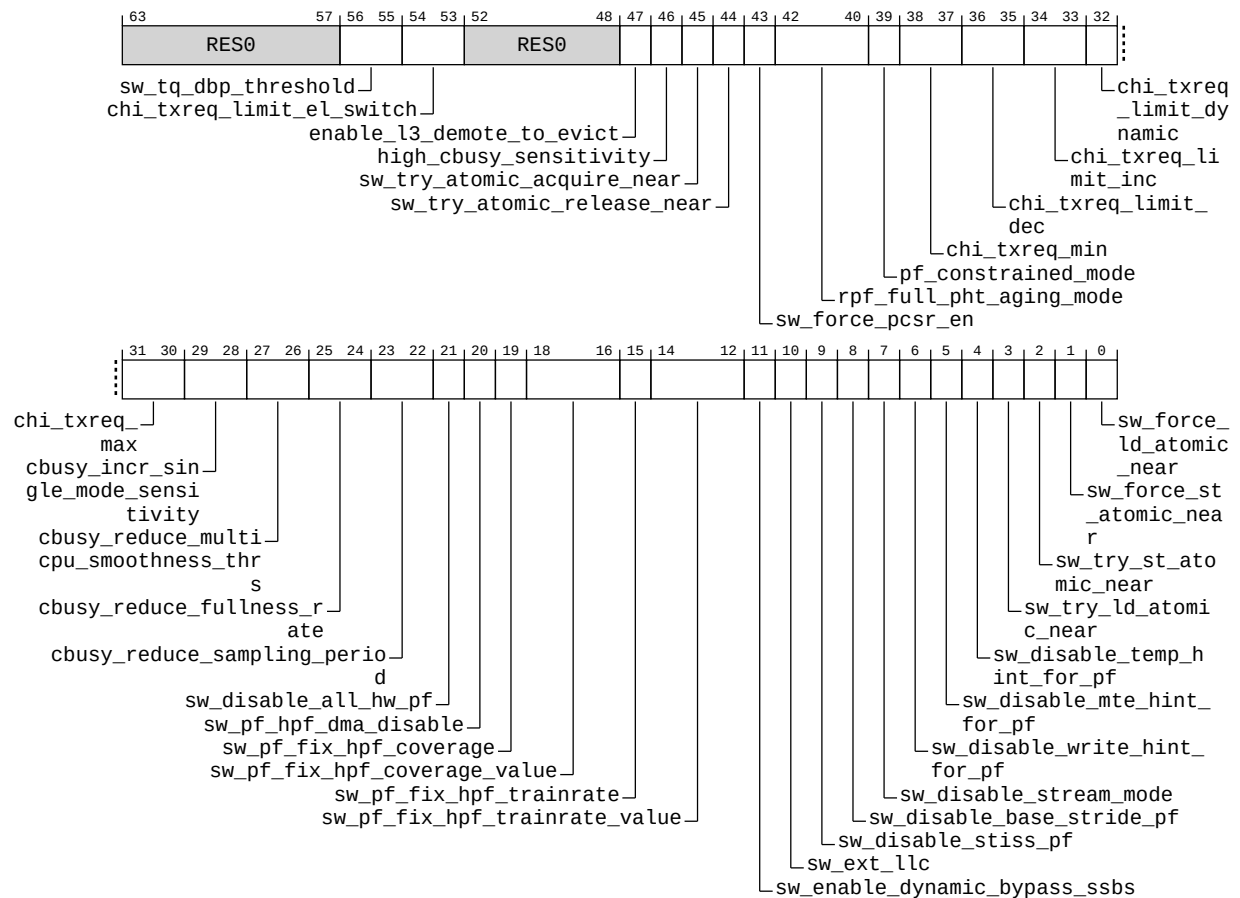
See bit descriptions

Reset value

xxxx	xxx0	100x	xxxx	1011	0011	0000	0000	0010	0110	0100	0111	0111	0100	0000	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions**Figure A-12: AArch64_imp_cpuctlr2_el1 bit assignments****Table A-39: IMP_CPUECTLR2_EL1 bit descriptions**

Bits	Name	Description	Reset
[63:57]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[56:55]	sw_tq_dbp_threshold	Configure thresholds for making L2 evictions dataless 0b00 Evicted data for a total of L2_CACHE_SIZE/4 0b01 Evicted data for a total of L2_CACHE_SIZE/2 0b10 Evicted data for a total of L2_CACHE_SIZE 0b11 Disable	0b01
[54:53]	chi_txreq_limit_el_switch	Behavior of outstanding TXREQ limit on EL switch. 0b00 Exception Level switches do not affect the outstanding TXREQ limit - This is the default value 0b01 Exception Level switches reset the outstanding TXREQ limit to the initial value 0b10 Exception Level switches restore the outstanding TXREQ limit to the previously saved value for that Exception Level 0b11 Reserved	0b00
[52:48]	RES0	Reserved	RES0
[47]	enable_l3_demote_to_evict	Convert L3 demotions into dataless Evict if PF is conservative 0b0 L3 demotions can be converted into Evict 0b1 L3 demotions cannot be converted into Evict	0b1
[46]	high_cbusy_sensitivity	Higher sensitivity to CBUSY feedback. 0b0 High sensitivity to CBUSY is disabled 0b1 High sensitivity to CBUSY is enabled	0b0
[45]	sw_try_atomic_acquire_near	Try atomic acquire as near even if misses in cache. If the line is evicted, move to far 0b0 Don't try to execute atomic acquire in near mode if miss 0b1 Try to execute atomic acquire in near mode if miss	0b1

Bits	Name	Description	Reset
[44]	sw_try_atomic_release_near	Try atomic release as near even if misses in cache. If the line is evicted, move to far 0b0 Don't try to execute atomic release in near mode if miss 0b1 Try to execute atomic release in near mode if miss	0b1
[43]	sw_force_pcsr_en	Force pcsr_enable 0b0 PCSR is not updated before the first PCSR read. It is then updated each time an instruction is executed. This is the normal update process. 0b1 Force PCSR to be updated each time an instruction is executed.	0b0
[42:40]	rpf_full_pht_aging_mode	RPF mode to enable full pht aging 0b000 Mode = 0, PF off; it enables full pht aging only if PF is off 0b001 Mode = 1, it enables full pht aging only if mode is lower or equal than 1 0b010 Mode = 2, it enables full pht aging only if mode is lower or equal than 2 0b011 Mode = 3, it enables full pht aging only if mode is lower or equal than 3 0b100 Mode = 4, it enables full pht aging only if mode is lower or equal than 4 0b101 Mode = 5, most aggressive mode; it enables rpf full pht aging at each training 0b110 Invalid 0b111 Invalid	0b011
[39]	pf_constrained_mode	Prefetcher throttling for constrained configurations. 0b0 Prefetcher throttling for constrained configurations is disabled 0b1 Prefetcher throttling for constrained configurations is enabled	0b0

Bits	Name	Description	Reset
[38:37]	chi_txreq_min	Minimum number of TXREQ transactions outstanding when the dynamic limit is enabled. 0b00 1/4 of L2 TQ size - This is the default value 0b01 1/8 of L2 TQ size 0b10 1/16 of L2 TQ size 0b11 1/32 of L2 TQ size	0b00
[36:35]	chi_txreq_limit_dec	Dynamic TXREQ limit decrement. Controls how quickly the dynamic TXREQ limit is decreased when CBusy indicates value of 3. 0b00 4 - This is the default value 0b01 8 0b10 16 0b11 2	0b00
[34:33]	chi_txreq_limit_inc	Dynamic TXREQ limit increment. Controls how quickly the dynamic TXREQ limit is increased when CBusy indicates values less than 2. 0b00 4 - This is the default value 0b01 8 0b10 16 0b11 2	0b00
[32]	chi_txreq_limit_dynamic	Selects static or dynamic control of TXREQ limit. Dynamic TXREQ limit will adjust based on CBusy responses on RXDAT and RXRSP in the range of limit selected. 0b0 maximum number of TXREQ transactions set 0b1 maximum number of TXREQ transactions dynamically controlled	0b0

Bits	Name	Description	Reset
[31:30]	chi_txreq_max	Maximum number of TXREQ transactions outstanding from the L2 Transaction Queue. 0b00 2 * L2 TQ size - This is the default value 0b01 3/4 of L2 TQ size 0b10 1/2 of L2 TQ size 0b11 1/4 of L2 TQ size	0b00
[29:28]	cbusy_incr_single_mode_sensitivity	Reduces the sensitivity to the single mode detection for streaming to System Level Cache and to DRAM by reducing sampling period of multi cpu state 0b00 Fraction of multi cpu states set to the sampling period 0b01 Fraction of multi cpu states set to 1/2 of the sampling period 0b10 Fraction of multi cpu states set to 1/4 of the sampling period 0b11 Fraction of multi cpu states set to 1/8 of the sampling period	0b10
[27:26]	cbusy_reduce_multicpu_smoothness_thrs	Reduces the smoothness period in multi CPU cluster 0b00 Sampling period set to 1023 0b01 Sampling period set to 511 0b10 Sampling period set to 255 0b11 Sampling period set to 127	0b01
[25:24]	cbusy_reduce_fullness_rate	Reduces the fraction of CBusy responses in a given sampling period necessary to define a given CBusy state 0b00 Fraction of CBusy responses set to 1/4 of the sampling period 0b01 Fraction of CBusy responses set to 1/8 of the sampling period 0b10 Fraction of CBusy responses set to 1/16 of the sampling period 0b11 Fraction of CBusy responses set to 1/32 of the sampling period	0b10

Bits	Name	Description	Reset
[23:22]	cbusy_reduce_sampling_period	Reduces sampling period of CBusy responses from maximum value (511) through a logarithmic scale 0b00 Sampling period set to 511 0b01 Sampling period set to 255 0b10 Sampling period set to 127 0b11 Sampling period set to 63	0b01
[21]	sw_disable_all_hw_pf	Disable all hardware prefetchers (dside, L2, MMU) 0b0 HW prefetcher are not disabled 0b1 HW prefetcher are all disabled	0b0
[20]	sw_pf_hpf_dma_disable	Disable HPF Dynamic Metadata Aging (HPF DMA) and set DMA Threshold to static value (DMAT[7]). 0b0 Dynamic behaviour is enabled 0b1 Dynamic behaviour is disable and DMA Threshold is fixed to a static value (DMAT[7])	0b0
[19]	sw_pf_fix_hpf_coverage	Set L2 HPF Coverage for HPF Dynamic Metadata Aging (HPF DMA) to a static value. 0b0 Dynamic behaviour is enabled 0b1 Coverage feedback is fixed to a static value	0b0

Bits	Name	Description	Reset
[18:16]	sw_pf_fix_hpf_coverage_value	Static value for L2 HPF Coverage if dynamic behaviour is disabled 0b000 Coverage = 0 (less conservative), DMAT will fully depends on Training Rate and ranges from DMAT[0] to DMAT[6]. 0b001 Coverage = 1, Training Rate[0,5[%: DMAT[1]; Training Rate[5,10[%: DMAT[2]; Training Rate[10,20[%: DMAT[3]; ...; Training Rate[60,100[%: DMAT[7]. 0b010 Coverage = 2, Training Rate[0,5[%: DMAT[2]; Training Rate[5,10[%: DMAT[3]; Training Rate[10,20[%: DMAT[4]; ...; Training Rate[45,100[%: DMAT[7]. 0b011 Coverage = 3, Training Rate[0,5[%: DMAT[3]; Training Rate[5,10[%: DMAT[4]; Training Rate[10,20[%: DMAT[5]; ...; Training Rate[30,100[%: DMAT[7]. 0b100 Coverage = 4, Training Rate[0,5[%: DMAT[4]; Training Rate[5,10[%: DMAT[5]; Training Rate[10,20[%: DMAT[6]; Training Rate[20,100[%: DMAT[7]. 0b101 Coverage = 5, DMA will be set to highest value (DMAT[7]) unless: Training Rate[0,5[%: DMAT[5]; Training Rate[5,10[%: DMAT[6]. 0b110 Coverage = 6 (most conservative), DMA Threshold (DMAT) will be set to highest value (DMAT[7]) unless Training Rate[0,5[% (DMAT[6]). 0b111 Invalid	0b111
[15]	sw_pf_fix_hpf_trainrate	Set L2 HPF Training Rate for HPF Dynamic Metadata Aging (HPF DMA) to a static value. 0b0 Dynamic behaviour is enabled 0b1 Training Rate feedback is fixed to a static value	0b0

Bits	Name	Description	Reset
[14:12]	sw_pf_fix_hpf_trainrate_value	Static value for L2 HPF Training Rate if dynamic behaviour is disabled 0b000 Training Rate = 0 (less conservative), DMAT will fully depends on Coverage and ranges from DMAT[0] to DMAT[6]. 0b001 Training Rate = 1, Coverage[0,5[%: DMAT[1]; Coverage[5,10[%: DMAT[2]; Coverage[10,20[%: DMAT[3]; ...; Coverage[60,100[%: DMAT[7]. 0b010 Training Rate = 2, Coverage[0,5[%: DMAT[2]; Coverage[5,10[%: DMAT[3]; Coverage[10,20[%: DMAT[4]; ...; Coverage[45,100[%: DMAT[7]. 0b011 Training Rate = 3, Coverage[0,5[%: DMAT[3]; Coverage[5,10[%: DMAT[4]; Coverage[10,20[%: DMAT[5]; ...; Coverage[30,100[%: DMAT[7]. 0b100 Training Rate = 4, Coverage[0,5[%: DMAT[4]; Coverage[5,10[%: DMAT[5]; Coverage[10,20[%: DMAT[6]; Coverage[20,100[%: DMAT[7]. 0b101 Training Rate = 5, DMA will be set to highest value (DMAT[7]) unless: Coverage[0,5[%: DMAT[5]; Coverage[5,10[%: DMAT[6]. 0b110 Training Rate = 6 (most conservative), DMA Threshold (DMAT) will be set to highest value (DMAT[7]) unless Coverage[0,5[% (DMAT[6]). 0b111 Invalid	0b111
[11]	sw_enable_dynamic_bypass_ssbs	Enable dynamic bypass SSBS mode 0b0 Dynamic Bypass SSBS is disabled 0b1 Dynamic Bypass SSBS is enabled	0b0
[10]	sw_ext_llc	External last level cache 0b0 Last Level Cache is not external 0b1 Last Level Cache is external	0b1
[9]	sw_disable_stiss_pf	Disable Store Issue Prefetcher 0b0 Store Issue Prefetcher is enabled 0b1 Store Issue Prefetcher is disabled	0b0

Bits	Name	Description	Reset
[8]	sw_disable_base_stride_pf	Disable Stride Prefetcher 0b0 Stride PF is enabled 0b1 Stride PF is disabled	0b0
[7]	sw_disable_stream_mode	Disable stream of writes to L2 and beyond 0b0 Stream mode is enabled 0b1 Stream mode is disabled	0b0
[6]	sw_disable_write_hint_for_pf	Disable write access hint for prefetch. 0b0 Write access hint is enabled 0b1 Write access hint is disabled	0b0
[5]	sw_disable_mte_hint_for_pf	Disable mte access hint for prefetch. 0b0 MTE access hint is enabled 0b1 MTE access hint is disabled	0b0
[4]	sw_disable_temp_hint_for_pf	Disable temp access hint for prefetch. 0b0 Temp access hint is enabled 0b1 Temp access hint is disabled	0b0
[3]	sw_try_ld_atomic_near	Try atomic load as near even if misses in cache. If the line is evicted, move to far 0b0 Don't try to execute atomic load in near mode if miss 0b1 Try to execute atomic load in near mode if miss	0b0
[2]	sw_try_st_atomic_near	Try atomic store as near even if misses in cache. If the line is evicted, move to far 0b0 Don't try to execute atomic store in near mode if miss 0b1 Try to execute atomic store in near mode if miss	0b0
[1]	sw_force_st_atomic_near	Always execute atomic store in near mode 0b0 Don't force atomic store to execute in near mode 0b1 Force atomic store to execute in near mode	0b0

Bits	Name	Description	Reset
[0]	sw_force_ld_atomic_near	Always execute atomic load in near mode 0b0 Don't force atomic load to execute in near mode 0b1 Force atomic load to execute in near mode	0b1

Access

MSR S3_0_C15_C1_5, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b101

MRS <Xt>, S3_0_C15_C1_5

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b101

Accessibility

MSR S3_0_C15_C1_5, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ECTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ECTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ECTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR2_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ECTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUACTLR2_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR2_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C1_5

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else

```



```
        return IMP_CPUECTLR2_EL1;
    elsif PSTATE.EL == EL2 then
        return IMP_CPUECTLR2_EL1;
    elsif PSTATE.EL == EL3 then
        return IMP_CPUECTLR2_EL1;
```

A.1.13 IMP_CPUBUSQOS_EL1, Quality of Service Bus Control Register, EL1

This register controls QoS values sent on Txreq for various request sources

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1110	1010	1110
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-13: AArch64_imp_cpubusqos_el1 bit assignments

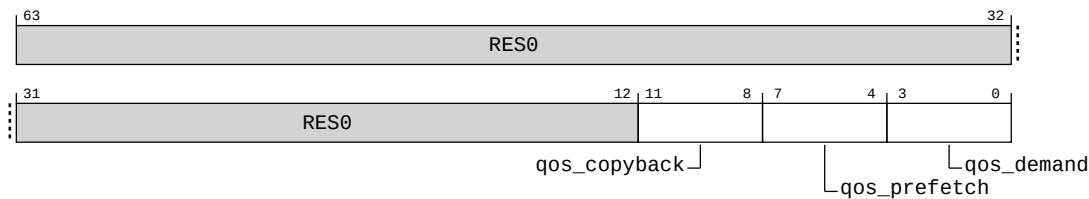


Table A-42: IMP_CPUBUSQOS_EL1 bit descriptions

Bits	Name	Description	Reset
[63:12]	RES0	Reserved	RES0
[11:8]	qos_copyback	QOS value sent on copyback requests	0b1110
[7:4]	qos_prefetch	QOS value sent on prefetch requests	0b1010
[3:0]	qos_demand	QOS value sent on demand requests	0b1110

Access

MSR S3_0_C15_C1_7, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b111

MRS <Xt>, S3_0_C15_C1_7

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0001	0b111

Accessibility

MSR S3_0_C15_C1_7, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.QOSEN == '0' then
        UNDEFINED;
    elsif EL2Enabled() && ACTLR_EL2.QOSEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif ACTLR_EL3.QOSEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUBUSQOS_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if ACTLR_EL3.QOSEN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUBUSQOS_EL1 = X[t];
    elsif PSTATE.EL == EL3 then
        IMP_CPUBUSQOS_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C1_7

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);

```

```
else
    return IMP_CPUBUSQOS_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUBUSQOS_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUBUSQOS_EL1;
```

A.1.14 IMP_CPUPWRCTLR_EL1, CPU Power Control Register

This register controls various power aspects of the core.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-14: AArch64_imp_cpupwrctlr_el1 bit assignments

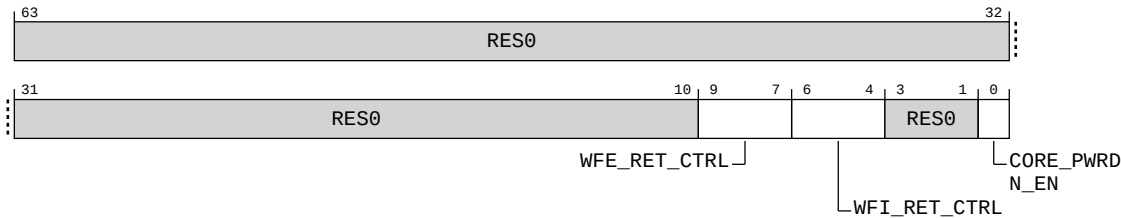


Table A-45: IMP_CPUPWRCTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:10]	RES0	Reserved	RES0
[9:7]	WFE_RET_CTRL	Wait for Event retention control. 0b000 Dynamic retention is disabled. 0b001 2 system counter ticks are required before retention entry. 0b010 8 system counter ticks are required before retention entry. 0b011 32 system counter ticks are required before retention entry. 0b100 64 system counter ticks are required before retention entry. 0b101 128 system counter ticks are required before retention entry. 0b110 256 system counter ticks are required before retention entry. 0b111 512 system counter ticks are required before retention entry.	xxx
[6:4]	WFI_RET_CTRL	Wait for Interrupt retention control. 0b000 Dynamic retention is disabled. 0b001 2 system counter ticks are required before retention entry. 0b010 8 system counter ticks are required before retention entry. 0b011 32 system counter ticks are required before retention entry. 0b100 64 system counter ticks are required before retention entry. 0b101 128 system counter ticks are required before retention entry. 0b110 256 system counter ticks are required before retention entry. 0b111 512 system counter ticks are required before retention entry.	xxx
[3:1]	RES0	Reserved	RES0
[0]	CORE_PWRDN_EN	Indicates to the power controller if the CPU wants to power down when it enters WFE/WFI state.	x

Access

MSR S3_0_C15_C2_7, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0010	0b111

MRS <Xt>, S3_0_C15_C2_7

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0010	0b111

Accessibility

MSR S3_0_C15_C2_7, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.PWREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.PWREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.PWREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUPWRCTLR_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.PWREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUPWRCTLR_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUPWRCTLR_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C2_7

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUPWRCTLR_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUPWRCTLR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPWRCTLR_EL1;

```

A.1.15 IMP_CLUSTERACTLR_EL1, Cluster Auxiliary Control Register

These register bits are reserved for Arm test purposes only and must not be used except under direction from Arm.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-15: AArch64_imp_clusteractlr_el1 bit assignments

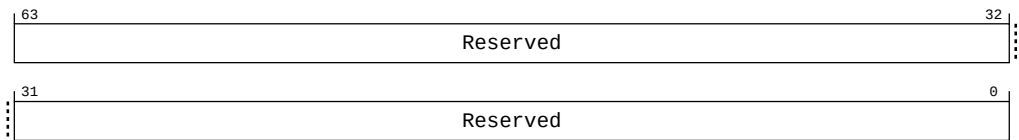


Table A-48: IMP_CLUSTERACTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MRS <Xt>, S3_0_C15_C3_3

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0011	0b011

MSR S3_0_C15_C3_3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0011	0b011

Accessibility

MRS <Xt>, S3_0_C15_C3_3

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CLUSTERACTLR_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CLUSTERACTLR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CLUSTERACTLR_EL1;

```

MSR S3_0_C15_C3_3, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CLUSTERACTLR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CLUSTERACTLR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_CLUSTERACTLR_EL1 = X[t];

```

A.1.16 IMP_ATCR_EL1, CPU Auxiliary Translation Control Register

This register controls the values of the PBHA signals for memory accesses generated by translation table walks in the EL1 translation regime.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-16: AArch64_imp_atcr_el1 bit assignments

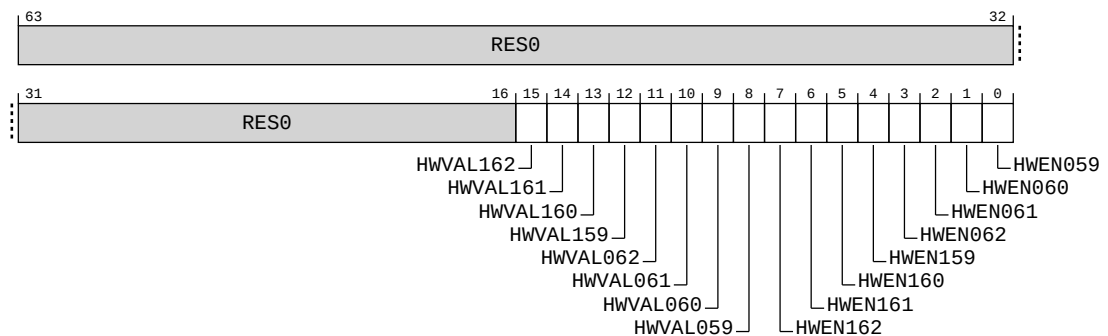


Table A-51: IMP_ATCR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15]	HWVAL162	Value of PBHA[3] on memory accesses due to translation table walks using TTBR1_EL1 if HWEN162 is set.	0b0

Bits	Name	Description	Reset
[14]	HWVAL161	Value of PBHA[2] on memory accesses due to translation table walks using TTBR1_EL1 if HWEN161 is set.	0b0
[13]	HWVAL160	Value of PBHA[1] on memory accesses due to translation table walks using TTBR1_EL1 if HWEN160 is set.	0b0
[12]	HWVAL159	Value of PBHA[0] on memory accesses due to translation table walks using TTBR1_EL1 if HWEN159 is set.	0b0
[11]	HWVAL062	Value of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL1 if HWEN062 is set.	0b0
[10]	HWVAL061	Value of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL1 if HWEN061 is set.	0b0
[9]	HWVAL060	Value of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL1 if HWEN060 is set.	0b0
[8]	HWVAL059	Value of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL1 if HWEN059 is set.	0b0
[7]	HWEN162	Enable use of PBHA[3] on memory accesses due to translation table walks using TTBR1_EL1. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[6]	HWEN161	Enable use of PBHA[2] on memory accesses due to translation table walks using TTBR1_EL1. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[5]	HWEN160	Enable use of PBHA[1] on memory accesses due to translation table walks using TTBR1_EL1. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[4]	HWEN159	Enable use of PBHA[0] on memory accesses due to translation table walks using TTBR1_EL1. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0
[3]	HWEN062	Enable use of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL1. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[2]	HWEN061	Enable use of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL1. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[1]	HWEN060	Enable use of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL1. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[0]	HWEN059	Enable use of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL1. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic IMP_ATCR_EL1 or IMP_ATCR_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, S3_0_C15_C7_0

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0111	0b000

MSR S3_0_C15_C7_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0111	0b000

MRS <Xt>, S3_5_C15_C7_0

op0	op1	CRn	CRm	op2
0b11	0b101	0b1111	0b0111	0b000

MSR S3_5_C15_C7_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b101	0b1111	0b0111	0b000

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL3 using the mnemonic IMP_ATCR_EL1 or IMP_ATCR_EL12 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, S3_0_C15_C7_0

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_ATCR_EL1;
elsif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return IMP_ATCR_EL2;
    else
        return IMP_ATCR_EL1;
elsif PSTATE.EL == EL3 then
    return IMP_ATCR_EL1;

```

MSR S3_0_C15_C7_0, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        IMP_ATCR_EL1 = X[t];
elsif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        IMP_ATCR_EL2 = X[t];
    else
        IMP_ATCR_EL1 = X[t];
elsif PSTATE.EL == EL3 then
        IMP_ATCR_EL1 = X[t];

```

MRS <Xt>, S3_5_C15_C7_0

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elsif PSTATE.EL == EL2 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        return IMP_ATCR_EL1;
    else
        UNDEFINED;
elsif PSTATE.EL == EL3 then

```

```
if EL2Enabled() && HCR_EL2.E2H == '1' then
    return IMP_ATCR_EL1;
else
    UNDEFINED;
```

MSR S3_5_C15_C7_0, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        IMP_ATCR_EL1 = X[t];
    else
        UNDEFINED;
elseif PSTATE.EL == EL3 then
    if EL2Enabled() && HCR_EL2.E2H == '1' then
        IMP_ATCR_EL1 = X[t];
    else
        UNDEFINED;
```

A.1.17 IMP_CPUACTLR5_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-17: AArch64_imp_cpuctlr5_el1 bit assignments

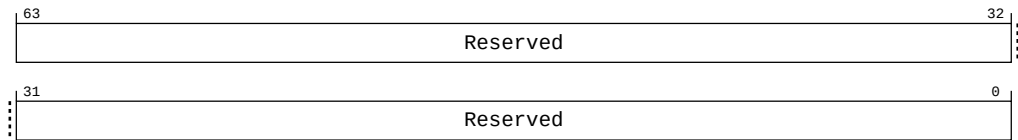


Table A-56: IMP_CPUACTLR5_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MSR S3_0_C15_C8_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b000

MRS <Xt>, S3_0_C15_C8_0

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b000

Accessibility

MSR S3_0_C15_C8_0, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLRN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLRN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLRN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR5_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLRN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR5_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR5_EL1 = X[t];
  
```

MRS <Xt>, S3_0_C15_C8_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR5_EL1;
elsif PSTATE.EL == EL2 then
    return IMP_CPUACTLR5_EL1;
elsif PSTATE.EL == EL3 then
    return IMP_CPUACTLR5_EL1;
```

A.1.18 IMP_CPUACTLR6_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-18: AArch64_imp_cpuactlr6_el1 bit assignments

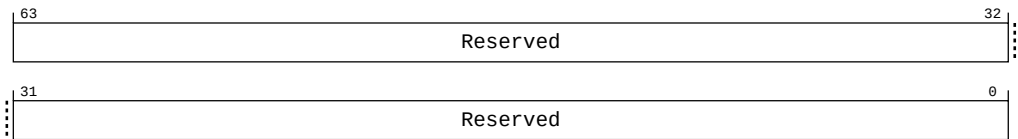


Table A-59: IMP_CPUACTLR6_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MSR S3_0_C15_C8_1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b001

MRS <Xt>, S3_0_C15_C8_1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b001

Accessibility

MSR S3_0_C15_C8_1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR6_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUACTLR6_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR6_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C8_1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR6_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUACTLR6_EL1;
elseif PSTATE.EL == EL3 then

```

```
return IMP_CPUACTLR6_EL1;
```

A.1.19 IMP_CPUACTLR7_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

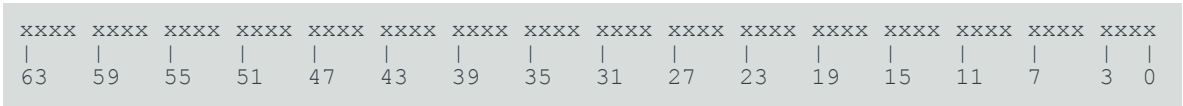
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-19: AArch64_imp_cpuactlr7_el1 bit assignments

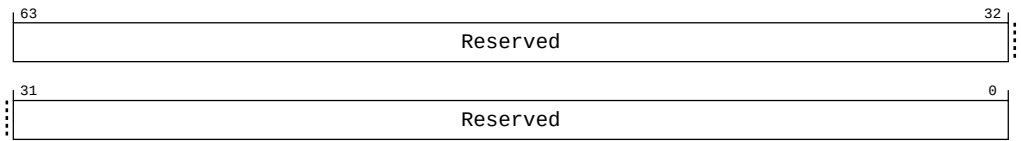


Table A-62: IMP_CPUACTLR7_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MSR S3_0_C15_C8_2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b010

MRS <Xt>, S3_0_C15_C8_2

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b010

Accessibility

MSR S3_0_C15_C8_2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR7_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUACTLR7_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR7_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C8_2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR7_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUACTLR7_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUACTLR7_EL1;

```


A.1.20 IMP_CPUACTLR8_EL1, CPU Auxiliary Control Register

This register contains control bits that affect the CPU behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

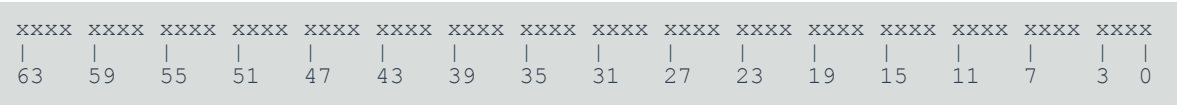
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-20: AArch64_imp_cpuctlr8_el1 bit assignments

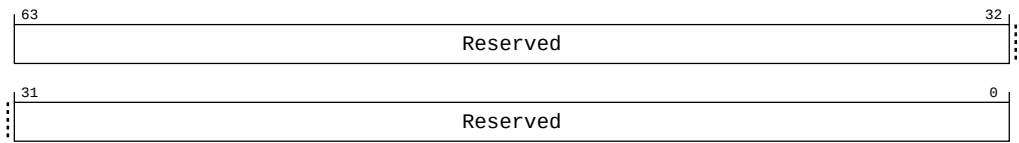


Table A-65: IMP_CPUACTLR8_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MSR S3_0_C15_C8_3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b011

MRS <Xt>, S3_0_C15_C8_3

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b1000	0b011

Accessibility

MSR S3_0_C15_C8_3, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ACTLREN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ACTLREN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ACTLREN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUACTLR8_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if ACTLR_EL3.ACTLREN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                IMP_CPUACTLR8_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        IMP_CPUACTLR8_EL1 = X[t];

```

MRS <Xt>, S3_0_C15_C8_3

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUACTLR8_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUACTLR8_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUACTLR8_EL1;

```

A.1.21 AIDR_EL1, Auxiliary ID Register

Provides **IMPLEMENTATION DEFINED** identification information.

The value of this register must be interpreted in conjunction with the value of AArch64-MIDR_EL1.

Configurations

This register is available in all configurations.

Attributes

Width

64

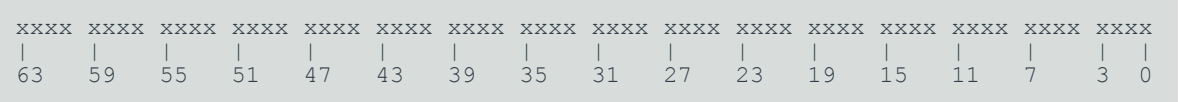
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-21: AArch64_aidr_el1 bit assignments

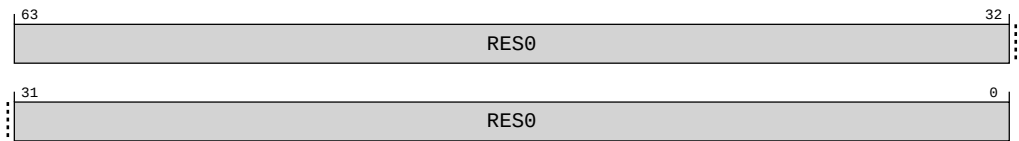


Table A-68: AIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, AIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b001	0b0000	0b0000	0b111

Accessibility

MRS <Xt>, AIDR_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
```

```
if EL2Enabled() && HCR_EL2.TID1 == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.AIDR_EL1 == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
else
    return AIDR_EL1;
elseif PSTATE.EL == EL2 then
    return AIDR_EL1;
elseif PSTATE.EL == EL3 then
    return AIDR_EL1;
```

A.1.22 RNDR, Random Number

Random Number. Returns a 64-bit random number which is reseeded from the True Random Number source at an **IMPLEMENTATION DEFINED** rate.

If the hardware returns a genuine random number, PSTATE.NZCV is set to 0b0000.

If the instruction cannot return a genuine random number in a reasonable period of time, PSTATE.NZCV is set to 0b0100 and the data value returned is 0.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-22: AArch64_rndr bit assignments

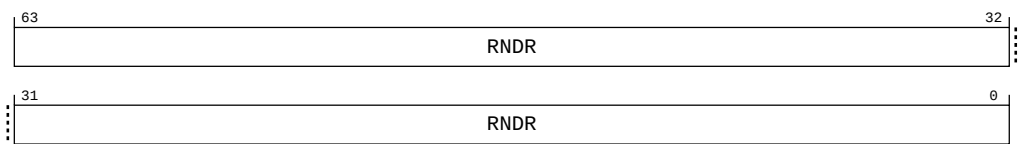


Table A-70: RNDR bit descriptions

Bits	Name	Description	Reset
[63:0]	RNDR	Random Number. Returns a 64-bit Random Number which is reseeded from the True Random Number source at an IMPLEMENTATION DEFINED rate.	64 {x}

Access

MRS <Xt>, RNDR

op0	op1	CRn	CRm	op2
0b11	0b011	0b0010	0b0100	0b000

Accessibility

MRS <Xt>, RNDR

```
if PSTATE.EL == EL0 then
    if SCR_EL3.TRNDR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDR;
elseif PSTATE.EL == EL1 then
    if SCR_EL3.TRNDR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDR;
elseif PSTATE.EL == EL2 then
    if SCR_EL3.TRNDR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDR;
elseif PSTATE.EL == EL3 then
    if SCR_EL3.TRNDR == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDR;
```

A.1.23 RNDRRS, Reseeded Random Number

Reseeded Random Number. Returns a 64-bit random number which is reseeded from the True Random Number source immediately before the read of the random number.

If the hardware returns a genuine random number, PSTATE.NZCV is set to 0b0000.

If the instruction cannot return a genuine random number in a reasonable period of time, PSTATE.NZCV is set to 0b0100 and the data value returned is 0.

When FEAT_RNG_TRAP is implemented and AArch64-SCR_EL3.TRNDR is 1, reads of this register are trapped to EL3.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-23: AArch64_rndrrs bit assignments

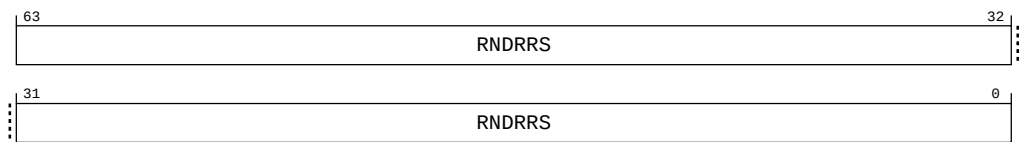


Table A-72: RNDRRS bit descriptions

Bits	Name	Description	Reset
[63:0]	RNDRRS	Reseeded Random Number. Returns a 64-bit Random Number which is reseeded from the True Random Number source immediately before this read.	64 {x}

Access

MRS <Xt>, RNDRRS

op0	op1	CRn	CRm	op2
0b11	0b011	0b0010	0b0100	0b001

Accessibility

MRS <Xt>, RNDRRS

```

if PSTATE.EL == EL0 then
    if SCR_EL3.TRNDR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDRRS;
elseif PSTATE.EL == EL1 then
    if SCR_EL3.TRNDR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDRRS;
elseif PSTATE.EL == EL2 then
    if SCR_EL3.TRNDR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDRRS;
elseif PSTATE.EL == EL3 then
    if SCR_EL3.TRNDR == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return RNDRRS;

```

A.1.24 FPCR, Floating-point Control Register

Controls floating-point behavior.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x000	0000	0000	xxxx	xxxx	xxxx	x000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-24: AArch64_fpcr bit assignments

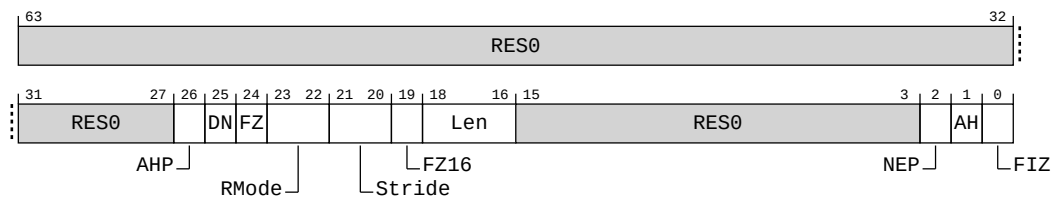


Table A-74: FPCR bit descriptions

Bits	Name	Description	Reset
[63:27]	RES0	Reserved	RES0
[26]	AHP	Alternative half-precision control bit. 0b0 IEEE half-precision format selected. 0b1 Alternative half-precision format selected. This bit is used only for conversions between half-precision floating-point and other floating-point formats. The data-processing instructions added as part of the FEAT_FP16 extension always use the IEEE half-precision format, and ignore the value of this bit.	0b0

Bits	Name	Description	Reset
[25]	DN	Default NaN use for NaN propagation. 0b0 NaN operands propagate through to the output of a floating-point operation. 0b1 Any operation involving one or more NaNs returns the Default NaN. This bit has no effect on the output of FABS, FMAX*, FMIN*, and FNEG instructions, and a default NaN is never returned as a result of these instructions. The value of this bit controls both scalar and Advanced SIMD floating-point arithmetic.	0b0
[24]	FZ	Flushing denormalized numbers to zero control bit. 0b0 If FPCR.AH is 0, the flushing to zero of single-precision and double-precision denormalized inputs to, and outputs of, floating-point instructions not enabled by this control, but other factors might cause the input denormalized numbers to be flushed to zero. If FPCR.AH is 1, the flushing to zero of single-precision and double-precision denormalized outputs of floating-point instructions not enabled by this control, but other factors might cause the input denormalized numbers to be flushed to zero. 0b1 If FPCR.AH is 0, denormalized single-precision and double-precision inputs to, and outputs from, floating-point instructions are flushed to zero. If FPCR.AH is 1, denormalized single-precision and double-precision outputs from floating-point instructions are flushed to zero. For more information, see <i>Flushing denormalized numbers to zero</i> in the Arm® Architecture Reference Manual for A-profile architecture and the pseudocode of the floating-point instructions.	0b0
[23:22]	RMode	Rounding Mode control field. 0b00 Round to Nearest (RN) mode. 0b01 Round towards Plus Infinity (RP) mode. 0b10 Round towards Minus Infinity (RM) mode. 0b11 Round towards Zero (RZ) mode. The specified rounding mode is used by both scalar and Advanced SIMD floating-point instructions. If FPCR.AH is 1, then the following instructions use Round to Nearest mode regardless of the value of this bit: • The FRECPPE, FRECPSP, FRECPX, FRSQRTE, and FRSQRTS instructions. • The BFCVT, BFCVTN, BFCVTN2, BFCVTNT, BFMLALB, and BFMLALT instructions.	0b00
[21:20]	Stride	This field has no function in AArch64 state, and non-zero values are ignored during execution in AArch64 state.	0b00

Bits	Name	Description	Reset
[19]	FZ16	Flushing denormalized numbers to zero control bit on half-precision data-processing instructions. 0b0 For some instructions, this bit disables flushing to zero of inputs and outputs that are half-precision denormalized numbers. 0b1 Flushing denormalized numbers to zero enabled. For some instructions that do not convert a half-precision input to a higher precision output, this bit enables flushing to zero of inputs and outputs that are half-precision denormalized numbers. The value of this bit applies to both scalar and Advanced SIMD floating-point half-precision calculations. For more information, see <i>Flushing denormalized numbers to zero</i> in the Arm® Architecture Reference Manual for A-profile architecture and the pseudocode of the floating-point instructions.	0b0
[18:16]	Len	This field has no function in AArch64 state, and non-zero values are ignored during execution in AArch64 state.	0b000
[15:3]	RES0	Reserved	RES0
[2]	NEP	Controls how the output elements other than the lowest element of the vector are determined for Advanced SIMD scalar instructions. 0b0 Does not affect how the output elements other than the lowest are determined for Advanced SIMD scalar instructions. 0b1 The output elements other than the lowest are taken from the following registers: - For 3-input scalar versions of the FMLA (by element) and FMLS (by element) instructions, the <Hd>, <Sd>, or <Dd> register. - For 3-input versions of the FMADD, FMSUB, FNMADD, and FNMSUB instructions, the <Ha>, <Sa>, or <Da> register. - For 2-input scalar versions of the FACGE, FACGT, FCMEQ (register), FCMGE (register), and FCMGT (register) instructions, the <Hm>, <Sm>, or <Dm> register. - For 2-input scalar versions of the FABD, FADD (scalar), FDIV (scalar), FMAX (scalar), FMAXNM (scalar), FMIN (scalar), FMINNM (scalar), FMUL (by element), FMUL (scalar), FMULX (by element), FMULX, FNMUL (scalar), FRECPs, FRSQRts, and FSUB (scalar) instructions, the <Hn>, <Sn>, or <Dn> register. - For 1-input scalar versions of the following instructions, the <Hd>, <Sd>, or <Dd> register: - The (vector) versions of the FCVtAS, FCVtAU, FCVtMS, FCVtMU, FCVtNS, FCVtNU, FCVtPS, and FCVtPU instructions. - The (vector, fixed-point) and (vector, integer) versions of the FCVtZS, FCVtZU, SCVtF, and UCVtF instructions. - The (scalar) versions of the FABS, FNEG, FRINT32X, FRINT32Z, FRINT64X, FRINT64Z, FRINTA, FRINTI, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ, and FSQRT instructions. - The (scalar, fixed-point) and (scalar, integer) versions of the SCVtF and UCVtF instructions. - The BFCVT, FCVT, FCVtXN, FRECPe, FRECPX, and FRSQRTE instructions.	0b0

Bits	Name	Description	Reset
[1]	AH	Alternate Handling. Controls alternate handling of floating-point numbers. The Arm architecture supports two models for handling some of the corner cases of the floating-point behaviors, such as the nature of flushing of denormalized numbers, the detection of tininess and other exceptions and a range of other behaviors. The value of the FPCR.AH bit selects between these models. For more information on the FPCR.AH bit, see <i>Flushing denormalized numbers to zero</i> in the Arm® Architecture Reference Manual for A-profile architecture, <i>Floating-point exceptions and exception traps</i> in the Arm® Architecture Reference Manual for A-profile architecture and the pseudocode of the floating-point instructions.	0b0
[0]	FIZ	Flush Inputs to Zero. Controls whether single-precision, double-precision and BFloat16 input operands that are denormalized numbers are flushed to zero. 0b0 The flushing to zero of single-precision and double-precision denormalized inputs to floating-point instructions not enabled by this control, but other factors might cause the input denormalized numbers to be flushed to zero. 0b1 Denormalized single-precision and double-precision inputs to most floating-point instructions flushed to zero. For more information, see <i>Flushing denormalized numbers to zero</i> in the Arm® Architecture Reference Manual for A-profile architecture and the pseudocode of the floating-point instructions.	0b0

Access

MRS <Xt>, FPCR

op0	op1	CRn	CRm	op2
0b11	0b011	0b0100	0b0100	0b000

MSR FPCR, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b011	0b0100	0b0100	0b000

Accessibility

MRS <Xt>, FPCR

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TFP == '1' then
        UNDEFINED;
    elsif !(EL2Enabled() && HCR_EL2.<E2H,TGE> == '11') && CPACR_EL1.FPEN != '11'
    then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x00);
        else
            AArch64.SystemAccessTrap(EL1, 0x07);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> == '11' && CPTR_EL2.FPEN != '11' then
            AArch64.SystemAccessTrap(EL2, 0x07);
        elsif EL2Enabled() && HCR_EL2.E2H == '1' && CPTR_EL2.FPEN == 'x0' then
            AArch64.SystemAccessTrap(EL2, 0x07);
        elsif EL2Enabled() && HCR_EL2.E2H != '1' && CPTR_EL2.TFP == '1' then
            AArch64.SystemAccessTrap(EL2, 0x07);
        elsif CPTR_EL3.TFP == '1' then
            if Halted() && EDSCR.SDD == '1' then

```

```

        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x07);
    else
        return FPCR;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TFP == '1' then
        UNDEFINED;
    elseif CPACR_EL1.FPEN == 'x0' then
        AArch64.SystemAccessTrap(EL1, 0x07);
    elseif EL2Enabled() && HCR_EL2.E2H != '1' && CPTR_EL2.TFP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif EL2Enabled() && HCR_EL2.E2H == '1' && CPTR_EL2.FPEN == 'x0' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif CPTR_EL3.TFP == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x07);
        else
            return FPCR;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TFP == '1' then
        UNDEFINED;
    elseif HCR_EL2.E2H == '0' && CPTR_EL2.TFP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif HCR_EL2.E2H == '1' && CPTR_EL2.FPEN == 'x0' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif CPTR_EL3.TFP == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x07);
        else
            return FPCR;
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TFP == '1' then
        AArch64.SystemAccessTrap(EL3, 0x07);
    else
        return FPCR;

```

MSR FPCR, <Xt>

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TFP == '1' then
        UNDEFINED;
    elseif !(EL2Enabled() && HCR_EL2.<E2H,TGE> == '11') && CPACR_EL1.FPEN != '11'
    then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x00);
        else
            AArch64.SystemAccessTrap(EL1, 0x07);
    elseif EL2Enabled() && HCR_EL2.<E2H,TGE> == '11' && CPTR_EL2.FPEN != '11' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif EL2Enabled() && HCR_EL2.E2H == '1' && CPTR_EL2.FPEN == 'x0' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif EL2Enabled() && HCR_EL2.E2H != '1' && CPTR_EL2.TFP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif CPTR_EL3.TFP == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x07);
        else
            FPCR = X[t];
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TFP == '1' then
        UNDEFINED;

```

```

elseif CPACR_EL1.FPEN == 'x0' then
    AArch64.SystemAccessTrap(EL1, 0x07);
elseif EL2Enabled() && HCR_EL2.E2H != '1' && CPTR_EL2.TFP == '1' then
    AArch64.SystemAccessTrap(EL2, 0x07);
elseif EL2Enabled() && HCR_EL2.E2H == '1' && CPTR_EL2.FPEN == 'x0' then
    AArch64.SystemAccessTrap(EL2, 0x07);
elseif CPTR_EL3.TFP == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x07);
    else
        FPCR = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TFP == '1' then
        UNDEFINED;
    elseif HCR_EL2.E2H == '0' && CPTR_EL2.TFP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif HCR_EL2.E2H == '1' && CPTR_EL2.FPEN == 'x0' then
        AArch64.SystemAccessTrap(EL2, 0x07);
    elseif CPTR_EL3.TFP == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x07);
        else
            FPCR = X[t];
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TFP == '1' then
        AArch64.SystemAccessTrap(EL3, 0x07);
    else
        FPCR = X[t];

```

A.1.25 ACTLR_EL2, Auxiliary Control Register (EL2)

Provides **IMPLEMENTATION DEFINED** configuration and control options for EL2.



Note

Arm recommends the contents of this register are updated to apply to EL0 when AArch64-HCR_EL2.{E2H, TGE} is {1, 1}, gaining configuration and control fields from the AArch64-ACTLR_EL1. This avoids the need for software to manage the contents of these register when switching between a Guest OS and a Host OS.

Configurations

If EL2 is not implemented, this register is RES0 from EL3.

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxx0	0x00	0xxx	xx00
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-25: AArch64_actlr_el2 bit assignments

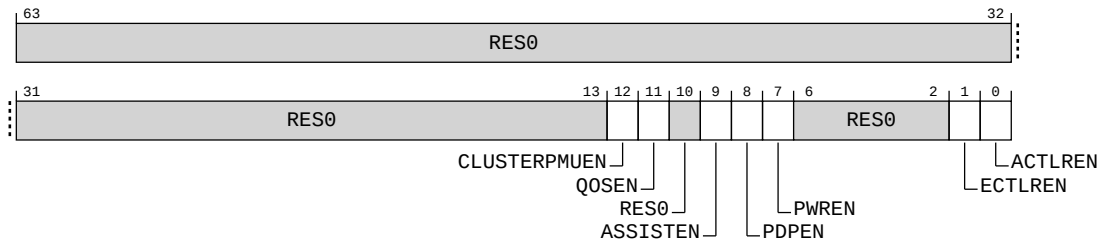


Table A-77: ACTLR_EL2 bit descriptions

Bits	Name	Description	Reset
[63:13]	RES0	Reserved	RES0
[12]	CLUSTERPMUEN	Cluster PMU Registers enable. Traps EL1 writes to cluster PMU registers IMP_CLUSTERPM* to EL2. Possible values of this bit are: 0b0 This control causes writes to IMP_CLUSTERPM* at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[11]	QOSEN	Cluster Bus QoS Registers enable. Traps EL1 writes to AArch64-IMP_CLUSTERBUSQOS_EL1 to EL2. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CLUSTERBUSQOS_EL1 at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[10]	RES0	Reserved	RES0
[9]	ASSISTEN	Assist feature enable. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUSYNCASSISTCR_EL1 at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0

Bits	Name	Description	Reset
[8]	PDPEN	Performance defined power enable. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUPMPDPCR_EL1 at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[7]	PWREN	Power Control Registers enable. Traps EL1 writes to power control registers AArch64-IMP_CPUPWRCTLR_EL1, AArch64-IMP_CLUSTERPWRCtrlr_EL1, AArch64-IMP_CLUSTERPWRDN_EL1 and IMP_CLUSTERL3*_EL1 to EL2. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUPWRCTLR_EL1, AArch64-IMP_CLUSTERPWRCtrlr_EL1, AArch64-IMP_CLUSTERPWRDN_EL1 and IMP_CLUSTERL3*_EL1 at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[6:2]	RES0	Reserved	RES0
[1]	ECTLREN	Extended Control Registers enable. Traps EL1 writes to AArch64-IMP_CPUECTLR_EL1, AArch64-IMP_CMPXECTLR_EL1 and AArch64-IMP_CLUSTERECTLR_EL1 to EL2. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUECTLR_EL1, AArch64-IMP_CMPXECTLR_EL1 and AArch64-IMP_CLUSTERECTLR_EL1 at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[0]	ACTLREN	Auxiliary Control Registers enable. Traps EL1 writes to AArch64-IMP_CPUACTLR_EL1, AArch64-IMP_CPUACTLR2_EL1, AArch64-IMP_CMPXACTLR_EL1 and AArch64-IMP_CLUSTERACTLR_EL1 to EL2. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUACTLR_EL1, AArch64-IMP_CPUACTLR2_EL1, AArch64-IMP_CMPXACTLR_EL1 and AArch64-IMP_CLUSTERACTLR_EL1 at EL1 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0

Access

MRS <Xt>, ACTLR_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b0001	0b0000	0b001

MSR ACTLR_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b0001	0b0000	0b001

Accessibility

MRS <Xt>, ACTLR_EL2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    return ACTLR_EL2;
elsif PSTATE.EL == EL3 then
    return ACTLR_EL2;
```

MSR ACTLR_EL2, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    ACTLR_EL2 = X[t];
elsif PSTATE.EL == EL3 then
    ACTLR_EL2 = X[t];
```

A.1.26 HACR_EL2, Hypervisor Auxiliary Control Register

Controls trapping to EL2 of **IMPLEMENTATION DEFINED** aspects of EL1 or EL0 operation.



Note

Arm recommends that the values in this register do not cause unnecessary traps to EL2 when AArch64-HCR_EL2.{E2H, TGE} == {1, 1}.

Configurations

If EL2 is not implemented, this register is RES0 from EL3.

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

Functional group

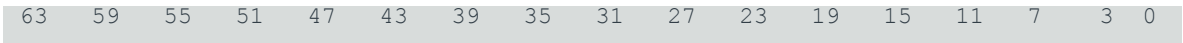
Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-26: AArch64_hacr_el2 bit assignments

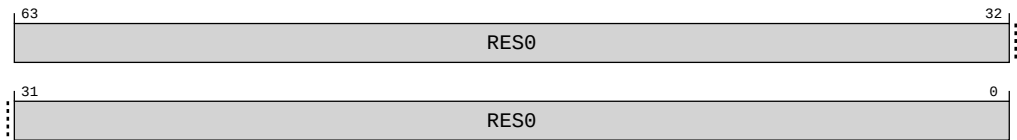


Table A-80: HACR_EL2 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, HACR_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b0001	0b0001	0b111

MSR HACR_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b0001	0b0001	0b111

Accessibility

MRS <Xt>, HACR_EL2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    return HACR_EL2;
elseif PSTATE.EL == EL3 then
    return HACR_EL2;
```

MSR HACR_EL2, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
```

```
    UNDEFINED;
    elsif PSTATE.EL == EL2 then
        HACR_EL2 = X[t];
    elsif PSTATE.EL == EL3 then
        HACR_EL2 = X[t];
```

A.1.27 AFSR0_EL2, Auxiliary Fault Status Register 0 (EL2)

Provides additional **IMPLEMENTATION DEFINED** fault status information for exceptions taken to EL2.

Configurations

If EL2 is not implemented, this register is RES0 from EL3.

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

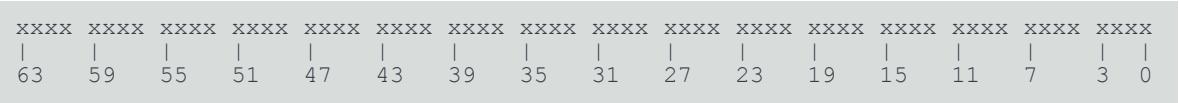
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-27: AArch64_afsr0_el2 bit assignments

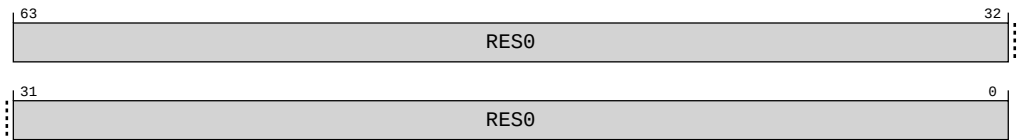


Table A-83: AFSR0_EL2 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic AFSRO_EL2 or AFSRO_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSRO_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b0101	0b0001	0b000

MSR AFSRO_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b0101	0b0001	0b000

MRS <Xt>, AFSRO_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b000

MSR AFSRO_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b000

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic AFSRO_EL2 or AFSRO_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSRO_EL2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    return AFSRO_EL2;
elsif PSTATE.EL == EL3 then
    return AFSRO_EL2;

```

MSR AFSRO_EL2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    AFSRO_EL2 = X[t];
elsif PSTATE.EL == EL3 then
    AFSRO_EL2 = X[t];

```

MRS <Xt>, AFSR0_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.AFSR0_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return AFSR0_EL1;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AFSR0_EL2;
    else
        return AFSR0_EL1;
elseif PSTATE.EL == EL3 then
    return AFSR0_EL1;

```

MSR AFSR0_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.AFSR0_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AFSR0_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AFSR0_EL2 = X[t];
    else
        AFSR0_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    AFSR0_EL1 = X[t];

```

A.1.28 AFSR1_EL2, Auxiliary Fault Status Register 1 (EL2)

Provides additional **IMPLEMENTATION DEFINED** fault status information for exceptions taken to EL2.

Configurations

If EL2 is not implemented, this register is RES0 from EL3.

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

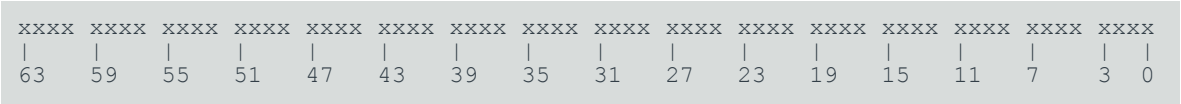
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-28: AArch64_afsr1_el2 bit assignments

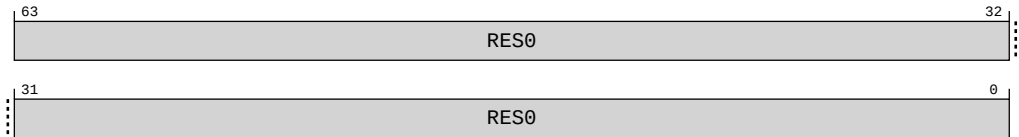


Table A-88: AFSR1_EL2 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic AFSR1_EL2 or AFSR1_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSR1_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b0101	0b0001	0b001

MSR AFSR1_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b0101	0b0001	0b001

MRS <Xt>, AFSR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b001

MSR AFSR1_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0001	0b001

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic AFSR1_EL2 or AFSR1_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AFSR1_EL2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    return AFSR1_EL2;
elseif PSTATE.EL == EL3 then
    return AFSR1_EL2;
```

MSR AFSR1_EL2, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    AFSR1_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    AFSR1_EL2 = X[t];
```

MRS <Xt>, AFSR1_EL1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.AFSR1_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return AFSR1_EL1;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AFSR1_EL2;
    else
        return AFSR1_EL1;
elseif PSTATE.EL == EL3 then
    return AFSR1_EL1;
```

MSR AFSR1_EL1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.AFSR1_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
```

```
        AFSR1_EL1 = X[t];
    elif PSTATE.EL == EL2 then
        if HCR_EL2.E2H == '1' then
            AFSR1_EL2 = X[t];
        else
            AFSR1_EL1 = X[t];
    elif PSTATE.EL == EL3 then
        AFSR1_EL1 = X[t];
```

A.1.29 AMAIR_EL2, Auxiliary Memory Attribute Indirection Register (EL2)

Provides **IMPLEMENTATION DEFINED** memory attributes for the memory regions specified by AArch64-MAIR_EL2.

Configurations

If EL2 is not implemented, this register is RES0 from EL3.

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

MAIR_EL2 is permitted to be cached in a TLB.

Figure A-29: AArch64_amair_el2 bit assignments

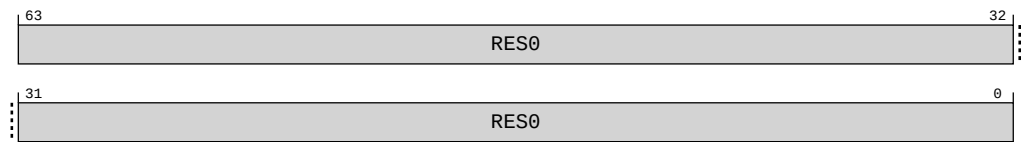


Table A-93: AMAIR_EL2 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic AMAIR_EL2 or AMAIR_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AMAIR_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0011	0b000

MSR AMAIR_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0011	0b000

MRS <Xt>, AMAIR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1010	0b0011	0b000

MSR AMAIR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1010	0b0011	0b000

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic AMAIR_EL2 or AMAIR_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, AMAIR_EL2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    return AMAIR_EL2;
elseif PSTATE.EL == EL3 then
    return AMAIR_EL2;

```

MSR AMAIR_EL2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then

```



```

    UNDEFINED;
elseif PSTATE.EL == EL2 then
    AMAIR_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    AMAIR_EL2 = X[t];

```

MRS <Xt>, AMAIR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TRVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.AMAIR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return AMAIR_EL1;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return AMAIR_EL2;
    else
        return AMAIR_EL1;
elseif PSTATE.EL == EL3 then
    return AMAIR_EL1;

```

MSR AMAIR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.AMAIR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AMAIR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        AMAIR_EL2 = X[t];
    else
        AMAIR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    AMAIR_EL1 = X[t];

```

A.1.30 IMP_ATCR_EL2, CPU Auxiliary Translation Control Register

This register controls the values of the PBHA signals for memory accesses generated by translation table walks in the EL2 translation regime.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

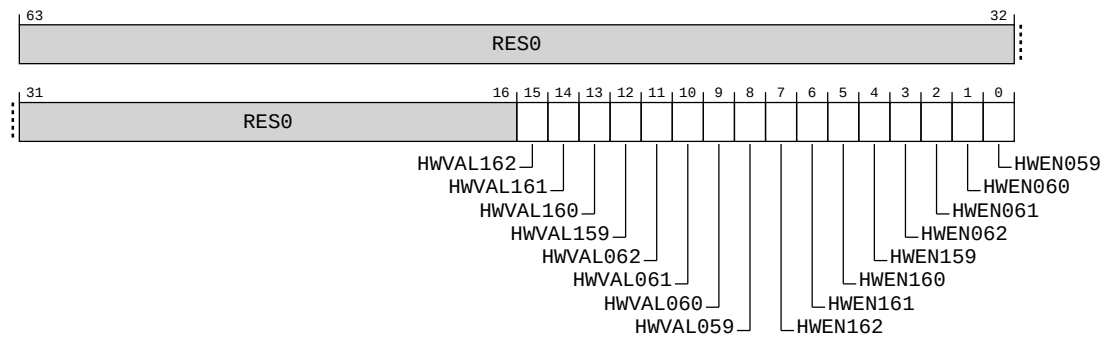
Generic System Control

Access type

See bit descriptions

Bit descriptions

When AArch64-HCR_EL2.E2H == 1

Figure A-30: AArch64_imp_atcr_el2 bit assignments**Table A-98: IMP_ATCR_EL2 bit descriptions**

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15]	HWVAL162	Value of PBHA[3] on memory accesses due to translation table walks using TTBR1_EL2 if HWEN162 is set.	0b0
[14]	HWVAL161	Value of PBHA[2] on memory accesses due to translation table walks using TTBR1_EL2 if HWEN161 is set.	0b0
[13]	HWVAL160	Value of PBHA[1] on memory accesses due to translation table walks using TTBR1_EL2 if HWEN160 is set.	0b0
[12]	HWVAL159	Value of PBHA[0] on memory accesses due to translation table walks using TTBR1_EL2 if HWEN159 is set.	0b0
[11]	HWVAL062	Value of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN062 is set.	0b0
[10]	HWVAL061	Value of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN061 is set.	0b0
[9]	HWVAL060	Value of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN060 is set.	0b0
[8]	HWVAL059	Value of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN059 is set.	0b0
[7]	HWEN162	Enable use of PBHA[3] on memory accesses due to translation table walks using TTBR1_EL2. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[6]	HWEN161	Enable use of PBHA[2] on memory accesses due to translation table walks using TTBR1_EL2. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[5]	HWEN160	Enable use of PBHA[1] on memory accesses due to translation table walks using TTBR1_EL2. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[4]	HWEN159	Enable use of PBHA[0] on memory accesses due to translation table walks using TTBR1_EL2. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0
[3]	HWEN062	Enable use of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[2]	HWEN061	Enable use of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0

Bits	Name	Description	Reset
[1]	HWEN060	Enable use of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[0]	HWEN059	Enable use of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0

When AArch64-HCR_EL2.E2H == 0

This view of the register is only valid from Armv8.1 when HCR_EL2.E2H is 1.

Any of the bits in TCR_EL2 are permitted to be cached in a TLB.

Figure A-31: AArch64_imp_atcr_el2 bit assignments

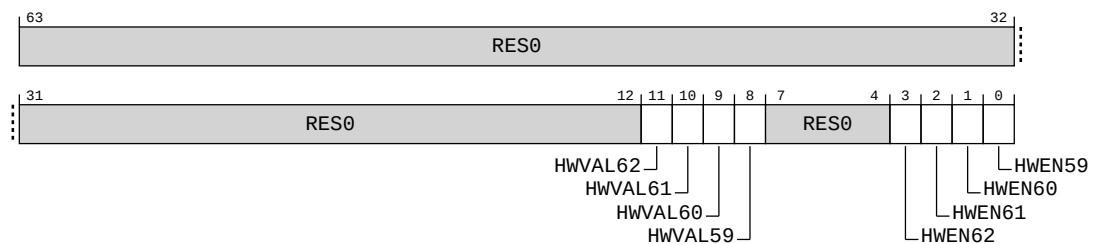


Table A-99: IMP_ATCR_EL2 bit descriptions

Bits	Name	Description	Reset
[63:12]	RES0	Reserved	RES0
[11]	HWVAL62	Value of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN62 is set.	0b0
[10]	HWVAL61	Value of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN61 is set.	0b0
[9]	HWVAL60	Value of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN60 is set.	0b0
[8]	HWVAL59	Value of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL2 if HWEN59 is set.	0b0
[7:4]	RES0	Reserved	RES0
[3]	HWEN62	Enable use of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[2]	HWEN61	Enable use of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[1]	HWEN60	Enable use of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[0]	HWEN59	Enable use of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL2. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0

Access

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic IMP_ATCR_EL2 or IMP_ATCR_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, S3_4_C15_C7_0

op0	op1	CRn	CRm	op2
0b11	0b100	0b1111	0b0111	0b000

MSR S3_4_C15_C7_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b1111	0b0111	0b000

MRS <Xt>, S3_0_C15_C7_0

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0111	0b000

MSR S3_0_C15_C7_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0111	0b000

Accessibility

When AArch64-HCR_EL2.E2H is 1, without explicit synchronization, access from EL2 using the mnemonic IMP_ATCR_EL2 or IMP_ATCR_EL1 are not guaranteed to be ordered with respect to accesses using the other mnemonic.

MRS <Xt>, S3_4_C15_C7_0

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    return IMP_ATCR_EL2;
elseif PSTATE.EL == EL3 then
    return IMP_ATCR_EL2;

```

MSR S3_4_C15_C7_0, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    IMP_ATCR_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_ATCR_EL2 = X[t];

```

MRS <Xt>, S3_0_C15_C7_0

```

if PSTATE.EL == EL0 then

```

```

    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.TVMM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_ATCR_EL1;
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        return IMP_ATCR_EL2;
    else
        return IMP_ATCR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_ATCR_EL1;

```

MSR S3_0_C15_C7_0, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.TVM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        IMP_ATCR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if HCR_EL2.E2H == '1' then
        IMP_ATCR_EL2 = X[t];
    else
        IMP_ATCR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_ATCR_EL1 = X[t];

```

A.1.31 IMP_AVTCR_EL2, CPU Auxiliary Virtualization Translation Control Register

This register controls the values of the PBHA signals for memory accesses generated by stage 2 translation table walks.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

```

xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx 0000 0000 0000 0000

```



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-32: AArch64_imp_avtcr_el2 bit assignments

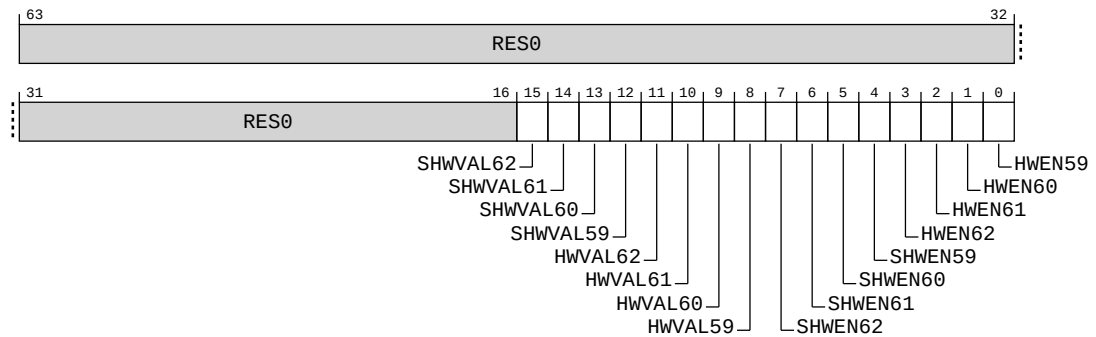


Table A-104: IMP_AVTCR_EL2 bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15]	SHWVAL62	Value of PBHA[3] on memory accesses due to translation table walks using VSTTBR_EL2 if SHWEN62 is set.	0b0
[14]	SHWVAL61	Value of PBHA[2] on memory accesses due to translation table walks using VSTTBR_EL2 if SHWEN61 is set.	0b0
[13]	SHWVAL60	Value of PBHA[1] on memory accesses due to translation table walks using VSTTBR_EL2 if SHWEN60 is set.	0b0
[12]	SHWVAL59	Value of PBHA[0] on memory accesses due to translation table walks using VSTTBR_EL2 if SHWEN59 is set.	0b0
[11]	HWVAL62	Value of PBHA[3] on memory accesses due to translation table walks using VTTBR_EL2 if HWEN62 is set.	0b0
[10]	HWVAL61	Value of PBHA[2] on memory accesses due to translation table walks using VTTBR_EL2 if HWEN61 is set.	0b0
[9]	HWVAL60	Value of PBHA[1] on memory accesses due to translation table walks using VTTBR_EL2 if HWEN60 is set.	0b0
[8]	HWVAL59	Value of PBHA[0] on memory accesses due to translation table walks using VTTBR_EL2 if HWEN59 is set.	0b0
[7]	SHWEN62	Enable use of PBHA[3] on memory accesses due to translation table walks using VSTTBR_EL2. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[6]	SHWEN61	Enable use of PBHA[2] on memory accesses due to translation table walks using VSTTBR_EL2. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[5]	SHWEN60	Enable use of PBHA[1] on memory accesses due to translation table walks using VSTTBR_EL2. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[4]	SHWEN59	Enable use of PBHA[0] on memory accesses due to translation table walks using VSTTBR_EL2. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0

Bits	Name	Description	Reset
[3]	HWEN62	Enable use of PBHA[3] on memory accesses due to translation table walks using VTTBR_EL2. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[2]	HWEN61	Enable use of PBHA[2] on memory accesses due to translation table walks using VTTBR_EL2. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[1]	HWEN60	Enable use of PBHA[1] on memory accesses due to translation table walks using VTTBR_EL2. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[0]	HWEN59	Enable use of PBHA[0] on memory accesses due to translation table walks using VTTBR_EL2. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0

Access

MRS <Xt>, S3_4_C15_C7_1

op0	op1	CRn	CRm	op2
0b11	0b100	0b1111	0b0111	0b001

MSR S3_4_C15_C7_1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b1111	0b0111	0b001

Accessibility

MRS <Xt>, S3_4_C15_C7_1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    return IMP_AVTCR_EL2;
elseif PSTATE.EL == EL3 then
    return IMP_AVTCR_EL2;

```

MSR S3_4_C15_C7_1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    IMP_AVTCR_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_AVTCR_EL2 = X[t];

```

A.1.32 ACTLR_EL3, Auxiliary Control Register (EL3)

Provides **IMPLEMENTATION DEFINED** configuration and control options for EL3.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxx0	0x00	0xxx	xx00
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-33: AArch64_actlr_el3 bit assignments

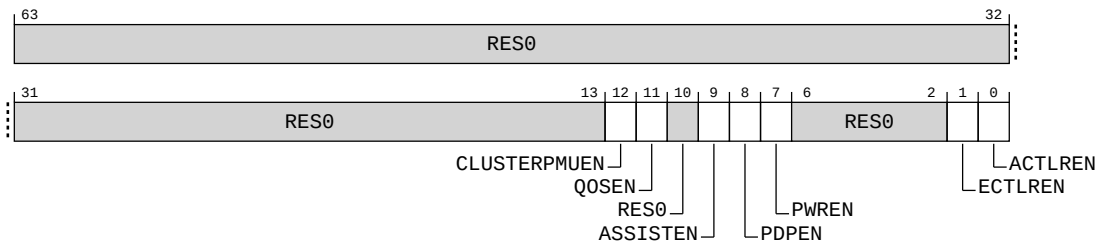


Table A-107: ACTLR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:13]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[12]	CLUSTERPMUEN	Cluster PMU Registers enable. Traps EL1 and EL2 writes to cluster PMU registers IMP_CLUSTERPM* to EL3. Possible values of this bit are: 0b0 This control causes writes to IMP_CLUSTERPM* at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[11]	QOSEN	Cluster Bus QoS Registers enable. Traps EL1 and EL2 writes to AArch64-IMP_CLUSTERBUSQOS_EL1 to EL3. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CLUSTERBUSQOS_EL1 at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[10]	RES0	Reserved	RES0
[9]	ASSISTEN	Assist feature enable. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUSYNCASSISTCR_EL1 at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[8]	PDPEN	Performance defined power enable. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUPPMPDPCR_EL1 at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[7]	PWREN	Power Control Registers enable. Traps EL1 and EL2 writes to power control registers AArch64-IMP_CPUPWRCTLR_EL1, AArch64-IMP_CLUSTERPWRCtrlr_EL1, AArch64-IMP_CLUSTERPWRDN_EL1 and IMP_CLUSTERL3*_EL1 to EL3. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUPWRCTLR_EL1, AArch64-IMP_CLUSTERPWRCtrlr_EL1, AArch64-IMP_CLUSTERPWRDN_EL1 and IMP_CLUSTERL3*_EL1 at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0
[6:2]	RES0	Reserved	RES0
[1]	ECTLREN	Extended Control Registers enable. Traps EL1 and EL2 writes to AArch64-IMP_CPUECTLR_EL1, AArch64-IMP_CMPXECTLR_EL1 and AArch64-IMP_CLUSTERECTLR_EL1 to EL3. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUECTLR_EL1, AArch64-IMP_CMPXECTLR_EL1 and AArch64-IMP_CLUSTERECTLR_EL1 at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0

Bits	Name	Description	Reset
[0]	ACTLREN	Auxiliary Control Registers enable. Traps EL1 and EL2 writes to AArch64-IMP_CPUACTLR_EL1, AArch64-IMP_CPUACTLR2_EL1, AArch64-IMP_CMPXACTLR_EL1 and AArch64-IMP_CLUSTERACTLR_EL1 to EL3. Possible values of this bit are: 0b0 This control causes writes to AArch64-IMP_CPUACTLR_EL1, AArch64-IMP_CPUACTLR2_EL1, AArch64-IMP_CMPXACTLR_EL1 and AArch64-IMP_CLUSTERACTLR_EL1 at EL1 and EL2 to be trapped. 0b1 This control does not cause any instructions to be trapped.	0b0

Access

MRS <Xt>, ACTLR_EL3

op0	op1	CRn	CRm	op2
0b11	0b110	0b0001	0b0000	0b001

MSR ACTLR_EL3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b0001	0b0000	0b001

Accessibility

MRS <Xt>, ACTLR_EL3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return ACTLR_EL3;
```

MSR ACTLR_EL3, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    ACTLR_EL3 = X[t];
```

A.1.33 AFSR0_EL3, Auxiliary Fault Status Register 0 (EL3)

Provides additional **IMPLEMENTATION DEFINED** fault status information for exceptions taken to EL3.

Configurations

This register is available in all configurations.

Attributes

Width

64

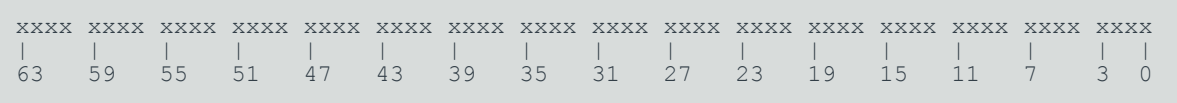
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-34: AArch64_afsr0_el3 bit assignments

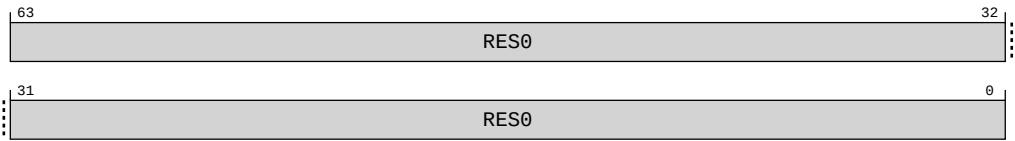


Table A-110: AFSR0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, AFSR0_EL3

op0	op1	CRn	CRm	op2
0b11	0b110	0b0101	0b0001	0b000

MSR AFSRO_EL3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b0101	0b0001	0b000

Accessibility

MRS <Xt>, AFSRO_EL3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return AFSRO_EL3;
```

MSR AFSRO_EL3, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    AFSRO_EL3 = X[t];
```

A.1.34 AFSR1_EL3, Auxiliary Fault Status Register 1 (EL3)

Provides additional **IMPLEMENTATION DEFINED** fault status information for exceptions taken to EL3.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-35: AArch64_afsr1_el3 bit assignments

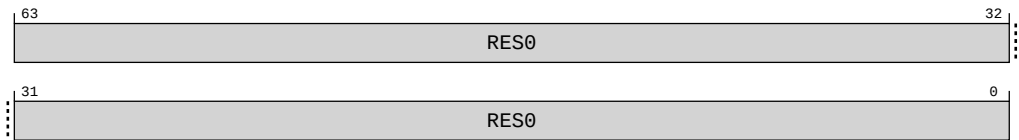


Table A-113: AFSR1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, AFSR1_EL3

op0	op1	CRn	CRm	op2
0b11	0b110	0b0101	0b0001	0b001

MSR AFSR1_EL3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b0101	0b0001	0b001

Accessibility

MRS <Xt>, AFSR1_EL3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return AFSR1_EL3;
```

MSR AFSR1_EL3, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
```

```
elseif PSTATE.EL == EL3 then
    AFSR1_EL3 = X[t];
```

A.1.35 AMAIR_EL3, Auxiliary Memory Attribute Indirection Register (EL3)

Provides **IMPLEMENTATION DEFINED** memory attributes for the memory regions specified by AArch64-MAIR_EL3.

Configurations

This register is available in all configurations.

Attributes

Width

64

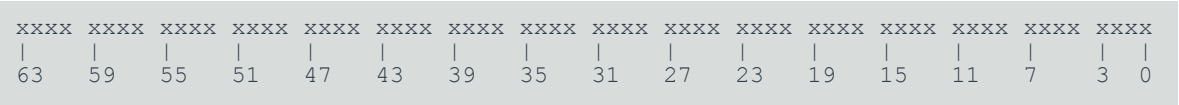
Functional group

Generic System Control

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

AMAIR_EL3 is permitted to be cached in a TLB.

Figure A-36: AArch64_amair_el3 bit assignments

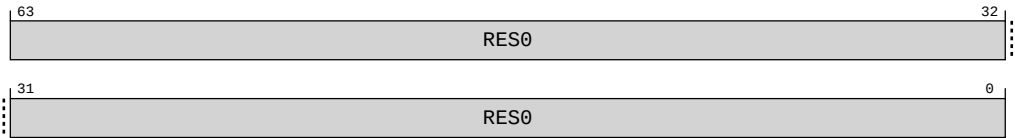


Table A-116: AMAIR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, AMAIR_EL3

op0	op1	CRn	CRm	op2
0b11	0b110	0b1010	0b0011	0b000

MSR AMAIR_EL3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1010	0b0011	0b000

Accessibility

MRS <Xt>, AMAIR_EL3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return AMAIR_EL3;
```

MSR AMAIR_EL3, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    AMAIR_EL3 = X[t];
```

A.1.36 RMR_EL3, Reset Management Register (EL3)

If EL3 is the implemented and this register is implemented:

- A write to the register at EL3 can request a Warm reset.
- If EL3 can use all Execution states, this register specifies the Execution state that the PE boots into on a Warm reset.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group
Generic System Control

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx01
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-37: AArch64_rmr_el3 bit assignments

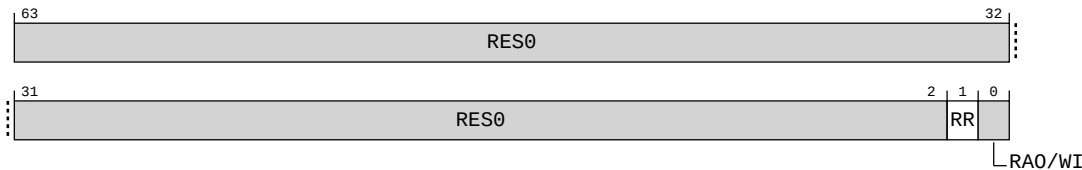


Table A-119: RMR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:2]	RES0	Reserved	RES0
[1]	RR	Reset Request. Setting this bit to 1 requests a Warm reset.	0b0
[0]	RAO/WI	Reserved	RAO/WI

Access
MRS <Xt>, RMR_EL3

op0	op1	CRn	CRm	op2
0b11	0b110	0b1100	0b0000	0b010

MSR RMR_EL3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1100	0b0000	0b010

Accessibility

MRS <Xt>, RMR_EL3

```
if PSTATE.EL == EL3 then
    return RMR_EL3;
else
    UNDEFINED;
```

MSR RMR_EL3, <Xt>

```
if PSTATE.EL == EL3 then
    RMR_EL3 = X[t];
else
    UNDEFINED;
```

A.1.37 IMP_ISIDE_DATA0_EL3, RAMINDEX Instruction Data register 0

Returns the data from a RAMINDEX instruction using RAMID=0x0,0x1

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L1 instruction cache tag

Figure A-38: AArch64_imp_iside_data0_el3 bit assignments

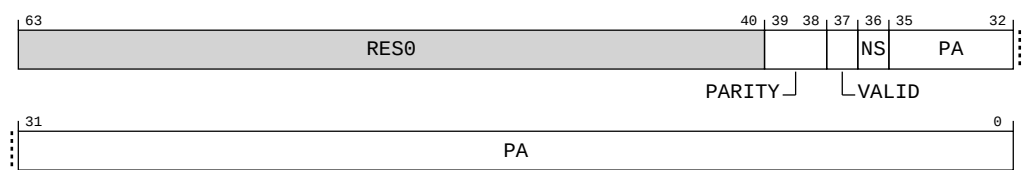


Table A-122: IMP_ISIDE_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:40]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[39:38]	PARITY	Parity bit	xx
[37]	VALID	Validity bit	x
[36]	NS	Non-secure identifier for the physical address	x
[35:0]	PA	Physical address [47:12]	36 {x}

When L1 instruction cache data

Figure A-39: AArch64_imp_iside_data0_el3 bit assignments

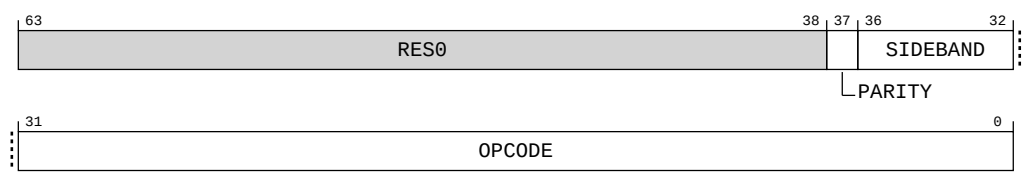


Table A-123: IMP_ISIDE_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:38]	RES0	Reserved	RES0
[37]	PARITY	Parity bit	x
[36:32]	SIDEBAND	If this field is not 5'b10000 or 5'b10110 then AArch64-IMP_ISIDE_DATA0_EL3.OPCODE represents the AArch64 instruction opcode	5 {x}
[31:0]	OPCODE	Aarch64 opcode at {Aarch64-RAMINDEX.VA[48:3], 1'b0} if AArch64-IMP_ISIDE_DATA0_EL3.SIDEBAND is not 5'b10000 or 5'b10110	32 {x}

Access

MRS <Xt>, S3_6_C15_CO_0

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0000	0b000

Accessibility

MRS <Xt>, S3_6_C15_CO_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_ISIDE_DATA0_EL3;
```

A.1.38 IMP_ISIDE_DATA1_EL3, RAMINDEX Instruction Data register 1

Returns the data from a RAMINDEX instruction using RAMID=0x0,0x1

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L1 instruction cache tag

Figure A-40: AArch64_imp_iside_data1_el3 bit assignments

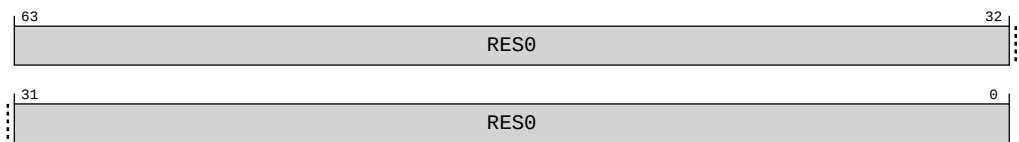


Table A-125: IMP_ISIDE_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

When L1 instruction cache data

Figure A-41: AArch64_imp_iside_data1_el3 bit assignments

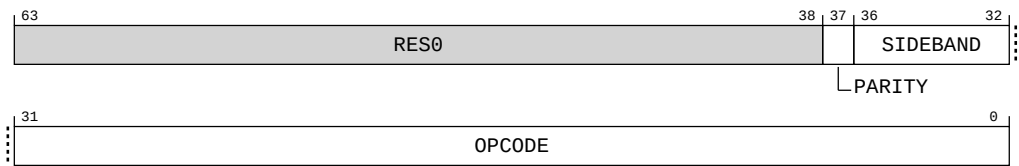


Table A-126: IMP_ISIDE_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:38]	RES0	Reserved	RES0
[37]	PARITY	Parity bit	x

Bits	Name	Description	Reset
[36:32]	SIDEBAND	If this field is not 5'b10000 or 5'b10110 then AArch64-IMP_ISIDE_DATA1_EL3.OPCODE represents the AArch64 instruction opcode	5{x}
[31:0]	OPCODE	Aarch64 opcode at {Aarch64-RAMINDEX.VA[48:3], 1'b1} if AArch64-IMP_ISIDE_DATA1_EL3.SIDEBAND is not 5'b10000 or 5'b10110	32{x}

Access

MRS <Xt>, S3_6_C15_CO_1

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0000	0b001

Accessibility

MRS <Xt>, S3_6_C15_CO_1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_ISIDE_DATA1_EL3;
```

A.1.39 IMP_ISIDE_DATA2_EL3, RAMINDEX Instruction Data register 2

Returns the data from a RAMINDEX instruction using RAMID=0x0,0x1

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-42: AArch64_imp_iside_data2_el3 bit assignments

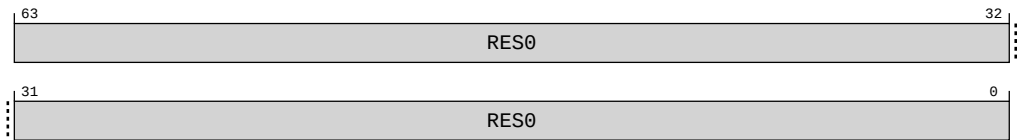


Table A-128: IMP_ISIDE_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, S3_6_C15_C0_2

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0000	0b010

Accessibility

MRS <Xt>, S3_6_C15_C0_2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_ISIDE_DATA2_EL3;
```

A.1.40 IMP_MMU_DATA0_EL3, RAMINDEX TLB Data register 0

Returns the data from a RAMINDEX instruction using RAMID=0x18

Configurations

This register is available in all configurations.

Attributes

Width
64

Functional group
Generic System Control

Access type
See bit descriptions

Bit descriptions
When L2 TLB TCSP (small pages)

Figure A-43: AArch64_imp_mmu_data0_el3 bit assignments

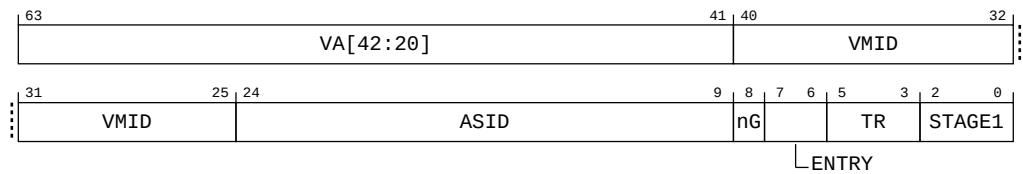


Table A-130: IMP_MMU_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:41]	VA[42:20]	Virtual Address [42:20]	23 {x}
[40:25]	VMID	VMID	16 {x}
[24:9]	ASID	ASID	16 {x}
[8]	nG	Non-Global	x
[7:6]	ENTRY	Entry Size 0b00 4KB 0b01 16KB 0b10 64KB 0b11 Reserved	xx

Bits	Name	Description	Reset
[5:3]	TR	Translation Regime 0b000 Reserved 0b001 Secure EL1 0b010 Secure EL2 0b011 Secure EL3 0b100 Reserved 0b101 Non-Secure EL1 0b110 Non-Secure EL2 0b111 Reserved	xxx
[2:0]	STAGE1	Stage 1 description encoding 0b000 Entry is found at level 3 of 4KB granule (stage1 size = 4KB) 0b001 Entry is found at level 3 of 16KB granule (stage1 size = 16KB) 0b010 Entry is found at level 3 of 64KB granule (stage1 size = 64KB) 0b011 Reserved 0b100 Entry is found at level 2 of 4KB granule (stage1 size = 2MB) 0b101 Entry is found at level 2 of 16KB granule (stage1 size = 32MB) 0b110 Entry is found at level 2 of 64KB granule (stage1 size = 512MB) 0b111 Entry is found at level 1 of 4KB granule (stage1 size = 1GB)	xxx

When L2 TLB TCMP (medium pages)

Figure A-44: AArch64_imp_mmu_data0_el3 bit assignments

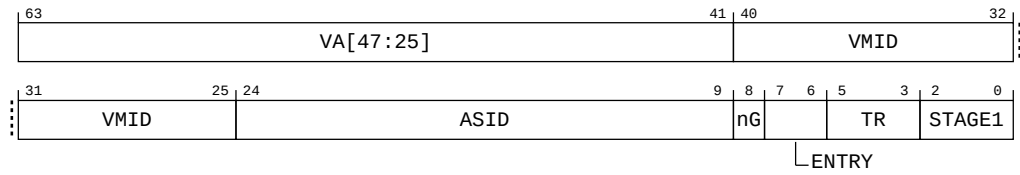


Table A-131: IMP_MMU_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:41]	VA[47:25]	Virtual Address [47:25]	23 {x}
[40:25]	VMID	VMID	16 {x}
[24:9]	ASID	ASID	16 {x}
[8]	nG	Non-Global	x
[7:6]	ENTRY	Entry Size 0b00 2MB 0b01 32MB 0b10 512MB 0b11 Normal entries: Reserved, IPA entries: 2MB originating from a 1GB block	xx
[5:3]	TR	Translation Regime 0b000 Reserved 0b001 Secure EL1 0b010 Secure EL2 0b011 Secure EL3 0b100 Reserved 0b101 Non-Secure EL1 0b110 Non-Secure EL2 0b111 Reserved	xxx

Bits	Name	Description	Reset
[2:0]	STAGE1	Stage 1 description encoding 0b000 Entry is found at level 3 of 4KB granule (stage1 size = 4KB) 0b001 Entry is found at level 3 of 16KB granule (stage1 size = 16KB) 0b010 Entry is found at level 3 of 64KB granule (stage1 size = 64KB) 0b011 Entry is an IPA to PA translation 0b100 Entry is found at level 2 of 4KB granule (stage1 size = 2MB) 0b101 Entry is found at level 2 of 16KB granule (stage1 size = 32MB) 0b110 Entry is found at level 2 of 64KB granule (stage1 size = 512MB) 0b111 Entry is found at level 1 of 4KB granule (stage1 size = 1GB)	xxx

Access

MRS <Xt>, S3_6_C15_CO_3

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0000	0b011

Accessibility

MRS <Xt>, S3_6_C15_CO_3

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_MMU_DATA0_EL3;

```

A.1.41 IMP_MMU_DATA1_EL3, RAMINDEX TLB Data register 1

Returns the data from a RAMINDEX instruction using RAMID=0x18

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L2 TLB TCSP (small pages)

Figure A-45: AArch64_imp_mmu_data1_el3 bit assignments

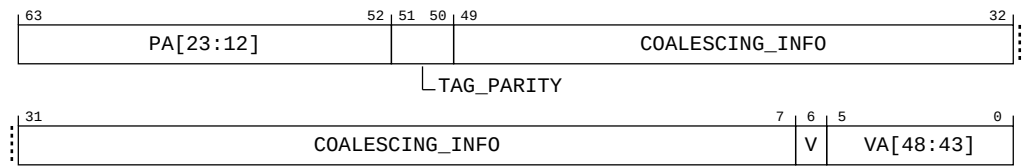


Table A-133: IMP_MMU_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:52]	PA[23:12]	Physical Address[23:12] In case of a single entry, this represents the output physical address of the entry. In case of a coalesced entry, when IMP_CPUECTLR_EL1[63:62] = 00, the [17:12] range represents the PA offset of cluster entry 0, while the [23:18] range represents the common part of the output physical address. In case of a coalesced entry, when IMP_CPUECTLR_EL1[63:62] = 01, the [22:12] range represents the PA offset of cluster entry 0, while the [23:23] range represents the common part of the output physical address. Note that the PA offset of remaining cluster entries is stored in COALESCING_INFO field.	12 {x}
[51:50]	TAG_PARITY	Tag Parity	xx

Bits	Name	Description	Reset
[49:7]	COALESCING_INFO	Coalescing information Decoding of coalescing information depends on IMP_CPUCTLR_EL1[63:62] bits When IMP_CPUCTLR_EL1[63:62] = 00: - COALESCING_INFO[42:8] represents the PA offsets of entries 1 to 7. Note that the PA offset of cluster entry 0 is stored in PA field. - COALESCING_INFO[7] represents the TLB entry type (i.e. single or coalesced). When set to 1, the TLB entry is coalesced, otherwise, the TLB entry contains a single translation. - COALESCING_INFO[6:0] represents the validity vector of cluster entries 1 to 7. Note that the validity of cluster entry 0 is stored in V field. When IMP_CPUCTLR_EL1[63:62] = 01: - COALESCING_INFO[42:4] represents the PA offsets of entries 1 to 3. Note that the PA offset of cluster entry 0 is stored in PA field. - COALESCING_INFO[3] represents the TLB entry type (i.e. single or coalesced). When set to 1, the TLB entry is coalesced, otherwise, the TLB entry contains a single translation. - COALESCING_INFO[2:0] represents the validity vector of cluster entries 1 to 3. Note that the validity of cluster entry 0 is stored in V field.	43 {x}
[6]	V	Validity bit In case of a single entry, this represents the validity of the entry. In case of a coalesced entry, this represents the validity of the cluster entry 0. Note that the validity of remaining cluster entries is stored in COALESCING_INFO field.	x
[5:0]	VA[48:43]	Virtual Address [48:43]	6 {x}

When L2 TLB TCMP (medium pages) and AArch64-IMP_MMU_DATA1_EL3.CACHEABILITY == 0b0

Figure A-46: AArch64_imp_mmu_data1_el3 bit assignments

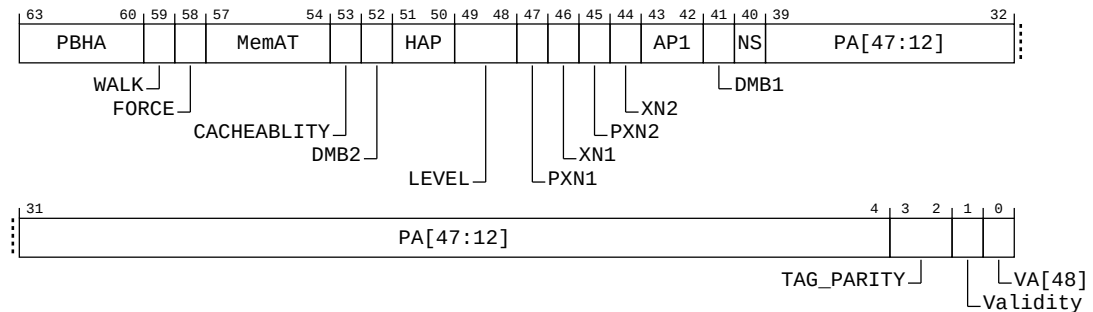


Table A-134: IMP_MMU_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:60]	PBHA	PBHA	xxxx
[59]	WALK	Walk cache entry	x
[58]	FORCE	Force write-back	x

Bits	Name	Description	Reset
[57:54]	MemAT	Memory attributes information 0b0000 Normal memory - Non-Cacheable due to stage 1. Originally not outer cacheable (WB/ WT) 0b0001 Device nGnRnE with stage 1 being device 0b0010 Device nGnRnE with stage 1 being normal non-cacheable 0b0011 Device nGnRnE with stage 1 being normal cacheable 0b0100 Normal memory - Non-Cacheable due to stage 1. Originally outer cacheable (WB/ WT) 0b0101 Device nGnRE with stage 1 being device 0b0110 Device nGnRE with stage 1 being normal non-cacheable 0b0111 Device nGnRE with stage 1 being normal cacheable 0b1000 Normal memory - Non-Cacheable due to stage 2. Originally not outer cacheable (WB/ WT) 0b1001 Device nGRE with stage 1 being device 0b1010 Device nGRE with stage 1 being normal non-cacheable 0b1011 Device nGRE with stage 1 being normal cacheable 0b1100 Normal memory - Non-Cacheable due to stage 2. Originally outer cacheable (WB/ WT) 0b1101 Device GRE with stage 1 being device 0b1110 Device GRE with stage 1 being normal non-cacheable 0b1111 Device GRE with stage 1 being normal cacheable	xxxx
[53]	CACHEABILITY	Cacheability	x
[52]	DMB2	Stage 2 dirty bit Modifier	x
[51:50]	HAP	Stage 2 Access Permissions	xx

Bits	Name	Description	Reset
[49:48]	LEVEL	Stage 2 level 0b00 Level 3 0b01 Level 2 0b10 Level 1 0b11 Level 0	xx
[47]	PXN1	Stage 1 privileged Execute-Never	x
[46]	XN1	Stage 1 Execute-Never	x
[45]	PXN2	Stage 2 privileged Execute-Never	x
[44]	XN2	Stage 2 Execute-Never	x
[43:42]	AP1	Stage 1 Access Permission	xx
[41]	DMB1	Stage 1 dirty bit Modifier	x
[40]	NS	Non-Secure which determines whether the physical address is in secure spece or non-secure space	x
[39:4]	PA[47:12]	Physical Address[47:12]	36{x}
[3:2]	TAG_PARITY	Tag Parity	xx
[1]	Validity	Valididy	x
[0]	VA[48]	Virtual Address [48]	x

When L2 TLB TCMP (medium pages) and AArch64-IMP_MMU_DATA1_EL3.CACHEABLITY == 0b1

Figure A-47: AArch64_imp_mmu_data1_el3 bit assignments

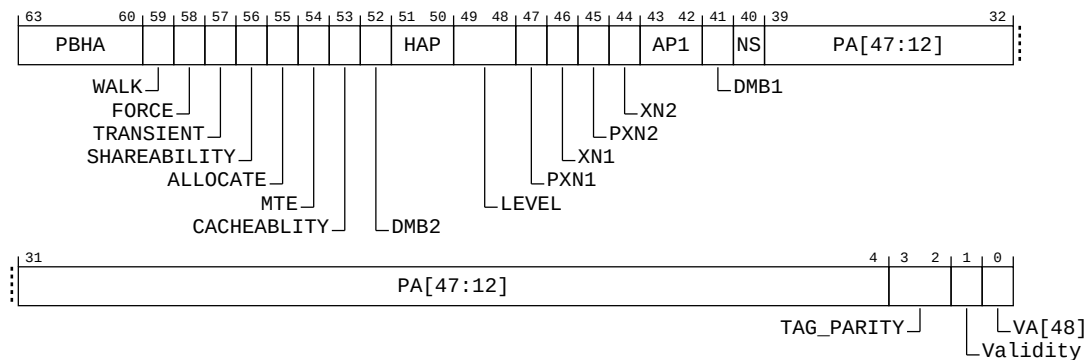


Table A-135: IMP_MMU_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:60]	PBHA	PBHA	xxxx
[59]	WALK	Walk cache entry	x
[58]	FORCE	Force write-back	x

Bits	Name	Description	Reset
[57]	TRANSIENT	Inner transient hint	x
[56]	SHAREABILITY	Shareability	x
[55]	ALLOCATE	Allocate hint	x
[54]	MTE	MTE tag	x
[53]	CACHEABILITY	Cacheability	x
[52]	DMB2	Stage 2 dirty bit Modifier	x
[51:50]	HAP	Stage 2 Access Permissions	xx
[49:48]	LEVEL	Stage 2 level 0b00 Level 3 0b01 Level 2 0b10 Level 1 0b11 Level 0	xx
[47]	PXN1	Stage 1 privileged Execute-Never	x
[46]	XN1	Stage 1 Execute-Never	x
[45]	PXN2	Stage 2 privileged Execute-Never	x
[44]	XN2	Stage 2 Execute-Never	x
[43:42]	AP1	Stage 1 Access Permission	xx
[41]	DMB1	Stage 1 dirty bit Modifier	x
[40]	NS	Non-Secure which determines whether the physical address is in secure spece or non-secure space	x
[39:4]	PA[47:12]	Physical Address[47:12]	36{x}
[3:2]	TAG_PARITY	Tag Parity	xx
[1]	Validity	Valididy	x
[0]	VA[48]	Virtual Address [48]	x

Access

MRS <Xt>, S3_6_C15_CO_4

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0000	0b100

Accessibility

MRS <Xt>, S3_6_C15_CO_4

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;

```

```
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_MMU_DATA1_EL3;
```

A.1.42 IMP_MMU_DATA2_EL3, RAMINDEX TLB Data register 2

Returns the data from a RAMINDEX instruction using RAMID=0x18

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L2 TLB TCSP (small pages) and AArch64-IMP_MMU_DATA1_EL3.CACHEABILITY == 0b0

Figure A-48: AArch64_imp_mmu_data2_el3 bit assignments

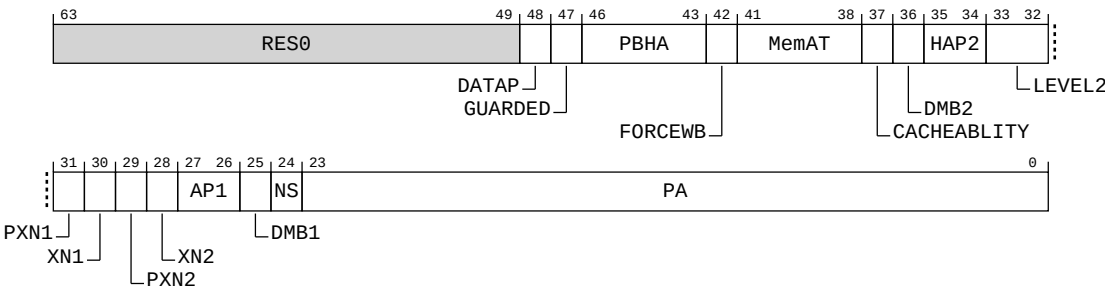


Table A-137: IMP_MMU_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:49]	RES0	Reserved	RES0
[48]	DATAP	Data Parity	x
[47]	GUARDED	Guarded page	x
[46:43]	PBHA	PBHA	xxxx
[42]	FORCEWB	Force Write-back	x

Bits	Name	Description	Reset
[41:38]	MemAT	Memory attributes information 0b0000 Normal memory - Non-Cacheable due to stage 1. Originally not outer cacheable (WB/ WT) 0b0001 Device nGnRnE with stage 1 being device 0b0010 Device nGnRnE with stage 1 being normal non-cacheable 0b0011 Device nGnRnE with stage 1 being normal cacheable 0b0100 Normal memory - Non-Cacheable due to stage 1. Originally outer cacheable (WB/ WT) 0b0101 Device nGnRE with stage 1 being device 0b0110 Device nGnRE with stage 1 being normal non-cacheable 0b0111 Device nGnRE with stage 1 being normal cacheable 0b1000 Normal memory - Non-Cacheable due to stage 2. Originally not outer cacheable (WB/ WT) 0b1001 Device nGRE with stage 1 being device 0b1010 Device nGRE with stage 1 being normal non-cacheable 0b1011 Device nGRE with stage 1 being normal cacheable 0b1100 Normal memory - Non-Cacheable due to stage 2. Originally outer cacheable (WB/ WT) 0b1101 Device GRE with stage 1 being device 0b1110 Device GRE with stage 1 being normal non-cacheable 0b1111 Device GRE with stage 1 being normal cacheable	xxxx
[37]	CACHEABILITY	Cacheability	x
[36]	DMB2	Stage 2 dirty bit Modifier	x
[35:34]	HAP2	Stage 2 Access Permissions	xx

Bits	Name	Description	Reset
[33:32]	LEVEL2	Stage 2 level 0b00 Level 3 0b01 Level 2 0b10 Level 1 0b11 Level 0	xx
[31]	PXN1	Stage 1 privileged Execute-Never	x
[30]	XN1	Stage 1 Execute-Never	x
[29]	PXN2	Stage 2 privileged Execute-Never	x
[28]	XN2	Stage 2 Execute-Never	x
[27:26]	AP1	Stage 1 Access Permission	xx
[25]	DMB1	Stage 1 dirty bit Modifier	x
[24]	NS	Non-Secure which determines whether the physical address is in secure spece or non-secure space	x
[23:0]	PA	Physical Address [47:24]	24 {x}

When L2 TLB TCSP (small pages) and AArch64-IMP_MMU_DATA1_EL3.CACHEABLITY == 0b1

Figure A-49: AArch64_imp_mmu_data2_el3 bit assignments

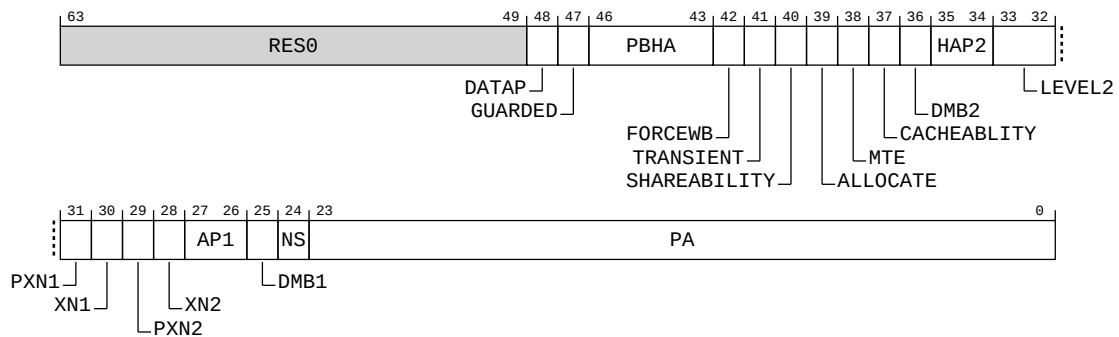


Table A-138: IMP_MMU_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:49]	RES0	Reserved	RES0
[48]	DATAP	Data Parity	x
[47]	GUARDED	Guarded page	x
[46:43]	PBHA	PBHA	xxxx
[42]	FORCEWB	Force Write-back	x
[41]	TRANSIENT	Inner transient hint	x
[40]	SHAREABILITY	Shareability	x
[39]	ALLOCATE	Allocate hint	x

Bits	Name	Description	Reset
[38]	MTE	MTE tag	x
[37]	CACHEABILITY	Cacheability	x
[36]	DMB2	Stage 2 dirty bit Modifier	x
[35:34]	HAP2	Stage 2 Access Permissions	xx
[33:32]	LEVEL2	Stage 2 level 0b00 Level 3 0b01 Level 2 0b10 Level 1 0b11 Level 0	xx
[31]	PXN1	Stage 1 privileged Execute-Never	x
[30]	XN1	Stage 1 Execute-Never	x
[29]	PXN2	Stage 2 privileged Execute-Never	x
[28]	XN2	Stage 2 Execute-Never	x
[27:26]	AP1	Stage 1 Access Permission	xx
[25]	DMB1	Stage 1 dirty bit Modifier	x
[24]	NS	Non-Secure which determines whether the physical address is in secure spece or non-secure space	x
[23:0]	PA	Physical Address [47:24]	24 {x}

When L2 TLB TCMP (medium pages)

Figure A-50: AArch64_imp_mmu_data2_el3 bit assignments

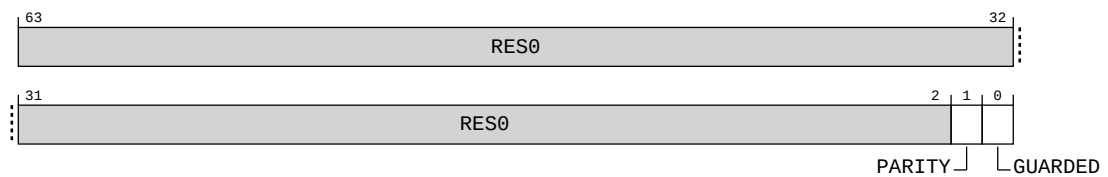


Table A-139: IMP_MMU_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:2]	RES0	Reserved	RES0
[1]	PARITY	Data Parity	x
[0]	GUARDED	Guarded page	x

Access

MRS <Xt>, S3_6_C15_C0_5

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0000	0b101

Accessibility

MRS <Xt>, S3_6_C15_CO_5

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_MMU_DATA2_EL3;
```

A.1.43 IMP_DSIDE_DATA0_EL3, RAMINDEX L1D Data register 0

Returns the data from a RAMINDEX instruction using RAMID=0x8,0x9

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L1 Data cache tag

Figure A-51: AArch64_imp_dside_data0_el3 bit assignments

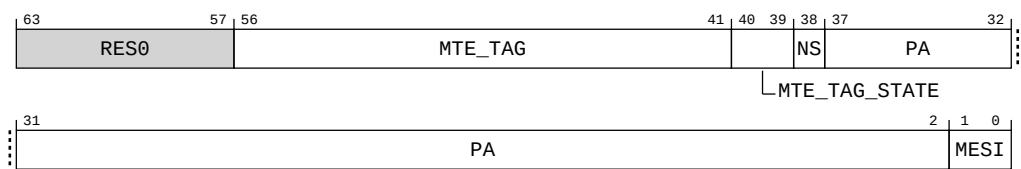
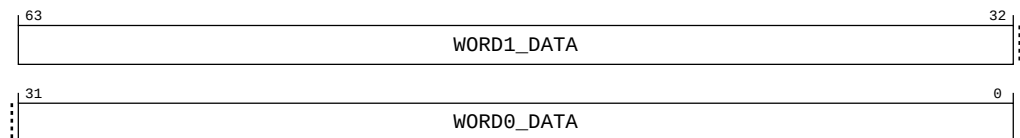


Table A-141: IMP_DSIDE_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:57]	RES0	Reserved	RES0
[56:41]	MTE_TAG	MTE tag data	16 {x}
[40:39]	MTE_TAG_STATE	MTE tag state 0b00 Invalid 0b01 Invalid 0b10 Clean 0b11 Dirty state	xx
[38]	NS	Non-secure identifier for the physical address	x
[37:2]	PA	Physical address [47:20]	36 {x}
[1:0]	MESI	MESI 0b00 Invalid 0b01 Shared 0b10 Modified (unique dirty) 0b11 Exclusive (unique clean)	xx

When L1 data cache data

Figure A-52: AArch64_imp_dside_data0_el3 bit assignments**Table A-142: IMP_DSIDE_DATA0_EL3 bit descriptions**

Bits	Name	Description	Reset
[63:32]	WORD1_DATA	Word 1 data 31:0	32 {x}
[31:0]	WORD0_DATA	Word 0 data 31:0	32 {x}

Access

MRS <Xt>, S3_6_C15_C1_0

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0001	0b000

Accessibility

MRS <Xt>, S3_6_C15_C1_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_DSIDE_DATA0_EL3;
```

A.1.44 IMP_DSIDE_DATA1_EL3, RAMINDEX L1D Data register 1

Returns the data from a RAMINDEX instruction using RAMID=0x8,0x9

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L1 data cache tag

Figure A-53: AArch64_imp_dside_data1_el3 bit assignments

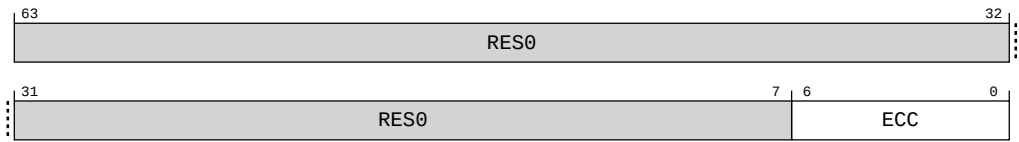
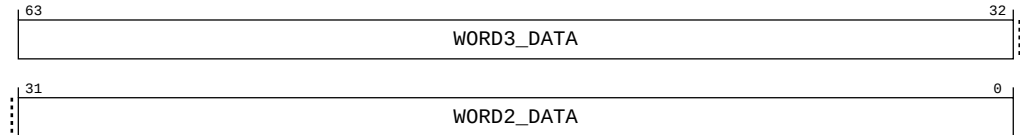


Table A-144: IMP_DSIDE_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:7]	RES0	Reserved	RES0
[6:0]	ECC	ECC	7 {x}

When L1 data cache data

Figure A-54: AArch64_imp_dside_data1_el3 bit assignments**Table A-145: IMP_DSIDE_DATA1_EL3 bit descriptions**

Bits	Name	Description	Reset
[63:32]	WORD3_DATA	Word 3 data 31:0	32 {x}
[31:0]	WORD2_DATA	Word 2 data 31:0	32 {x}

Access

MRS <Xt>, S3_6_C15_C1_1

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0001	0b001

Accessibility

MRS <Xt>, S3_6_C15_C1_1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
    endif
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_DSIDE_DATA1_EL3;
endif
  
```

A.1.45 IMP_DSIDE_DATA2_EL3, RAMINDEX L1D Data register 2

Returns the data from a RAMINDEX instruction using RAMID=0x8,0x9

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L1 data cache tag

Figure A-55: AArch64_imp_dside_data2_el3 bit assignments

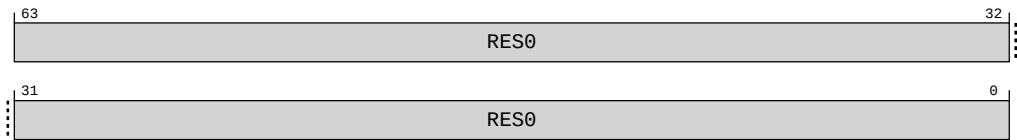


Table A-147: IMP_DSIDE_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

When L1 data cache data

Figure A-56: AArch64_imp_dside_data2_el3 bit assignments

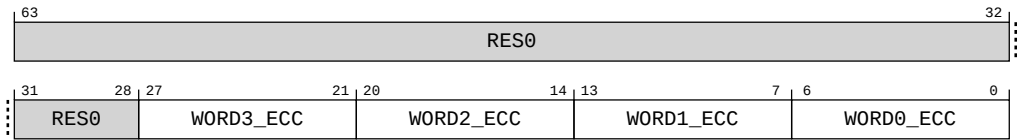


Table A-148: IMP_DSIDE_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:28]	RES0	Reserved	RES0
[27:21]	WORD3_ECC	Word 3 ECC	7 {x}
[20:14]	WORD2_ECC	Word 2 ECC	7 {x}
[13:7]	WORD1_ECC	Word 1 ECC	7 {x}
[6:0]	WORD0_ECC	Word 0 ECC	7 {x}

Access

MRS <Xt>, S3_6_C15_C1_2

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0001	0b010

Accessibility

MRS <Xt>, S3_6_C15_C1_2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_DSIDE_DATA2_EL3;

```

A.1.46 IMP_L2_DATA0_EL3, RAMINDEX L2 Data register 0

Returns the data from a RAMINDEX instruction using RAMID=0x10 or RAMID=0x11

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L2 tag cache for 2048KB RAM

Figure A-57: AArch64_imp_l2_data0_el3 bit assignments

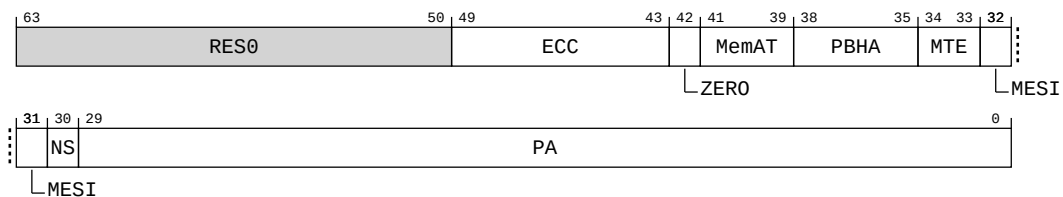
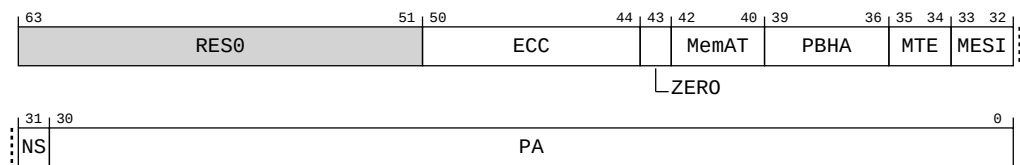


Table A-150: IMP_L2_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:50]	RES0	Reserved	RES0
[49:43]	ECC	ECC	7 {x}
[42]	ZERO	Data is zero	x
[41:39]	MemAT	Memory attribute	xxx
[38:35]	PBHA	PBHA	xxxx
[34:33]	MTE	MTE state 0b00 Invalid 0b01 Invalid 0b10 Clean 0b11 Dirty	xx
[32:31]	MESI	MESI 0b00 Invalid 0b01 Shared 0b10 Modified (unique dirty) 0b11 Exclusive (unique clean)	xx
[30]	NS	Non-Secure identifier	x
[29:0]	PA	Physical address [47:18]	30 {x}

When L2 tag cache for 1024KB RAM

Figure A-58: AArch64_imp_l2_data0_el3 bit assignments**Table A-151: IMP_L2_DATA0_EL3 bit descriptions**

Bits	Name	Description	Reset
[63:51]	RES0	Reserved	RES0
[50:44]	ECC	ECC	7 {x}
[43]	ZERO	Data is zero	x

Bits	Name	Description	Reset
[42:40]	MemAT	Memory attribute	xxx
[39:36]	PBHA	PBHA	xxxx
[35:34]	MTE	MTE state 0b00 Invalid 0b01 Invalid 0b10 Clean 0b11 Dirty	xx
[33:32]	MESI	MESI 0b00 Invalid 0b01 Shared 0b10 Modified (unique dirty) 0b11 Exclusive (unique clean)	xx
[31]	NS	Non-Secure identifier	x
[30:0]	PA	Physical address [47:17]	31 {x}

When L2 tag cache for 512KB RAM

Figure A-59: AArch64_imp_l2_data0_el3 bit assignments

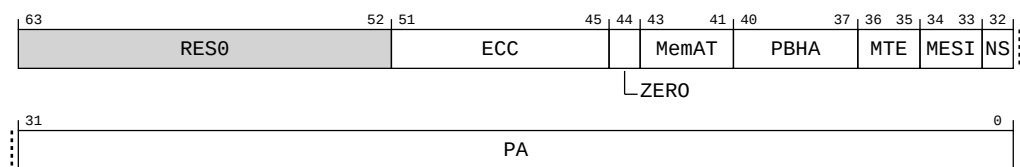


Table A-152: IMP_L2_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:52]	RES0	Reserved	RES0
[51:45]	ECC	ECC	7 {x}
[44]	ZERO	Data is zero	x
[43:41]	MemAT	Memory attribute	xxx
[40:37]	PBHA	PBHA	xxxx

Bits	Name	Description	Reset
[36:35]	MTE	MTE state 0b00 Invalid 0b01 Invalid 0b10 Clean 0b11 Dirty	xx
[34:33]	MESI	MESI 0b00 Invalid 0b01 Shared 0b10 Modified (unique dirty) 0b11 Exclusive (unique clean)	xx
[32]	NS	Non-Secure identifier	x
[31:0]	PA	Physical address [47:16]	32 { x }

When L2 tag cache for 256KB RAM

Figure A-60: AArch64_imp_l2_data0_el3 bit assignments

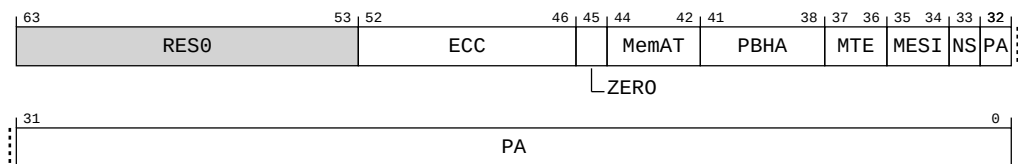


Table A-153: IMP_L2_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:53]	RES0	Reserved	RES0
[52:46]	ECC	ECC	7 { x }
[45]	ZERO	Data is zero	x
[44:42]	MemAT	Memory attribute	xxx
[41:38]	PBHA	PBHA	xxxxx

Bits	Name	Description	Reset
[37:36]	MTE	MTE state 0b00 Invalid 0b01 Invalid 0b10 Clean 0b11 Dirty	xx
[35:34]	MESI	MESI 0b00 Invalid 0b01 Shared 0b10 Modified (unique dirty) 0b11 Exclusive (unique clean)	xx
[33]	NS	Non-Secure identifier	x
[32:0]	PA	Physical address [47:15]	33 {x}

When L2 tag cache for 128KB RAM

Figure A-61: AArch64_imp_l2_data0_el3 bit assignments

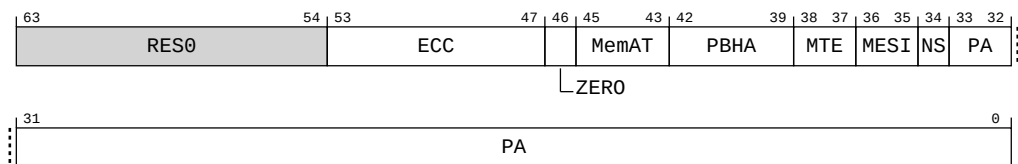


Table A-154: IMP_L2_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:54]	RES0	Reserved	RES0
[53:47]	ECC	ECC	7 {x}
[46]	ZERO	Data is zero	x
[45:43]	MemAT	Memory attribute	xxx
[42:39]	PBHA	PBHA	xxxxx

Bits	Name	Description	Reset
[38:37]	MTE	MTE state 0b00 Invalid 0b01 Invalid 0b10 Clean 0b11 Dirty	xx
[36:35]	MESI	MESI 0b00 Invalid 0b01 Shared 0b10 Modified (unique dirty) 0b11 Exclusive (unique clean)	xx
[34]	NS	Non-Secure identifier	x
[33:0]	PA	Physical address [47:14]	34 { x }

When L2 data cache

Figure A-62: AArch64_imp_l2_data0_el3 bit assignments

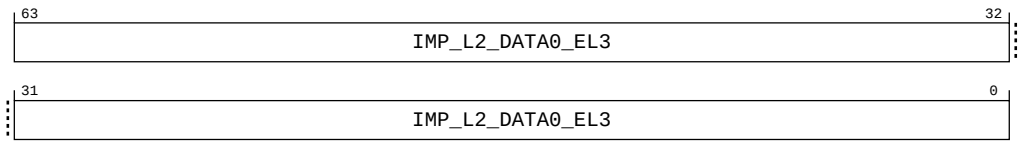


Table A-155: IMP_L2_DATA0_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	IMP_L2_DATA0_EL3	Least significant 8-bytes of the 16-bytes data granule of the line	64 { x }

Access
MRS <Xt>, S3_6_C15_C1_3

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0001	0b011

Accessibility

MRS <Xt>, S3_6_C15_C1_3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elsif PSTATE.EL == EL2 then
    UNDEFINED;
elsif PSTATE.EL == EL3 then
    return IMP_L2_DATA0_EL3;
```

A.1.47 IMP_L2_DATA2_EL3, RAMINDEX L2 Data register 2

Returns the data from a RAMINDEX instruction using RAMID=0x10 or RAMID=0x11

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L2 tag cache

Figure A-63: AArch64_imp_l2_data2_el3 bit assignments

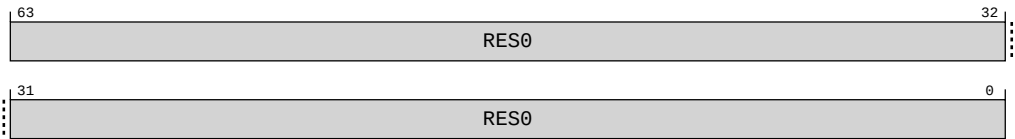


Table A-157: IMP_L2_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

When L2 data cache when L2_DATA_ECC_GRANULE=128

Figure A-64: AArch64_imp_l2_data2_el3 bit assignments

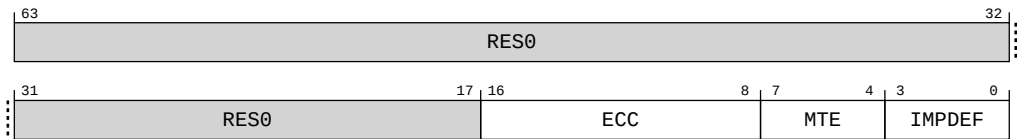


Table A-158: IMP_L2_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:17]	RES0	Reserved	RES0
[16:8]	ECC	ECC	9{x}
[7:4]	MTE	MTE allocation tags for the 16-bytes granule	xxxx
[3:0]	IMPDEF	For granule 0 and 3: replacement policy meta data For granule 1 and 2: MPAM	xxxx

When L2 data cache when L2_DATA_ECC_GRANULE=256

Figure A-65: AArch64_imp_l2_data2_el3 bit assignments

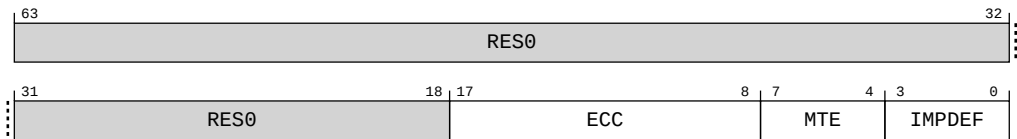


Table A-159: IMP_L2_DATA2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:18]	RES0	Reserved	RES0
[17:8]	ECC	For granule 0 and 2: RES0 For granule 1 and 3: ECC bits	10{x}
[7:4]	MTE	MTE allocation tags for the 16-bytes granule	xxxx
[3:0]	IMPDEF	For granule 0 and 3: replacement policy meta data For granule 1 and 2: MPAM	xxxx

Access

MRS <Xt>, S3_6_C15_C1_4

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0001	0b100

Accessibility

MRS <Xt>, S3_6_C15_C1_4

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elsif PSTATE.EL == EL2 then
    UNDEFINED;
elsif PSTATE.EL == EL3 then
    return IMP_L2_DATA2_EL3;
```

A.1.48 IMP_L2_DATA1_EL3, RAMINDEX L2 Data register 1

Returns the data from a RAMINDEX instruction using RAMID=0x10 or RAMID=0x11

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Bit descriptions

When L2 tag cache

Figure A-66: AArch64_imp_l2_data1_el3 bit assignments

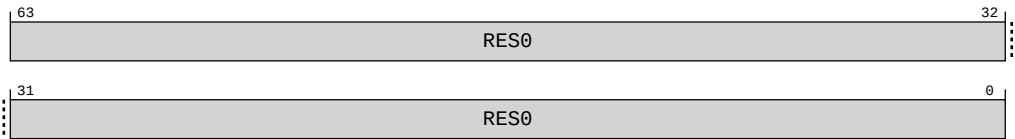


Table A-161: IMP_L2_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

When L2 data cache

Figure A-67: AArch64_imp_l2_data1_el3 bit assignments

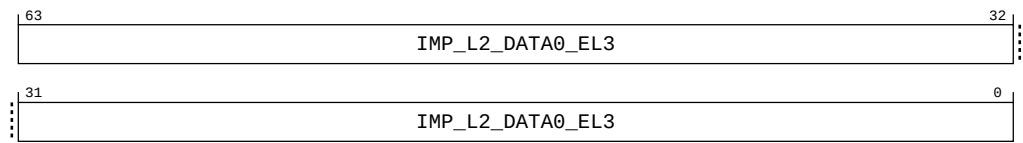


Table A-162: IMP_L2_DATA1_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	IMP_L2_DATA0_EL3	Most significant 8-bytes of the 16-bytes data granule of the line	64 { x }

Access
MRS <Xt>, S3_6_C15_C1_5

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0001	0b101

Accessibility
MRS <Xt>, S3_6_C15_C1_5

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_L2_DATA1_EL3;
```

A.1.49 IMP_CPURNDBR_EL3, Random Number Base Register, EL3

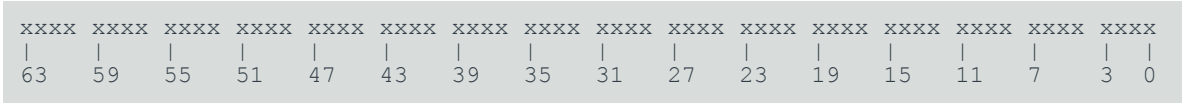
This register controls the base physical address which is used to construct the address for RNDR accesses

Configurations
This register is available in all configurations.

Attributes
Width
64
Functional group
Generic System Control

Access type
See bit descriptions

Reset value



 Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-68: AArch64_imp_cpurndb_r_el3 bit assignments

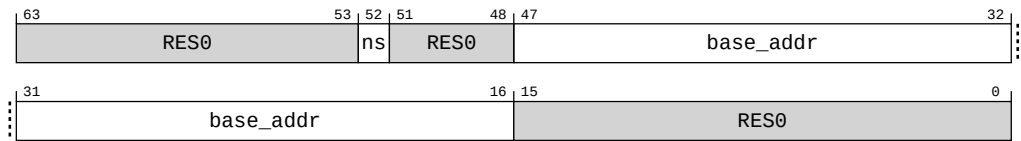


Table A-164: IMP_CPURNDBR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:53]	RES0	Reserved	RES0
[52]	ns	Non-Secure identifier for the physical address	x
[51:48]	RES0	Reserved	RES0
[47:16]	base_addr	Base physical address	32 { x }
[15:0]	RES0	Reserved	RES0

Access
MSR S3_6_C15_C3_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0011	0b000

MRS <Xt>, S3_6_C15_C3_0

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0011	0b000

Accessibility
MSR S3_6_C15_C3_0, <Xt>

```
if PSTATE.EL == EL0 then
```

```
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPURNDBR_EL3 = X[t];
```

MRS <Xt>, S3_6_C15_C3_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPURNDBR_EL3;
```

A.1.50 IMP_CPURNDPEID_EL3, Random Number Packet Identification Register, EL3

This register controls the unique hardware identification which is used to construct the address for RNDR accesses

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-69: AArch64_imp_cpurndpeid_el3 bit assignments

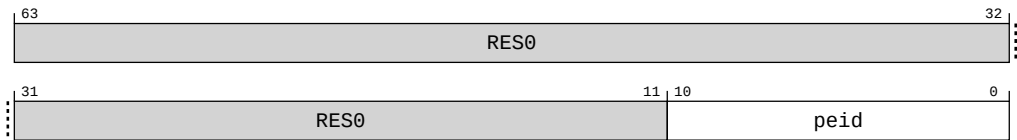


Table A-167: IMP_CPURNDPEID_EL3 bit descriptions

Bits	Name	Description	Reset
[63:11]	RES0	Reserved	RES0
[10:0]	peid	Unique PE ID	11 {x}

Access

MSR S3_6_C15_C3_1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0011	0b001

MRS <Xt>, S3_6_C15_C3_1

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0011	0b001

Accessibility

MSR S3_6_C15_C3_1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPURNDPEID_EL3 = X[t];
```

MRS <Xt>, S3_6_C15_C3_1

```
if PSTATE.EL == EL0 then
```

```
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPURNDPEID_EL3;
```

A.1.51 IMP_CLUSTERDBG_EL3, Cluster Cache Debug Register

Can be used to read the contents of the L3 cache RAMs and snoop filter RAMs. The register must be written with the information of which RAM is to be read. Then the same register should be read to read the contents of that RAM.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000
0000

Bit descriptions

Figure A-70: AArch64_imp_clustercdbg_el3 bit assignments

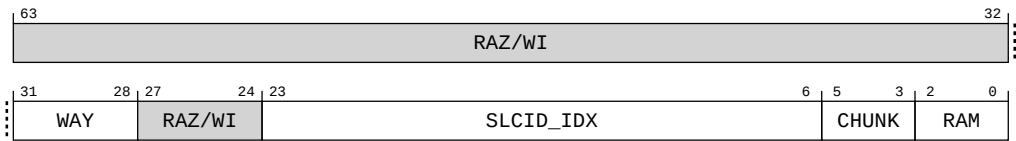


Table A-170: IMP_CLUSTERDBG_EL3 bit descriptions

Bits	Name	Description	Reset
[63:32]	RAZ/WI	Reserved	RAZ/WI
[31:28]	WAY	Way of RAM being accessed.	0b0000
[27:24]	RAZ/WI	Reserved	RAZ/WI

Bits	Name	Description	Reset
[23:6]	SLCID_IDX	The L3 cache Set locations in each cache slice are all power-of-2 in size and therefore can be identified using contiguous index locations. The Set index values for slice 0 start from value zero in this field, followed by the index locations for slice 1, then slice 2, and so on. The total index width varies depending on the size of the RAM being accessed. The cache slice identification number, Slice ID, forms the upper used bits of the cache location encoding in this field. For a Tag RAM or Data RAM access this field will encode as {0, SLICE_ID_W, TagRAM_IDX_W} For a Snoop Filter RAM access this field will encode as {0, SLICE_ID_W, SFRAM_IDX_W}.	0b000000000000000000
[5:3]	CHUNK	Select of 64-bit data chunk to read from 512-bit Data RAM cache line. Only used when accessing Data RAM data. 0b000 Data[63:0] 0b001 Data[127:64] 0b010 Data[191:128] 0b011 Data[255:192] 0b100 Data[319:256] 0b101 Data[383:320] 0b110 Data[447:384] 0b111 Data[511:448]	0b000
[2:0]	RAM	RAM to be accessed. All other values are reserved. 0b001 Snoop Filter RAM 0b010 Tag RAM 0b011 Data RAM - accessing cacheline data 0b111 Data RAM - accessing cacheline MTE tags	0b000

Access

MRS <Xt>, S3_6_C15_C4_7

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0100	0b111

MSR S3_6_C15_C4_7, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0100	0b111

Accessibility

MRS <Xt>, S3_6_C15_C4_7

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CLUSTERDBG_EL3;

```

MSR S3_6_C15_C4_7, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CLUSTERDBG_EL3 = X[t];

```

A.1.52 IMP_ATCR_EL3, CPU Auxiliary Translation Control Register

This register controls the values of the PBHA signals for memory accesses generated by translation table walks in the EL3 translation regime.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	xxxx	0000	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions**Figure A-71: AArch64_imp_atcr_el3 bit assignments****Table A-173: IMP_ATCR_EL3 bit descriptions**

Bits	Name	Description	Reset
[63:12]	RES0	Reserved	RES0
[11]	HWVAL62	Value of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL3 if HWEN62 is set.	0b0
[10]	HWVAL61	Value of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL3 if HWEN61 is set.	0b0
[9]	HWVAL60	Value of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL3 if HWEN60 is set.	0b0
[8]	HWVAL59	Value of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL3 if HWEN59 is set.	0b0
[7:4]	RES0	Reserved	RES0
[3]	HWEN62	Enable use of PBHA[3] on memory accesses due to translation table walks using TTBR0_EL3. If this bit is clear, PBHA[3] will be 0 on translation table walks.	0b0
[2]	HWEN61	Enable use of PBHA[2] on memory accesses due to translation table walks using TTBR0_EL3. If this bit is clear, PBHA[2] will be 0 on translation table walks.	0b0
[1]	HWEN60	Enable use of PBHA[1] on memory accesses due to translation table walks using TTBR0_EL3. If this bit is clear, PBHA[1] will be 0 on translation table walks.	0b0
[0]	HWEN59	Enable use of PBHA[0] on memory accesses due to translation table walks using TTBR0_EL3. If this bit is clear, PBHA[0] will be 0 on translation table walks.	0b0

Access

MRS <Xt>, S3_6_C15_C7_0

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0111	0b000

MSR S3_6_C15_C7_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0111	0b000

Accessibility

MRS <Xt>, S3_6_C15_C7_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_ATCR_EL3;
```

MSR S3_6_C15_C7_0, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_ATCR_EL3 = X[t];
```

A.1.53 IMP_CPUPSELR_EL3, Selected Instruction Private Control Register

Selects the current instruction patch register for subsequent accesses to AArch64-IMP_CPUPCR_EL3, AArch64-IMP_CPUPOR_EL3, AArch64-IMP_CPUPMR_EL3, AArch64-IMP_CPUPOR2_EL3, AArch64-IMP_CPUPMR2_EL3, and AArch64-IMP_CPUPFR_EL3

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-72: AArch64_imp_cpupselr_el3 bit assignments



Table A-176: IMP_CPUPSELR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MRS <Xt>, S3_6_C15_C8_0

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b000

MSR S3_6_C15_C8_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b000

Accessibility

MRS <Xt>, S3_6_C15_C8_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
```

```
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPSELR_EL3;
```

MSR S3_6_C15_C8_0, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPUPSELR_EL3 = X[t];
```

A.1.54 IMP_CPUPCR_EL3, Selected Instruction Patch Control Register

Configures current Instruction Patch selected by AArch64-IMP_CPUPSELR_EL3.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-73: AArch64_imp_cpupcr_el3 bit assignments



Table A-179: IMP_CPUPCR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MRS <Xt>, S3_6_C15_C8_1

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b001

MSR S3_6_C15_C8_1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b001

Accessibility

MRS <Xt>, S3_6_C15_C8_1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPCR_EL3;
```

MSR S3_6_C15_C8_1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
```

```
IMP_CPUPCR_EL3 = X[t];
```

A.1.55 IMP_CPUPOR_EL3, Selected Instruction Patch Opcode Register

Opcode for current Instruction Patch selected by AArch64-IMP_CPUPSELR_EL3.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-74: AArch64_imp_cpupor_el3 bit assignments

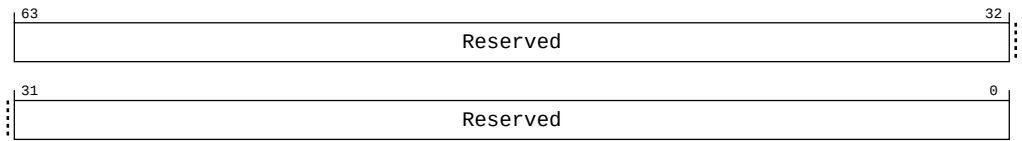


Table A-182: IMP_CPUPOR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MRS <Xt>, S3_6_C15_C8_2

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b010

MSR S3_6_C15_C8_2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b010

Accessibility

MRS <Xt>, S3_6_C15_C8_2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPOR_EL3;

```

MSR S3_6_C15_C8_2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPUPOR_EL3 = X[t];

```

A.1.56 IMP_CPUPMR_EL3, Selected Instruction Patch Mask Register

Mask for current Instruction Patch selected by AArch64-IMP_CPUPSELR_EL3.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-75: AArch64_imp_cpupmr_el3 bit assignments

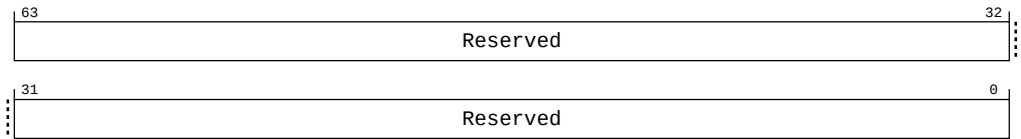


Table A-185: IMP_CPUPMR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MRS <Xt>, S3_6_C15_C8_3

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b011

MSR S3_6_C15_C8_3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b011

Accessibility

MRS <Xt>, S3_6_C15_C8_3

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
```

```
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPMR_EL3;
```

MSR S3_6_C15_C8_3, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPUPMR_EL3 = X[t];
```

A.1.57 IMP_CPUPOR2_EL3, Selected Instruction Patch Opcode Register 2

Opcode exclusion for current Instruction Patch selected by AArch64-IMP_CPUPSELR_EL3.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-76: AArch64_imp_cpupor2_el3 bit assignments

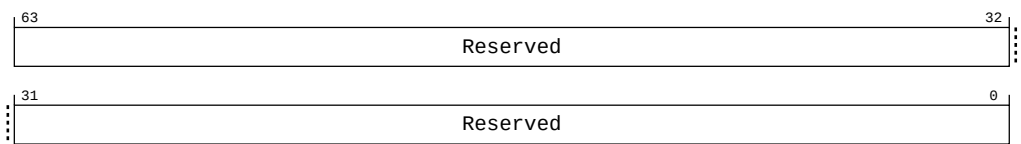


Table A-188: IMP_CPUPOR2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 {x}

Access

MRS <Xt>, S3_6_C15_C8_4

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b100

MSR S3_6_C15_C8_4, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b100

Accessibility

MRS <Xt>, S3_6_C15_C8_4

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPOR2_EL3;
```

MSR S3_6_C15_C8_4, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
```

```
IMP_CPUPOR2_EL3 = X[t];
```

A.1.58 IMP_CPUPMR2_EL3, Selected Instruction Patch Mask Register 2

Mask exclusion for current Instruction Patch selected by AArch64-IMP_CPUPSELR_EL3.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-77: AArch64_imp_cpupmr2_el3 bit assignments

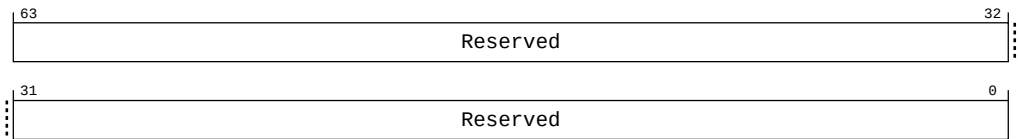


Table A-191: IMP_CPUPMR2_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MRS <Xt>, S3_6_C15_C8_5

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b101

MSR S3_6_C15_C8_5, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b101

Accessibility

MRS <Xt>, S3_6_C15_C8_5

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPMR2_EL3;
```

MSR S3_6_C15_C8_5, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPUPMR2_EL3 = X[t];
```

A.1.59 IMP_CPUPFR_EL3, Selected Instruction Private Flag Register

Instruction Patch flags for current Instruction Patch selected by AArch64-IMP_CPUPSELR_EL3.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Generic System Control

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-78: AArch64_imp_cpupfr_el3 bit assignments

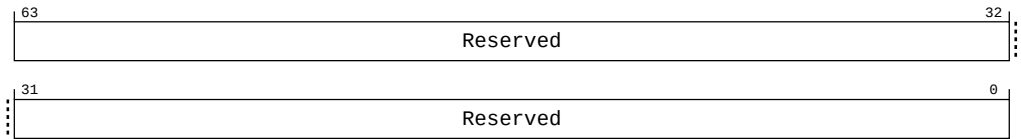


Table A-194: IMP_CPUPFR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MRS <Xt>, S3_6_C15_C8_6

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b110

MSR S3_6_C15_C8_6, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b1000	0b110

Accessibility

MRS <Xt>, S3_6_C15_C8_6

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
```

```

    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPFR_EL3;

```

MSR S3_6_C15_C8_6, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPUPFR_EL3 = X[t];

```

A.2 AArch64 Special-purpose registers summary

The summary table provides an overview of all Special-purpose registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-197: Special-purpose registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
SPSR_EL1	3	0	C4	C0	0	See individual bit resets.	64-bit	Saved Program Status Register (EL1)
ELR_EL1	3	0	C4	C0	1	See individual bit resets.	64-bit	Exception Link Register (EL1)
SP_ELO	3	0	C4	C1	0	See individual bit resets.	64-bit	Stack Pointer (EL0)
IMP_CPUPMPDPCR_EL1	3	0	C15	C2	4	See individual bit resets.	64-bit	Global PMPDPC Configuration Register
IMP_CPUSYNCASSISTCR_EL1	3	0	C15	C2	6	See individual bit resets.	64-bit	Synchronization Assist Configuration Register
DSPSR_ELO	3	3	C4	C5	0	See individual bit resets.	64-bit	Debug Saved Program Status Register
DLR_ELO	3	3	C4	C5	1	See individual bit resets.	64-bit	Debug Link Register
SPSR_EL2	3	4	C4	C0	0	See individual bit resets.	64-bit	Saved Program Status Register (EL2)
ELR_EL2	3	4	C4	C0	1	See individual bit resets.	64-bit	Exception Link Register (EL2)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
SP_EL1	3	4	C4	C1	0	See individual bit resets.	64-bit	Stack Pointer (EL1)
SPSR_irq	3	4	C4	C3	0	See individual bit resets.	64-bit	Saved Program Status Register (IRQ mode)
SPSR_abt	3	4	C4	C3	1	See individual bit resets.	64-bit	Saved Program Status Register (Abort mode)
SPSR_und	3	4	C4	C3	2	See individual bit resets.	64-bit	Saved Program Status Register (Undefined mode)
SPSR_fiq	3	4	C4	C3	3	See individual bit resets.	64-bit	Saved Program Status Register (FIQ mode)
SPSR_EL3	3	6	C4	C0	0	See individual bit resets.	64-bit	Saved Program Status Register (EL3)
ELR_EL3	3	6	C4	C0	1	See individual bit resets.	64-bit	Exception Link Register (EL3)
SP_EL2	3	6	C4	C1	0	See individual bit resets.	64-bit	Stack Pointer (EL2)
IMP_CPUPPMPCR_EL3	3	6	C15	C2	0	See individual bit resets.	64-bit	Global PPM Configuration Register
IMP_CPUMPMPCR_EL3	3	6	C15	C2	1	See individual bit resets.	64-bit	Global MPMM Configuration Register

A.2.1 IMP_CPUPMPDPCR_EL1, Global PPMPDP Configuration Register

This register controls the aggressiveness of PDP features.

Configurations

AArch64 register IMP_CPUPMPDPCR_EL1 bits [63:0] are architecturally mapped to External System register [B.1.3 CPUPMPDPCR, Global PPMPDP Configuration Register](#) on page 613 bits [63:0].

Attributes

Width

64

Functional group

Special-purpose registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx00	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx00
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-79: AArch64_imp_cpupmpdpcr_el1 bit assignments

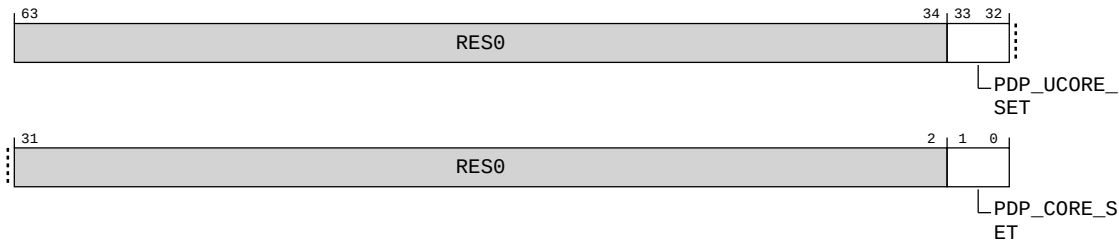


Table A-198: IMP_CPUPMPDPCR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:34]	RES0	Reserved	RES0
[33:32]	PDP_UCORE_SET	Uncore PDP Aggressiveness 0b00 Disable PDP. 0b01 Engage PDP at low aggressiveness 0b10 Engage PDP at medium aggressiveness 0b11 Engage PDP at high aggressiveness	0b00
[31:2]	RES0	Reserved	RES0
[1:0]	PDP_CORE_SET	Core PDP Aggressiveness 0b00 Disable PDP. 0b01 Engage PDP at low aggressiveness. 0b10 Engage PDP at medium aggressiveness. 0b11 Engage PDP at high aggressiveness.	0b00

Access

MRS <Xt>, S3_0_C15_C2_4

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0010	0b100

MSR S3_0_C15_C2_4, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0010	0b100

Accessibility

MRS <Xt>, S3_0_C15_C2_4

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUPPMPDPCR_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUPPMPDPCR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPPMPDPCR_EL1;

```

MSR S3_0_C15_C2_4, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.PDPEN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.PDPEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.PDPEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUPPMPDPCR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if ACTLR_EL3.PDPEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUPPMPDPCR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_CPUPPMPDPCR_EL1 = X[t];

```


A.2.2 IMP_CPUSYNCASSISTCR_EL1, Synchronization Assist Configuration Register

This register controls the deferring mechanisms used by specific tests to assist in synchronizing power-related events.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Special-purpose registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-80: AArch64_imp_cpusyncassistcr_el1 bit assignments

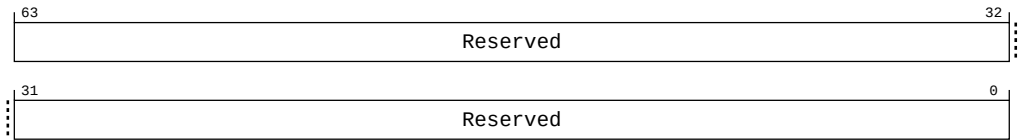


Table A-201: IMP_CPUSYNCASSISTCR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Reserved	Reserved for Arm internal use	64 { x }

Access

MRS <Xt>, S3_0_C15_C2_6

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0010	0b110

MSR S3_0_C15_C2_6, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1111	0b0010	0b110

Accessibility

MRS <Xt>, S3_0_C15_C2_6

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return IMP_CPUSYNCCASSISTCR_EL1;
elseif PSTATE.EL == EL2 then
    return IMP_CPUSYNCCASSISTCR_EL1;
elseif PSTATE.EL == EL3 then
    return IMP_CPUSYNCCASSISTCR_EL1;

```

MSR S3_0_C15_C2_6, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif Halted() && EDSCR.SDD == '1' && ACTLR_EL3.ASSISTEN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && ACTLR_EL2.ASSISTEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif ACTLR_EL3.ASSISTEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUSYNCCASSISTCR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if ACTLR_EL3.ASSISTEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            IMP_CPUSYNCCASSISTCR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    IMP_CPUSYNCCASSISTCR_EL1 = X[t];

```

A.2.3 IMP_CPUPPMCR_EL3, Global PPM Configuration Register

This register controls global PPM features and allows discovery of some PPM implementation details.

Configurations

AArch64 register IMP_CPUPPMCR_EL3 bits [63:0] are architecturally mapped to External System register [B.1.1 CPUPPMCR, Global PPM Configuration Register](#) on page 609 bits [63:0].

Attributes

Width

64

Functional group

Special-purpose registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx00
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-81: AArch64_imp_cpuppmcr_el3 bit assignments

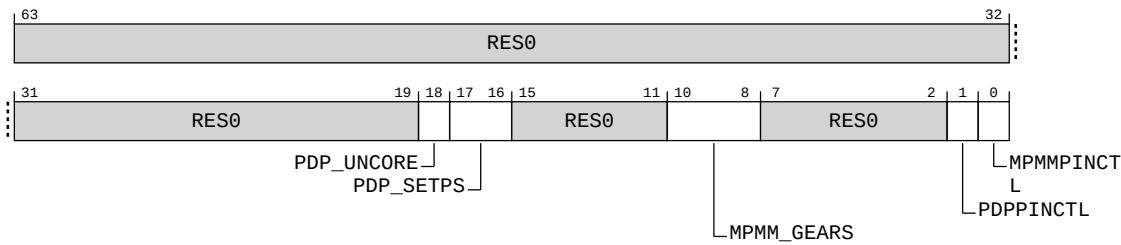


Table A-204: IMP_CPUPPMCR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:19]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[18]	PDP_UNCORE	Indicates whether PDP uncore is implemented 0b1 PDP has separate uncore and core controls. Access to this field is: RO	x
[17:16]	PDP_SETPS	Number of PDP Setpoints implemented 0b11 3 PDP are enabled. Access to this field is: RO	xx
[15:11]	RES0	Reserved	RES0
[10:8]	MPMM_GEARs	Number of MPMM Gears implemented 0b011 3 MPMM are enabled. Access to this field is: RO	xxx
[7:2]	RES0	Reserved	RES0
[1]	PDPPINCTL	PDP Pin Control Enabled 0b0 PDP control through SPR and utility bus 0b1 PDP control through pin only.	0b0
[0]	MPMMPINCTL	MPMM Pin Control Enabled 0b0 MPMM control through SPR and utility bus. 0b1 MPMM control through pin only.	0b0

Access

MRS <Xt>, S3_6_C15_C2_0

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0010	0b000

MSR S3_6_C15_C2_0, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0010	0b000

Accessibility

MRS <Xt>, S3_6_C15_C2_0

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
```

```
if EL2Enabled() && HCR_EL2.TIDCP == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
else
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    return IMP_CPUPPMCR_EL3;
```

MSR S3_6_C15_C2_0, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    IMP_CPUPPMCR_EL3 = X[t];
```

A.2.4 IMP_CPUMPMMCR_EL3, Global MPMM Configuration Register

This register is used to change MPMM gears or disable MPMM.

Configurations

AArch64 register IMP_CPUMPMMCR_EL3 bits [63:0] are architecturally mapped to External System register [B.1.2 CPUMPMMCR, Global MPMM Configuration Register](#) on page 611 bits [63:0].

Attributes

Width

64

Functional group

Special-purpose registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-82: AArch64_imp_cpumpmmcr_el3 bit assignments

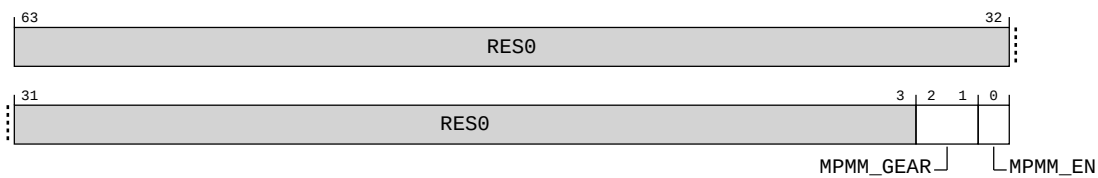


Table A-207: IMP_CPUMPMMCR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:3]	RES0	Reserved	RES0
[2:1]	MPMM_GEAR	MPMM Gear Select 0b00 Select MPMM Gear 0. 0b01 Select MPMM Gear 1. 0b10 Select MPMM Gear 2. 0b11 Do not throttle.	0b00
[0]	MPMM_EN	MPMM Enable 0b0 MPMM is not enabled. 0b1 MPMM is enabled.	0b0

Access

MRS <Xt>, S3_6_C15_C2_1

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0010	0b001

MSR S3_6_C15_C2_1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1111	0b0010	0b001

Accessibility

MRS <Xt>, S3_6_C15_C2_1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
```

```
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
    elsif PSTATE.EL == EL2 then
        UNDEFINED;
    elsif PSTATE.EL == EL3 then
        return IMP_CPUMPMCR_EL3;
```

MSR S3_6_C15_C2_1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elsif PSTATE.EL == EL2 then
    UNDEFINED;
elsif PSTATE.EL == EL3 then
    IMP_CPUMPMCR_EL3 = X[t];
```

A.3 AArch64 System instructions summary

The summary table provides an overview of all System instructions in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-210: System instructions summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
RAMINDEX	1	6	C15	C0	0	See individual bit resets.	64-bit	RAMINDEX system instruction

A.3.1 RAMINDEX, RAMINDEX system instruction

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group
System instructions

Access type
See bit descriptions

Bit descriptions
When AArch64-RAMINDEX.ID == 0x0 and 64KB

Figure A-83: AArch64_ramindex bit assignments

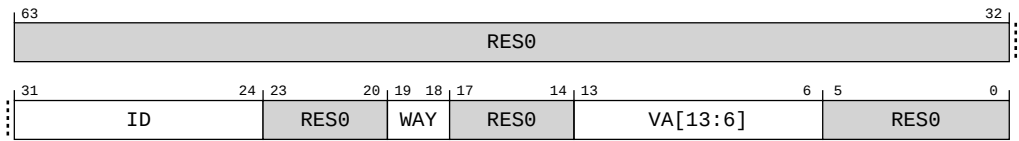


Table A-211: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00000000 L1_I TAG	8 { x }
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17:14]	RES0	Reserved	RES0
[13:6]	VA[13:6]	Virtual Address bits[13:6]	8 { x }
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x0 and 32KB

Figure A-84: AArch64_ramindex bit assignments

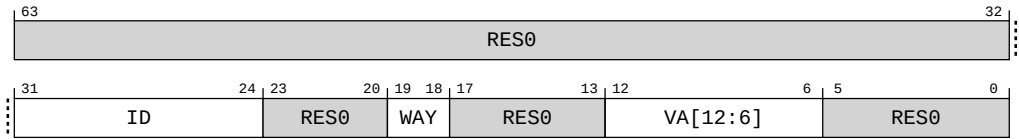


Table A-212: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00000000 L1_I TAG	8 { x }

Bits	Name	Description	Reset
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17:13]	RES0	Reserved	RES0
[12:6]	VA[12:6]	Virtual Address bits[12:6]	7 {x}
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x1 and 64KB

Figure A-85: AArch64_ramindex bit assignments

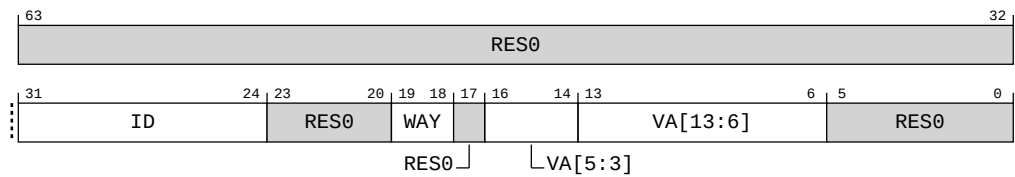


Table A-213: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00000001 L1_I Data	8 {x}
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17]	RES0	Reserved	RES0
[16:14]	VA[5:3]	Virtual Address bits[5:3]	xxx
[13:6]	VA[13:6]	Virtual Address bits[13:6]	8 {x}
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x1 and 32KB

Figure A-86: AArch64_ramindex bit assignments

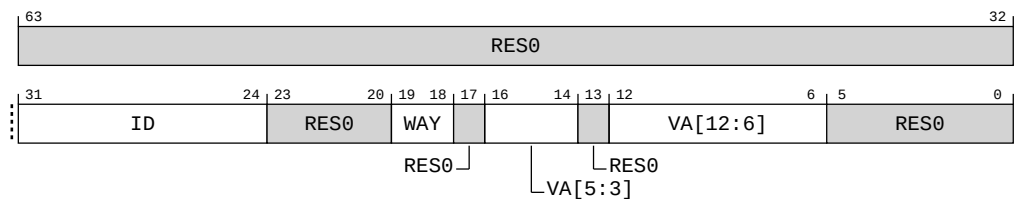


Table A-214: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[31:24]	ID	ID of the selected memory 0b00000001 L1_I Data	8 {x}
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17]	RES0	Reserved	RES0
[16:14]	VA[5:3]	Virtual Address bits[5:3]	xxx
[13]	RES0	Reserved	RES0
[12:6]	VA[12:6]	Virtual Address bits[12:6]	7 {x}
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x8 and 64KB

Figure A-87: AArch64_ramindex bit assignments

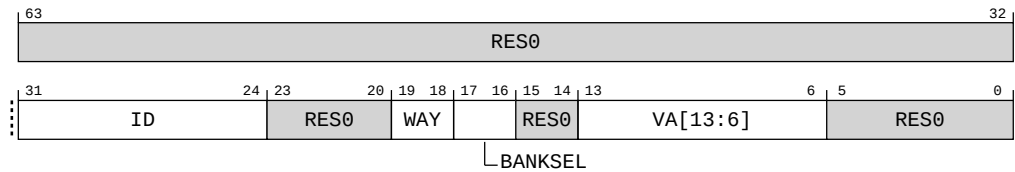


Table A-215: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00001000 L1_D Tag	8 {x}
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17:16]	BANKSEL	Bank selection 0b00 Tag RAM 0 0b01 Tag RAM 1 0b10 Tag RAM 2	xx
[15:14]	RES0	Reserved	RES0
[13:6]	VA[13:6]	Virtual Address bits[13:6]	8 {x}
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x8 and 32KB

Figure A-88: AArch64_ramindex bit assignments

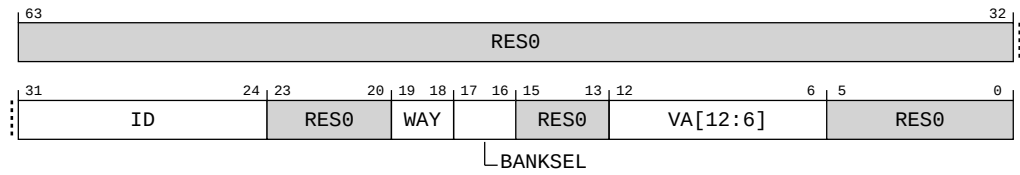


Table A-216: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00001000 L1_D Tag	8 {x}
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17:16]	BANKSEL	Bank selection 0b00 Tag RAM 0 0b01 Tag RAM 1 0b10 Tag RAM 2	xx
[15:13]	RES0	Reserved	RES0
[12:6]	VA[12:6]	Virtual Address bits[12:6]	7 {x}
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x9 and 64KB

Figure A-89: AArch64_ramindex bit assignments

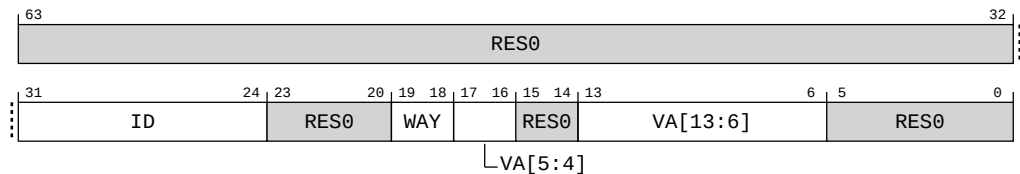


Table A-217: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00001001 L1_D Data	8 {x}

Bits	Name	Description	Reset
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17:16]	VA[5:4]	Virtual Address bits[5:4]	xx
[15:14]	RES0	Reserved	RES0
[13:6]	VA[13:6]	Virtual Address bits[13:6]	8 { x }
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x9 and 32KB

Figure A-90: AArch64_ramindex bit assignments

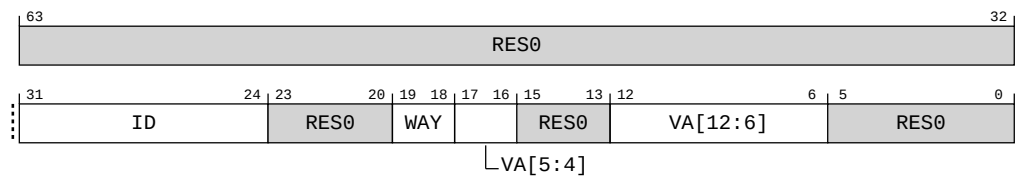


Table A-218: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00001001 L1_D Data	8 { x }
[23:20]	RES0	Reserved	RES0
[19:18]	WAY	Way	xx
[17:16]	VA[5:4]	Virtual Address bits[5:4]	xx
[15:13]	RES0	Reserved	RES0
[12:6]	VA[12:6]	Virtual Address bits[12:6]	7 { x }
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x10 or AArch64-RAMINDEX.ID == 0x11 and 1024KB

Figure A-91: AArch64_ramindex bit assignments

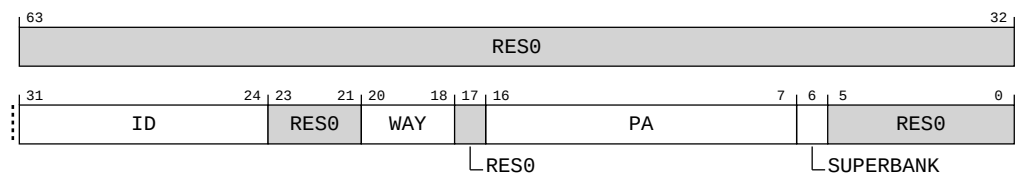
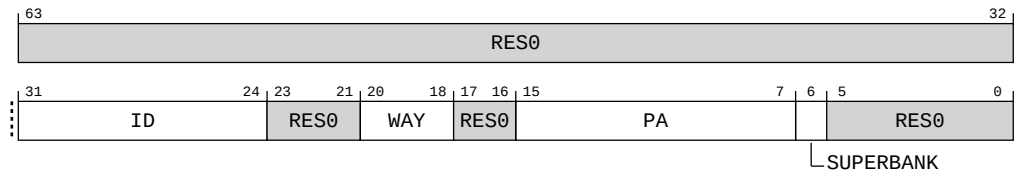


Table A-219: RAMINDEX bit descriptions

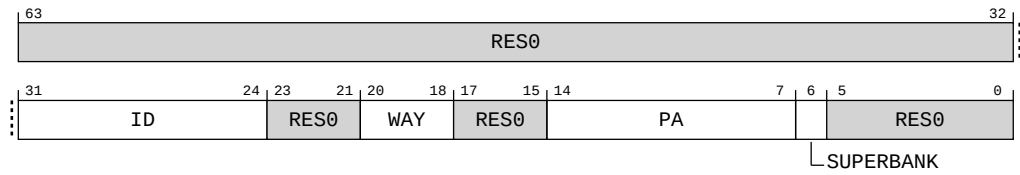
Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00010000 L2 Tag 0b00010001 L2 Data	8 {x}
[23:21]	RES0	Reserved	RES0
[20:18]	WAY	Way	xxx
[17]	RES0	Reserved	RES0
[16:7]	PA	Physical Address bits[16:7]	10 {x}
[6]	SUPERBANK	Physical Address bit[6]	x
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x10 or AArch64-RAMINDEX.ID == 0x11 and 512KB

Figure A-92: AArch64_ramindex bit assignments**Table A-220: RAMINDEX bit descriptions**

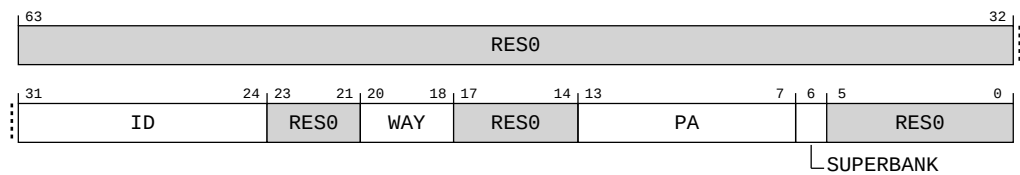
Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00010000 L2 Tag 0b00010001 L2 Data	8 {x}
[23:21]	RES0	Reserved	RES0
[20:18]	WAY	Way	xxx
[17:16]	RES0	Reserved	RES0
[15:7]	PA	Physical Address bits[15:7]	9 {x}
[6]	SUPERBANK	Physical Address bit[6]	x
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x10 or AArch64-RAMINDEX.ID == 0x11 and 256KB

Figure A-93: AArch64_ramindex bit assignments**Table A-221: RAMINDEX bit descriptions**

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00010000 L2 Tag 0b00010001 L2 Data	8 { x }
[23:21]	RES0	Reserved	RES0
[20:18]	WAY	Way	xxx
[17:15]	RES0	Reserved	RES0
[14:7]	PA	Physical Address bits[14:7]	8 { x }
[6]	SUPERBANK	Physical Address bit[6]	x
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x10 or AArch64-RAMINDEX.ID == 0x11 and 128KB

Figure A-94: AArch64_ramindex bit assignments**Table A-222: RAMINDEX bit descriptions**

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00010000 L2 Tag 0b00010001 L2 Data	8 { x }
[23:21]	RES0	Reserved	RES0
[20:18]	WAY	Way	xxx

Bits	Name	Description	Reset
[17:14]	RES0	Reserved	RES0
[13:7]	PA	Physical Address bits[13:7]	7 { x }
[6]	SUPERBANK	Physical Address bit[6]	x
[5:0]	RES0	Reserved	RES0

When AArch64-RAMINDEX.ID == 0x18

Figure A-95: AArch64_ramindex bit assignments

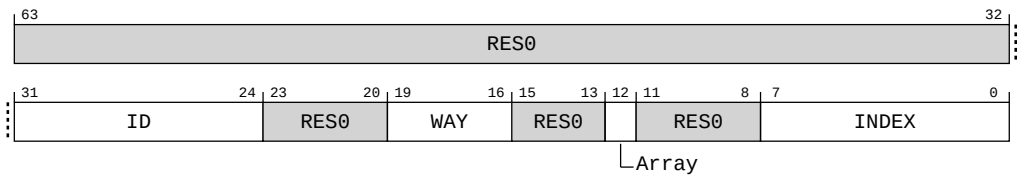


Table A-223: RAMINDEX bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	ID	ID of the selected memory 0b00011000 TLB	8 { x }
[23:20]	RES0	Reserved	RES0
[19:16]	WAY	Way	xxxx
[15:13]	RES0	Reserved	RES0
[12]	Array	Array 0b0 TCSP 0b1 TCMP	x
[11:8]	RES0	Reserved	RES0
[7:0]	INDEX	Index	8 { x }

Access

SYS #6, C15, C0, #0{, <Xt>}

op0	op1	CRn	CRm	op2
0b01	0b110	0b1111	0b0000	0b000

Accessibility

SYS #6, C15, C0, #0{, <Xt>}

```
if PSTATE.EL == EL0 then
    UNDEFINED;
```

```

elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TIDCP == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;
elseif PSTATE.EL == EL2 then
    UNDEFINED;
elseif PSTATE.EL == EL3 then
    RAMINDEX(X[t]);

```

A.4 AArch64 Performance Monitors registers summary

The summary table provides an overview of all Performance Monitors registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-225: Performance Monitors registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMINTENSET_EL1	3	0	C9	C14	1	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Set register
PMINTENCLR_EL1	3	0	C9	C14	2	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Clear register
PMMIR_EL1	3	0	C9	C14	6	See individual bit resets.	64-bit	Performance Monitors Machine Identification Register
IMP_CLUSTERPMCR_EL1	3	0	C15	C5	0	See individual bit resets.	64-bit	Performance Monitors Control Register
IMP_CLUSTERPMCNTENSET_EL1	3	0	C15	C5	1	See individual bit resets.	64-bit	Performance Monitors Count Enable Set Register
IMP_CLUSTERPMCNTENCLR_EL1	3	0	C15	C5	2	See individual bit resets.	64-bit	Performance Monitors Count Enable Clear Register
IMP_CLUSTERPMOVSSSET_EL1	3	0	C15	C5	3	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Set Register
IMP_CLUSTERPMOVSCCLR_EL1	3	0	C15	C5	4	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Clear Register
IMP_CLUSTERPMSELR_EL1	3	0	C15	C5	5	See individual bit resets.	64-bit	Performance Monitors Event Counter Selection Register
IMP_CLUSTERPMINTENSET_EL1	3	0	C15	C5	6	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Set Register
IMP_CLUSTERPMINTENCLR_EL1	3	0	C15	C5	7	See individual bit resets.	64-bit	Performance Monitors Interrupt Enable Clear Register
IMP_CLUSTERPMCCNTR_EL1	3	0	C15	C6	0	See individual bit resets.	64-bit	Performance Monitors Cycle Count Register
IMP_CLUSTERPMXEVTYPYPER_EL1	3	0	C15	C6	1	See individual bit resets.	64-bit	Performance Monitors Selected Event Type Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
IMP_CLUSTERPMXEVCNTR_EL1	3	0	C15	C6	2	See individual bit resets.	64-bit	Performance Monitors Selected Event Count Register
IMP_CLUSTERPMCEID0_EL1	3	0	C15	C6	4	See individual bit resets.	64-bit	Performance Monitors Common Event Identification Register 0
IMP_CLUSTERPMCEID1_EL1	3	0	C15	C6	5	See individual bit resets.	64-bit	Performance Monitors Common Event Identification Register 1
IMP_CLUSTERRSVD_6_6_EL1	3	0	C15	C6	6	See individual bit resets.	64-bit	Reserved
IMP_CLUSTERRSVD_6_7_EL1	3	0	C15	C6	7	See individual bit resets.	64-bit	Reserved
PMCR_ELO	3	3	C9	C12	0	See individual bit resets.	64-bit	Performance Monitors Control Register
PMCNTENSET_ELO	3	3	C9	C12	1	See individual bit resets.	64-bit	Performance Monitors Count Enable Set register
PMCNTENCLR_ELO	3	3	C9	C12	2	See individual bit resets.	64-bit	Performance Monitors Count Enable Clear register
PMOVSLR_ELO	3	3	C9	C12	3	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Clear Register
PMSWINC_ELO	3	3	C9	C12	4	See individual bit resets.	64-bit	Performance Monitors Software Increment register
PMSELR_ELO	3	3	C9	C12	5	See individual bit resets.	64-bit	Performance Monitors Event Counter Selection Register
PMCEID0_ELO	3	3	C9	C12	6	See individual bit resets.	64-bit	Performance Monitors Common Event Identification register 0
PMCEID1_ELO	3	3	C9	C12	7	See individual bit resets.	64-bit	Performance Monitors Common Event Identification register 1
PMCCNTR_ELO	3	3	C9	C13	0	See individual bit resets.	64-bit	Performance Monitors Cycle Count Register
PMXEVTYPER_ELO	3	3	C9	C13	1	See individual bit resets.	64-bit	Performance Monitors Selected Event Type Register
PMXEVCNTR_ELO	3	3	C9	C13	2	See individual bit resets.	64-bit	Performance Monitors Selected Event Count Register
PMUSERENR_ELO	3	3	C9	C14	0	See individual bit resets.	64-bit	Performance Monitors User Enable Register
PMOVSSET_ELO	3	3	C9	C14	3	See individual bit resets.	64-bit	Performance Monitors Overflow Flag Status Set register
PMEVCNTR0_ELO	3	3	C14	C8	0	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR1_ELO	3	3	C14	C8	1	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR2_ELO	3	3	C14	C8	2	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR3_ELO	3	3	C14	C8	3	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR4_ELO	3	3	C14	C8	4	See individual bit resets.	64-bit	Performance Monitors Event Count Registers

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMEVCNTR5_ELO	3	3	C14	C8	5	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR6_ELO	3	3	C14	C8	6	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR7_ELO	3	3	C14	C8	7	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR8_ELO	3	3	C14	C9	0	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR9_ELO	3	3	C14	C9	1	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR10_ELO	3	3	C14	C9	2	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR11_ELO	3	3	C14	C9	3	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR12_ELO	3	3	C14	C9	4	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR13_ELO	3	3	C14	C9	5	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR14_ELO	3	3	C14	C9	6	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR15_ELO	3	3	C14	C9	7	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR16_ELO	3	3	C14	C10	0	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR17_ELO	3	3	C14	C10	1	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR18_ELO	3	3	C14	C10	2	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVCNTR19_ELO	3	3	C14	C10	3	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
PMEVTYPER0_ELO	3	3	C14	C12	0	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER1_ELO	3	3	C14	C12	1	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER2_ELO	3	3	C14	C12	2	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER3_ELO	3	3	C14	C12	3	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER4_ELO	3	3	C14	C12	4	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER5_ELO	3	3	C14	C12	5	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER6_ELO	3	3	C14	C12	6	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER7_ELO	3	3	C14	C12	7	See individual bit resets.	64-bit	Performance Monitors Event Type Registers

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMEVTYPER8_ELO	3	3	C14	C13	0	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER9_ELO	3	3	C14	C13	1	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER10_ELO	3	3	C14	C13	2	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER11_ELO	3	3	C14	C13	3	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER12_ELO	3	3	C14	C13	4	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER13_ELO	3	3	C14	C13	5	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER14_ELO	3	3	C14	C13	6	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER15_ELO	3	3	C14	C13	7	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER16_ELO	3	3	C14	C14	0	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER17_ELO	3	3	C14	C14	1	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER18_ELO	3	3	C14	C14	2	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMEVTYPER19_ELO	3	3	C14	C14	3	See individual bit resets.	64-bit	Performance Monitors Event Type Registers
PMCCFILTR_ELO	3	3	C14	C15	7	See individual bit resets.	64-bit	Performance Monitors Cycle Count Filter Register

A.4.1 PMMIR_EL1, Performance Monitors Machine Identification Register

Describes Performance Monitors parameters specific to the implementation to software.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Performance Monitors registers

Access type

See bit descriptions

Reset value

```

xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx xxxx 0000 0000 0000 0000 0101
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |

```

6359555147433935312723191511730



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-96: AArch64_pmmir_el1 bit assignments

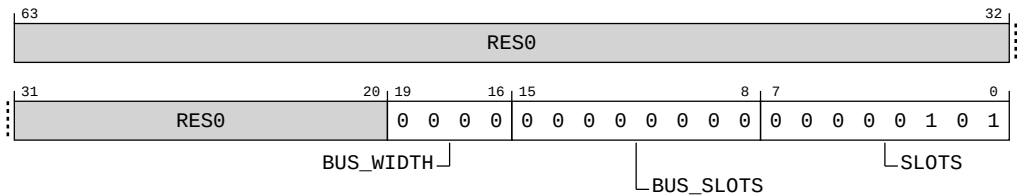


Table A-226: PMMIR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:20]	RES0	Reserved	RES0
[19:16]	BUS_WIDTH	Bus width. Indicates the number of bytes each BUS_ACCESS event relates to. Encoded as Log ₂ (number of bytes), plus one. Defined values are: 0b0000 The information is not available.	0b0000
[15:8]	BUS_SLOTS	Bus count. The largest value by which the BUS_ACCESS event might increment in a single BUS_CYCLES cycle. 0b00000000	0x00
[7:0]	SLOTS	Operation width. The largest value by which the STALL_SLOT event might increment in a single cycle. If the STALL_SLOT event is not implemented, this field might read as zero. 0b00000101 The largest value by which the STALL_SLOT PMU event may increment in one cycle is 5.	0x05

Access

MRS <Xt>, PMMIR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1110	0b110

Accessibility

MRS <Xt>, PMMIR_EL1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
```

```
        UNDEFINED;
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.PMMIR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif MDCR_EL3.TPM == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return PMMIR_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
            UNDEFINED;
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMMIR_EL1;
    elsif PSTATE.EL == EL3 then
        return PMMIR_EL1;
```

A.4.2 PMCR_EL0, Performance Monitors Control Register

Provides details of the Performance Monitors implementation, including the number of counters implemented, and configures and controls the counters.

Configurations

AArch64 register PMCR_EL0 bits [7:0] are architecturally mapped to External System register [B.4.27 PMCR_EL0, Performance Monitors Control Register](#) on page 685 bits [7:0].

Attributes

Width

64

Functional group

Performance Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0000	xxxx	xxxx	xxxx	xxxx	xxx0	x000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-97: AArch64_pmcr_el0 bit assignments

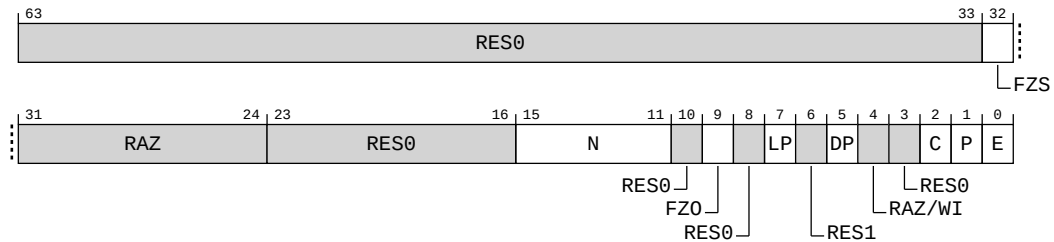


Table A-228: PMCR_EL0 bit descriptions

Bits	Name	Description	Reset
[63:33]	RES0	Reserved	RES0
[32]	FZS	When IsFeatureImplemented(FEAT_SPEv1p2) Freeze-on-SPE event. Stop counters when AArch64-PMBLIMITR_EL1.{PMFZ,E} == {1,1} and AArch64-PMBSR_EL1.S == 1. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_EL0.N. 0b0 Do not freeze on Statistical Profiling Buffer Management event. 0b1 Event counter AArch64-PMEVCNTR<n>_ELO does not count following a Statistical Profiling Buffer Management event if n is in the range of affected event counters. If PMN is not 0, this field affects the operation of event counters in the range [0 .. (PMN-1)]. This field does not affect the operation of other event counters and AArch64-PMCCNTR_EL0. The operation of this field applies even when EL2 is disabled in the current Security state. Otherwise RES0	xxxx
[31:24]	RAZ	Reserved	RAZ
[23:16]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[15:11]	N	Indicates the number of event counters implemented. This value is in the range of 0b00000-0b11111. If the value is 0b00000, then only AArch64-PMCCNTR_ELO is implemented. If the value is 0b11111, then AArch64-PMCCNTR_ELO and 31 event counters are implemented. When EL2 is implemented and enabled for the current Security state, reads of this field from EL1 and EL0 return the value of AArch64-MDCR_EL2.HPMN. 0b10100 Twenty PMU Counters Implemented 0b00110 Six PMU Counters Implemented	The reset values can be the following: 0b10100, 0b00110, respective to the value.
[10]	RES0	Reserved	RES0
[9]	FZO	Freeze-on-overflow. Stop event counters on overflow. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 Do not freeze on overflow. 0b1 Event counter AArch64-PMEVCNTR<n>_ELO does not count when AArch64-PMOVSLR_ELO[(PMN-1):0] is nonzero and n is in the range of affected event counters. If PMN is not 0, this field affects the operation of event counters in the range [0 .. (PMN-1)]. This field does not affect the operation of other event counters and AArch64-PMCCNTR_ELO. The operation of this field applies even when EL2 is disabled in the current Security state.	x
[8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7]	LP	Long event counter enable. Determines when unsigned overflow is recorded by an event counter overflow bit. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 Event counter overflow on increment that causes unsigned overflow of AArch64-PMEVCNTR<n>_ELO[31:0]. 0b1 Event counter overflow on increment that causes unsigned overflow of AArch64-PMEVCNTR<n>_ELO[63:0]. If PMN is not 0, this field affects the operation of event counters in the range [0 .. (PMN-1)]. This field does not affect the operation of other event counters and AArch64-PMCCNTR_ELO. The operation of this field applies even when EL2 is disabled in the current Security state.	x
[6]	RES1	Reserved	RES1
[5]	DP	Disable cycle counter when event counting is prohibited. 0b0 Cycle counting by AArch64-PMCCNTR_ELO is not affected by this mechanism. 0b1 Cycle counting by AArch64-PMCCNTR_ELO is disabled in prohibited regions: - If FEAT_PMUv3p1 is implemented, EL2 is implemented, and AArch64-MDCR_EL2.HPMD is 1, then cycle counting by AArch64-PMCCNTR_ELO is disabled at EL2. - If FEAT_PMUv3p7 is implemented, EL3 is implemented and using AArch64, and AArch64-MDCR_EL3.MPMX is 1, then cycle counting by AArch64-PMCCNTR_ELO is disabled at EL3. - If EL3 is implemented, AArch64-MDCR_EL3.SPME is 0, and either FEAT_PMUv3p7 is not implemented or AArch64-MDCR_EL3.MPMX is 0, then cycle counting by AArch64-PMCCNTR_ELO is disabled at EL3 and in Secure state. If AArch64-MDCR_EL2.HPMN is not 0, this is when event counting by event counters in the range [0..(AArch64-MDCR_EL2.HPMN-1)] is prohibited. For more information see <i>Prohibiting event counting</i> in the Arm® Architecture Reference Manual for A-profile architecture.	x
[4]	RAZ/WI	Reserved	RAZ/WI
[3]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[2]	C	Cycle counter reset. The effects of writing to this bit are: 0b0 No action. 0b1 Reset AArch64-PMCCNTR_ELO to zero. Note: Resetting AArch64-PMCCNTR_ELO does not change the cycle counter overflow bit. If FEAT_PMUV3p5 is implemented, the value of PMCR_ELO.LC is ignored, and bits [63:0] of the cycle counter are reset. Access to this field is: WO/RAZ	0b0
[1]	P	Event counter reset. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 No action. 0b1 If n is in the range of affected event counters, resets each event counter AArch64-PMEVCNTR<n> to zero. The effects of writing to this bit are: - If EL2 is implemented and enabled in the current Security state, in EL0 and EL1, if PMN is not 0, a write of 1 to this bit resets event counters in the range [0 .. (PMN-1)]. - If EL2 is disabled in the current Security state, a write of 1 to this bit resets all the event counters. - In EL2 and EL3, a write of 1 to this bit resets all the event counters. - This field does not affect the operation of other event counters and AArch64-PMCCNTR_ELO. Note: Resetting the event counters does not change the event counter overflow bits. If FEAT_PMUV3p5 is implemented, the values of AArch64-MDCR_EL2.HLP and PMCR_ELO.LP are ignored, and bits [63:0] of all affected event counters are reset. Access to this field is: WO/RAZ	0b0

Bits	Name	Description	Reset
[0]	E	Enable. If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 AArch64-PMCCNTR_ELO is disabled and event counters AArch64-PMEVCNTR<n>_ELO, where n is in the range of affected event counters, are disabled. 0b1 AArch64-PMCCNTR_ELO and event counters AArch64-PMEVCNTR<n>_ELO, where n is in the range of affected event counters, are enabled by AArch64-PMCNTENSET_ELO. If PMN is not 0, this field affects the operation of event counters in the range [0 .. (PMN-1)]. This field does not affect the operation of other event counters. The operation of this field applies even when EL2 is disabled in the current Security state.	0b0

Access

MRS <Xt>, PMCR_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1001	0b1100	0b000

MSR PMCR_ELO, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b011	0b1001	0b1100	0b000

Accessibility

MRS <Xt>, PMCR_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
        UNDEFINED;
    elsif PMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPMCR == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMCR_ELO;
        elsif PSTATE.EL == EL1 then

```

```

    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
        UNDEFINED;
    elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && MDCR_EL2.TPMCR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif MDCR_EL3.TPM == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return PMCR_EL0;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
            UNDEFINED;
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return PMCR_EL0;
    elsif PSTATE.EL == EL3 then
        return PMCR_EL0;

```

MSR PMCR_EL0, <Xt>

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
        UNDEFINED;
    elsif PMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
HDFGWTR_EL2.PMCR_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPMCR == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            PMCR_EL0 = X[t];
    elsif PSTATE.EL == EL1 then
        if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
            UNDEFINED;
        elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.PMCR_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPMCR == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            PMCR_EL0 = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then

```

```

        UNDEFINED;
    elsif MDCR_EL3.TPM == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            PMCR_EL0 = X[t];
    elsif PSTATE.EL == EL3 then
        PMCR_EL0 = X[t];

```

A.4.3 PMCEID0_ELO, Performance Monitors Common Event Identification register 0

Defines which Common architectural events and Common microarchitectural events are implemented, or counted, using PMU events in the ranges 0x0000 to 0x001F and 0x4000 to 0x401F.

For more information about the Common events and the use of the PMCEID<n>_ELO registers see *The PMU event number space and common events* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

AArch64 register PMCEID0_ELO bits [31:0] are architecturally mapped to External System register [B.4.28 PMCEID0, Performance Monitors Common Event Identification register 0](#) on page 688 bits [31:0].

AArch64 register PMCEID0_ELO bits [63:32] are architecturally mapped to External System register [B.4.30 PMCEID2, Performance Monitors Common Event Identification register 2](#) on page 697 bits [31:0].

Attributes

Width

64

Functional group

Performance Monitors registers

Access type

See bit descriptions

Reset value

0000	1111	0000	1111	0001	1110	0111	xxxx	0111	1011	1111	1111	0111	1111	0011	1111
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-98: AArch64_pmceid0_el0 bit assignments

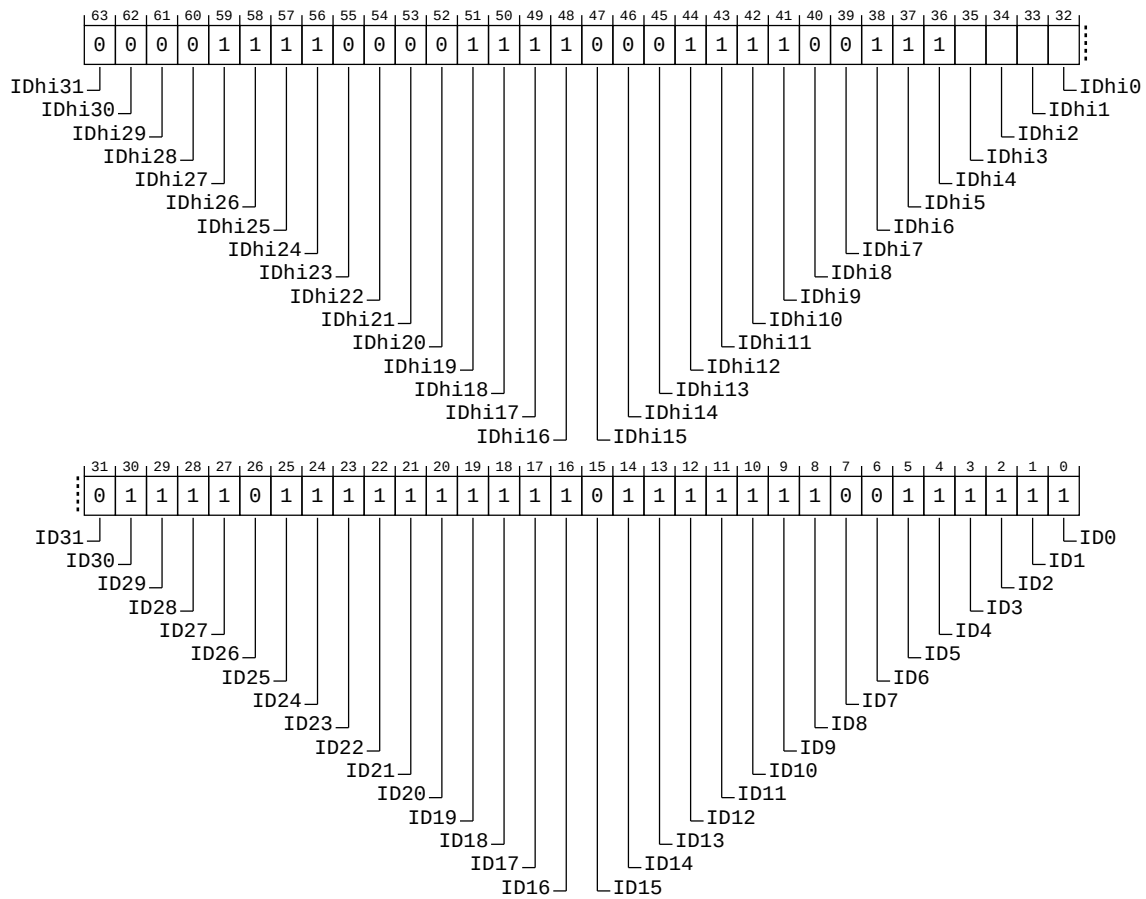


Table A-231: PMCEID0_ELO bit descriptions

Bits	Name	Description	Reset
[63]	IDhi31	IDhi31 corresponds to a Reserved Event event (0x401f) 0b0 The Common event is not implemented, or not counted.	0b0
[62]	IDhi30	IDhi30 corresponds to a Reserved Event event (0x401e) 0b0 The Common event is not implemented, or not counted.	0b0

Bits	Name	Description	Reset
[61]	IDhi29	IDhi29 corresponds to a Reserved Event event (0x401d) 0b0 The Common event is not implemented, or not counted.	0b0
[60]	IDhi28	IDhi28 corresponds to a Reserved Event event (0x401c) 0b0 The Common event is not implemented, or not counted.	0b0
[59]	IDhi27	IDhi27 corresponds to common event (0x401b) CTI_TRIGOUT7 0b1 The Common event is implemented.	0b1
[58]	IDhi26	IDhi26 corresponds to common event (0x401a) CTI_TRIGOUT6 0b1 The Common event is implemented.	0b1
[57]	IDhi25	IDhi25 corresponds to common event (0x4019) CTI_TRIGOUT5 0b1 The Common event is implemented.	0b1
[56]	IDhi24	IDhi24 corresponds to common event (0x4018) CTI_TRIGOUT4 0b1 The Common event is implemented.	0b1
[55]	IDhi23	IDhi23 corresponds to a Reserved Event event (0x4017) 0b0 The Common event is not implemented, or not counted.	0b0
[54]	IDhi22	IDhi22 corresponds to a Reserved Event event (0x4016) 0b0 The Common event is not implemented, or not counted.	0b0
[53]	IDhi21	IDhi21 corresponds to a Reserved Event event (0x4015) 0b0 The Common event is not implemented, or not counted.	0b0
[52]	IDhi20	IDhi20 corresponds to a Reserved Event event (0x4014) 0b0 The Common event is not implemented, or not counted.	0b0
[51]	IDhi19	IDhi19 corresponds to common event (0x4013) TRCEXTOUT3 0b1 The Common event is implemented.	0b1
[50]	IDhi18	IDhi18 corresponds to common event (0x4012) TRCEXTOUT2 0b1 The Common event is implemented.	0b1
[49]	IDhi17	IDhi17 corresponds to common event (0x4011) TRCEXTOUT1 0b1 The Common event is implemented.	0b1
[48]	IDhi16	IDhi16 corresponds to common event (0x4010) TRCEXTOUT0 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[47]	IDhi15	IDhi15 corresponds to common event (0x400f) PMU_HOVFS 0b0 The Common event is not implemented, or not counted.	0b0
[46]	IDhi14	IDhi14 corresponds to common event (0x400e) TRB_TRIG 0b0 The Common event is not implemented, or not counted.	0b0
[45]	IDhi13	IDhi13 corresponds to common event (0x400d) PMU_OVFS 0b0 The Common event is not implemented, or not counted.	0b0
[44]	IDhi12	IDhi12 corresponds to common event (0x400c) TRB_WRAP 0b1 The Common event is implemented.	0b1
[43]	IDhi11	IDhi11 corresponds to common event (0x400b) L3D_CACHE_LMISS_RD 0b1 The Common event is implemented.	0b1
[42]	IDhi10	IDhi10 corresponds to common event (0x400a) L2I_CACHE_LMISS 0b1 The Common event is implemented.	0b1
[41]	IDhi9	IDhi9 corresponds to common event (0x4009) L2D_CACHE_LMISS_RD 0b1 The Common event is implemented.	0b1
[40]	IDhi8	IDhi8 corresponds to common event (0x4008) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[39]	IDhi7	IDhi7 corresponds to common event (0x4007) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[38]	IDhi6	IDhi6 corresponds to common event (0x4006) L1I_CACHE_LMISS 0b1 The Common event is implemented.	0b1
[37]	IDhi5	IDhi5 corresponds to common event (0x4005) STALL_BACKEND_MEM 0b1 The Common event is implemented.	0b1
[36]	IDhi4	IDhi4 corresponds to common event (0x4004) CNT_CYCLES 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[35]	IDhi3	IDhi3 corresponds to common event (0x4003) SAMPLE_COLLISION 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.
[34]	IDhi2	IDhi2 corresponds to common event (0x4002) SAMPLE_FILTRATE 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.
[33]	IDhi1	IDhi1 corresponds to common event (0x4001) SAMPLE_FEED 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.
[32]	IDhi0	IDhi0 corresponds to common event (0x4000) SAMPLE_POP 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.
[31]	ID31	ID31 corresponds to common event (0x1f) L1D_CACHE_ALLOCATE 0b0 The Common event is not implemented, or not counted.	0b0
[30]	ID30	ID30 corresponds to common event (0x1e) CHAIN 0b1 The Common event is implemented.	0b1
[29]	ID29	ID29 corresponds to common event (0x1d) BUS_CYCLES 0b1 The Common event is implemented.	0b1
[28]	ID28	ID28 corresponds to common event (0x1c) TTBR_WRITE_RETIRED 0b1 The Common event is implemented.	0b1
[27]	ID27	ID27 corresponds to common event (0x1b) INST_SPEC 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[26]	ID26	ID26 corresponds to common event (0x1a) MEMORY_ERROR 0b0 The Common event is not implemented, or not counted.	0b0
[25]	ID25	ID25 corresponds to common event (0x19) BUS_ACCESS 0b1 The Common event is implemented.	0b1
[24]	ID24	ID24 corresponds to common event (0x18) L2D_CACHE_WB 0b1 The Common event is implemented.	0b1
[23]	ID23	ID23 corresponds to common event (0x17) L2D_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[22]	ID22	ID22 corresponds to common event (0x16) L2D_CACHE 0b1 The Common event is implemented.	0b1
[21]	ID21	ID21 corresponds to common event (0x15) L1D_CACHE_WB 0b1 The Common event is implemented.	0b1
[20]	ID20	ID20 corresponds to common event (0x14) L1I_CACHE 0b1 The Common event is implemented.	0b1
[19]	ID19	ID19 corresponds to common event (0x13) MEM_ACCESS 0b1 The Common event is implemented.	0b1
[18]	ID18	ID18 corresponds to common event (0x12) BR_PRED 0b1 The Common event is implemented.	0b1
[17]	ID17	ID17 corresponds to common event (0x11) CPU_CYCLES 0b1 The Common event is implemented.	0b1
[16]	ID16	ID16 corresponds to common event (0x10) BR_MIS_PRED 0b1 The Common event is implemented.	0b1
[15]	ID15	ID15 corresponds to common event (0xf) UNALIGNED_LDST_RETIRED 0b0 The Common event is not implemented, or not counted.	0b0
[14]	ID14	ID14 corresponds to common event (0xe) BR_RETURN_RETIRED 0b1 The Common event is implemented.	0b1
[13]	ID13	ID13 corresponds to common event (0xd) BR_IMMED_RETIRED 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[12]	ID12	ID12 corresponds to common event (0xc) PC_WRITE_RETIRED 0b1 The Common event is implemented.	0b1
[11]	ID11	ID11 corresponds to common event (0xb) CID_WRITE_RETIRED 0b1 The Common event is implemented.	0b1
[10]	ID10	ID10 corresponds to common event (0xa) EXC_RETURN 0b1 The Common event is implemented.	0b1
[9]	ID9	ID9 corresponds to common event (0x9) EXC_TAKEN 0b1 The Common event is implemented.	0b1
[8]	ID8	ID8 corresponds to common event (0x8) INST_RETIRED 0b1 The Common event is implemented.	0b1
[7]	ID7	ID7 corresponds to common event (0x7) ST_RETIRED 0b0 The Common event is not implemented, or not counted.	0b0
[6]	ID6	ID6 corresponds to common event (0x6) LD_RETIRED 0b0 The Common event is not implemented, or not counted.	0b0
[5]	ID5	ID5 corresponds to common event (0x5) L1D_TLB_REFILL 0b1 The Common event is implemented.	0b1
[4]	ID4	ID4 corresponds to common event (0x4) L1D_CACHE 0b1 The Common event is implemented.	0b1
[3]	ID3	ID3 corresponds to common event (0x3) L1D_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[2]	ID2	ID2 corresponds to common event (0x2) L1I_TLB_REFILL 0b1 The Common event is implemented.	0b1
[1]	ID1	ID1 corresponds to common event (0x1) L1I_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[0]	ID0	ID0 corresponds to common event (0x0) SW_INCR 0b1 The Common event is implemented.	0b1

Access

MRS <Xt>, PMCEID0_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1001	0b1100	0b110

Accessibility

MRS <Xt>, PMCEID0_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
        UNDEFINED;
    elsif PMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
HDFGRTR_EL2.PMCEIDn_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMCEID0_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.PMCEIDn_EL0 == '1'
then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif MDCR_EL3.TPM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return PMCEID0_ELO;
            elsif PSTATE.EL == EL2 then
                if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
                    UNDEFINED;
                elsif MDCR_EL3.TPM == '1' then
                    if Halted() && EDSCR.SDD == '1' then
                        UNDEFINED;
                    else
                        AArch64.SystemAccessTrap(EL3, 0x18);
                    else
                        return PMCEID0_ELO;
            elsif PSTATE.EL == EL3 then
                return PMCEID0_ELO;

```

A.4.4 PMCEID1_ELO, Performance Monitors Common Event Identification register 1

Defines which Common architectural events and Common microarchitectural events are implemented, or counted, using PMU events in the ranges 0x0020 to 0x003F and 0x4020 to 0x403F.

For more information about the Common events and the use of the PMCEID<n>_ELO registers see *The PMU event number space and common events* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

AArch64 register PMCEID1_ELO bits [31:0] are architecturally mapped to External System register [B.4.29 PMCEID1, Performance Monitors Common Event Identification register 1](#) on page 693 bits [31:0].

AArch64 register PMCEID1_ELO bits [63:32] are architecturally mapped to External System register [B.4.31 PMCEID3, Performance Monitors Common Event Identification register 3](#) on page 701 bits [31:0].

Attributes

Width

64

Functional group

Performance Monitors registers

Access type

See bit descriptions

Reset value

0000 0000 0000 0000 0000 0000 0111 0111 1111 1110 1111 1110 1010 1111 1111 1111

Bit descriptions

Figure A-99: AArch64_pmceid1_el0 bit assignments

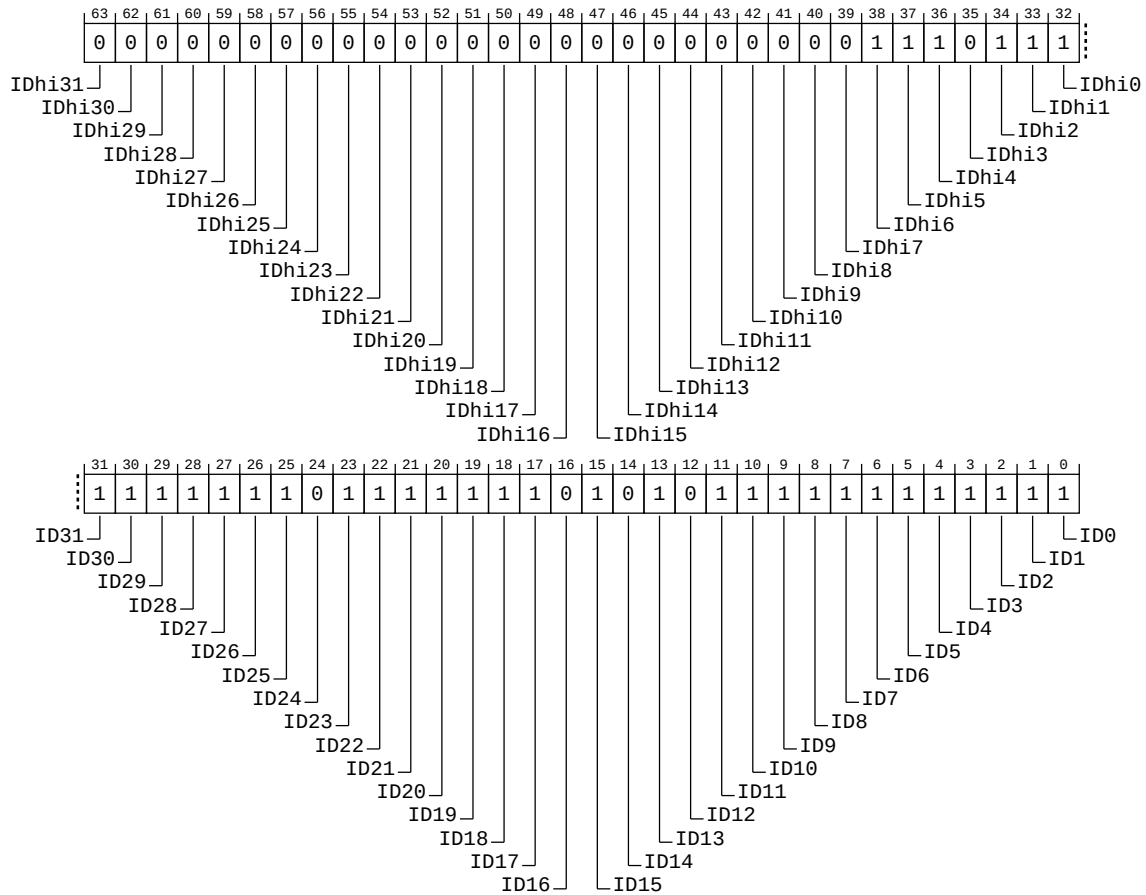


Table A-233: PMCEID1_ELO bit descriptions

Bits	Name	Description	Reset
[63]	IDHi31	IDHi31 corresponds to a Reserved Event event (0x403f) 0b0 The Common event is not implemented, or not counted.	0b0
[62]	IDHi30	IDHi30 corresponds to a Reserved Event event (0x403e) 0b0 The Common event is not implemented, or not counted.	0b0
[61]	IDHi29	IDHi29 corresponds to a Reserved Event event (0x403d) 0b0 The Common event is not implemented, or not counted.	0b0
[60]	IDHi28	IDHi28 corresponds to a Reserved Event event (0x403c) 0b0 The Common event is not implemented, or not counted.	0b0

Bits	Name	Description	Reset
[59]	IDHi27	IDHi27 corresponds to a Reserved Event event (0x403b) 0b0 The Common event is not implemented, or not counted.	0b0
[58]	IDHi26	IDHi26 corresponds to a Reserved Event event (0x403a) 0b0 The Common event is not implemented, or not counted.	0b0
[57]	IDHi25	IDHi25 corresponds to a Reserved Event event (0x4039) 0b0 The Common event is not implemented, or not counted.	0b0
[56]	IDHi24	IDHi24 corresponds to a Reserved Event event (0x4038) 0b0 The Common event is not implemented, or not counted.	0b0
[55]	IDHi23	IDHi23 corresponds to a Reserved Event event (0x4037) 0b0 The Common event is not implemented, or not counted.	0b0
[54]	IDHi22	IDHi22 corresponds to a Reserved Event event (0x4036) 0b0 The Common event is not implemented, or not counted.	0b0
[53]	IDHi21	IDHi21 corresponds to a Reserved Event event (0x4035) 0b0 The Common event is not implemented, or not counted.	0b0
[52]	IDHi20	IDHi20 corresponds to a Reserved Event event (0x4034) 0b0 The Common event is not implemented, or not counted.	0b0
[51]	IDHi19	IDHi19 corresponds to a Reserved Event event (0x4033) 0b0 The Common event is not implemented, or not counted.	0b0
[50]	IDHi18	IDHi18 corresponds to a Reserved Event event (0x4032) 0b0 The Common event is not implemented, or not counted.	0b0
[49]	IDHi17	IDHi17 corresponds to a Reserved Event event (0x4031) 0b0 The Common event is not implemented, or not counted.	0b0
[48]	IDHi16	IDHi16 corresponds to a Reserved Event event (0x4030) 0b0 The Common event is not implemented, or not counted.	0b0
[47]	IDHi15	IDHi15 corresponds to a Reserved Event event (0x402f) 0b0 The Common event is not implemented, or not counted.	0b0
[46]	IDHi14	IDHi14 corresponds to a Reserved Event event (0x402e) 0b0 The Common event is not implemented, or not counted.	0b0

Bits	Name	Description	Reset
[45]	IDhi13	IDhi13 corresponds to a Reserved Event event (0x402d) 0b0 The Common event is not implemented, or not counted.	0b0
[44]	IDhi12	IDhi12 corresponds to a Reserved Event event (0x402c) 0b0 The Common event is not implemented, or not counted.	0b0
[43]	IDhi11	IDhi11 corresponds to a Reserved Event event (0x402b) 0b0 The Common event is not implemented, or not counted.	0b0
[42]	IDhi10	IDhi10 corresponds to a Reserved Event event (0x402a) 0b0 The Common event is not implemented, or not counted.	0b0
[41]	IDhi9	IDhi9 corresponds to a Reserved Event event (0x4029) 0b0 The Common event is not implemented, or not counted.	0b0
[40]	IDhi8	IDhi8 corresponds to a Reserved Event event (0x4028) 0b0 The Common event is not implemented, or not counted.	0b0
[39]	IDhi7	IDhi7 corresponds to a Reserved Event event (0x4027) 0b0 The Common event is not implemented, or not counted.	0b0
[38]	IDhi6	IDhi6 corresponds to common event (0x4026) MEM_ACCESS_CHECKED_WR 0b1 The Common event is implemented.	0b1
[37]	IDhi5	IDhi5 corresponds to common event (0x4025) MEM_ACCESS_CHECKED_RD 0b1 The Common event is implemented.	0b1
[36]	IDhi4	IDhi4 corresponds to common event (0x4024) MEM_ACCESS_CHECKED 0b1 The Common event is implemented.	0b1
[35]	IDhi3	IDhi3 corresponds to common event (0x4023) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[34]	IDhi2	IDhi2 corresponds to common event (0x4022) ST_ALIGN_LAT 0b1 The Common event is implemented.	0b1
[33]	IDhi1	IDhi1 corresponds to common event (0x4021) LD_ALIGN_LAT 0b1 The Common event is implemented.	0b1
[32]	IDhi0	IDhi0 corresponds to common event (0x4020) LDST_ALIGN_LAT 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[31]	ID31	ID31 corresponds to common event (0x3f) STALL_SLOT 0b1 The Common event is implemented.	0b1
[30]	ID30	ID30 corresponds to common event (0x3e) STALL_SLOT_FRONTEND 0b1 The Common event is implemented.	0b1
[29]	ID29	ID29 corresponds to common event (0x3d) STALL_SLOT_BACKEND 0b1 The Common event is implemented.	0b1
[28]	ID28	ID28 corresponds to common event (0x3c) STALL 0b1 The Common event is implemented.	0b1
[27]	ID27	ID27 corresponds to common event (0x3b) OP_SPEC 0b1 The Common event is implemented.	0b1
[26]	ID26	ID26 corresponds to common event (0x3a) OP_RETIRED 0b1 The Common event is implemented.	0b1
[25]	ID25	ID25 corresponds to common event (0x39) L1D_CACHE_LMISS_RD 0b1 The Common event is implemented.	0b1
[24]	ID24	ID24 corresponds to common event (0x38) REMOTE_ACCESS_RD 0b0 The Common event is not implemented, or not counted.	0b0
[23]	ID23	ID23 corresponds to common event (0x37) LL_CACHE_MISS_RD 0b1 The Common event is implemented.	0b1
[22]	ID22	ID22 corresponds to common event (0x36) LL_CACHE_RD 0b1 The Common event is implemented.	0b1
[21]	ID21	ID21 corresponds to common event (0x35) ITLB_WLK 0b1 The Common event is implemented.	0b1
[20]	ID20	ID20 corresponds to common event (0x34) DTLB_WLK 0b1 The Common event is implemented.	0b1
[19]	ID19	ID19 corresponds to common event (0x33) LL_CACHE_MISS 0b1 The Common event is implemented.	0b1
[18]	ID18	ID18 corresponds to common event (0x32) LL_CACHE 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[17]	ID17	ID17 corresponds to common event (0x31) REMOTE_ACCESS 0b1 The Common event is implemented.	0b1
[16]	ID16	ID16 corresponds to common event (0x30) L2I_TLB 0b0 The Common event is not implemented, or not counted.	0b0
[15]	ID15	ID15 corresponds to common event (0x2f) L2D_TLB 0b1 The Common event is implemented.	0b1
[14]	ID14	ID14 corresponds to common event (0x2e) L2I_TLB_REFILL 0b0 The Common event is not implemented, or not counted.	0b0
[13]	ID13	ID13 corresponds to common event (0x2d) L2D_TLB_REFILL 0b1 The Common event is implemented.	0b1
[12]	ID12	ID12 corresponds to common event (0x2c) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[11]	ID11	ID11 corresponds to common event (0x2b) L3D_CACHE 0b1 The Common event is implemented.	0b1
[10]	ID10	ID10 corresponds to common event (0x2a) L3D_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[9]	ID9	ID9 corresponds to common event (0x29) L3D_CACHE_ALLOCATE 0b1 The Common event is implemented.	0b1
[8]	ID8	ID8 corresponds to common event (0x28) L2I_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[7]	ID7	ID7 corresponds to common event (0x27) L2I_CACHE 0b1 The Common event is implemented.	0b1
[6]	ID6	ID6 corresponds to common event (0x26) L1I_TLB 0b1 The Common event is implemented.	0b1
[5]	ID5	ID5 corresponds to common event (0x25) L1D_TLB 0b1 The Common event is implemented.	0b1
[4]	ID4	ID4 corresponds to common event (0x24) STALL_BACKEND 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[3]	ID3	ID3 corresponds to common event (0x23) STALL_FRONTEND 0b1 The Common event is implemented.	0b1
[2]	ID2	ID2 corresponds to common event (0x22) BR_MIS_PRED_RETIRED 0b1 The Common event is implemented.	0b1
[1]	ID1	ID1 corresponds to common event (0x21) BR_RETIRED 0b1 The Common event is implemented.	0b1
[0]	ID0	ID0 corresponds to common event (0x20) L2D_CACHE_ALLOCATE 0b1 The Common event is implemented.	0b1

Access

MRS <Xt>, PMCEID1_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1001	0b1100	0b111

Accessibility

MRS <Xt>, PMCEID1_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
        UNDEFINED;
    elsif PMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
HDFGRTR_EL2.PMCEIDn_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif MDCR_EL3.TPM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMCEID1_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.PMCEIDn_EL0 == '1'
then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif EL2Enabled() && MDCR_EL2.TPM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif MDCR_EL3.TPM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);

```

```

else
    return PMCEID1_EL0;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && MDCR_EL3.TPM == '1' then
        UNDEFINED;
    elseif MDCR_EL3.TPM == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return PMCEID1_EL0;
elseif PSTATE.EL == EL3 then
    return PMCEID1_EL0;

```

A.5 AArch64 Other system control registers summary

The summary table provides an overview of all Other system control registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-235: Other system control registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
SCTLR_EL1	3	0	C1	C0	0	See individual bit resets.	64-bit	System Control Register (EL1)
CPACR_EL1	3	0	C1	C0	2	See individual bit resets.	64-bit	Architectural Feature Access Control Register
ZCR_EL1	3	0	C1	C2	0	See individual bit resets.	64-bit	SVE Control Register (EL1)
SCTLR_EL2	3	4	C1	C0	0	See individual bit resets.	64-bit	System Control Register (EL2)
HCR_EL2	3	4	C1	C1	0	See individual bit resets.	64-bit	Hypervisor Configuration Register
CPTR_EL2	3	4	C1	C1	2	See individual bit resets.	64-bit	Architectural Feature Trap Register (EL2)
HSTR_EL2	3	4	C1	C1	3	See individual bit resets.	64-bit	Hypervisor System Trap Register
HFGRTR_EL2	3	4	C1	C1	4	See individual bit resets.	64-bit	Hypervisor Fine-Grained Read Trap Register
HFGWTR_EL2	3	4	C1	C1	5	See individual bit resets.	64-bit	Hypervisor Fine-Grained Write Trap Register
HFGITR_EL2	3	4	C1	C1	6	See individual bit resets.	64-bit	Hypervisor Fine-Grained Instruction Trap Register
ZCR_EL2	3	4	C1	C2	0	See individual bit resets.	64-bit	SVE Control Register (EL2)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
HCRX_EL2	3	4	C1	C2	2	See individual bit resets.	64-bit	Extended Hypervisor Configuration Register
HDFGRTR_EL2	3	4	C3	C1	4	See individual bit resets.	64-bit	Hypervisor Debug Fine-Grained Read Trap Register
HDFGWTR_EL2	3	4	C3	C1	5	See individual bit resets.	64-bit	Hypervisor Debug Fine-Grained Write Trap Register
HAFGRTR_EL2	3	4	C3	C1	6	See individual bit resets.	64-bit	Hypervisor Activity Monitors Fine-Grained Read Trap Register
SCTLR_EL3	3	6	C1	C0	0	See individual bit resets.	64-bit	System Control Register (EL3)
ZCR_EL3	3	6	C1	C2	0	See individual bit resets.	64-bit	SVE Control Register (EL3)

A.6 AArch64 GIC system registers summary

The summary table provides an overview of all GIC system registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-236: GIC system registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ICC_PMR_EL1	3	0	C4	C6	0	See individual bit resets.	64-bit	Interrupt Controller Interrupt Priority Mask Register
ICV_PMR_EL1	3	0	C4	C6	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Priority Mask Register
ICC_IARO_EL1	3	0	C12	C8	0	See individual bit resets.	64-bit	Interrupt Controller Interrupt Acknowledge Register 0
ICV_IARO_EL1	3	0	C12	C8	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Acknowledge Register 0
ICC_EOIRO_EL1	3	0	C12	C8	1	See individual bit resets.	64-bit	Interrupt Controller End Of Interrupt Register 0
ICV_EOIRO_EL1	3	0	C12	C8	1	See individual bit resets.	64-bit	Interrupt Controller Virtual End Of Interrupt Register 0
ICC_HPPIRO_EL1	3	0	C12	C8	2	See individual bit resets.	64-bit	Interrupt Controller Highest Priority Pending Interrupt Register 0
ICV_HPPIRO_EL1	3	0	C12	C8	2	See individual bit resets.	64-bit	Interrupt Controller Virtual Highest Priority Pending Interrupt Register 0
ICC_BPRO0_EL1	3	0	C12	C8	3	See individual bit resets.	64-bit	Interrupt Controller Binary Point Register 0

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ICV_BPR0_EL1	3	0	C12	C8	3	See individual bit resets.	64-bit	Interrupt Controller Virtual Binary Point Register 0
ICC_AP0R0_EL1	3	0	C12	C8	4	See individual bit resets.	64-bit	Interrupt Controller Active Priorities Group 0 Registers
ICV_AP0R0_EL1	3	0	C12	C8	4	See individual bit resets.	64-bit	Interrupt Controller Virtual Active Priorities Group 0 Registers
ICC_AP1R0_EL1	3	0	C12	C9	0	See individual bit resets.	64-bit	Interrupt Controller Active Priorities Group 1 Registers
ICV_AP1R0_EL1	3	0	C12	C9	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Active Priorities Group 1 Registers
ICC_DIR_EL1	3	0	C12	C11	1	See individual bit resets.	64-bit	Interrupt Controller Deactivate Interrupt Register
ICV_DIR_EL1	3	0	C12	C11	1	See individual bit resets.	64-bit	Interrupt Controller Deactivate Virtual Interrupt Register
ICC_RPR_EL1	3	0	C12	C11	3	See individual bit resets.	64-bit	Interrupt Controller Running Priority Register
ICV_RPR_EL1	3	0	C12	C11	3	See individual bit resets.	64-bit	Interrupt Controller Virtual Running Priority Register
ICC_SGI1R_EL1	3	0	C12	C11	5	See individual bit resets.	64-bit	Interrupt Controller Software Generated Interrupt Group 1 Register
ICC_ASGI1R_EL1	3	0	C12	C11	6	See individual bit resets.	64-bit	Interrupt Controller Alias Software Generated Interrupt Group 1 Register
ICC_SGI0R_EL1	3	0	C12	C11	7	See individual bit resets.	64-bit	Interrupt Controller Software Generated Interrupt Group 0 Register
ICC_IAR1_EL1	3	0	C12	C12	0	See individual bit resets.	64-bit	Interrupt Controller Interrupt Acknowledge Register 1
ICV_IAR1_EL1	3	0	C12	C12	0	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Acknowledge Register 1
ICC_EOIR1_EL1	3	0	C12	C12	1	See individual bit resets.	64-bit	Interrupt Controller End Of Interrupt Register 1
ICV_EOIR1_EL1	3	0	C12	C12	1	See individual bit resets.	64-bit	Interrupt Controller Virtual End Of Interrupt Register 1
ICC_HPIR1_EL1	3	0	C12	C12	2	See individual bit resets.	64-bit	Interrupt Controller Highest Priority Pending Interrupt Register 1
ICV_HPIR1_EL1	3	0	C12	C12	2	See individual bit resets.	64-bit	Interrupt Controller Virtual Highest Priority Pending Interrupt Register 1
ICC_BPR1_EL1	3	0	C12	C12	3	See individual bit resets.	64-bit	Interrupt Controller Binary Point Register 1
ICV_BPR1_EL1	3	0	C12	C12	3	See individual bit resets.	64-bit	Interrupt Controller Virtual Binary Point Register 1
ICC_CTLR_EL1	3	0	C12	C12	4	See individual bit resets.	64-bit	Interrupt Controller Control Register (EL1)
ICV_CTLR_EL1	3	0	C12	C12	4	See individual bit resets.	64-bit	Interrupt Controller Virtual Control Register
ICC_SRE_EL1	3	0	C12	C12	5	See individual bit resets.	64-bit	Interrupt Controller System Register Enable register (EL1)

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ICC_IGRPEN0_EL1	3	0	C12	C12	6	See individual bit resets.	64-bit	Interrupt Controller Interrupt Group 0 Enable register
ICV_IGRPEN0_EL1	3	0	C12	C12	6	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Group 0 Enable register
ICC_IGRPEN1_EL1	3	0	C12	C12	7	See individual bit resets.	64-bit	Interrupt Controller Interrupt Group 1 Enable register
ICV_IGRPEN1_EL1	3	0	C12	C12	7	See individual bit resets.	64-bit	Interrupt Controller Virtual Interrupt Group 1 Enable register
ICH_AP0R0_EL2	3	4	C12	C8	0	See individual bit resets.	64-bit	Interrupt Controller Hyp Active Priorities Group 0 Registers
ICH_AP1R0_EL2	3	4	C12	C9	0	See individual bit resets.	64-bit	Interrupt Controller Hyp Active Priorities Group 1 Registers
ICC_SRE_EL2	3	4	C12	C9	5	See individual bit resets.	64-bit	Interrupt Controller System Register Enable register (EL2)
ICH_HCR_EL2	3	4	C12	C11	0	See individual bit resets.	64-bit	Interrupt Controller Hyp Control Register
ICH_VTR_EL2	3	4	C12	C11	1	See individual bit resets.	64-bit	Interrupt Controller VGIC Type Register
ICH_MISR_EL2	3	4	C12	C11	2	See individual bit resets.	64-bit	Interrupt Controller Maintenance Interrupt State Register
ICH_EISR_EL2	3	4	C12	C11	3	See individual bit resets.	64-bit	Interrupt Controller End of Interrupt Status Register
ICH_ELSR_EL2	3	4	C12	C11	5	See individual bit resets.	64-bit	Interrupt Controller Empty List Register Status Register
ICH_VMCR_EL2	3	4	C12	C11	7	See individual bit resets.	64-bit	Interrupt Controller Virtual Machine Control Register
ICH_LR0_EL2	3	4	C12	C12	0	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICH_LR1_EL2	3	4	C12	C12	1	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICH_LR2_EL2	3	4	C12	C12	2	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICH_LR3_EL2	3	4	C12	C12	3	See individual bit resets.	64-bit	Interrupt Controller List Registers
ICC_CTLR_EL3	3	6	C12	C12	4	See individual bit resets.	64-bit	Interrupt Controller Control Register (EL3)
ICC_SRE_EL3	3	6	C12	C12	5	See individual bit resets.	64-bit	Interrupt Controller System Register Enable register (EL3)
ICC_IGRPEN1_EL3	3	6	C12	C12	7	See individual bit resets.	64-bit	Interrupt Controller Interrupt Group 1 Enable register (EL3)

A.6.1 ICC_AP0R0_EL1, Interrupt Controller Active Priorities Group 0 Registers

Provides information about Group 0 active priorities.

Configurations

This register is available in all configurations.

Attributes

Width

64

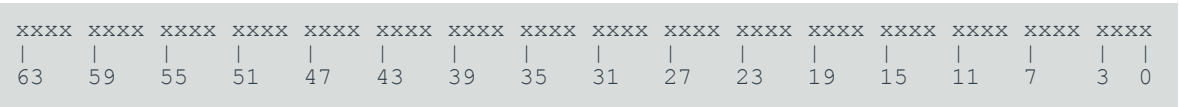
Functional group

GIC system registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-100: AArch64_icc_ap0r0_el1 bit assignments

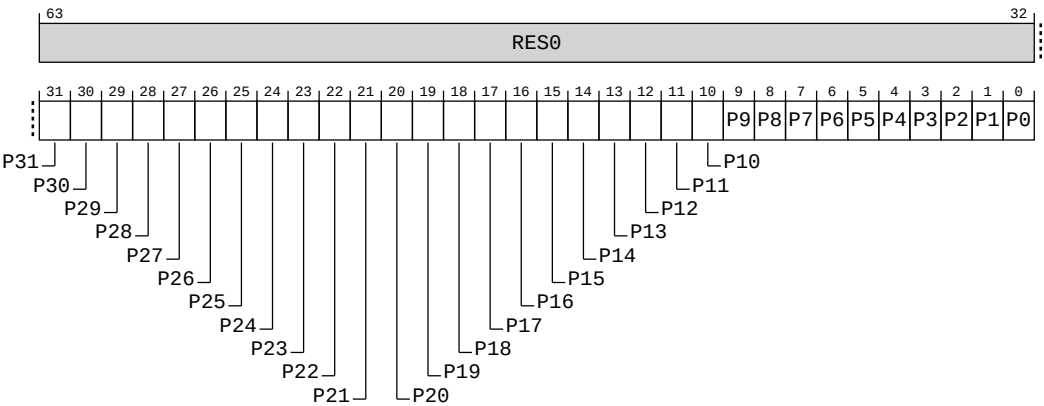


Table A-237: ICC_AP0R0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:0]	P<x>	Provides the access to the active priorities for Group 0 interrupts. Possible values of each bit are: 0b0 There is no Group 0 interrupt active with this priority level, or all active Group 0 interrupts with this priority level have undergone priority-drop. 0b1 There is a Group 0 interrupt active with this priority level which has not undergone priority drop. There are 32 preemption levels, and the active state of these preemption levels are held in the bits corresponding to Priority[7:3].	32 {x}

The contents of these registers are **IMPLEMENTATION DEFINED** with the one architectural requirement that the value 0x00000000 is consistent with no interrupts being active.

Access

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 0 active priorities) might result in **UNPREDICTABLE** behavior of the interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICC_AP0R1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICC_AP0R2_EL1 and ICC_AP0R3_EL1 are only implemented in implementations that support 7 or more bits of priority. Unimplemented registers are **UNDEFINED**.



Note

The number of bits of preemption is indicated by AArch64-ICH_VTR_EL2.PREbits.

Writing to the active priority registers in any order other than the following order will result in **UNPREDICTABLE** behavior:

- ICC_AP0R<n>_EL1.
- Secure AArch64-ICC_AP1R<n>_EL1.
- Non-secure AArch64-ICC_AP1R<n>_EL1.

MRS <Xt>, ICC_AP0R0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1000	0b100

MSR ICC_AP0R0_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1000	0b100

Accessibility

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 0 active priorities) might result in UNPREDICTABLE behavior of the interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICC_AP0R1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICC_AP0R2_EL1 and ICC_AP0R3_EL1 are only implemented in implementations that support 7 or more bits of priority. Unimplemented registers are UNDEFINED.



Note

The number of bits of preemption is indicated by AArch64-ICH_VTR_EL2.PREbits.

Writing to the active priority registers in any order other than the following order will result in UNPREDICTABLE behavior:

- ICC_AP0R<n>_EL1.
- Secure AArch64-ICC_AP1R<n>_EL1.
- Non-secure AArch64-ICC_AP1R<n>_EL1.

MRS <Xt>, ICC_AP0R0_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TALL0 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FMO == '1' then
        return ICV_AP0R0_EL1;
    elseif SCR_EL3.FIQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ICC_AP0R0_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
            UNDEFINED;
        elseif SCR_EL3.FIQ == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ICC_AP0R0_EL1;
    elseif PSTATE.EL == EL3 then

```

```
return ICC_AP0R0_EL1;
```

MSR ICC_AP0R0_EL1, <Xt>

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
        UNDEFINED;
    elsif EL2Enabled() && ICH_HCR_EL2.TALL0 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && HCR_EL2.FMO == '1' then
        ICV_AP0R0_EL1 = X[t];
    elsif SCR_EL3.FIQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ICC_AP0R0_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
            UNDEFINED;
        elsif SCR_EL3.FIQ == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                ICC_AP0R0_EL1 = X[t];
    elsif PSTATE.EL == EL3 then
        ICC_AP0R0_EL1 = X[t];
```

A.6.2 ICV_AP0R0_EL1, Interrupt Controller Virtual Active Priorities Group 0 Registers

Provides information about virtual Group 0 active priorities.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

GIC system registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-101: AArch64_icv_ap0r0_el1 bit assignments

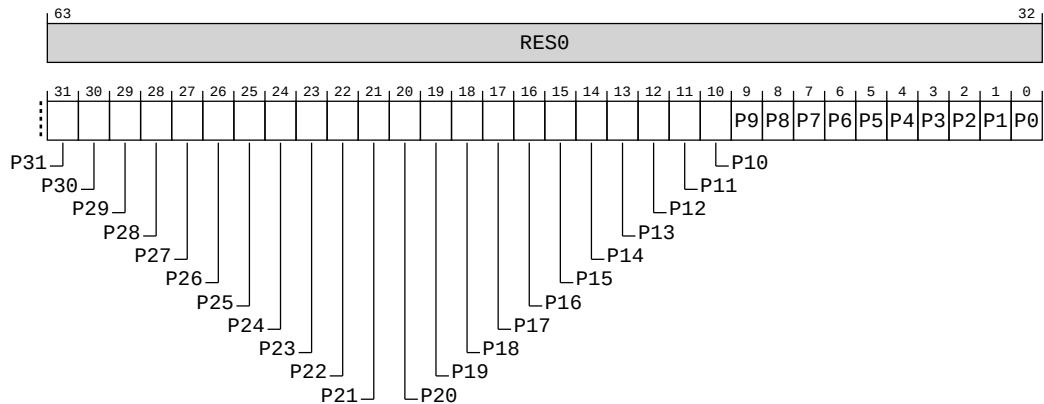


Table A-240: ICV_AP0R0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:0]	P<x>	Provides the access to the virtual active priorities for Group 0 interrupts. Possible values of each bit are: 0b0 There is no Group 0 interrupt active with this priority level, or all active Group 0 interrupts with this priority level have undergone priority-drop. 0b1 There is a Group 0 interrupt active with this priority level which has not undergone priority drop. There are 32 preemption levels, and the active state of these preemption levels are held in the bits corresponding to Priority[7:3].	32 {x}

The contents of these registers are **IMPLEMENTATION DEFINED** with the one architectural requirement that the value 0x00000000 is consistent with no interrupts being active.

Access

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 0 active priorities) might result in **UNPREDICTABLE** behavior of the virtual interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICV_APOR1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICV_APOR2_EL1 and ICV_APOR3_EL1 are only implemented in implementations that support 7 bits of priority. Unimplemented registers are **UNDEFINED**.

Writing to the active priority registers in any order other than the following order might result in **UNPREDICTABLE** behavior of the interrupt prioritization system:

- ICV_APOR<n>_EL1.
- AArch64-ICV_AP1R<n>_EL1.

MRS <Xt>, ICC_APOR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1000	0b100

MSR ICC_APOR0_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1000	0b100

Accessibility

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 0 active priorities) might result in **UNPREDICTABLE** behavior of the virtual interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICV_APOR1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICV_APOR2_EL1 and ICV_APOR3_EL1 are only implemented in implementations that support 7 bits of priority. Unimplemented registers are **UNDEFINED**.

Writing to the active priority registers in any order other than the following order might result in **UNPREDICTABLE** behavior of the interrupt prioritization system:

- ICV_APOR<n>_EL1.
- AArch64-ICV_AP1R<n>_EL1.

MRS <Xt>, ICC_APOR0_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TALL0 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FMO == '1' then
        return ICV_APOR0_EL1;
    elseif SCR_EL3.FIQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else

```

```

        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return ICC_AP0R0_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
            UNDEFINED;
        elseif SCR_EL3.FIQ == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ICC_AP0R0_EL1;
    elseif PSTATE.EL == EL3 then
        return ICC_AP0R0_EL1;

```

MSR ICC_AP0R0_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TALL0 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FMO == '1' then
        ICC_AP0R0_EL1 = X[t];
    elseif SCR_EL3.FIQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ICC_AP0R0_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIQ == '1' then
        UNDEFINED;
    elseif SCR_EL3.FIQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ICC_AP0R0_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    ICC_AP0R0_EL1 = X[t];

```

A.6.3 ICC_AP1R0_EL1, Interrupt Controller Active Priorities Group 1 Registers

Provides information about Group 1 active priorities.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

GIC system registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-102: AArch64_icc_ap1r0_el1 bit assignments

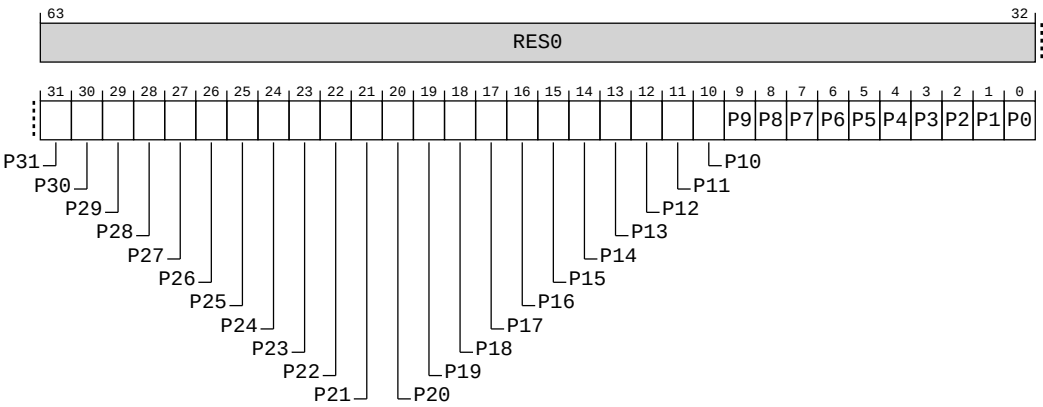


Table A-243: ICC_AP1R0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[31:0]	P<x>	Group 1 interrupt active priorities. When AArch64-SCR_EL3.NS == '1', accesses the priorities for Non-secure Group 1 interrupts, and when AArch64-SCR_EL3.NS == '0' accesses the priorities for Secure Group 1 interrupts. Possible values of each bit are: 0b0 There is no Group 1 interrupt active with this priority level, or all active Group 1 interrupts with this priority level have undergone priority-drop. 0b1 There is a Group 1 interrupt active with this priority level which has not undergone priority drop. There are 32 preemption levels, and the active state of these preemption levels are held in the bits corresponding to Priority[7:3]. When accessed from non-secure EL2 or EL1, only the 16 lowest-priority interrupts are visible in bits [15:0] of this register.	32 {x}

The contents of these registers are **IMPLEMENTATION DEFINED** with the one architectural requirement that the value 0x00000000 is consistent with no interrupts being active.

Access

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 1 active priorities) might result in **UNPREDICTABLE** behavior of the interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICC_AP1R1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICC_AP1R2_EL1 and ICC_AP1R3_EL1 are only implemented in implementations that support 7 or more bits of priority. Unimplemented registers are **UNDEFINED**.



Note

The number of bits of preemption is indicated by AArch64-ICH_VTR_EL2.PREbits.

Writing to the active priority registers in any order other than the following order will result in **UNPREDICTABLE** behavior:

- AArch64-ICC_AP0R<n>_EL1.
- Secure ICC_AP1R<n>_EL1.
- Non-secure ICC_AP1R<n>_EL1.

MRS <Xt>, ICC_AP1R0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1001	0b000

MSR ICC_AP1R0_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1001	0b000

Accessibility

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 1 active priorities) might result in UNPREDICTABLE behavior of the interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICC_AP1R1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICC_AP1R2_EL1 and ICC_AP1R3_EL1 are only implemented in implementations that support 7 or more bits of priority. Unimplemented registers are UNDEFINED.



Note

The number of bits of preemption is indicated by AArch64-ICH_VTR_EL2.PREbits.

Writing to the active priority registers in any order other than the following order will result in UNPREDICTABLE behavior:

- AArch64-ICC_AP0R<n>_EL1.
- Secure ICC_AP1R<n>_EL1.
- Non-secure ICC_AP1R<n>_EL1.

MRS <Xt>, ICC_AP1R0_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TALL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.IMO == '1' then
        return ICV_AP1R0_EL1;
    elseif SCR_EL3.IRQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            if SCR_EL3.NS == '0' then
                return ICC_AP1R0_EL1_S;
            else
                return ICC_AP1R0_EL1_NS;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
            UNDEFINED;
        elseif SCR_EL3.IRQ == '1' then
            if Halted() && EDSCR.SDD == '1' then

```



```

        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            return ICC_AP1R0_EL1_S;
        else
            return ICC_AP1R0_EL1_NS;
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        return ICC_AP1R0_EL1_S;
    else
        return ICC_AP1R0_EL1_NS;

```

MSR ICC_AP1R0_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TALL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.IMO == '1' then
        ICV_AP1R0_EL1 = X[t];
    elseif SCR_EL3.IRQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_AP1R0_EL1_S = X[t];
        else
            ICC_AP1R0_EL1_NS = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
        UNDEFINED;
    elseif SCR_EL3.IRQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_AP1R0_EL1_S = X[t];
        else
            ICC_AP1R0_EL1_NS = X[t];
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        ICC_AP1R0_EL1_S = X[t];
    else
        ICC_AP1R0_EL1_NS = X[t];

```

A.6.4 ICV_AP1R0_EL1, Interrupt Controller Virtual Active Priorities Group 1 Registers

Provides information about virtual Group 1 active priorities.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

GIC system registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-103: AArch64_icv_ap1r0_el1 bit assignments

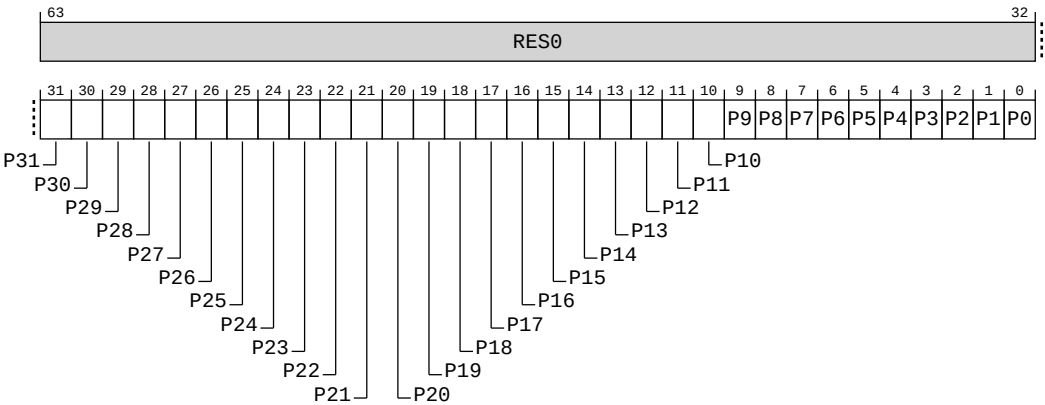


Table A-246: ICV_AP1R0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[31:0]	P<x>	Group 1 interrupt active priorities. Possible values of each bit are: 0b0 There is no Group 1 interrupt active with this priority level, or all active Group 1 interrupts with this priority level have undergone priority-drop. 0b1 There is a Group 1 interrupt active with this priority level which has not undergone priority drop. There are 32 preemption levels, and the active state of these preemption levels are held in the bits corresponding to Priority[7:3].	32 {x}

The contents of these registers are **IMPLEMENTATION DEFINED** with the one architectural requirement that the value 0x00000000 is consistent with no interrupts being active.

Access

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 1 active priorities) might result in **UNPREDICTABLE** behavior of the virtual interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.
- Interrupts that should not preempt execution to preempt execution.

ICV_AP1R1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICV_AP1R2_EL1 and ICV_AP1R3_EL1 are only implemented in implementations that support 7 bits of priority. Unimplemented registers are **UNDEFINED**.

Writing to the active priority registers in any order other than the following order might result in **UNPREDICTABLE** behavior of the interrupt prioritization system:

- AArch64-ICV_AP0R<n>_EL1.
- ICV_AP1R<n>_EL1.

MRS <Xt>, ICC_AP1R0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1001	0b000

MSR ICC_AP1R0_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1001	0b000

Accessibility

Writing to these registers with any value other than the last read value of the register (or 0x00000000 when there are no Group 1 active priorities) might result in **UNPREDICTABLE** behavior of the virtual interrupt prioritization system, causing:

- Interrupts that should preempt execution to not preempt execution.

- Interrupts that should not preempt execution to preempt execution.

ICV_AP1R1_EL1 is only implemented in implementations that support 6 or more bits of priority. ICV_AP1R2_EL1 and ICV_AP1R3_EL1 are only implemented in implementations that support 7 bits of priority. Unimplemented registers are UNDEFINED.

Writing to the active priority registers in any order other than the following order might result in UNPREDICTABLE behavior of the interrupt prioritization system:

- AArch64-ICV_AP0R<n>_EL1.
- ICV_AP1R<n>_EL1.

MRS <Xt>, ICC_AP1R0_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
        UNDEFINED;
    elsif EL2Enabled() && ICH_HCR_EL2.TALL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && HCR_EL2.IMO == '1' then
        return ICV_AP1R0_EL1;
    elsif SCR_EL3.IRQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            if SCR_EL3.NS == '0' then
                return ICC_AP1R0_EL1_S;
            else
                return ICC_AP1R0_EL1_NS;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
            UNDEFINED;
        elsif SCR_EL3.IRQ == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            if SCR_EL3.NS == '0' then
                return ICC_AP1R0_EL1_S;
            else
                return ICC_AP1R0_EL1_NS;
    elsif PSTATE.EL == EL3 then
        if SCR_EL3.NS == '0' then
            return ICC_AP1R0_EL1_S;
        else
            return ICC_AP1R0_EL1_NS;

```

MSR ICC_AP1R0_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
        UNDEFINED;
    elsif EL2Enabled() && ICH_HCR_EL2.TALL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && HCR_EL2.IMO == '1' then
        ICV_AP1R0_EL1 = X[t];

```

```

elseif SCR_EL3.IRQ == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_AP1R0_EL1_S = X[t];
        else
            ICC_AP1R0_EL1_NS = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.IRQ == '1' then
        UNDEFINED;
    elseif SCR_EL3.IRQ == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_AP1R0_EL1_S = X[t];
        else
            ICC_AP1R0_EL1_NS = X[t];
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        ICC_AP1R0_EL1_S = X[t];
    else
        ICC_AP1R0_EL1_NS = X[t];

```

A.6.5 ICC_CTLR_EL1, Interrupt Controller Control Register (EL1)

Controls aspects of the behavior of the GIC CPU interface and provides information about the features implemented.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

GIC system registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-104: AArch64_icc_ctlr_el1 bit assignments

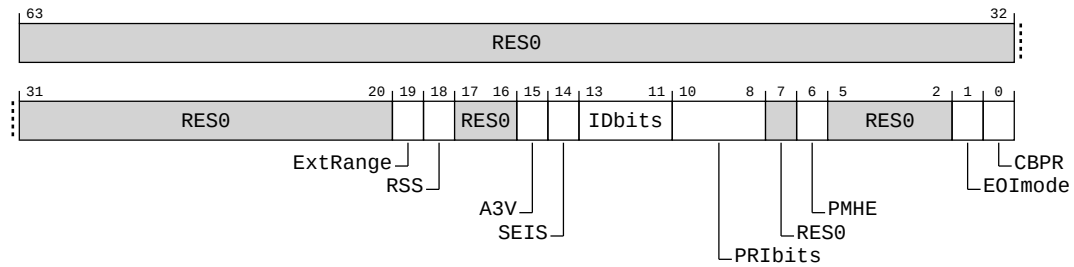


Table A-249: ICC_CTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:20]	RES0	Reserved	RES0
[19]	ExtRange	Extended INTID range (read-only). 0b1 CPU interface supports INTIDs in the range 1024..8191 All INTIDs in the range 1024..8191 are treated as requiring deactivation.	x
[18]	RSS	Range Selector Support. Possible values are: 0b0 Targeted SGLs with affinity level 0 values of 0 - 15 are supported.	x
[17:16]	RES0	Reserved	RES0
[15]	A3V	Affinity 3 Valid. Read-only and writes are ignored. Possible values are: 0b1 The CPU interface logic supports non-zero values of Affinity 3 in SGI generation System registers.	x
[14]	SEIS	SEI Support. Read-only and writes are ignored. Indicates whether the CPU interface supports local generation of SEIs: 0b0 The CPU interface logic does not support local generation of SEIs.	x
[13:11]	IDbits	Identifier bits. Read-only and writes are ignored. The number of physical interrupt identifier bits supported: 0b000 16 bits.	xxx

Bits	Name	Description	Reset
[10:8]	PRIbits	Priority bits. Read-only and writes are ignored. The number of priority bits implemented, minus one. An implementation that supports two Security states must implement at least 32 levels of physical priority (5 priority bits). An implementation that supports only a single Security state must implement at least 16 levels of physical priority (4 priority bits). Note: This field always returns the number of priority bits implemented, regardless of the Security state of the access or the value of ext-GICD_CTLR.DS. For physical accesses, this field determines the minimum value of AArch64-ICC_BPR0_EL1. If EL3 is implemented, physical accesses return the value from AArch64-ICC_CTLR_EL3.PRIbits. 0b100 5 bits of priority are implemented	xxx
[7]	RES0	Reserved	RES0
[6]	PMHE	Priority Mask Hint Enable. Controls whether the priority mask register is used as a hint for interrupt distribution: 0b0 Disables use of AArch64-ICC_PMR_EL1 as a hint for interrupt distribution. 0b1 Enables use of AArch64-ICC_PMR_EL1 as a hint for interrupt distribution. If EL3 is implemented, this bit is an alias of AArch64-ICC_CTLR_EL3.PMHE. Whether this bit can be written as part of an access to this register depends on the value of ext-GICD_CTLR.DS: - If ext-GICD_CTLR.DS == 0, this bit is read-only. - If ext-GICD_CTLR.DS == 1, this bit is read/write.	x
[5:2]	RES0	Reserved	RES0
[1]	EOImode	EOI mode for the current Security state. Controls whether a write to an End of Interrupt register also deactivates the interrupt: 0b0 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide both priority drop and interrupt deactivation functionality. Accesses to AArch64-ICC_DIR_EL1 are UNPREDICTABLE. 0b1 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide priority drop functionality only. AArch64-ICC_DIR_EL1 provides interrupt deactivation functionality. The Secure AArch64-ICC_CTLR_EL1.EOImode is an alias of AArch64-ICC_CTLR_EL3.EOImode_EL1S. The Non-secure AArch64-ICC_CTLR_EL1.EOImode is an alias of AArch64-ICC_CTLR_EL3.EOImode_EL1NS	x

Bits	Name	Description	Reset
[0]	CBPR	Common Binary Point Register. Controls whether the same register is used for interrupt preemption of both Group 0 and Group 1 interrupts: 0b0 AArch64-ICC_BPRO_EL1 determines the preemption group for Group 0 interrupts only. AArch64-ICC_BPR1_EL1 determines the preemption group for Group 1 interrupts. 0b1 AArch64-ICC_BPRO_EL1 determines the preemption group for both Group 0 and Group 1 interrupts. If EL3 is implemented: - This bit is an alias of AArch64-ICC_CTLR_EL3.CBPR_EL1{S,NS} where S or NS corresponds to the current Security state. - If ext-GICD_CTLR.DS == 0, this bit is read-only. - If ext-GICD_CTLR.DS == 1, this bit is read/write.	x

Access

MRS <Xt>, ICC_CTLR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1100	0b100

MSR ICC_CTLR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1100	0b100

Accessibility

MRS <Xt>, ICC_CTLR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elsif EL2Enabled() && ICH_HCR_EL2.TC == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && HCR_EL2.FMO == '1' then
        return ICV_CTLR_EL1;
    elsif EL2Enabled() && HCR_EL2.IMO == '1' then
        return ICV_CTLR_EL1;
    elsif SCR_EL3.<IRQ,FIQ> == '11' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end
    else
        if SCR_EL3.NS == '0' then
            return ICC_CTLR_EL1_S;
        else
            return ICC_CTLR_EL1_NS;
        end
    end
elsif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elsif SCR_EL3.<IRQ,FIQ> == '11' then

```



```

        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            return ICC_CTLR_EL1_S;
        else
            return ICC_CTLR_EL1_NS;
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        return ICC_CTLR_EL1_S;
    else
        return ICC_CTLR_EL1_NS;

```

MSR ICC_CTLR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TC == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FMO == '1' then
        ICC_CTLR_EL1 = X[t];
    elseif EL2Enabled() && HCR_EL2.IMO == '1' then
        ICC_CTLR_EL1 = X[t];
    elseif SCR_EL3.<IRQ,FIQ> == '11' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_CTLR_EL1_S = X[t];
        else
            ICC_CTLR_EL1_NS = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elseif SCR_EL3.<IRQ,FIQ> == '11' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_CTLR_EL1_S = X[t];
        else
            ICC_CTLR_EL1_NS = X[t];
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        ICC_CTLR_EL1_S = X[t];
    else
        ICC_CTLR_EL1_NS = X[t];

```

A.6.6 ICV_CTLR_EL1, Interrupt Controller Virtual Control Register

Controls aspects of the behavior of the GIC virtual CPU interface and provides information about the features implemented.

Configurations

This register is available in all configurations.

Attributes

Width

64

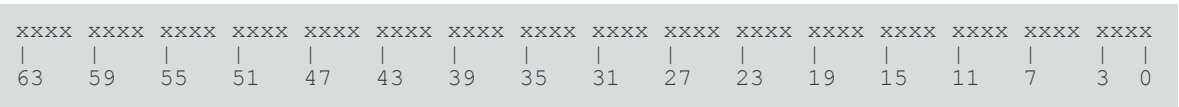
Functional group

GIC system registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-105: AArch64_icv_ctlr_el1 bit assignments

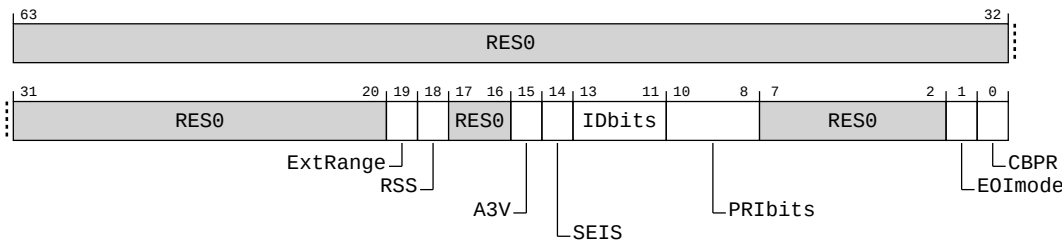


Table A-252: ICV_CTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:20]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[19]	ExtRange	Extended INTID range (read-only). 0b1 CPU interface supports INTIDs in the range 1024..8191 All INTIDs in the range 1024..8191 are treated as requiring deactivation.	x
[18]	RSS	Range Selector Support. Possible values are: 0b0 Targeted SGLs with affinity level 0 values of 0 - 15 are supported.	x
[17:16]	RES0	Reserved	RES0
[15]	A3V	Affinity 3 Valid. Read-only and writes are ignored. Possible values are: 0b1 The virtual CPU interface logic supports non-zero values of Affinity 3 in SGI generation System registers.	x
[14]	SEIS	SEI Support. Read-only and writes are ignored. Indicates whether the virtual CPU interface supports local generation of SEIs: 0b0 The virtual CPU interface logic does not support local generation of SEIs.	x
[13:11]	IDbits	Identifier bits. Read-only and writes are ignored. The number of virtual interrupt identifier bits supported: 0b000 16 bits.	xxx
[10:8]	PRIbits	Priority bits. Read-only and writes are ignored. The number of priority bits implemented, minus one. An implementation must implement at least 32 levels of physical priority (5 priority bits). Note: This field always returns the number of priority bits implemented. The division between group priority and subpriority is defined in the binary point registers AArch64-ICV_BPR0_EL1 and AArch64-ICV_BPR1_EL1. 0b100 5 bits of priority are implemented	xxx
[7:2]	RES0	Reserved	RES0
[1]	EOImode	Virtual EOI mode. Controls whether a write to an End of Interrupt register also deactivates the virtual interrupt: 0b0 AArch64-ICV_EOIR0_EL1 and AArch64-ICV_EOIR1_EL1 provide both priority drop and interrupt deactivation functionality. Accesses to AArch64-ICV_DIR_EL1 are UNPREDICTABLE . 0b1 AArch64-ICV_EOIR0_EL1 and AArch64-ICV_EOIR1_EL1 provide priority drop functionality only. AArch64-ICV_DIR_EL1 provides interrupt deactivation functionality.	x

Bits	Name	Description	Reset
[0]	CBPR	Common Binary Point Register. Controls whether the same register is used for interrupt preemption of both virtual Group 0 and virtual Group 1 interrupts: 0b0 AArch64-ICV_BPR1_EL1 determines the preemption group for virtual Group 1 interrupts. 0b1 Non-secure reads of AArch64-ICV_BPR1_EL1 return AArch64-ICV_BPR0_EL1 plus one, saturated to 0b111. Non-secure writes to AArch64-ICV_BPR1_EL1 are ignored. Secure reads of AArch64-ICV_BPR1_EL1 return AArch64-ICV_BPR0_EL1. Secure writes of AArch64-ICV_BPR1_EL1 modify AArch64-ICV_BPR0_EL1.	x

Access

MRS <Xt>, ICC_CTLR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1100	0b100

MSR ICC_CTLR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1100	0b1100	0b100

Accessibility

MRS <Xt>, ICC_CTLR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TC == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FMO == '1' then
        return ICV_CTLR_EL1;
    elseif EL2Enabled() && HCR_EL2.IMO == '1' then
        return ICV_CTLR_EL1;
    elseif SCR_EL3.<IRQ,FIQ> == '11' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            if SCR_EL3.NS == '0' then
                return ICC_CTLR_EL1_S;
            else
                return ICC_CTLR_EL1_NS;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
            UNDEFINED;
        elseif SCR_EL3.<IRQ,FIQ> == '11' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
        else
            if SCR_EL3.NS == '0' then

```

```

        return ICC_CTLR_EL1_S;
    else
        return ICC_CTLR_EL1_NS;
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        return ICC_CTLR_EL1_S;
    else
        return ICC_CTLR_EL1_NS;

```

MSR ICC_CTLR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elseif EL2Enabled() && ICH_HCR_EL2.TC == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.FMO == '1' then
        ICV_CTLR_EL1 = X[t];
    elseif EL2Enabled() && HCR_EL2.IMO == '1' then
        ICV_CTLR_EL1 = X[t];
    elseif SCR_EL3.<IRQ,FIQ> == '11' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_CTLR_EL1_S = X[t];
        else
            ICC_CTLR_EL1_NS = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.<IRQ,FIQ> == '11' then
        UNDEFINED;
    elseif SCR_EL3.<IRQ,FIQ> == '11' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        if SCR_EL3.NS == '0' then
            ICC_CTLR_EL1_S = X[t];
        else
            ICC_CTLR_EL1_NS = X[t];
elseif PSTATE.EL == EL3 then
    if SCR_EL3.NS == '0' then
        ICC_CTLR_EL1_S = X[t];
    else
        ICC_CTLR_EL1_NS = X[t];

```

A.6.7 ICH_VTR_EL2, Interrupt Controller VGIC Type Register

Reports supported GIC virtualization features.

Configurations

If EL2 is not implemented, all bits in this register are RES0 from EL3, except for nV4, which is RES1 from EL3.

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

Functional group

GIC system registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1001	0000	0010	10xx	xxxx	xxxx	xxx0	0011
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-106: AArch64_ich_vtr_el2 bit assignments

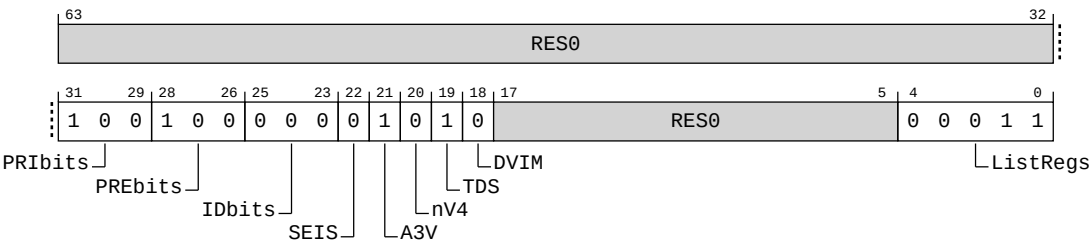


Table A-255: ICH_VTR_EL2 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:29]	PRIbits	Priority bits. The number of virtual priority bits implemented, minus one. An implementation must implement at least 32 levels of virtual priority (5 priority bits). This field is an alias of AArch64-ICV_CTLR_EL1.PRIbits. 0b100 5 virtual priority bits are implemented	0b100

Bits	Name	Description	Reset
[28:26]	PREbits	The number of virtual preemption bits implemented, minus one. An implementation must implement at least 32 levels of virtual preemption priority (5 preemption bits). The value of this field must be less than or equal to the value of ICH_VTR_EL2.PRIBits. The maximum value of this field is 6, indicating 7 bits of preemption. This field determines the minimum value of AArch64-ICH_VMCR_EL2.VBPR0. 0b100 5 virtual preemption bits are implemented	0b100
[25:23]	IDbits	The number of virtual interrupt identifier bits supported: 0b000 16 bits.	0b000
[22]	SEIS	SEI Support. Indicates whether the virtual CPU interface supports generation of SEIs: 0b0 The virtual CPU interface logic does not support generation of SEIs.	0b0
[21]	A3V	Affinity 3 Valid. Possible values are: 0b1 The virtual CPU interface logic supports non-zero values of Affinity 3 in SGI generation System registers.	0b1
[20]	nV4	Direct injection of virtual interrupts not supported. Possible values are: 0b0 The CPU interface logic supports direct injection of virtual interrupts.	0b0
[19]	TDS	Separate trapping of EL1 writes to AArch64-ICV_DIR_EL1 supported. 0b1 Implementation supports AArch64-ICH_HCR_EL2.TDIR.	0b1
[18]	DVIM	Masking of directly-injected virtual interrupts. 0b0 Masking of Directly-injected Virtual Interrupts not supported.	0b0
[17:5]	RES0	Reserved	RES0
[4:0]	ListRegs	The number of implemented List registers, minus one. For example, a value of 0b01111 indicates that the maximum of 16 List registers are implemented. 0b00011 Four list registers are implemented.	0b00011

Access

MRS <Xt>, ICH_VTR_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b1100	0b1011	0b001

Accessibility

MRS <Xt>, ICH_VTR_EL2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    return ICH_VTR_EL2;
elsif PSTATE.EL == EL3 then
    return ICH_VTR_EL2;
```

A.6.8 ICC_CTLR_EL3, Interrupt Controller Control Register (EL3)

Controls aspects of the behavior of the GIC CPU interface and provides information about the features implemented.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

GIC system registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x0xx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-107: AArch64_icc_ctlr_el3 bit assignments

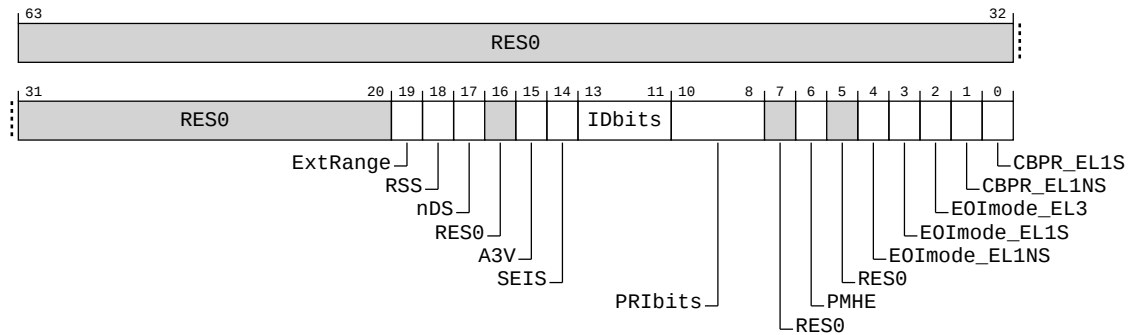


Table A-257: ICC_CTLR_EL3 bit descriptions

Bits	Name	Description	Reset
[63:20]	RES0	Reserved	RES0
[19]	ExtRange	Extended INTID range (read-only). 0b1 CPU interface supports INTIDs in the range 1024..8191 All INTIDs in the range 1024..8191 are treated as requiring deactivation.	x
[18]	RSS	Range Selector Support. 0b0 Targeted SGIs with affinity level 0 values of 0-15 are supported.	x
[17]	nDS	Disable Security not supported. Read-only and writes are ignored. 0b1 The CPU interface logic does not support disabling of security, and requires that security is not disabled.	x
[16]	RES0	Reserved	RES0
[15]	A3V	Affinity 3 Valid. Read-only and writes are ignored. 0b1 The CPU interface logic supports non-zero values of the Aff3 field in SGI generation System registers.	x
[14]	SEIS	SEI Support. Read-only and writes are ignored. Indicates whether the CPU interface supports generation of SEIs: 0b0 The CPU interface logic does not support generation of SEIs.	x
[13:11]	IDbits	Identifier bits. Read-only and writes are ignored. Indicates the number of physical interrupt identifier bits supported. 0b000 16 bits.	xxx

Bits	Name	Description	Reset
[10:8]	PRIbits	Priority bits. Read-only and writes are ignored. The number of priority bits implemented, minus one. An implementation that supports two Security states must implement at least 32 levels of physical priority (5 priority bits). An implementation that supports only a single Security state must implement at least 16 levels of physical priority (4 priority bits). Note: This field always returns the number of priority bits implemented, regardless of the value of SCR_EL3.NS or the value of ext-GICD_CTLR.DS. The division between group priority and subpriority is defined in the binary point registers AArch64-ICC_BPRO_EL1 and AArch64-ICC_BPR1_EL1. This field determines the minimum value of ICC_BPRO_EL1. 0b100 5 bits of priority are implemented	xxx
[7]	RES0	Reserved	RES0
[6]	PMHE	Priority Mask Hint Enable. 0b0 Disables use of the priority mask register as a hint for interrupt distribution. 0b1 Enables use of the priority mask register as a hint for interrupt distribution. Software must write AArch64-ICC_PMR_EL1 to 0xFF before clearing this field to 0. - An implementation might choose to make this field RAO/WI if priority-based routing is always used - An implementation might choose to make this field RAZ/WI if priority-based routing is never used If EL3 is present, AArch64-ICC_CTLR_EL1.PMHE is an alias of ICC_CTLR_EL3.PMHE.	0b0
[5]	RES0	Reserved	RES0
[4]	EOImode_EL1NS	EOI mode for interrupts handled at Non-secure EL1 and EL2. Controls whether a write to an End of Interrupt register also deactivates the interrupt. 0b0 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide both priority drop and interrupt deactivation functionality. Accesses to AArch64-ICC_DIR_EL1 are UNPREDICTABLE. 0b1 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide priority drop functionality only. AArch64-ICC_DIR_EL1 provides interrupt deactivation functionality. If EL3 is present, AArch64-ICC_CTLR_EL1(NS).EOImode is an alias of ICC_CTLR_EL3.EOImode_EL1NS.	x

Bits	Name	Description	Reset
[3]	EOImode_EL1S	EOI mode for interrupts handled at Secure EL1 and EL2. Controls whether a write to an End of Interrupt register also deactivates the interrupt. 0b0 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide both priority drop and interrupt deactivation functionality. Accesses to AArch64-ICC_DIR_EL1 are UNPREDICTABLE . 0b1 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide priority drop functionality only. AArch64-ICC_DIR_EL1 provides interrupt deactivation functionality. If EL3 is present, AArch64-ICC_CTLR_EL1(S).EOImode is an alias of ICC_CTLR_EL3.EOImode_EL1S.	x
[2]	EOImode_EL3	EOI mode for interrupts handled at EL3. Controls whether a write to an End of Interrupt register also deactivates the interrupt. 0b0 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide both priority drop and interrupt deactivation functionality. Accesses to AArch64-ICC_DIR_EL1 are UNPREDICTABLE . 0b1 AArch64-ICC_EOIR0_EL1 and AArch64-ICC_EOIR1_EL1 provide priority drop functionality only. AArch64-ICC_DIR_EL1 provides interrupt deactivation functionality.	x
[1]	CBPR_EL1NS	Common Binary Point Register, EL1 Non-secure. Controls whether the same register is used for interrupt preemption of both Group 0 and Group 1 Non-secure interrupts at EL1 and EL2. 0b0 AArch64-ICC_BPR0_EL1 determines the preemption group for Group 0 interrupts only. AArch64-ICC_BPR1_EL1 determines the preemption group for Non-secure Group 1 interrupts. 0b1 AArch64-ICC_BPR0_EL1 determines the preemption group for Group 0 interrupts and Non-secure Group 1 interrupts. Non-secure accesses to ext-GICC_BPR and AArch64-ICC_BPR1_EL1 access the state of AArch64-ICC_BPR0_EL1. If EL3 is present, AArch64-ICC_CTLR_EL1(NS).CBPR is an alias of ICC_CTLR_EL3.CBPR_EL1NS.	x
[0]	CBPR_EL1S	Common Binary Point Register, EL1 Secure. Controls whether the same register is used for interrupt preemption of both Group 0 and Group 1 Secure interrupts at EL1 and EL2. 0b0 AArch64-ICC_BPR0_EL1 determines the preemption group for Group 0 interrupts only. AArch64-ICC_BPR1_EL1 determines the preemption group for Secure Group 1 interrupts. 0b1 AArch64-ICC_BPR0_EL1 determines the preemption group for Group 0 interrupts and Secure Group 1 interrupts. Secure EL1 accesses to AArch64-ICC_BPR1_EL1 access the state of AArch64-ICC_BPR0_EL1. If EL3 is present, AArch64-ICC_CTLR_EL1(S).CBPR is an alias of ICC_CTLR_EL3.CBPR_EL1S.	x

Access

MRS <Xt>, ICC_CTLR_EL3

op0	op1	CRn	CRm	op2
0b11	0b110	0b1100	0b1100	0b100

MSR ICC_CTLR_EL3, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b110	0b1100	0b1100	0b100

Accessibility

MRS <Xt>, ICC_CTLR_EL3

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    UNDEFINED;
elsif PSTATE.EL == EL3 then
    return ICC_CTLR_EL3;

```

MSR ICC_CTLR_EL3, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    UNDEFINED;
elsif PSTATE.EL == EL2 then
    UNDEFINED;
elsif PSTATE.EL == EL3 then
    ICC_CTLR_EL3 = X[t];

```

A.7 AArch64 Identification registers summary

The summary table provides an overview of all Identification registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-260: Identification registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
MIDR_EL1	3	0	C0	C0	0	See individual bit resets.	64-bit	Main ID Register
MPIDR_EL1	3	0	C0	C0	5	See individual bit resets.	64-bit	Multiprocessor Affinity Register
REVIDR_EL1	3	0	C0	C0	6	See individual bit resets.	64-bit	Revision ID Register
ID_PFR0_EL1	3	0	C0	C1	0	See individual bit resets.	64-bit	AArch32 Processor Feature Register 0

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ID_PFR1_EL1	3	0	C0	C1	1	See individual bit resets.	64-bit	AArch32 Processor Feature Register 1
ID_DFR0_EL1	3	0	C0	C1	2	See individual bit resets.	64-bit	AArch32 Debug Feature Register 0
ID_AFR0_EL1	3	0	C0	C1	3	See individual bit resets.	64-bit	AArch32 Auxiliary Feature Register 0
ID_MMFR0_EL1	3	0	C0	C1	4	See individual bit resets.	64-bit	AArch32 Memory Model Feature Register 0
ID_MMFR1_EL1	3	0	C0	C1	5	See individual bit resets.	64-bit	AArch32 Memory Model Feature Register 1
ID_MMFR2_EL1	3	0	C0	C1	6	See individual bit resets.	64-bit	AArch32 Memory Model Feature Register 2
ID_MMFR3_EL1	3	0	C0	C1	7	See individual bit resets.	64-bit	AArch32 Memory Model Feature Register 3
ID_ISAR0_EL1	3	0	C0	C2	0	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 0
ID_ISAR1_EL1	3	0	C0	C2	1	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 1
ID_ISAR2_EL1	3	0	C0	C2	2	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 2
ID_ISAR3_EL1	3	0	C0	C2	3	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 3
ID_ISAR4_EL1	3	0	C0	C2	4	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 4
ID_ISAR5_EL1	3	0	C0	C2	5	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 5
ID_MMFR4_EL1	3	0	C0	C2	6	See individual bit resets.	64-bit	AArch32 Memory Model Feature Register 4
ID_ISAR6_EL1	3	0	C0	C2	7	See individual bit resets.	64-bit	AArch32 Instruction Set Attribute Register 6
MVFR0_EL1	3	0	C0	C3	0	See individual bit resets.	64-bit	AArch32 Media and VFP Feature Register 0
MVFR1_EL1	3	0	C0	C3	1	See individual bit resets.	64-bit	AArch32 Media and VFP Feature Register 1
MVFR2_EL1	3	0	C0	C3	2	See individual bit resets.	64-bit	AArch32 Media and VFP Feature Register 2
ID_PFR2_EL1	3	0	C0	C3	4	See individual bit resets.	64-bit	AArch32 Processor Feature Register 2
ID_DFR1_EL1	3	0	C0	C3	5	See individual bit resets.	64-bit	Debug Feature Register 1
ID_MMFR5_EL1	3	0	C0	C3	6	See individual bit resets.	64-bit	AArch32 Memory Model Feature Register 5
ID_AA64PFR0_EL1	3	0	C0	C4	0	See individual bit resets.	64-bit	AArch64 Processor Feature Register 0
ID_AA64PFR1_EL1	3	0	C0	C4	1	See individual bit resets.	64-bit	AArch64 Processor Feature Register 1
ID_AA64ZFR0_EL1	3	0	C0	C4	4	See individual bit resets.	64-bit	SVE Feature ID register 0
ID_AA64SMFR0_EL1	3	0	C0	C4	5	See individual bit resets.	64-bit	SME Feature ID register 0
ID_AA64DFR0_EL1	3	0	C0	C5	0	See individual bit resets.	64-bit	AArch64 Debug Feature Register 0
ID_AA64DFR1_EL1	3	0	C0	C5	1	See individual bit resets.	64-bit	AArch64 Debug Feature Register 1
ID_AA64AFR0_EL1	3	0	C0	C5	4	See individual bit resets.	64-bit	AArch64 Auxiliary Feature Register 0
ID_AA64AFR1_EL1	3	0	C0	C5	5	See individual bit resets.	64-bit	AArch64 Auxiliary Feature Register 1
ID_AA64ISAR0_EL1	3	0	C0	C6	0	See individual bit resets.	64-bit	AArch64 Instruction Set Attribute Register 0
ID_AA64ISAR1_EL1	3	0	C0	C6	1	See individual bit resets.	64-bit	AArch64 Instruction Set Attribute Register 1
ID_AA64ISAR2_EL1	3	0	C0	C6	2	See individual bit resets.	64-bit	AArch64 Instruction Set Attribute Register 2
ID_AA64MMFR0_EL1	3	0	C0	C7	0	See individual bit resets.	64-bit	AArch64 Memory Model Feature Register 0
ID_AA64MMFR1_EL1	3	0	C0	C7	1	See individual bit resets.	64-bit	AArch64 Memory Model Feature Register 1
ID_AA64MMFR2_EL1	3	0	C0	C7	2	See individual bit resets.	64-bit	AArch64 Memory Model Feature Register 2
ID_AA64MMFR3_EL1	3	0	C0	C7	3	See individual bit resets.	64-bit	AArch64 Memory Model Feature Register 2
MPAMIDR_EL1	3	0	C10	C4	4	See individual bit resets.	64-bit	MPAM ID Register (EL1)
CCSIDR_EL1	3	1	C0	C0	0	See individual bit resets.	64-bit	Current Cache Size ID Register
CLIDR_EL1	3	1	C0	C0	1	See individual bit resets.	64-bit	Cache Level ID Register
GMID_EL1	3	1	C0	C0	4	See individual bit resets.	64-bit	Multiple tag transfer ID register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
CSSELR_EL1	3	2	C0	C0	0	See individual bit resets.	64-bit	Cache Size Selection Register
CTR_EL0	3	3	C0	C0	1	See individual bit resets.	64-bit	Cache Type Register
DCZID_EL0	3	3	C0	C0	7	See individual bit resets.	64-bit	Data Cache Zero ID register
VPIDR_EL2	3	4	C0	C0	0	See individual bit resets.	64-bit	Virtualization Processor ID Register
VMPIDR_EL2	3	4	C0	C0	5	See individual bit resets.	64-bit	Virtualization Multiprocessor ID Register

A.7.1 MIDR_EL1, Main ID Register

Provides identification information for the PE, including an implementer code for the device and a device ID number.

Configurations

AArch64 register MIDR_EL1 bits [31:0] are architecturally mapped to External System register [B.3.4 MIDR_EL1, Main ID Register](#) on page 623 bits [31:0].

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0100	0001	0000	1111	1101	1000	1110	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-108: AArch64_midr_el1 bit assignments

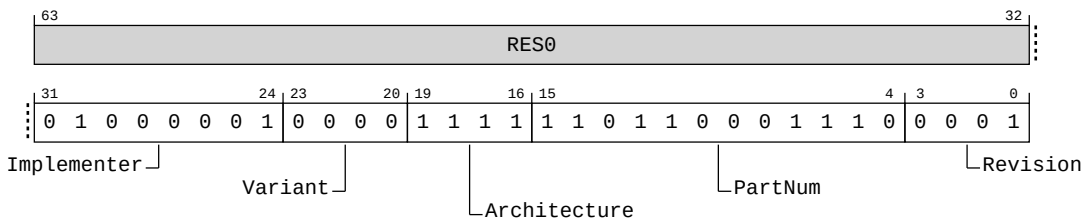


Table A-261: MIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	Implementer	The Implementer code. This field must hold an implementer code that has been assigned by Arm. Assigned codes include the following: 0b01000001 Arm Limited.	0x41
[23:20]	Variant	Variant number. Typically, this field is used to distinguish between different product variants, or major revisions of a product. 0b0000	0b0000
[19:16]	Architecture	Architecture version. Defined values are: 0b1111 Architectural features are individually identified in the ID_* registers.	0b1111
[15:4]	PartNum	Primary Part Number for the device. On processors implemented by Arm, if the top four bits of the primary part number are 0x0 or 0x7, the variant and architecture are encoded differently. 0b110110001110 Neoverse N3	0xD8E
[3:0]	Revision	Indicates the minor revision of the product. 0b0001 rOp1	0b0001

Access

MRS <Xt>, MIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0000	0b000

Accessibility

MRS <Xt>, MIDR_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
```

```
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif PSTATE.EL == EL1 then
        if EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.MIDR_EL1 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() then
            return VPIDR_EL2;
        else
            return MIDR_EL1;
    elsif PSTATE.EL == EL2 then
        return MIDR_EL1;
    elsif PSTATE.EL == EL3 then
        return MIDR_EL1;
```

A.7.2 MPIDR_EL1, Multiprocessor Affinity Register

In a multiprocessor system, provides an additional PE identification mechanism for scheduling purposes.

Configurations

In a uniprocessor system, Arm recommends that each Aff<n> field of this register returns a value of 0.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-109: AArch64_mpidr_el1 bit assignments

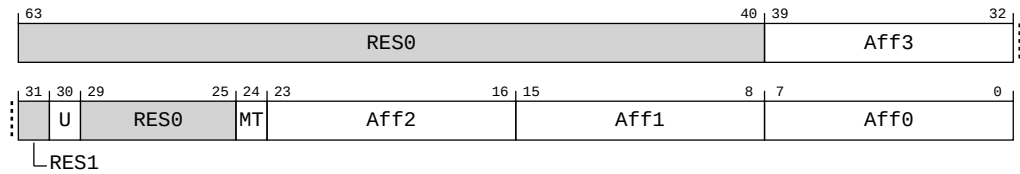


Table A-263: MPIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:40]	RES0	Reserved	RES0
[39:32]	Aff3	Affinity level 3. See the description of Aff0 for more information. The value will be determined by the CLUSTERIDAFF3 configuration pins.	8 {x}
[31]	RES1	Reserved	RES1
[30]	U	Indicates a Uniprocessor system, as distinct from PE 0 in a multiprocessor system. 0b0 Processor is part of a multiprocessor system.	x
[29:25]	RES0	Reserved	RES0
[24]	MT	Indicates whether the lowest level of affinity consists of logical PEs that are implemented using a multithreading type approach. See the description of Aff0 for more information about affinity levels. 0b1 Performance of PEs with different affinity level 0 values, and the same values for affinity level 1 and higher, is very interdependent.	x
[23:16]	Aff2	Affinity level 2. See the description of Aff0 for more information. The value will be determined by the CLUSTERIDAFF2 configuration pins.	8 {x}
[15:8]	Aff1	Affinity level 1. See the description of Aff0 for more information. Identification number for each core in an cluster counting from zero.	8 {x}
[7:0]	Aff0	Affinity level 0. This is the affinity level that is most significant for determining PE behavior. Higher affinity levels are increasingly less significant in determining PE behavior. The assigned value of the MPIDR.{Aff2, Aff1, Aff0} or AArch64-MPIDR_EL1.{Aff3, Aff2, Aff1, Aff0} set of fields of each PE must be unique within the system as a whole. 0b00000000 Thread 0. Neoverse N3 is single-threaded.	8 {x}

Access

MRS <Xt>, MPIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0000	0b101

Accessibility

MRS <Xt>, MPIDR_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.MPIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() then
        return VMPIDR_EL2;
    else
        return MPIDR_EL1;
elseif PSTATE.EL == EL2 then
    return MPIDR_EL1;
elseif PSTATE.EL == EL3 then
    return MPIDR_EL1;
```

A.7.3 REVIDR_EL1, Revision ID Register

Provides implementation-specific minor revision information.

Configurations

If REVIDR_EL1 has the same value as AArch64-MIDR_EL1, then its contents have no significance.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-110: AArch64_revidr_el1 bit assignments

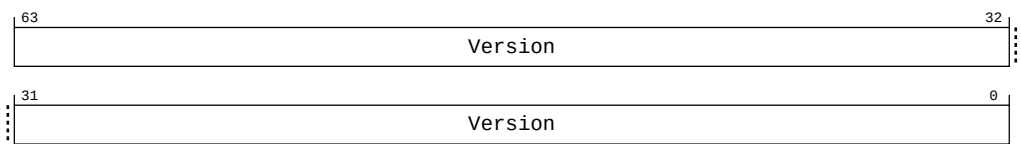


Table A-265: REVIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	Version	Identifies errata fixes present in this implementation. Refer to the Software Developer's Errata Notice or Product Errata Notice for information on how to interpret this field.	64 {x}

Access

MRS <Xt>, REVIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0000	0b110

Accessibility

MRS <Xt>, REVIDR_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGRTR_EL2.REVIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return REVIDR_EL1;
elseif PSTATE.EL == EL2 then
    return REVIDR_EL1;
elseif PSTATE.EL == EL3 then
    return REVIDR_EL1;
```

A.7.4 MVFR0_EL1, AArch32 Media and VFP Feature Register 0

Describes the features provided by the AArch32 Advanced SIMD and Floating-point implementation.

Must be interpreted with AArch64-MVFR1_EL1 and AArch64-MVFR2_EL1.

For general information about the interpretation of the ID registers see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

In an implementation where at least one Exception level supports execution in AArch32 state, but there is no support for Advanced SIMD and floating-point operation, this register is RAZ.

Attributes

Width

64

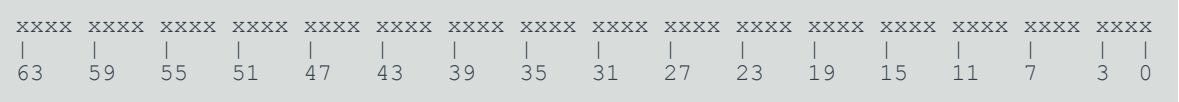
Functional group

Identification registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-111: AArch64_mvfr0_el1 bit assignments

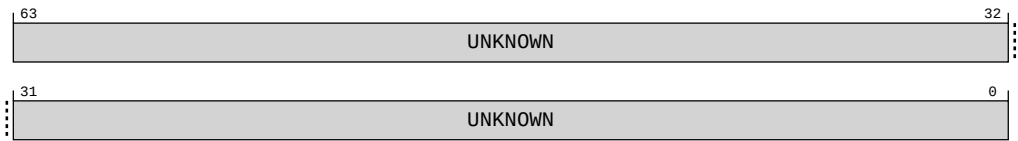


Table A-267: MVFR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	UNKNOWN	Reserved	UNKNOWN

Access

MRS <Xt>, MVFR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0011	0b000

Accessibility

MRS <Xt>, MVFR0_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return MVFR0_EL1;
elseif PSTATE.EL == EL2 then
    return MVFR0_EL1;
elseif PSTATE.EL == EL3 then
    return MVFR0_EL1;

```

A.7.5 MVFR1_EL1, AArch32 Media and VFP Feature Register 1

Describes the features provided by the AArch32 Advanced SIMD and Floating-point implementation.

Must be interpreted with AArch64-MVFR0_EL1 and AArch64-MVFR2_EL1.

For general information about the interpretation of the ID registers see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

In an implementation where at least one Exception level supports execution in AArch32 state, but there is no support for Advanced SIMD and floating-point operation, this register is RAZ.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-112: AArch64_mvfr1_el1 bit assignments

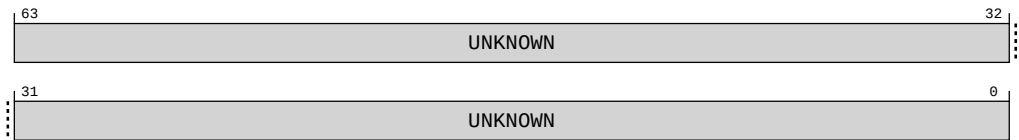


Table A-269: MVFR1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	UNKNOWN	Reserved	UNKNOWN

Access

MRS <Xt>, MVFR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0011	0b001

Accessibility

MRS <Xt>, MVFR1_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return MVFR1_EL1;
elseif PSTATE.EL == EL2 then
    return MVFR1_EL1;
elseif PSTATE.EL == EL3 then
    return MVFR1_EL1;
```

A.7.6 MVFR2_EL1, AArch32 Media and VFP Feature Register 2

Describes the features provided by the AArch32 Advanced SIMD and Floating-point implementation.

Must be interpreted with AArch64-MVFR0_EL1 and AArch64-MVFR1_EL1.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

In an implementation where at least one Exception level supports execution in AArch32 state, but there is no support for Advanced SIMD and floating-point operation, this register is RAZ.

Attributes

Width

64

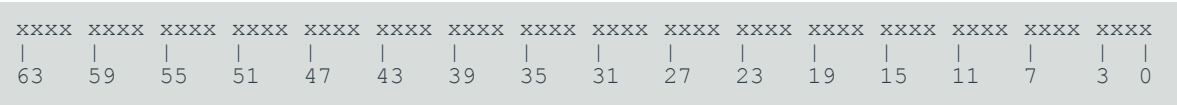
Functional group

Identification registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-113: AArch64_mvfr2_el1 bit assignments

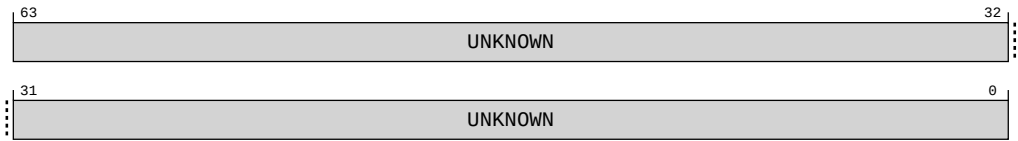


Table A-271: MVFR2_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	UNKNOWN	Reserved	UNKNOWN

Access

MRS <Xt>, MVFR2_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0011	0b010

Accessibility

MRS <Xt>, MVFR2_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return MVFR2_EL1;
elseif PSTATE.EL == EL2 then
    return MVFR2_EL1;
elseif PSTATE.EL == EL3 then
    return MVFR2_EL1;
```

A.7.7 ID_AA64PFR0_EL1, AArch64 Processor Feature Register 0

Provides additional information about implemented PE features in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

The external register ext-EDPFR gives information from this register.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0001	0010	0000	0001	0001	0001	0001	0001	0010	xxxx	0001	0001	0001	0001	0001	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-114: AArch64_id_aa64pfr0_el1 bit assignments

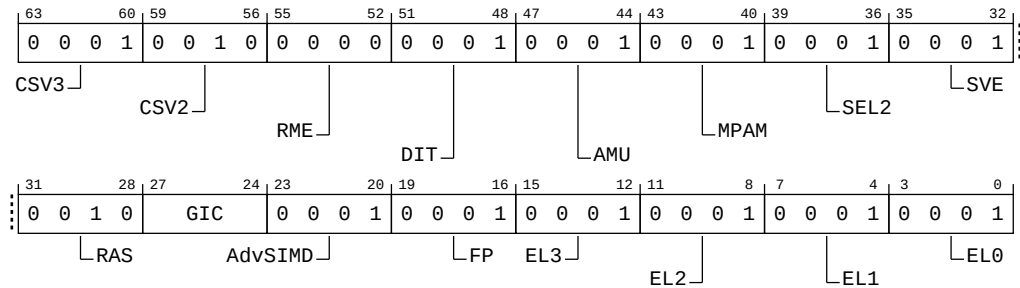


Table A-273: ID_AA64PFR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	CSV3	Speculative use of faulting data. Defined values are: 0b0001 Data loaded under speculation with a permission or domain fault cannot be used to form an address, generate condition codes, or generate SVE predicate values to be used by other instructions in the speculative sequence. The execution timing of any other instructions in the speculative sequence is not a function of the data loaded under speculation.	0b0001
[59:56]	CSV2	Speculative use of out of context branch targets. Defined values are: 0b0010 Branch targets trained in one hardware-described context can exploitatively control speculative execution in a different hardware-described context only in a hard-to-determine way. The SCXTNUM_ELx registers are supported and the contexts include the SCXTNUM_ELx register contexts.	0b0010
[55:52]	RME	Realm Management Extension (RME). Defined values are: 0b0000 Realm Management Extension not implemented.	0b0000
[51:48]	DIT	Data Independent Timing. Defined values are: 0b0001 AArch64 provides the PSTATE.DIT mechanism to guarantee constant execution time of certain instructions.	0b0001
[47:44]	AMU	Indicates support for Activity Monitors Extension. Defined values are: 0b0001 FEAT_AMUv1 is implemented.	0b0001
[43:40]	MPAM	Indicates support for MPAM Extension. Defined values are: 0b0001 If AArch64-ID_AA64PFR1_EL1.MPAM_frac == 0b0000, MPAM Extension version 1.0 is implemented. If AArch64-ID_AA64PFR1_EL1.MPAM_frac == 0b0001, MPAM Extension version 1.1 is implemented.	0b0001

Bits	Name	Description	Reset
[39:36]	SEL2	Secure EL2. Defined values are: 0b0001 Secure EL2 is implemented.	0b0001
[35:32]	SVE	Scalable Vector Extension. Defined values are: 0b0001 SVE architectural state and programmers' model are implemented.	0b0001
[31:28]	RAS	RAS Extension version. Defined values are: 0b0010 FEAT_RASv1p1 is implemented.	0b0010
[27:24]	GIC	System register GIC CPU interface. Defined values are: 0b0000 GIC CPU interface system registers not implemented. This value is reported when the GICCDISABLE input is HIGH. 0b0011 System register interface to version 4.1 of the GIC CPU interface is supported. This value is reported when the GICCDISABLE input is LOW.	The reset values can be the following: 0b0000, 0b0011, respective to the value.
[23:20]	AdvSIMD	Advanced SIMD. Defined values are: 0b0001 Advanced SIMD is implemented, including support for the following SISD and SIMD operations: - Integer byte, halfword, word and doubleword element operations. - Half-precision, single-precision and double-precision floating-point arithmetic. - Conversions between single-precision and half-precision data types, and double-precision and half-precision data types.	0b0001
[19:16]	FP	Floating-point. Defined values are: 0b0001 Floating-point is implemented, and includes support for: - Half-precision, single-precision and double-precision floating-point types. - Conversions between single-precision and half-precision data types, and double-precision and half-precision data types.	0b0001
[15:12]	EL3	EL3 Exception level handling. Defined values are: 0b0001 EL3 can be executed in AArch64 state only.	0b0001
[11:8]	EL2	EL2 Exception level handling. Defined values are: 0b0001 EL2 can be executed in AArch64 state only.	0b0001
[7:4]	EL1	EL1 Exception level handling. Defined values are: 0b0001 EL1 can be executed in AArch64 state only.	0b0001
[3:0]	ELO	ELO Exception level handling. Defined values are: 0b0001 ELO can be executed in AArch64 state only.	0b0001

Access

MRS <Xt>, ID_AA64PFR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0100	0b000

Accessibility

MRS <Xt>, ID_AA64PFR0_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64PFR0_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64PFR0_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64PFR0_EL1;
```

A.7.8 ID_AA64PFR1_EL1, AArch64 Processor Feature Register 1

Reserved for future expansion of information about implemented PE features in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0001	0000	xxxx	0001	xxxx	xxxx	0010	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-115: AArch64_id_aa64pfr1_el1 bit assignments

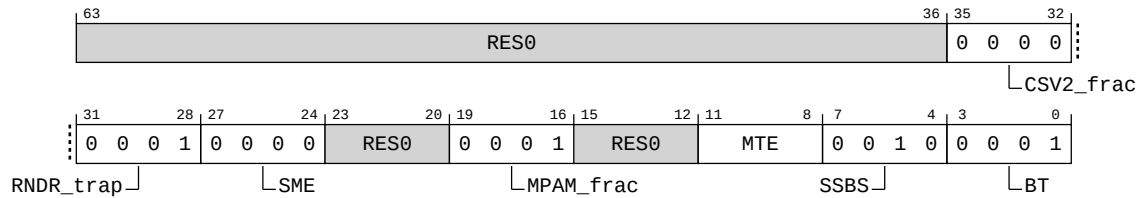


Table A-275: ID_AA64PFR1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:36]	RES0	Reserved	RES0
[35:32]	CSV2_frac	CSV2 fractional field. Defined values are: 0b0000 This PE does not disclose whether branch targets trained in one hardware-described context can exploitatively control speculative execution in a different hardware-described context. The SCXTNUM_ELx registers are not supported.	0b0000
[31:28]	RNDR_trap	Random Number trap to EL3 field. Defined values are: 0b0001 Trapping of AArch64-RNDR and AArch64-RNDRRS to EL3 is supported. AArch64-SCR_EL3.TRNDR is present.	0b0001
[27:24]	SME	Scalable Matrix Extension field. Defined values are: 0b0000 SME architectural state and programmers' model are not implemented.	0b0000
[23:20]	RES0	Reserved	RES0
[19:16]	MPAM_frac	MPAM Extension fractional field. Defined values are: 0b0001 If AArch64-ID_AA64PFR0_EL1.MPAM == 0b0000, implements MPAM v0.1, which is like v1.1 but reduces support for Secure PARTIDs. If AArch64-ID_AA64PFR0_EL1.MPAM == 0b0001, implements MPAM v1.1 and adds support for AArch64-MPAM2_EL2.TIDR to provide trapping of AArch64-MPAMIDR_EL1 when AArch64-MPAMHCR_EL2 is not present.	0b0001
[15:12]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[11:8]	MTE	Support for the Memory Tagging Extension. Defined values are: 0b0001 Memory Tagging Extension instructions accessible at EL0 are implemented. Instructions and System Registers defined by the extension not configurably accessible at EL0 are Unallocated and other System Register fields defined by the extension are RES0 . This value is reported when the BROADCASTMTE input is LOW. 0b0011 Memory Tagging Extension is implemented with support for asymmetric Tag Check Fault handling. This value is reported when the BROADCASTMTE input is HIGH.	The reset values can be the following: 0b0001, 0b0011, respectively to the value.
[7:4]	SSBS	Speculative Store Bypassing controls in AArch64 state. Defined values are: 0b0010 As 0b0001, and adds the MSR and MRS instructions to directly read and write the PSTATE.SSBS field.	0b0010
[3:0]	BT	Branch Target Identification mechanism support in AArch64 state. Defined values are: 0b0001 The Branch Target Identification mechanism is implemented.	0b0001

Access

MRS <Xt>, ID_AA64PFR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0100	0b001

Accessibility

MRS <Xt>, ID_AA64PFR1_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64PFR1_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64PFR1_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64PFR1_EL1;

```

A.7.9 ID_AA64ZFR0_EL1, SVE Feature ID register 0

Provides additional information about the implemented features of the AArch64 Scalable Vector Extension, when the AArch64-ID_AA64PFR0_EL1.SVE field is not zero.

For general information about the interpretation of the ID registers see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations



Note

Prior to the introduction of the features described by this register, this register was unnamed and reserved, RES0 from EL1, EL2, and EL3.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	0001	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0001	xxxx	xxxx	xxxx	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-116: AArch64_id_aa64zfr0_el1 bit assignments

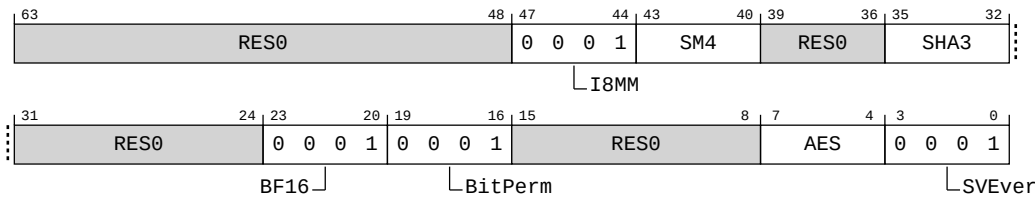


Table A-277: ID_AA64ZFR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:48]	RES0	Reserved	RES0
[47:44]	I8MM	Indicates support for SVE Int8 matrix multiplication instructions. Defined values are: 0b0001 SMMLA, SUDOT, UMMLA, USMMLA, and USDOT instructions are implemented.	0b0001

Bits	Name	Description	Reset
[43:40]	SM4	Indicates support for SVE SM4 instructions. Defined values are: 0b0000 SVE2 SM4 instructions are not implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0001 SVE2 SM4E and SM4EKEY instructions are implemented. This value is reported when the Cryptographic Extension is implemented and enabled.	The reset values can be the following: 0b0000, 0b0001, respective to the value.
[39:36]	RES0	Reserved	RES0
[35:32]	SHA3	Indicates support for the SVE SHA3 instructions. Defined values are: 0b0000 SVE2 SHA3 instructions are not implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0001 SVE2 RAX1 instruction is implemented. This value is reported when the Cryptographic Extension is implemented and enabled.	The reset values can be the following: 0b0000, 0b0001, respective to the value.
[31:24]	RES0	Reserved	RES0
[23:20]	BF16	Indicates support for SVE BFloat16 instructions. Defined values are: 0b0001 BFCVT, BFCVTNT, BFDOT, BFMLALB, BFMLALT, and BFMMLA instructions are implemented.	0b0001
[19:16]	BitPerm	Indicates support for SVE bit permute instructions. Defined values are: 0b0001 SVE BDEP, BEXT, and BGRP instructions are implemented.	0b0001
[15:8]	RES0	Reserved	RES0
[7:4]	AES	Indicates support for SVE AES instructions. Defined values are: 0b0000 SVE2-AES instructions are not implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0010 SVE2 AESE, AESD, AESMC, and AESIMC instructions are implemented plus SVE2 PMULLB and PMULLT instructions with 64-bit source. This value is reported when the Cryptographic Extension is implemented and enabled.	The reset values can be the following: 0b0000, 0b0010, respective to the value.
[3:0]	SVEver	Indicates support for SVE. Defined values are: 0b0001 The SVE and non-optional SVE2 instructions are implemented.	0b0001

Access

MRS <Xt>, ID_AA64ZFR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0100	0b100

Accessibility

MRS <Xt>, ID_AA64ZFR0_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64ZFR0_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64ZFR0_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64ZFR0_EL1;
```

A.7.10 ID_AA64DFR0_EL1, AArch64 Debug Feature Register 0

Provides top-level information about the debug system in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

The external register ext-EDDFR gives information from this register.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0001	xxxx	0000	1111	0001	0001	1111	xxxx	0001	xxxx	0011	xxxx	0101	0111	0001	1001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-117: AArch64_id_aa64dfr0_el1 bit assignments

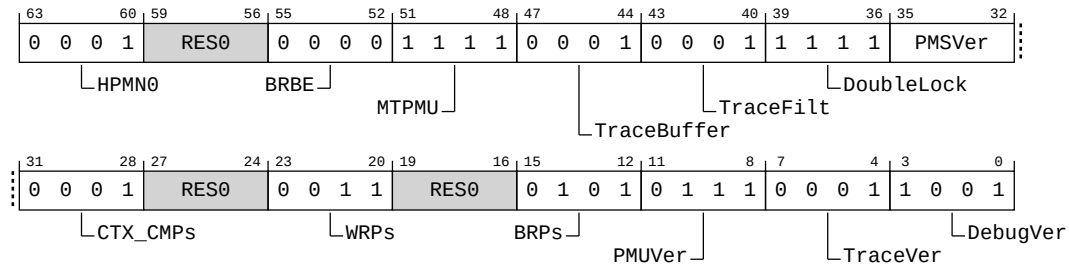


Table A-279: ID_AA64DFR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	HPMN0	Zero PMU event counters for a Guest operating system. Defined values are: 0b0001 Setting AArch64-MDCR_EL2.HPMN to zero has defined behavior.	0b0001
[59:56]	RES0	Reserved	RES0
[55:52]	BRBE	Branch Record Buffer Extension. Defined values are: 0b0000 Branch Record Buffer Extension not implemented.	0b0000
[51:48]	MTPMU	Multi-threaded PMU extension. Defined values are: 0b1111 FEAT_MTPMU not implemented. If FEAT_PMUv3 is implemented, AArch64-PMEVTYPER<n>_EL0.MT and AArch32-PMEVTYPER<n>_MT are RES0.	0b1111
[47:44]	TraceBuffer	Trace Buffer Extension. Defined values are: 0b0001 Trace Buffer Extension implemented, FEAT_TRBE.	0b0001
[43:40]	TraceFilt	Armv8.4 Self-hosted Trace Extension version. Defined values are: 0b0001 Armv8.4 Self-hosted Trace Extension implemented.	0b0001
[39:36]	DoubleLock	OS Double Lock implemented. Defined values are: 0b1111 OS Double Lock not implemented. AArch64-OSDLR_EL1 is RAZ/WI.	0b1111
[35:32]	PMSVer	Statistical Profiling Extension version. Defined values are: 0b0000 Statistical Profiling Extension not implemented. 0b0011 Statistical Profiling Extension version SPEv1p2 is implemented.	The reset values can be the following: 0b0000, 0b0011, respective to the value.
[31:28]	CTX_CMPs	Number of breakpoints that are context-aware, minus 1. These are the highest numbered breakpoints. 0b0001 Two context-aware breakpoints are included	0b0001

Bits	Name	Description	Reset
[27:24]	RES0	Reserved	RES0
[23:20]	WRPs	Number of watchpoints, minus 1. The value of 0b0000 is reserved. 0b0011 Four watchpoints	0b0011
[19:16]	RES0	Reserved	RES0
[15:12]	BRPs	Number of breakpoints, minus 1. The value of 0b0000 is reserved. 0b0101 Six breakpoints	0b0101
[11:8]	PMUVer	Performance Monitors Extension version. Defined value is: 0b0111 Performance Monitors Extension implemented, PMUv3 for Armv8.7	0b0111
[7:4]	TraceVer	Trace support. Indicates whether System register interface to a PE trace unit is implemented. Defined values are: 0b0001 PE trace unit System registers implemented.	0b0001
[3:0]	DebugVer	Debug architecture version. Indicates presence of Armv8 debug architecture. Defined values are: 0b1001 Armv8.4 debug architecture, FEAT_Debugv8p4.	0b1001

Access

MRS <Xt>, ID_AA64DFR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0101	0b000

Accessibility

MRS <Xt>, ID_AA64DFR0_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64DFR0_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64DFR0_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64DFR0_EL1;

```

A.7.11 ID_AA64DFR1_EL1, AArch64 Debug Feature Register 1

Reserved for future expansion of top-level information about the debug system in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

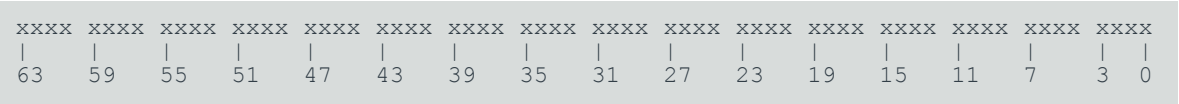
Functional group

Identification registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-118: AArch64_id_aa64dfr1_el1 bit assignments

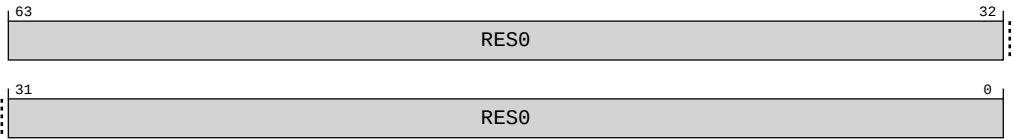


Table A-281: ID_AA64DFR1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, ID_AA64DFR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0101	0b001

Accessibility

MRS <Xt>, ID_AA64DFR1_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64DFR1_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64DFR1_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64DFR1_EL1;

```

A.7.12 ID_AA64AFR0_EL1, AArch64 Auxiliary Feature Register 0

Provides information about the **IMPLEMENTATION DEFINED** features of the PE in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-119: AArch64_id_aa64afr0_el1 bit assignments

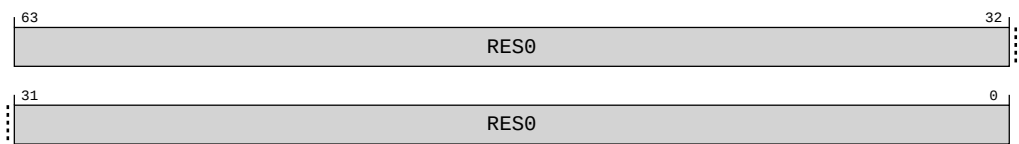


Table A-283: ID_AA64AFR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, ID_AA64AFR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0101	0b100

Accessibility

MRS <Xt>, ID_AA64AFR0_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64AFR0_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64AFR0_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64AFR0_EL1;
```

A.7.13 ID_AA64AFR1_EL1, AArch64 Auxiliary Feature Register 1

Reserved for future expansion of information about the **IMPLEMENTATION DEFINED** features of the PE in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

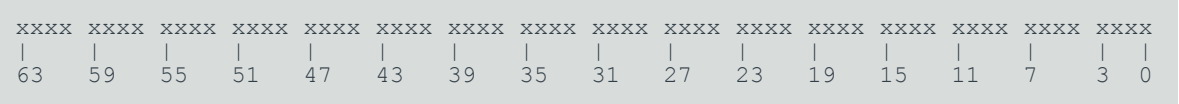
Functional group

Identification registers

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-120: AArch64_id_aa64afr1_el1 bit assignments

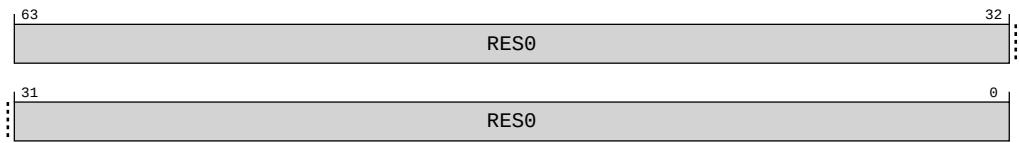


Table A-285: ID_AA64AFR1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, ID_AA64AFR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0101	0b101

Accessibility

MRS <Xt>, ID_AA64AFR1_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
```

```
if EL2Enabled() && HCR_EL2.TID3 == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
else
    return ID_AA64AFR1_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64AFR1_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64AFR1_EL1;
```

A.7.14 ID_AA64ISAR0_EL1, AArch64 Instruction Set Attribute Register 0

Provides information about the instructions implemented in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0001	0010	0010	0001	0001	xxxx	xxxx	xxxx	0001	0000	0010	0001	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-121: AArch64_id_aa64isar0_el1 bit assignments

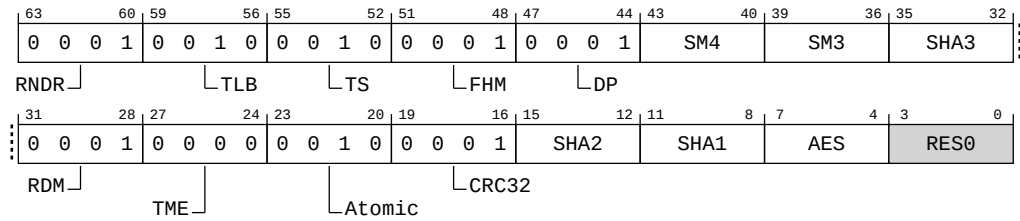


Table A-287: ID_AA64ISAR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	RNDR	Indicates support for Random Number instructions in AArch64 state. When FEAT_RNG_TRAP is implemented, the value returned by a direct read of ID_AA64ISAR0_EL1.RNDR is further controlled by the value of AArch64-SCR_EL3.TRNDR. Defined values are: 0b0001 AArch64-RNDR and AArch64-RNDRS registers are implemented.	0b0001
[59:56]	TLB	Indicates support for Outer Shareable and TLB range maintenance instructions. Defined values are: 0b0010 Outer Shareable and TLB range maintenance instructions are implemented.	0b0010
[55:52]	TS	Indicates support for flag manipulation instructions. Defined values are: 0b0010 CFINV, RMIF, SETF16, SETF8, AXFLAG, and XAFLAG instructions are implemented.	0b0010
[51:48]	FHM	Indicates support for FMLAL and FMLSL instructions. Defined values are: 0b0001 FMLAL and FMLSL instructions are implemented.	0b0001
[47:44]	DP	Indicates support for Dot Product instructions in AArch64 state. Defined values are: 0b0001 UDOT and SDOT instructions implemented.	0b0001
[43:40]	SM4	Indicates support for SM4 instructions in AArch64 state. Defined values are: 0b0000 No SM4 instructions implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0001 SM4E and SM4EKEY instructions implemented. This value is reported when the Cryptographic Extension is implemented and enabled. When the CRYPTO configuration parameter is true and the CRYPTODISABLE input is low at reset the Cryptographic Extension is implemented	The reset values can be the following: 0b0000, 0b0001, respective to the value.

Bits	Name	Description	Reset
[39:36]	SM3	Indicates support for SM3 instructions in AArch64 state. Defined values are: 0b0000 No SM3 instructions implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0001 SM3SS1, SM3TT1A, SM3TT1B, SM3TT2A, SM3TT2B, SM3PARTW1, and SM3PARTW2 instructions implemented. This value is reported when the Cryptographic Extension is implemented and enabled. When the CRYPTO configuration parameter is true and the CRYPTODISABLE input is low at reset the Cryptographic Extension is implemented	The reset values can be the following: 0b0000, 0b0001, respective to the value.
[35:32]	SHA3	Indicates support for SHA3 instructions in AArch64 state. Defined values are: 0b0000 No SHA3 instructions implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0001 EOR3, RAX1, XAR, and BCAX instructions implemented. This value is reported when the Cryptographic Extension is implemented and enabled. When the CRYPTO configuration parameter is true and the CRYPTODISABLE input is low at reset the Cryptographic Extension is implemented	The reset values can be the following: 0b0000, 0b0001, respective to the value.
[31:28]	RDM	Indicates support for SQRDMLAH and SQRDMLSH instructions in AArch64 state. Defined values are: 0b0001 SQRDMLAH and SQRDMLSH instructions implemented.	0b0001
[27:24]	TME	Indicates support for TME instructions. Defined values are: 0b0000 TME instructions are not implemented.	0b0000
[23:20]	Atomic	Indicates support for Atomic instructions in AArch64 state. Defined values are: 0b0010 LDADD, LDCLR, LDEOR, LDSET, LDSMAX, LDSMIN, LDUMAX, LDUMIN, CAS, CASP, and SWP instructions implemented.	0b0010
[19:16]	CRC32	Indicates support for CRC32 instructions in AArch64 state. Defined values are: 0b0001 CRC32B, CRC32H, CRC32W, CRC32X, CRC32CB, CRC32CH, CRC32CW, and CRC32CX instructions implemented.	0b0001
[15:12]	SHA2	Indicates support for SHA2 instructions in AArch64 state. Defined values are: 0b0000 No SHA2 instructions implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0010 SHA256H, SHA256H2, SHA256SU0, SHA256SU1, SHA512H, SHA512H2, SHA512SU0, and SHA512SU1 instructions implemented. This value is reported when the Cryptographic Extension is implemented and enabled. When the CRYPTO configuration parameter is true and the CRYPTODISABLE input is low at reset the Cryptographic Extension is implemented	The reset values can be the following: 0b0000, 0b0010, respective to the value.

Bits	Name	Description	Reset
[11:8]	SHA1	Indicates support for SHA1 instructions in AArch64 state. Defined values are: 0b0000 No SHA1 instructions implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0001 SHA1C, SHA1P, SHA1M, SHA1H, SHA1SU0, and SHA1SU1 instructions implemented. This value is reported when the Cryptographic Extension is implemented and enabled. When the CRYPTO configuration parameter is true and the CRYPTODISABLE input is low at reset the Cryptographic Extension is implemented	The reset values can be the following: 0b0000, 0b0001, respective to the value.
[7:4]	AES	Indicates support for AES instructions in AArch64 state. Defined values are: 0b0000 No AES instructions implemented. This value is reported when the Cryptographic Extension is not implemented or is disabled. 0b0010 AESE, AESD, AESMC, and AESIMC instructions are implemented plus PMULL/PMULL2 instructions operating on 64-bit data quantities. This value is reported when the Cryptographic Extension is implemented and enabled. When the CRYPTO configuration parameter is true and the CRYPTODISABLE input is low at reset the Cryptographic Extension is implemented	The reset values can be the following: 0b0000, 0b0010, respective to the value.
[3:0]	RES0	Reserved	RES0

Access

MRS <Xt>, ID_AA64ISAR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0110	0b000

Accessibility

MRS <Xt>, ID_AA64ISAR0_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64ISAR0_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64ISAR0_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64ISAR0_EL1;

```

A.7.15 ID_AA64ISAR1_EL1, AArch64 Instruction Set Attribute Register 1

Provides information about the features and instructions implemented in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0000 0001 0001 0001 0001 0001 0001 0001 0000 0000 0010 0001 0001 0000 0000 0010

Bit descriptions

Figure A-122: AArch64_id_aa64isar1_el1 bit assignments

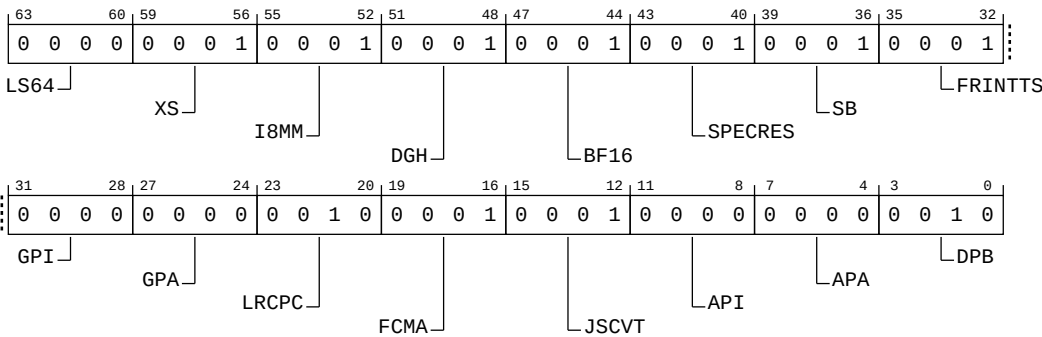


Table A-289: ID_AA64ISAR1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	LS64	Indicates support for LD64B and ST64B* instructions, and the AArch64-ACCDATA_EL1 register. Defined values of this field are: 0b0000 The LD64B and ST64B* instructions, the AArch64-ACCDATA_EL1 register, and associated traps are not supported.	0b0000

Bits	Name	Description	Reset
[59:56]	XS	Indicates support for the XS attribute, the TLBI and DSB instructions with the nXS qualifier, and the AArch64-HCRX_EL2.{FGTnXS, FnXS} fields in AArch64 state. Defined values are: 0b0001 The XS attribute, the TLBI and DSB instructions with the nXS qualifier, and the AArch64-HCRX_EL2.{FGTnXS, FnXS} fields are supported.	0b0001
[55:52]	I8MM	Indicates support for Advanced SIMD and Floating-point Int8 matrix multiplication instructions in AArch64 state. Defined values are: 0b0001 SMMLA, SUDOT, UMMLA, USMMLA, and USDOT instructions are implemented.	0b0001
[51:48]	DGH	Indicates support for the Data Gathering Hint instruction. Defined values are: 0b0001 Data Gathering Hint is implemented.	0b0001
[47:44]	BF16	Indicates support for Advanced SIMD and Floating-point BFloat16 instructions in AArch64 state. Defined values are: 0b0001 BFCVT, BFCVTN, BFCVTN2, BFDOT, BFMLALB, BFMLALT, and BFMMLA instructions are implemented.	0b0001
[43:40]	SPECRES	Indicates support for prediction invalidation instructions in AArch64 state. Defined values are: 0b0001 CFP RCTX, DVP RCTX, and CPP RCTX instructions are implemented.	0b0001
[39:36]	SB	Indicates support for SB instruction in AArch64 state. Defined values are: 0b0001 SB instruction is implemented.	0b0001
[35:32]	FRINTTS	Indicates support for the FRINT32Z, FRINT32X, FRINT64Z, and FRINT64X instructions are implemented. Defined values are: 0b0001 FRINT32Z, FRINT32X, FRINT64Z, and FRINT64X instructions are implemented.	0b0001
[31:28]	GPI	Indicates support for an IMPLEMENTATION DEFINED algorithm is implemented in the PE for generic code authentication in AArch64 state. Defined values are: 0b0000 Generic Authentication using an IMPLEMENTATION DEFINED algorithm is not implemented.	0b0000
[27:24]	GPA	Indicates whether the QARMA5 algorithm is implemented in the PE for generic code authentication in AArch64 state. Defined values are: 0b0000 Generic Authentication using the QARMA5 algorithm is not implemented.	0b0000
[23:20]	LRCPC	Indicates support for weaker release consistency, RCpc, based model. Defined values are: 0b0010 The LDAPUR, STLUR, and LDAPR* instructions are implemented.	0b0010
[19:16]	FCMA	Indicates support for complex number addition and multiplication, where numbers are stored in vectors. Defined values are: 0b0001 The FCMLA and FCADD instructions are implemented.	0b0001

Bits	Name	Description	Reset
[15:12]	JSCVT	Indicates support for JavaScript conversion from double precision floating point values to integers in AArch64 state. Defined values are: 0b0001 The FJCVTZS instruction is implemented.	0b0001
[11:8]	API	Indicates whether an IMPLEMENTATION DEFINED algorithm is implemented in the PE for address authentication, in AArch64 state. This applies to all Pointer Authentication instructions other than the PACGA instruction. Defined values are: 0b0000 Address Authentication using an IMPLEMENTATION DEFINED algorithm is not implemented.	0b0000
[7:4]	APA	Indicates whether the QARMA5 algorithm is implemented in the PE for address authentication, in AArch64 state. This applies to all Pointer Authentication instructions other than the PACGA instruction. Defined values are: 0b0000 Address Authentication using the QARMA5 algorithm is not implemented.	0b0000
[3:0]	DPB	Data Persistence writeback. Indicates support for the DC CVAP and DC CVADP instructions in AArch64 state. Defined values are: 0b0010 DC CVAP and DC CVADP supported	0b0010

Access

MRS <Xt>, ID_AA64ISAR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0110	0b001

Accessibility

MRS <Xt>, ID_AA64ISAR1_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64ISAR1_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64ISAR1_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64ISAR1_EL1;

```

A.7.16 ID_AA64ISAR2_EL1, AArch64 Instruction Set Attribute Register 2

Provides information about the features and instructions implemented in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations



Prior to the introduction of the features described by this register, this register was unnamed and reserved, RES0 from EL1, EL2, and EL3.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	xxxx	xxxx	0101	0001	0000	0010
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-123: AArch64_id_aa64isar2_el1 bit assignments

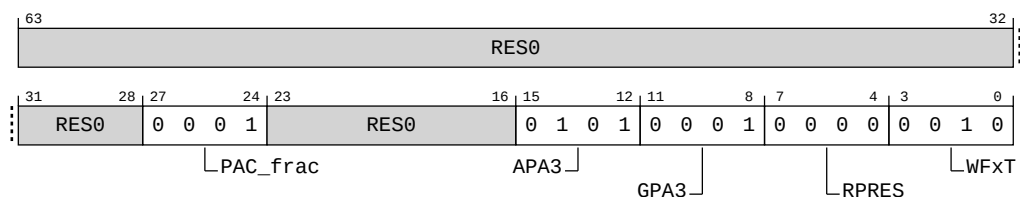


Table A-291: ID_AA64ISAR2_EL1 bit descriptions

Bits	Name	Description	Reset
[63:28]	RES0	Reserved	RES0
[27:24]	PAC_frac	Indicates whether the ConstPACField() function used as part of the PAC addition returns FALSE or TRUE. 0b0001 ConstPACField() returns TRUE.	0b0001
[23:16]	RES0	Reserved	RES0
[15:12]	APA3	Indicates whether the QARMA3 algorithm is implemented in the PE for address authentication in AArch64 state. This applies to all Pointer Authentication instructions other than the PACGA instruction. Defined values are: 0b0101 Address Authentication using the QARMA3 algorithm is implemented, with the HaveEnhancedPAC2() function returning TRUE, the HaveFPAC() function returning TRUE, the HaveFPACCombined() function returning TRUE, and the HaveEnhancedPAC() function returning FALSE.	0b0101
[11:8]	GPA3	Indicates whether the QARMA3 algorithm is implemented in the PE for generic code authentication in AArch64 state. Defined values are: 0b0001 Generic Authentication using the QARMA3 algorithm is implemented. This includes the PACGA instruction.	0b0001
[7:4]	RPRES	When AArch64-FPCR.AH is 1, indicates support for 12 bits of mantissa in reciprocal and reciprocal square root instructions in AArch64 state. Defined values are: 0b0000 Reciprocal and reciprocal square root estimates give 8 bits of mantissa.	0b0000
[3:0]	WFXT	Indicates support for the WFET and WFIT instructions in AArch64 state. Defined values are: 0b0010 WFET and WFIT are supported, and the register number is reported in the ESR_ELx on exceptions.	0b0010

Access

MRS <Xt>, ID_AA64ISAR2_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0110	0b010

Accessibility

MRS <Xt>, ID_AA64ISAR2_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64ISAR2_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64ISAR2_EL1;
elseif PSTATE.EL == EL3 then

```

```
return ID_AA64ISAR2_EL1;
```

A.7.17 ID_AA64MMFR0_EL1, AArch64 Memory Model Feature Register 0

Provides information about the implemented memory model and memory management support in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0010	0001	xxxx	xxxx	0000	0010	0010	0010	0000	0000	0001	0000	0001	0001	0010	0101
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-124: AArch64_id_aa64mmfr0_el1 bit assignments

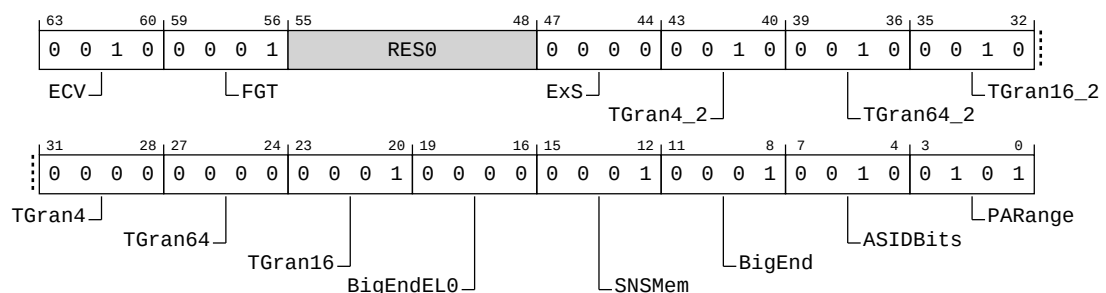


Table A-293: ID_AA64MMFR0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	ECV	Indicates presence of Enhanced Counter Virtualization. Defined values are: 0b0010 As 0b0001, and also includes support for AArch64-CNTHCTL_EL2.ECV and AArch64-CNTPOFF_EL2.	0b0010
[59:56]	FGT	Indicates presence of the Fine-Grained Trap controls: - If EL2 is implemented, the AArch64-HAFGRTR_EL2, AArch64-HDFGRTR_EL2, AArch64-HDFGWTR_EL2, AArch64-HFGRTR_EL2, AArch64-HFGITR_EL2 and AArch64-HFGWTR_EL2 registers, and their associated traps. - If EL2 is implemented, AArch64-MDCR_EL2.TDCC. - If EL3 is implemented, AArch64-MDCR_EL3.TDCC. - If both EL2 and EL3 are implemented, AArch64-SCR_EL3.FGTEn. Defined values are: 0b0001 The fine-grained trap controls are implemented.	0b0001
[55:48]	RES0	Reserved	RES0
[47:44]	ExS	Indicates support for disabling context synchronizing exception entry and exit. Defined values are: 0b0000 All exception entries and exits are context synchronization events.	0b0000
[43:40]	TGran4_2	Indicates support for 4KB memory granule size at stage 2. Defined values are: 0b0010 4KB granule supported at stage 2.	0b0010
[39:36]	TGran64_2	Indicates support for 64KB memory granule size at stage 2. Defined values are: 0b0010 64KB granule supported at stage 2.	0b0010
[35:32]	TGran16_2	Indicates support for 16KB memory granule size at stage 2. Defined values are: 0b0010 16KB granule supported at stage 2.	0b0010
[31:28]	TGran4	Indicates support for 4KB memory translation granule size. Defined values are: 0b0000 4KB granule supported.	0b0000
[27:24]	TGran64	Indicates support for 64KB memory translation granule size. Defined values are: 0b0000 64KB granule supported.	0b0000
[23:20]	TGran16	Indicates support for 16KB memory translation granule size. Defined values are: 0b0001 16KB granule supported.	0b0001
[19:16]	BigEndELO	Indicates support for mixed-endian at EL0 only. Defined values are: 0b0000 No mixed-endian support at EL0. The AArch64-SCTLR_EL1.EOE bit has a fixed value.	0b0000

Bits	Name	Description	Reset
[15:12]	SNSMem	Indicates support for a distinction between Secure and Non-secure Memory. Defined values are: 0b0001 Does support a distinction between Secure and Non-secure Memory.	0b0001
[11:8]	BigEnd	Indicates support for mixed-endian configuration. Defined values are: 0b0001 Mixed-endian support. The SCTLR_ElxE.EE and AArch64-SCTLR_EL1.EOE bits can be configured.	0b0001
[7:4]	ASIDBits	Number of ASID bits. Defined values are: 0b0010 16 bits.	0b0010
[3:0]	PARange	Physical Address range supported. Defined values are: 0b0101 48 bits, 256TB.	0b0101

Access

MRS <Xt>, ID_AA64MMFR0_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0111	0b000

Accessibility

MRS <Xt>, ID_AA64MMFR0_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64MMFR0_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64MMFR0_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64MMFR0_EL1;

```

A.7.18 ID_AA64MMFR1_EL1, AArch64 Memory Model Feature Register 1

Provides information about the implemented memory model and memory management support in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

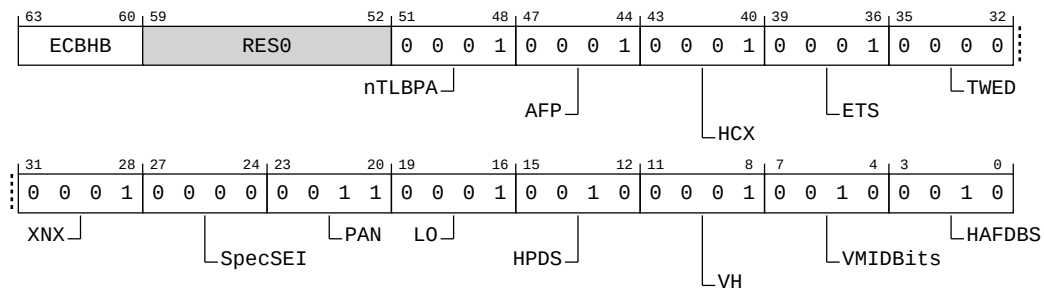
Reset value

0001	xxxx	xxxx	0001	0001	0001	0001	0000	0001	0000	0011	0001	0010	0001	0010	0010
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-125: AArch64_id_aa64mmfr1_el1 bit assignments**Table A-295: ID_AA64MMFR1_EL1 bit descriptions**

Bits	Name	Description	Reset
[63:60]	ECBHB	Indicates support for Exploitative Control using Branch History information between exception levels. Defined values are: 0b0001 The branch history information created in a context before an exception to a higher exception level using AArch64 cannot be used by code before that exception to exploitatively control the execution of any code in a different context after the exception. All other values are reserved. FEAT_ECBHB implements the functionality identified by the value 0b0001.	0b0001
[59:52]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[51:48]	nTLBPA	Indicates support for intermediate caching of translation table walks. Defined values are: 0b0001 The intermediate caching of translation table walks does not include non-coherent caches of previous valid translation table entries since the last completed TLBI applicable to the PE where either: - The caching is indexed by the physical address of the location holding the translation table entry. - The caching is used for stage 1 translations and is indexed by the intermediate physical address of the location holding the translation table entry.	0b0001
[47:44]	AFP	Indicates support for AArch64-FPCR.{AH, FIZ, NEP}. Defined values are: 0b0001 The AArch64-FPCR.{AH, FIZ, NEP} fields are supported.	0b0001
[43:40]	H CX	Indicates support for AArch64-HCRX_EL2 and its associated EL3 trap. Defined values are: 0b0001 AArch64-HCRX_EL2 and its associated EL3 trap are supported.	0b0001
[39:36]	ETS	Indicates support for Enhanced Translation Synchronization. Defined values are: 0b0001 Enhanced Translation Synchronization is supported.	0b0001
[35:32]	TWED	Indicates support for the configurable delayed trapping of WFE. Defined values are: 0b0000 Configurable delayed trapping of WFE is not supported.	0b0000
[31:28]	XNX	Indicates support for execute-never control distinction by Exception level at stage 2. Defined values are: 0b0001 Distinction between EL0 and EL1 execute-never control at stage 2 supported.	0b0001
[27:24]	SpecSEI	Describes whether the PE can generate SError interrupt exceptions from speculative reads of memory, including speculative instruction fetches. The defined values of this field are: 0b0000 The PE never generates an SError interrupt due to an External abort on a speculative read.	0b0000
[23:20]	PAN	Privileged Access Never. Indicates support for the PAN bit in PSTATE, AArch64-SPSR_EL1, AArch64-SPSR_EL2, AArch64-SPSR_EL3, and AArch64-DSPSR_EL0. Defined values are: 0b0011 PAN supported, AT S1E1RP and AT S1E1WP instructions supported, and AArch64-SCTLR_EL1.EPAN and AArch64-SCTLR_EL2.EPAN bits supported.	0b0011
[19:16]	LO	LORegions. Indicates support for LORegions. Defined values are: 0b0001 LORegions supported.	0b0001
[15:12]	HPDS	Hierarchical Permission Disables. Indicates support for disabling hierarchical controls in translation tables. Defined values are: 0b0010 Disabling of hierarchical controls supported with the TCR_EL1.{HPD1, HPD0}, TCR_EL2.HPD or TCR_EL2.{HPD1, HPD0}, and TCR_EL3.HPD bits and adds possible hardware allocation of bits[62:59] of the translation table descriptors from the final lookup level for IMPLEMENTATION DEFINED use.	0b0010

Bits	Name	Description	Reset
[11:8]	VH	Virtualization Host Extensions. Defined values are: 0b0001 Virtualization Host Extensions supported.	0b0001
[7:4]	VMIDBits	Number of VMID bits. Defined values are: 0b0010 16 bits	0b0010
[3:0]	HAFDBS	Hardware updates to Access flag and Dirty state in translation tables. Defined values are: 0b0010 Hardware update of both the Access flag and dirty state is supported.	0b0010

Access

MRS <Xt>, ID_AA64MMFR1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0111	0b001

Accessibility

MRS <Xt>, ID_AA64MMFR1_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64MMFR1_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64MMFR1_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64MMFR1_EL1;

```

A.7.19 ID_AA64MMFR2_EL1, AArch64 Memory Model Feature Register 2

Provides information about the implemented memory model and memory management support in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations



Note

Prior to the introduction of the features described by this register, this register was unnamed and reserved, RES0 from EL1, EL2, and EL3.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0001	0010	0010	0001	xxxx	0001	0001	0001	0001	0000	0001	0000	0001	0000	0001	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-126: AArch64_id_aa64mmfr2_el1 bit assignments

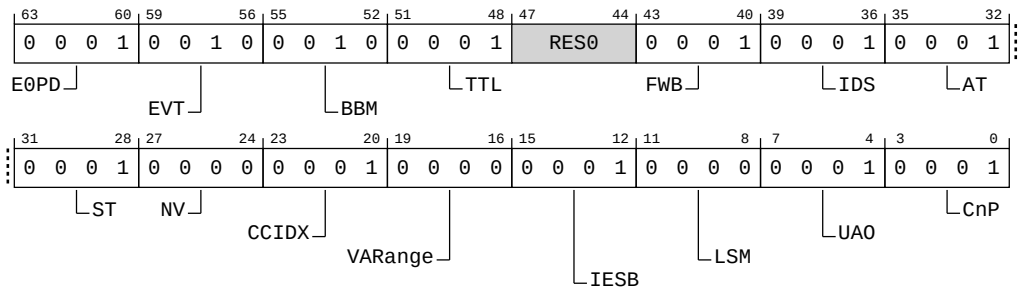


Table A-297: ID_AA64MMFR2_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	EOPD	Indicates support for the EOPD mechanism. Defined values are: 0b0001 EOPDx mechanism is implemented.	0b0001

Bits	Name	Description	Reset
[59:56]	EVT	Enhanced Virtualization Traps. If EL2 is implemented, indicates support for the AArch64-HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps. Defined values are: 0b0010 AArch64-HCR_EL2.{TTLBOS, TTLBIS, TOCU, TICAB, TID4} traps are supported.	0b0010
[55:52]	BBM	Allows identification of the requirements of the hardware to have break-before-make sequences when changing block size for a translation. 0b0010 Level 2 support for changing block size is supported.	0b0010
[51:48]	TTL	Indicates support for TTL field in address operations. Defined values are: 0b0001 TLB maintenance instructions by address have bits[47:44] holding the TTL field.	0b0001
[47:44]	RES0	Reserved	RES0
[43:40]	FWB	Indicates support for AArch64-HCR_EL2.FWB. Defined values are: 0b0001 AArch64-HCR_EL2.FWB is supported.	0b0001
[39:36]	IDS	Indicates the value of ESR_ELx.EC that reports an exception generated by a read access to the feature ID space. Defined values are: 0b0001 All exceptions generated by an AArch64 read access to the feature ID space are reported by ESR_ELx.EC == 0x18.	0b0001
[35:32]	AT	Identifies support for unaligned single-copy atomicity and atomic functions. Defined values are: 0b0001 Unaligned single-copy atomicity and atomic functions with a 16-byte address range aligned to 16-bytes are supported.	0b0001
[31:28]	ST	Identifies support for small translation tables. Defined values are: 0b0001 The maximum value of the TCR_ELx.{T0SZ,T1SZ} and VTCR_EL2.T0SZ fields is 48 for 4KB and 16KB granules, and 47 for 64KB granules.	0b0001
[27:24]	NV	Nested Virtualization. If EL2 is implemented, indicates support for the use of nested virtualization. Defined values are: 0b0000 Nested virtualization is not supported.	0b0000
[23:20]	CCIDX	Support for the use of revised AArch64-CCSIDR_EL1 register format. Defined values are: 0b0001 64-bit format implemented for all levels of the CCSIDR_EL1.	0b0001
[19:16]	VARange	Indicates support for a larger virtual address. Defined values are: 0b0000 VMSAv8-64 supports 48-bit VAs.	0b0000
[15:12]	IESB	Indicates support for the IESB bit in the SCTLR_ELx registers. Defined values are: 0b0001 IESB bit in the SCTLR_ELx registers is supported.	0b0001

Bits	Name	Description	Reset
[11:8]	LSM	Indicates support for LSMAOE and nTLSMD bits in AArch64-SCTLR_EL1 and AArch64-SCTLR_EL2. Defined values are: 0b0000 LSMAOE and nTLSMD bits not supported.	0b0000
[7:4]	UAO	User Access Override. Defined values are: 0b0001 UAO supported.	0b0001
[3:0]	CnP	Indicates support for Common not Private translations. Defined values are: 0b0001 Common not Private translations supported.	0b0001

Access

MRS <Xt>, ID_AA64MMFR2_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0111	0b010

Accessibility

MRS <Xt>, ID_AA64MMFR2_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64MMFR2_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64MMFR2_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64MMFR2_EL1;

```

A.7.20 ID_AA64MMFR3_EL1, AArch64 Memory Model Feature Register 2

Provides information about the implemented memory model and memory management support in AArch64 state.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations



Note

Prior to the introduction of the features described by this register, this register was unnamed and reserved, RES0 from EL1, EL2, and EL3.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

0001	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-127: AArch64_id_aa64mmfr3_el1 bit assignments

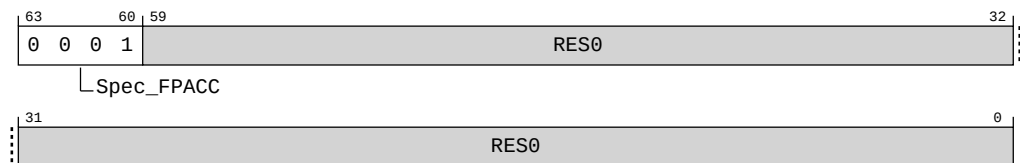


Table A-299: ID_AA64MMFR3_EL1 bit descriptions

Bits	Name	Description	Reset
[63:60]	Spec_FPACC	Speculative behavior in the event of a PAC authentication failure in an implementation that includes FEAT_FPACCOMBINE. Defined values are: 0b0001 The speculative use of pointers processed by a PAC Authentication is not materially different in terms of the impact on cached microarchitectural state between passing and failing of the PAC Authentication. All other values are reserved. For the purpose of this definition, cached microarchitecture state is the state of caching agents such as instruction caches, data caches and TLBs which can be altered as a result of speculation caused by a mispredicted execution, but is not restored to the state prior to the speculation when the misprediction is corrected.	0b0001
[59:0]	RES0	Reserved	RES0

Access

MRS <Xt>, ID_AA64MMFR3_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0000	0b0111	0b011

Accessibility

MRS <Xt>, ID_AA64MMFR3_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID3 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return ID_AA64MMFR3_EL1;
elseif PSTATE.EL == EL2 then
    return ID_AA64MMFR3_EL1;
elseif PSTATE.EL == EL3 then
    return ID_AA64MMFR3_EL1;

```

A.7.21 MPAMIDR_EL1, MPAM ID Register (EL1)

Indicates the presence and maximum PARTID and PMG values supported in the implementation. It also indicates whether the implementation supports MPAM virtualization.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

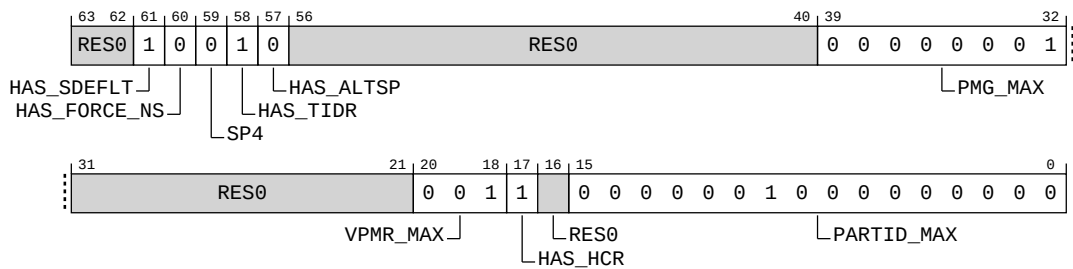
xx10	010x	xxxx	xxxx	xxxx	xxxx	0000	0001	xxxx	xxxx	xxx0	011x	0000	0010	0000	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

MPAMIDR_EL1 indicates the MPAM implementation parameters of the PE.

Figure A-128: AArch64_mpamidr_el1 bit assignments**Table A-301: MPAMIDR_EL1 bit descriptions**

Bits	Name	Description	Reset
[63:62]	RES0	Reserved	RES0
[61]	HAS_SDEFLT	HAS_SDEFLT indicates support for AArch64-MPAM3_EL3.SDEFLT bit. Defined values are: 0b1 The SDEFLT bit is implemented in AArch64-MPAM3_EL3.	0b1
[60]	HAS_FORCE_NS	HAS_FORCE_NS indicates support for AArch64-MPAM3_EL3.FORCE_NS bit. Defined values are: 0b0 The FORCE_NS bit is not implemented in AArch64-MPAM3_EL3.	0b0
[59]	SP4	Supports 4 MPAM PARTID spaces. 0b0 MPAM supports 2 PARTID spaces.	0b0

Bits	Name	Description	Reset
[58]	HAS_TIDR	HAS_TIDR indicates support for AArch64-MPAM2_EL2.TIDR bit. Defined values are: 0b1 The TIDR bit is implemented in AArch64-MPAM2_EL2.	0b1
[57]	HAS_ALTSP	HAS_ALTSP indicates support for alternative PARTID spaces. 0b0 Alternative PARTID spaces are not implemented.	0b0
[56:40]	RES0	Reserved	RES0
[39:32]	PMG_MAX	The largest value of PMG that the implementation can generate. The PMG_I and PMG_D fields of every MPAMn_ELx must implement at least enough bits to represent PMG_MAX. 0b00000001 Max PMG field is 1	0x01
[31:21]	RES0	Reserved	RES0
[20:18]	VPMR_MAX	Indicates the maximum register index n for the MPAMVPM<n>_EL2 registers. 0b001 2 MPAMVPMn_EL2 registers are implemented	0b001
[17]	HAS_HCR	HAS_HCR indicates that the PE implementation supports MPAM virtualization, including AArch64-MPAMHCR_EL2, AArch64-MPAMVPMV_EL2, and MPAMVPM<n>_EL2 with n in the range 0 to VPMR_MAX. Must be 0 if EL2 is not implemented in either Security state. 0b1 MPAM virtualization is supported.	0b1
[16]	RES0	Reserved	RES0
[15:0]	PARTID_MAX	The largest value of PARTID that the implementation can generate. The PARTID_I and PARTID_D fields of every MPAMn_ELx must implement at least enough bits to represent PARTID_MAX. 0b00000001000000000 Max PARTID field is 512	0x0200

Access

MRS <Xt>, MPAMIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1010	0b0100	0b100

Accessibility

MRS <Xt>, MPAMIDR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    elseif EL2Enabled() && MPAMHCR_EL2.TRAP_MPAMIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && MPAM2_EL2.TIDR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        UNDEFINED;

```

```
        return MPAMIDR_EL1;
    elseif PSTATE.EL == EL2 then
        if MPAM3_EL3.TRAPLOWER == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            end
        elseif PSTATE.EL == EL3 then
            return MPAMIDR_EL1;
        end
    end
```

A.7.22 CCSIDR_EL1, Current Cache Size ID Register

Provides information about the architecture of the currently selected cache.

Configurations

The implementation includes one CCSIDR_EL1 for each cache that it can access. AArch64-CSSELR_EL1 selects which Cache Size ID Register is accessible.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions



Note

The parameters NumSets, Associativity, and LineSize in these registers define the architecturally visible parameters that are required for the cache maintenance by Set/Way instructions. They are not guaranteed to represent the actual microarchitectural features of a design. You cannot make any inference about the actual sizes of caches based on these parameters.

Figure A-129: AArch64_ccsidr_el1 bit assignments

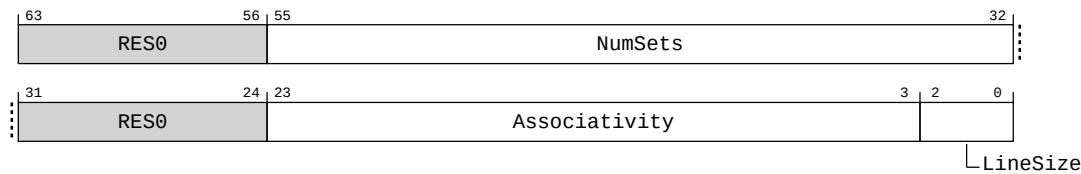


Table A-303: CCSIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:56]	RES0	Reserved	RES0
[55:32]	NumSets	(Number of sets in cache) - 1, therefore a value of 0 indicates 1 set in the cache. The number of sets does not have to be a power of 2.	24 {x}
[31:24]	RES0	Reserved	RES0
[23:3]	Associativity	(Associativity of cache) - 1, therefore a value of 0 indicates an associativity of 1. The associativity does not have to be a power of 2.	21 {x}
[2:0]	LineSize	(Log ₂ (Number of bytes in cache line)) - 4. For example: - For a line length of 16 bytes: Log₂(16) = 4, LineSize entry = 0. This is the minimum line length. - For a line length of 32 bytes: Log₂(32) = 5, LineSize entry = 1. When FEAT_MTE2 is implemented and enabled, where a cache only holds Allocation tags, this field is RES0.	xxx

Access

If AArch64-CSSELR_EL1.Level is programmed to a cache level that is not implemented, then on a read of the CCSIDR_EL1 the behavior is **CONSTRAINED UNPREDICTABLE**, and can be one of the following:

- The CCSIDR_EL1 read is treated as NOP.
- The CCSIDR_EL1 read is UNDEFINED.
- The CCSIDR_EL1 read returns an **UNKNOWN** value.

MRS <Xt>, CCSIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b001	0b0000	0b0000	0b000

Accessibility

If AArch64-CSSELR_EL1.Level is programmed to a cache level that is not implemented, then on a read of the CCSIDR_EL1 the behavior is **CONSTRAINED UNPREDICTABLE**, and can be one of the following:

- The CCSIDR_EL1 read is treated as NOP.
- The CCSIDR_EL1 read is UNDEFINED.
- The CCSIDR_EL1 read returns an **UNKNOWN** value.

MRS <Xt>, CCSIDR_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID2 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.TID4 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.CCSIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return CCSIDR_EL1;
elseif PSTATE.EL == EL2 then
    return CCSIDR_EL1;
elseif PSTATE.EL == EL3 then
    return CCSIDR_EL1;

```

A.7.23 CLIDR_EL1, Cache Level ID Register

Identifies the type of cache, or caches, that are implemented at each level and can be managed using the architected cache maintenance instructions that operate by set/way, up to a maximum of seven levels. Also identifies the Level of Coherence (LoC) and Level of Unification (LoU) for the cache hierarchy.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	x000	0000	0001	0100	1000	0010	0000	0000	0000	0000	0010	0011
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-130: AArch64_clidr_el1 bit assignments

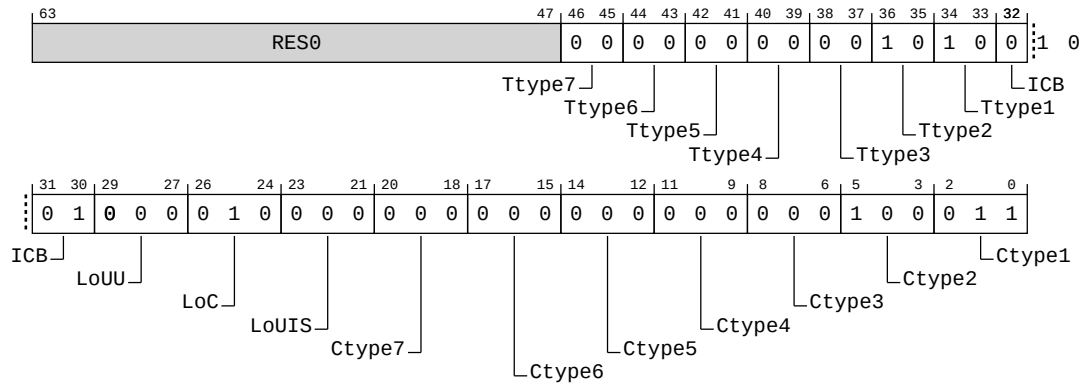


Table A-305: CLIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:47]	RES0	Reserved	RES0
[46:45]	Ttype7	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b00 No Tag Cache.	0b00
[44:43]	Ttype6	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b00 No Tag Cache.	0b00
[42:41]	Ttype5	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b00 No Tag Cache.	0b00
[40:39]	Ttype4	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b00 No Tag Cache.	0b00
[38:37]	Ttype3	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b00 No Tag Cache.	0b00

Bits	Name	Description	Reset
[36:35]	Ttype2	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b10 Unified Allocation Tag and Data cache, Allocation Tags and Data in unified lines.	0b10
[34:33]	Ttype1	Tag cache type. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. 0b10 Unified Allocation Tag and Data cache, Allocation Tags and Data in unified lines.	0b10
[32:30]	ICB	Inner cache boundary. This field indicates the boundary for caching Inner Cacheable memory regions. 0b010 L2 cache is the highest Inner Cacheable level.	0b010
[29:27]	LoUU	Level of Unification Uniprocessor for the cache hierarchy. Note: When FEAT_S2FWB is implemented, the architecture requires that this field is zero so that no levels of data cache need to be cleaned in order to manage coherency with instruction fetches. 0b000 Level of Unification Uniprocessor is before the L1 data cache.	0b000
[26:24]	LoC	Level of Coherence for the cache hierarchy. 0b010 Level of Coherency is after the L2 cache.	0b010
[23:21]	LoUIS	Level of Unification Inner Shareable for the cache hierarchy. Note: When FEAT_S2FWB is implemented, the architecture requires that this field is zero so that no levels of data cache need to be cleaned in order to manage coherency with instruction fetches. 0b000 Level of Unification Inner Shareable is before the L1 data cache.	0b000
[20:18]	Ctype7	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b000 No cache.	0b000
[17:15]	Ctype6	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b000 No cache.	0b000
[14:12]	Ctype5	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b000 No cache.	0b000

Bits	Name	Description	Reset
[11:9]	Ctype4	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b000 No cache.	0b000
[8:6]	Ctype3	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b000 No cache.	0b000
[5:3]	Ctype2	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b100 Unified cache.	0b100
[2:0]	Ctype1	Cache Type fields. Indicate the type of cache that is implemented and can be managed using the architected cache maintenance instructions that operate by set/way at each level, from Level 1 up to a maximum of seven levels of cache hierarchy. Possible values of each field are: 0b011 Separate instruction and data caches.	0b011

Access

MRS <Xt>, CLIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b001	0b0000	0b0000	0b001

Accessibility

MRS <Xt>, CLIDR_EL1

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID2 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.TID4 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.CLIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return CLIDR_EL1;
elseif PSTATE.EL == EL2 then
    return CLIDR_EL1;
elseif PSTATE.EL == EL3 then
    return CLIDR_EL1;

```

A.7.24 GMID_EL1, Multiple tag transfer ID register

Indicates the block size that is accessed by the LDGM and STGM System instructions.

Configurations

This register is available in all configurations.

Attributes

Width

64

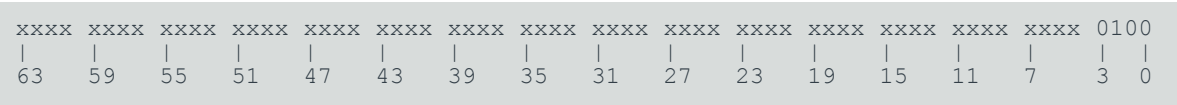
Functional group

Identification registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-131: AArch64_gmid_el1 bit assignments

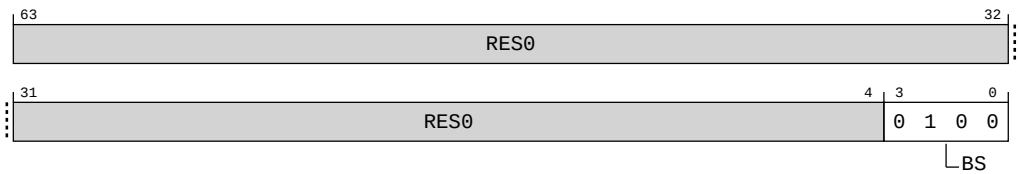


Table A-307: GMID_EL1 bit descriptions

Bits	Name	Description	Reset
[63:4]	RES0	Reserved	RES0
[3:0]	BS	Log ₂ of the block size in words. The minimum supported size is 16B (value == 2) and the maximum is 256B (value == 6). 0b0100 64 bytes.	0b0100

Access

MRS <Xt>, GMID_EL1

CRn	op0	op1	op2	CRm
0b0000	0b11	0b001	0b100	0b0000

Accessibility

MRS <Xt>, GMID_EL1

```
if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.TGE == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        AArch64.SystemAccessTrap(EL1, 0x18);
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID5 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return GMID_EL1;
elseif PSTATE.EL == EL2 then
    return GMID_EL1;
elseif PSTATE.EL == EL3 then
    return GMID_EL1;
```

A.7.25 CSSELR_EL1, Cache Size Selection Register

Selects the current Cache Size ID Register, AArch64-CCSIDR_EL1, by specifying the required cache level and the cache type (either instruction or data cache).

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-132: AArch64_cselr_el1 bit assignments

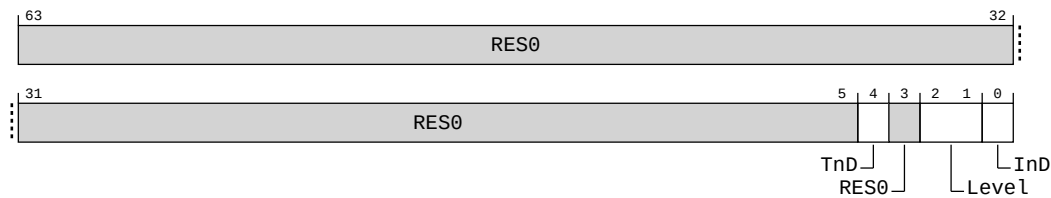


Table A-309: CSELR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:5]	RES0	Reserved	RES0
[4]	TnD	Allocation Tag not Data bit. 0b0 Data, Instruction or Unified cache.	x
[3]	RES0	Reserved	RES0
[2:1]	Level	Cache level of required cache. 0b00 Level 1 cache. 0b01 Level 2 cache. 0b10 Level 3 cache. All other values are reserved. If CSELR_EL1.Level is programmed to a cache level that is not implemented, then the value for this field on a read of CSELR_EL1 is UNKNOWN .	xx
[0]	InD	Instruction not Data bit. 0b0 Data or unified cache. 0b1 Instruction cache. If CSELR_EL1.Level is programmed to a cache level that is not implemented, then a read of CSELR_EL1 is CONSTRAINED UNPREDICTABLE, and returns UNKNOWN values for CSELR_EL1.{Level, InD}.	x

Access

MRS <Xt>, CSELR_EL1

op0	op1	CRn	CRm	op2
0b11	0b010	0b0000	0b0000	0b000

MSR CSSELR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b010	0b0000	0b0000	0b000

Accessibility

MRS <Xt>, CSSELR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID2 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.TID4 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.CSSELR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return CSSELR_EL1;
elseif PSTATE.EL == EL2 then
    return CSSELR_EL1;
elseif PSTATE.EL == EL3 then
    return CSSELR_EL1;

```

MSR CSSELR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && HCR_EL2.TID2 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && HCR_EL2.TID4 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.CSSELR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        CSSELR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    CSSELR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    CSSELR_EL1 = X[t];

```

A.7.26 CTR_EL0, Cache Type Register

Provides information about the architecture of the caches.

Configurations

This register is available in all configurations.

Attributes

Width

64

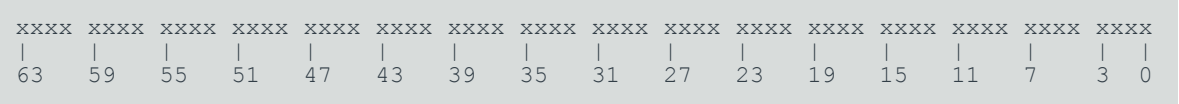
Functional group

Identification registers

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-133: AArch64_ctr_el0 bit assignments

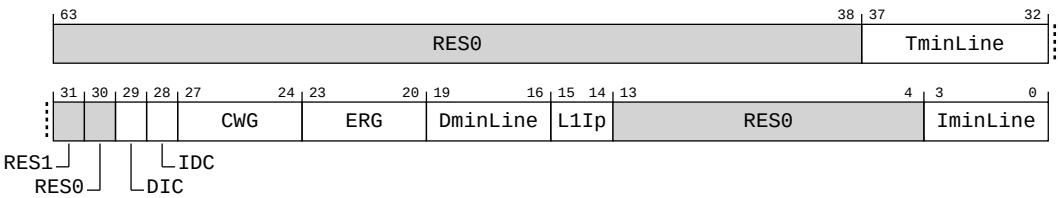


Table A-312: CTR_EL0 bit descriptions

Bits	Name	Description	Reset
[63:38]	RES0	Reserved	RES0
[37:32]	TminLine	Tag minimum Line. Log ₂ of the number of words covered by Allocation Tags in the smallest cache line of all caches which can contain Allocation tags that are controlled by the PE. 0b000100 64 bytes.	6{x}
[31]	RES1	Reserved	RES1
[30]	RES0	Reserved	RES0
[29]	DIC	Instruction cache invalidation requirements for data to instruction coherence. 0b0 Instruction cache invalidation to the Point of Unification is required for data to instruction coherence.	x
[28]	IDC	Data cache clean requirements for instruction to data coherence. The meaning of this bit is: 0b1 Data cache clean to the Point of Unification is not required for instruction to data coherence.	x

Bits	Name	Description	Reset
[27:24]	CWG	Cache writeback granule. Log ₂ of the number of words of the maximum size of memory that can be overwritten as a result of the eviction of a cache entry that has had a memory location in it modified. 0b0100 64 bytes.	xxxx
[23:20]	ERG	Exclusives reservation granule. Log ₂ of the number of words of the maximum size of the reservation granule for the Load-Exclusive and Store-Exclusive instructions. 0b0100 64 bytes.	xxxx
[19:16]	DminLine	Log ₂ of the number of words in the smallest cache line of all the data caches and unified caches that are controlled by the PE. 0b0100 64 bytes.	xxxx
[15:14]	L1Ip	Level 1 instruction cache policy. Indicates the indexing and tagging policy for the L1 instruction cache. Possible values of this field are: 0b11 Physical Index, Physical Tag (PIPT).	xx
[13:4]	RES0	Reserved	RES0
[3:0]	IminLine	Log ₂ of the number of words in the smallest cache line of all the instruction caches that are controlled by the PE. 0b0100 64 bytes.	xxxx

Access

MRS <Xt>, CTR_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b0000	0b0000	0b001

Accessibility

MRS <Xt>, CTR_ELO

```

if PSTATE.EL == EL0 then
    if !(EL2Enabled() && HCR_EL2.<E2H,TGE> == '11') && SCTLR_EL1.UCT == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && HCR_EL2.TID2 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
            HFGRTR_EL2.CTR_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> == '11' && SCTLR_EL2.UCT == '0' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            return CTR_EL0;
    elsif PSTATE.EL == EL1 then
        if EL2Enabled() && HCR_EL2.TID2 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGRTR_EL2.CTR_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);

```



```
else
    return CTR_EL0;
elseif PSTATE.EL == EL2 then
    return CTR_EL0;
elseif PSTATE.EL == EL3 then
    return CTR_EL0;
```

A.7.27 DCZID_EL0, Data Cache Zero ID register

Indicates the block size that is written with byte values of 0 by the DC ZVA (Data Cache Zero by Address) System instruction.

If FEAT_MTE is implemented, this register also indicates the granularity at which the DC GVA and DC GZVA instructions write.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Identification registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxx0	0100
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-134: AArch64_dcqid_el0 bit assignments

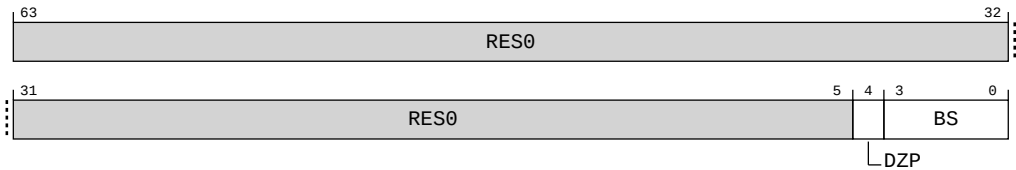


Table A-314: DCZID_ELO bit descriptions

Bits	Name	Description	Reset
[63:5]	RES0	Reserved	RES0
[4]	DZP	Data Zero Prohibited. This field indicates whether use of DC ZVA instructions is permitted or prohibited. If FEAT_MTE is implemented, this field also indicates whether use of the DC GVA and DC GZVA instructions are permitted or prohibited. 0b0 Instructions are permitted. 0b1 Instructions are prohibited. The value read from this field is governed by the access state and the values of the AArch64-HCR_EL2.TDZ and AArch64-SCTLR_EL1.DZE bits.	0b0
[3:0]	BS	$\log_2$ of the block size in words. The maximum size supported is 2KB (value == 9). 0b0100 64 bytes.	0b0100

Access

MRS <Xt>, DCZID_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b0000	0b0000	0b111

Accessibility

MRS <Xt>, DCZID_ELO

```

if PSTATE.EL == EL0 then
    if EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
        HFGTR_EL2.DCZID_ELO == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return DCZID_ELO;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.DCZID_ELO == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return DCZID_ELO;
elseif PSTATE.EL == EL2 then
    return DCZID_ELO;
elseif PSTATE.EL == EL3 then
    return DCZID_ELO;

```

A.8 AArch64 Generic Timer registers summary

The summary table provides an overview of all Generic Timer registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-316: Generic Timer registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
CNTKCTL_EL1	3	0	C14	C1	0	See individual bit resets.	64-bit	Counter-timer Kernel Control register
CNTFRQ_ELO	3	3	C14	C0	0	See individual bit resets.	64-bit	Counter-timer Frequency register
CNTPCT_ELO	3	3	C14	C0	1	See individual bit resets.	64-bit	Counter-timer Physical Count register
CNTVCT_ELO	3	3	C14	C0	2	See individual bit resets.	64-bit	Counter-timer Virtual Count register
CNTPCTSS_ELO	3	3	C14	C0	5	See individual bit resets.	64-bit	Counter-timer Self-Synchronized Physical Count register
CNTVCTSS_ELO	3	3	C14	C0	6	See individual bit resets.	64-bit	Counter-timer Self-Synchronized Virtual Count register
CNTP_TVAL_ELO	3	3	C14	C2	0	See individual bit resets.	64-bit	Counter-timer Physical Timer TimerValue register
CNTP_CTL_ELO	3	3	C14	C2	1	See individual bit resets.	64-bit	Counter-timer Physical Timer Control register
CNTP_CVAL_ELO	3	3	C14	C2	2	See individual bit resets.	64-bit	Counter-timer Physical Timer CompareValue register
CNTV_TVAL_ELO	3	3	C14	C3	0	See individual bit resets.	64-bit	Counter-timer Virtual Timer TimerValue register
CNTV_CTL_ELO	3	3	C14	C3	1	See individual bit resets.	64-bit	Counter-timer Virtual Timer Control register
CNTV_CVAL_ELO	3	3	C14	C3	2	See individual bit resets.	64-bit	Counter-timer Virtual Timer CompareValue register
CNTVOFF_EL2	3	4	C14	C0	3	See individual bit resets.	64-bit	Counter-timer Virtual Offset register
CNTPOFF_EL2	3	4	C14	C0	6	See individual bit resets.	64-bit	Counter-timer Physical Offset register
CNTHCTL_EL2	3	4	C14	C1	0	See individual bit resets.	64-bit	Counter-timer Hypervisor Control register
CNTHP_TVAL_EL2	3	4	C14	C2	0	See individual bit resets.	64-bit	Counter-timer Physical Timer TimerValue register (EL2)
CNTHP_CTL_EL2	3	4	C14	C2	1	See individual bit resets.	64-bit	Counter-timer Hypervisor Physical Timer Control register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
CNTHP_CVAL_EL2	3	4	C14	C2	2	See individual bit resets.	64-bit	Counter-timer Physical Timer CompareValue register (EL2)
CNTHV_TVAL_EL2	3	4	C14	C3	0	See individual bit resets.	64-bit	Counter-timer Virtual Timer TimerValue Register (EL2)
CNTHV_CTL_EL2	3	4	C14	C3	1	See individual bit resets.	64-bit	Counter-timer Virtual Timer Control register (EL2)
CNTHV_CVAL_EL2	3	4	C14	C3	2	See individual bit resets.	64-bit	Counter-timer Virtual Timer CompareValue register (EL2)
CNTHVS_TVAL_EL2	3	4	C14	C4	0	See individual bit resets.	64-bit	Counter-timer Secure Virtual Timer TimerValue register (EL2)
CNTHVS_CTL_EL2	3	4	C14	C4	1	See individual bit resets.	64-bit	Counter-timer Secure Virtual Timer Control register (EL2)
CNTHVS_CVAL_EL2	3	4	C14	C4	2	See individual bit resets.	64-bit	Counter-timer Secure Virtual Timer CompareValue register (EL2)
CNTHPS_TVAL_EL2	3	4	C14	C5	0	See individual bit resets.	64-bit	Counter-timer Secure Physical Timer TimerValue register (EL2)
CNTHPS_CTL_EL2	3	4	C14	C5	1	See individual bit resets.	64-bit	Counter-timer Secure Physical Timer Control register (EL2)
CNTHPS_CVAL_EL2	3	4	C14	C5	2	See individual bit resets.	64-bit	Counter-timer Secure Physical Timer CompareValue register (EL2)
CNTPS_TVAL_EL1	3	7	C14	C2	0	See individual bit resets.	64-bit	Counter-timer Physical Secure Timer TimerValue register
CNTPS_CTL_EL1	3	7	C14	C2	1	See individual bit resets.	64-bit	Counter-timer Physical Secure Timer Control register
CNTPS_CVAL_EL1	3	7	C14	C2	2	See individual bit resets.	64-bit	Counter-timer Physical Secure Timer CompareValue register

A.9 AArch64 Debug registers summary

The summary table provides an overview of all Debug registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-317: Debug registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
OSDTRRX_EL1	2	0	C0	C0	2	See individual bit resets.	64-bit	OS Lock Data Transfer Register, Receive
DBGBVR0_EL1	2	0	C0	C0	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR0_EL1	2	0	C0	C0	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR0_EL1	2	0	C0	C0	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
DBGWCR0_EL1	2	0	C0	C0	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
DBGBVR1_EL1	2	0	C0	C1	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR1_EL1	2	0	C0	C1	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR1_EL1	2	0	C0	C1	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR1_EL1	2	0	C0	C1	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
MDCCINT_EL1	2	0	C0	C2	0	See individual bit resets.	64-bit	Monitor DCC Interrupt Enable Register
MDSCR_EL1	2	0	C0	C2	2	See individual bit resets.	64-bit	Monitor Debug System Control Register
DBGBVR2_EL1	2	0	C0	C2	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR2_EL1	2	0	C0	C2	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR2_EL1	2	0	C0	C2	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR2_EL1	2	0	C0	C2	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
OSDTRTX_EL1	2	0	C0	C3	2	See individual bit resets.	64-bit	OS Lock Data Transfer Register, Transmit
DBGBVR3_EL1	2	0	C0	C3	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR3_EL1	2	0	C0	C3	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGWVR3_EL1	2	0	C0	C3	6	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
DBGWCR3_EL1	2	0	C0	C3	7	See individual bit resets.	64-bit	Debug Watchpoint Control Registers
DBGBVR4_EL1	2	0	C0	C4	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR4_EL1	2	0	C0	C4	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
DBGBVR5_EL1	2	0	C0	C5	4	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
DBGBCR5_EL1	2	0	C0	C5	5	See individual bit resets.	64-bit	Debug Breakpoint Control Registers
OSECCR_EL1	2	0	C0	C6	2	See individual bit resets.	64-bit	OS Lock Exception Catch Control Register
MDRAR_EL1	2	0	C1	C0	0	See individual bit resets.	64-bit	Monitor Debug ROM Address Register
OSLAR_EL1	2	0	C1	C0	4	See individual bit resets.	64-bit	OS Lock Access Register
OSLSR_EL1	2	0	C1	C1	4	See individual bit resets.	64-bit	OS Lock Status Register
OSDLR_EL1	2	0	C1	C3	4	See individual bit resets.	64-bit	OS Double Lock Register
DBGPRCR_EL1	2	0	C1	C4	4	See individual bit resets.	64-bit	Debug Power Control Register
DBGCLAIMSET_EL1	2	0	C7	C8	6	See individual bit resets.	64-bit	Debug CLAIM Tag Set register
DBGCLAIMCLR_EL1	2	0	C7	C9	6	See individual bit resets.	64-bit	Debug CLAIM Tag Clear register
DBGAUTHSTATUS_EL1	2	0	C7	C14	6	See individual bit resets.	64-bit	Debug Authentication Status register
MDCCSR_ELO	2	3	C0	C1	0	See individual bit resets.	64-bit	Monitor DCC Status Register
DBGDTR_ELO	2	3	C0	C4	0	See individual bit resets.	64-bit	Debug Data Transfer Register, half-duplex
DBGDTRRX_ELO	2	3	C0	C5	0	See individual bit resets.	64-bit	Debug Data Transfer Register, Receive
DBGDTRTX_ELO	2	3	C0	C5	0	See individual bit resets.	64-bit	Debug Data Transfer Register, Transmit
TRFCR_EL1	3	0	C1	C2	1	See individual bit resets.	64-bit	Trace Filter Control Register (EL1)
MDCR_EL2	3	4	C1	C1	1	See individual bit resets.	64-bit	Monitor Debug Configuration Register (EL2)
TRFCR_EL2	3	4	C1	C2	1	See individual bit resets.	64-bit	Trace Filter Control Register (EL2)

A.10 AArch64 Activity Monitors registers summary

The summary table provides an overview of all Activity Monitors registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-318: Activity Monitors registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AMCR_ELO	3	3	C13	C2	0	See individual bit resets.	64-bit	Activity Monitors Control Register
AMCFGR_ELO	3	3	C13	C2	1	See individual bit resets.	64-bit	Activity Monitors Configuration Register
AMCGCR_ELO	3	3	C13	C2	2	See individual bit resets.	64-bit	Activity Monitors Counter Group Configuration Register
AMUSERENR_ELO	3	3	C13	C2	3	See individual bit resets.	64-bit	Activity Monitors User Enable Register
AMCNTENCLR0_ELO	3	3	C13	C2	4	See individual bit resets.	64-bit	Activity Monitors Count Enable Clear Register 0
AMCNTENSET0_ELO	3	3	C13	C2	5	See individual bit resets.	64-bit	Activity Monitors Count Enable Set Register 0
AMCNTENCLR1_ELO	3	3	C13	C3	0	See individual bit resets.	64-bit	Activity Monitors Count Enable Clear Register 1
AMCNTENSET1_ELO	3	3	C13	C3	1	See individual bit resets.	64-bit	Activity Monitors Count Enable Set Register 1
AMEVCNTR00_ELO	3	3	C13	C4	0	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVCNTR01_ELO	3	3	C13	C4	1	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVCNTR02_ELO	3	3	C13	C4	2	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVCNTR03_ELO	3	3	C13	C4	3	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 0
AMEVTYPER00_ELO	3	3	C13	C6	0	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVTYPER01_ELO	3	3	C13	C6	1	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVTYPER02_ELO	3	3	C13	C6	2	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVTYPER03_ELO	3	3	C13	C6	3	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 0
AMEVCNTR10_ELO	3	3	C13	C12	0	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 1

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
AMEVCNTR11_ELO	3	3	C13	C12	1	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 1
AMEVCNTR12_ELO	3	3	C13	C12	2	See individual bit resets.	64-bit	Activity Monitors Event Counter Registers 1
AMEVTYPER10_ELO	3	3	C13	C14	0	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 1
AMEVTYPER11_ELO	3	3	C13	C14	1	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 1
AMEVTYPER12_ELO	3	3	C13	C14	2	See individual bit resets.	64-bit	Activity Monitors Event Type Registers 1

A.10.1 AMCFGR_ELO, Activity Monitors Configuration Register

Global configuration register for the activity monitors.

Provides information on supported features, the number of counter groups implemented, the total number of activity monitor event counters implemented, and the size of the counters. AMCFGR_ELO is applicable to both the architected and the auxiliary counter groups.

Configurations

AArch64 register AMCFGR_ELO bits [31:0] are architecturally mapped to External System register [B.5.9 AMCFGR, Activity Monitors Configuration Register](#) on page 740 bits [31:0].

Attributes

Width

64

Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	xxx1	0000	0000	0011	1111	0000	0110
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-135: AArch64_amcfgr_el0 bit assignments

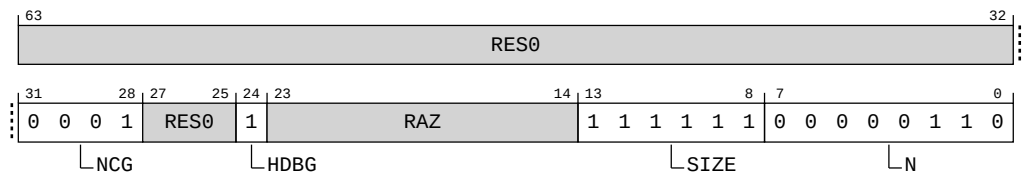


Table A-319: AMCFGR_ELO bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:28]	NCG	Defines the number of counter groups. The following value is specified for this product. 0b0001 Two counter groups are implemented	0b0001
[27:25]	RES0	Reserved	RES0
[24]	HDBG	Halt-on-debug supported. This feature must be supported, and so this bit is 0b1. 0b1 AArch64-AMCR_ELO.HDBG is read/write.	0b1
[23:14]	RAZ	Reserved	RAZ
[13:8]	SIZE	Defines the size of activity monitor event counters. The size of the activity monitor event counters implemented by the activity monitors Extension is [AMCFGR_ELO.SIZE + 1]. Note: Software also uses this field to determine the spacing of counters in the memory-map. From Armv8, the counters are at doubleword-aligned addresses. 0b111111	0b111111
[7:0]	N	Defines the number of activity monitor event counters. The total number of counters implemented in all groups by the Activity Monitors Extension is [AMCFGR_ELO.N + 1]. 0b00000110 Seven activity monitor event counters	0x06

Access

MRS <Xt>, AMCFGR_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b0010	0b001

Accessibility

MRS <Xt>, AMCFGR_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMCFGR_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMCFGR_ELO;
        elsif PSTATE.EL == EL2 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMCFGR_ELO;
        elsif PSTATE.EL == EL3 then
            return AMCFGR_ELO;

```

A.10.2 AMCGCR_ELO, Activity Monitors Counter Group Configuration Register

Provides information on the number of activity monitor event counters implemented within each counter group.

Configurations

AArch64 register AMCGCR_ELO bits [31:0] are architecturally mapped to External System register [B.5.8 AMCGCR, Activity Monitors Counter Group Configuration Register](#) on page 739 bits [31:0].

Attributes

Width

64

Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0011	0000	0100
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-136: AArch64_amcgcr_el0 bit assignments

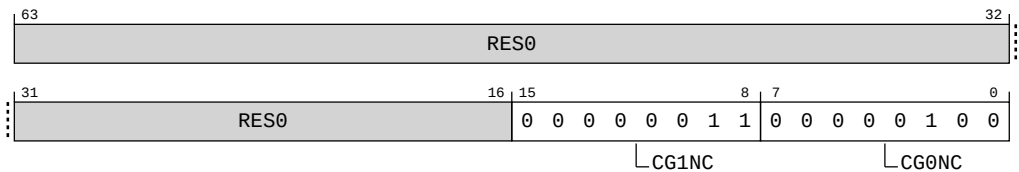


Table A-321: AMCGCR_EL0 bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15:8]	CG1NC	Counter Group 1 Number of Counters. The number of counters in the auxiliary counter group. In an implementation that includes FEAT_AMUV1, the permitted range of values is 0x0 to 0x10. 0b00000011 Three counters in the auxiliary counter group	0x03
[7:0]	CG0NC	Counter Group 0 Number of Counters. The number of counters in the architected counter group. 0b00000100	0x04

Access

MRS <Xt>, AMCGCR_EL0

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b0010	0b010

Accessibility

MRS <Xt>, AMCGCR_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMCGCR_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMCGCR_ELO;
        elsif PSTATE.EL == EL2 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMCGCR_ELO;
        elsif PSTATE.EL == EL3 then
            return AMCGCR_ELO;

```

A.10.3 AMEVTYPER00_ELO, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR00_ELO counts.

Configurations

AArch64 register AMEVTYPER00_ELO bits [31:0] are architecturally mapped to External System register [B.5.1 AMEVTYPER00, Activity Monitors Event Type Registers 0](#) on page 727 bits [31:0].

Attributes

Width

64

Functional group
Activity Monitors registers

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-137: AArch64_amevtyper00_el0 bit assignments

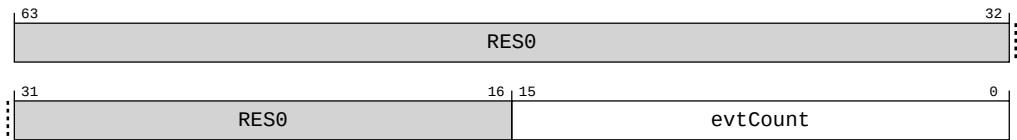


Table A-323: AMEVTYPER00_EL0 bit descriptions

Bits	Name	Description	Reset		
[63:16]	RES0	Reserved	RES0		
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter AArch64-AMEVCNTR0<n>_ELO. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: <table><tr><td>0b000000000000010001</td><td>Processor frequency cycles</td></tr></table>	0b000000000000010001	Processor frequency cycles	The reset values can be the following: 0b00000000000010001, respective to the value.
0b000000000000010001	Processor frequency cycles				

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are **UNDEFINED**.



AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPEPER00_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b11101	0b0110	0b000

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPEPER0<n>_ELO are UNDEFINED.



AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPEPER00_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPEPER00_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return AMEVTYPEPER00_ELO;
            elsif PSTATE.EL == EL2 then
                if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                    UNDEFINED;
                elsif CPTR_EL3.TAM == '1' then
                    if Halted() && EDSCR.SDD == '1' then
                        UNDEFINED;
                    else
                        AArch64.SystemAccessTrap(EL3, 0x18);
                    else
                        return AMEVTYPEPER00_ELO;
            elsif PSTATE.EL == EL3 then
                return AMEVTYPEPER00_ELO;

```

A.10.4 AMEVTYPER01_ELO, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR01_ELO counts.

Configurations

AArch64 register AMEVTYPER01_ELO bits [31:0] are architecturally mapped to External System register [B.5.2 AMEVTYPER01, Activity Monitors Event Type Registers 0](#) on page 729 bits [31:0].

Attributes

Width

64

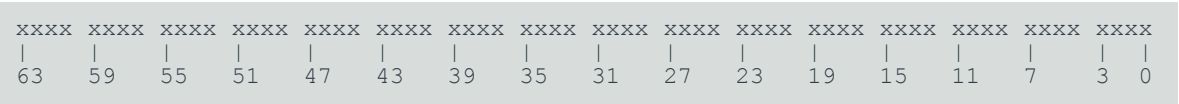
Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-138: AArch64_amevtyper01_el0 bit assignments

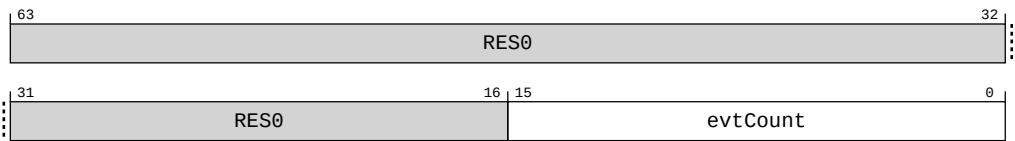


Table A-325: AMEVTYPER01_ELO bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter AArch64-AMEVCNTR0<n>_ELO. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: 0b0100000000000100 Constant frequency cycles	The reset values can be the following: 0b0100000000000100, respective to the value.

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are **UNDEFINED**.



Note

AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPER01_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b0110	0b001

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are **UNDEFINED**.



Note

AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPER01_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else

```

```
        return AMEVTYPER01_EL0;
    elseif PSTATE.EL == EL1 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
            UNDEFINED;
        elseif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPER01_EL0;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
            UNDEFINED;
        elseif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPER01_EL0;
    elseif PSTATE.EL == EL3 then
        return AMEVTYPER01_EL0;
```

A.10.5 AMEVTYPER02_EL0, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR02_EL0 counts.

Configurations

AArch64 register AMEVTYPER02_EL0 bits [31:0] are architecturally mapped to External System register [B.5.3 AMEVTYPER02, Activity Monitors Event Type Registers 0](#) on page 730 bits [31:0].

Attributes

Width

64

Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-139: AArch64_amevtyper02_el0 bit assignments

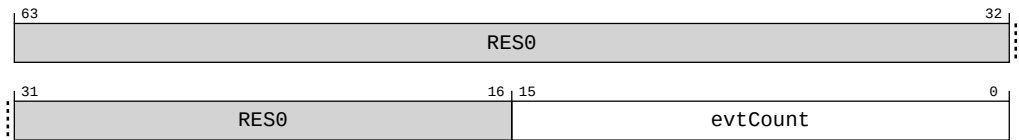


Table A-327: AMEVTYPER02_ELO bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter AArch64-AMEVCNTR0<n>_ELO. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: 0b00000000000001000 Instructions retired	The reset values can be the following: 0b0000000000001000, relative to the value.

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are **UNDEFINED**.



Note

AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPER02_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b0110	0b010

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are **UNDEFINED**.



Note

AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPER02_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPER02_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return AMEVTYPER02_ELO;
            elsif PSTATE.EL == EL2 then
                if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                    UNDEFINED;
                elsif CPTR_EL3.TAM == '1' then
                    if Halted() && EDSCR.SDD == '1' then
                        UNDEFINED;
                    else
                        AArch64.SystemAccessTrap(EL3, 0x18);
                    else
                        return AMEVTYPER02_ELO;
            elsif PSTATE.EL == EL3 then
                return AMEVTYPER02_ELO;

```

A.10.6 AMEVTYPER03_ELO, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR03_ELO counts.

Configurations

AArch64 register AMEVTYPER03_ELO bits [31:0] are architecturally mapped to External System register [B.5.4 AMEVTYPER03, Activity Monitors Event Type Registers 0](#) on page 732 bits [31:0].

Attributes

Width

64

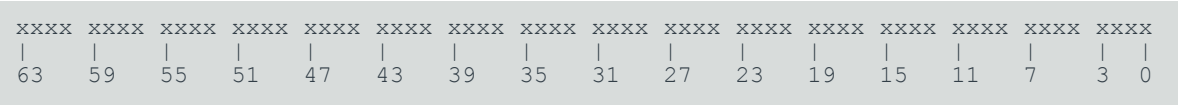
Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-140: AArch64_amevtyper03_el0 bit assignments

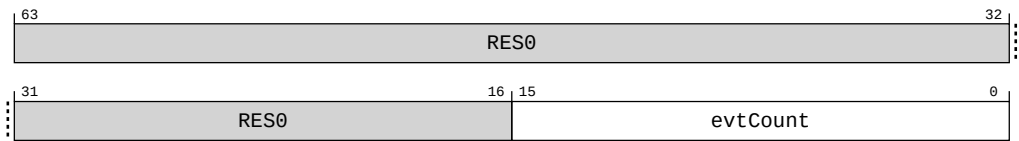


Table A-329: AMEVTYPER03_ELO bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter AArch64-AMEVCNTR0<n>_ELO. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: 0b0100000000000101 Memory stall cycles	The reset values can be the following: 0b0100000000000101, respective to the value.

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are **UNDEFINED**.



AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPER03_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b0110	0b011

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads and writes of AMEVTYPER0<n>_ELO are UNDEFINED.



AArch64-AMCGCR_ELO.CG0NC identifies the number of architected activity monitor event counters.

MRS <Xt>, AMEVTYPER03_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPER03_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return AMEVTYPER03_ELO;
            elsif PSTATE.EL == EL2 then
                if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                    UNDEFINED;
                elsif CPTR_EL3.TAM == '1' then
                    if Halted() && EDSCR.SDD == '1' then
                        UNDEFINED;
                    else
                        AArch64.SystemAccessTrap(EL3, 0x18);
                    else
                        return AMEVTYPER03_ELO;
            elsif PSTATE.EL == EL3 then
                return AMEVTYPER03_ELO;

```

A.10.7 AMEVTYPER10_ELO, Activity Monitors Event Type Registers 1

Provides information on the events that an auxiliary activity monitor event counter AArch64-AMEVCNTR10_ELO counts.

Configurations

AArch64 register AMEVTYPER10_ELO bits [31:0] are architecturally mapped to External System register [B.5.5 AMEVTYPER10, Activity Monitors Event Type Registers 1](#) on page 734 bits [31:0].

Attributes

Width

64

Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0011	0000	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-141: AArch64_amevtyper10_el0 bit assignments

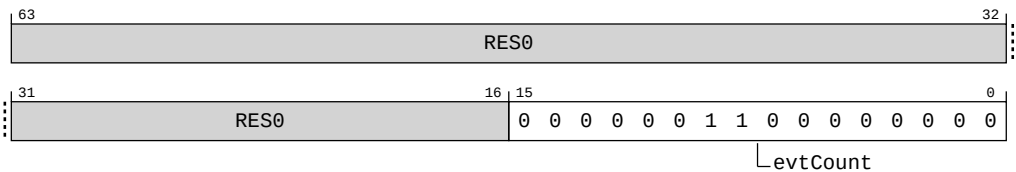


Table A-331: AMEVTYPER10_ELO bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the auxiliary activity monitor event counter AMEVCNTR10_ELO. 0b0000000110000000 MPMM gear 0 period threshold exceeded	0x0300

Access

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads and writes of AMEVTYPER1<n>_ELO are **UNDEFINED**.



AArch64-AMCGCR_ELO.CG1NC identifies the number of auxiliary activity monitor event counters.

MRS <Xt>, AMEVTYPER10_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b11101	0b11110	0b000

Accessibility

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads and writes of AMEVTYPER1<n>_ELO are **UNDEFINED**.



AArch64-AMCGCR_ELO.CG1NC identifies the number of auxiliary activity monitor event counters.

MRS <Xt>, AMEVTYPER10_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
            HAFGRTR_EL2.AMEVTYPER10_ELO == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPER10_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HAFGRTR_EL2.AMEVTYPER10_ELO == '1'
            then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then

```

```
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return AMEVTYPYPER10_EL0;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elseif CPTR_EL3.TAM == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return AMEVTYPYPER10_EL0;
elseif PSTATE.EL == EL3 then
    return AMEVTYPYPER10_EL0;
```

A.10.8 AMEVTYPYPER11_EL0, Activity Monitors Event Type Registers 1

Provides information on the events that an auxiliary activity monitor event counter AArch64-AMEVCNTR11_EL0 counts.

Configurations

AArch64 register AMEVTYPYPER11_EL0 bits [31:0] are architecturally mapped to External System register [B.5.6 AMEVTYPYPER11, Activity Monitors Event Type Registers 1](#) on page 736 bits [31:0].

Attributes

Width

64

Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0011	0000	0001
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-142: AArch64_amevtyper11_el0 bit assignments

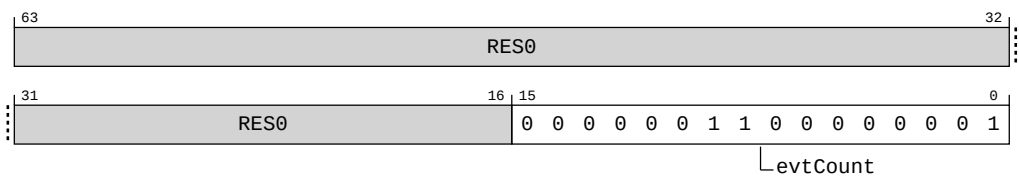


Table A-333: AMEVTYPER11_ELO bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the auxiliary activity monitor event counter AMEVCNTR11_ELO. 0b0000000110000001 MPMM gear 1 period threshold exceeded	0x0301

Access

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads and writes of AMEVTYPER1<n>_ELO are **UNDEFINED**.



Note

AArch64-AMCGCR_ELO.CG1NC identifies the number of auxiliary activity monitor event counters.

MRS <Xt>, AMEVTYPER11_ELO

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b1110	0b001

Accessibility

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads and writes of AMEVTYPER1<n>_ELO are UNDEFINED.



Note

AArch64-AMCGCR_ELO.CG1NC identifies the number of auxiliary activity monitor event counters.

MRS <Xt>, AMEVTYPER11_ELO

```
if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
```



```

        UNDEFINED;
    elseif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elseif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
        HAFGRTR_EL2.AMEVTPER11_EL0 == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTPER11_EL0;
        elseif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elseif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HAFGRTR_EL2.AMEVTPER11_EL0 == '1'
            then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elseif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return AMEVTPER11_EL0;
        elseif PSTATE.EL == EL2 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elseif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return AMEVTPER11_EL0;
        elseif PSTATE.EL == EL3 then
            return AMEVTPER11_EL0;

```

A.10.9 AMEVTPER12_EL0, Activity Monitors Event Type Registers 1

Provides information on the events that an auxiliary activity monitor event counter AArch64-AMEVCNTR12_EL0 counts.

Configurations

AArch64 register AMEVTPER12_EL0 bits [31:0] are architecturally mapped to External System register [B.5.7 AMEVTPER12, Activity Monitors Event Type Registers 1](#) on page 737 bits [31:0].

Attributes

Width

64

Functional group

Activity Monitors registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0011	0000	0010
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-143: AArch64_amevtyper12_el0 bit assignments

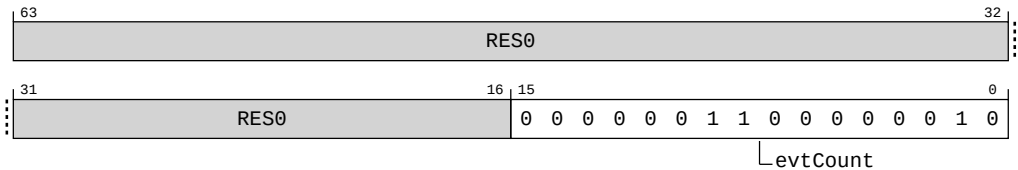


Table A-335: AMEVTYPER12_EL0 bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the auxiliary activity monitor event counter AMEVCNTR12_EL0. 0b0000000110000010 MPMM gear 2 period threshold exceeded	0x0302

Access

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads and writes of AMEVTYPER1<n>_ELO are **UNDEFINED**.



AArch64-AMCGCR_EL0.CG1NC identifies the number of auxiliary activity monitor event counters.

MRS <Xt>, AMEVTYPER12_EL0

op0	op1	CRn	CRm	op2
0b11	0b011	0b1101	0b1110	0b010

Accessibility

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads and writes of AMEVTYPER1<n>_ELO are UNDEFINED.



AArch64-AMCGCR_ELO.CG1NC identifies the number of auxiliary activity monitor event counters.

MRS <Xt>, AMEVTYPER12_ELO

```

if PSTATE.EL == EL0 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
        UNDEFINED;
    elsif AMUSERENR_EL0.EN == '0' then
        if EL2Enabled() && HCR_EL2.TGE == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        else
            AArch64.SystemAccessTrap(EL1, 0x18);
        elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif EL2Enabled() && HCR_EL2.<E2H,TGE> != '11' && SCR_EL3.FGTEn == '1' &&
            HAFGRTR_EL2.AMEVTYPER12_ELO == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TAM == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return AMEVTYPER12_ELO;
        elsif PSTATE.EL == EL1 then
            if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                UNDEFINED;
            elsif EL2Enabled() && CPTR_EL2.TAM == '1' then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HAFGRTR_EL2.AMEVTYPER12_ELO == '1'
            then
                AArch64.SystemAccessTrap(EL2, 0x18);
            elsif CPTR_EL3.TAM == '1' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
                else
                    return AMEVTYPER12_ELO;
            elsif PSTATE.EL == EL2 then
                if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TAM == '1' then
                    UNDEFINED;
                elsif CPTR_EL3.TAM == '1' then
                    if Halted() && EDSCR.SDD == '1' then
                        UNDEFINED;
                    else
                        AArch64.SystemAccessTrap(EL3, 0x18);
                    else
                        return AMEVTYPER12_ELO;
            elsif PSTATE.EL == EL3 then
                return AMEVTYPER12_ELO;

```

A.11 AArch64 Trace unit registers summary

The summary table provides an overview of all Trace unit registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-337: Trace unit registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCTRACEIDR	2	1	C0	C0	1	See individual bit resets.	64-bit	Trace ID Register
TRCVICTLR	2	1	C0	C0	2	See individual bit resets.	64-bit	ViewInst Main Control Register
TRCSEQEVR0	2	1	C0	C0	4	See individual bit resets.	64-bit	Sequencer State Transition Control Register <n>
TRCCNTRLDVRO	2	1	C0	C0	5	See individual bit resets.	64-bit	Counter Reload Value Register <n>
TRCIDR8	2	1	C0	C0	6	See individual bit resets.	64-bit	ID Register 8
TRCIMSPEC0	2	1	C0	C0	7	See individual bit resets.	64-bit	IMP DEF Register 0
TRCPRGCTLR	2	1	C0	C1	0	See individual bit resets.	64-bit	Programming Control Register
TRCVIIECTLR	2	1	C0	C1	2	See individual bit resets.	64-bit	ViewInst Include/Exclude Control Register
TRCSEQEVR1	2	1	C0	C1	4	See individual bit resets.	64-bit	Sequencer State Transition Control Register <n>
TRCCNTRLDVR1	2	1	C0	C1	5	See individual bit resets.	64-bit	Counter Reload Value Register <n>
TRCIDR9	2	1	C0	C1	6	See individual bit resets.	64-bit	ID Register 9
TRCVISSCTLR	2	1	C0	C2	2	See individual bit resets.	64-bit	ViewInst Start/Stop Control Register
TRCSEQEVR2	2	1	C0	C2	4	See individual bit resets.	64-bit	Sequencer State Transition Control Register <n>
TRCIDR10	2	1	C0	C2	6	See individual bit resets.	64-bit	ID Register 10
TRCSTATR	2	1	C0	C3	0	See individual bit resets.	64-bit	Trace Status Register
TRCIDR11	2	1	C0	C3	6	See individual bit resets.	64-bit	ID Register 11
TRCCONFIGR	2	1	C0	C4	0	See individual bit resets.	64-bit	Trace Configuration Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCCNTCTLR0	2	1	C0	C4	5	See individual bit resets.	64-bit	Counter Control Register <n>
TRCIDR12	2	1	C0	C4	6	See individual bit resets.	64-bit	ID Register 12
TRCCNTCTLR1	2	1	C0	C5	5	See individual bit resets.	64-bit	Counter Control Register <n>
TRCIDR13	2	1	C0	C5	6	See individual bit resets.	64-bit	ID Register 13
TRCAUXCTLR	2	1	C0	C6	0	See individual bit resets.	64-bit	Auxiliary Control Register
TRCSEQRSTEV	2	1	C0	C6	4	See individual bit resets.	64-bit	Sequencer Reset Control Register
TRCSEQSTR	2	1	C0	C7	4	See individual bit resets.	64-bit	Sequencer State Register
TRCEVENTCTLR0	2	1	C0	C8	0	See individual bit resets.	64-bit	Event Control 0 Register
TRCEXTINSEL0	2	1	C0	C8	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCCNTVR0	2	1	C0	C8	5	See individual bit resets.	64-bit	Counter Value Register <n>
TRCIDR0	2	1	C0	C8	7	See individual bit resets.	64-bit	ID Register 0
TRCEVENTCTLR1	2	1	C0	C9	0	See individual bit resets.	64-bit	Event Control 1 Register
TRCEXTINSEL1	2	1	C0	C9	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCCNTVR1	2	1	C0	C9	5	See individual bit resets.	64-bit	Counter Value Register <n>
TRCIDR1	2	1	C0	C9	7	See individual bit resets.	64-bit	ID Register 1
TRCRSR	2	1	C0	C10	0	See individual bit resets.	64-bit	Resources Status Register
TRCEXTINSEL2	2	1	C0	C10	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCIDR2	2	1	C0	C10	7	See individual bit resets.	64-bit	ID Register 2
TRCEXTINSEL3	2	1	C0	C11	4	See individual bit resets.	64-bit	External Input Select Register <n>
TRCIDR3	2	1	C0	C11	7	See individual bit resets.	64-bit	ID Register 3
TRCTSCTLR	2	1	C0	C12	0	See individual bit resets.	64-bit	Timestamp Control Register
TRCIDR4	2	1	C0	C12	7	See individual bit resets.	64-bit	ID Register 4
TRCSYNCP	2	1	C0	C13	0	See individual bit resets.	64-bit	Synchronization Period Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCIDR5	2	1	C0	C13	7	See individual bit resets.	64-bit	ID Register 5
TRCCCCTLR	2	1	C0	C14	0	See individual bit resets.	64-bit	Cycle Count Control Register
TRCIDR6	2	1	C0	C14	7	See individual bit resets.	64-bit	ID Register 6
TRCBBCTLR	2	1	C0	C15	0	See individual bit resets.	64-bit	Branch Broadcast Control Register
TRCIDR7	2	1	C0	C15	7	See individual bit resets.	64-bit	ID Register 7
TRCSSCCRO	2	1	C1	C0	2	See individual bit resets.	64-bit	Single-shot Comparator Control Register <n>
TRCOSLSR	2	1	C1	C1	4	See individual bit resets.	64-bit	Trace OS Lock Status Register
TRCRSCTLR2	2	1	C1	C2	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR3	2	1	C1	C3	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR4	2	1	C1	C4	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR5	2	1	C1	C5	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR6	2	1	C1	C6	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR7	2	1	C1	C7	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR8	2	1	C1	C8	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCSSCSR0	2	1	C1	C8	2	See individual bit resets.	64-bit	Single-shot Comparator Control Status Register <n>
TRCRSCTLR9	2	1	C1	C9	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR10	2	1	C1	C10	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR11	2	1	C1	C11	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR12	2	1	C1	C12	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR13	2	1	C1	C13	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR14	2	1	C1	C14	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCRSCTLR15	2	1	C1	C15	0	See individual bit resets.	64-bit	Resource Selection Control Register <n>
TRCACVR0	2	1	C2	C0	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCACATR0	2	1	C2	C0	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR1	2	1	C2	C2	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR1	2	1	C2	C2	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR2	2	1	C2	C4	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR2	2	1	C2	C4	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR3	2	1	C2	C6	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR3	2	1	C2	C6	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR4	2	1	C2	C8	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR4	2	1	C2	C8	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR5	2	1	C2	C10	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR5	2	1	C2	C10	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR6	2	1	C2	C12	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR6	2	1	C2	C12	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCACVR7	2	1	C2	C14	0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
TRCACATR7	2	1	C2	C14	2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
TRCCIDCVR0	2	1	C3	C0	0	See individual bit resets.	64-bit	Context Identifier Comparator Value Registers <n>
TRCVMIDCVR0	2	1	C3	C0	1	See individual bit resets.	64-bit	Virtual Context Identifier Comparator Value Register <n>
TRCCIDCCTLR0	2	1	C3	C0	2	See individual bit resets.	64-bit	Context Identifier Comparator Control Register 0
TRCVMIDCCTLR0	2	1	C3	C2	2	See individual bit resets.	64-bit	Virtual Context Identifier Comparator Control Register 0
TRCDEVID	2	1	C7	C2	7	See individual bit resets.	64-bit	Device Configuration Register
TRCCLAIMSET	2	1	C7	C8	6	See individual bit resets.	64-bit	Claim Tag Set Register
TRCCLAIMCLR	2	1	C7	C9	6	See individual bit resets.	64-bit	Claim Tag Clear Register
TRCAUTHSTATUS	2	1	C7	C14	6	See individual bit resets.	64-bit	Authentication Status Register

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRCDEVARCH	2	1	C7	C15	6	See individual bit resets.	64-bit	Device Architecture Register

A.11.1 TRCIDR8, ID Register 8

Returns the maximum speculation depth of the instruction trace element stream.

Configurations

AArch64 register TRCIDR8 bits [31:0] are architecturally mapped to External System register [B.6.2 TRCIDR8, ID Register 8](#) on page 762 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	0000	0000	0000	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-144: AArch64_trcidr8 bit assignments



Table A-338: TRCIDR8 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[31:0]	MAXSPEC	Indicates the maximum speculation depth of the instruction trace element stream. This is the maximum number of PO elements in the trace element stream that can be speculative at any time. 0b00000000000000000000000000000000	0x00000000

Access

MRS <Xt>, TRCIDR8

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0000	0b110

Accessibility

MRS <Xt>, TRCIDR8

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR8;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR8;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR8;

```

A.11.2 TRCIMSPECO, IMP DEF Register 0

TRCIMSPECO shows the presence of any **IMPLEMENTATION DEFINED** features, and provides an interface to enable the features that are provided.

Configurations

AArch64 register TRCIMSPECO bits [31:0] are architecturally mapped to External System register [B.6.8 TRCIMSPECO, IMP DEF Register 0](#) on page 769 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-145: AArch64_trcimspec0 bit assignments

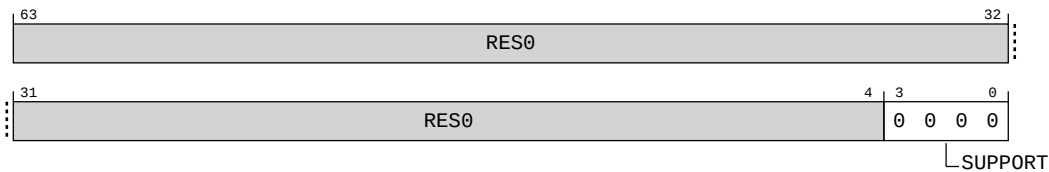


Table A-340: TRCIMSPECO bit descriptions

Bits	Name	Description	Reset
[63:4]	RES0	Reserved	RES0
[3:0]	SUPPORT	Indicates whether the implementation supports IMPLEMENTATION DEFINED features. 0b0000 No IMPLEMENTATION DEFINED features are supported.	0b0000

Access

MRS <Xt>, TRCIMSPECO

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0000	0b111

MSR TRCIMSPECO, <Xt>

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0000	0b111

Accessibility

MRS <Xt>, TRCIMSPECO

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCIMSPECn == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIMSPECO;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIMSPECO;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIMSPECO;

```

MSR TRCIMSPECO, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then

```

```

    AArch64.SystemAccessTrap(EL2, 0x18);
elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.TRCIMSPECn == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif CPTR_EL3.TTA == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        TRCIMSPEC0 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCIMSPEC0 = X[t];
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        TRCIMSPEC0 = X[t];

```

A.11.3 TRCIDR9, ID Register 9

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR9 bits [31:0] are architecturally mapped to External System register [B.6.3 TRCIDR9, ID Register 9](#) on page 763 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-146: AArch64_trcidr9 bit assignments

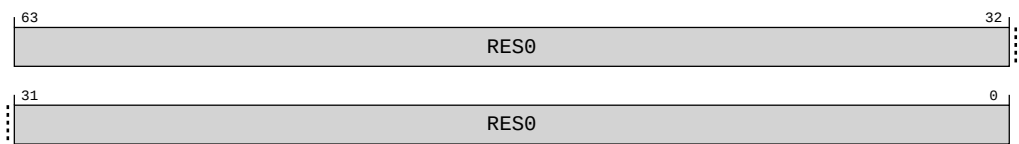


Table A-343: TRCIDR9 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCIDR9

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0001	0b110

Accessibility

MRS <Xt>, TRCIDR9

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end
    else
        return TRCIDR9;
    end
elsif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end
    else
        return TRCIDR9;
    end
elsif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
```

```
return TRCIDR9;
```

A.11.4 TRCIDR10, ID Register 10

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR10 bits [31:0] are architecturally mapped to External System register [B.6.4 TRCIDR10, ID Register 10](#) on page 765 bits [31:0].

Attributes

Width

64

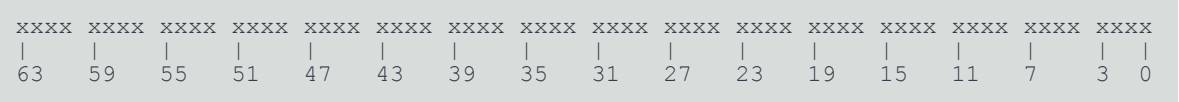
Functional group

Trace unit registers

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-147: AArch64_trcidr10 bit assignments

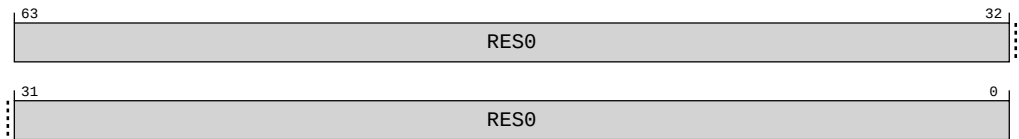


Table A-345: TRCIDR10 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCIDR10

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0010	0b110

Accessibility

MRS <Xt>, TRCIDR10

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR10;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elsif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR10;
    elsif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR10;

```

A.11.5 TRCIDR11, ID Register 11

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR11 bits [31:0] are architecturally mapped to External System register [B.6.5 TRCIDR11, ID Register 11](#) on page 766 bits [31:0].

Attributes

Width

64

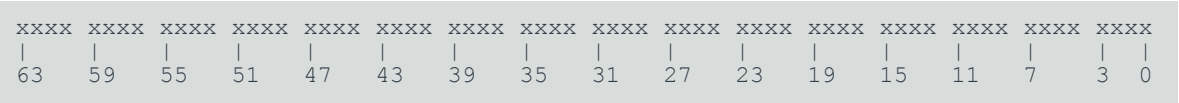
Functional group

Trace unit registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-148: AArch64_trcidr11 bit assignments

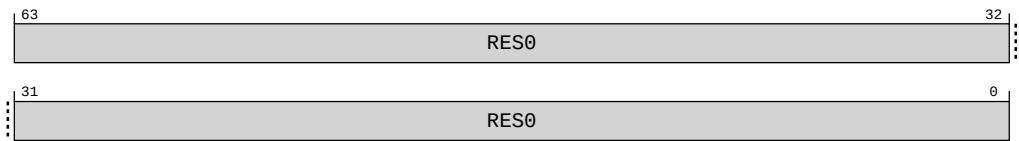


Table A-347: TRCIDR11 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCIDR11

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0011	0b110

Accessibility

MRS <Xt>, TRCIDR11

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
```



```
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCIDR11;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elsif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR11;
        elsif PSTATE.EL == EL3 then
            if CPTR_EL3.TTA == '1' then
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR11;
```

A.11.6 TRCIDR12, ID Register 12

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR12 bits [31:0] are architecturally mapped to External System register [B.6.6 TRCIDR12, ID Register 12](#) on page 767 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-149: AArch64_trcidr12 bit assignments

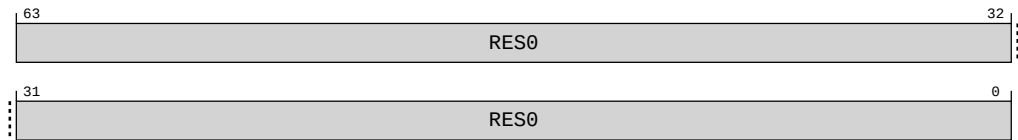


Table A-349: TRCIDR12 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCIDR12

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0100	0b110

Accessibility

MRS <Xt>, TRCIDR12

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR12;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elsif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR12;
    elsif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else

```

```
return TRCIDR12;
```

A.11.7 TRCIDR13, ID Register 13

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR13 bits [31:0] are architecturally mapped to External System register [B.6.7 TRCIDR13, ID Register 13](#) on page 768 bits [31:0].

Attributes

Width

64

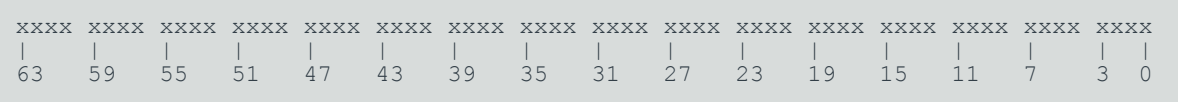
Functional group

Trace unit registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-150: AArch64_trcidr13 bit assignments

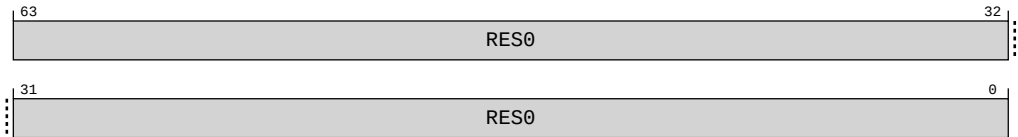


Table A-351: TRCIDR13 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCIDR13

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0101	0b110

Accessibility

MRS <Xt>, TRCIDR13

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR13;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elsif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR13;
        elsif PSTATE.EL == EL3 then
            if CPTR_EL3.TTA == '1' then
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR13;

```

A.11.8 TRCAUXCTLR, Auxiliary Control Register

The function of this register is **IMPLEMENTATION DEFINED**.

Configurations

AArch64 register TRCAUXCTLR bits [31:0] are architecturally mapped to External System register [B.6.1 TRCAUXCTLR, Auxiliary Control Register](#) on page 761 bits [31:0].

Attributes

Width

64

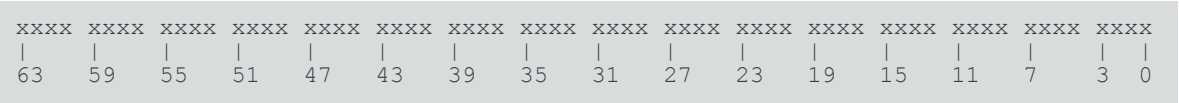
Functional group

Trace unit registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-151: AArch64_trcauxctlr bit assignments

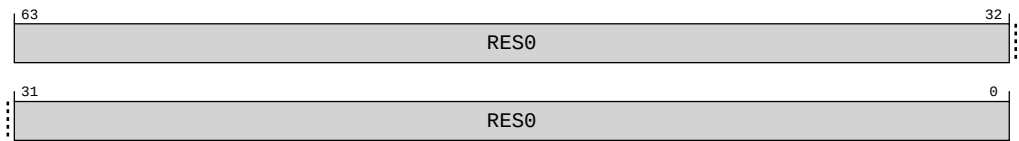


Table A-353: TRCAUXCTLR bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

If this register is nonzero then it might cause the behavior of a trace unit to contradict this architecture specification. See the documentation of the specific implementation for information about the **IMPLEMENTATION DEFINED** support for this register.

MRS <Xt>, TRCAUXCTLR

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0110	0b000

MSR TRCAUXCTLR, <Xt>

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b0110	0b000

Accessibility

If this register is nonzero then it might cause the behavior of a trace unit to contradict this architecture specification. See the documentation of the specific implementation for information about the **IMPLEMENTATION DEFINED** support for this register.

MRS <Xt>, TRCAUXCTLR

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCAUXCTLR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCAUXCTLR;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCAUXCTLR;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCAUXCTLR;

```

MSR TRCAUXCTLR, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.TRCAUXCTLR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCAUXCTLR = X[t];
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else

```

```
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        TRCAUXCTLR = X[t];
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        TRCAUXCTLR = X[t];
```

A.11.9 TRCIDR0, ID Register 0

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR0 bits [31:0] are architecturally mapped to External System register [B.6.9 TRCIDR0, ID Register 0](#) on page 771 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x010	1000	1xxx	xxx0	00xx	111x	1010	000x
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-152: AArch64_trcidr0 bit assignments

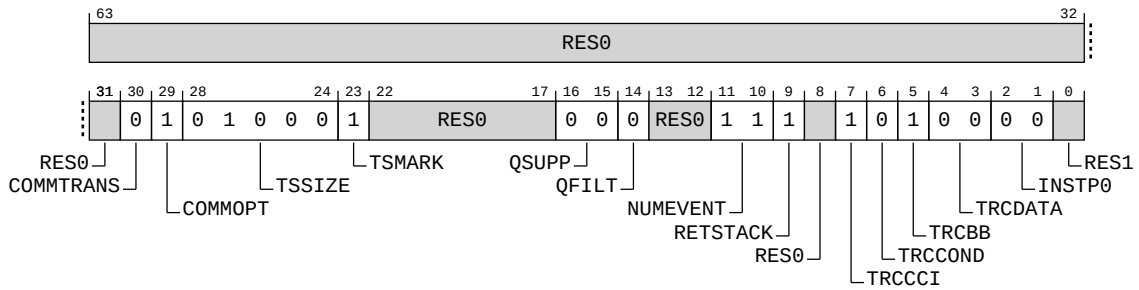


Table A-356: TRCIDR0 bit descriptions

Bits	Name	Description	Reset
[63:31]	RES0	Reserved	RES0
[30]	COMMTRANS	Transaction Start element behavior. 0b0 Transaction Start elements are P0 elements.	0b0
[29]	COMMOPT	Indicates the contents and encodings of Cycle count packets. 0b1 Commit mode 1.	0b1
[28:24]	TSSIZE	Indicates that the trace unit implements Global timestamping and the size of the timestamp value. 0b01000 Global timestamping implemented with a 64-bit timestamp value.	0b01000
[23]	TSMARK	Indicates whether Timestamp Marker elements are generated. 0b1 Timestamp Marker elements are generated.	0b1
[22:17]	RES0	Reserved	RES0
[16:15]	QSUPP	Indicates that the trace unit implements Q element support. 0b00 Q element support is not implemented.	0b00
[14]	QFILT	Indicates if the trace unit implements Q element filtering. 0b0 Q element filtering is not implemented.	0b0
[13:12]	RES0	Reserved	RES0
[11:10]	NUMEVENT	Indicates the number of ETEEvents implemented. 0b11 The trace unit supports 4 ETEEvents.	0b11
[9]	RETSTACK	Indicates if the trace unit supports the return stack. 0b1 Return stack implemented.	0b1
[8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7]	TRCCCI	Indicates if the trace unit implements cycle counting. 0b1 Cycle counting implemented.	0b1
[6]	TRCCOND	Indicates if the trace unit implements conditional instruction tracing. Conditional instruction tracing is not implemented in ETE and this field is reserved for other trace architectures. 0b0 Conditional instruction tracing not implemented.	0b0
[5]	TRCBB	Indicates if the trace unit implements branch broadcasting. 0b1 Branch broadcasting implemented.	0b1
[4:3]	TRCDATA	Indicates if the trace unit implements data tracing. Data tracing is not implemented in ETE and this field is reserved for other trace architectures. 0b00 Tracing of data addresses and data values is not implemented.	0b00
[2:1]	INSTPO	Indicates if load and store instructions are PO instructions. Load and store instructions as PO instructions is not implemented in ETE and this field is reserved for other trace architectures. 0b00 Load and store instructions are not PO instructions.	0b00
[0]	RES1	Reserved	RES1

Access

MRS <Xt>, TRCIDRO

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1000	0b111

Accessibility

MRS <Xt>, TRCIDRO

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end
    else
        return TRCIDRO;
    end
elsif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    end
end

```

```
    elsif CPTR_EL3.TTA == '1' then
      if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
      else
        AArch64.SystemAccessTrap(EL3, 0x18);
      else
        return TRCIDR0;
    elsif PSTATE.EL == EL3 then
      if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
      else
        return TRCIDR0;
```

A.11.10 TRCIDR1, ID Register 1

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR1 bits [31:0] are architecturally mapped to External System register [B.6.10 TRCIDR1, ID Register 1](#) on page 773 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0100	0001	xxxx	xxxx	xxxx	1111	1111	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-153: AArch64_trcidr1 bit assignments

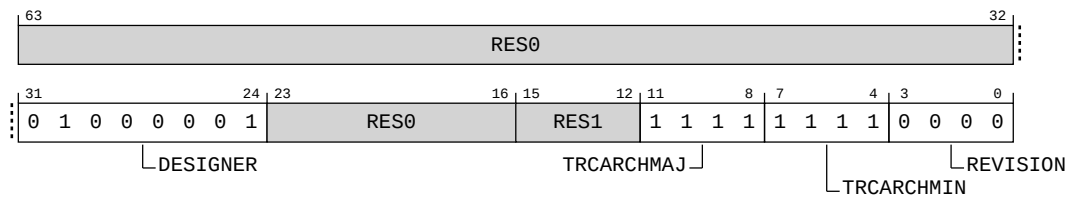


Table A-358: TRCIDR1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:24]	DESIGNER	Indicates which company designed the trace unit. The permitted values of this field are the same as AArch64-MIDR_EL1.Implementer. 0b01000001 Arm Limited	0x41
[23:16]	RES0	Reserved	RES0
[15:12]	RES1	Reserved	RES1
[11:8]	TRCARCHMAJ	Major architecture version. 0b1111 If both TRCARCHMAJ and TRCARCHMIN == 0xF then refer to AArch64-TRCDEVARCH.	0b1111
[7:4]	TRCARCHMIN	Minor architecture version. 0b1111 If both TRCARCHMAJ and TRCARCHMIN == 0xF then refer to AArch64-TRCDEVARCH.	0b1111
[3:0]	REVISION	Indicates the major revision of the product 0b0000 rOp1	0b0000

Access

MRS <Xt>, TRCIDR1

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1001	0b111

Accessibility

MRS <Xt>, TRCIDR1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
```

```

elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif CPTR_EL3.TTA == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCIDR1;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR1;
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCIDR1;

```

A.11.11 TRCIDR2, ID Register 2

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR2 bits [31:0] are architecturally mapped to External System register [B.6.11 TRCIDR2, ID Register 2](#) on page 775 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1100	000x	xxxx	xxxx	x001	0000	1000	1000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-154: AArch64_trcidr2 bit assignments

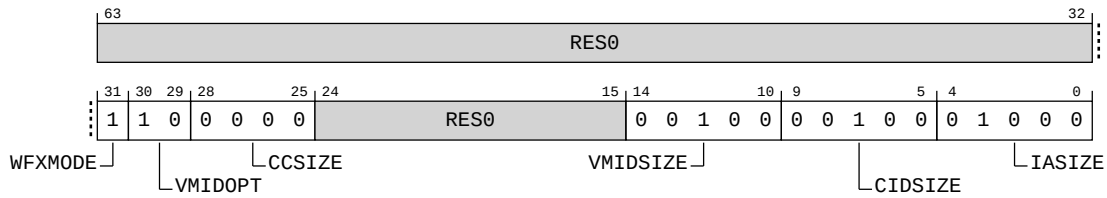


Table A-360: TRCIDR2 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31]	WFXMODE	Indicates whether WFI and WFE instructions are classified as PO instructions: 0b1 WFI and WFE instructions are classified as PO instructions.	0b1
[30:29]	VMIDOPT	Indicates the options for Virtual context identifier selection. 0b10 Virtual context identifier selection not supported. AArch64-TRCCONFIGR.VMIDOPT is RES1 .	0b10
[28:25]	CCSIZE	Indicates the size of the cycle counter. 0b0000 The cycle counter is 12 bits in length.	0b0000
[24:15]	RES0	Reserved	RES0
[14:10]	VMIDSIZE	Indicates the trace unit Virtual context identifier size. 0b00100 32-bit Virtual context identifier size.	0b00100
[9:5]	CIDSIZE	Indicates the Context identifier size. 0b00100 32-bit Context identifier size.	0b00100
[4:0]	IASIZE	Virtual instruction address size. 0b01000 Maximum of 64-bit instruction address size.	0b01000

Access

MRS <Xt>, TRCIDR2

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1010	0b111

Accessibility

MRS <Xt>, TRCIDR2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
```

```

elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR2;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR2;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR2;

```

A.11.12 TRCIDR3, ID Register 3

Returns the base architecture of the trace unit.

Configurations

AArch64 register TRCIDR3 bits [31:0] are architecturally mapped to External System register [B.6.12 TRCIDR3, ID Register 3](#) on page 776 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

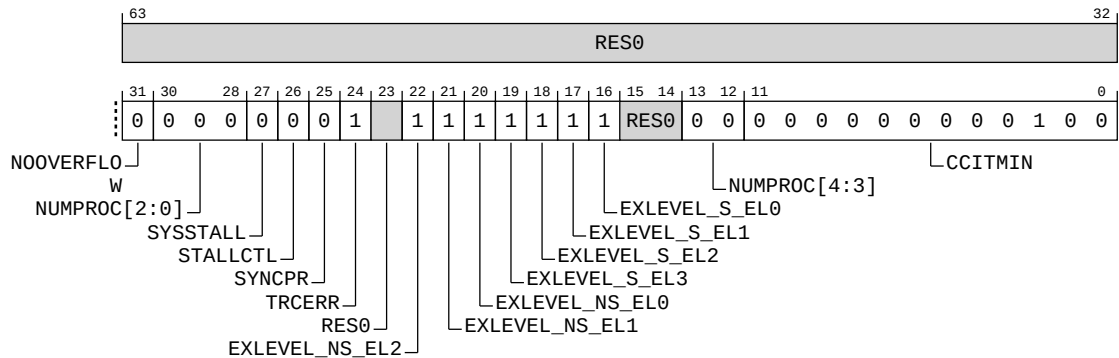
Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0001	x111	1111	xx00	0000	0000	0100
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-155: AArch64_trcidr3 bit assignments**Table A-362: TRCIDR3 bit descriptions**

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31]	NOOVERFLOW	Indicates if overflow prevention is implemented. 0b0 Overflow prevention is not implemented.	0b0
[27]	SYSSTALL	Indicates if stalling of the PE is permitted. 0b0 Stalling of the PE is not permitted.	0b0
[26]	STALLCTL	Indicates if trace unit implements stalling of the PE. 0b0 Stalling of the PE is not implemented.	0b0
[25]	SYNCPR	Indicates if an implementation has a fixed synchronization period. 0b0 AArch64-TRCSYNCPR is read/write so software can change the synchronization period.	0b0
[24]	TRCERR	Indicates forced tracing of System Error exceptions is implemented. 0b1 Forced tracing of System Error exceptions is implemented.	0b1
[23]	RES0	Reserved	RES0
[22]	EXLEVEL_NS_EL2	Indicates if Non-secure EL2 is implemented. 0b1 Non-secure EL2 is implemented.	0b1

Bits	Name	Description	Reset
[21]	EXLEVEL_NS_EL1	Indicates if Non-secure EL1 is implemented. 0b1 Non-secure EL1 is implemented.	0b1
[20]	EXLEVEL_NS_ELO	Indicates if Non-secure ELO is implemented. 0b1 Non-secure ELO is implemented.	0b1
[19]	EXLEVEL_S_EL3	Indicates if EL3 is implemented. 0b1 EL3 is implemented.	0b1
[18]	EXLEVEL_S_EL2	Indicates if Secure EL2 is implemented. 0b1 Secure EL2 is implemented.	0b1
[17]	EXLEVEL_S_EL1	Indicates if Secure EL1 is implemented. 0b1 Secure EL1 is implemented.	0b1
[16]	EXLEVEL_S_ELO	Indicates if Secure ELO is implemented. 0b1 Secure ELO is implemented.	0b1
[15:14]	RES0	Reserved	RES0
[13:12, 30:28]	NUMPROC	Indicates the number of PEs available for tracing. 0b00000 The trace unit can trace one PE.	0b00000
[11:0]	CCITMIN	Indicates the minimum value that can be programmed in AArch64-TRCCCCTLR.THRESHOLD. If AArch64-TRCIDR0.TRCCCI == 1 then the minimum value of this field is 0x001. If AArch64-TRCIDR0.TRCCCI == 0 then this field is zero. 0b000000000100	0x004

Access

MRS <Xt>, TRCIDR3

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1011	0b111

Accessibility

MRS <Xt>, TRCIDR3

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then

```



```

    AArch64.SystemAccessTrap(EL1, 0x18);
elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif CPTR_EL3.TTA == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCIDR3;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR3;
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCIDR3;

```

A.11.13 TRCIDR4, ID Register 4

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR4 bits [31:0] are architecturally mapped to External System register [B.6.13 TRCIDR4, ID Register 4](#) on page 779 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

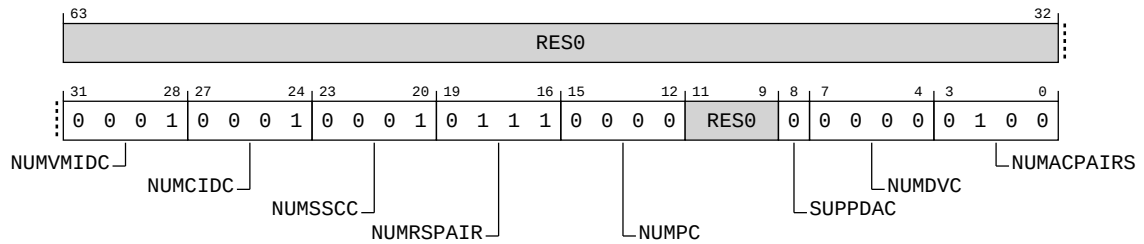
Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0001	0001	0111	0000	xxx0	0000	0100
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-156: AArch64_trcidr4 bit assignments**Table A-364: TRCIDR4 bit descriptions**

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:28]	NUMVMIDC	Indicates the number of Virtual Context Identifier Comparators that are available for tracing. 0b0001 The implementation has one Virtual Context Identifier Comparator.	0b0001
[27:24]	NUMCIDC	Indicates the number of Context Identifier Comparators that are available for tracing. 0b0001 The implementation has one Context Identifier Comparator.	0b0001
[23:20]	NUMSSCC	Indicates the number of Single-shot Comparator Controls that are available for tracing. 0b0001 The implementation has one Single-shot Comparator Control.	0b0001
[19:16]	NUMRSPAIR	Indicates the number of resource selector pairs that are available for tracing. 0b0111 The implementation has eight resource selector pairs.	0b0111
[15:12]	NUMPC	Indicates the number of PE Comparator Inputs that are available for tracing. 0b0000 No PE Comparator Inputs are available.	0b0000
[11:9]	RES0	Reserved	RES0
[8]	SUPPDAC	Indicates whether data address comparisons are implemented. Data address comparisons are not implemented in ETE and are reserved for other trace architectures. 0b0 Data address comparisons not implemented.	0b0

Bits	Name	Description	Reset
[7:4]	NUMDVC	Indicates the number of data value comparators. Data value comparators are not implemented in ETE and are reserved for other trace architectures. Allocated in other trace architectures. 0b0000 No data value comparators implemented.	0b0000
[3:0]	NUMACPAIRS	Indicates the number of Address Comparator pairs that are available for tracing. 0b0100 The implementation has four Address Comparator pairs.	0b0100

Access

MRS <Xt>, TRCIDR4

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1100	0b111

Accessibility

MRS <Xt>, TRCIDR4

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR4;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR4;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR4;

```

A.11.14 TRCIDR5, ID Register 5

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR5 bits [31:0] are architecturally mapped to External System register [B.6.14 TRCIDR5, ID Register 5](#) on page 781 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x010	100x	0100	0111	xxxx	1001	1111	1111
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-157: AArch64_trcidr5 bit assignments

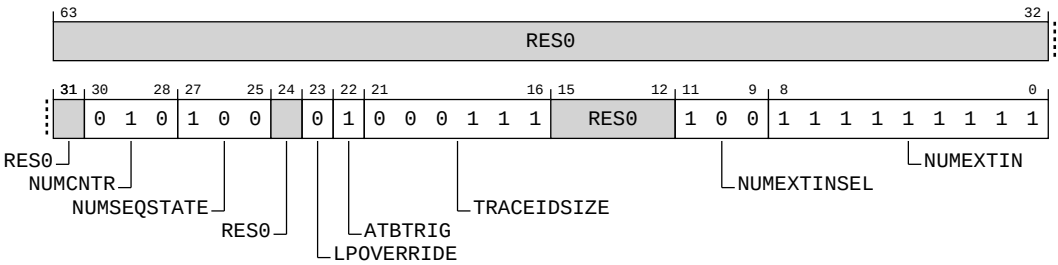


Table A-366: TRCIDR5 bit descriptions

Bits	Name	Description	Reset
[63:31]	RES0	Reserved	RES0
[30:28]	NUMCNTR	Indicates the number of Counters that are available for tracing. 0b010 Two Counters implemented.	0b010

Bits	Name	Description	Reset
[27:25]	NUMSEQSTATE	Indicates if the Sequencer is implemented and the number of Sequencer states that are implemented. 0b100 Four Sequencer states are implemented.	0b100
[24]	RES0	Reserved	RES0
[23]	LPOVERRIDE	Indicates support for Low-power Override Mode. 0b0 The trace unit does not support Low-power Override Mode.	0b0
[22]	ATBTRIG	Indicates if the implementation can support ATB triggers. 0b1 The implementation supports ATB triggers.	0b1
[21:16]	TRACEIDSIZE	Indicates the trace ID width. 0b000111 The implementation supports a 7-bit trace ID.	0b000111
[15:12]	RES0	Reserved	RES0
[11:9]	NUMEXTINSEL	Indicates how many External Input Selector resources are implemented. 0b100 4 External Input Selector resources are available.	0b100
[8:0]	NUMEXTIN	Indicates how many External Inputs are implemented. 0b11111111 Unified PMU event selection.	0b11111111

Access

MRS <Xt>, TRCIDR5

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1101	0b111

Accessibility

MRS <Xt>, TRCIDR5

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR5;

```

```
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR5;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCIDR5;
```

A.11.15 TRCIDR6, ID Register 6

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR6 bits [31:0] are architecturally mapped to External System register [B.6.15 TRCIDR6, ID Register 6](#) on page 783 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-158: AArch64_trcidr6 bit assignments

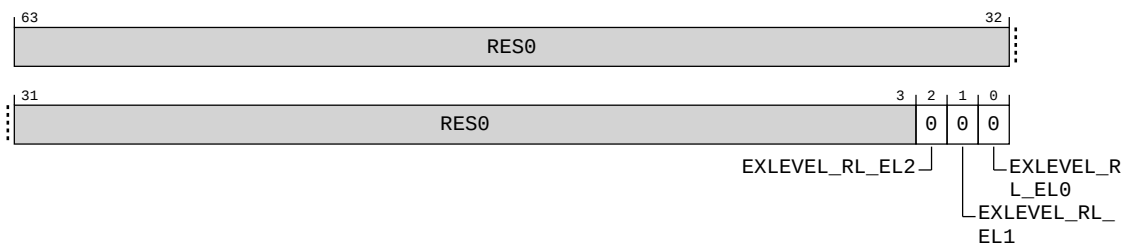


Table A-368: TRCIDR6 bit descriptions

Bits	Name	Description	Reset
[63:3]	RES0	Reserved	RES0
[2]	EXLEVEL_RL_EL2	Indicates if Realm EL2 is implemented. 0b0 Realm EL2 is not implemented.	0b0
[1]	EXLEVEL_RL_EL1	Indicates if Realm EL1 is implemented. 0b0 Realm EL1 is not implemented.	0b0
[0]	EXLEVEL_RL_EL0	Indicates if Realm EL0 is implemented. 0b0 Realm EL0 is not implemented.	0b0

Access

MRS <Xt>, TRCIDR6

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1110	0b111

Accessibility

MRS <Xt>, TRCIDR6

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
```

```

    else
        return TRCIDR6;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elsif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            end
        else
            return TRCIDR6;
        elsif PSTATE.EL == EL3 then
            if CPTR_EL3.TTA == '1' then
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCIDR6;
            end
        end
    end
end

```

A.11.16 TRCIDR7, ID Register 7

Returns the tracing capabilities of the trace unit.

Configurations

AArch64 register TRCIDR7 bits [31:0] are architecturally mapped to External System register [B.6.16 TRCIDR7, ID Register 7](#) on page 784 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
0															



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-159: AArch64_trcidr7 bit assignments

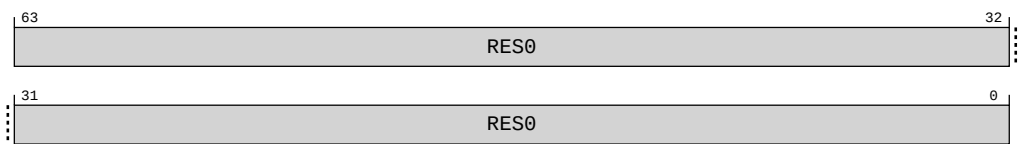


Table A-370: TRCIDR7 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCIDR7

op0	op1	CRn	CRm	op2
0b10	0b001	0b0000	0b1111	0b111

Accessibility

MRS <Xt>, TRCIDR7

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end
    else
        return TRCIDR7;
    end
elsif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end
    else
        return TRCIDR7;
    end
elsif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
```

```
return TRCIDR7;
```

A.11.17 TRCDEVID, Device Configuration Register

Provides discovery information for the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

AArch64 register TRCDEVID bits [31:0] are architecturally mapped to External System register [B.6.23 TRCDEVID, Device Configuration Register](#) on page 795 bits [31:0].

Attributes

Width

64

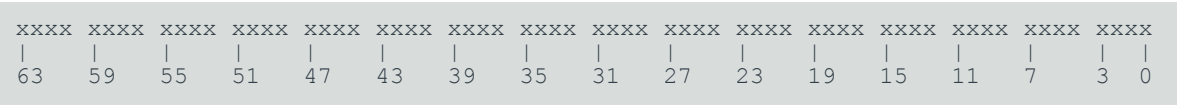
Functional group

Trace unit registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-160: AArch64_trcdevid bit assignments

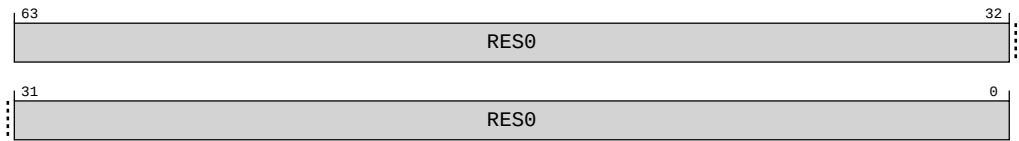


Table A-372: TRCDEVID bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

MRS <Xt>, TRCDEVID

op0	op1	CRn	CRm	op2
0b10	0b001	0b0111	0b0010	0b111

Accessibility

MRS <Xt>, TRCDEVID

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCDEVID;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCDEVID;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCDEVID;

```

A.11.18 TRCCLAIMSET, Claim Tag Set Register

In conjunction with AArch64-TRCCLAIMCLR, provides Claim Tag bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent.

For additional information, see the CoreSight Architecture Specification.

Configurations

The number of claim tag bits implemented is four, that is, SET[3:0] reads as 0b1111.

AArch64 register TRCCLAIMSET bits [31:0] are architecturally mapped to External System register [B.6.18 TRCCLAIMSET, Claim Tag Set Register](#) on page 787 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

RAOW1S

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	0000	0000	0000	1111
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-161: AArch64_trclaimset bit assignments

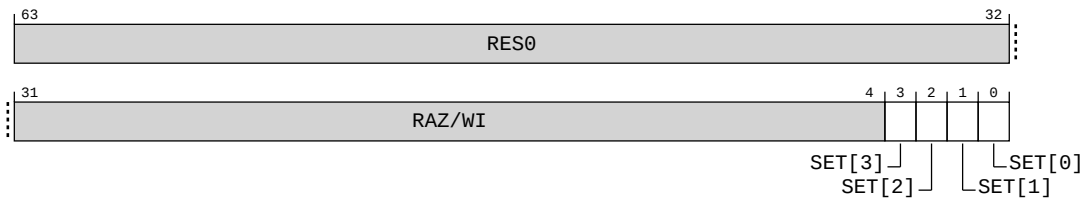


Table A-374: TRCCLAIMSET bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:4]	RAZ/WI	Reserved	RAZ/WI
[3]	SET[3]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1

Bits	Name	Description	Reset
[2]	SET[2]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1
[1]	SET[1]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1
[0]	SET[0]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1

Access

MRS <Xt>, TRCCLAIMSET

op0	op1	CRn	CRm	op2
0b10	0b001	0b0111	0b1000	0b110

MSR TRCCLAIMSET, <Xt>

op0	op1	CRn	CRm	op2
0b10	0b001	0b0111	0b1000	0b110

Accessibility

MRS <Xt>, TRCCLAIMSET

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    
```

```

elseif CPACR_EL1.TTA == '1' then
    AArch64.SystemAccessTrap(EL1, 0x18);
elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCLAIM == '1' then
    AArch64.SystemAccessTrap(EL2, 0x18);
elseif CPTR_EL3.TTA == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCLAIMSET;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCLAIMSET;
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return TRCLAIMSET;

```

MSR TRCLAIMSET, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.TRCLAIM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCLAIMSET = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCLAIMSET = X[t];
elseif PSTATE.EL == EL3 then
    if CPTR_EL3.TTA == '1' then
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        TRCLAIMSET = X[t];

```

A.11.19 TRCCLAIMCLR, Claim Tag Clear Register

In conjunction with AArch64-TRCCLAIMSET, provides Claim Tag bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent.

For additional information, see the CoreSight Architecture Specification.

Configurations

AArch64 register TRCCLAIMCLR bits [31:0] are architecturally mapped to External System register B.6.19 TRCCLAIMCLR, Claim Tag Clear Register on page 789 bits [31:0].

Attributes

Width

64

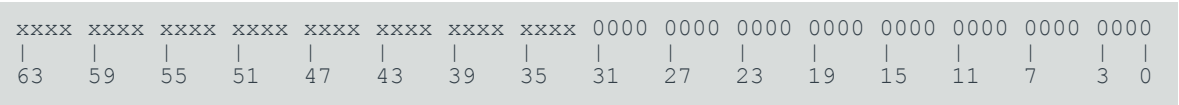
Functional group

Trace unit registers

Access type

RW1C

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-162: AArch64_trcclaimclr bit assignments

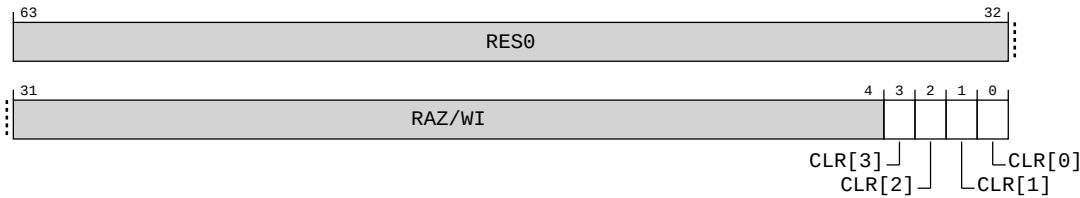


Table A-377: TRCCLAIMCLR bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:4]	RAZ/WI	Reserved	RAZ/WI

Bits	Name	Description	Reset
[3]	CLR[3]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0
[2]	CLR[2]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0
[1]	CLR[1]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0
[0]	CLR[0]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0

Access

MRS <Xt>, TRCCLAIMCLR

op0	op1	CRn	CRm	op2
0b10	0b001	0b0111	0b1001	0b110

MSR TRCCLAIMCLR, <Xt>

op0	op1	CRn	CRm	op2
0b10	0b001	0b0111	0b1001	0b110

Accessibility

MRS <Xt>, TRCCLAIMCLR

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCCLAIM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCCLAIMCLR;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elsif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elsif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCCLAIMCLR;
    elsif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCCLAIMCLR;

```

MSR TRCCLAIMCLR, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elsif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elsif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.TRCCLAIM == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCCLAIMCLR = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then

```

```
        UNDEFINED;
    elsif CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCLAIMCLR = X[t];
    elsif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            TRCLAIMCLR = X[t];
```

A.11.20 TRCDEVARCH, Device Architecture Register

Provides discovery information for the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

AArch64 register TRCDEVARCH bits [31:0] are architecturally mapped to External System register [B.6.20 TRCDEVARCH, Device Architecture Register](#) on page 791 bits [31:0].

Attributes

Width

64

Functional group

Trace unit registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-163: AArch64_trcdevarch bit assignments

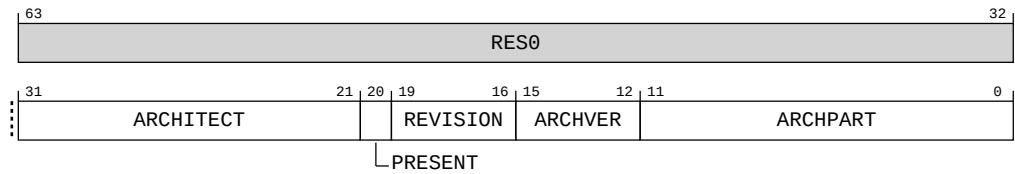


Table A-380: TRCDEVARCH bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:21]	ARCHITECT	Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code. 0b01000111011 JEP106 continuation code 0x4, ID code 0x3B. Arm Limited. Other values are defined by the JEDEC JEP106 standard. This field reads as 0x23B.	11 {x}
[20]	PRESENT	DEVARCH Present. Defines that the DEVARCH register is present. 0b1 Device Architecture information present.	x
[19:16]	REVISION	Revision. Defines the architecture revision of the component. 0b0000 ETEv1.0, FEAT_ETE. 0b0001 ETEv1.1, FEAT_ETEv1p1. 0b0010 ETEv1.2, FEAT_ETEv1p2.	xxxx
[15:12]	ARCHVER	Architecture Version. Defines the architecture version of the component. 0b0101 ETEv1. ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12]. This field reads as 0x5.	xxxx
[11:0]	ARCHPART	Architecture Part. Defines the architecture of the component. 0b101000010011 Arm PE trace architecture. ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0]. This field reads as 0xA13.	12 {x}

Access

MRS <Xt>, TRCDEVARCH

op0	op1	CRn	CRm	op2
0b10	0b001	0b0111	0b1111	0b110

Accessibility

MRS <Xt>, TRCDEVARCH

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
        UNDEFINED;
    elseif CPACR_EL1.TTA == '1' then
        AArch64.SystemAccessTrap(EL1, 0x18);
    elseif EL2Enabled() && CPTR_EL2.TTA == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRCID == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif CPTR_EL3.TTA == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCDEVARCH;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && CPTR_EL3.TTA == '1' then
            UNDEFINED;
        elseif CPTR_EL2.TTA == '1' then
            AArch64.SystemAccessTrap(EL2, 0x18);
        elseif CPTR_EL3.TTA == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return TRCDEVARCH;
    elseif PSTATE.EL == EL3 then
        if CPTR_EL3.TTA == '1' then
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return TRCDEVARCH;

```

A.12 AArch64 Memory Partitioning and Monitoring registers summary

The summary table provides an overview of all Memory Partitioning and Monitoring registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-382: Memory Partitioning and Monitoring registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
MPAM1_EL1	3	0	C10	C5	0	See individual bit resets.	64-bit	MPAM1 Register (EL1)
MPAM0_EL1	3	0	C10	C5	1	See individual bit resets.	64-bit	MPAM0 Register (EL1)
MPAMHCR_EL2	3	4	C10	C4	0	See individual bit resets.	64-bit	MPAM Hypervisor Control Register (EL2)
MPAMVPMV_EL2	3	4	C10	C4	1	See individual bit resets.	64-bit	MPAM Virtual Partition Mapping Valid Register
MPAM2_EL2	3	4	C10	C5	0	See individual bit resets.	64-bit	MPAM2 Register (EL2)
MPAMVPM0_EL2	3	4	C10	C6	0	See individual bit resets.	64-bit	MPAM Virtual PARTID Mapping Register 0
MPAMVPM1_EL2	3	4	C10	C6	1	See individual bit resets.	64-bit	MPAM Virtual PARTID Mapping Register 1
MPAM3_EL3	3	6	C10	C5	0	See individual bit resets.	64-bit	MPAM3 Register (EL3)

A.12.1 MPAMVPMV_EL2, MPAM Virtual Partition Mapping Valid Register

Valid bits for virtual PARTID mapping entries. Each bit m corresponds to virtual PARTID mapping entry m in the MPAMVPM<n>_EL2 registers where n = m >> 2.

Configurations

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

Functional group

Memory Partitioning and Monitoring registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-164: AArch64_mpmavpmv_el2 bit assignments

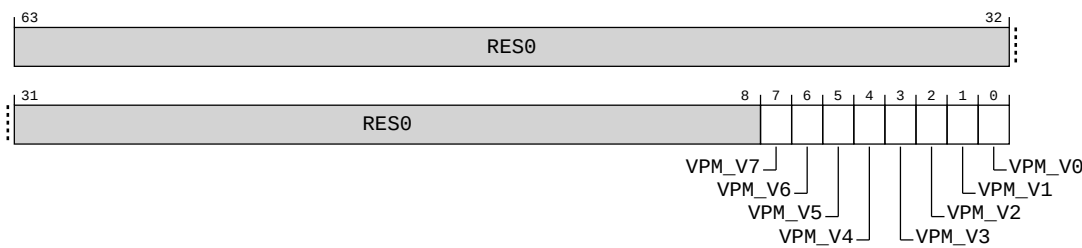


Table A-383: MPAMVPMV_EL2 bit descriptions

Bits	Name	Description	Reset
[63:8]	RES0	Reserved	RES0
[7]	VPM_V7	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[6]	VPM_V6	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[5]	VPM_V5	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[4]	VPM_V4	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[3]	VPM_V3	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[2]	VPM_V2	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[1]	VPM_V1	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x
[0]	VPM_V0	Contains valid bit for virtual PARTID mapping entry corresponding to virtual PARTID<m>.	x

Access

MRS <Xt>, MPAMVPMV_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0100	0b001

MSR MPAMVPMV_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0100	0b001

Accessibility

MRS <Xt>, MPAMVPMV_EL2

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    end if
end if
```

```

else
    return MPAMVPMV_EL2;
elseif PSTATE.EL == EL3 then
    return MPAMVPMV_EL2;

```

MSR MPAMVPMV_EL2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        MPAMVPMV_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    MPAMVPMV_EL2 = X[t];

```

A.12.2 MPAMVPMO_EL2, MPAM Virtual PARTID Mapping Register 0

MPAMVPMO_EL2 provides mappings from virtual PARTIDs 0 - 3 to physical PARTIDs.

AArch64-MPAMIDR_EL1.VPMR_MAX field gives the index of the highest implemented MPAMVPM<n>_EL2 register. VPMR_MAX can be as large as 7 (8 registers) or 32 virtual PARTIDs. If AArch64-MPAMIDR_EL1.VPMR_MAX == 0, there is only a single MPAMVPM<n>_EL2 register, AArch64-MPAMVPMO_EL2.

Virtual PARTID mapping is enabled by AArch64-MPAMHCR_EL2.EL1_VPMEN for PARTIDs in AArch64-MPAM1_EL1 and by AArch64-MPAMHCR_EL2.EL0_VPMEN for PARTIDs in AArch64-MPAMO_EL1.

A virtual-to-physical PARTID mapping entry, PhyPARTID<n>, is valid only when the AArch64-MPAMVPMV_EL2.VPM_V bit in bit position n is set to 1.

Configurations

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

Functional group

Memory Partitioning and Monitoring registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-165: AArch64_mpamvpm0_el2 bit assignments

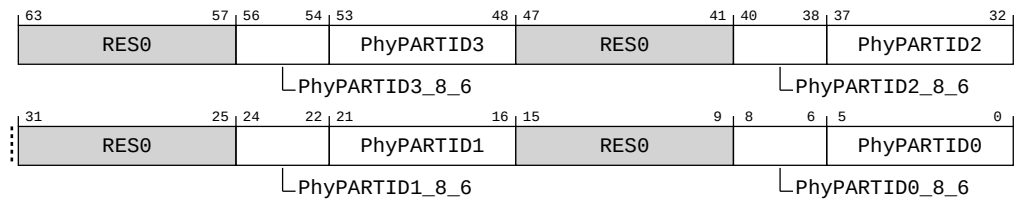


Table A-386: MPAMVPM0_EL2 bit descriptions

Bits	Name	Description	Reset
[63:57]	RES0	Reserved	RES0
[56:54]	PhyPARTID3_8_6	Virtual PARTID Mapping Entry for virtual PARTID 3. PhyPARTID3 gives the mapping of virtual PARTID 3 to a physical PARTID.	xxx
[53:48]	PhyPARTID3	Virtual PARTID Mapping Entry for virtual PARTID 3. PhyPARTID3 gives the mapping of virtual PARTID 3 to a physical PARTID.	6 {x}
[47:41]	RES0	Reserved	RES0
[40:38]	PhyPARTID2_8_6	Virtual PARTID Mapping Entry for virtual PARTID 2. PhyPARTID2 gives the mapping of virtual PARTID 2 to a physical PARTID.	xxx
[37:32]	PhyPARTID2	Virtual PARTID Mapping Entry for virtual PARTID 2. PhyPARTID2 gives the mapping of virtual PARTID 2 to a physical PARTID.	6 {x}
[31:25]	RES0	Reserved	RES0
[24:22]	PhyPARTID1_8_6	Virtual PARTID Mapping Entry for virtual PARTID 1. PhyPARTID1 gives the mapping of virtual PARTID 1 to a physical PARTID.	xxx
[21:16]	PhyPARTID1	Virtual PARTID Mapping Entry for virtual PARTID 1. PhyPARTID1 gives the mapping of virtual PARTID 1 to a physical PARTID.	6 {x}
[15:9]	RES0	Reserved	RES0
[8:6]	PhyPARTID0_8_6	Virtual PARTID Mapping Entry for virtual PARTID 0. PhyPARTID0 gives the mapping of virtual PARTID 0 to a physical PARTID.	xxx
[5:0]	PhyPARTID0	Virtual PARTID Mapping Entry for virtual PARTID 0. PhyPARTID0 gives the mapping of virtual PARTID 0 to a physical PARTID.	6 {x}

Access

MRS <Xt>, MPAMVPM0_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0110	0b000

MSR MPAMVPMO_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0110	0b000

Accessibility

MRS <Xt>, MPAMVPMO_EL2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return MPAMVPMO_EL2;
elseif PSTATE.EL == EL3 then
    return MPAMVPMO_EL2;

```

MSR MPAMVPMO_EL2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        MPAMVPMO_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    MPAMVPMO_EL2 = X[t];

```

A.12.3 MPAMVPM1_EL2, MPAM Virtual PARTID Mapping Register 1

MPAMVPM1_EL2 provides mappings from virtual PARTIDs 4 - 7 to physical PARTIDs.

AArch64-MPAMIDR_EL1.VPMR_MAX field gives the index of the highest implemented AArch64-MPAMVPMO_EL2 to AArch64-MPAMVPM7_EL2 registers. VPMR_MAX can be as large as 7 (8 registers) or 32 virtual PARTIDs. If AArch64-MPAMIDR_EL1.VPMR_MAX == 0, there is only a single MPAMVPM<n>_EL2 register, AArch64-MPAMVPMO_EL2.

Virtual PARTID mapping is enabled by AArch64-MPAMHCR_EL2.EL1_VPMEN for PARTIDs in AArch64-MPAM1_EL1 and by MPAMHCR_EL2.ELO_VPMEN for PARTIDs in AArch64-MPAMO_EL1.

A virtual-to-physical PARTID mapping entry, PhyPARTID<n>, is valid only when the AArch64-MPAMVPMV_EL2.VPM_V bit in bit position n is set to 1.

Configurations

This register has no effect if EL2 is not enabled in the current Security state.

Attributes

Width

64

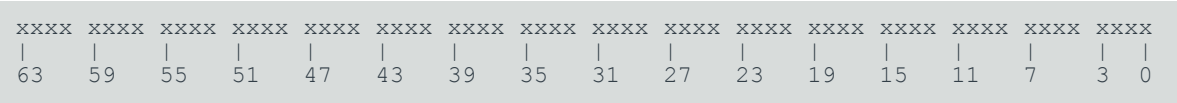
Functional group

Memory Partitioning and Monitoring registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-166: AArch64_mpamvpm1_el2 bit assignments

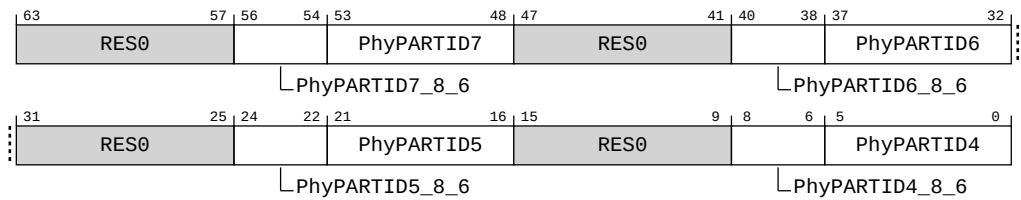


Table A-389: MPAMVPM1_EL2 bit descriptions

Bits	Name	Description	Reset
[63:57]	RES0	Reserved	RES0
[56:54]	PhyPARTID7_8_6	Virtual PARTID Mapping Entry for virtual PARTID 7. PhyPARTID7 gives the mapping of virtual PARTID 7 to a physical PARTID.	xxx

Bits	Name	Description	Reset
[53:48]	PhyPARTID7	Virtual PARTID Mapping Entry for virtual PARTID 7. PhyPARTID7 gives the mapping of virtual PARTID 7 to a physical PARTID.	6 {x}
[47:41]	RES0	Reserved	RES0
[40:38]	PhyPARTID6_8_6	Virtual PARTID Mapping Entry for virtual PARTID 6. PhyPARTID6 gives the mapping of virtual PARTID 6 to a physical PARTID.	xxx
[37:32]	PhyPARTID6	Virtual PARTID Mapping Entry for virtual PARTID 6. PhyPARTID6 gives the mapping of virtual PARTID 6 to a physical PARTID.	6 {x}
[31:25]	RES0	Reserved	RES0
[24:22]	PhyPARTID5_8_6	Virtual PARTID Mapping Entry for virtual PARTID 5. PhyPARTID5 gives the mapping of virtual PARTID 5 to a physical PARTID.	xxx
[21:16]	PhyPARTID5	Virtual PARTID Mapping Entry for virtual PARTID 5. PhyPARTID5 gives the mapping of virtual PARTID 5 to a physical PARTID.	6 {x}
[15:9]	RES0	Reserved	RES0
[8:6]	PhyPARTID4_8_6	Virtual PARTID Mapping Entry for virtual PARTID 4. PhyPARTID4 gives the mapping of virtual PARTID 4 to a physical PARTID.	xxx
[5:0]	PhyPARTID4	Virtual PARTID Mapping Entry for virtual PARTID 4. PhyPARTID4 gives the mapping of virtual PARTID 4 to a physical PARTID.	6 {x}

Access

MRS <Xt>, MPAMVPM1_EL2

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0110	0b001

MSR MPAMVPM1_EL2, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b100	0b1010	0b0110	0b001

Accessibility

MRS <Xt>, MPAMVPM1_EL2

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return MPAMVPM1_EL2;
    elseif PSTATE.EL == EL3 then
        return MPAMVPM1_EL2;

```

MSR MPAMVPM1_EL2, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    UNDEFINED;
elseif PSTATE.EL == EL2 then
    if MPAM3_EL3.TRAPLOWER == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        MPAMVPM1_EL2 = X[t];
elseif PSTATE.EL == EL3 then
    MPAMVPM1_EL2 = X[t];

```

A.13 AArch64 RAS registers summary

The summary table provides an overview of all RAS registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-392: RAS registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
ERRIDR_EL1	3	0	C5	C3	0	See individual bit resets.	64-bit	Error Record ID Register
ERRSELR_EL1	3	0	C5	C3	1	See individual bit resets.	64-bit	Error Record Select Register
ERXFR_EL1	3	0	C5	C4	0	See individual bit resets.	64-bit	Selected Error Record Feature Register
ERXCTLR_EL1	3	0	C5	C4	1	See individual bit resets.	64-bit	Selected Error Record Control Register
ERXSTATUS_EL1	3	0	C5	C4	2	See individual bit resets.	64-bit	Selected Error Record Primary Status Register
ERXADDR_EL1	3	0	C5	C4	3	See individual bit resets.	64-bit	Selected Error Record Address Register
ERXPFGF_EL1	3	0	C5	C4	4	See individual bit resets.	64-bit	Selected Pseudo-fault Generation Feature register
ERXPFGCTL_EL1	3	0	C5	C4	5	See individual bit resets.	64-bit	Selected Pseudo-fault Generation Control register
ERXPFGCDN_EL1	3	0	C5	C4	6	See individual bit resets.	64-bit	Selected Pseudo-fault Generation Countdown register
ERXMISCO_EL1	3	0	C5	C5	0	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 0
ERXMISC1_EL1	3	0	C5	C5	1	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 1
ERXMISC2_EL1	3	0	C5	C5	2	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 2
ERXMISC3_EL1	3	0	C5	C5	3	See individual bit resets.	64-bit	Selected Error Record Miscellaneous Register 3
DISR_EL1	3	0	C12	C1	1	See individual bit resets.	64-bit	Deferred Interrupt Status Register
VSESR_EL2	3	4	C5	C2	3	See individual bit resets.	64-bit	Virtual SError Exception Syndrome Register
VDISR_EL2	3	4	C12	C1	1	See individual bit resets.	64-bit	Virtual Deferred Interrupt Status Register

A.13.1 ERRIDR_EL1, Error Record ID Register

Defines the highest numbered index of the error records that can be accessed through the Error Record System registers.

Configurations

This register is available in all configurations.

Attributes

Width

64

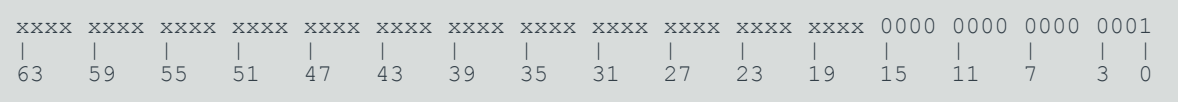
Functional group

RAS registers

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-167: AArch64_erridr_el1 bit assignments

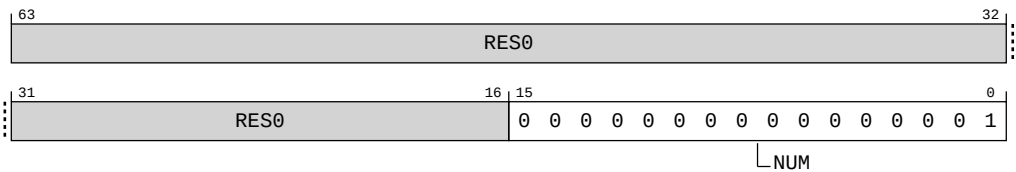


Table A-393: ERRIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:16]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[15:0]	NUM	Highest numbered index of the records that can be accessed through the Error Record System registers plus one. Zero indicates no records can be accessed through the Error Record System registers. Each implemented record is owned by a node. A node might own multiple records. 0b0000000000000001 One Record Present.	0x0001

Access

MRS <Xt>, ERRIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0011	0b000

Accessibility

MRS <Xt>, ERRIDR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERRIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERRIDR_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elsif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERRIDR_EL1;
    elsif PSTATE.EL == EL3 then
        return ERRIDR_EL1;

```

A.13.2 ERRSELR_EL1, Error Record Select Register

Selects an error record to be accessed through the Error Record System registers.

Configurations

This register is available in all configurations.

Attributes

Width

64

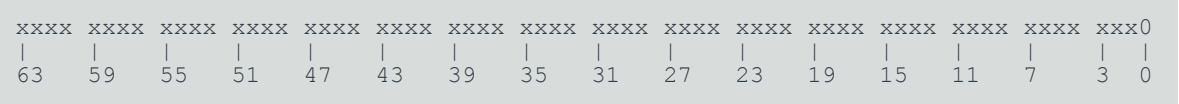
Functional group

RAS registers

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-168: AArch64_errselr_el1 bit assignments

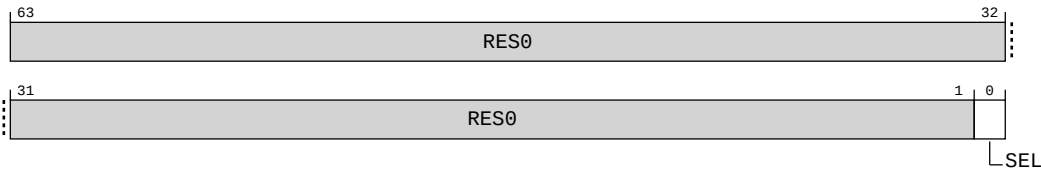


Table A-395: ERRSELR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:1]	RES0	Reserved	RES0
[0]	SEL	0b0 Selects record 0, containing errors from Core RAMs	0b0

Access

MRS <Xt>, ERRSELR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0011	0b001

MSR ERRSELR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0011	0b001

Accessibility

MRS <Xt>, ERRSELR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERRSELR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERRSELR_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elsif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERRSELR_EL1;
    elsif PSTATE.EL == EL3 then
        return ERRSELR_EL1;

```

MSR ERRSELR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERRSELR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERRSELR_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elsif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                ERRSELR_EL1 = X[t];
    elsif PSTATE.EL == EL3 then
        ERRSELR_EL1 = X[t];

```


A.13.3 ERXFR_EL1, Selected Error Record Feature Register

Accesses ext-ERR<n>FR for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	x101	0001	xxxx	xxxx	xxxx	xxxx	1xxx	xx00	0001	0000	1010	1001	1010	xx10
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-169: AArch64_erxfr_el1 bit assignments

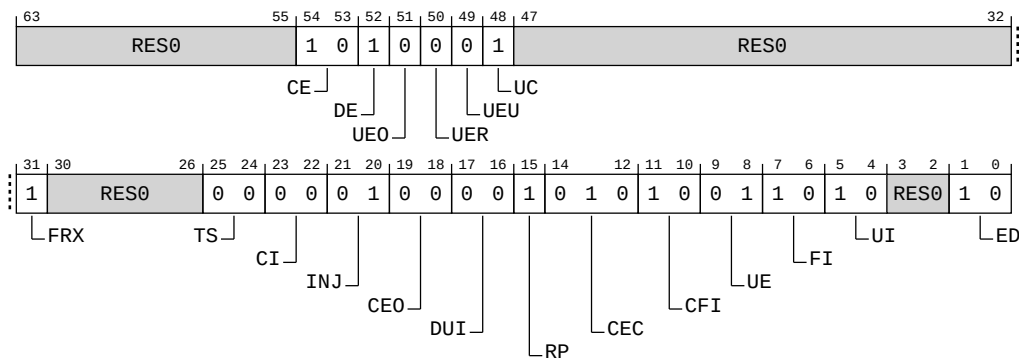


Table A-398: ERXFR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:55]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[54:53]	CE	Corrected Error recording. Describes the types of Corrected errors the node can record, if any. 0b10 Records only non-specific Corrected errors. That is, Corrected errors recorded by setting ext-ERXSTATUS_EL1.CE to 0b10.	0b10
[52]	DE	Deferred Error recording. Describes whether the node supports recording Deferred errors. 0b1 Records Deferred errors.	0b1
[51]	UEO	Latent or Restartable Error recording. Describes whether the node supports recording Latent or Restartable errors. 0b0 Does not record Latent or Restartable errors.	0b0
[50]	UER	Signaled or Recoverable Error recording. Describes whether the node supports recording Signaled or Recoverable errors. 0b0 Does not record Signaled or Recoverable errors.	0b0
[49]	UEU	Unrecoverable Error recording. Describes whether the node supports recording Unrecoverable errors. 0b0 Does not record Unrecoverable errors.	0b0
[48]	UC	Uncontainable Error recording. Describes whether the node supports recording Uncontainable errors. 0b1 Records Uncontainable errors.	0b1
[47:32]	RES0	Reserved	RES0
[31]	FRX	Feature Register extension. Defines whether AArch64-ERXFR_EL1[63:48] are architecturally defined. 0b1 AArch64-ERXFR_EL1[63:48] are defined by the architecture.	0b1
[30:26]	RES0	Reserved	RES0
[25:24]	TS	Timestamp Extension. Indicates whether, for each error record <m> owned by this node, AArch64-ERXMISC3_EL1 is used as the timestamp register, and, if it is, the timebase used by the timestamp. 0b00 Does not support a timestamp register. All other values are reserved.	0b00
[23:22]	CI	Critical error interrupt. Indicates whether the critical error interrupt and associated controls are implemented by the node. 0b00 Does not support the critical error interrupt. AArch64-ERXCTLR_EL1.CI is RES0 . All other values are reserved.	0b00
[21:20]	INJ	Fault Injection Extension. Indicates whether the Common Fault Injection Model Extension is implemented by the node. 0b01 Supports the Common Fault Injection Model Extension. See AArch64-ERXPFGF_EL1 for more information. All other values are reserved.	0b01

Bits	Name	Description	Reset
[19:18]	CEO	Corrected Error overwrite. Indicates the behavior of the node when a second or subsequent Corrected error is recorded and a first Corrected error has previously been recorded by an error record <m> owned by the node. 0b00 Keeps the previous error syndrome. All other values are reserved. The second or subsequent Corrected error is counted by the Corrected error counter, regardless of the value of this field. If counting the error causes unsigned overflow of the counter, then AArch64-ERXSTATUS_EL1.OF is set to 1. This means that, if no other error is subsequently recorded that overwrites the syndrome: - If AArch64-ERXFR_EL1.CEO is 0b00, the error record holds the syndrome for the first recorded Corrected error. - If AArch64-ERXFR_EL1.CEO is 0b01, the error record holds the syndrome for the most recently recorded Corrected error before the counter overflows.	0b00
[17:16]	DUI	Error recovery interrupt for deferred errors control. Indicates whether the enabling and disabling of error recovery interrupts on deferred errors is supported by the node. 0b00 Does not support the enabling and disabling of error recovery interrupts on deferred errors. AArch64-ERXCTLR_EL1.DUI is RESO. All other values are reserved.	0b00
[15]	RP	Repeat counter. Indicates whether the node implements a second Corrected error counter in AArch64-ERXMISCO_EL1 for each error record <m> owned by the node that can record countable errors 0b1 Implements a first (repeat) counter and a second (other) counter in AArch64-ERXMISCO_EL1 for each error record <m> owned by the node that can record countable errors. The repeat counter is the same size as the primary error counter.	0b1
[14:12]	CEC	Corrected Error Counter. Indicates whether the node implements the standard Corrected error counter mechanisms in AArch64-ERXMISCO_EL1 for each error record <m> owned by the node that can record countable errors. 0b010 Implements an 8-bit Corrected error counter in AArch64-ERXMISCO_EL1[39:32] for each error record <m> owned by the node that can record countable errors. All other values are reserved. Note: Implementations might include other error counter models, or might include the standard model and not indicate this in AArch64-ERXFR_EL1.	0b010
[11:10]	CFI	Fault handling interrupt for corrected errors control. Indicates whether the enabling and disabling of fault handling interrupts on corrected errors is supported by the node. 0b10 Enabling and disabling of fault handling interrupts on corrected errors is supported and controllable using AArch64-ERXCTLR_EL1.CFI. All other values are reserved.	0b10

Bits	Name	Description	Reset
[9:8]	UE	In-band error response (External Abort). Indicates whether the in-band error response and associated controls are implemented by the node. 0b01 In-band error response is supported and always enabled. AArch64-ERXCTLR_EL1.UE is RES0 .	0b01
[7:6]	FI	Fault handling interrupt. Indicates whether the fault handling interrupt and associated controls are implemented by the node. 0b10 Fault handling interrupt is supported and controllable using AArch64-ERXCTLR_EL1.FI.	0b10
[5:4]	UI	Error recovery interrupt for uncorrected errors. Indicates whether the error handling interrupt and associated controls are implemented by the node. 0b10 Error handling interrupt is supported and controllable using AArch64-ERXCTLR_EL1.UI.	0b10
[3:2]	RES0	Reserved	RES0
[1:0]	ED	Error reporting and logging. Indicates error record <n> is the first record owned the node, and whether the node implements the controls for enabling and disabling error reporting and logging. 0b10 Error reporting and logging is controllable using AArch64-ERXCTLR_EL1.ED. All other values are reserved.	0b10

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXFR_EL1 is RAZ.
- Direct reads of ERXFR_EL1 are NOPs.
- Direct reads of ERXFR_EL1 are UNDEFINED.

MRS <Xt>, ERXFR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b000

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXFR_EL1 is RAZ.
- Direct reads of ERXFR_EL1 are NOPs.
- Direct reads of ERXFR_EL1 are UNDEFINED.

MRS <Xt>, ERXFR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXFR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXFR_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXFR_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXFR_EL1;

```

A.13.4 ERXCTLR_EL1, Selected Error Record Control Register

Accesses ext-ERR<n>CTLR for the error record <n> selected by AArch64-ERRSEL_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-170: AArch64_erxctlr_el1 bit assignments

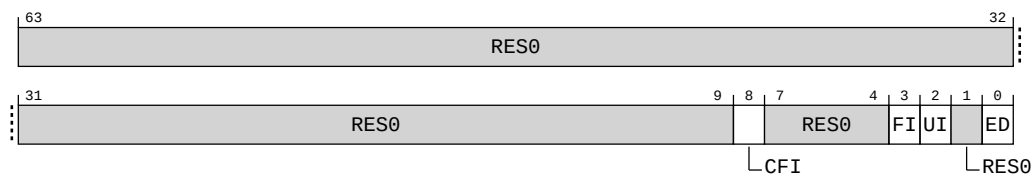


Table A-400: ERXCTLR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:9]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[8]	CFI	When AArch64-ERXFR.CFI == '11' Fault handling interrupt for Corrected errors on reads enable. When AArch64-ERXFR.CFI == 0b11, this field is named RCFI. When enabled: - If the node implements Corrected error counters for reads, then the fault handling interrupt is generated when a counter overflows and the overflow bit for the counter is set to 1. For more information, see AArch64-ERXMISC0. - Otherwise, the fault handling interrupt is also generated for errors recorded as Corrected error on reads. 0b0 Fault handling interrupt not generated for Corrected errors on reads. 0b1 Fault handling interrupt generated for Corrected errors on reads. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. When AArch64-ERXFR.CFI == '10' Fault handling interrupt for Corrected errors enable. When AArch64-ERXFR.CFI == 0b10, this control applies to errors arising from both reads and writes. When enabled: - If the node implements Corrected error counters, then the fault handling interrupt is generated when a counter overflows and the overflow bit for the counter is set to 1. For more information, see AArch64-ERXMISC0. - Otherwise, the fault handling interrupt is also generated for all errors recorded as Corrected error. 0b0 Fault handling interrupt not generated for Corrected errors. 0b1 Fault handling interrupt generated for Corrected errors. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. Otherwise RES0	x
[7:4]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[3]	FI	When AArch64-ERXFR.FI == '11' Fault handling interrupt on reads enable. When AArch64-ERXFR.FI == 0b11, this field is named RFI. When enabled: - The fault handling interrupt is generated for errors recorded as either Deferred error or Uncorrected error on reads. - If the corresponding fault handling interrupt for Corrected errors control is not implemented: - If the node implements Corrected error counters for reads, then the fault handling interrupt is also generated when a counter overflows and the overflow bit for the counter is set to 1. - Otherwise, the fault handling interrupt is also generated for errors recorded as Corrected error on reads. 0b0 Fault handling interrupt on reads disabled. 0b1 Fault handling interrupt on reads enabled. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. When AArch64-ERXFR.FI == '10' Fault handling interrupt enable. When AArch64-ERXFR.FI == 0b10, this control applies to errors arising from both reads and writes. When enabled: - The fault handling interrupt is generated for all errors recorded as either Deferred error or Uncorrected error. - If the fault handling interrupt for Corrected errors control is not implemented: - If the node implements Corrected error counters, then the fault handling interrupt is also generated when a counter overflows and the overflow bit for the counter is set to 1. - Otherwise, the fault handling interrupt is also generated for all errors recorded as Corrected error. 0b0 Fault handling interrupt disabled. 0b1 Fault handling interrupt enabled. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. Otherwise RES0	x

Bits	Name	Description	Reset
[2]	UI	When AArch64-ERXFR.UI == '10' Uncorrected error recovery interrupt enable. When AArch64-ERXFR.UI == 0b10, this control applies to errors arising from both reads and writes. When enabled, the error recovery interrupt is generated for all errors recorded as Uncorrected error. 0b0 Error recovery interrupt disabled. 0b1 Error recovery interrupt enabled. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. When AArch64-ERXFR.UI == '11' Uncorrected error recovery interrupt on reads enable. When AArch64-ERXFR.UI == 0b11, this field is named RUI. When enabled, the error recovery interrupt is generated for errors recorded as Uncorrected error on reads. 0b0 Error recovery interrupt on reads disabled. 0b1 Error recovery interrupt on reads enabled. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error. Otherwise RES0	x
[1]	RES0	Reserved	RES0
[0]	ED	When AArch64-ERXFR.ED == '10' Error reporting and logging enable. When disabled, the node behaves as if error detection and correction are disabled, and no errors are recorded or signaled by the node. 0b0 Error reporting disabled. 0b1 Error reporting enabled. This field is set to 0 on Cold reset Otherwise RES0	x

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXCTLR_EL1 is RAZ/WI.
- Direct reads and writes of ERXCTLR_EL1 are NOPs.
- Direct reads and writes of ERXCTLR_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>CTLR is not present, meaning reads and writes of ERXCTLR_EL1 are **RES0**.

MRS <Xt>, ERXCTLR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b001

MSR ERXCTLR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b001

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXCTLR_EL1 is RAZ/WI.
- Direct reads and writes of ERXCTLR_EL1 are NOPs.
- Direct reads and writes of ERXCTLR_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>CTLR is not present, meaning reads and writes of ERXCTLR_EL1 are RES0.

MRS <Xt>, ERXCTLR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEN == '1' && HFGTR_EL2.ERXCTLR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXCTLR_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXCTLR_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXCTLR_EL1;

```

MSR ERXCTLR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXCTLR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXCTLR_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                ERXCTLR_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        ERXCTLR_EL1 = X[t];

```

A.13.5 ERXSTATUS_EL1, Selected Error Record Primary Status Register

Accesses ext-ERR<n>STATUS for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

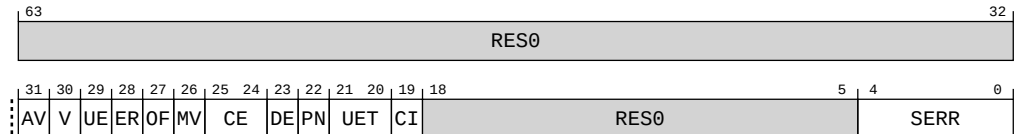
Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	00xx	x0xx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-171: AArch64_erxstatus_el1 bit assignments**Table A-403: ERXSTATUS_EL1 bit descriptions**

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31]	AV	Address Valid. 0b0 ext-ERXADDR not valid. 0b1 ext-ERXADDR contains an address associated with the highest priority error recorded by this record. When AArch64-ERXSTATUS.DE == '0' && AArch64-ERXSTATUS.UE == '0' && AArch64-ERXSTATUS.CE != '00' && ERXSTATUS.CE is not being cleared to 0b00 in the same write Access to this field is: RO When AArch64-ERXSTATUS.UE == '0' && AArch64-ERXSTATUS.DE != '0' && ERXSTATUS.DE is not being cleared to 0b0 in the same write Access to this field is: RO When AArch64-ERXSTATUS.UE != '0' && ERXSTATUS.UE is not being cleared to 0b0 in the same write Access to this field is: RO Otherwise Access to this field is: W1C	0b0
[30]	V	Status Register Valid. 0b0 ERXSTATUS not valid. 0b1 ERXSTATUS valid. At least one error has been recorded. Access to this field is: W1C	0b0

Bits	Name	Description	Reset
[29]	UE	Uncorrected Error. 0b0 No errors have been detected, or all detected errors have been either corrected or deferred. 0b1 At least one detected error was not corrected and not deferred. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x
[28]	ER	Error Reported. 0b0 No in-band error response (External Abort) signaled to the Requester making the access or other transaction. 0b1 An in-band error response was signaled by the component to the Requester making the access or other transaction. This can be because any of the following are true: - The applicable one of the ERR<q>CTLR.{WUE, RUE, UE} fields is implemented and was 1 when an error was detected and not corrected. - The applicable one of the ERR<q>CTLR.{WUE, RUE, UE} fields is not implemented and the component always reports errors. Note: An in-band error response signaled by the component might be masked and not generate any exception. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When AArch64-ERXSTATUS.UE == '0' && AArch64-ERXSTATUS.DE == '0' && this field can be set to 0b1 by a Deferred error Access to this field is: UNKNOWN/WI When AArch64-ERXSTATUS.UE == '0' && this field is never set to 0b1 by a Deferred error Access to this field is: UNKNOWN/WI When AArch64-ERXSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x

Bits	Name	Description	Reset
[27]	OF	Overflow. Indicates that multiple errors have been detected. This field is set to 1 when one of the following occurs: - A Corrected error counter is implemented, an error is counted, and the counter overflows. - ERXSTATUS.V was previously 1, a Corrected error counter is not implemented, and a Corrected error is recorded. - ERXSTATUS.V was previously 1, and a type of error other than a Corrected error is recorded. Otherwise, this field is unchanged when an error is recorded. If a Corrected error counter is implemented: - A direct write that modifies the counter overflow flag indirectly might set this field to an UNKNOWN value. - A direct write to this field that clears this field to zero might indirectly set the counter overflow flag to an UNKNOWN value. 0b0 Since this field was last cleared to zero, no error syndrome has been discarded and, if a Corrected error counter is implemented, it has not overflowed. 0b1 Since this field was last cleared to zero, at least one error syndrome has been discarded or, if a Corrected error counter is implemented, it might have overflowed. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' Access to this field is: UNKNOWN/W1 Otherwise Access to this field is: W1C	x
[26]	MV	Miscellaneous Registers Valid. 0b0 ERXMISC<m> not valid. 0b1 The contents of the ERXMISC<m> registers contains additional information for an error recorded by this record. Note: If the ERXMISC<m> registers can contain additional information for a previously recorded error, then the contents must be self-describing to software or a user. For example, certain fields might relate only to Corrected errors, and other fields only to the most recent error that was not discarded. Access to this field is: W1C	0b0

Bits	Name	Description	Reset
[25:24]	CE	Corrected Error. 0b00 No errors were corrected. 0b10 At least one error was corrected. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write ones to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	xx
[23]	DE	Deferred Error. 0b0 No errors were deferred. 0b1 At least one error was not corrected and deferred. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x
[22]	PN	Poison. 0b0 Uncorrected error or Deferred error recorded because a corrupt value was detected, for example, by an error detection code (EDC), or Corrected error recorded. 0b1 Uncorrected error or Deferred error recorded because a poison value was detected. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' (AArch64-ERXSTATUS.DE == '0' && AArch64-ERXSTATUS.UE == '0') Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x

Bits	Name	Description	Reset
[21:20]	UET	Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error. 0b00 Uncorrected error, Uncontainable error (UC). 0b01 Uncorrected error, Unrecoverable error (UEU). 0b10 Uncorrected error, Latent or Restartable error (UEO). 0b11 Uncorrected error, Signaled or Recoverable error (UER). Note: Software might use the information in the error record registers to determine what recovery is necessary. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write ones to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' AArch64-ERXSTATUS.UE == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	xx
[19]	CI	Critical Error. Indicates whether a critical error condition has been recorded. 0b0 No critical error condition. 0b1 Critical error condition. When clearing ERXSTATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When AArch64-ERXSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x
[18:5]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[4:0]	SERR	Architecturally-defined primary error code. The primary error code might be used by a fault handling agent to triage an error without requiring device-specific code. For example, to count and threshold corrected errors in software, or generate a short log entry. 0b00000 No error. 0b00010 Data value from (non-associative) internal memory. For example, ECC from on-chip SRAM or buffer. 0b00110 Data value from associative memory. For example, ECC error on cache data. 0b00111 Address/control value from associative memory. For example, ECC error on cache tag. 0b01000 Data value from a TLB. For example, ECC error on TLB data. 0b01001 Address/control value from a TLB. For example, ECC error on TLB tag 0b10010 Error response from Completer of access. For example, error response from cache write-back. 0b11000 Deferred error from Requester passed through. For example, poisoned data received from the Requester of an access and deferred to the Completer. All other values are reserved. This field is not valid and reads UNKNOWN if ERXSTATUS.V == 0b0.	5{x}

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXSTATUS_EL1 is RAZ/WI.
- Direct reads and writes of ERXSTATUS_EL1 are NOPs.
- Direct reads and writes of ERXSTATUS_EL1 are UNDEFINED.

ext-ERR<n>STATUS describes additional constraints that also apply when ext-ERR<n>STATUS is accessed through ERXSTATUS_EL1.

MRS <Xt>, ERXSTATUS_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b010

MSR ERXSTATUS_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b010

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXSTATUS_EL1 is RAZ/WI.
- Direct reads and writes of ERXSTATUS_EL1 are NOPs.
- Direct reads and writes of ERXSTATUS_EL1 are UNDEFINED.

ext-ERR<n>STATUS describes additional constraints that also apply when ext-ERR<n>STATUS is accessed through ERXSTATUS_EL1.

MRS <Xt>, ERXSTATUS_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEN == '1' && HFGTR_EL2.ERXSTATUS_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXSTATUS_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXSTATUS_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXSTATUS_EL1;

```

MSR ERXSTATUS_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEN == '1' && HFGWTR_EL2.ERXSTATUS_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);

```

```

elseif SCR_EL3.TERR == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ERXSTATUS_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXSTATUS_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    ERXSTATUS_EL1 = X[t];

```

A.13.6 ERXADDR_EL1, Selected Error Record Address Register

Accesses ext-ERR<n>ADDR for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-172: AArch64_ernaddr_el1 bit assignments

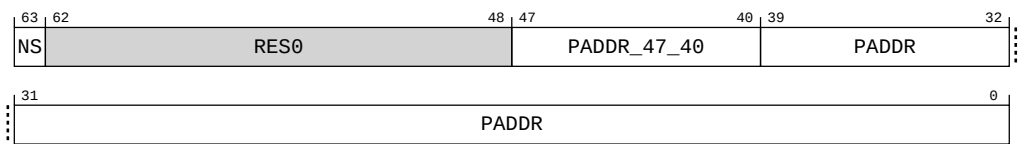


Table A-406: ERXADDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63]	NS	Non-secure attribute. 0b0 The address is Secure. 0b1 The address is Non-secure.	x
[62:48]	RES0	Reserved	RES0
[47:40]	PADDR_47_40	Physical Address. Address of the recorded location.	8 {x}
[39:0]	PADDR	Physical Address. Address of the recorded location.	40 {x}

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXADDR_EL1 is RAZ/WI.
- Direct reads and writes of ERXADDR_EL1 are NOPs.
- Direct reads and writes of ERXADDR_EL1 are UNDEFINED.

ext-ERR<n>ADDR describes additional constraints that also apply when ext-ERR<n>ADDR is accessed through ERXADDR_EL1.

MRS <Xt>, ERXADDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b011

MSR ERXADDR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b011

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXADDR_EL1 is RAZ/WI.
- Direct reads and writes of ERXADDR_EL1 are NOPs.
- Direct reads and writes of ERXADDR_EL1 are UNDEFINED.

ext-ERR<n>ADDR describes additional constraints that also apply when ext-ERR<n>ADDR is accessed through ERXADDR_EL1.

MRS <Xt>, ERXADDR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXADDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXADDR_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elsif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXADDR_EL1;
    elsif PSTATE.EL == EL3 then
        return ERXADDR_EL1;

```

MSR ERXADDR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXADDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXADDR_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elsif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;

```

```

else
    AArch64.SystemAccessTrap(EL3, 0x18);
else
    ERXADDR_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    ERXADDR_EL1 = X[t];

```

A.13.7 ERXPFGF_EL1, Selected Pseudo-fault Generation Feature register

Accesses ext-ERR<n>PFGF for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x111	xxxx	xxxx	xxxx	xxx1	1010	0110	0011
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-173: AArch64_erxpfgf_el1 bit assignments

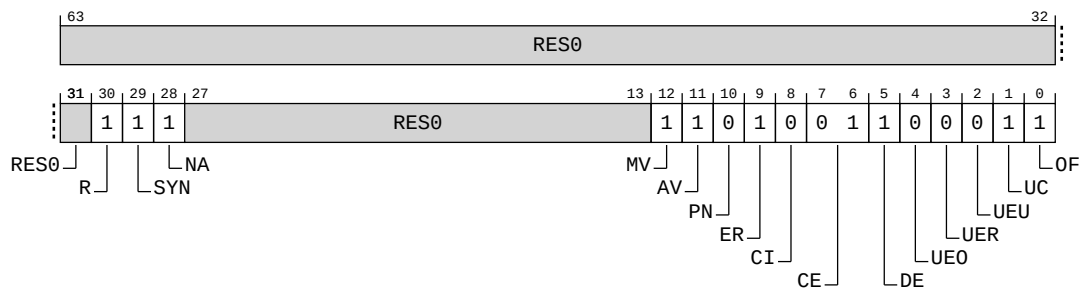


Table A-409: ERXPFGF_EL1 bit descriptions

Bits	Name	Description	Reset
[63:31]	RES0	Reserved	RES0
[30]	R	Restartable. Support for Error Generation Counter restart mode. 0b1 Error Generation Counter restart mode is implemented and is controlled by AARCH64-ERXPFGCTL_EL1.R. AARCH64-ERXPFGCTL_EL1.R is a read/write field.	0b1
[29]	SYN	Syndrome. Fault syndrome injection. 0b1 When an injected error is recorded, the node does not update the AARCH64-ERXSTATUS_EL1.{IERR, SERR} fields. AARCH64-ERXSTATUS_EL1.{IERR, SERR} are writable when AARCH64-ERXSTATUS_EL1.V is 0.	0b1
[28]	NA	No access required. Defines whether this component fakes detection of the error on an access to the component or spontaneously in the fault injection state. 0b1 The component fakes detection of the error spontaneously in the fault injection state.	0b1
[27:13]	RES0	Reserved	RES0
[12]	MV	Miscellaneous syndrome. Additional syndrome injection. Defines whether software can control all or part of the syndrome recorded in the ERR<n>MISC<m> registers when an injected error is recorded. 0b1 When an injected error is recorded, the node does not update AARCH64-ERXMISCO_EL1 and AARCH64-ERXSTATUS_EL1.MV is set to AARCH64-ERXPFGCTL_EL1.MV. AARCH64-ERXPFGCTL_EL1.MV is a read/write field.	0b1
[11]	AV	Address syndrome. Address syndrome injection. 0b1 When an injected error is recorded, the node does not update AARCH64-ERXADDR_EL1 and AARCH64-ERXSTATUS_EL1.AV is set to AARCH64-ERXPFGCTL_EL1.AV. AARCH64-ERXPFGCTL_EL1.AV is a read/write field. Note: If ERR<n>PFGF.AV is 1, software can write a specific address value into AARCH64-ERXADDR_EL1 when setting up a fault injection event.	0b1
[10]	PN	Poison flag. Describes how the fault generation feature of the node sets the AARCH64-ERXSTATUS_EL1.PN status flag. 0b0 When an injected error is recorded the node sets AARCH64-ERXSTATUS_EL1.PN to 0. AARCH64-ERXPFGCTL_EL1.PN is RES0. This behavior replaces the architecture-defined rules for setting the AARCH64-ERXSTATUS_EL1.PN bit.	0b0

Bits	Name	Description	Reset
[9]	ER	Error Reported flag. Describes how the fault generation feature of the node sets the AARCH64-ERXSTATUS_EL1.ER status flag. 0b1 When an injected error is recorded, AARCH64-ERXSTATUS_EL1.ER is set to AARCH64-ERXPFGCTL_EL1.ER. This behavior replaces the architecture-defined rules for setting the ER field. AARCH64-ERXPFGCTL_EL1.ER is a read/write field.	0b1
[8]	CI	Critical Error flag. Describes how the fault generation feature of the node sets the AARCH64-ERXSTATUS_EL1.CI status flag. 0b0 When an injected error is recorded, the node sets AARCH64-ERXSTATUS_EL1.CI to 0. AARCH64-ERXPFGCTL_EL1.CI is RES0. This behavior replaces the architecture-defined rules for setting the AARCH64-ERXSTATUS_EL1.CI bit.	0b0
[7:6]	CE	Corrected Error generation. Describes the types of Corrected error that the fault generation feature of the node can generate. 0b01 The fault generation feature of the node allows generation of a non-specific Corrected error, that is, a Corrected error that is recorded by setting AARCH64-ERXSTATUS_EL1.CE to 0b10. AARCH64-ERXPFGCTL_EL1.CE is a read/write field. The values 0b10 and 0b11 in AARCH64-ERXPFGCTL_EL1.CE are reserved. All other values are reserved. If AARCH64-ERXFR_EL1.FRX is 1, then AARCH64-ERXFR_EL1.CE indicates whether the node supports this type of error.	0b01
[5]	DE	Deferred Error generation. Describes whether the fault generation feature of the node can generate Deferred errors. 0b1 The fault generation feature of the node allows generation of Deferred errors. AARCH64-ERXPFGCTL_EL1.DE is a read/write field. If AARCH64-ERXFR_EL1.FRX is 1, then AARCH64-ERXFR_EL1.DE indicates whether the node supports this type of error.	0b1
[4]	UEO	Latent or Restartable Error generation. Describes whether the fault generation feature of the node can generate Latent or Restartable errors. 0b0 The fault generation feature of the node does not generate Latent or Restartable errors. AARCH64-ERXPFGCTL_EL1.UEO is RES0. If AARCH64-ERXFR_EL1.FRX is 1, then AARCH64-ERXFR_EL1.UEO indicates whether the node supports this type of error.	0b0
[3]	UER	Signaled or Recoverable Error generation. Describes whether the fault generation feature of the node can generate Signaled or Recoverable errors. 0b0 The fault generation feature of the node does not generate Signaled or Recoverable errors. AARCH64-ERXPFGCTL_EL1.UER is RES0. If AARCH64-ERXFR_EL1.FRX is 1, then AARCH64-ERXFR_EL1.UER indicates whether the node supports this type of error.	0b0

Bits	Name	Description	Reset
[2]	UEU	Unrecoverable Error generation. Describes whether the fault generation feature of the node can generate Unrecoverable errors. 0b0 The fault generation feature of the node does not generate Unrecoverable errors. AARCH64-ERXPFGCTL_EL1.UEU is RES0 . If AARCH64-ERXFR.FRX_EL1 is 1, then AARCH64-ERXFR_EL1.UEU indicates whether the node supports this type of error.	0b0
[1]	UC	Uncontainable Error generation. Describes whether the fault generation feature of the node can generate Uncontainable errors. 0b1 The fault generation feature of the node allows generation of Uncontainable errors. AARCH64-ERXPFGCTL_EL1.UC is a read/write field. If AARCH64-ERXFR.FRX_EL1 is 1, then AARCH64-ERXFR_EL1.UC indicates whether the node supports this type of error.	0b1
[0]	OF	Overflow flag. Describes how the fault generation feature of the node sets the AARCH64-ERXSTATUS_EL1.OF status flag. 0b1 When an injected error is recorded, AARCH64-ERXSTATUS_EL1.OF is set to AARCH64-ERXPFGCTL_EL1.OF. This behavior replaces the architecture-defined rules for setting the OF field. AARCH64-ERXPFGCTL_EL1.OF is a read/write field.	0b1

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXPFGF_EL1 is RAZ.
- Direct reads of ERXPFGF_EL1 are NOPs.
- Direct reads of ERXPFGF_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL selects an error record owned by a node that does not implement the Common Fault Injection Model Extension, then one of the following occurs:

- ERXPFGF_EL1 is RAZ.
- Direct reads of ERXPFGF_EL1 are NOPs.
- Direct reads of ERXPFGF_EL1 are **UNDEFINED**.



Note

A node does not implement the Common Fault Injection Model Extension if ERR<q>FR.INJ reads as 0b00. <q> is the index of the first error record owned by the same node as error record <n>, where <n> is the value in AArch64-ERRSELR_EL1.SEL. If the node owns a single record, then q = n.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>PFGF is not present, meaning reads of ERXPFGF_EL1 are **RES0**.

ext-ERR<n>PFGF describes additional constraints that also apply when ext-ERR<n>PFGF is accessed through ERXPFGF_EL1.

MRS <Xt>, ERXPFGF_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b100

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXPFGF_EL1 is RAZ.
- Direct reads of ERXPFGF_EL1 are NOPs.
- Direct reads of ERXPFGF_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL selects an error record owned by a node that does not implement the Common Fault Injection Model Extension, then one of the following occurs:

- ERXPFGF_EL1 is RAZ.
- Direct reads of ERXPFGF_EL1 are NOPs.
- Direct reads of ERXPFGF_EL1 are UNDEFINED.



Note

A node does not implement the Common Fault Injection Model Extension if ERR<q>FR.INJ reads as 0b00. <q> is the index of the first error record owned by the same node as error record <n>, where <n> is the value in AArch64-ERRSELR_EL1.SEL. If the node owns a single record, then q = n.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>PFGF is not present, meaning reads of ERXPFGF_EL1 are RES0.

ext-ERR<n>PFGF describes additional constraints that also apply when ext-ERR<n>PFGF is accessed through ERXPFGF_EL1.

MRS <Xt>, ERXPFGF_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.FIEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXPFGF_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.FIEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        end if
    end if
end if

```

```
    else
        return ERXPFGF_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
            UNDEFINED;
        elsif SCR_EL3.FIEN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXPFGF_EL1;
        elsif PSTATE.EL == EL3 then
            return ERXPFGF_EL1;
```

A.13.8 ERXPFGCTL_EL1, Selected Pseudo-fault Generation Control register

Accesses ext-ERR<n>PFGCTL for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-174: AArch64_erxpgctl_el1 bit assignments

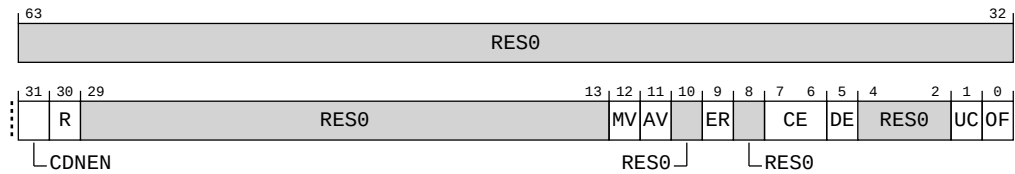


Table A-411: ERXPGCTL_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31]	CDNEN	Countdown Enable. Controls transfers of the value held in AArch64-ERXPGCDN_EL1 to the Error Generation Counter and enables this counter. 0b0 The Error Generation Counter is disabled. 0b1 The Error Generation Counter is enabled. On a write of 1 to this field, the Error Generation Counter is set to AArch64-ERXPGCDN_EL1.CDN.	x
[30]	R	Restartable. Support for Error Generation Counter restart mode. 0b0 The node does not support this feature. 0b1 Feature controllable.	x
[29:13]	RES0	Reserved	RES0
[12]	MV	Miscellaneous syndrome. The value written to AArch64-ERXSTATUS.MV when an injected error is recorded. 0b0 AArch64-ERXSTATUS.MV is set to 0 when an injected error is recorded. 0b1 AArch64-ERXSTATUS.MV is set to 1 when an injected error is recorded.	x
[11]	AV	Address syndrome. The value written to AArch64-ERXSTATUS.AV when an injected error is recorded. 0b0 AArch64-ERXSTATUS.AV is set to 0 when an injected error is recorded. 0b1 AArch64-ERXSTATUS.AV is set to 1 when an injected error is recorded.	x
[10]	RES0	Reserved	RES0
[9]	ER	Error Reported flag. The value written to AArch64-ERXSTATUS.ER when an injected error is recorded. 0b0 AArch64-ERXSTATUS.ER is set to 0 when an injected error is recorded. 0b1 AArch64-ERXSTATUS.ER is set to 1 when an injected error is recorded.	x
[8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:6]	CE	Corrected Error generation. Describes the types of Corrected Error that the fault generation feature of the node can generate. 0b00 The fault generation feature of the node cannot generate this type of error. 0b01 The fault generation feature of the node allows generation of a non-specific Corrected Error, that is, a Corrected Error that is recorded as AArch64-ERXSTATUS_EL1.CE == 0b10. All other values are reserved. If AArch64-ERXFR_EL1.FRX is 0b1 then AArch64-ERXFR_EL1.CE indicates whether the node supports this type of error. This field reads-as-zeros if the node does not support this type of error.	xx
[5]	DE	Deferred Error generation. Describes whether the fault generation feature of the node can generate this type of error. 0b0 The fault generation feature of the node cannot generate this type of error. 0b1 The fault generation feature of the node allows generation of this type of error. If AArch64-ERXFR_EL1.FRX is 0b1 then AArch64-ERXFR_EL1.DE indicates whether the node supports this type of error. This bit reads-as-zero if the node does not support this type of error.	x
[4:2]	RES0	Reserved	RES0
[1]	UC	Uncontainable Error generation. Describes whether the fault generation feature of the node can generate this type of error. 0b0 The fault generation feature of the node cannot generate this type of error. 0b1 The fault generation feature of the node allows generation of this type of error. If AArch64-ERXFR_EL1.FRX is 0b1 then AArch64-ERXFR_EL1.UC indicates whether the node supports this type of error. This bit reads-as-zero if the node does not support this type of error.	x
[0]	OF	Overflow flag. The value written to AArch64-ERXSTATUS.OF when an injected error is recorded. 0b0 AArch64-ERXSTATUS.OF is set to 0 when an injected error is recorded. 0b1 AArch64-ERXSTATUS.OF is set to 1 when an injected error is recorded.	x

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.

- ERXPFGCTL_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCTL_EL1 are NOPs.
- Direct reads and writes of ERXPFGCTL_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL selects an error record owned by a node that does not implement the Common Fault Injection Model Extension, then one of the following occurs:

- ERXPFGCTL_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCTL_EL1 are NOPs.
- Direct reads and writes of ERXPFGCTL_EL1 are **UNDEFINED**.



A node does not implement the Common Fault Injection Model Extension if ERR<q>FR.INJ reads as 0b00. <q> is the index of the first error record owned by the same node as error record <n>, where <n> is the value in AArch64-ERRSELR_EL1.SEL. If the node owns a single record, then q = n.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>PFGCTL is not present, meaning reads and writes of ERXPFGCTL_EL1 are **RES0**.

ext-ERR<n>PFGCTL describes additional constraints that also apply when ext-ERR<n>PFGCTL is accessed through ERXPFGCTL_EL1.

MRS <Xt>, ERXPFGCTL_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b101

MSR ERXPFGCTL_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b101

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXPFGCTL_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCTL_EL1 are NOPs.
- Direct reads and writes of ERXPFGCTL_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL selects an error record owned by a node that does not implement the Common Fault Injection Model Extension, then one of the following occurs:

- ERXPFGCTL_EL1 is RAZ/WI.

- Direct reads and writes of ERXPFPGCTL_EL1 are NOPs.
- Direct reads and writes of ERXPFPGCTL_EL1 are UNDEFINED.

**Note**

A node does not implement the Common Fault Injection Model Extension if `ERR<q>FR.INJ` reads as `0b00`. `<q>` is the index of the first error record owned by the same node as error record `<n>`, where `<n>` is the value in `AArch64-ERRSELR_EL1.SEL`. If the node owns a single record, then `q = n`.

If `AArch64-ERRSELR_EL1.SEL` is not the index of the first error record owned by a node, then `ext-ERR<n>PFGCTL` is not present, meaning reads and writes of `ERXPFPGCTL_EL1` are RESO.

`ext-ERR<n>PFGCTL` describes additional constraints that also apply when `ext-ERR<n>PFGCTL` is accessed through `ERXPFPGCTL_EL1`.

MRS `<Xt>`, `ERXPFPGCTL_EL1`

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.FIEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXPFPGCTL_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.FIEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXPFPGCTL_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
            UNDEFINED;
        elseif SCR_EL3.FIEN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXPFPGCTL_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXPFPGCTL_EL1;

```

MSR `ERXPFPGCTL_EL1, <Xt>`

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.FIEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXPFPGCTL_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.FIEN == '0' then
        if Halted() && EDSCR.SDD == '1' then

```

```
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ERXPFPGCTL_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
        UNDEFINED;
    elseif SCR_EL3.FIEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ERXPFPGCTL_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    ERXPFPGCTL_EL1 = X[t];
```

A.13.9 ERXPFPGCDN_EL1, Selected Pseudo-fault Generation Countdown register

Accesses ext-ERR<n>PFGCDN for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-175: AArch64_erxpfgcdn_el1 bit assignments

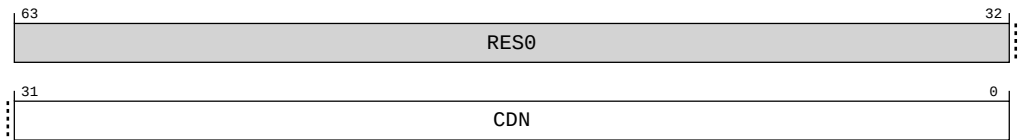


Table A-414: ERXPFGCDN_EL1 bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:0]	CDN	Countdown value. This field is copied to Error Generation Counter when either: • Software writes ERXPFGCTL.CDNEN with 1. • The Error Generation Counter decrements to zero and ERXPFGCTL.R == 1. Note: The current Error Generation Counter value is not visible to software.	32 {x}

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXPFGCDN_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCDN_EL1 are NOPs.
- Direct reads and writes of ERXPFGCDN_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL selects an error record owned by a node that does not implement the Common Fault Injection Model Extension, then one of the following occurs:

- ERXPFGCDN_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCDN_EL1 are NOPs.
- Direct reads and writes of ERXPFGCDN_EL1 are **UNDEFINED**.



A node does not implement the Common Fault Injection Model Extension if ERR<q>FR.INJ reads as 0b00. <q> is the index of the first error record owned by the same node as error record <n>, where <n> is the value in AArch64-ERRSELR_EL1.SEL. If the node owns a single record, then q = n.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>PFGCDN is not present, meaning reads and writes of ERXPFGCDN_EL1 are **RES0**.

ext-ERR<n>PFGCDN describes additional constraints that also apply when ext-ERR<n>PFGCDN is accessed through ERXPFGCDN_EL1.

MRS <Xt>, ERXPFGCDN_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b110

MSR ERXPFGCDN_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0100	0b110

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXPFGCDN_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCDN_EL1 are NOPs.
- Direct reads and writes of ERXPFGCDN_EL1 are UNDEFINED.

If AArch64-ERRSELR_EL1.SEL selects an error record owned by a node that does not implement the Common Fault Injection Model Extension, then one of the following occurs:

- ERXPFGCDN_EL1 is RAZ/WI.
- Direct reads and writes of ERXPFGCDN_EL1 are NOPs.
- Direct reads and writes of ERXPFGCDN_EL1 are UNDEFINED.



Note

A node does not implement the Common Fault Injection Model Extension if ERR<q>FR.INJ reads as 0b00. <q> is the index of the first error record owned by the same node as error record <n>, where <n> is the value in AArch64-ERRSELR_EL1.SEL. If the node owns a single record, then q = n.

If AArch64-ERRSELR_EL1.SEL is not the index of the first error record owned by a node, then ext-ERR<n>PFGCDN is not present, meaning reads and writes of ERXPFGCDN_EL1 are RES0.

ext-ERR<n>PFGCDN describes additional constraints that also apply when ext-ERR<n>PFGCDN is accessed through ERXPFGCDN_EL1.

MRS <Xt>, ERXPFGCDN_EL1

```
if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.FIEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
```

```

    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXPFPCDN_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.FIEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXPFPCDN_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
            UNDEFINED;
        elsif SCR_EL3.FIEN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXPFPCDN_EL1;
    elsif PSTATE.EL == EL3 then
        return ERXPFPCDN_EL1;

```

MSR ERXPFPCDN_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.FIEN == '0' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXPFPCDN_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.FIEN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXPFPCDN_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.FIEN == '0' then
            UNDEFINED;
        elsif SCR_EL3.FIEN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                ERXPFPCDN_EL1 = X[t];
    elsif PSTATE.EL == EL3 then
        ERXPFPCDN_EL1 = X[t];

```

A.13.10 ERXMISCO_EL1, Selected Error Record Miscellaneous Register 0

Accesses ext-ERR<n>MISCO for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width
64

Functional group
RAS registers

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-176: AArch64_erxmisc0_el1 bit assignments

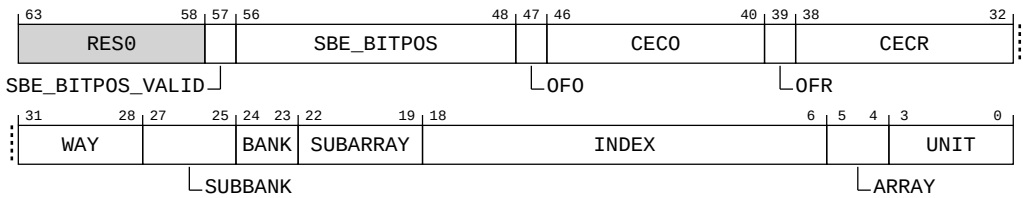


Table A-417: ERXMISC0_EL1 bit descriptions

Bits	Name	Description	Reset
[63:58]	RES0	Reserved	RES0
[57]	SBE_BITPOS_VALID	Bit position of a corrected error from a RAM protected by ECC Valid	x
[56:48]	SBE_BITPOS	Bit position of a corrected error from a RAM protected by ECC	9 { x }
[47]	OFO	Sticky overflow bit, other. Set to 1 when ERXMISC0_EL1.CECO is incremented and wraps through zero. 0b0 Other counter has not overflowed. 0b1 Other counter has overflowed. A direct write that modifies this bit might indirectly set ERXSTATUS_EL1.OF to an UNKNOWN value and a direct write to ERXSTATUS_EL1.OF that clears it to zero might indirectly set this bit to an UNKNOWN value. Unaffected by Warm reset.	0b0

Bits	Name	Description	Reset
[46:40]	CECO	Corrected error count, other. Incremented for each countable error that is not accounted for by incrementing ERXMISCO_EL1.CECR. Unaffected by Warm reset.	0b0000000
[39]	OFR	Sticky overflow bit, repeat. Set to 1 when ERXMISCO_EL1.CECR is incremented and wraps through zero. 0b0 Repeat counter has not overflowed. 0b1 Repeat counter has overflowed. A direct write that modifies this bit might indirectly set ERXSTATUS_EL1.OF to an UNKNOWN value and a direct write to ERXSTATUS_EL1.OF that clears it to zero might indirectly set this bit to an UNKNOWN value. Unaffected by Warm reset.	0b0
[38:32]	CECR	Corrected error count, repeat. Incremented for the first countable error, which also records other syndrome for the error, and subsequently for each countable error that matches the recorded other syndrome.	0b0000000
[31:28]	WAY	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L1 Data Cache] - Indicates which Tag RAM way or data RAM way detected the error. Upper 2 bits are unused. [L2 TLB] - Indicates which RAM detected an error. The possible values are 0 (RAM 1) to 9 (RAM 10). [L1 Instruction Cache] - Indicates which way detected the error. [L2 Tag Cache] - Indicates which way detected the error. Upper 1 bit unused. [L2 Data Cache] - Indicates which way detected the error. [L2 Transaction Queue Cache] - Unused Unaffected by Warm reset.	0b0000

Bits	Name	Description	Reset
[27:25]	SUBBANK	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L1 Instruction Cache] Indicates which subbank detected the error, valid for Instruction Data Cache. For Tag errors this field is zero. Unaffected by Warm reset.	0b000
[24:23]	BANK	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Cache] - Indicates which L2 bank detected the error. Upper 1 bit is unused. [L1 Instruction Cache] - Indicates which bank detected the error, valid for Instruction Cache. Upper 1 bit is unused Unaffected by Warm reset.	0b00
[22:19]	SUBARRAY	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Tag Cache] - Indicates which tag bank detected the error. Only values from 0b00 to 0b11 are possible. [L2 Data Cache] - Indicates which data granule detected the error. Possible values vary with the granule size. [L2 Transaction Queue Cache] - Indicates which data granule detected the error. Only values from 0b00 to 0b11 are possible. [L1 Tag Cache] - Indicates for L1 Data RAM which bank had the error detected. Only values 0b0 and 0b1 are possible. [L1 Data Cache] - Indicates for L1 Data RAM which granule had the error detected. All values are possible. [L2 TLB] - Indicates for L2 TLB RAM which word had the error detected. Upper 3 bits are unused. Unaffected by Warm reset.	0b0000

Bits	Name	Description	Reset
[18:6]	INDEX	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Tag Cache] - Indicates which index detected the error. Bits[n:6] are the index, n varies with the cache size. [L2 Data Cache] - Indicates which index detected the error. Bits[n:6] are the index, n varies with the cache size. [L2 Transaction Queue Cache] - Indicates which index detected the error. Bits[10:6] are the index, upper bits are unused. [L1 Data Cache] - Indicates which index detected the error. Upper bits of the index are unused depending on the cache size [L2 TLB] - Index of TLB RAM. Upper 4 bits are unused. [L1 Instruction Cache] - Indicates which index detected the error. Upper bits of the index are unused depending on the cache size. Unaffected by Warm reset.	0b00000000000000

Bits	Name	Description	Reset
[5:4]	ARRAY	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Cache] Indicates which array detected the error. The possible values are: 0b00 L2 Tag RAM. 0b01 L2 Data RAM. 0b10 Transaction Queue RAM. [L1 Data Cache] Indicates which array detected the error. The possible values are: 00 LS Tag RAM 0. 01 LS Tag RAM 1. 10 LS Data RAM. 11 LS Tag RAM 2. [L1 Instruction Cache] Indicates which array that detected the error, Data Array has higher priority. The possible values are: 0b00 Tag. 0b01 Data. Unaffected by Warm reset.	0b00
[3:0]	UNIT	Indicates the unit which detected the error. The possible values are: 0b0001 L1 Instruction Cache. 0b0010 L2 TLB. 0b0100 L1 Data Cache. 0b1000 L2 Cache.	0b0000

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXMISCO_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISCO_EL1 are NOPs.
- Direct reads and writes of ERXMISCO_EL1 are UNDEFINED.

ext-ERR<n>MISCO describes additional constraints that also apply when ext-ERR<n>MISCO is accessed through ERXMISCO_EL1.

MRS <Xt>, ERXMISCO_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b000

MSR ERXMISCO_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b000

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXMISCO_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISCO_EL1 are NOPs.
- Direct reads and writes of ERXMISCO_EL1 are UNDEFINED.

ext-ERR<n>MISCO describes additional constraints that also apply when ext-ERR<n>MISCO is accessed through ERXMISCO_EL1.

MRS <Xt>, ERXMISCO_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXMISCN_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXMISCO_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXMISCO_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXMISCO_EL1;

```

MSR ERXMISCO_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXMISCN_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXMISCO_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                ERXMISCO_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        ERXMISCO_EL1 = X[t];

```

A.13.11 ERXMISC1_EL1, Selected Error Record Miscellaneous Register 1

Accesses ext-ERR<n>MISC1 for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes**Width**

64

Functional group

RAS registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-177: AArch64_erxmisc1_el1 bit assignments

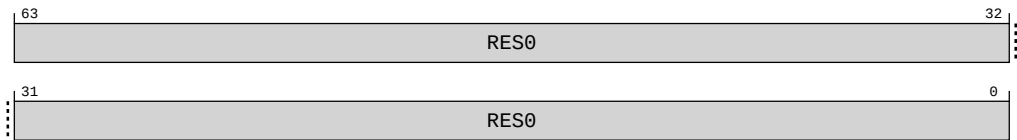


Table A-420: ERXMISC1_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXMISC1_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISC1_EL1 are NOPs.
- Direct reads and writes of ERXMISC1_EL1 are UNDEFINED.

ext-ERR<n>MISC1 describes additional constraints that also apply when ext-ERR<n>MISC1 is accessed through ERXMISC1_EL1.

MRS <Xt>, ERXMISC1_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b001

MSR ERXMISC1_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b001

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.

- ERXMISC1_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISC1_EL1 are NOPs.
- Direct reads and writes of ERXMISC1_EL1 are UNDEFINED.

ext-ERR<n>MISC1 describes additional constraints that also apply when ext-ERR<n>MISC1 is accessed through ERXMISC1_EL1.

MRS <Xt>, ERXMISC1_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXMISCn_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXMISC1_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXMISC1_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXMISC1_EL1;

```

MSR ERXMISC1_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXMISCn_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXMISC1_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;

```

```
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ERXMISC1_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    ERXMISC1_EL1 = X[t];
```

A.13.12 ERXMISC2_EL1, Selected Error Record Miscellaneous Register 2

Accesses ext-ERR<n>MISC2 for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

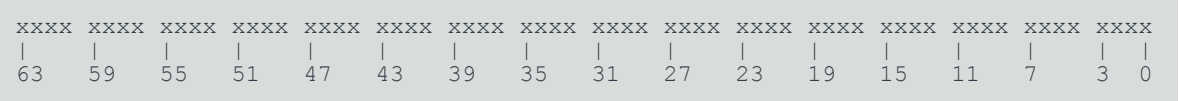
Functional group

RAS registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-178: AArch64_erxmisc2_el1 bit assignments

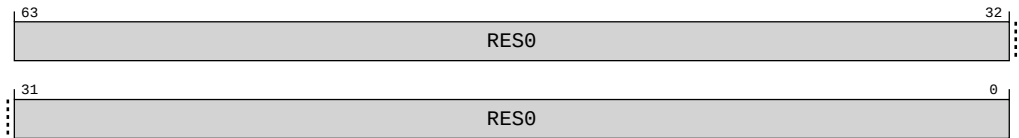


Table A-423: ERXMISC2_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSEL_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXMISC2_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISC2_EL1 are NOPs.
- Direct reads and writes of ERXMISC2_EL1 are UNDEFINED.

ext-ERR<n>MISC2 describes additional constraints that also apply when ext-ERR<n>MISC2 is accessed through ERXMISC2_EL1.

MRS <Xt>, ERXMISC2 EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b010

MSR ERXMISC2 EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b010

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXMISC2_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISC2_EL1 are NOPs.
- Direct reads and writes of ERXMISC2_EL1 are UNDEFINED.

ext-ERR<n>MISC2 describes additional constraints that also apply when ext-ERR<n>MISC2 is accessed through ERXMISC2_EL1.

MRS <Xt>, ERXMISC2 EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elsif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXMISCN_EL1 == '1'
then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else

```

```

        return ERXMISC2_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXMISC2_EL1;
        elseif PSTATE.EL == EL3 then
            return ERXMISC2_EL1;

```

MSR ERXMISC2_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXMISCn_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            ERXMISC2_EL1 = X[t];
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                ERXMISC2_EL1 = X[t];
    elseif PSTATE.EL == EL3 then
        ERXMISC2_EL1 = X[t];

```

A.13.13 ERXMISC3_EL1, Selected Error Record Miscellaneous Register 3

Accesses ext-ERR<n>MISC3 for the error record <n> selected by AArch64-ERRSELR_EL1.SEL.

Configurations

This register is available in all configurations.

Attributes

Width

64

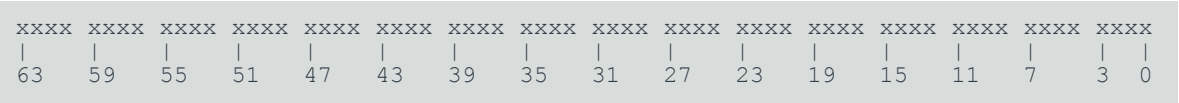
Functional group

RAS registers

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-179: AArch64_erxmisc3_el1 bit assignments

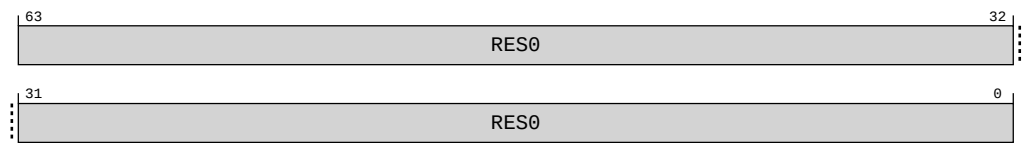


Table A-426: ERXMISC3_EL1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An **UNKNOWN** error record is selected.
- ERXMISC3_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISC3_EL1 are NOPs.
- Direct reads and writes of ERXMISC3_EL1 are UNDEFINED.

ext-ERR<n>MISC3 describes additional constraints that also apply when ext-ERR<n>MISC3 is accessed through ERXMISC3_EL1.

MRS <Xt>, ERXMISC3_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b011

MSR ERXMISC3_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b0101	0b0101	0b011

Accessibility

If AArch64-ERRIDR_EL1.NUM is 0x0000 or AArch64-ERRSELR_EL1.SEL is greater than or equal to AArch64-ERRIDR_EL1.NUM, then one of the following occurs:

- An UNKNOWN error record is selected.
- ERXMISC3_EL1 is RAZ/WI.
- Direct reads and writes of ERXMISC3_EL1 are NOPs.
- Direct reads and writes of ERXMISC3_EL1 are UNDEFINED.

ext-ERR<n>MISC3 describes additional constraints that also apply when ext-ERR<n>MISC3 is accessed through ERXMISC3_EL1.

MRS <Xt>, ERXMISC3_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGTR_EL2.ERXMISCN_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return ERXMISC3_EL1;
    elseif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
            UNDEFINED;
        elseif SCR_EL3.TERR == '1' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return ERXMISC3_EL1;
    elseif PSTATE.EL == EL3 then
        return ERXMISC3_EL1;

```

MSR ERXMISC3_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif EL2Enabled() && HCR_EL2.TERR == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HFGWTR_EL2.ERXMISCN_EL1 == '1'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);

```

```

elseif SCR_EL3.TERR == '1' then
    if Halted() && EDSCR.SDD == '1' then
        UNDEFINED;
    else
        AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ERXMISC3_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && SCR_EL3.TERR == '1' then
        UNDEFINED;
    elseif SCR_EL3.TERR == '1' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        ERXMISC3_EL1 = X[t];
elseif PSTATE.EL == EL3 then
    ERXMISC3_EL1 = X[t];

```

A.14 AArch64 Statistical Profiling Extension registers summary

The summary table provides an overview of all Statistical Profiling Extension registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-429: Statistical Profiling Extension registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
PMSCR_EL1	3	0	C9	C9	0	See individual bit resets.	64-bit	Statistical Profiling Control Register (EL1)
PMSNEVFR_EL1	3	0	C9	C9	1	See individual bit resets.	64-bit	Sampling Inverted Event Filter Register
PMSICR_EL1	3	0	C9	C9	2	See individual bit resets.	64-bit	Sampling Interval Counter Register
PMSIRR_EL1	3	0	C9	C9	3	See individual bit resets.	64-bit	Sampling Interval Reload Register
PMSFCR_EL1	3	0	C9	C9	4	See individual bit resets.	64-bit	Sampling Filter Control Register
PMSEVFR_EL1	3	0	C9	C9	5	See individual bit resets.	64-bit	Sampling Event Filter Register
PMSLATFR_EL1	3	0	C9	C9	6	See individual bit resets.	64-bit	Sampling Latency Filter Register
PMSIDR_EL1	3	0	C9	C9	7	See individual bit resets.	64-bit	Sampling Profiling ID Register
PMBLIMITR_EL1	3	0	C9	C10	0	See individual bit resets.	64-bit	Profiling Buffer Limit Address Register
PMBPTR_EL1	3	0	C9	C10	1	See individual bit resets.	64-bit	Profiling Buffer Write Pointer Register
PMBSR_EL1	3	0	C9	C10	3	See individual bit resets.	64-bit	Profiling Buffer Status/syndrome Register
PMBIDR_EL1	3	0	C9	C10	7	See individual bit resets.	64-bit	Profiling Buffer ID Register
PMSCR_EL2	3	4	C9	C9	0	See individual bit resets.	64-bit	Statistical Profiling Control Register (EL2)

A.14.1 PMSNEVFR_EL1, Sampling Inverted Event Filter Register

Controls sample filtering by events. The overall filter is the logical AND of these filters. For example, if E[3] and E[5] are both set to 1, only samples that have both event 3 (Level 1 unified or data cache refill) and event 5 (TLB walk) clear are recorded.

Configurations

This register is present only when FEAT_SPE is implemented. Otherwise, direct accesses to PMBIDR_EL1 are undefined

Attributes

Width

64

Functional group

Statistical Profiling Extension registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	xxxx	xxxx	0000	0xx0	xxxx	x000	xxx0	x0x0
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-180: AArch64_pmsnevfr_el1 bit assignments

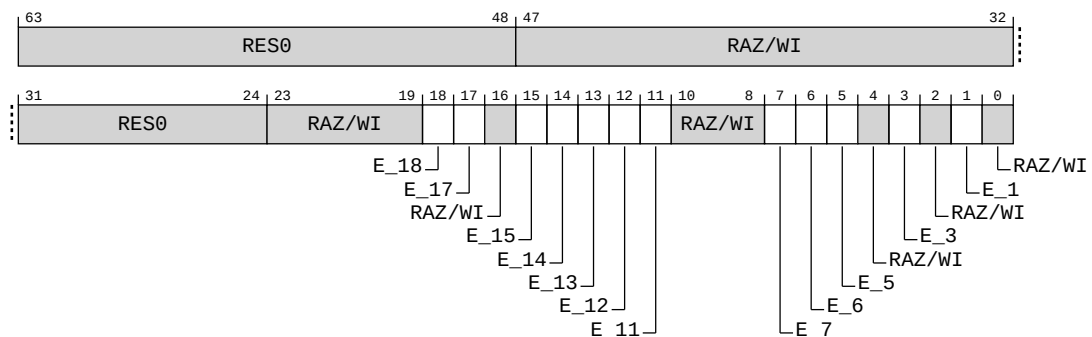


Table A-430: PMSNEVFR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:48]	RES0	Reserved	RES0
[47:32]	RAZ/WI	Reserved	RAZ/WI
[31:24]	RES0	Reserved	RES0
[23:19]	RAZ/WI	Reserved	RAZ/WI
[18]	E_18	None	x
[17]	E_17	None	x
[16]	RAZ/WI	Reserved	RAZ/WI
[15]	E_15	None	x
[14]	E_14	None	x
[13]	E_13	None	x
[12]	E_12	None	x
[11]	E_11	None	x
[10:8]	RAZ/WI	Reserved	RAZ/WI
[7]	E_7	None	x
[6]	E_6	None	x
[5]	E_5	None	x
[4]	RAZ/WI	Reserved	RAZ/WI
[3]	E_3	None	x
[2]	RAZ/WI	Reserved	RAZ/WI
[1]	E_1	None	x
[0]	RAZ/WI	Reserved	RAZ/WI

Access

MRS <Xt>, PMSNEVFR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1001	0b001

MSR PMSNEVFR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1001	0b001

Accessibility

MRS <Xt>, PMSNEVFR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
        UNDEFINED;
    elseif Halted() && EDSCR.SDD == '1' && MDCR_EL3.EnPMSN == '0' then
        UNDEFINED;

```

```

    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.nPMSNEVFR_EL1 == '0'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && MDCR_EL2.TPMS == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        elsif MDCR_EL3.EnPMSN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMSNEVFR_EL1;
        elsif PSTATE.EL == EL2 then
            if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
            MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
                UNDEFINED;
            elsif Halted() && EDSCR.SDD == '1' && MDCR_EL3.EnPMSN == '0' then
                UNDEFINED;
            elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
            elsif MDCR_EL3.EnPMSN == '0' then
                if Halted() && EDSCR.SDD == '1' then
                    UNDEFINED;
                else
                    AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMSNEVFR_EL1;
        elsif PSTATE.EL == EL3 then
            return PMSNEVFR_EL1;

```

MSR PMSNEVFR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
    MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
        UNDEFINED;
    elsif Halted() && EDSCR.SDD == '1' && MDCR_EL3.EnPMSN == '0' then
        UNDEFINED;
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.nPMSNEVFR_EL1 == '0'
    then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && MDCR_EL2.TPMS == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    elsif MDCR_EL3.EnPMSN == '0' then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        PMSNEVFR_EL1 = X[t];
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
        MDCR_EL3.NSPB[1] != SCR_EL3.NS) then

```

```

        UNDEFINED;
    elsif Halted() && EDSCR.SDD == '1' && MDCR_EL3.EnPMSN == '0' then
        UNDEFINED;
    elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        elsif MDCR_EL3.EnPMSN == '0' then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                PMSNEVFR_EL1 = X[t];
        elsif PSTATE.EL == EL3 then
            PMSNEVFR_EL1 = X[t];

```

A.14.2 PMSEVFR_EL1, Sampling Event Filter Register

Controls sample filtering by events. The overall filter is the logical AND of these filters. For example, if E[3] and E[5] are both set to 1, only samples that have both event 3 (Level 1 unified or data cache refill) and event 5 set (TLB walk) are recorded.

Configurations

This register is present only when FEAT_SPE is implemented. Otherwise, direct accesses to PMBIDR_EL1 are undefined

Attributes

Width

64

Functional group

Statistical Profiling Extension registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	xxxx	xxxx	0000	0xx0	xxxx	x000	xxx0	x0x0
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-181: AArch64_pmsevfr_el1 bit assignments

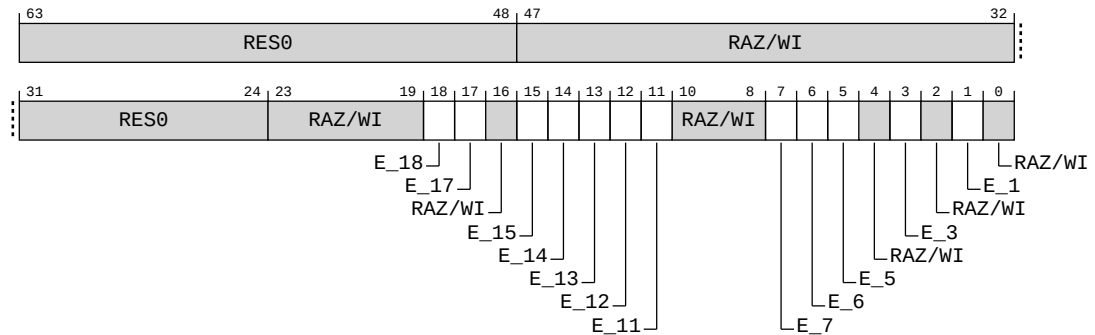


Table A-433: PMSEVFR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:48]	RES0	Reserved	RES0
[47:32]	RAZ/WI	Reserved	RAZ/WI
[31:24]	RES0	Reserved	RES0
[23:19]	RAZ/WI	Reserved	RAZ/WI
[18]	E_18	None	x
[17]	E_17	None	x
[16]	RAZ/WI	Reserved	RAZ/WI
[15]	E_15	None	x
[14]	E_14	None	x
[13]	E_13	None	x
[12]	E_12	None	x
[11]	E_11	None	x
[10:8]	RAZ/WI	Reserved	RAZ/WI
[7]	E_7	None	x
[6]	E_6	None	x
[5]	E_5	None	x
[4]	RAZ/WI	Reserved	RAZ/WI
[3]	E_3	None	x
[2]	RAZ/WI	Reserved	RAZ/WI
[1]	E_1	None	x
[0]	RAZ/WI	Reserved	RAZ/WI

Access

MRS <Xt>, PMSEVFR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1001	0b101

MSR PMSEVFR_EL1, <Xt>

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1001	0b101

Accessibility

MRS <Xt>, PMSEVFR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
        UNDEFINED;
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.PMSEVFR_EL1 == '1'
then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && MDCR_EL2.TPMS == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return PMSEVFR_EL1;
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
        UNDEFINED;
    elseif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        return PMSEVFR_EL1;
elseif PSTATE.EL == EL3 then
    return PMSEVFR_EL1;

```

MSR PMSEVFR_EL1, <Xt>

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
        UNDEFINED;
    elseif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGWTR_EL2.PMSEVFR_EL1 == '1'
then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif EL2Enabled() && MDCR_EL2.TPMS == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elseif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
    else
        PMSEVFR_EL1 = X[t];
elseif PSTATE.EL == EL2 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then

```



```
        UNDEFINED;
    elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            PMSEVFR_EL1 = X[t];
    elsif PSTATE.EL == EL3 then
        PMSEVFR_EL1 = X[t];
```

A.14.3 PMSIDR_EL1, Sampling Profiling ID Register

Describes the Statistical Profiling implementation to software

Configurations

This register is present only when FEAT_SPE is implemented. Otherwise, direct accesses to PMBIDR_EL1 are undefined

Attributes

Width

64

Functional group

Statistical Profiling Extension registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxx0	0000	0010	0110	0100	x101	0xxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-182: AArch64_pmsidr_el1 bit assignments

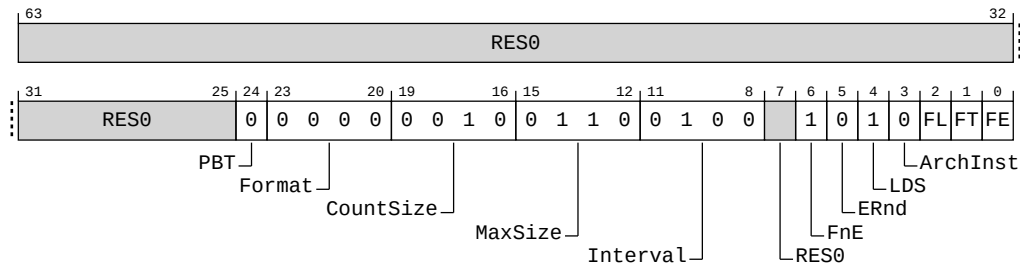


Table A-436: PMSIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:25]	RES0	Reserved	RES0
[24]	PBT	Previous branch target Address packet. Defined values are: 0b0 Previous branch target Address packet not supported.	0b0
[23:20]	Format	Defines the format of the sample records. Defined values are: 0b0000 Format 0. All other values are reserved.	0b0000
[19:16]	CountSize	Defines the size of the counters. 0b0010 12-bit saturating counters.	0b0010
[15:12]	MaxSize	Defines the largest size for a single record, rounded up to a power-of-two. If this is the same as the minimum alignment (AArch64-PMBIDR_EL1.Align), then each record is exactly this size. Defined values are: 0b0110 64 bytes.	0b0110
[11:8]	Interval	Recommended minimum sampling interval. This provides guidance from the implementer to the smallest minimum sampling interval, N. Defined values are: 0b0100 1,024.	0b0100
[7]	RES0	Reserved	RES0
[6]	FnE	Filtering by events, inverted. Defined values are: 0b1 AArch64-PMSNEVFR_EL1 and AArch64-PMSFCR_EL1.FnE are implemented.	0b1
[5]	ERnd	Defines how the random number generator is used in determining the interval between samples, when enabled by AArch64-PMSIRR_EL1.RND. Defined values are: 0b0 The random number is added at the start of the interval, and the sample is taken and a new interval started when the combined interval expires.	0b0

Bits	Name	Description	Reset
[4]	LDS	Data source indicator for sampled load instructions. Defined values are: 0b1 Loaded data source implemented.	0b1
[3]	ArchInst	Architectural instruction profiling. Defined values are: 0b0 Micro-op sampling implemented.	0b0
[2]	FL	Filtering by latency. This bit is RAO .	x
[1]	FT	Filtering by operation type. This bit is RAO .	x
[0]	FE	Filtering by events. This bit is RAO .	x

Access

MRS <Xt>, PMSIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1001	0b111

Accessibility

MRS <Xt>, PMSIDR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elsif PSTATE.EL == EL1 then
    if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
        UNDEFINED;
    elsif EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.PMSIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif EL2Enabled() && MDCR_EL2.TPMS == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
        if Halted() && EDSCR.SDD == '1' then
            UNDEFINED;
        else
            AArch64.SystemAccessTrap(EL3, 0x18);
        else
            return PMSIDR_EL1;
    elsif PSTATE.EL == EL2 then
        if Halted() && EDSCR.SDD == '1' && (MDCR_EL3.NSPB[0] == '0' ||
MDCR_EL3.NSPB[1] != SCR_EL3.NS) then
            UNDEFINED;
        elsif MDCR_EL3.NSPB[0] == '0' || MDCR_EL3.NSPB[1] != SCR_EL3.NS then
            if Halted() && EDSCR.SDD == '1' then
                UNDEFINED;
            else
                AArch64.SystemAccessTrap(EL3, 0x18);
            else
                return PMSIDR_EL1;
    elsif PSTATE.EL == EL3 then
        return PMSIDR_EL1;

```

A.14.4 PMBIDR_EL1, Profiling Buffer ID Register

Provides information to software as to whether the buffer can be programmed at the current Exception level.

Configurations

This register is present only when FEAT_SPE is implemented. Otherwise, direct accesses to PMBIDR_EL1 are undefined

Attributes

Width

64

Functional group

Statistical Profiling Extension registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx10	0110
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-183: AArch64_pmbidr_el1 bit assignments

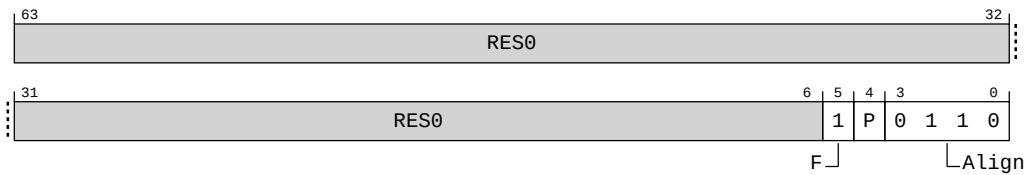


Table A-438: PMBIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:6]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[5]	F	Flag updates. Describes how address translations performed by the Statistical Profiling Extension manage the Access flag and dirty state. 0b1 Hardware management of the Access flag and dirty state for accesses made by the Statistical Profiling Extension is controlled in the same way as explicit memory accesses in the Profiling Buffer owning translation regime.	0b1
[4]	P	Programming not allowed. When read at EL3, this field reads as zero. Otherwise, indicates that the Profiling Buffer is owned by a higher Exception level or another Security state. Defined values are: 0b0 Programming is allowed. 0b1 Programming not allowed. The value read from this field depends on the current Exception level and the Effective values of AArch64-MDCR_EL3.NSPB and AArch64-MDCR_EL2.E2PB: - If EL3 is implemented, and the owning Security state is Secure state, this field reads as one from: - Non-secure EL1 and Non-secure EL2. - If Secure EL2 is implemented and enabled, and AArch64-MDCR_EL2.E2PB is 0b00, Secure EL1. - If EL3 is implemented, and the owning Security state is Non-secure state, this field reads as one from: - Secure EL1. - If Secure EL2 is implemented, Secure EL2. - If EL2 is implemented and AArch64-MDCR_EL2.E2PB is 0b00, Non-secure EL1. - If EL3 is not implemented, EL2 is implemented, and AArch64-MDCR_EL2.E2PB is 0b00, this field reads as one from EL1. - Otherwise, this field reads as zero.	0b0
[3:0]	Align	Defines the minimum alignment constraint for writes to AArch64-PMBPTR_EL1. Defined values are: 0b0110 64 bytes.	0b0110

Access

MRS <Xt>, PMBIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1010	0b111

Accessibility

MRS <Xt>, PMBIDR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.PMBIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return PMBIDR_EL1;
elseif PSTATE.EL == EL2 then
    return PMBIDR_EL1;
elseif PSTATE.EL == EL3 then

```

```
return PMBIDR_EL1;
```

A.15 AArch64 Trace Buffer Extension registers summary

The summary table provides an overview of all Trace Buffer Extension registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table A-440: Trace Buffer Extension registers summary

Name	Op0	Op1	CRn	CRm	Op2	Reset	Width	Description
TRBLIMITR_EL1	3	0	C9	C11	0	See individual bit resets.	64-bit	Trace Buffer Limit Address Register
TRBPTR_EL1	3	0	C9	C11	1	See individual bit resets.	64-bit	Trace Buffer Write Pointer Register
TRBBASER_EL1	3	0	C9	C11	2	See individual bit resets.	64-bit	Trace Buffer Base Address Register
TRBSR_EL1	3	0	C9	C11	3	See individual bit resets.	64-bit	Trace Buffer Status/syndrome Register
TRBMAR_EL1	3	0	C9	C11	4	See individual bit resets.	64-bit	Trace Buffer Memory Attribute Register
TRBTRG_EL1	3	0	C9	C11	6	See individual bit resets.	64-bit	Trace Buffer Trigger Counter Register
TRBIDR_EL1	3	0	C9	C11	7	See individual bit resets.	64-bit	Trace Buffer ID Register

A.15.1 TRBIDR_EL1, Trace Buffer ID Register

Describes constraints on using the Trace Buffer Unit to software, including whether the Trace Buffer Unit can be programmed at the current Exception level.

Configurations

This register is available in all configurations.

Attributes

Width

64

Functional group

Trace Buffer Extension registers

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx10	0110

63 59 55 51 47 43 39 35 31 27 23 19 15 11 7 3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure A-184: AArch64_trbldr_el1 bit assignments

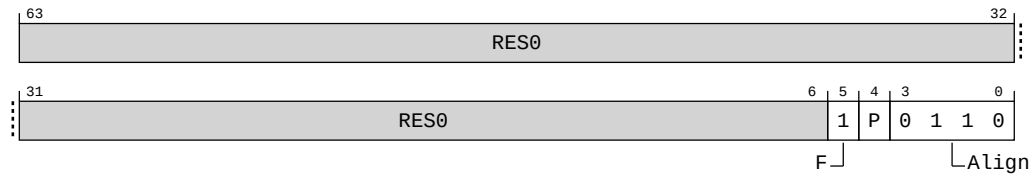


Table A-441: TRBIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[63:6]	RES0	Reserved	RES0
[5]	F	Flag updates. Describes how address translations performed by the Trace Buffer Unit manage the Access flag and dirty state. 0b1 Hardware management of the Access flag and dirty state for accesses made by the Trace Buffer Unit is controlled in the same way as explicit memory accesses in the trace buffer owning translation regime.	0b1
[4]	P	Programming not allowed. When read at EL3, this field reads as zero. Otherwise, indicates that the trace buffer is owned by a higher Exception level or another Security state. Defined values are: 0b0 Programming is allowed. 0b1 Programming not allowed. The value read from this field depends on the current Exception level and the Effective values of AArch64-MDCR_EL3.NSTB and AArch64-MDCR_EL2.E2TB: - If EL3 is implemented, and the owning Security state is Secure state, this field reads as one from: - Non-secure EL1 and Non-secure EL2. - If Secure EL2 is implemented and enabled, and AArch64-MDCR_EL2.E2TB is 0b00, Secure EL1. - If EL3 is implemented, and the owning Security state is Non-secure state, this field reads as one from: - Secure EL1. - If Secure EL2 is implemented, Secure EL2. - If EL2 is implemented and AArch64-MDCR_EL2.E2TB is 0b00, Non-secure EL1. - If EL3 is not implemented, EL2 is implemented, and AArch64-MDCR_EL2.E2TB is 0b00, this field reads as one from EL1. - Otherwise, this field reads as zero.	0b0

Bits	Name	Description	Reset
[3:0]	Align	Defines the minimum alignment constraint for writes to AArch64-TRBPTR_EL1 and AArch64-TRBTRG_EL1. Defined values are: 0b0110 64 bytes.	0b0110

Access

MRS <Xt>, TRBIDR_EL1

op0	op1	CRn	CRm	op2
0b11	0b000	0b1001	0b1011	0b111

Accessibility

MRS <Xt>, TRBIDR_EL1

```

if PSTATE.EL == EL0 then
    UNDEFINED;
elseif PSTATE.EL == EL1 then
    if EL2Enabled() && SCR_EL3.FGTEn == '1' && HDFGRTR_EL2.TRBIDR_EL1 == '1' then
        AArch64.SystemAccessTrap(EL2, 0x18);
    else
        return TRBIDR_EL1;
elseif PSTATE.EL == EL2 then
    return TRBIDR_EL1;
elseif PSTATE.EL == EL3 then
    return TRBIDR_EL1;

```


Appendix B External registers

This appendix contains the descriptions for the Neoverse™ N3 external registers.

This manual does not provide a complete list of registers. Read this manual together with the [Arm® Architecture Reference Manual for A-profile architecture](#).

B.1 External MPMM registers summary

The summary table provides an overview of all memory-mapped MPMM registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-1: MPMM registers summary

Offset	Name	Reset	Width	Description
0x000	CPUPPMCR	See individual bit resets.	32-bit	Global PPM Configuration Register
0x010	CPUMPMCR	See individual bit resets.	32-bit	Global MPMM Configuration Register
0x020	CPUPMPDPCR [31:0]	See individual bit resets.	32-bit	Global PPMPDP Configuration Register
0x024	CPUPMPDPCR [63:32]	See individual bit resets.	32-bit	Global PPMPDP Configuration Register

B.1.1 CPUPPMCR, Global PPM Configuration Register

This register controls global PPM features and allows discovery of some PPM implementation details.

Configurations

External register CPUPPMCR bits [31:0] are architecturally mapped to AArch64 System register [A.2.3 IMP_CPUPPMCR_EL3, Global PPM Configuration Register](#) on page 322 bits [31:0].

Attributes

Width

32

Component

MPMM

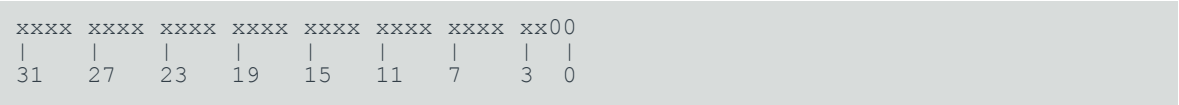
Register offset

0x000

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-1: ext_cpuppmcr bit assignments

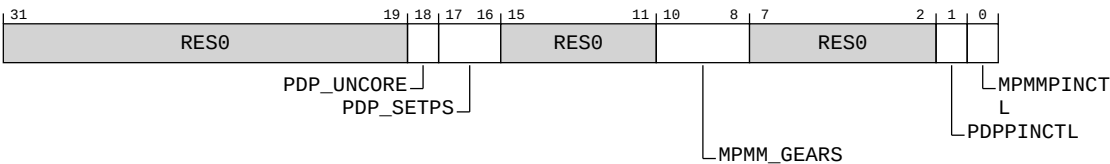


Table B-2: CPUPPMCR bit descriptions

Bits	Name	Description	Reset
[31:19]	RES0	Reserved	RES0
[18]	PDP_UNCORE	Indicates whether PDP uncore is implemented 0b1 PDP has separate uncore and core controls. Access to this field is: RO	x
[17:16]	PDP_SETPS	Number of PDP Setpoints implemented 0b11 3 PDP are enabled. Access to this field is: RO	xx
[15:11]	RES0	Reserved	RES0
[10:8]	MPMM_GEARs	Number of MPMM Gears implemented 0b011 3 MPMM are enabled. Access to this field is: RO	xxx
[7:2]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[1]	PDPPINCTL	PDP Pin Control Enabled 0b0 PDP control through SPR and utility bus 0b1 PDP control through pin only.	0b0
[0]	MPMMPINCTL	MPMM Pin Control Enabled 0b0 MPMM control through SPR and utility bus. 0b1 MPMM control through pin only.	0b0

Accessibility

Component	Offset	Instance	Range
MPMM	0x000	CPUPPMCR	None

This interface is accessible as follows:

When IsCorePowered() && IsAccessSecure()

RW

When IsCorePowered() && !IsAccessSecure()

RAZ/WI

Otherwise

ERROR

B.1.2 CPUMPMMCR, Global MPMM Configuration Register

This register is used to change MPMM gears or disable MPMM.

Configurations

External register CPUMPMMCR bits [31:0] are architecturally mapped to AArch64 System register [A.2.4 IMP_CPUMPMMCR_EL3, Global MPMM Configuration Register](#) on page 325 bits [31:0].

Attributes

Width

32

Component

MPMM

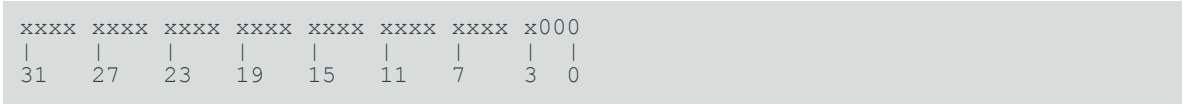
Register offset

0x010

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-2: ext_cpumpmmcr bit assignments



Table B-4: CPUMPMMCR bit descriptions

Bits	Name	Description	Reset
[31:3]	RES0	Reserved	RES0
[2:1]	MPMM_GEAR	MPMM Gear Select 0b00 Select MPMM Gear 0. 0b01 Select MPMM Gear 1. 0b10 Select MPMM Gear 2. 0b11 Do not throttle.	0b00
[0]	MPMM_EN	MPMM Enable 0b0 MPMM is not enabled. 0b1 MPMM is enabled.	0b0

Accessibility

Component	Offset	Instance	Range
MPMM	0x010	CPUMPMMCR	None

This interface is accessible as follows:

When IsCorePowered() && IsAccessSecure()

RW

When IsCorePowered() && !IsAccessSecure()
RAZ/WI

Otherwise
ERROR

B.1.3 CPUPPMPDPCR, Global PPMPDP Configuration Register

This register controls the aggressiveness of PDP features.

Configurations

External register CPUPPMPDPCR bits [63:0] are architecturally mapped to AArch64 System register [A.2.1 IMP_CPUPPMPDPCR_EL1, Global PPMPDP Configuration Register](#) on page 318 bits [63:0].

Attributes

Width
64

Component
MPMM

Register offsets (2)
0x020,0x024

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx00	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xx00
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-3: ext_cpupmpdpcr bit assignments

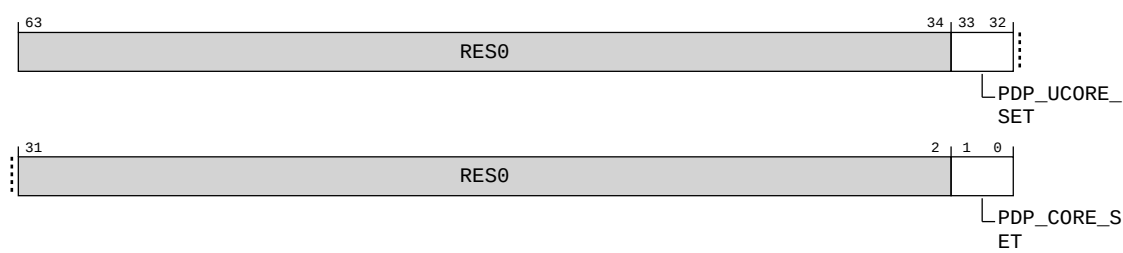


Table B-6: CPUPMPDPCR bit descriptions

Bits	Name	Description	Reset
[63:34]	RES0	Reserved	RES0
[33:32]	PDP_UCORE_SET	Uncore PDP Aggressiveness 0b00 Disable PDP. 0b01 Engage PDP at low aggressiveness 0b10 Engage PDP at medium aggressiveness 0b11 Engage PDP at high aggressiveness	0b00
[31:2]	RES0	Reserved	RES0
[1:0]	PDP_CORE_SET	Core PDP Aggressiveness 0b00 Disable PDP. 0b01 Engage PDP at low aggressiveness. 0b10 Engage PDP at medium aggressiveness. 0b11 Engage PDP at high aggressiveness.	0b00

Accessibility

Component	Offset	Instance	Range
MPMM	0x020	CPUPMPDPCR	31:0

This interface is accessible as follows:

When `IsCorePowered()` && `IsAccessSecure()`
RW

When IsCorePowered() && !IsAccessSecure()

RAZ/WI

Otherwise

ERROR

Component	Offset	Instance	Range
MPMM	0x024	CPUPPMPDPCR	63:32

This interface is accessible as follows:

When IsCorePowered() && IsAccessSecure()

RW

When IsCorePowered() && !IsAccessSecure()

RAZ/WI

Otherwise

ERROR

B.2 External CTI registers summary

The summary table provides an overview of all memory-mapped CTI registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-9: CTI registers summary

Offset	Name	Reset	Width	Description
0x000	CTICONTROL	See individual bit resets.	32-bit	CTI Control register
0x010	CTIINTACK	See individual bit resets.	32-bit	CTI Output Trigger Acknowledge register
0x014	CTIAPPSET	See individual bit resets.	32-bit	CTI Application Trigger Set register
0x018	CTIAPPCLEAR	See individual bit resets.	32-bit	CTI Application Trigger Clear register
0x01C	CTIAPPULSE	See individual bit resets.	32-bit	CTI Application Pulse register
0x020 + (4 * n)	CTIINEN_n_	See individual bit resets.	32-bit	CTI Input Trigger to Output Channel Enable registers
0x0A0 + (4 * n)	CTIOUTEN_n_	See individual bit resets.	32-bit	CTI Input Channel to Output Trigger Enable registers
0x130	CTITRIGINSTATUS	See individual bit resets.	32-bit	CTI Trigger In Status register
0x134	CTITRIGOUTSTATUS	See individual bit resets.	32-bit	CTI Trigger Out Status register
0x138	CTICHINSTATUS	See individual bit resets.	32-bit	CTI Channel In Status register
0x13C	CTICHOUTSTATUS	See individual bit resets.	32-bit	CTI Channel Out Status register
0x140	CTIGATE	See individual bit resets.	32-bit	CTI Channel Gate Enable register

Offset	Name	Reset	Width	Description
0x150	CTIDEVCTL	See individual bit resets.	32-bit	CTI Device Control register
0xFA0	CTICLAIMSET	See individual bit resets.	32-bit	CTI CLAIM Tag Set register
0xFA4	CTICLAIMCLR	See individual bit resets.	32-bit	CTI CLAIM Tag Clear register
0xFA8	CTIDEVAFF0	See individual bit resets.	32-bit	CTI Device Affinity register 0
0xFAC	CTIDEVAFF1	See individual bit resets.	32-bit	CTI Device Affinity register 1
0xFB0	CTILAR	See individual bit resets.	32-bit	CTI Lock Access Register
0xFB4	CTILSR	See individual bit resets.	32-bit	CTI Lock Status Register
0xFB8	CTIAUTHSTATUS	See individual bit resets.	32-bit	CTI Authentication Status register
0xFBC	CTIDEVARCH	See individual bit resets.	32-bit	CTI Device Architecture register
0xFC0	CTIDEVID2	See individual bit resets.	32-bit	CTI Device ID register 2
0xFC4	CTIDEVID1	See individual bit resets.	32-bit	CTI Device ID register 1
0xFC8	CTIDEVID	See individual bit resets.	32-bit	CTI Device ID register 0
0xFCC	CTIDEVTYPE	See individual bit resets.	32-bit	CTI Device Type register
0xFD0	CTIPIDR4	See individual bit resets.	32-bit	CTI Peripheral Identification Register 4
0xFE0	CTIPIDR0	See individual bit resets.	32-bit	CTI Peripheral Identification Register 0
0xFE4	CTIPIDR1	See individual bit resets.	32-bit	CTI Peripheral Identification Register 1
0xFE8	CTIPIDR2	See individual bit resets.	32-bit	CTI Peripheral Identification Register 2
0xFEC	CTIPIDR3	See individual bit resets.	32-bit	CTI Peripheral Identification Register 3
0xFF0	CTICIDR0	See individual bit resets.	32-bit	CTI Component Identification Register 0
0xFF4	CTICIDR1	See individual bit resets.	32-bit	CTI Component Identification Register 1
0xFF8	CTICIDR2	See individual bit resets.	32-bit	CTI Component Identification Register 2
0xFFC	CTICIDR3	See individual bit resets.	32-bit	CTI Component Identification Register 3

B.3 External Debug registers summary

The summary table provides an overview of all memory-mapped Debug registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-10: Debug registers summary

Offset	Name	Reset	Width	Description
0x020	EDESR	See individual bit resets.	32-bit	External Debug Event Status Register
0x024	EDECR	See individual bit resets.	32-bit	External Debug Execution Control Register
0x030	EDWAR [31:0]	See individual bit resets.	32-bit	External Debug Watchpoint Address Register
0x034	EDWAR [63:32]	See individual bit resets.	32-bit	External Debug Watchpoint Address Register
0x080	DBGDTRRX_ELO	See individual bit resets.	32-bit	Debug Data Transfer Register, Receive

Offset	Name	Reset	Width	Description
0x084	EDITR	See individual bit resets.	32-bit	External Debug Instruction Transfer Register
0x088	EDSCR	See individual bit resets.	32-bit	External Debug Status and Control Register
0x08C	DBGDTRTX_ELO	See individual bit resets.	32-bit	Debug Data Transfer Register, Transmit
0x090	EDRCR	See individual bit resets.	32-bit	External Debug Reserve Control Register
0x094	EDACR	See individual bit resets.	32-bit	External Debug Auxiliary Control Register
0x098	EDECCR	See individual bit resets.	32-bit	External Debug Exception Catch Control Register
0x300	OSLAR_EL1	See individual bit resets.	32-bit	OS Lock Access Register
0x310	EDPRCR	See individual bit resets.	32-bit	External Debug Power/Reset Control Register
0x314	EDPRSR	See individual bit resets.	32-bit	External Debug Processor Status Register
0x400	DBGBVR0_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x408	DBGBCR0_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x410	DBGBVR1_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x418	DBGBCR1_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x420	DBGBVR2_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x428	DBGBCR2_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x430	DBGBVR3_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x438	DBGBCR3_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x440	DBGBVR4_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x448	DBGBCR4_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x450	DBGBVR5_EL1 [63:0]	See individual bit resets.	64-bit	Debug Breakpoint Value Registers
0x458	DBGBCR5_EL1	See individual bit resets.	32-bit	Debug Breakpoint Control Registers
0x800	DBGWVR0_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x808	DBGWCR0_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0x810	DBGWVR1_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x818	DBGWCR1_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0x820	DBGWVR2_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x828	DBGWCR2_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0x830	DBGWVR3_EL1 [63:0]	See individual bit resets.	64-bit	Debug Watchpoint Value Registers
0x838	DBGWCR3_EL1	See individual bit resets.	32-bit	Debug Watchpoint Control Registers
0xD00	MIDR_EL1	See individual bit resets.	32-bit	Main ID Register
0xD20	EDPFR [31:0]	See individual bit resets.	32-bit	External Debug Processor Feature Register
0xD24	EDPFR [63:32]	See individual bit resets.	32-bit	External Debug Processor Feature Register
0xD28	EDDFR [31:0]	See individual bit resets.	32-bit	External Debug Feature Register
0xD2C	EDDFR [63:32]	See individual bit resets.	32-bit	External Debug Feature Register
0xD60	EDAA32PFR	See individual bit resets.	64-bit	External Debug Auxiliary Processor Feature Register
0xFA0	DBGCLAIMSET_EL1	See individual bit resets.	32-bit	Debug CLAIM Tag Set register
0xFA4	DBGCLAIMCLR_EL1	See individual bit resets.	32-bit	Debug CLAIM Tag Clear register
0xFA8	EDDEVAFF0	See individual bit resets.	32-bit	External Debug Device Affinity register 0
0xFAC	EDDEVAFF1	See individual bit resets.	32-bit	External Debug Device Affinity register 1
0xFB0	EDLAR	See individual bit resets.	32-bit	External Debug Lock Access Register

Offset	Name	Reset	Width	Description
0xFB4	EDLSR	See individual bit resets.	32-bit	External Debug Lock Status Register
0xFB8	DBGAUTHSTATUS_EL1	See individual bit resets.	32-bit	Debug Authentication Status register
0xFBC	EDDEVARCH	See individual bit resets.	32-bit	External Debug Device Architecture register
0xFC0	EDDEVID2	See individual bit resets.	32-bit	External Debug Device ID register 2
0xFC4	EDDEVID1	See individual bit resets.	32-bit	External Debug Device ID register 1
0xFC8	EDDEVID	See individual bit resets.	32-bit	External Debug Device ID register 0
0xFCC	EDDEVTYPE	See individual bit resets.	32-bit	External Debug Device Type register
0xFD0	EDPIDR4	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 4
0xFE0	EDPIDR0	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 0
0xFE4	EDPIDR1	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 1
0xFE8	EDPIDR2	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 2
0xFEC	EDPIDR3	See individual bit resets.	32-bit	External Debug Peripheral Identification Register 3
0xFF0	EDCIDR0	See individual bit resets.	32-bit	External Debug Component Identification Register 0
0xFF4	EDCIDR1	See individual bit resets.	32-bit	External Debug Component Identification Register 1
0xFF8	EDCIDR2	See individual bit resets.	32-bit	External Debug Component Identification Register 2
0xFFC	EDCIDR3	See individual bit resets.	32-bit	External Debug Component Identification Register 3

B.3.1 EDRCR, External Debug Reserve Control Register

This register is used to allow imprecise entry to Debug state and clear sticky bits in ext-EDSCR.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

Debug

Register offset

0x090

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3
							0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-4: ext_edrcr bit assignments

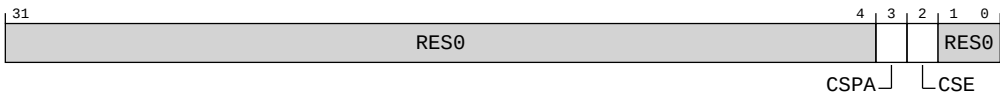


Table B-11: EDRCR bit descriptions

Bits	Name	Description	Reset
[31:4]	RES0	Reserved	RES0
[3]	CSPA	Clear Sticky Pipeline Advance. This bit is used to clear the ext-EDSCR.PipeAdv bit to 0. The actions on writing to this bit are: 0b0 No action. 0b1 Clear the ext-EDSCR.PipeAdv bit to 0.	x
[2]	CSE	Clear Sticky Error. Used to clear the ext-EDSCR cumulative error bits to 0. The actions on writing to this bit are: 0b0 No action. 0b1 Clear the ext-EDSCR.{TXU, RXO, ERR} bits, and, if the PE is in Debug state, the ext-EDSCR.ITO bit, to 0.	x
[1:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
Debug	0x090	EDRCR	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && SoftwareLockStatus()
 WI

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && !SoftwareLockStatus()
 WO

Otherwise
 ERROR

B.3.2 EDACR, External Debug Auxiliary Control Register

Allows implementations to support **IMPLEMENTATION DEFINED** controls.

Configurations

This register is implemented in the Core power domain since FEAT_DoPD is implemented.

Attributes

Width

32

Component

Debug

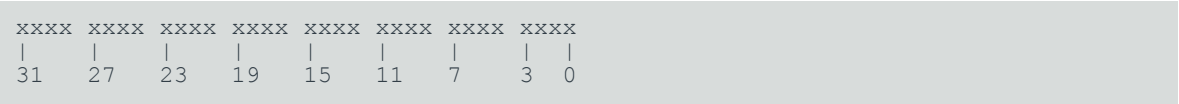
Register offset

0x094

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-5: ext_edacr bit assignments



Table B-13: EDACR bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
Debug	0x094	EDACR	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && SoftwareLockStatus()
RO

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && !SoftwareLockStatus()
RW

Otherwise
ImplementationDefined

B.3.3 EDPRCR, External Debug Power/Reset Control Register

Controls the PE functionality related to powerup, reset, and powerdown.

Configurations
If FEAT_DoPD is implemented then all fields in this register are in the Core power domain.

CORENPDRQ is the only field that is mapped between the EDPRCR and DBGPRCR and DBGPRCR_EL1.

Attributes

Width
32

Component
Debug

Register offset
0x310

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-6: ext_edprcr bit assignments

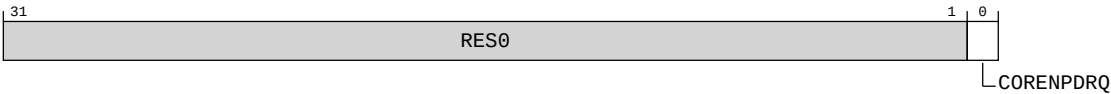


Table B-15: EDPRCR bit descriptions

Bits	Name	Description	Reset
[31:1]	RES0	Reserved	RES0
[0]	CORENPDRQ	Core no powerdown request. Requests emulation of powerdown. 0b0 If the system responds to a powerdown request, it powers down Core power domain. 0b1 If the system responds to a powerdown request, it does not powerdown the Core power domain, but instead emulates a powerdown of that domain. This field is in the Core power domain, and permitted accesses to this field map to the AArch64-DBGPRCR_EL1.CORENPDRQ field. When OSLockStatus() Access to this field is: UNKNOWN/WI When SoftwareLockStatus() Access to this field is: RO Otherwise Access to this field is: RW	x ²

Accessibility

On permitted accesses to the register, other access controls affect the behavior of some fields. See the field descriptions for more information.

Component	Offset	Instance	Range
Debug	0x310	EDPRCR	None

This interface is accessible as follows:

When IsCorePowered() && SoftwareLockStatus()

RO

When IsCorePowered() && !SoftwareLockStatus()

RW

Otherwise

ERROR

² On a Cold reset, if the powerup request is implemented and the powerup request has been asserted, this field is an **IMPLEMENTATION DEFINED** choice of 0 or 1. If the powerup request is not asserted, this field is set to 0.

B.3.4 MIDR_EL1, Main ID Register

Provides identification information for the PE, including an implementer code for the device and a device ID number.

Configurations

External register MIDR_EL1 bits [31:0] are architecturally mapped to AArch64 System register [A.7.1 MIDR_EL1, Main ID Register](#) on page 398 bits [31:0].

Attributes

Width

32

Component

Debug

Register offset

0xD00

Access type

See bit descriptions

Reset value

0100 0001 0000 1111 1101 1000 1110 0001

Bit descriptions

Figure B-7: ext_midr_el1 bit assignments

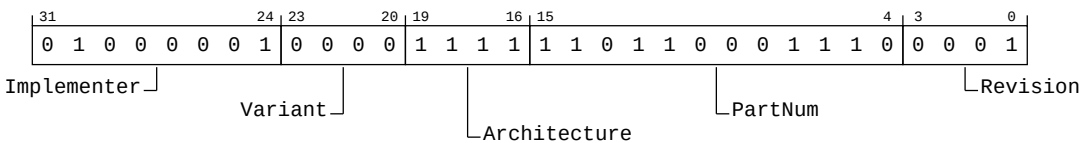


Table B-17: MIDR_EL1 bit descriptions

Bits	Name	Description	Reset
[31:24]	Implementer	The Implementer code. This field must hold an implementer code that has been assigned by Arm. Assigned codes include the following: 0b01000001 Arm Limited.	0x41
[23:20]	Variant	Variant number. Typically, this field is used to distinguish between different product variants, or major revisions of a product. 0b0000	0b0000
[19:16]	Architecture	Architecture version. Defined values are: 0b1111 Architectural features are individually identified in the ID_* registers.	0b1111

Bits	Name	Description	Reset
[15:4]	PartNum	Primary Part Number for the device. On processors implemented by Arm, if the top four bits of the primary part number are 0x0 or 0x7, the variant and architecture are encoded differently. 0b110110001110 Neoverse N3	0xD8E
[3:0]	Revision	Indicates the minor revision of the product. 0b0001 rOp1	0b0001

Accessibility

Component	Offset	Instance	Range
Debug	0xD00	MIDR_EL1	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus()
RO

Otherwise
ImplementationDefined

B.3.5 EDPFR, External Debug Processor Feature Register

Provides information about implemented PE features.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

Debug

Register offsets (2)

0xD20,0xD24

Access type

See bit descriptions

Reset value

```
xxxx xxxx xxxx xxxx 0001 xxxx 0001 0001 xxxx xxxx 0001 0001 0001 0001 0001 0001
```




Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-8: ext_edpfr bit assignments

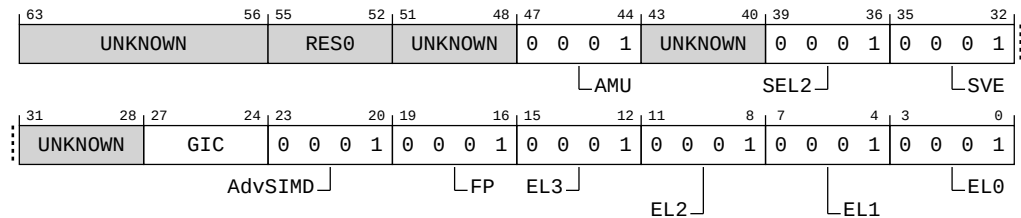


Table B-19: EDPFR bit descriptions

Bits	Name	Description	Reset
[63:56]	UNKNOWN	Reserved	UNKNOWN
[55:52]	RES0	Reserved	RES0
[51:48]	UNKNOWN	Reserved	UNKNOWN
[47:44]	AMU	Indicates support for Activity Monitors Extension. Defined values are: 0b0001 FEAT_AMUv1 is implemented.	0b0001
[43:40]	UNKNOWN	Reserved	UNKNOWN
[39:36]	SEL2	Secure EL2. Defined values are: 0b0001 Secure EL2 is implemented.	0b0001
[35:32]	SVE	Scalable Vector Extension. Defined values are: 0b0001 SVE is implemented.	0b0001
[31:28]	UNKNOWN	Reserved	UNKNOWN
[27:24]	GIC	System register GIC interface support. Defined values are: 0b0000 GIC CPU interface system registers not implemented. 0b0011 System register interface to version 4.1 of the GIC CPU interface is supported.	The reset values can be the following: 0b0000, 0b0011, respective to the value.

Bits	Name	Description	Reset
[23:20]	AdvSIMD	Advanced SIMD. Defined values are: 0b0001 Advanced SIMD is implemented, including support for the following SIMD and SIMD operations: - Integer byte, halfword, word and doubleword element operations. - Half-precision, single-precision and double-precision floating-point arithmetic. - Conversions between single-precision and half-precision data types, and double-precision and half-precision data types.	0b0001
[19:16]	FP	Floating-point. Defined values are: 0b0001 Floating-point is implemented, and includes support for: - Half-precision, single-precision and double-precision floating-point types. - Conversions between single-precision and half-precision data types, and double-precision and half-precision data types.	0b0001
[15:12]	EL3	AArch64 EL3 Exception level handling. Defined values are: 0b0001 EL3 can be executed in AArch64 state only.	0b0001
[11:8]	EL2	AArch64 EL2 Exception level handling. Defined values are: 0b0001 EL2 can be executed in AArch64 state only.	0b0001
[7:4]	EL1	AArch64 EL1 Exception level handling. Defined values are: 0b0001 EL1 can be executed in AArch64 state only.	0b0001
[3:0]	ELO	AArch64 ELO Exception level handling. Defined values are: 0b0001 ELO can be executed in AArch64 state only.	0b0001

Accessibility

Component	Offset	Instance	Range
Debug	0xD20	EDPFR	31:0

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus()

RO

Otherwise

ImplementationDefined

Component	Offset	Instance	Range
Debug	0xD24	EDPFR	63:32

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus()
RO

Otherwise
ImplementationDefined

B.3.6 EDDFR, External Debug Feature Register

Provides top-level information about the debug system.



Debuggers must use ext-EDDEVARCH to determine the Debug architecture version.

For general information about the interpretation of the ID registers, see *Principles of the ID scheme for fields in ID registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width
64

Component
Debug

Register offsets (2)
0xD28,0xD2C

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	0001	xxxx	xxxx	0001	xxxx	0011	xxxx	0101	0111	0001	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-9: ext_eddfr bit assignments

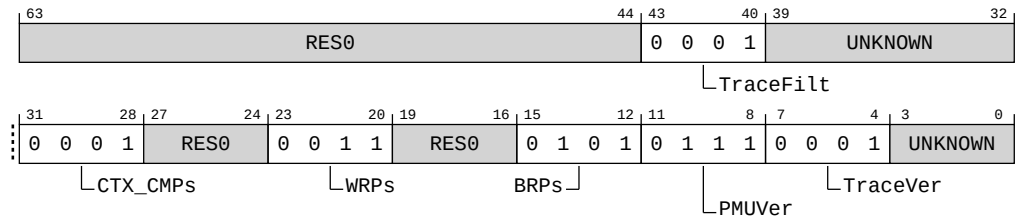


Table B-22: EDDFR bit descriptions

Bits	Name	Description	Reset
[63:44]	RES0	Reserved	RES0
[43:40]	TraceFilt	Armv8.4 Self-hosted Trace Extension version. Defined values are: 0b0001 Armv8.4 Self-hosted Trace Extension is implemented.	0b0001
[39:32]	UNKNOWN	Reserved	UNKNOWN
[31:28]	CTX_CMPS	Number of breakpoints that are context-aware, minus 1. These are the highest numbered breakpoints. In an Armv8-A implementation that supports AArch64, this field returns the value of AArch64-ID_AA64DFR0_EL1.CTX_CMPS. 0b0001 Two context-aware breakpoints are included	0b0001
[27:24]	RES0	Reserved	RES0
[23:20]	WRPs	Number of watchpoints, minus 1. The value of 0b0000 is reserved. In an Armv8-A implementation that supports AArch64, this field returns the value of AArch64-ID_AA64DFR0_EL1.WRPs. 0b0011 Four watchpoints	0b0011
[19:16]	RES0	Reserved	RES0
[15:12]	BRPs	Number of breakpoints, minus 1. The value of 0b0000 is reserved. In an Armv8-A implementation that supports AArch64, this field returns the value of AArch64-ID_AA64DFR0_EL1.BRPs. 0b0101 Six breakpoints	0b0101
[11:8]	PMUVer	Performance Monitors Extension version. Defined value is: 0b0111 Performance Monitors Extension implemented, PMUv3 for Armv8.7	0b0111
[7:4]	TraceVer	Trace support. Indicates whether System register interface to a PE trace unit is implemented. Defined values are: 0b0001 PE trace unit System registers implemented.	0b0001

Bits	Name	Description	Reset
[3:0]	UNKNOWN	Reserved	UNKNOWN

Accessibility

Component	Offset	Instance	Range
Debug	0xD28	EDDFR	31:0

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus()

RO

Otherwise

ImplementationDefined

Component	Offset	Instance	Range
Debug	0xD2C	EDDFR	63:32

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus()

RO

Otherwise

ImplementationDefined

B.3.7 EDDEVARCH, External Debug Device Architecture register

Identifies the programmers' model architecture of the external debug component.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain.

If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width

32

Component

Debug

Register offset

0xFBC

Access type

See bit descriptions

Reset value

0100	0111	0111	0000	1001	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-10: ext_eddevarch bit assignments

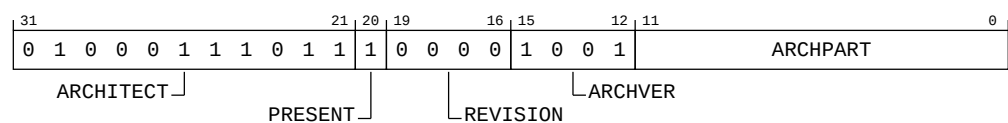


Table B-25: EDDEVARCH bit descriptions

Bits	Name	Description	Reset
[31:21]	ARCHITECT	Defines the architecture of the component. For debug, this is Arm Limited. Bits [31:28] are the JEP106 continuation code, 0x4. Bits [27:21] are the JEP106 ID code, 0x3B. 0b01000111011	0b01000111011
[20]	PRESENT	Indicates that the DEVARCH is present. 0b1	0b1
[19:16]	REVISION	Defines the architecture revision. For architectures defined by Arm this is the minor revision. For debug, the revision defined by Armv8 is 0x0. All other values are reserved. 0b0000	0b0000
[15:12]	ARCHVER	Architecture Version. Defines the architecture version of the component. Defined values are: 0b1001 Armv8.4 debug architecture, FEAT_Debugv8p4.	0b1001

Bits	Name	Description	Reset
[11:0]	ARCHPART	Architecture Part. Defines the architecture of the component. 0b101000010101 Armv8-A debug architecture. EDDEVARCH.ARCHVER and EDDEVARCH.ARCHPART are also defined as a single field, EDDEVARCH.ARCHID, so that EDDEVARCH.ARCHPART is EDDEVARCH.ARCHID[11:0]. Armv8-A debug architecture.	The reset values can be the following: 0b101000010101, respective to the value.

Accessibility

Component	Offset	Instance	Range
Debug	0xFBC	EDDEVARCH	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.8 EDDEVID2, External Debug Device ID register 2

Reserved for future descriptions of features of the debug implementation.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width

32

Component

Debug

Register offset

0xFC0

Access type

See bit descriptions

Reset value





Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-11: ext_eddevvid2 bit assignments

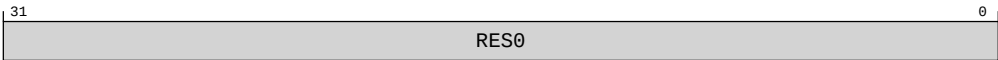


Table B-27: EDDEVID2 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
Debug	0xFC0	EDDEVID2	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.9 EDDEVID1, External Debug Device ID register 1

Provides extra information for external debuggers about features of the debug implementation.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain.

If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width

32

Component

Debug

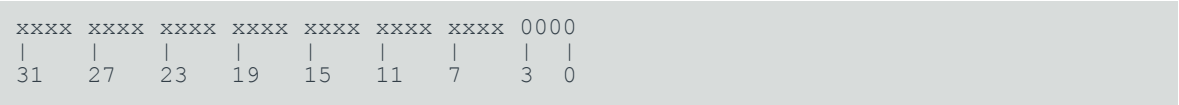
Register offset

0xFC4

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-12: ext_eddevid1 bit assignments



Table B-29: EDDEVID1 bit descriptions

Bits	Name	Description	Reset
[31:4]	RES0	Reserved	RES0
[3:0]	PCSRoffset	This field indicates the offset applied to PC samples returned by reads of ext-EDPCSR. Permitted values of this field in Armv8 are: 0b0000 ext-EDPCSR not implemented.	0b0000

Accessibility

Component	Offset	Instance	Range
Debug	0xFC4	EDDEVID1	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.10 EDDEVID, External Debug Device ID register 0

Provides extra information for external debuggers about features of the debug implementation.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain.

If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width

32

Component

Debug

Register offset

0xFC8

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-13: ext_eddevid bit assignments

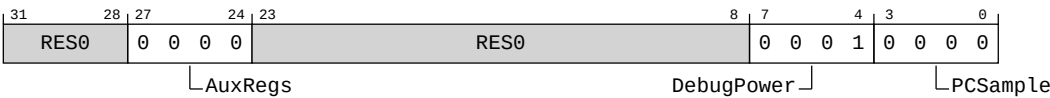


Table B-31: EDDEVID bit descriptions

Bits	Name	Description	Reset
[31:28]	RES0	Reserved	RES0
[27:24]	AuxRegs	Indicates support for Auxiliary registers. Defined values are: 0b0000 None supported.	0b0000
[23:8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:4]	DebugPower	Indicates support for the FEAT_DoPD feature. Defined values are: 0b0001 FEAT_DoPD implemented. All registers in the external debug interface register map are implemented in the Core power domain.	0b0001
[3:0]	PCSample	Indicates the level of PC Sample-based Profiling support using external debug registers. Defined values are: 0b0000 PC Sample-based Profiling Extension is not implemented in the external debug registers space.	0b0000

Accessibility

Component	Offset	Instance	Range
Debug	0xFC8	EDDEVID	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.11 EDDEVTYPE, External Debug Device Type register

Indicates to a debugger that this component is part of a PEs debug logic.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width

32

Component

Debug

Register offset

0xFCC

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-14: ext_eddevtype bit assignments



Table B-33: EDDEVTYPE bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SUB	Subtype. Indicates this is a component within a PE. 0b0001	0b0001
[3:0]	MAJOR	Major type. Indicates this is a debug logic component. 0b0101	0b0101

Accessibility

Component	Offset	Instance	Range
Debug	0xFCC	EDDEVTYPE	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.12 EDPIDR4, External Debug Peripheral Identification Register 4

Provides information to identify an external debug component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

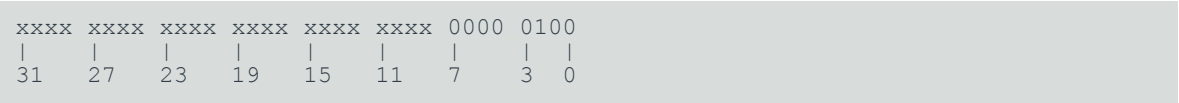
Register offset

0xFD0

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-15: ext_edpidr4 bit assignments



Table B-35: EDPIDR4 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SIZE	Size of the component. Log ₂ of the number of 4KB pages from the start of the component to the end of the component ID registers. 0b0000	0b0000
[3:0]	DES_2	Designer, JEP106 continuation code, least significant nibble. For Arm Limited, this field is 0b0100. 0b0100 Arm Limited	0b0100

Accessibility

Component	Offset	Instance	Range
Debug	0xFD0	EDPIDR4	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.13 EDPIDR0, External Debug Peripheral Identification Register 0

Provides information to identify an external debug component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

Register offset

0xFE0

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1000	1110
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-16: ext_edpidr0 bit assignments

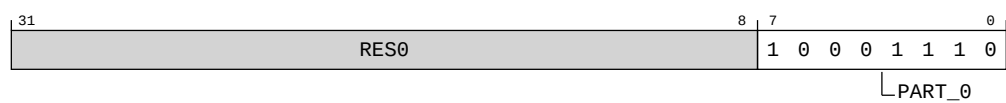


Table B-37: EDPIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PART_0	Part number, least significant byte. 0b10001110 Neoverse N3	0x8E

Accessibility

Component	Offset	Instance	Range
Debug	0xFE0	EDPIDR0	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.14 EDPIDR1, External Debug Peripheral Identification Register 1

Provides information to identify an external debug component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

Register offset

0xFE4

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1011	1101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-17: ext_edpidr1 bit assignments

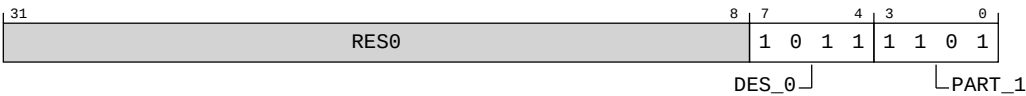


Table B-39: EDPIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	DES_0	Designer, least significant nibble of JEP106 ID code. For Arm Limited, this field is 0b1011. 0b1011 Arm Limited	0b1011
[3:0]	PART_1	Part number, most significant nibble. 0b1101 Neoverse N3	0b1101

Accessibility

Component	Offset	Instance	Range
Debug	0xFE4	EDPIDR1	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.15 EDPIDR2, External Debug Peripheral Identification Register 2

Provides information to identify an external debug component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

Register offset

0xFE8

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-18: ext_edpidr2 bit assignments

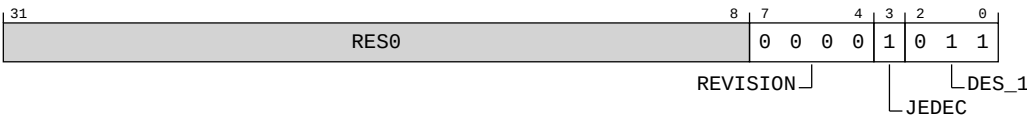


Table B-41: EDPIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:4]	REVISION	Part major revision. Parts can also use this field to extend Part number to 16-bits. 0b0000 rOp1	0b0000
[3]	JEDEC	Indicates a JEP106 identity code is used. 0b1	0b1
[2:0]	DES_1	Designer, most significant bits of JEP106 ID code. For Arm Limited, this field is 0b011. 0b011 Arm Limited	0b011

Accessibility

Component	Offset	Instance	Range
Debug	0xFE8	EDPIDR2	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.16 EDPIDR3, External Debug Peripheral Identification Register 3

Provides information to identify an external debug component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

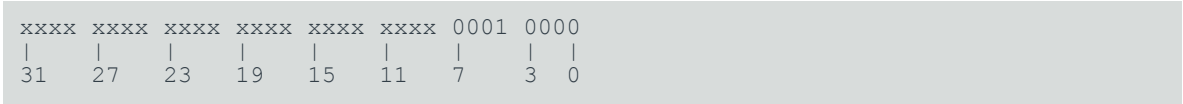
Register offset

0xFEC

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-19: ext_edpidr3 bit assignments



Table B-43: EDPIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVAND	Part minor revision. Parts using ext-EDPIDR2.REVISION as an extension to the Part number must use this field as a major revision number. 0b0001 rOp1	0b0001
[3:0]	CMOD	Customer modified. Indicates someone other than the Designer has modified the component. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
Debug	0xFEC	EDPIDR3	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.17 EDCIDR0, External Debug Component Identification Register 0

Provides information to identify an external debug component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

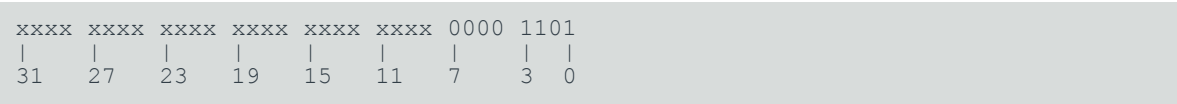
Register offset

0xFF0

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-20: ext_edcidr0 bit assignments

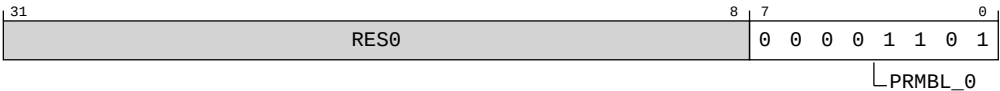


Table B-45: EDCIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:0]	PRMBL_0	Preamble. 0b00001101	0x0D

Accessibility

Component	Offset	Instance	Range
Debug	0xFF0	EDCIDR0	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.18 EDCIDR1, External Debug Component Identification Register 1

Provides information to identify an external debug component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

Register offset

0xFF4

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1001	0000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-21: ext_edcldr1 bit assignments



Table B-47: EDCIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	CLASS	Component class. 0b1001 CoreSight component. Other values are defined by the CoreSight Architecture. This field reads as 0x9.	0b1001
[3:0]	PRMBL_1	Preamble. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
Debug	0xFF4	EDCIDR1	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.19 EDCIDR2, External Debug Component Identification Register 2

Provides information to identify an external debug component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

Register offset

0xFF8

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-22: ext_edcldr2 bit assignments



Table B-49: EDCIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_2	Preamble. 0b00000101	0x05

Accessibility

Component	Offset	Instance	Range
Debug	0xFF8	EDCIDR2	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.3.20 EDCIDR3, External Debug Component Identification Register 3

Provides information to identify an external debug component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

Debug

Register offset

0xFFC

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1011	0001
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-23: ext_edcldr3 bit assignments

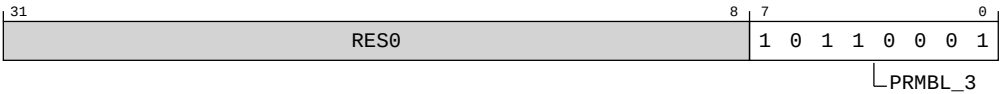


Table B-51: EDCIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_3	Preamble. 0b10110001	0xB1

Accessibility

Component	Offset	Instance	Range
Debug	0xFFC	EDCIDR3	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4 External PMU registers summary

The summary table provides an overview of all memory-mapped PMU registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-53: PMU registers summary

Offset	Name	Reset	Width	Description
0x0	PMEVCNTR0_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x8	PMEVCNTR1_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x10	PMEVCNTR2_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x18	PMEVCNTR3_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x20	PMEVCNTR4_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers

Offset	Name	Reset	Width	Description
0x28	PMEVCNTR5_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x30	PMEVCNTR6_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x38	PMEVCNTR7_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x40	PMEVCNTR8_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x48	PMEVCNTR9_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x50	PMEVCNTR10_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x58	PMEVCNTR11_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x60	PMEVCNTR12_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x68	PMEVCNTR13_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x70	PMEVCNTR14_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x78	PMEVCNTR15_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x80	PMEVCNTR16_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x88	PMEVCNTR17_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x90	PMEVCNTR18_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x98	PMEVCNTR19_ELO	See individual bit resets.	64-bit	Performance Monitors Event Count Registers
0x0F8	PMCCNTR_ELO [31:0]	See individual bit resets.	32-bit	Performance Monitors Cycle Counter
0x0FC	PMCCNTR_ELO [63:32]	See individual bit resets.	32-bit	Performance Monitors Cycle Counter
0x200	PMPCSR [31:0]	See individual bit resets.	32-bit	Program Counter Sample Register
0x204	PMPCSR [63:32]	See individual bit resets.	32-bit	Program Counter Sample Register
0x220	PMPCSR [31:0]	See individual bit resets.	32-bit	Program Counter Sample Register
0x224	PMPCSR [63:32]	See individual bit resets.	32-bit	Program Counter Sample Register
0x208	PMCID1SR	See individual bit resets.	32-bit	CONTEXTIDR_EL1 Sample Register
0x228	PMCID1SR	See individual bit resets.	32-bit	CONTEXTIDR_EL1 Sample Register
0x20C	PMVIDSR	See individual bit resets.	32-bit	VMID Sample Register
0x22C	PMCID2SR	See individual bit resets.	32-bit	CONTEXTIDR_EL2 Sample Register
0x400	PMEVTYPER0_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x404	PMEVTYPER1_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x408	PMEVTYPER2_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x40C	PMEVTYPER3_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x410	PMEVTYPER4_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x414	PMEVTYPER5_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x418	PMEVTYPER6_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x41C	PMEVTYPER7_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x420	PMEVTYPER8_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x424	PMEVTYPER9_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x428	PMEVTYPER10_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x42C	PMEVTYPER11_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x430	PMEVTYPER12_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x434	PMEVTYPER13_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x438	PMEVTYPER14_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers

Offset	Name	Reset	Width	Description
0x43C	PMEVTYPEPER15_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x440	PMEVTYPEPER16_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x444	PMEVTYPEPER17_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x448	PMEVTYPEPER18_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x44C	PMEVTYPEPER19_ELO	See individual bit resets.	32-bit	Performance Monitors Event Type Registers
0x47C	PMCCFILTR_ELO	See individual bit resets.	32-bit	Performance Monitors Cycle Counter Filter Register
0x600	PMPCSSR	See individual bit resets.	64-bit	Snapshot Program Counter Sample Register
0x608	PMCIDSSR	See individual bit resets.	32-bit	Snapshot CONTEXTIDR_EL1 Sample Register
0x60C	PMCID2SSR	See individual bit resets.	32-bit	Snapshot CONTEXTIDR_EL2 Sample Register
0x610	PMSSSR	See individual bit resets.	32-bit	PMU Snapshot Status Register
0x618	PMCCNTSR	See individual bit resets.	64-bit	PMU Cycle Counter Snapshot Register
0x620	PMEVCNTSR0	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x628	PMEVCNTSR1	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x630	PMEVCNTSR2	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x638	PMEVCNTSR3	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x640	PMEVCNTSR4	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x648	PMEVCNTSR5	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x650	PMEVCNTSR6	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x658	PMEVCNTSR7	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x660	PMEVCNTSR8	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x668	PMEVCNTSR9	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x670	PMEVCNTSR10	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x678	PMEVCNTSR11	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x680	PMEVCNTSR12	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x688	PMEVCNTSR13	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x690	PMEVCNTSR14	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x698	PMEVCNTSR15	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6A0	PMEVCNTSR16	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6A8	PMEVCNTSR17	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6B0	PMEVCNTSR18	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0x6B8	PMEVCNTSR19	See individual bit resets.	64-bit	PMU Event Counter Snapshot Register
0xC00	PMCNTENSET_ELO	See individual bit resets.	32-bit	Performance Monitors Count Enable Set register
0xC20	PMCNTENCLR_ELO	See individual bit resets.	32-bit	Performance Monitors Count Enable Clear register
0xC40	PMINTENSET_EL1	See individual bit resets.	32-bit	Performance Monitors Interrupt Enable Set register
0xC60	PMINTENCLR_EL1	See individual bit resets.	32-bit	Performance Monitors Interrupt Enable Clear register
0xC80	PMOVSLR_ELO	See individual bit resets.	32-bit	Performance Monitors Overflow Flag Status Clear register
0xCA0	PMSWINC_ELO	See individual bit resets.	32-bit	Performance Monitors Software Increment register
0xCC0	PMOVSSET_ELO	See individual bit resets.	32-bit	Performance Monitors Overflow Flag Status Set register
0xE00	PMCFGR	See individual bit resets.	32-bit	Performance Monitors Configuration Register
0xE04	PMCR_ELO	See individual bit resets.	32-bit	Performance Monitors Control Register

Offset	Name	Reset	Width	Description
0xE20	PMCEID0	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 0
0xE24	PMCEID1	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 1
0xE28	PMCEID2	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 2
0xE2C	PMCEID3	See individual bit resets.	32-bit	Performance Monitors Common Event Identification register 3
0xE30	PMSSCR	See individual bit resets.	32-bit	PMU Snapshot Capture Register
0xE40	PMMIR	See individual bit resets.	32-bit	Performance Monitors Machine Identification Register
0xFA8	PMDEVAFF0	See individual bit resets.	32-bit	Performance Monitors Device Affinity register 0
0xFAC	PMDEVAFF1	See individual bit resets.	32-bit	Performance Monitors Device Affinity register 1
0xFB0	PMLAR	See individual bit resets.	32-bit	Performance Monitors Lock Access Register
0xFB4	PMLSR	See individual bit resets.	32-bit	Performance Monitors Lock Status Register
0xFB8	PMAUTHSTATUS	See individual bit resets.	32-bit	Performance Monitors Authentication Status register
0xFBC	PMDEVARCH	See individual bit resets.	32-bit	Performance Monitors Device Architecture register
0xFC8	PMDEVID	See individual bit resets.	32-bit	Performance Monitors Device ID register
0xFCC	PMDEVTYPE	See individual bit resets.	32-bit	Performance Monitors Device Type register
0xFD0	PMPIDR4	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 4
0xFE0	PMPIDR0	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 0
0xFE4	PMPIDR1	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 1
0xFE8	PMPIDR2	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 2
0xFEC	PMPIDR3	See individual bit resets.	32-bit	Performance Monitors Peripheral Identification Register 3
0xFF0	PMCIDR0	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 0
0xFF4	PMCIDR1	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 1
0xFF8	PMCIDR2	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 2
0xFFC	PMCIDR3	See individual bit resets.	32-bit	Performance Monitors Component Identification Register 3

B.4.1 PMPCSSR, Snapshot Program Counter Sample Register

Captured copy of the Program Counter.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x600

Access type

Read

R

Write

RESERVED

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-24: ext_pmpcssr bit assignments

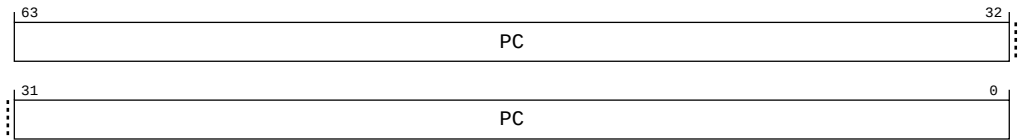


Table B-54: PMPCSSR bit descriptions

Bits	Name	Description	Reset
[63:0]	PC	Sampled PC. The instruction address for the sampled instruction. The sampled instruction is an instruction recently architecturally executed by the PE. PCSR is not updated before the first read. It is then updated each time an instruction is executed. Note: The ARM architecture does not define recently executed.	64 {x}

Accessibility

PMPCSSR is set to an UNKNOWN value by a read of the EDPCSRlo or PMPCSR[31:0] register. This interface is accessible as follows:

RO

B.4.2 PMCIDSSR, Snapshot CONTEXTIDR_EL1 Sample Register

Captured copy of the CONTEXTIDR_EL1 register.

The value captured must relate to the instruction captured in PMPCSSR.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

PMU

Register offset

0x608

Access type

Read

R

Write

RESERVED

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-25: ext_pmcidssr bit assignments

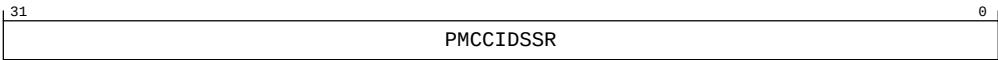


Table B-55: PMCIDSSR bit descriptions

Bits	Name	Description	Reset
[31:0]	PMCCIDSSR	PMCIDSR sample. Sampled CONTEXTIDR_EL1 snapshot.	32 {x}

Accessibility

PMCIDSSR is set to an UNKNOWN value by a read of the EDPCSRlo or PMPCSR[31:0] register.
This interface is accessible as follows:

RO

B.4.3 PMCID2SSR, Snapshot CONTEXTIDR_EL2 Sample Register

Captured copy of the CONTEXTIDR_EL2 register.

The value captured must relate to the instruction captured in PMPCSSR.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

PMU

Register offset

0x60C

Access type

Read

R

Write

RESERVED

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-26: ext_pmcid2ssr bit assignments

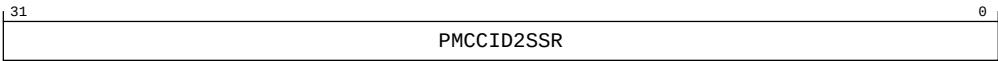


Table B-56: PMCID2SSR bit descriptions

Bits	Name	Description	Reset
[31:0]	PMCCID2SSR	PMCID2SR sample. Sampled CONTEXTIDR_EL2 snapshot.	32 {x}

Access

If ARMv8.2 is not implemented, this location is used for PMVIDSSR.

PMCID2SSR is set to an **UNKNOWN** value by a read of the EDPCSRlo or PMPCSR[31:0] register.

Accessibility

If ARMv8.2 is not implemented, this location is used for PMVIDSSR.

PMCID2SSR is set to an UNKNOWN value by a read of the EDPCSRlo or PMPCSR[31:0] register.
This interface is accessible as follows:

RO

B.4.4 PMSSSR, PMU Snapshot Status Register

Holds status information about the captured counters.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

PMU

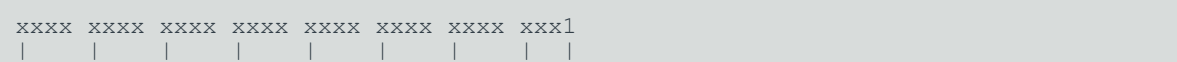
Register offset

0x610

Access type

RO

Reset value



312723191511730



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-27: ext_pmsssr bit assignments

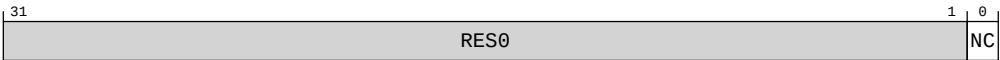


Table B-57: PMSSSR bit descriptions

Bits	Name	Description	Reset
[31:1]	RES0	Reserved	RES0
[0]	NC	No capture. Indicates whether the PMU counters have been captured. 0b0 PMU counters captured. 0b1 PMU counters not captured. The event counters are only not captured by the PE in the event of a security violation. The external Monitor is responsible for keeping track of whether it managed to capture the snapshot registers from the PE. PMSSSR.NC does not reflect the status of the captured Program Counter Sample registers. PMSSSR.NC is reset to 1 by PE Warm reset, but is overwritten at the first capture. Tools need to be aware that capturing over reset or power-down might lose data, as they are reliant on software saving and restoring the PMU state (including PMSSCR). There is no sampled sticky reset bit.	0b1

Accessibility

This interface is accessible as follows:

RO

B.4.5 PMCCNTRSR, PMU Cycle Counter Snapshot Register

Captured copy of PMCCNTR_ELO. Once captured, the value in PMCCNTRSR is unaffected by writes to PMCCNTR_ELO and PMCR_ELO.C.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

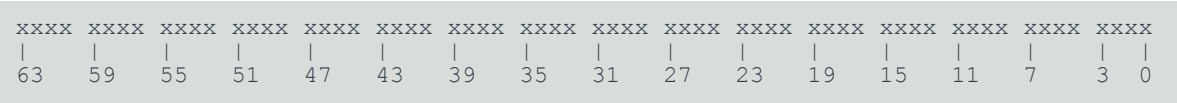
Register offset

0x618

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-28: ext_pmcntr bit assignments

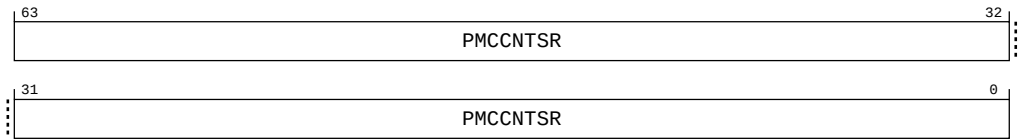


Table B-58: PMCCNTR bit descriptions

Bits	Name	Description	Reset
[63:0]	PMCCNTR	PMCCNTR_ELO sample. Sampled cycle count.	64 { x }

Accessibility

This interface is accessible as follows:

RO

B.4.6 PMEVCNTR0, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

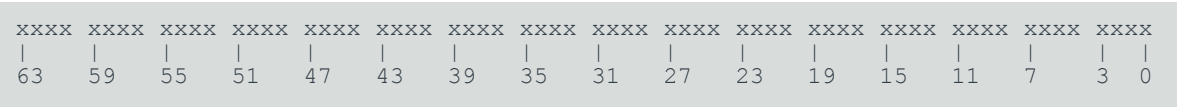
Register offset

0x620

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-29: ext_pmevcntr0 bit assignments

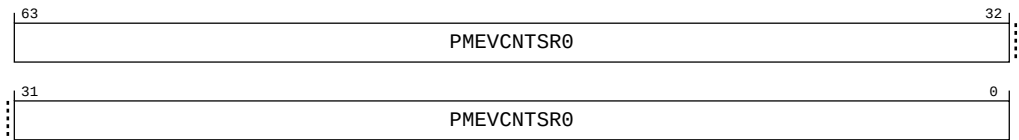


Table B-59: PMEVCNTR0 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR0	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x620	PMEVCNTR0	None

This interface is accessible as follows:

RO

B.4.7 PMEVCNTR1, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x628

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-30: ext_pmevcntr1 bit assignments

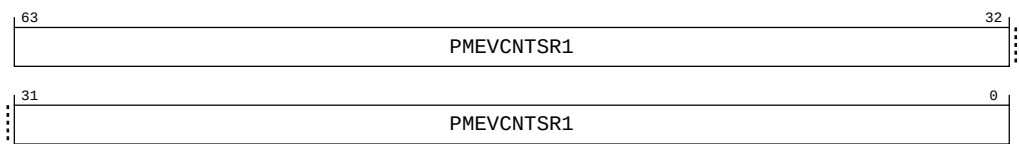


Table B-61: PMEVCNTR1 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR1	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x628	PMEVCNTR1	None

This interface is accessible as follows:

RO

B.4.8 PMEVCNTR2, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

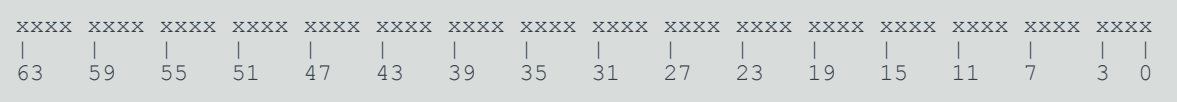
Register offset

0x630

Access type

RO

Reset value





Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-31: ext_pmevcntr2 bit assignments

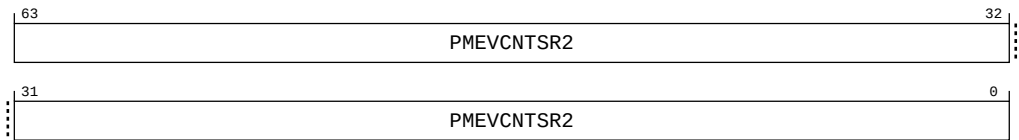


Table B-63: PMEVCNTR2 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR2	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x630	PMEVCNTR2	None

This interface is accessible as follows:

RO

B.4.9 PMEVCNTR3, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x638

Access type
RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-32: ext_pmevcntr3 bit assignments

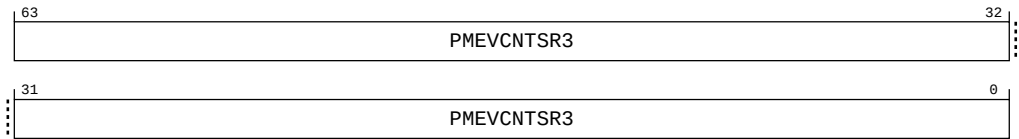


Table B-65: PMEVCNTR3 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR3	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x638	PMEVCNTR3	None

This interface is accessible as follows:

RO

B.4.10 PMEVCNTR4, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

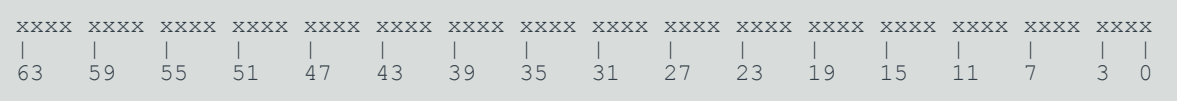
Register offset

0x640

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-33: ext_pmevcntr4 bit assignments

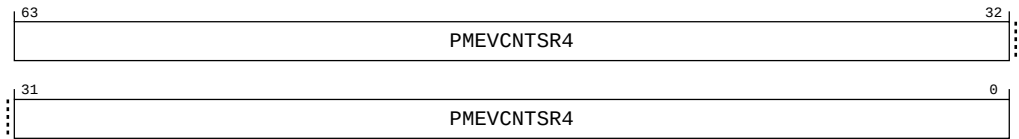


Table B-67: PMEVCNTR4 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR4	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x640	PMEVCNTR4	None

This interface is accessible as follows:

RO

B.4.11 PMEVCNTR5, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

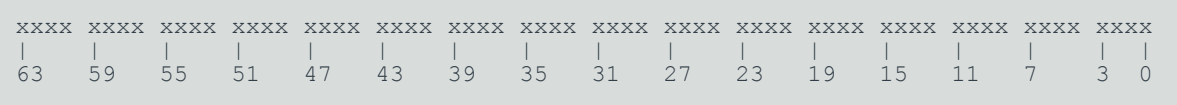
Register offset

0x648

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-34: ext_pmevcntr5 bit assignments

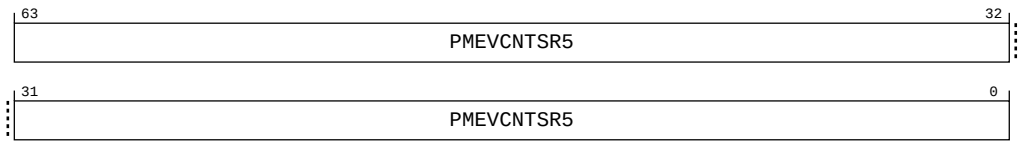


Table B-69: PMEVCNTR5 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR5	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x648	PMEVCNTR5	None

This interface is accessible as follows:

RO

B.4.12 PMEVCNTR6, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x650

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-35: ext_pmevcntrs6 bit assignments

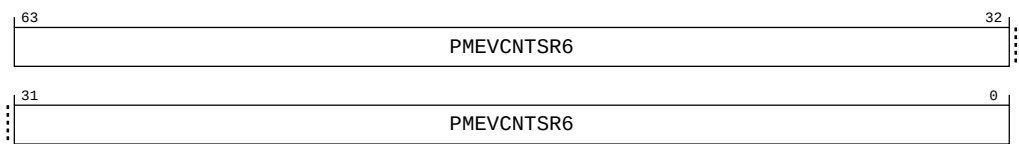


Table B-71: PMEVCNTR6 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR6	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x650	PMEVCNTR6	None

This interface is accessible as follows:

RO

B.4.13 PMEVCNTR7, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

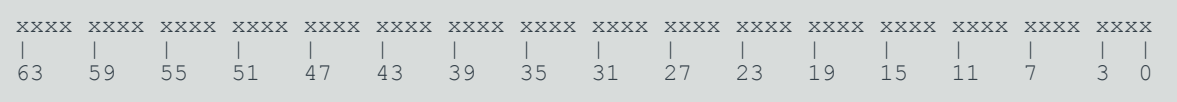
Register offset

0x658

Access type

RO

Reset value





Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-36: ext_pmevcntr7 bit assignments

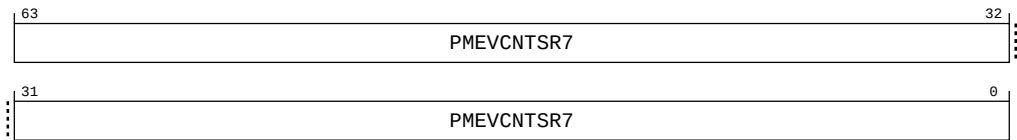


Table B-73: PMEVCNTR7 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR7	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x658	PMEVCNTR7	None

This interface is accessible as follows:

RO

B.4.14 PMEVCNTR8, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x660

Access type
RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-37: ext_pmevcntrs8 bit assignments

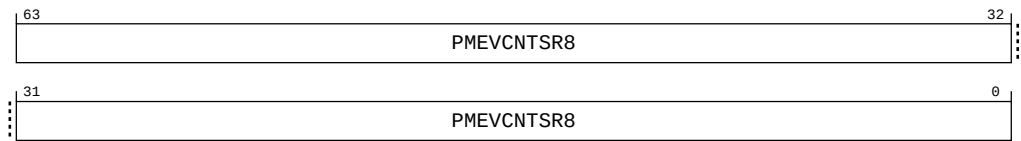


Table B-75: PMEVCNTRS8 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTRS8	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x660	PMEVCNTRS8	None

This interface is accessible as follows:

RO

B.4.15 PMEVCNTRS9, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

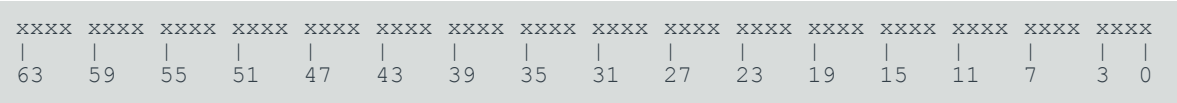
Register offset

0x668

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-38: ext_pmevcntr9 bit assignments

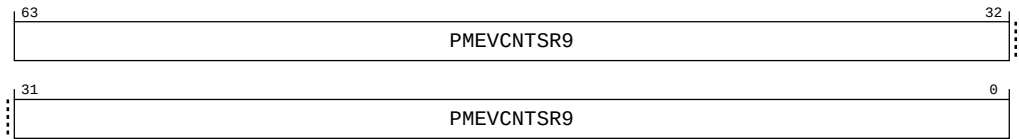


Table B-77: PMEVCNTR9 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR9	PMEVCNTR<n>_ELO sample. Sampled event count.	64 { x }

Accessibility

Component	Offset	Instance	Range
PMU	0x668	PMEVCNTR9	None

This interface is accessible as follows:

RO

B.4.16 PMEVCNTR10, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

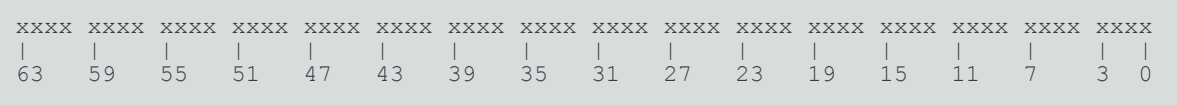
Register offset

0x670

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-39: ext_pmevcntr10 bit assignments

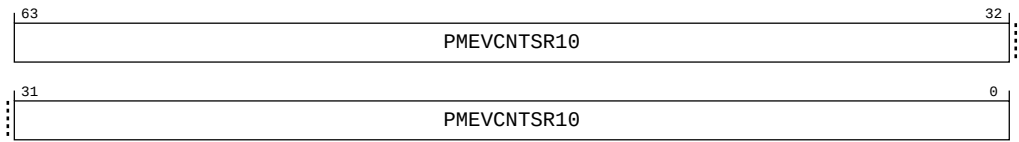


Table B-79: PMEVCNTR10 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR10	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x670	PMEVCNTR10	None

This interface is accessible as follows:

RO

B.4.17 PMEVCNTR11, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x678

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-40: ext_pmevcntr11 bit assignments

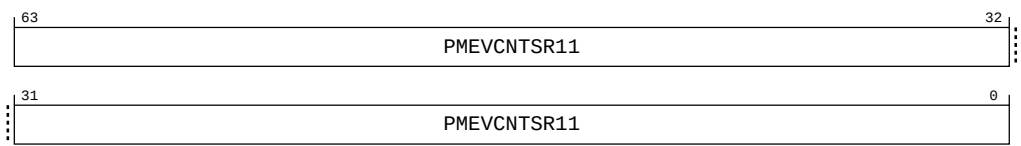


Table B-81: PMEVCNTR11 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR11	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x678	PMEVCNTR11	None

This interface is accessible as follows:

RO

B.4.18 PMEVCNTR12, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

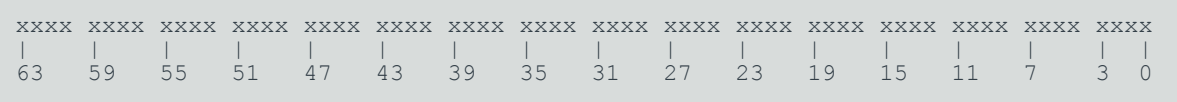
Register offset

0x680

Access type

RO

Reset value





Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-41: ext_pmevcntr12 bit assignments

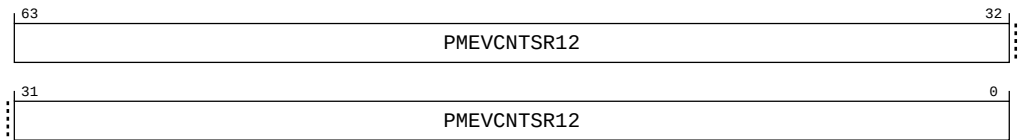


Table B-83: PMEVCNTR12 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR12	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x680	PMEVCNTR12	None

This interface is accessible as follows:

RO

B.4.19 PMEVCNTR13, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

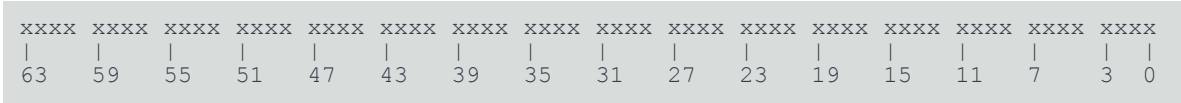
PMU

Register offset

0x688

Access type
RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-42: ext_pmevcntr13 bit assignments

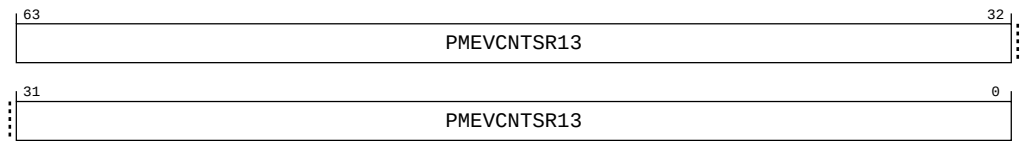


Table B-85: PMEVCNTR13 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR13	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x688	PMEVCNTR13	None

This interface is accessible as follows:

RO

B.4.20 PMEVCNTR14, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

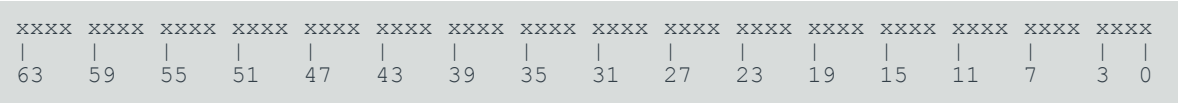
Register offset

0x690

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-43: ext_pmevcntr14 bit assignments

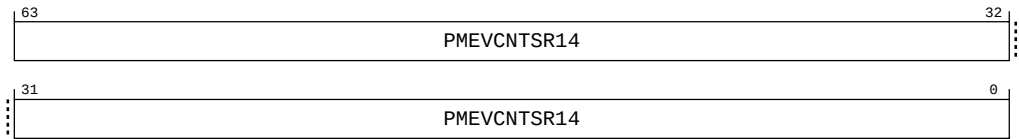


Table B-87: PMEVCNTR14 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR14	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x690	PMEVCNTR14	None

This interface is accessible as follows:

RO

B.4.21 PMEVCNTR15, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

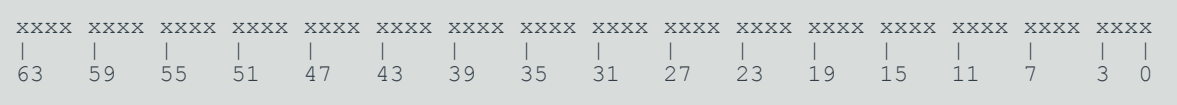
Register offset

0x698

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-44: ext_pmevcntr15 bit assignments

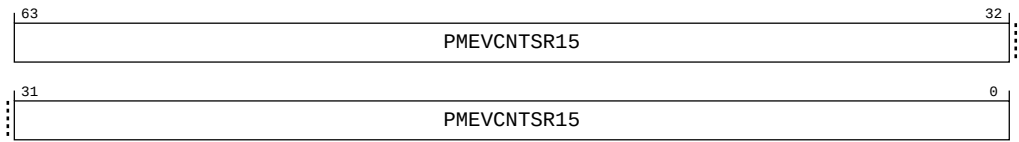


Table B-89: PMEVCNTR15 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR15	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x698	PMEVCNTR15	None

This interface is accessible as follows:

RO

B.4.22 PMEVCNTR16, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x6A0

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-45: ext_pmevcntr16 bit assignments

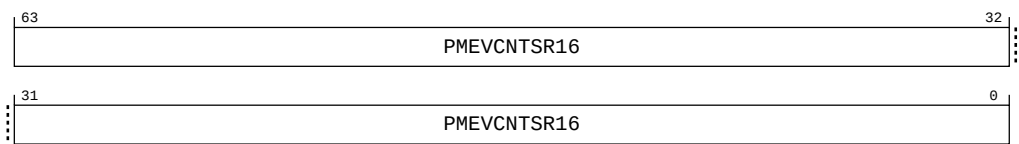


Table B-91: PMEVCNTR16 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR16	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x6A0	PMEVCNTR16	None

This interface is accessible as follows:

RO

B.4.23 PMEVCNTR17, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

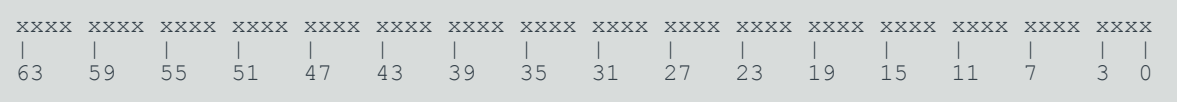
Register offset

0x6A8

Access type

RO

Reset value





Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-46: ext_pmevcntr17 bit assignments

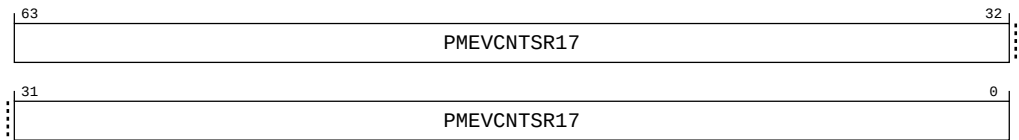


Table B-93: PMEVCNTR17 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR17	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x6A8	PMEVCNTR17	None

This interface is accessible as follows:

RO

B.4.24 PMEVCNTR18, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

Register offset

0x6B0

Access type
RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-47: ext_pmevcntrs18 bit assignments

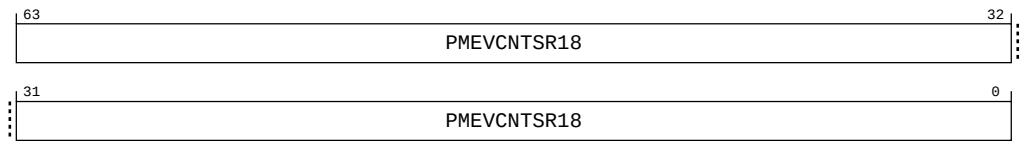


Table B-95: PMEVCNTR18 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR18	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x6B0	PMEVCNTR18	None

This interface is accessible as follows:

RO

B.4.25 PMEVCNTR19, PMU Event Counter Snapshot Register

Captured copy of PMEVCNTR<n>_ELO. Once captured, the value in PMSSEVCNTR<n> is unaffected by writes to PMSSEVCNTR<n>_ELO and PMCR_ELO.P.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

PMU

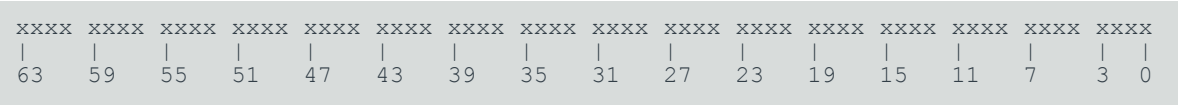
Register offset

0x6B8

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-48: ext_pmevcntsr19 bit assignments

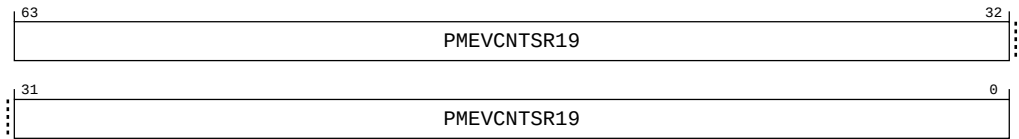


Table B-97: PMEVCNTR19 bit descriptions

Bits	Name	Description	Reset
[63:0]	PMEVCNTR19	PMEVCNTR<n>_ELO sample. Sampled event count.	64 {x}

Accessibility

Component	Offset	Instance	Range
PMU	0x6B8	PMEVCNTR19	None

This interface is accessible as follows:

RO

B.4.26 PMCFGR, Performance Monitors Configuration Register

Contains PMU-specific configuration data.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

PMU

Register offset

0xE00

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-49: ext_pmcfgr bit assignments

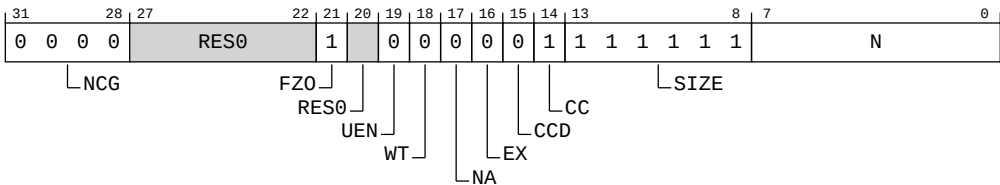


Table B-99: PMCFGR bit descriptions

Bits	Name	Description	Reset
[31:28]	NCG	This feature is not supported, so this field is RAZ . 0b0000	0b0000
[27:22]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[21]	FZO	Freeze-on-overflow supported. Defined values are: 0b1 Freeze-on-overflow mechanism is supported. ext-PMCR_ELO.FZO is RW.	0b1
[20]	RES0	Reserved	RES0
[19]	UEN	User-mode Enable Register supported. AArch64-PMUSERENR_ELO is not visible in the external debug interface, so this bit is RAZ . 0b0	0b0
[18]	WT	This feature is not supported, so this bit is RAZ . 0b0	0b0
[17]	NA	This feature is not supported, so this bit is RAZ . 0b0	0b0
[16]	EX	Export supported. Value is IMPLEMENTATION DEFINED . 0b0 Export Not supported	0b0
[15]	CCD	Cycle counter has prescale. This field is RAZ 0b0 ext-PMCR_ELO.D is RES0 .	0b0
[14]	CC	Dedicated cycle counter (counter 31) supported. 0b1	0b1
[13:8]	SIZE	Size of counters, minus one. This field defines the size of the largest counter implemented by the Performance Monitors Unit. From Armv8, the largest counter is 64-bits, so the value of this field is 0b111111. This field is used by software to determine the spacing of the counters in the memory-map. From Armv8, the counters are a doubleword-aligned addresses. 0b111111	0b111111
[7:0]	N	Number of counters implemented in addition to the cycle counter, ext-PMCCNTR_ELO. The maximum number of event counters is 31. 0b00010100 Twenty PMU Counters Implemented 0b00000110 Six PMU Counters Implemented	The reset values can be the following: 0b00010100, 0b00000110, respective to the value.

Access

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Accessibility

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Component	Offset	Instance	Range
PMU	0xE00	PMCFGR	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() &&
AllowExternalPMUAccess()

RO

Otherwise

ERROR

B.4.27 PMCR_ELO, Performance Monitors Control Register

Provides details of the Performance Monitors implementation, including the number of counters implemented, and configures and controls the counters.

Configurations

External register PMCR_ELO bits [7:0] are architecturally mapped to AArch64 System register [A.4.2 PMCR_ELO, Performance Monitors Control Register](#) on page 341 bits [7:0].

Attributes

Width

32

Component

PMU

Register offset

0xE04

Access type

See bit descriptions

Reset value

0000	0000	0000	0000	0000	0xxx	xxx0	x000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-50: ext_pmc_r_el0 bit assignments

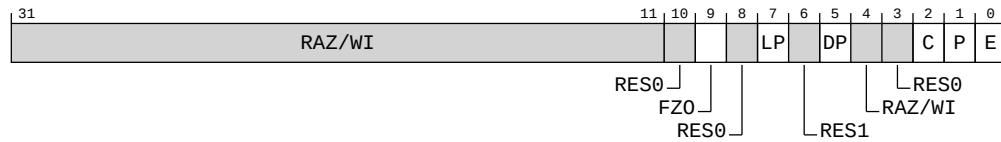


Table B-101: PMCR_ELO bit descriptions

Bits	Name	Description	Reset
[31:11]	RAZ/WI	Reserved	RAZ/WI
[10]	RES0	Reserved	RES0
[9]	FZO	Freeze-on-overflow. Stop event counters on overflow. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 Do not freeze on overflow. 0b1 Event counter ext-PMEVCNTR<n>_ELO does not count when AArch64-PMOVSLR_ELO[(PMN-1):0] is nonzero and n is in the range of affected event counters. If PMN is not 0, this field affects the operation of event counters in the range [0 .. (PMN-1)]. This field does not affect the operation of other event counters and AArch64-PMCCNTR_ELO. The operation of this field applies even when EL2 is disabled in the current Security state.	x
[8]	RES0	Reserved	RES0
[7]	LP	Long event counter enable. Determines when unsigned overflow is recorded by an event counter overflow bit. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 Event counter overflow on increment that causes unsigned overflow of ext-PMEVCNTR<n>_ELO[31:0]. 0b1 Event counter overflow on increment that causes unsigned overflow of ext-PMEVCNTR<n>_ELO[63:0]. If PMN is not 0, this bit affects the operation of event counters in the range [0 .. (PMN-1)]. The field does not affect the operation of other event counters and ext-PMCCNTR_ELO. The operation of this field applies even when EL2 is disabled in the current Security state.	x

Bits	Name	Description	Reset
[6]	RES1	Reserved	RES1
[5]	DP	Disable cycle counter when event counting is prohibited. The possible values of this bit are: 0b0 Cycle counting by ext-PMCCNTR_ELO is not affected by this mechanism. 0b1 Cycle counting by ext-PMCCNTR_ELO is disabled in prohibited regions: - If FEAT_PMUv3p1 is implemented, EL2 is implemented, and AArch64-MDCR_EL2.HPMD is 1, then cycle counting by ext-PMCCNTR_ELO is disabled at EL2. - If FEAT_PMUv3p7 is implemented, EL3 is implemented and using AArch64, and AArch64-MDCR_EL3.MPMX is 1, then cycle counting by ext-PMCCNTR_ELO is disabled at EL3. - If EL3 is implemented, AArch64-MDCR_EL3.SPME is 0, and either FEAT_PMUv3p7 is not implemented or AArch64-MDCR_EL3.MPMX is 0, then cycle counting by ext-PMCCNTR_ELO is disabled at EL3 and in Secure state. If AArch64-MDCR_EL2.HPMN is not 0, this is when event counting by event counters in the range [0..(AArch64-MDCR_EL2.HPMN-1)] is prohibited. For more information, see <i>Prohibiting event counting</i> in the Arm® Architecture Reference Manual for A-profile architecture.	x
[4]	RAZ/WI	Reserved	RAZ/WI
[3]	RES0	Reserved	RES0
[2]	C	Cycle counter reset. The effects of writing to this bit are: 0b0 No action. 0b1 Reset ext-PMCCNTR_ELO to zero. Note: Resetting ext-PMCCNTR_ELO does not change the cycle counter overflow bit. If FEAT_PMUv3p5 is implemented, the value of PMCR_ELO.LC is ignored, and bits [63:0] of the cycle counter are reset. Access to this field is: WO/RAZ	0b0
[1]	P	Event counter reset. The effects of writing to this bit are: 0b0 No action. 0b1 Reset all event counters, not including ext-PMCCNTR_ELO, to zero. Note: Resetting the event counters does not change the event counter overflow bits. If FEAT_PMUv3p5 is implemented, the value of AArch64-MDCR_EL2.HLP, or PMCR_ELO.LP is ignored and bits [63:0] of all affected event counters are reset. Access to this field is: WO/RAZ	0b0

Bits	Name	Description	Reset
[0]	E	Enable. In the description of this field: - If EL2 is implemented and is using AArch64, PMN is AArch64-MDCR_EL2.HPMN. - If EL2 is not implemented, PMN is PMCR_ELO.N. 0b0 ext-PMCCNTR_ELO is disabled and event counters ext-PMEVCNTR<n>_ELO, where n is in the range of affected event counters, are disabled. 0b1 ext-PMCCNTR_ELO and event counters ext-PMEVCNTR<n>_ELO, where n is in the range of affected event counters, are enabled by ext-PMCNSET_ELO. If PMN is not 0, this field affects the operation of event counters in the range [0 .. (PMN-1)]. This field does not affect the operation of other event counters. The operation of this field applies even when EL2 is disabled in the current Security state.	0b0

Access

SoftwareLockStatus() depends on the type of access attempted and AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Accessibility

SoftwareLockStatus() depends on the type of access attempted and AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Component	Offset	Instance	Range
PMU	0xE04	PMCR_ELO	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && AllowExternalPMUAccess() && SoftwareLockStatus()

RO

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && AllowExternalPMUAccess() && !SoftwareLockStatus()

RW

Otherwise

ERROR

B.4.28 PMCEID0, Performance Monitors Common Event Identification register 0

Defines which Common architectural events and Common microarchitectural events are implemented, or counted, using PMU events in the range 0x0000 to 0x001F.

For more information about the Common events and the use of the PMCEIDn registers, see *The PMU event number space and common events* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



This view of the register was previously called PMCEID0_ELO.

Configurations

External register PMCEID0 bits [31:0] are architecturally mapped to AArch64 System register [A.4.3 PMCEID0_ELO, Performance Monitors Common Event Identification register 0](#) on page 348 bits [31:0].

Attributes

Width

32

Component

PMU

Register offset

0xE20

Access type

See bit descriptions

Reset value

0111 1011 1111 1111 0111 1111 0011 1111

Bit descriptions

Figure B-51: ext_pmceid0 bit assignments

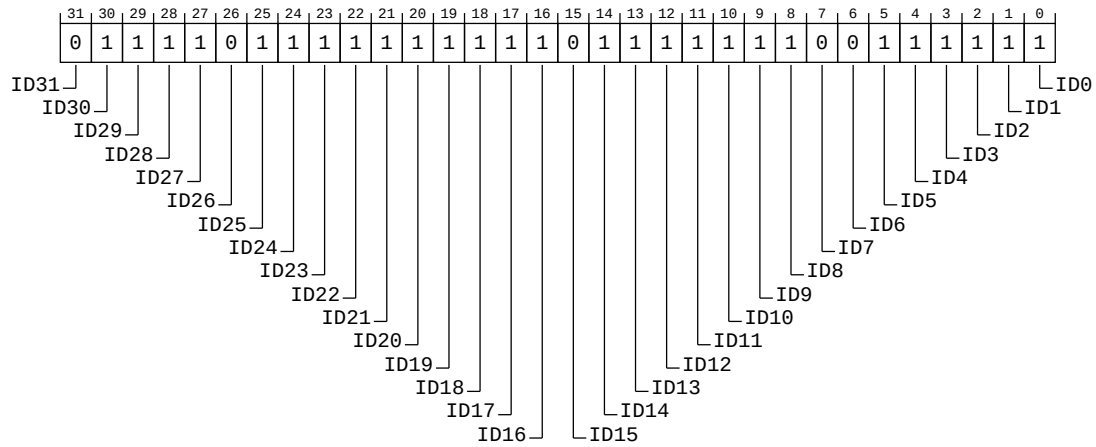


Table B-103: PMCEID0 bit descriptions

Bits	Name	Description	Reset
[31]	ID31	ID31 corresponds to common event (0x1f) L1D_CACHE_ALLOCATE 0b0 The Common event is not implemented, or not counted.	0b0
[30]	ID30	ID30 corresponds to common event (0x1e) CHAIN 0b1 The Common event is implemented.	0b1
[29]	ID29	ID29 corresponds to common event (0x1d) BUS_CYCLES 0b1 The Common event is implemented.	0b1
[28]	ID28	ID28 corresponds to common event (0x1c) TTBR_WRITE_RETIRED 0b1 The Common event is implemented.	0b1
[27]	ID27	ID27 corresponds to common event (0x1b) INST_SPEC 0b1 The Common event is implemented.	0b1
[26]	ID26	ID26 corresponds to common event (0x1a) MEMORY_ERROR 0b0 The Common event is not implemented, or not counted.	0b0
[25]	ID25	ID25 corresponds to common event (0x19) BUS_ACCESS 0b1 The Common event is implemented.	0b1
[24]	ID24	ID24 corresponds to common event (0x18) L2D_CACHE_WB 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[23]	ID23	ID23 corresponds to common event (0x17) L2D_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[22]	ID22	ID22 corresponds to common event (0x16) L2D_CACHE 0b1 The Common event is implemented.	0b1
[21]	ID21	ID21 corresponds to common event (0x15) L1D_CACHE_WB 0b1 The Common event is implemented.	0b1
[20]	ID20	ID20 corresponds to common event (0x14) L1I_CACHE 0b1 The Common event is implemented.	0b1
[19]	ID19	ID19 corresponds to common event (0x13) MEM_ACCESS 0b1 The Common event is implemented.	0b1
[18]	ID18	ID18 corresponds to common event (0x12) BR_PRED 0b1 The Common event is implemented.	0b1
[17]	ID17	ID17 corresponds to common event (0x11) CPU_CYCLES 0b1 The Common event is implemented.	0b1
[16]	ID16	ID16 corresponds to common event (0x10) BR_MIS_PRED 0b1 The Common event is implemented.	0b1
[15]	ID15	ID15 corresponds to common event (0xf) UNALIGNED_LDST_RETIRED 0b0 The Common event is not implemented, or not counted.	0b0
[14]	ID14	ID14 corresponds to common event (0xe) BR_RETURN_RETIRED 0b1 The Common event is implemented.	0b1
[13]	ID13	ID13 corresponds to common event (0xd) BR_IMMED_RETIRED 0b1 The Common event is implemented.	0b1
[12]	ID12	ID12 corresponds to common event (0xc) PC_WRITE_RETIRED 0b1 The Common event is implemented.	0b1
[11]	ID11	ID11 corresponds to common event (0xb) CID_WRITE_RETIRED 0b1 The Common event is implemented.	0b1
[10]	ID10	ID10 corresponds to common event (0xa) EXC_RETURN 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[9]	ID9	ID9 corresponds to common event (0x9) EXC_TAKEN 0b1 The Common event is implemented.	0b1
[8]	ID8	ID8 corresponds to common event (0x8) INST_RETIRED 0b1 The Common event is implemented.	0b1
[7]	ID7	ID7 corresponds to common event (0x7) ST_RETIRED 0b0 The Common event is not implemented, or not counted.	0b0
[6]	ID6	ID6 corresponds to common event (0x6) LD_RETIRED 0b0 The Common event is not implemented, or not counted.	0b0
[5]	ID5	ID5 corresponds to common event (0x5) L1D_TLB_REFILL 0b1 The Common event is implemented.	0b1
[4]	ID4	ID4 corresponds to common event (0x4) L1D_CACHE 0b1 The Common event is implemented.	0b1
[3]	ID3	ID3 corresponds to common event (0x3) L1D_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[2]	ID2	ID2 corresponds to common event (0x2) L1I_TLB_REFILL 0b1 The Common event is implemented.	0b1
[1]	ID1	ID1 corresponds to common event (0x1) L1I_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[0]	ID0	ID0 corresponds to common event (0x0) SW_INCR 0b1 The Common event is implemented.	0b1

Access

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Accessibility

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Component	Offset	Instance	Range
PMU	0xE20	PMCEID0	None

This interface is accessible as follows:

**When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() &&
AllowExternalPMUAccess()**

RO

Otherwise

ERROR

B.4.29 PMCEID1, Performance Monitors Common Event Identification register 1

Defines which Common architectural events and Common microarchitectural events are implemented, or counted, using PMU events in the range 0x020 to 0x03F.

For more information about the Common events and the use of the PMCEIDn registers, see *The PMU event number space and common events* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



Note

This view of the register was previously called PMCEID1_EL0.

Configurations

External register PMCEID1 bits [31:0] are architecturally mapped to AArch64 System register [A.4.4 PMCEID1_EL0, Performance Monitors Common Event Identification register 1](#) on page 355 bits [31:0].

Attributes

Width

32

Component

PMU

Register offset

0xE24

Access type

See bit descriptions

Reset value

1111 1110 1111 1110 1010 1111 1111 1111

Bit descriptions

Figure B-52: ext_pmceid1 bit assignments

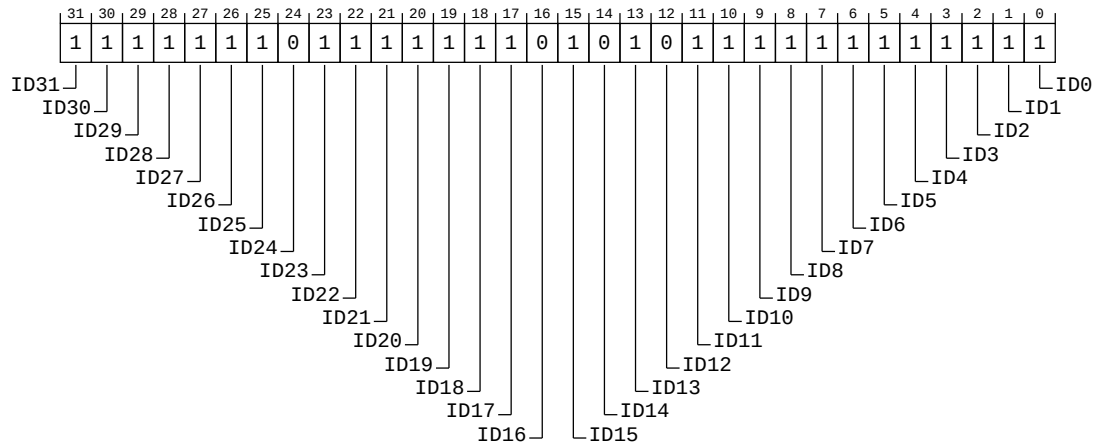


Table B-105: PMCEID1 bit descriptions

Bits	Name	Description	Reset
[31]	ID31	ID31 corresponds to common event (0x3f) STALL_SLOT 0b1 The Common event is implemented.	0b1
[30]	ID30	ID30 corresponds to common event (0x3e) STALL_SLOT_FRONTEND 0b1 The Common event is implemented.	0b1
[29]	ID29	ID29 corresponds to common event (0x3d) STALL_SLOT_BACKEND 0b1 The Common event is implemented.	0b1
[28]	ID28	ID28 corresponds to common event (0x3c) STALL 0b1 The Common event is implemented.	0b1
[27]	ID27	ID27 corresponds to common event (0x3b) OP_SPEC 0b1 The Common event is implemented.	0b1
[26]	ID26	ID26 corresponds to common event (0x3a) OP_RETIRED 0b1 The Common event is implemented.	0b1
[25]	ID25	ID25 corresponds to common event (0x39) L1D_CACHE_LMISS_RD 0b1 The Common event is implemented.	0b1
[24]	ID24	ID24 corresponds to common event (0x38) REMOTE_ACCESS_RD 0b0 The Common event is not implemented, or not counted.	0b0

Bits	Name	Description	Reset
[23]	ID23	ID23 corresponds to common event (0x37) LL_CACHE_MISS_RD 0b1 The Common event is implemented.	0b1
[22]	ID22	ID22 corresponds to common event (0x36) LL_CACHE_RD 0b1 The Common event is implemented.	0b1
[21]	ID21	ID21 corresponds to common event (0x35) ITLB_WLK 0b1 The Common event is implemented.	0b1
[20]	ID20	ID20 corresponds to common event (0x34) DTLB_WLK 0b1 The Common event is implemented.	0b1
[19]	ID19	ID19 corresponds to common event (0x33) LL_CACHE_MISS 0b1 The Common event is implemented.	0b1
[18]	ID18	ID18 corresponds to common event (0x32) LL_CACHE 0b1 The Common event is implemented.	0b1
[17]	ID17	ID17 corresponds to common event (0x31) REMOTE_ACCESS 0b1 The Common event is implemented.	0b1
[16]	ID16	ID16 corresponds to common event (0x30) L2I_TLB 0b0 The Common event is not implemented, or not counted.	0b0
[15]	ID15	ID15 corresponds to common event (0x2f) L2D_TLB 0b1 The Common event is implemented.	0b1
[14]	ID14	ID14 corresponds to common event (0x2e) L2I_TLB_REFILL 0b0 The Common event is not implemented, or not counted.	0b0
[13]	ID13	ID13 corresponds to common event (0x2d) L2D_TLB_REFILL 0b1 The Common event is implemented.	0b1
[12]	ID12	ID12 corresponds to common event (0x2c) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[11]	ID11	ID11 corresponds to common event (0x2b) L3D_CACHE 0b1 The Common event is implemented.	0b1
[10]	ID10	ID10 corresponds to common event (0x2a) L3D_CACHE_REFILL 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[9]	ID9	ID9 corresponds to common event (0x29) L3D_CACHE_ALLOCATE 0b1 The Common event is implemented.	0b1
[8]	ID8	ID8 corresponds to common event (0x28) L2I_CACHE_REFILL 0b1 The Common event is implemented.	0b1
[7]	ID7	ID7 corresponds to common event (0x27) L2I_CACHE 0b1 The Common event is implemented.	0b1
[6]	ID6	ID6 corresponds to common event (0x26) L1I_TLB 0b1 The Common event is implemented.	0b1
[5]	ID5	ID5 corresponds to common event (0x25) L1D_TLB 0b1 The Common event is implemented.	0b1
[4]	ID4	ID4 corresponds to common event (0x24) STALL_BACKEND 0b1 The Common event is implemented.	0b1
[3]	ID3	ID3 corresponds to common event (0x23) STALL_FRONTEND 0b1 The Common event is implemented.	0b1
[2]	ID2	ID2 corresponds to common event (0x22) BR_MIS_PRED_RETIRED 0b1 The Common event is implemented.	0b1
[1]	ID1	ID1 corresponds to common event (0x21) BR_RETIRED 0b1 The Common event is implemented.	0b1
[0]	ID0	ID0 corresponds to common event (0x20) L2D_CACHE_ALLOCATE 0b1 The Common event is implemented.	0b1

Access

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Accessibility

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Component	Offset	Instance	Range
PMU	0xE24	PMCEID1	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() &&
AllowExternalPMUAccess()

RO

Otherwise

ERROR

B.4.30 PMCEID2, Performance Monitors Common Event Identification
register 2

Defines which Common architectural events and Common microarchitectural events are implemented, or counted, using PMU events in the range 0x4000 to 0x401F.

For more information about the Common events and the use of the PMCEIDn registers, see *The PMU event number space and common events* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

External register PMCEID2 bits [31:0] are architecturally mapped to AArch64 System register [A.4.3 PMCEID0_ELO, Performance Monitors Common Event Identification register 0](#) on page 348 bits [63:32].

Attributes

Width

32

Component

PMU

Register offset

0xE28

Access type

See bit descriptions

Reset value

0000	1111	0000	1111	0001	1110	0111	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-53: ext_pmceid2 bit assignments

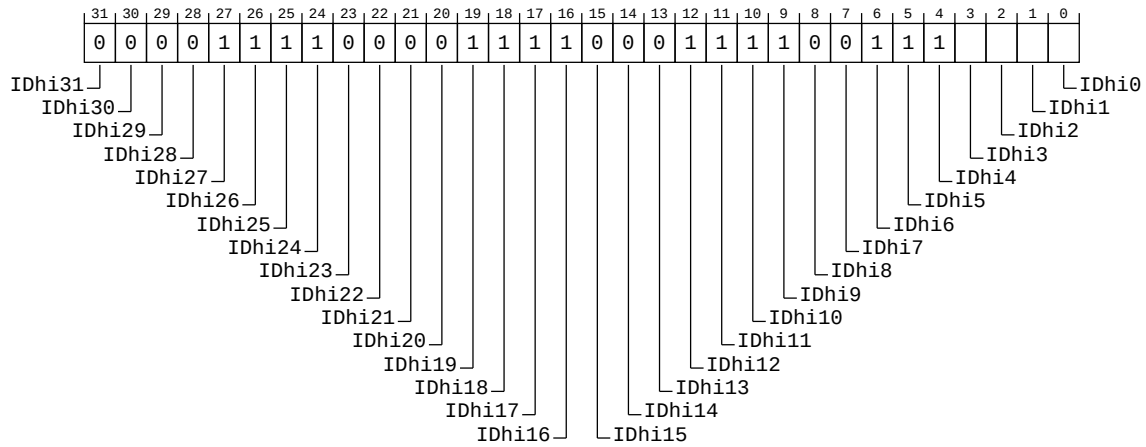


Table B-107: PMCEID2 bit descriptions

Bits	Name	Description	Reset
[31]	IDHi31	IDHi31 corresponds to a Reserved Event event (0x401f) 0b0 The Common event is not implemented, or not counted.	0b0
[30]	IDHi30	IDHi30 corresponds to a Reserved Event event (0x401e) 0b0 The Common event is not implemented, or not counted.	0b0
[29]	IDHi29	IDHi29 corresponds to a Reserved Event event (0x401d) 0b0 The Common event is not implemented, or not counted.	0b0
[28]	IDHi28	IDHi28 corresponds to a Reserved Event event (0x401c) 0b0 The Common event is not implemented, or not counted.	0b0
[27]	IDHi27	IDHi27 corresponds to common event (0x401b) CTI_TRIGOUT7 0b1 The Common event is implemented.	0b1
[26]	IDHi26	IDHi26 corresponds to common event (0x401a) CTI_TRIGOUT6 0b1 The Common event is implemented.	0b1
[25]	IDHi25	IDHi25 corresponds to common event (0x4019) CTI_TRIGOUT5 0b1 The Common event is implemented.	0b1
[24]	IDHi24	IDHi24 corresponds to common event (0x4018) CTI_TRIGOUT4 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[23]	IDhi23	IDhi23 corresponds to a Reserved Event event (0x4017) 0b0 The Common event is not implemented, or not counted.	0b0
[22]	IDhi22	IDhi22 corresponds to a Reserved Event event (0x4016) 0b0 The Common event is not implemented, or not counted.	0b0
[21]	IDhi21	IDhi21 corresponds to a Reserved Event event (0x4015) 0b0 The Common event is not implemented, or not counted.	0b0
[20]	IDhi20	IDhi20 corresponds to a Reserved Event event (0x4014) 0b0 The Common event is not implemented, or not counted.	0b0
[19]	IDhi19	IDhi19 corresponds to common event (0x4013) TRCEXTOUT3 0b1 The Common event is implemented.	0b1
[18]	IDhi18	IDhi18 corresponds to common event (0x4012) TRCEXTOUT2 0b1 The Common event is implemented.	0b1
[17]	IDhi17	IDhi17 corresponds to common event (0x4011) TRCEXTOUT1 0b1 The Common event is implemented.	0b1
[16]	IDhi16	IDhi16 corresponds to common event (0x4010) TRCEXTOUT0 0b1 The Common event is implemented.	0b1
[15]	IDhi15	IDhi15 corresponds to common event (0x400f) PMU_HOVFS 0b0 The Common event is not implemented, or not counted.	0b0
[14]	IDhi14	IDhi14 corresponds to common event (0x400e) TRB_TRIG 0b0 The Common event is not implemented, or not counted.	0b0
[13]	IDhi13	IDhi13 corresponds to common event (0x400d) PMU_OVFS 0b0 The Common event is not implemented, or not counted.	0b0
[12]	IDhi12	IDhi12 corresponds to common event (0x400c) TRB_WRAP 0b1 The Common event is implemented.	0b1
[11]	IDhi11	IDhi11 corresponds to common event (0x400b) L3D_CACHE_LMISS_RD 0b1 The Common event is implemented.	0b1
[10]	IDhi10	IDhi10 corresponds to common event (0x400a) L2I_CACHE_LMISS 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[9]	IDhi9	IDhi9 corresponds to common event (0x4009) L2D_CACHE_LMISS_RD 0b1 The Common event is implemented.	0b1
[8]	IDhi8	IDhi8 corresponds to common event (0x4008) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[7]	IDhi7	IDhi7 corresponds to common event (0x4007) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[6]	IDhi6	IDhi6 corresponds to common event (0x4006) L1I_CACHE_LMISS 0b1 The Common event is implemented.	0b1
[5]	IDhi5	IDhi5 corresponds to common event (0x4005) STALL_BACKEND_MEM 0b1 The Common event is implemented.	0b1
[4]	IDhi4	IDhi4 corresponds to common event (0x4004) CNT_CYCLES 0b1 The Common event is implemented.	0b1
[3]	IDhi3	IDhi3 corresponds to common event (0x4003) SAMPLE_COLLISION 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.
[2]	IDhi2	IDhi2 corresponds to common event (0x4002) SAMPLE_FILTRATE 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.
[1]	IDhi1	IDhi1 corresponds to common event (0x4001) SAMPLE_FEED 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.

Bits	Name	Description	Reset
[0]	IDhi0	IDhi0 corresponds to common event (0x4000) SAMPLE_POP 0b0 The common event is not implemented, or not counted. This value is reported if the SPE is not implemented. 0b1 The common event is implemented. This value is reported if SPE is implemented.	The reset values can be the following: 0b0, 0b1, respective to the value.

Access

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Accessibility

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Component	Offset	Instance	Range
PMU	0xE28	PMCEID2	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && AllowExternalPMUAccess()

RO

Otherwise

ERROR

B.4.31 PMCEID3, Performance Monitors Common Event Identification register 3

Defines which Common architectural events and Common microarchitectural events are implemented, or counted, using PMU events in the range 0x4020 to 0x403F.

For more information about the Common events and the use of the PMCEIDn registers, see *The PMU event number space and common events* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

External register PMCEID3 bits [31:0] are architecturally mapped to AArch64 System register [A.4.4 PMCEID1_ELO, Performance Monitors Common Event Identification register 1](#) on page 355 bits [63:32].

Attributes**Width**

32

Component

PMU

Register offset

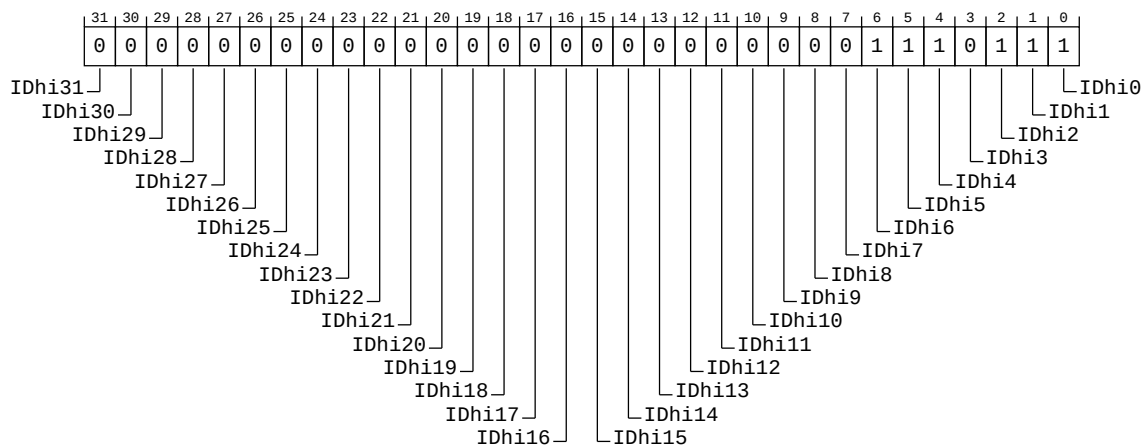
0xE2C

Access type

See bit descriptions

Reset value

0000 0000 0000 0000 0000 0000 0111 0111

Bit descriptions**Figure B-54: ext_pmceid3 bit assignments****Table B-109: PMCEID3 bit descriptions**

Bits	Name	Description	Reset
[31]	IDhi31	IDhi31 corresponds to a Reserved Event event (0x403f) 0b0 The Common event is not implemented, or not counted.	0b0
[30]	IDhi30	IDhi30 corresponds to a Reserved Event event (0x403e) 0b0 The Common event is not implemented, or not counted.	0b0
[29]	IDhi29	IDhi29 corresponds to a Reserved Event event (0x403d) 0b0 The Common event is not implemented, or not counted.	0b0

Bits	Name	Description	Reset
[28]	IDHi28	IDHi28 corresponds to a Reserved Event event (0x403c) 0b0 The Common event is not implemented, or not counted.	0b0
[27]	IDHi27	IDHi27 corresponds to a Reserved Event event (0x403b) 0b0 The Common event is not implemented, or not counted.	0b0
[26]	IDHi26	IDHi26 corresponds to a Reserved Event event (0x403a) 0b0 The Common event is not implemented, or not counted.	0b0
[25]	IDHi25	IDHi25 corresponds to a Reserved Event event (0x4039) 0b0 The Common event is not implemented, or not counted.	0b0
[24]	IDHi24	IDHi24 corresponds to a Reserved Event event (0x4038) 0b0 The Common event is not implemented, or not counted.	0b0
[23]	IDHi23	IDHi23 corresponds to a Reserved Event event (0x4037) 0b0 The Common event is not implemented, or not counted.	0b0
[22]	IDHi22	IDHi22 corresponds to a Reserved Event event (0x4036) 0b0 The Common event is not implemented, or not counted.	0b0
[21]	IDHi21	IDHi21 corresponds to a Reserved Event event (0x4035) 0b0 The Common event is not implemented, or not counted.	0b0
[20]	IDHi20	IDHi20 corresponds to a Reserved Event event (0x4034) 0b0 The Common event is not implemented, or not counted.	0b0
[19]	IDHi19	IDHi19 corresponds to a Reserved Event event (0x4033) 0b0 The Common event is not implemented, or not counted.	0b0
[18]	IDHi18	IDHi18 corresponds to a Reserved Event event (0x4032) 0b0 The Common event is not implemented, or not counted.	0b0
[17]	IDHi17	IDHi17 corresponds to a Reserved Event event (0x4031) 0b0 The Common event is not implemented, or not counted.	0b0
[16]	IDHi16	IDHi16 corresponds to a Reserved Event event (0x4030) 0b0 The Common event is not implemented, or not counted.	0b0
[15]	IDHi15	IDHi15 corresponds to a Reserved Event event (0x402f) 0b0 The Common event is not implemented, or not counted.	0b0

Bits	Name	Description	Reset
[14]	IDhi14	IDhi14 corresponds to a Reserved Event event (0x402e) 0b0 The Common event is not implemented, or not counted.	0b0
[13]	IDhi13	IDhi13 corresponds to a Reserved Event event (0x402d) 0b0 The Common event is not implemented, or not counted.	0b0
[12]	IDhi12	IDhi12 corresponds to a Reserved Event event (0x402c) 0b0 The Common event is not implemented, or not counted.	0b0
[11]	IDhi11	IDhi11 corresponds to a Reserved Event event (0x402b) 0b0 The Common event is not implemented, or not counted.	0b0
[10]	IDhi10	IDhi10 corresponds to a Reserved Event event (0x402a) 0b0 The Common event is not implemented, or not counted.	0b0
[9]	IDhi9	IDhi9 corresponds to a Reserved Event event (0x4029) 0b0 The Common event is not implemented, or not counted.	0b0
[8]	IDhi8	IDhi8 corresponds to a Reserved Event event (0x4028) 0b0 The Common event is not implemented, or not counted.	0b0
[7]	IDhi7	IDhi7 corresponds to a Reserved Event event (0x4027) 0b0 The Common event is not implemented, or not counted.	0b0
[6]	IDhi6	IDhi6 corresponds to common event (0x4026) MEM_ACCESS_CHECKED_WR 0b1 The Common event is implemented.	0b1
[5]	IDhi5	IDhi5 corresponds to common event (0x4025) MEM_ACCESS_CHECKED_RD 0b1 The Common event is implemented.	0b1
[4]	IDhi4	IDhi4 corresponds to common event (0x4024) MEM_ACCESS_CHECKED 0b1 The Common event is implemented.	0b1
[3]	IDhi3	IDhi3 corresponds to common event (0x4023) Reserved 0b0 The Common event is not implemented, or not counted.	0b0
[2]	IDhi2	IDhi2 corresponds to common event (0x4022) ST_ALIGN_LAT 0b1 The Common event is implemented.	0b1
[1]	IDhi1	IDhi1 corresponds to common event (0x4021) LD_ALIGN_LAT 0b1 The Common event is implemented.	0b1

Bits	Name	Description	Reset
[0]	IDhi0	IDhi0 corresponds to common event (0x4020) LDST_ALIGN_LAT 0b1 The Common event is implemented.	0b1

Access

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Accessibility

AllowExternalPMUAccess() has a new definition from Armv8.4. Refer to the Pseudocode definitions for more information.

Component	Offset	Instance	Range
PMU	0xE2C	PMCEID3	None

This interface is accessible as follows:

When IsCorePowered() && !DoubleLockStatus() && !OSLockStatus() && AllowExternalPMUAccess()

RO

Otherwise

ERROR

B.4.32 PMSSCR, PMU Snapshot Capture Register

Provides a mechanism for software to initiate a sample.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

PMU

Register offset

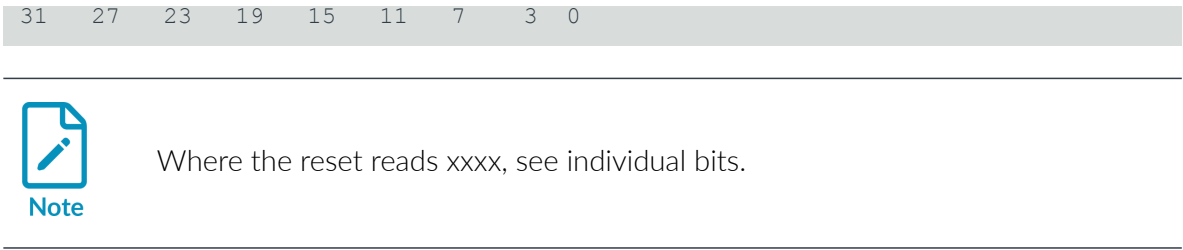
0xE30

Access type

RESERVEDW

Reset value





Bit descriptions

Figure B-55: ext_pmsscr bit assignments

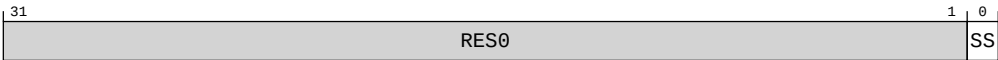


Table B-111: PMSSCR bit descriptions

Bits	Name	Description	Reset
[31:1]	RES0	Reserved	RES0
[0]	SS	Capture now. 0b0 Ignored. 0b1 Initiate a capture immediately.	x

Accessibility

This interface is accessible as follows:

WO

B.4.33 PMMIR, Performance Monitors Machine Identification Register

Describes Performance Monitors parameters specific to the implementation.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

PMU

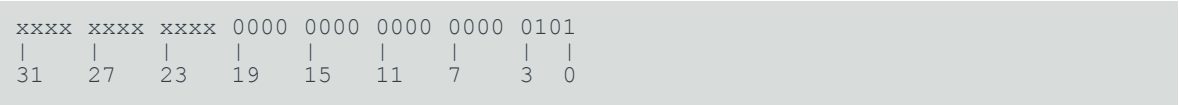
Register offset

0xE40

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-56: ext_pmmir bit assignments

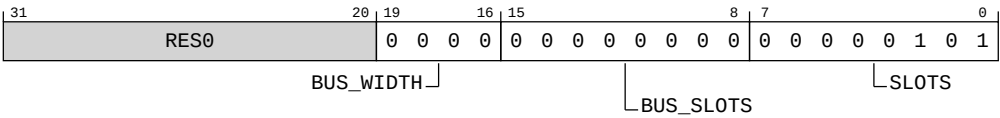


Table B-112: PMMIR bit descriptions

Bits	Name	Description	Reset
[31:20]	RES0	Reserved	RES0
[19:16]	BUS_WIDTH	Bus width. Indicates the number of bytes each BUS_ACCESS event relates to. Encoded as Log ₂ (number of bytes), plus one. Defined values are: 0b0000 The information is not available.	0b0000
[15:8]	BUS_SLOTS	Bus count. The largest value by which the BUS_ACCESS event might increment by in a single BUS_CYCLES cycle. 0b00000000	0x00
[7:0]	SLOTS	Operation width. The largest value by which the STALL_SLOT event might increment by in a single cycle. If the STALL_SLOT event is not implemented, this field might read as zero. 0b00000101 The largest value by which the STALL_SLOT PMU event may increment in one cycle is 5.	0x05

Accessibility

If the Core power domain is off or in a low-power state, access on this interface returns an Error.

Component	Offset	Instance	Range
PMU	0xE40	PMMIR	None

This interface is accessible as follows:

```
When !IsCorePowered() || DoubleLockStatus() || OSLockStatus() || !AllowExternalIPMUAccess()  
ERROR
```

Otherwise
RO

B.4.34 PMDEVARCH, Performance Monitors Device Architecture register

Identifies the programmers' model architecture of the Performance Monitor component.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width
32

Component
PMU

Register offset
0xFBC

Access type
See bit descriptions

Reset value

0100	0111	0111	0000	0010	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3
							0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-57: ext_pmdevarch bit assignments

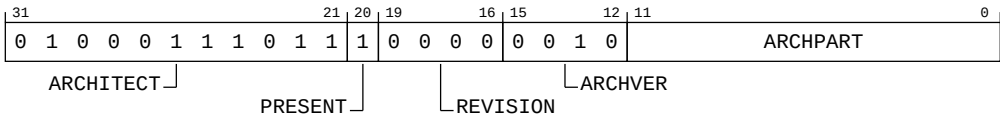


Table B-114: PMDEVARCH bit descriptions

Bits	Name	Description	Reset
[31:21]	ARCHITECT	Defines the architecture of the component. For Performance Monitors, this is Arm Limited. Bits [31:28] are the JEP106 continuation code, 0x4. Bits [27:21] are the JEP106 ID code, 0x3B. 0b01000111011	0b01000111011
[20]	PRESENT	Indicates that the DEVARCH is present. 0b1	0b1
[19:16]	REVISION	Defines the architecture revision. For architectures defined by Arm this is the minor revision. For Performance Monitors, the revision defined by Armv8 is 0x0. All other values are reserved. 0b0000	0b0000
[15:12]	ARCHVER	Architecture Version. Defines the architecture version of the component. 0b0010 Performance Monitors Extension version 3, PMUv3. All other values are reserved. PMDEVARCH.ARCHVER and PMDEVARCH.ARCHPART are also defined as a single field, PMDEVARCH.ARCHID, so that PMDEVARCH.ARCHVER is PMDEVARCH.ARCHID[15:12].	0b0010
[11:0]	ARCHPART	Architecture Part. Defines the architecture of the component. 0b101000010110 Armv8-A PE performance monitors. PMDEVARCH.ARCHVER and PMDEVARCH.ARCHPART are also defined as a single field, PMDEVARCH.ARCHID, so that PMDEVARCH.ARCHPART is PMDEVARCH.ARCHID[11:0].	The reset values can be the following: 0b101000010110, respective to the value.

Accessibility

Component	Offset	Instance	Range
PMU	0xFBC	PMDEVARCH	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.35 PMDEVID, Performance Monitors Device ID register

Provides information about features of the Performance Monitors implementation.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain.

If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required from Armv8.2 and in any implementation that includes FEAT_PCSRv8p2. Otherwise, its location is RES0.



Before Armv8.2, the PC Sample-based Profiling Extension can be implemented in the external debug register space, as indicated by the value of ext-EDDEVID.PCSample.

Attributes

Width

32

Component

PMU

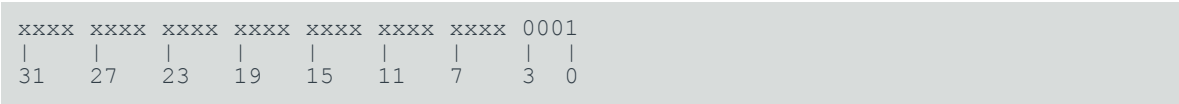
Register offset

0xFC8

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-58: ext_pmdeviid bit assignments



Table B-116: PMDEVID bit descriptions

Bits	Name	Description	Reset
[31:4]	RES0	Reserved	RES0
[3:0]	PCSample	Indicates the level of PC Sample-based Profiling support using Performance Monitors registers. 0b0001 PC Sample-based Profiling Extension is implemented in the Performance Monitors register space.	0b0001

Accessibility

Component	Offset	Instance	Range
PMU	0xFC8	PMDEVID	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.36 PMDEVTYPE, Performance Monitors Device Type register

Indicates to a debugger that this component is part of a PEs performance monitor interface.

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

Attributes

Width

32

Component

PMU

Register offset

0xFCC

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0110
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-59: ext_pmdevtype bit assignments



Table B-118: PMDEVTYPE bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SUB	Subtype. Indicates this is a component within a PE. 0b0001	0b0001
[3:0]	MAJOR	Major type. Indicates this is a performance monitor component. 0b0110	0b0110

Accessibility

Component	Offset	Instance	Range
PMU	0xFCC	PMDEVTYPE	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.37 PMPIDR4, Performance Monitors Peripheral Identification Register
4

Provides information to identify a Performance Monitor component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFD0

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-60: ext_pmpidr4 bit assignments



Table B-120: PMPIDR4 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SIZE	Size of the component. Log ₂ of the number of 4KB pages from the start of the component to the end of the component ID registers. 0b0000	0b0000
[3:0]	DES_2	Designer, JEP106 continuation code, least significant nibble. For Arm Limited, this field is 0b0100. 0b0100 Arm Limited	0b0100

Accessibility

Component	Offset	Instance	Range
PMU	0xFD0	PMPIDR4	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.38 PMPIDR0, Performance Monitors Peripheral Identification Register 0

Provides information to identify a Performance Monitor component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFE0

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1000	1110
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-61: ext_pmpidr0 bit assignments



Table B-122: PMPIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PART_0	Part number, least significant byte. 0b10001110 Neoverse N3	0x8E

Accessibility

Component	Offset	Instance	Range
PMU	0xFE0	PMPIDR0	None

This interface is accessible as follows:

When `IsCorePowered()`

RO

Otherwise

ERROR

B.4.39 PMPIDR1, Performance Monitors Peripheral Identification Register 1

Provides information to identify a Performance Monitor component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If `FEAT_DoPD` is implemented, this register is in the Core power domain. If `FEAT_DoPD` is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

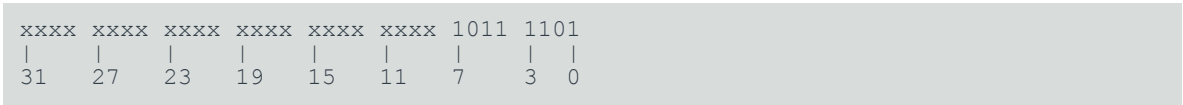
32

Component
PMU

Register offset
0xFE4

Access type
See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-62: ext_pmpidr1 bit assignments



Table B-124: PMPIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	DES_0	Designer, least significant nibble of JEP106 ID code. For Arm Limited, this field is 0b1011. 0b1011 Arm Limited	0b1011
[3:0]	PART_1	Part number, most significant nibble. 0b1101 Neoverse N3	0b1101

Accessibility

Component	Offset	Instance	Range
PMU	0xFE4	PMPIDR1	None

This interface is accessible as follows:

When IsCorePowered()
RO

Otherwise
ERROR

B.4.40 PMPIDR2, Performance Monitors Peripheral Identification Register
2

Provides information to identify a Performance Monitor component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFE8

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	1011
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-63: ext_pmpidr2 bit assignments

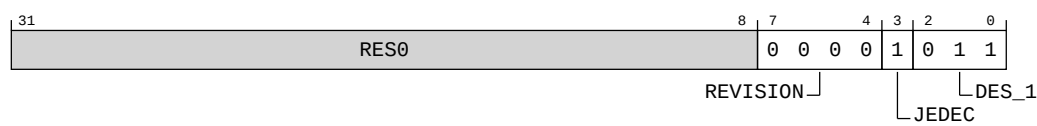


Table B-126: PMPIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVISION	Part major revision. Parts can also use this field to extend Part number to 16-bits. 0b0000 rOp1	0b0000
[3]	JEDEC	Indicates a JEP106 identity code is used. 0b1	0b1
[2:0]	DES_1	Designer, most significant bits of JEP106 ID code. For Arm Limited, this field is 0b011. 0b011 Arm Limited	0b011

Accessibility

Component	Offset	Instance	Range
PMU	0xFE8	PMPIDR2	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.41 PMPIDR3, Performance Monitors Peripheral Identification Register 3

Provides information to identify a Performance Monitor component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFEC

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-64: ext_pmpidr3 bit assignments

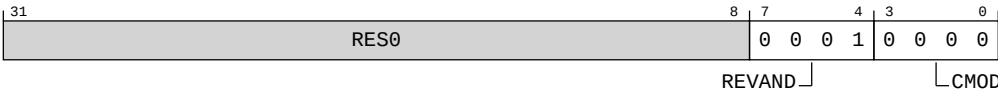


Table B-128: PMPIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVAND	Part minor revision. Parts using ext-PMPIDR2.REVISION as an extension to the Part number must use this field as a major revision number. 0b0001 rOp1	0b0001
[3:0]	CMOD	Customer modified. Indicates someone other than the Designer has modified the component. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
PMU	0xFEC	PMPIDR3	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.42 PMCIDR0, Performance Monitors Component Identification Register 0

Provides information to identify a Performance Monitor component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFF0

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	1101
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-65: ext_pmcidr0 bit assignments

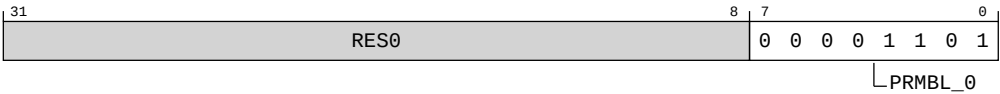


Table B-130: PMCIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_0	Preamble. 0b00001101	0x0D

Accessibility

Component	Offset	Instance	Range
PMU	0xFF0	PMCIDR0	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.43 PMCIDR1, Performance Monitors Component Identification Register 1

Provides information to identify a Performance Monitor component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFF4

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-66: ext_pmcidr1 bit assignments

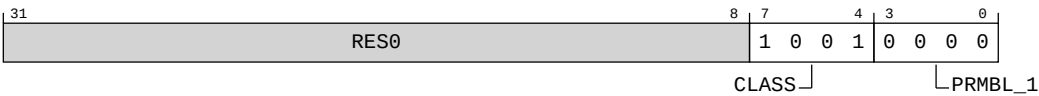


Table B-132: PMCIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	CLASS	Component class. 0b1001 CoreSight component. Other values are defined by the CoreSight Architecture. This field reads as 0x9.	0b1001
[3:0]	PRMBL_1	Preamble. RAZ . 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
PMU	0xFF4	PMCIDR1	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise
ERROR

B.4.44 PMCIDR2, Performance Monitors Component Identification Register 2

Provides information to identify a Performance Monitor component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width
32

Component
PMU

Register offset
0xFF8

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-67: ext_pmcidr2 bit assignments

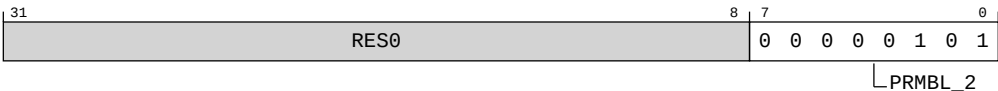


Table B-134: PMCIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_2	Preamble. 0b00000101	0x05

Accessibility

Component	Offset	Instance	Range
PMU	0xFF8	PMCIDR2	None

This interface is accessible as follows:

When IsCorePowered()

RO

Otherwise

ERROR

B.4.45 PMCIDR3, Performance Monitors Component Identification Register 3

Provides information to identify a Performance Monitor component.

For more information, see *About the Component Identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

If FEAT_DoPD is implemented, this register is in the Core power domain. If FEAT_DoPD is not implemented, this register is in the Debug power domain.

This register is required for CoreSight compliance.

Attributes

Width

32

Component

PMU

Register offset

0xFFC

Access type

See bit descriptions

Reset value

```
xxxx xxxx xxxx xxxx xxxx xxxx 1011 0001
```



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-68: ext_pmcidr3 bit assignments



Table B-136: PMCIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_3	Preamble. 0b10110001	0xB1

Accessibility

Component	Offset	Instance	Range
PMU	0xFFC	PMCIDR3	None

This interface is accessible as follows:

When IsCorePowered()
RO
Otherwise
ERROR

B.5 External AMU registers summary

The summary table provides an overview of all memory-mapped AMU registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-138: AMU registers summary

Offset	Name	Reset	Width	Description
0x0	AMEVCNTR00 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x4	AMEVCNTR00 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x8	AMEVCNTR01 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0xC	AMEVCNTR01 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x10	AMEVCNTR02 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x14	AMEVCNTR02 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x18	AMEVCNTR03 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x1C	AMEVCNTR03 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 0
0x100	AMEVCNTR10 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x104	AMEVCNTR10 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x108	AMEVCNTR11 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x10C	AMEVCNTR11 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x110	AMEVCNTR12 [31:0]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x114	AMEVCNTR12 [63:32]	See individual bit resets.	32-bit	Activity Monitors Event Counter Registers 1
0x400	AMEVTYPER00	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x404	AMEVTYPER01	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x408	AMEVTYPER02	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x40C	AMEVTYPER03	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 0
0x480	AMEVTYPER10	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 1
0x484	AMEVTYPER11	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 1
0x488	AMEVTYPER12	See individual bit resets.	32-bit	Activity Monitors Event Type Registers 1
0xC00	AMCNTENSET0	See individual bit resets.	32-bit	Activity Monitors Count Enable Set Register 0
0xC04	AMCNTENSET1	See individual bit resets.	32-bit	Activity Monitors Count Enable Set Register 1
0xC20	AMCNTENCLR0	See individual bit resets.	32-bit	Activity Monitors Count Enable Clear Register 0
0xC24	AMCNTENCLR1	See individual bit resets.	32-bit	Activity Monitors Count Enable Clear Register 1
0xCE0	AMCGCR	See individual bit resets.	32-bit	Activity Monitors Counter Group Configuration Register
0xE00	AMCFGR	See individual bit resets.	32-bit	Activity Monitors Configuration Register
0xE04	AMCR	See individual bit resets.	32-bit	Activity Monitors Control Register
0xE08	AMIIDR	See individual bit resets.	32-bit	Activity Monitors Implementation Identification Register
0xFA8	AMDEVAFF0	See individual bit resets.	32-bit	Activity Monitors Device Affinity Register 0
0xFAC	AMDEVAFF1	See individual bit resets.	32-bit	Activity Monitors Device Affinity Register 1
0xFBC	AMDEVARCH	See individual bit resets.	32-bit	Activity Monitors Device Architecture Register
0xFCC	AMDEVTYPE	See individual bit resets.	32-bit	Activity Monitors Device Type Register
0xFD0	AMPIDR4	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 4
0xFE0	AMPIDR0	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 0
0xFE4	AMPIDR1	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 1
0xFE8	AMPIDR2	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 2
0xFEC	AMPIDR3	See individual bit resets.	32-bit	Activity Monitors Peripheral Identification Register 3
0xFF0	AMCIDR0	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 0

Offset	Name	Reset	Width	Description
0xFF4	AMCIDR1	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 1
0xFF8	AMCIDR2	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 2
0xFFC	AMCIDR3	See individual bit resets.	32-bit	Activity Monitors Component Identification Register 3

B.5.1 AMEVTYPER00, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR00_ELO counts.

Configurations

External register AMEVTYPER00 bits [31:0] are architecturally mapped to AArch64 System register [A.10.3 AMEVTYPER00_ELO, Activity Monitors Event Type Registers 0](#) on page 467 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x400

Access type

Read

R

Write

RESERVED

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-69: ext_amevtyper00 bit assignments

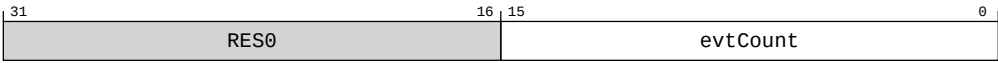


Table B-139: AMEVTYPER00 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter ext-AMEVCNTR0<n>. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: 0b000000000000010001 Processor frequency cycles	16 {x}

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as **RES0**. See *Access requirements for reserved and unallocated registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



Note

ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



Note

ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x400	AMEVTYPER00	None

This interface is accessible as follows:

RO

B.5.2 AMEVTYPER01, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR01_ELO counts.

Configurations

External register AMEVTYPER01 bits [31:0] are architecturally mapped to AArch64 System register [A.10.4 AMEVTYPER01_ELO, Activity Monitors Event Type Registers 0](#) on page 469 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x404

Access type

Read

R

Write

RESERVED

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-70: ext_amevtyper01 bit assignments

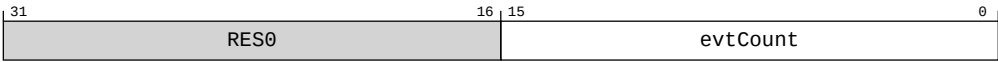


Table B-141: AMEVTYPER01 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0

Bits	Name	Description	Reset	
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter ext-AMEVCNTR0<n>. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: <table><tr><td>0b0100000000000100</td></tr></table> Constant frequency cycles	0b0100000000000100	16{x}
0b0100000000000100				

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as **RES0**. See *Access requirements for reserved and unallocated registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



Note

ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



Note

ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x404	AMEVTYPER01	None

This interface is accessible as follows:

RO

B.5.3 AMEVTYPER02, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR02_ELO counts.

Configurations

External register AMEVTYPER02 bits [31:0] are architecturally mapped to AArch64 System register [A.10.5 AMEVTYPER02_ELO, Activity Monitors Event Type Registers 0](#) on page 472 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x408

Access type

Read

R

Write

RESERVED

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-71: ext_amevtyper02 bit assignments

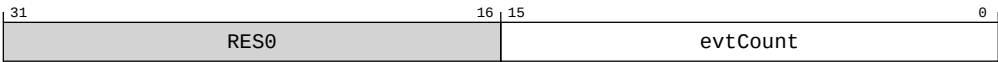


Table B-143: AMEVTYPER02 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter ext-AMEVCNTR0<n>. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: 0b00000000000001000 Instructions retired	16 {x}

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as **RES0**. See Access

requirements for reserved and unallocated registers in the [Arm® Architecture Reference Manual for A-profile architecture](#).



ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x408	AMEVTYPER02	None

This interface is accessible as follows:

RO

B.5.4 AMEVTYPER03, Activity Monitors Event Type Registers 0

Provides information on the events that an architected activity monitor event counter AArch64-AMEVCNTR03_ELO counts.

Configurations

External register AMEVTYPER03 bits [31:0] are architecturally mapped to AArch64 System register [A.10.6 AMEVTYPER03_ELO, Activity Monitors Event Type Registers 0](#) on page 474 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x40C

Access type

Read

R

Write

RESERVED

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-72: ext_amevtyper03 bit assignments

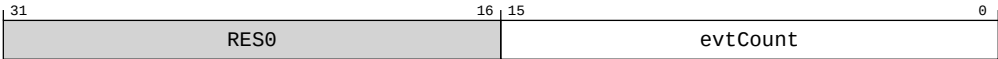


Table B-145: AMEVTYPER03 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the architected activity monitor event counter ext-AMEVCNTR0<n>. The value of this field is architecturally mandated for each architected counter. The following table shows the mapping between required event numbers and the corresponding counters: 0b01000000000000101 Memory stall cycles	16 {x}

Access

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as **RES0**. See *Access requirements for reserved and unallocated registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



Note

ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of architected activity monitor event counters, reads of AMEVTYPER0<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



Note

ext-AMCGCR.CG0NC identifies the number of architected activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x40C	AMEVTYPER03	None

This interface is accessible as follows:

RO

B.5.5 AMEVTYPER10, Activity Monitors Event Type Registers 1

Provides information on the events that an auxiliary activity monitor event counter AArch64-AMEVCNTR10_ELO counts.

Configurations

External register AMEVTYPER10 bits [31:0] are architecturally mapped to AArch64 System register [A.10.7 AMEVTYPER10_ELO, Activity Monitors Event Type Registers 1](#) on page 476 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x480

Access type

Read

R

Write

RESERVED

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-73: ext_amevtyper10 bit assignments

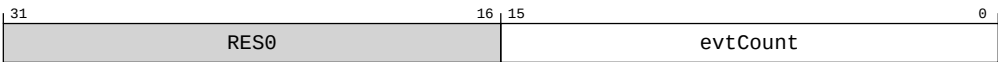


Table B-147: AMEVTYPER10 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the auxiliary activity monitor event counter AMEVCNTR10_ELO. 0b00000001100000000 MPMM gear 0 period threshold exceeded	16{x}

Access

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads of AMEVTYPER1<n> are RAZ. Software must treat reserved accesses as **RES0**. See *Access requirements for reserved and unallocated registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



ext-AMCGCR.CG1NC identifies the number of auxiliary activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads of AMEVTYPER1<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



ext-AMCGCR.CG1NC identifies the number of auxiliary activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x480	AMEVTYPER10	None

This interface is accessible as follows:

RO

B.5.6 AMEVTYPER11, Activity Monitors Event Type Registers 1

Provides information on the events that an auxiliary activity monitor event counter AArch64-AMEVCNTR11_ELO counts.

Configurations

External register AMEVTYPER11 bits [31:0] are architecturally mapped to AArch64 System register [A.10.8 AMEVTYPER11_ELO, Activity Monitors Event Type Registers 1](#) on page 479 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x484

Access type

Read

R

Write

RESERVED

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-74: ext_amevtyper11 bit assignments

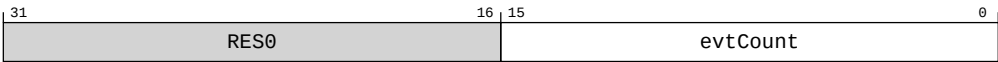


Table B-149: AMEVTYPER11 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the auxiliary activity monitor event counter AMEVCNTR11_ELO. 0b00000001100000001 MPMM gear 1 period threshold exceeded	16{x}

Access

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads of AMEVTYPER1<n> are RAZ. Software must treat reserved accesses as **RES0**. See *Access requirements for reserved and unallocated registers* in the [Arm® Architecture Reference Manual for A-profile architecture](#).



ext-AMCGCR.CG1NC identifies the number of auxiliary activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads of AMEVTYPER1<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



ext-AMCGCR.CG1NC identifies the number of auxiliary activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x484	AMEVTYPER11	None

This interface is accessible as follows:

RO

B.5.7 AMEVTYPER12, Activity Monitors Event Type Registers 1

Provides information on the events that an auxiliary activity monitor event counter AArch64-AMEVCNTR12_ELO counts.

Configurations

External register AMEVTYPER12 bits [31:0] are architecturally mapped to AArch64 System register [A.10.9 AMEVTYPER12_ELO, Activity Monitors Event Type Registers 1](#) on page 481 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0x488

Access type

Read

R

Write

RESERVED

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-75: ext_amevtyper12 bit assignments

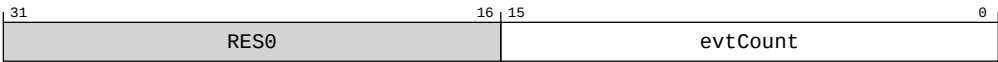


Table B-151: AMEVTYPER12 bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	evtCount	Event to count. The event number of the event that is counted by the auxiliary activity monitor event counter AMEVCNTR12_ELO. 0b00000001100000010 MPMM gear 2 period threshold exceeded	16{x}

Access

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads of AMEVTYPER1<n> are RAZ. Software must treat reserved accesses as **RES0**. See Access

requirements for reserved and unallocated registers in the [Arm® Architecture Reference Manual for A-profile architecture](#).



ext-AMCGCR.CG1NC identifies the number of auxiliary activity monitor event counters.

Accessibility

If <n> is greater than or equal to the number of auxiliary activity monitor event counters, reads of AMEVTYPER1<n> are RAZ. Software must treat reserved accesses as RES0. See 'Access requirements for reserved and unallocated registers'.



ext-AMCGCR.CG1NC identifies the number of auxiliary activity monitor event counters.

Component	Offset	Instance	Range
AMU	0x488	AMEVTYPER12	None

This interface is accessible as follows:

RO

B.5.8 AMCGCR, Activity Monitors Counter Group Configuration Register

Provides information on the number of activity monitor event counters implemented within each counter group.

Configurations

External register AMCGCR bits [31:0] are architecturally mapped to AArch64 System register [A.10.2 AMCGCR_ELO, Activity Monitors Counter Group Configuration Register](#) on page 465 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0xCE0

Access type
RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-76: ext_amcgcr bit assignments

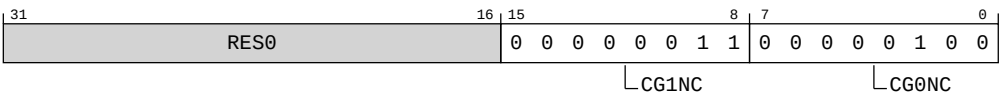


Table B-153: AMCGCR bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:8]	CG1NC	Counter Group 1 Number of Counters. The number of counters in the auxiliary counter group. In an implementation that includes FEAT_AMUv1, the permitted range of values is 0 to 16. 0b00000011 Three counters in the auxiliary counter group	0x03
[7:0]	CG0NC	Counter Group 0 Number of Counters. The number of counters in the architected counter group. 0b00000100	0x04

Accessibility

Component	Offset	Instance	Range
AMU	0xCE0	AMCGCR	None

This interface is accessible as follows:

RO

B.5.9 AMCFGR, Activity Monitors Configuration Register

Global configuration register for the activity monitors.

Provides information on supported features, the number of counter groups implemented, the total number of activity monitor event counters implemented, and the size of the counters. AMCFGR is applicable to both the architected and the auxiliary counter groups.

Configurations

External register AMCFGR bits [31:0] are architecturally mapped to AArch64 System register [A.10.1 AMCFGR_ELO, Activity Monitors Configuration Register](#) on page 463 bits [31:0].

Attributes

Width

32

Component

AMU

Register offset

0xE00

Access type

RO

Reset value

0001	xxx1	0000	0000	0011	1111	0000	0110
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-77: ext_amcfgr bit assignments

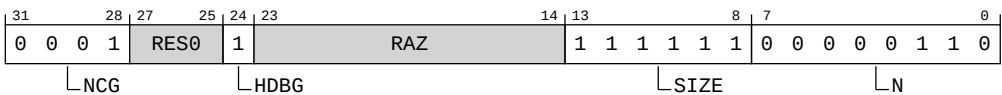


Table B-155: AMCFGR bit descriptions

Bits	Name	Description	Reset
[31:28]	NCG	Defines the number of counter groups. The following value is specified for this product. 0b0001 Two counter groups are implemented	0b0001
[27:25]	RES0	Reserved	RES0
[24]	HDBG	Halt-on-debug supported. This feature must be supported, and so this bit is 0b1. 0b1 ext-AMCR.HDBG is read/write.	0b1
[23:14]	RAZ	Reserved	RAZ
[13:8]	SIZE	Defines the size of activity monitor event counters. The size of the activity monitor event counters implemented by the Activity Monitors Extension is [AMCFGR.SIZE + 1]. The counters are 64-bit. Note: Software also uses this field to determine the spacing of counters in the memory-map. The counters are at doubleword-aligned addresses. 0b111111	0b111111
[7:0]	N	Defines the number of activity monitor event counters. The total number of counters implemented in all groups by the Activity Monitors Extension is [AMCFGR.N + 1]. 0b00000110 Seven activity monitor event counters	0x06

Accessibility

Component	Offset	Instance	Range
AMU	0xE00	AMCFGR	None

This interface is accessible as follows:

RO

B.5.10 AMIIDR, Activity Monitors Implementation Identification Register

Defines the implementer and revisions of the AMU.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xE08

Access type

RO

Reset value

1101 1000 1110 0000 0001 0100 0011 1011

Bit descriptions

Figure B-78: ext_amiidr bit assignments

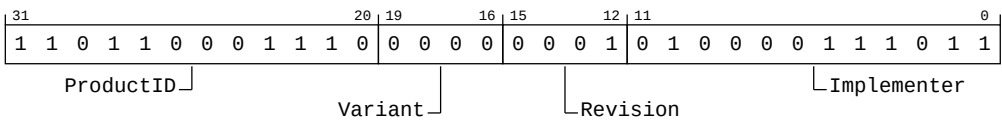


Table B-157: AMIIDR bit descriptions

Bits	Name	Description	Reset
[31:20]	ProductID	This field is an AMU part identifier. 0b110110001110 Neoverse N3	0xD8E
[19:16]	Variant	This field distinguishes product variants or major revisions of the product. 0b0000 rOp1	0b0000
[15:12]	Revision	This field distinguishes minor revisions of the product. 0b0001 rOp1	0b0001
[11:0]	Implementer	Contains the JEP106 code of the company that implemented the AMU. For an Arm implementation, this field reads as 0x43B. 0b010000111011 Arm Limited	0x43B

Accessibility

Component	Offset	Instance	Range
AMU	0xE08	AMIIDR	None

This interface is accessible as follows:

RO

B.5.11 AMDEVARCH, Activity Monitors Device Architecture Register

Identifies the programmers' model architecture of the AMU component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFBC

Access type

RO

Reset value

0100 0111 0111 0000 0000 1010 0110 0110

Bit descriptions

Figure B-79: ext_amdevarch bit assignments

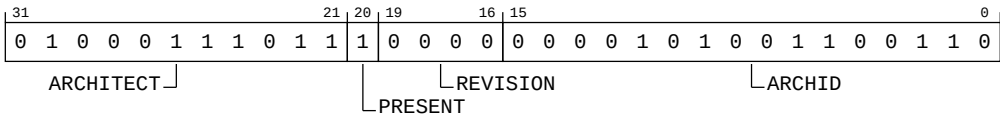


Table B-159: AMDEVARCH bit descriptions

Bits	Name	Description	Reset
[31:21]	ARCHITECT	Defines the architecture of the component. For AMU, this is Arm Limited. Bits [31:28] are the JEP106 continuation code, 0x4. Bits [27:21] are the JEP106 ID code, 0x3B. 0b01000111011	0b01000111011
[20]	PRESENT	Indicates that the DEVARCH is present. 0b1	0b1

Bits	Name	Description	Reset
[19:16]	REVISION	Defines the architecture revision. For architectures defined by Arm this is the minor revision. 0b0000 Architecture revision is AMUv1. All other values are reserved.	0b0000
[15:0]	ARCHID	Defines this part to be an AMU component. For architectures defined by Arm this is further subdivided. For AMU: - Bits [15:12] are the architecture version, 0x0. - Bits [11:0] are the architecture part number, 0xA66. This corresponds to AMU architecture version AMUv1. 0b00000101001100110	0x0A66

Accessibility

Component	Offset	Instance	Range
AMU	0xFBC	AMDEVARCH	None

This interface is accessible as follows:

RO

B.5.12 AMDEVTYPE, Activity Monitors Device Type Register

Indicates to a debugger that this component is part of a PE's performance monitor interface.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

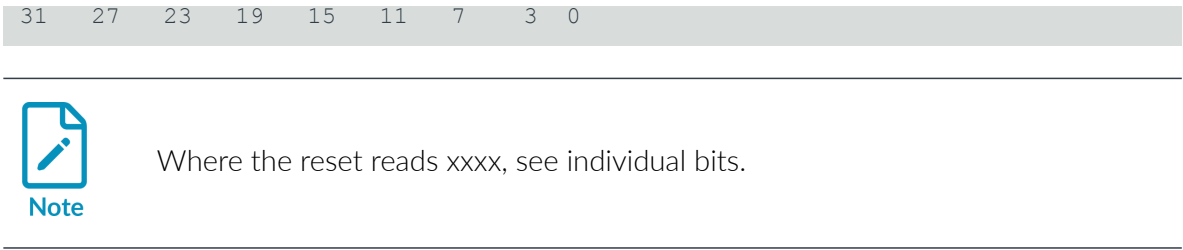
0xFCC

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0110



Bit descriptions

Figure B-80: ext_amdevtype bit assignments

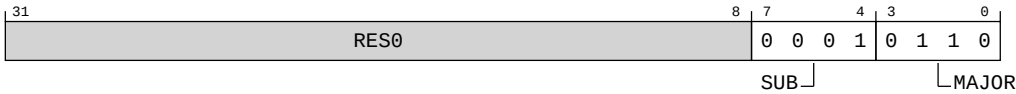


Table B-161: AMDEVTYPE bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SUB	Subtype. 0b0001 Component within a PE.	0b0001
[3:0]	MAJOR	Major type. 0b0110 Performance monitor component	0b0110

Accessibility

Component	Offset	Instance	Range
AMU	0xFCC	AMDEVTYPE	None

This interface is accessible as follows:

RO

B.5.13 AMPIDR4, Activity Monitors Peripheral Identification Register 4

Provides information to identify an activity monitors component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

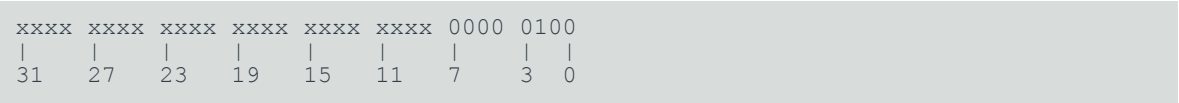
Register offset

0xFD0

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-81: ext_ampidr4 bit assignments



Table B-163: AMPIDR4 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SIZE	Size of the component. Log ₂ of the number of 4KB pages from the start of the component to the end of the component ID registers. 0b0000	0b0000
[3:0]	DES_2	Designer. JEP106 continuation code, least significant nibble. For Arm Limited, this field is 0b0100. 0b0100 Arm Limited	0b0100

Accessibility

Component	Offset	Instance	Range
AMU	0xFD0	AMPIDR4	None

This interface is accessible as follows:

RO

B.5.14 AMPIDR0, Activity Monitors Peripheral Identification Register 0

Provides information to identify an activity monitors component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFE0

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1000	1110
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-82: ext_ampidr0 bit assignments



Table B-165: AMPIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PART_0	Part number, least significant byte. 0b10001110 Neoverse N3	0x8E

Accessibility

Component	Offset	Instance	Range
AMU	0xFE0	AMPIDR0	None

This interface is accessible as follows:

RO

B.5.15 AMPIDR1, Activity Monitors Peripheral Identification Register 1

Provides information to identify an activity monitors component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFE4

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1011	1101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-83: ext_ampidr1 bit assignments



Table B-167: AMPIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	DES_0	Designer, least significant nibble of JEP106 ID code. For Arm Limited, this field is 0b1011. 0b1011 Arm Limited	0b1011
[3:0]	PART_1	Part number, most significant nibble. 0b1101 Neoverse N3	0b1101

Accessibility

Component	Offset	Instance	Range
AMU	0xFE4	AMPIDR1	None

This interface is accessible as follows:

RO

B.5.16 AMPIDR2, Activity Monitors Peripheral Identification Register 2

Provides information to identify an activity monitors component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFE8

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-84: ext_ampidr2 bit assignments

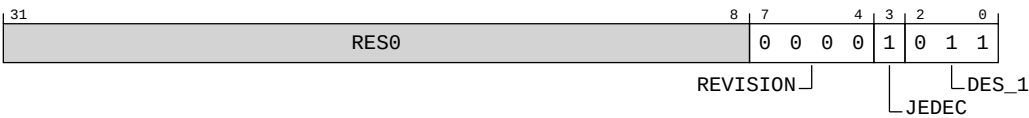


Table B-169: AMPIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVISION	Part major revision. Parts can also use this field to extend Part number to 16-bits. 0b0000 r0p1	0b0000
[3]	JEDEC	Indicates a JEP106 identity code is used. 0b1	0b1
[2:0]	DES_1	Designer, most significant bits of JEP106 ID code. For Arm Limited, this field is 0b011. 0b011 Arm Limited	0b011

Accessibility

Component	Offset	Instance	Range
AMU	0xFE8	AMPIDR2	None

This interface is accessible as follows:

RO

B.5.17 AMPIDR3, Activity Monitors Peripheral Identification Register 3

Provides information to identify an activity monitors component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFEC

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-85: ext_ampidr3 bit assignments

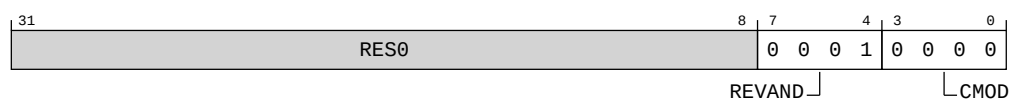


Table B-171: AMPIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVAND	Part minor revision. Parts using ext-AMPIDR2.REVISION as an extension to the Part number must use this field as a major revision number. 0b0001 rOp1	0b0001
[3:0]	CMOD	Customer modified. Indicates someone other than the Designer has modified the component. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
AMU	0xFEC	AMPIDR3	None

This interface is accessible as follows:

RO

B.5.18 AMCIDR0, Activity Monitors Component Identification Register 0

Provides information to identify an activity monitors component.

For more information, see *About the Component identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

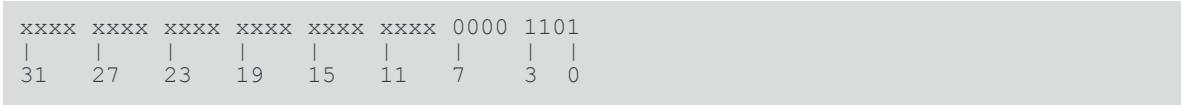
AMU

Register offset

0xFF0

Access type
RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-86: ext_amcldr0 bit assignments



Table B-173: AMCIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_0	Preamble. 0b00001101	0x0D

Accessibility

Component	Offset	Instance	Range
AMU	0xFF0	AMCIDR0	None

This interface is accessible as follows:

RO

B.5.19 AMCIDR1, Activity Monitors Component Identification Register 1

Provides information to identify an activity monitors component.

For more information, see *About the Component identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

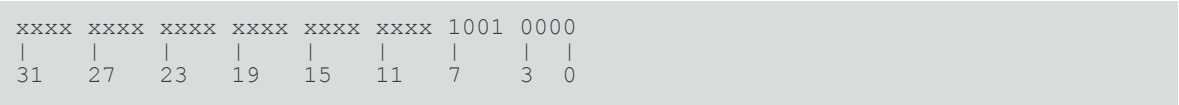
Register offset

0xFF4

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-87: ext_amcidr1 bit assignments



Table B-175: AMCIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	CLASS	Component class. 0b1001 CoreSight component. Other values are defined by the CoreSight Architecture. This field reads as 0x9.	0b1001
[3:0]	PRMBL_1	Preamble. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
AMU	0xFF4	AMCIDR1	None

This interface is accessible as follows:

RO

B.5.20 AMCIDR2, Activity Monitors Component Identification Register 2

Provides information to identify an activity monitors component.

For more information, see *About the Component identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFF8

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-88: ext_amcidr2 bit assignments



Table B-177: AMCIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_2	Preamble. 0b000000101	0x05

Accessibility

Component	Offset	Instance	Range
AMU	0xFF8	AMCIDR2	None

This interface is accessible as follows:

RO

B.5.21 AMCIDR3, Activity Monitors Component Identification Register 3

Provides information to identify an activity monitors component.

For more information, see *About the Component identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

AMU

Register offset

0xFFC

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1011	0001
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-89: ext_amcidr3 bit assignments

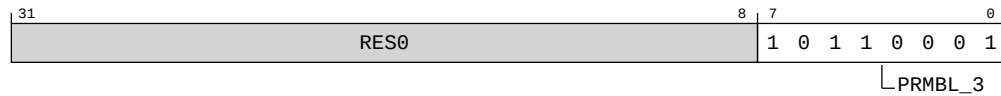


Table B-179: AMCIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_3	Preamble. 0b10110001	0xB1

Accessibility

Component	Offset	Instance	Range
AMU	0xFFC	AMCIDR3	None

This interface is accessible as follows:

RO

B.6 External ETE registers summary

The summary table provides an overview of all memory-mapped ETE registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-181: ETE registers summary

Offset	Name	Reset	Width	Description
0x004	TRCPRGCTLR	See individual bit resets.	32-bit	Programming Control Register
0x00C	TRCSTATR	See individual bit resets.	32-bit	Trace Status Register
0x010	TRCCONFIGR	See individual bit resets.	32-bit	Trace Configuration Register
0x018	TRCAUXCTLR	See individual bit resets.	32-bit	Auxiliary Control Register
0x020	TRCEVENTCTLOR	See individual bit resets.	32-bit	Event Control 0 Register
0x024	TRCEVENTCTL1R	See individual bit resets.	32-bit	Event Control 1 Register
0x028	TRCRSR	See individual bit resets.	32-bit	Resources Status Register
0x030	TRCTSCTLR	See individual bit resets.	32-bit	Timestamp Control Register

Offset	Name	Reset	Width	Description
0x034	TRCSYNCPR	See individual bit resets.	32-bit	Synchronization Period Register
0x038	TRCCCCTLR	See individual bit resets.	32-bit	Cycle Count Control Register
0x03C	TRCBBCTLR	See individual bit resets.	32-bit	Branch Broadcast Control Register
0x040	TRCTRACEIDR	See individual bit resets.	32-bit	Trace ID Register
0x044	TRCQCTLR	See individual bit resets.	32-bit	Q Element Control Register
0x080	TRCVICTLR	See individual bit resets.	32-bit	ViewInst Main Control Register
0x084	TRCVIICTLR	See individual bit resets.	32-bit	ViewInst Include/Exclude Control Register
0x088	TRCVISSCTLR	See individual bit resets.	32-bit	ViewInst Start/Stop Control Register
0x100	TRCSEQEVR0	See individual bit resets.	32-bit	Sequencer State Transition Control Register <n>
0x104	TRCSEQEVR1	See individual bit resets.	32-bit	Sequencer State Transition Control Register <n>
0x108	TRCSEQEVR2	See individual bit resets.	32-bit	Sequencer State Transition Control Register <n>
0x118	TRCSEQRSTEV	See individual bit resets.	32-bit	Sequencer Reset Control Register
0x11C	TRCSEQSTR	See individual bit resets.	32-bit	Sequencer State Register
0x120	TRCEXTINSEL0	See individual bit resets.	32-bit	External Input Select Register <n>
0x124	TRCEXTINSEL1	See individual bit resets.	32-bit	External Input Select Register <n>
0x128	TRCEXTINSEL2	See individual bit resets.	32-bit	External Input Select Register <n>
0x12C	TRCEXTINSEL3	See individual bit resets.	32-bit	External Input Select Register <n>
0x140	TRCCNTRLDVR0	See individual bit resets.	32-bit	Counter Reload Value Register <n>
0x144	TRCCNTRLDVR1	See individual bit resets.	32-bit	Counter Reload Value Register <n>
0x150	TRCCNTCTLR0	See individual bit resets.	32-bit	Counter Control Register <n>
0x154	TRCCNTCTLR1	See individual bit resets.	32-bit	Counter Control Register <n>
0x160	TRCCNTVR0	See individual bit resets.	32-bit	Counter Value Register <n>
0x164	TRCCNTVR1	See individual bit resets.	32-bit	Counter Value Register <n>
0x180	TRCIDR8	See individual bit resets.	32-bit	ID Register 8
0x184	TRCIDR9	See individual bit resets.	32-bit	ID Register 9
0x188	TRCIDR10	See individual bit resets.	32-bit	ID Register 10
0x18C	TRCIDR11	See individual bit resets.	32-bit	ID Register 11
0x190	TRCIDR12	See individual bit resets.	32-bit	ID Register 12
0x194	TRCIDR13	See individual bit resets.	32-bit	ID Register 13
0x1C0	TRCIMSPECO	See individual bit resets.	32-bit	IMP DEF Register 0
0x1E0	TRCIDR0	See individual bit resets.	32-bit	ID Register 0
0x1E4	TRCIDR1	See individual bit resets.	32-bit	ID Register 1
0x1E8	TRCIDR2	See individual bit resets.	32-bit	ID Register 2
0x1EC	TRCIDR3	See individual bit resets.	32-bit	ID Register 3
0x1F0	TRCIDR4	See individual bit resets.	32-bit	ID Register 4
0x1F4	TRCIDR5	See individual bit resets.	32-bit	ID Register 5
0x1F8	TRCIDR6	See individual bit resets.	32-bit	ID Register 6
0x1FC	TRCIDR7	See individual bit resets.	32-bit	ID Register 7
0x208	TRCRSCTLR2	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x20C	TRCRSCTLR3	See individual bit resets.	32-bit	Resource Selection Control Register <n>

Offset	Name	Reset	Width	Description
0x210	TRCRSCTLR4	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x214	TRCRSCTLR5	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x218	TRCRSCTLR6	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x21C	TRCRSCTLR7	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x220	TRCRSCTLR8	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x224	TRCRSCTLR9	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x228	TRCRSCTLR10	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x22C	TRCRSCTLR11	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x230	TRCRSCTLR12	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x234	TRCRSCTLR13	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x238	TRCRSCTLR14	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x23C	TRCRSCTLR15	See individual bit resets.	32-bit	Resource Selection Control Register <n>
0x280	TRCSSCCR0	See individual bit resets.	32-bit	Single-shot Comparator Control Register <n>
0x2A0	TRCSSCSR0	See individual bit resets.	32-bit	Single-shot Comparator Control Status Register <n>
0x304	TRCOSLSR	See individual bit resets.	32-bit	Trace OS Lock Status Register
0x310	TRCPDCR	See individual bit resets.	32-bit	PowerDown Control Register
0x314	TRCPDSR	See individual bit resets.	32-bit	PowerDown Status Register
0x400	TRCACVR0	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x408	TRCACVR1	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x410	TRCACVR2	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x418	TRCACVR3	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x420	TRCACVR4	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x428	TRCACVR5	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x430	TRCACVR6	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x438	TRCACVR7	See individual bit resets.	64-bit	Address Comparator Value Register <n>
0x480	TRCACATR0	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x488	TRCACATR1	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x490	TRCACATR2	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x498	TRCACATR3	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4A0	TRCACATR4	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4A8	TRCACATR5	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4B0	TRCACATR6	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x4B8	TRCACATR7	See individual bit resets.	64-bit	Address Comparator Access Type Register <n>
0x600	TRCCIDCVR0	See individual bit resets.	64-bit	Context Identifier Comparator Value Registers <n>
0x640	TRCVMIDCVR0	See individual bit resets.	64-bit	Virtual Context Identifier Comparator Value Register <n>
0x680	TRCCIDCCTLRO	See individual bit resets.	32-bit	Context Identifier Comparator Control Register 0
0x688	TRCVMIDCCTLRO	See individual bit resets.	32-bit	Virtual Context Identifier Comparator Control Register 0
0xF00	TRCITCTRL	See individual bit resets.	32-bit	Integration Mode Control Register
0xFA0	TRCCLAIMSET	See individual bit resets.	32-bit	Claim Tag Set Register
0xFA4	TRCCLAIMCLR	See individual bit resets.	32-bit	Claim Tag Clear Register

Offset	Name	Reset	Width	Description
0xFA8	TRCDEVAFF	See individual bit resets.	64-bit	Device Affinity Register
0xFB0	TRCLAR	See individual bit resets.	32-bit	Lock Access Register
0xFB4	TRCLSR	See individual bit resets.	32-bit	Lock Status Register
0xFB8	TRCAUTHSTATUS	See individual bit resets.	32-bit	Authentication Status Register
0xFBC	TRCDEVARCH	See individual bit resets.	32-bit	Device Architecture Register
0xFC0	TRCDEVID2	See individual bit resets.	32-bit	Device Configuration Register 2
0xFC4	TRCDEVID1	See individual bit resets.	32-bit	Device Configuration Register 1
0xFC8	TRCDEVID	See individual bit resets.	32-bit	Device Configuration Register
0xFCC	TRCDEVTYPE	See individual bit resets.	32-bit	Device Type Register
0xFD0	TRCPIDR4	See individual bit resets.	32-bit	Peripheral Identification Register 4
0xFD4	TRCPIDR5	See individual bit resets.	32-bit	Peripheral Identification Register 5
0xFD8	TRCPIDR6	See individual bit resets.	32-bit	Peripheral Identification Register 6
0xFDC	TRCPIDR7	See individual bit resets.	32-bit	Peripheral Identification Register 7
0xFE0	TRCPIDR0	See individual bit resets.	32-bit	Peripheral Identification Register 0
0xFE4	TRCPIDR1	See individual bit resets.	32-bit	Peripheral Identification Register 1
0xFE8	TRCPIDR2	See individual bit resets.	32-bit	Peripheral Identification Register 2
0xFEC	TRCPIDR3	See individual bit resets.	32-bit	Peripheral Identification Register 3
0xFF0	TRCCIDR0	See individual bit resets.	32-bit	Component Identification Register 0
0xFF4	TRCCIDR1	See individual bit resets.	32-bit	Component Identification Register 1
0xFF8	TRCCIDR2	See individual bit resets.	32-bit	Component Identification Register 2
0xFFC	TRCCIDR3	See individual bit resets.	32-bit	Component Identification Register 3

B.6.1 TRCAUXCTLR, Auxiliary Control Register

The function of this register is **IMPLEMENTATION DEFINED**.

Configurations

External register TRCAUXCTLR bits [31:0] are architecturally mapped to AArch64 System register [A.11.8 TRCAUXCTLR, Auxiliary Control Register](#) on page 500 bits [31:0].

Attributes

Width

32

Component

ETE

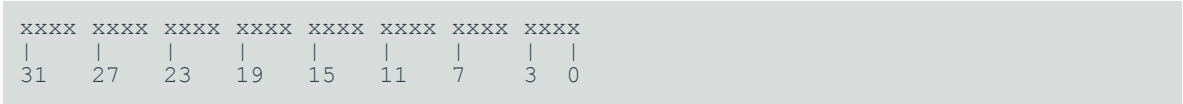
Register offset

0x018

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-90: ext_trcauxctlr bit assignments

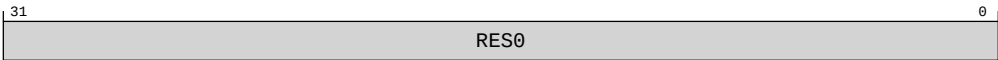


Table B-182: TRCAUXCTLR bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

If this register is nonzero then it might cause the behavior of a trace unit to contradict this architecture specification. See the documentation of the specific implementation for information about the IMPLEMENTATION DEFINED support for this register.

Component	Offset	Instance	Range
ETE	0x018	TRCAUXCTLR	None

This interface is accessible as follows:

When `OSLockStatus() || !AllowExternalTraceAccess() || !IsTraceCorePowered()`

ERROR

Otherwise

RW

B.6.2 TRCIDR8, ID Register 8

Returns the maximum speculation depth of the instruction trace element stream.

Configurations

External register TRCIDR8 bits [31:0] are architecturally mapped to AArch64 System register [A.11.1 TRCIDR8, ID Register 8](#) on page 488 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0x180

Access type

See bit descriptions

Reset value

0000 0000 0000 0000 0000 0000 0000 0000

Bit descriptions

Figure B-91: ext_trcldr8 bit assignments

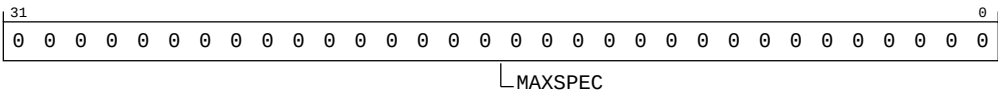


Table B-184: TRCIDR8 bit descriptions

Bits	Name	Description	Reset
[31:0]	MAXSPEC	Indicates the maximum speculation depth of the instruction trace element stream. This is the maximum number of PO elements in the trace element stream that can be speculative at any time. 0b00000000000000000000000000000000	0x00000000

Accessibility

Component	Offset	Instance	Range
ETE	0x180	TRCIDR8	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.3 TRCIDR9, ID Register 9

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR9 bits [31:0] are architecturally mapped to AArch64 System register [A.11.3 TRCIDR9, ID Register 9](#) on page 492 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0x184

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-92: ext_trcidr9 bit assignments



Table B-186: TRCIDR9 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
ETE	0x184	TRCIDR9	None

This interface is accessible as follows:

When `OSLockStatus() || !IsTraceCorePowered()`
ERROR

Otherwise
RO

B.6.4 TRCIDR10, ID Register 10

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR10 bits [31:0] are architecturally mapped to AArch64 System register [A.11.4 TRCIDR10, ID Register 10](#) on page 494 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0x188

Access type
See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-93: ext_trcidr10 bit assignments

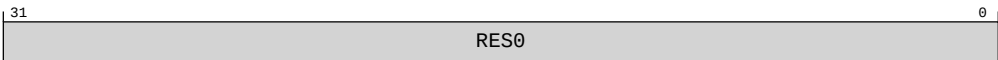


Table B-188: TRCIDR10 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
ETE	0x188	TRCIDR10	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.5 TRCIDR11, ID Register 11

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR11 bits [31:0] are architecturally mapped to AArch64 System register [A.11.5 TRCIDR11, ID Register 11](#) on page 495 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0x18C

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-94: ext_trcidr11 bit assignments

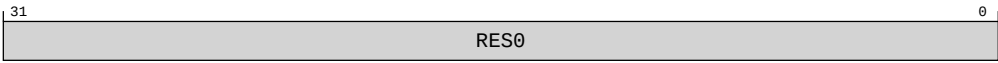


Table B-190: TRCIDR11 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
ETE	0x18C	TRCIDR11	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.6 TRCIDR12, ID Register 12

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR12 bits [31:0] are architecturally mapped to AArch64 System register [A.11.6 TRCIDR12, ID Register 12](#) on page 497 bits [31:0].

Attributes

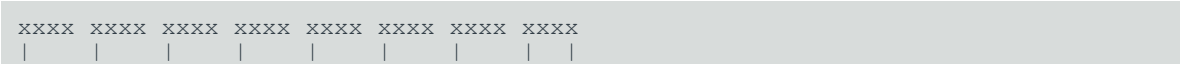
Width
32

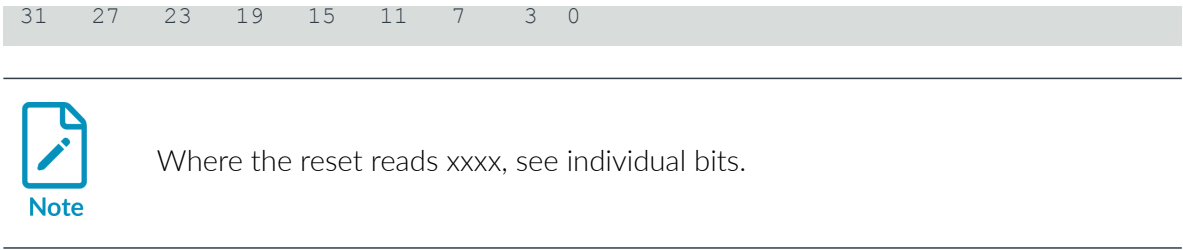
Component
ETE

Register offset
0x190

Access type
See bit descriptions

Reset value





Bit descriptions

Figure B-95: ext_trcidr12 bit assignments

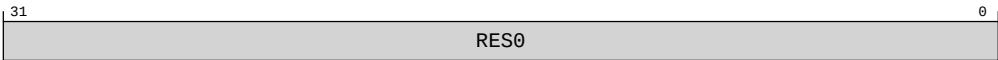


Table B-192: TRCIDR12 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
ETE	0x190	TRCIDR12	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.7 TRCIDR13, ID Register 13

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR13 bits [31:0] are architecturally mapped to AArch64 System register [A.11.7 TRCIDR13, ID Register 13](#) on page 499 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset

0x194

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-96: ext_trcidr13 bit assignments



Table B-194: TRCIDR13 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
ETE	0x194	TRCIDR13	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.8 TRCIMSPECO, IMP DEF Register 0

TRCIMSPECO shows the presence of any **IMPLEMENTATION DEFINED** features, and provides an interface to enable the features that are provided.

Configurations

External register TRCIMSPECO bits [31:0] are architecturally mapped to AArch64 System register [A.11.2 TRCIMSPECO, IMP DEF Register 0](#) on page 489 bits [31:0].

Attributes

Width

32

Component

ETE

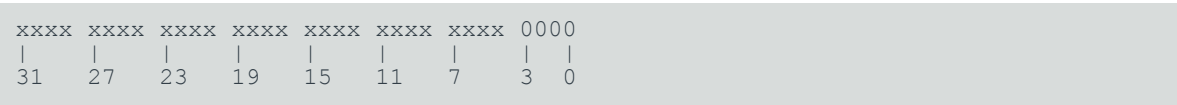
Register offset

0x1C0

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-97: ext_trcimspec0 bit assignments

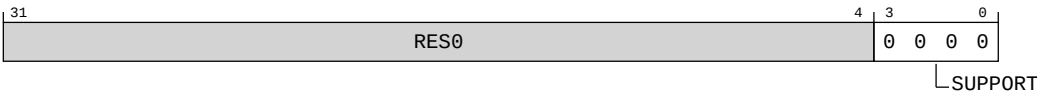


Table B-196: TRCIMSPECO bit descriptions

Bits	Name	Description	Reset
[31:4]	RES0	Reserved	RES0
[3:0]	SUPPORT	Indicates whether the implementation supports IMPLEMENTATION DEFINED features. 0b0000 No IMPLEMENTATION DEFINED features are supported.	0b0000

Accessibility

Component	Offset	Instance	Range
ETE	0x1C0	TRCIMSPEC0	None

This interface is accessible as follows:

When OSLockStatus() || !AllowExternalTraceAccess() || !IsTraceCorePowered()
ERROR

Otherwise
RW

B.6.9 TRCIDR0, ID Register 0

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR0 bits [31:0] are architecturally mapped to AArch64 System register [A.11.9 TRCIDR0, ID Register 0](#) on page 503 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0x1E0

Access type
See bit descriptions

Reset value

x010	1000	1xxx	xxx0	00xx	111x	1010	000x
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-98: ext_trcidr0 bit assignments

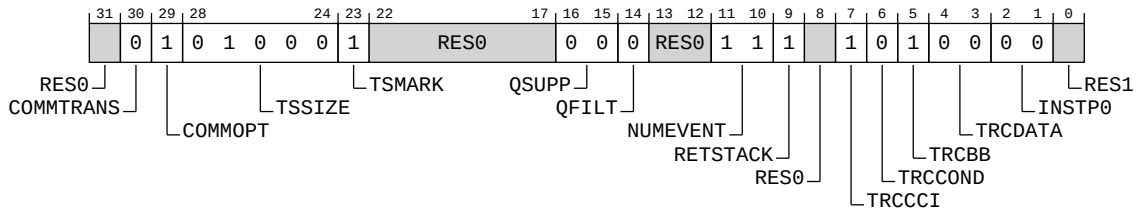


Table B-198: TRCIDR0 bit descriptions

Bits	Name	Description	Reset
[31]	RES0	Reserved	RES0
[30]	COMMTTRANS	Transaction Start element behavior. 0b0 Transaction Start elements are P0 elements.	0b0
[29]	COMMOPT	Indicates the contents and encodings of Cycle count packets. 0b1 Commit mode 1.	0b1
[28:24]	TSSIZE	Indicates that the trace unit implements Global timestamping and the size of the timestamp value. 0b01000 Global timestamping implemented with a 64-bit timestamp value.	0b01000
[23]	TSMARK	Indicates whether Timestamp Marker elements are generated. 0b1 Timestamp Marker elements are generated.	0b1
[22:17]	RES0	Reserved	RES0
[16:15]	QSUPP	Indicates that the trace unit implements Q element support. 0b00 Q element support is not implemented.	0b00
[14]	QFILT	Indicates if the trace unit implements Q element filtering. 0b0 Q element filtering is not implemented.	0b0
[13:12]	RES0	Reserved	RES0
[11:10]	NUMEVENT	Indicates the number of ETEEvents implemented. 0b11 The trace unit supports 4 ETEEvents.	0b11
[9]	RETSTACK	Indicates if the trace unit supports the return stack. 0b1 Return stack implemented.	0b1
[8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7]	TRCCCI	Indicates if the trace unit implements cycle counting. 0b1 Cycle counting implemented.	0b1
[6]	TRCCOND	Indicates if the trace unit implements conditional instruction tracing. Conditional instruction tracing is not implemented in ETE and this field is reserved for other trace architectures. 0b0 Conditional instruction tracing not implemented.	0b0
[5]	TRCBB	Indicates if the trace unit implements branch broadcasting. 0b1 Branch broadcasting implemented.	0b1
[4:3]	TRCDATA	Indicates if the trace unit implements data tracing. Data tracing is not implemented in ETE and this field is reserved for other trace architectures. 0b00 Tracing of data addresses and data values is not implemented.	0b00
[2:1]	INSTPO	Indicates if load and store instructions are PO instructions. Load and store instructions as PO instructions is not implemented in ETE and this field is reserved for other trace architectures. 0b00 Load and store instructions are not PO instructions.	0b00
[0]	RES1	Reserved	RES1

Accessibility

Component	Offset	Instance	Range
ETE	0x1E0	TRCIDR0	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.10 TRCIDR1, ID Register 1

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR1 bits [31:0] are architecturally mapped to AArch64 System register [A.11.10 TRCIDR1, ID Register 1](#) on page 506 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0x1E4

Access type

See bit descriptions

Reset value

0100	0001	xxxx	xxxx	xxxx	1111	1111	0000	
31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-99: ext_trcidr1 bit assignments

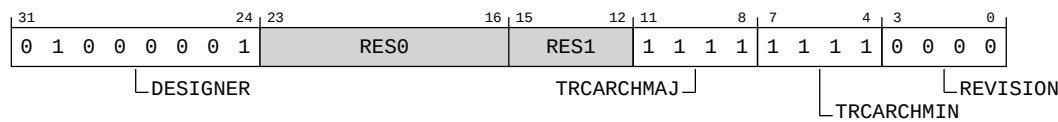


Table B-200: TRCIDR1 bit descriptions

Bits	Name	Description	Reset
[31:24]	DESIGNER	Indicates which company designed the trace unit. The permitted values of this field are the same as AArch64-MIDR_EL1.Implementer. 0b01000001 Arm Limited	0x41
[23:16]	RES0	Reserved	RES0
[15:12]	RES1	Reserved	RES1
[11:8]	TRCARCHMAJ	Major architecture version. 0b1111 If both TRCARCHMAJ and TRCARCHMIN == 0xF then refer to ext-TRCDEVARCH.	0b1111
[7:4]	TRCARCHMIN	Minor architecture version. 0b1111 If both TRCARCHMAJ and TRCARCHMIN == 0xF then refer to ext-TRCDEVARCH.	0b1111
[3:0]	REVISION	Indicates the major revision of the product 0b0000 r0p1	0b0000

Accessibility

Component	Offset	Instance	Range
ETE	0x1E4	TRCIDR1	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.11 TRCIDR2, ID Register 2

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR2 bits [31:0] are architecturally mapped to AArch64 System register [A.11.11 TRCIDR2, ID Register 2](#) on page 508 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0x1E8

Access type
See bit descriptions

Reset value

1100	000x	xxxx	xxxx	x001	0000	1000	1000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-100: ext_trcidr2 bit assignments

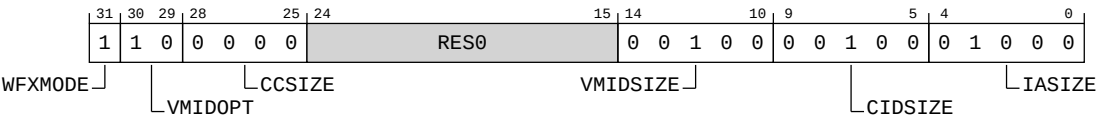


Table B-202: TRCIDR2 bit descriptions

Bits	Name	Description	Reset
[31]	WFXMODE	Indicates whether WFI and WFE instructions are classified as P0 instructions: 0b1 WFI and WFE instructions are classified as P0 instructions.	0b1
[30:29]	VMIDOPT	Indicates the options for Virtual context identifier selection. 0b10 Virtual context identifier selection not supported. ext-TRCCONFIGR.VMIDOPT is RES1 .	0b10
[28:25]	CCSIZE	Indicates the size of the cycle counter. 0b0000 The cycle counter is 12 bits in length.	0b0000
[24:15]	RES0	Reserved	RES0
[14:10]	VMIDSIZE	Indicates the trace unit Virtual context identifier size. 0b00100 32-bit Virtual context identifier size.	0b00100
[9:5]	CIDSIZE	Indicates the Context identifier size. 0b00100 32-bit Context identifier size.	0b00100
[4:0]	IASIZE	Virtual instruction address size. 0b01000 Maximum of 64-bit instruction address size.	0b01000

Accessibility

Component	Offset	Instance	Range
ETE	0x1E8	TRCIDR2	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.12 TRCIDR3, ID Register 3

Returns the base architecture of the trace unit.

Configurations

External register TRCIDR3 bits [31:0] are architecturally mapped to AArch64 System register [A.11.12 TRCIDR3, ID Register 3](#) on page 510 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0x1EC

Access type

See bit descriptions

Reset value

0000	0001	x111	1111	xx00	0000	0000	0100
31	27	23	19	15	11	7	3
							0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-101: ext_trcidr3 bit assignments

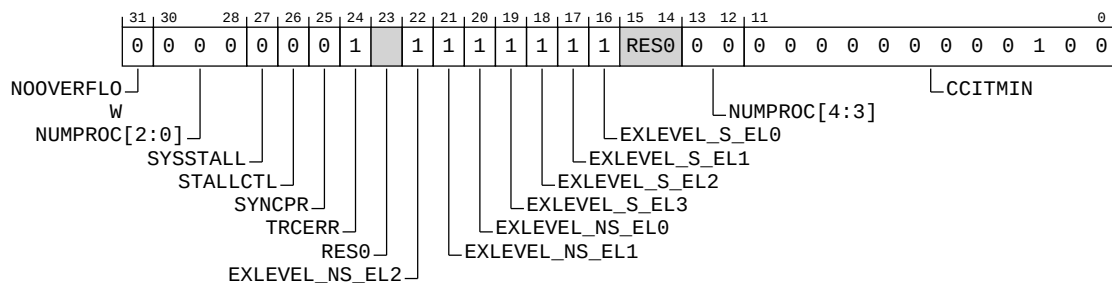


Table B-204: TRCIDR3 bit descriptions

Bits	Name	Description	Reset
[31]	NOOVERFLOW	Indicates if overflow prevention is implemented. 0b0 Overflow prevention is not implemented.	0b0
[27]	SYSSTALL	Indicates if stalling of the PE is permitted. 0b0 Stalling of the PE is not permitted.	0b0
[26]	STALLCTL	Indicates if trace unit implements stalling of the PE. 0b0 Stalling of the PE is not implemented.	0b0
[25]	SYNCPR	Indicates if an implementation has a fixed synchronization period. 0b0 ext-TRCSYNCPR is read/write so software can change the synchronization period.	0b0
[24]	TRCERR	Indicates forced tracing of System Error exceptions is implemented. 0b1 Forced tracing of System Error exceptions is implemented.	0b1
[23]	RES0	Reserved	RES0
[22]	EXLEVEL_NS_EL2	Indicates if Non-secure EL2 is implemented. 0b1 Non-secure EL2 is implemented.	0b1
[21]	EXLEVEL_NS_EL1	Indicates if Non-secure EL1 is implemented. 0b1 Non-secure EL1 is implemented.	0b1
[20]	EXLEVEL_NS_ELO	Indicates if Non-secure ELO is implemented. 0b1 Non-secure ELO is implemented.	0b1
[19]	EXLEVEL_S_EL3	Indicates if EL3 is implemented. 0b1 EL3 is implemented.	0b1
[18]	EXLEVEL_S_EL2	Indicates if Secure EL2 is implemented. 0b1 Secure EL2 is implemented.	0b1
[17]	EXLEVEL_S_EL1	Indicates if Secure EL1 is implemented. 0b1 Secure EL1 is implemented.	0b1
[16]	EXLEVEL_S_ELO	Indicates if Secure ELO is implemented. 0b1 Secure ELO is implemented.	0b1
[15:14]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[13:12, 30:28]	NUMPROC	Indicates the number of PEs available for tracing. 0b00000 The trace unit can trace one PE.	0b00000
[11:0]	CCITMIN	Indicates the minimum value that can be programmed in ext-TRCCCCTLR.THRESHOLD. If ext-TRCIDR0.TRCCCI == 1 then the minimum value of this field is 0x001. If ext-TRCIDR0.TRCCCI == 0 then this field is zero. 0b000000000100	0x004

Accessibility

Component	Offset	Instance	Range
ETE	0x1EC	TRCIDR3	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.13 TRCIDR4, ID Register 4

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR4 bits [31:0] are architecturally mapped to AArch64 System register [A.11.13 TRCIDR4, ID Register 4](#) on page 513 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0x1F0

Access type

See bit descriptions

Reset value

0001	0001	0001	0111	0000	xxx0	0000	0100
31	27	23	19	15	11	7	3 0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-102: ext_trcidr4 bit assignments

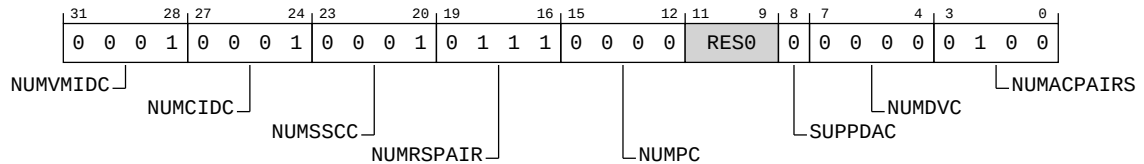


Table B-206: TRCIDR4 bit descriptions

Bits	Name	Description	Reset
[31:28]	NUMVMIDC	Indicates the number of Virtual Context Identifier Comparators that are available for tracing. 0b0001 The implementation has one Virtual Context Identifier Comparator.	0b0001
[27:24]	NUMCIDC	Indicates the number of Context Identifier Comparators that are available for tracing. 0b0001 The implementation has one Context Identifier Comparator.	0b0001
[23:20]	NUMSSCC	Indicates the number of Single-shot Comparator Controls that are available for tracing. 0b0001 The implementation has one Single-shot Comparator Control.	0b0001
[19:16]	NUMRSPAIR	Indicates the number of resource selector pairs that are available for tracing. 0b0111 The implementation has eight resource selector pairs.	0b0111
[15:12]	NUMPC	Indicates the number of PE Comparator Inputs that are available for tracing. 0b0000 No PE Comparator Inputs are available.	0b0000
[11:9]	RES0	Reserved	RES0
[8]	SUPPDAC	Indicates whether data address comparisons are implemented. Data address comparisons are not implemented in ETE and are reserved for other trace architectures. Allocated in other trace architectures. 0b0 Data address comparisons not implemented.	0b0
[7:4]	NUMDVC	Indicates the number of data value comparators. Data value comparators are not implemented in ETE and are reserved for other trace architectures. Allocated in other trace architectures. 0b0000 No data value comparators implemented.	0b0000

Bits	Name	Description	Reset
[3:0]	NUMACPAIRS	Indicates the number of Address Comparator pairs that are available for tracing. 0b0100 The implementation has four Address Comparator pairs.	0b0100

Accessibility

Component	Offset	Instance	Range
ETE	0x1F0	TRCIDR4	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RO

B.6.14 TRCIDR5, ID Register 5

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR5 bits [31:0] are architecturally mapped to AArch64 System register [A.11.14 TRCIDR5, ID Register 5](#) on page 515 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0x1F4

Access type
See bit descriptions

Reset value

x010	100x	0100	0111	xxxx	1001	1111	1111
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-103: ext_trcidr5 bit assignments

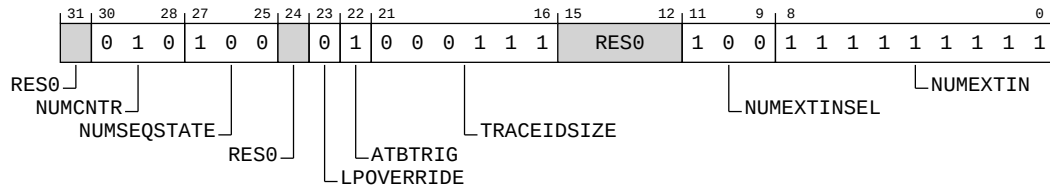


Table B-208: TRCIDR5 bit descriptions

Bits	Name	Description	Reset
[31]	RES0	Reserved	RES0
[30:28]	NUMCNTR	Indicates the number of Counters that are available for tracing. 0b010 Two Counters implemented.	0b010
[27:25]	NUMSEQSTATE	Indicates if the Sequencer is implemented and the number of Sequencer states that are implemented. 0b100 Four Sequencer states are implemented.	0b100
[24]	RES0	Reserved	RES0
[23]	LPOVERRIDE	Indicates support for Low-power Override Mode. 0b0 The trace unit does not support Low-power Override Mode.	0b0
[22]	ATBTRIG	Indicates if the implementation can support ATB triggers. 0b1 The implementation supports ATB triggers.	0b1
[21:16]	TRACEIDSIZE	Indicates the trace ID width. 0b000111 The implementation supports a 7-bit trace ID.	0b000111
[15:12]	RES0	Reserved	RES0
[11:9]	NUMEXTINSEL	Indicates how many External Input Selector resources are implemented. 0b100 4 External Input Selector resources are available.	0b100
[8:0]	NUMEXTIN	Indicates how many External Inputs are implemented. 0b11111111 Unified PMU event selection.	0b11111111

Accessibility

Component	Offset	Instance	Range
ETE	0x1F4	TRCIDR5	None

This interface is accessible as follows:

When `OSLockStatus() || !IsTraceCorePowered()`
ERROR

Otherwise
RO

B.6.15 TRCIDR6, ID Register 6

Returns the tracing capabilities of the trace unit.

Configurations
External register TRCIDR6 bits [31:0] are architecturally mapped to AArch64 System register [A.11.15 TRCIDR6, ID Register 6](#) on page 518 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0x1F8

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	x000
31	27	23	19	15	11	7	3 0


Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-104: ext_trcidr6 bit assignments

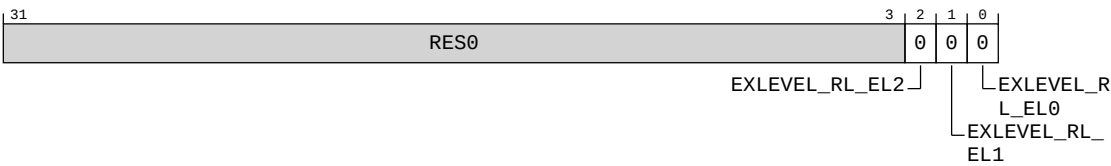


Table B-210: TRCIDR6 bit descriptions

Bits	Name	Description	Reset
[31:3]	RES0	Reserved	RES0
[2]	EXLEVEL_RL_EL2	Indicates if Realm EL2 is implemented. 0b0 Realm EL2 is not implemented.	0b0
[1]	EXLEVEL_RL_EL1	Indicates if Realm EL1 is implemented. 0b0 Realm EL1 is not implemented.	0b0
[0]	EXLEVEL_RL_ELO	Indicates if Realm ELO is implemented. 0b0 Realm ELO is not implemented.	0b0

Accessibility

Component	Offset	Instance	Range
ETE	0x1F8	TRCIDR6	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.16 TRCIDR7, ID Register 7

Returns the tracing capabilities of the trace unit.

Configurations

External register TRCIDR7 bits [31:0] are architecturally mapped to AArch64 System register [A.11.16 TRCIDR7, ID Register 7](#) on page 520 bits [31:0].

Attributes

Width

32

Component

ETE

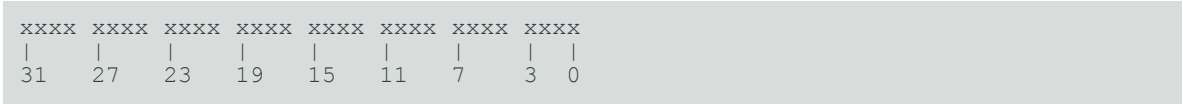
Register offset

0x1FC

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-105: ext_trcidr7 bit assignments

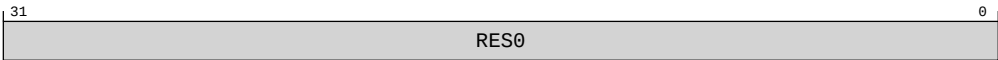


Table B-212: TRCIDR7 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

Component	Offset	Instance	Range
ETE	0x1FC	TRCIDR7	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.17 TRCITCTRL, Integration Mode Control Register

A component can use TRCITCTRL to dynamically switch between functional mode and integration mode. In integration mode, topology detection is enabled. After switching to integration mode and performing integration tests or topology detection, reset the system to ensure correct behavior of CoreSight and other connected system components.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xF00

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-106: ext_trcitctrl bit assignments

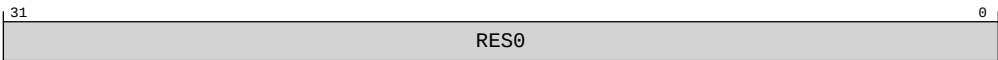


Table B-214: TRCITCTRL bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are IMPLEMENTATION DEFINED when the trace unit is not in the Idle state.

Component	Offset	Instance	Range
ETE	0xF00	TRCITCTRL	None

This interface is accessible as follows:

When `OSLockStatus() || !AllowExternalTraceAccess() || !IsTraceCorePowered()`

ERROR

Otherwise

RW

B.6.18 TRCCLAIMSET, Claim Tag Set Register

In conjunction with ext-TRCCLAIMCLR, provides Claim Tag bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent.

For additional information, see the CoreSight Architecture Specification.

Configurations

The number of claim tag bits implemented is four, that is, SET[3:0] reads as 0b1111.

External register TRCCLAIMSET bits [31:0] are architecturally mapped to AArch64 System register [A.11.18 TRCCLAIMSET, Claim Tag Set Register](#) on page 523 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0xFA0

Access type

RAOW1S

Reset value

0000 0000 0000 0000 0000 0000 0000 1111

Bit descriptions

Figure B-107: ext_trcclaimset bit assignments

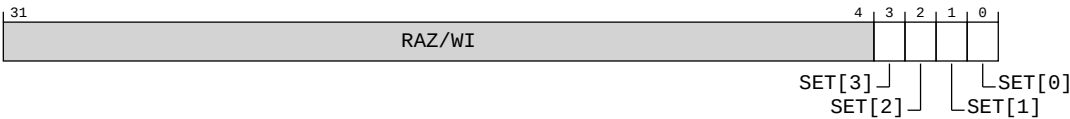


Table B-216: TRCCLAIMSET bit descriptions

Bits	Name	Description	Reset
[31:4]	RAZ/WI	Reserved	RAZ/WI

Bits	Name	Description	Reset
[3]	SET[3]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1
[2]	SET[2]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1
[1]	SET[1]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1
[0]	SET[0]	Claim Tag Set. Indicates whether Claim Tag bit <m> is implemented, and is used to set Claim Tag bit <m> to 1. 0b0 On a read: Claim Tag bit <m> is not implemented. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is implemented. On a write: Set Claim Tag bit <m> to 1.	0b1

Accessibility

Component	Offset	Instance	Range
ETE	0xFA0	TRCCCLAIMSET	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()

ERROR

Otherwise
RW

B.6.19 TRCCLAIMCLR, Claim Tag Clear Register

In conjunction with ext-TRCCLAIMSET, provides Claim Tag bits that can be separately set and cleared to indicate whether functionality is in use by a debug agent.

For additional information, see the CoreSight Architecture Specification.

Configurations

External register TRCCLAIMCLR bits [31:0] are architecturally mapped to AArch64 System register [A.11.19 TRCCLAIMCLR, Claim Tag Clear Register](#) on page 527 bits [31:0].

Attributes

Width

32

Component

ETE

Register offset

0xFA4

Access type

RW1C

Reset value

0000 0000 0000 0000 0000 0000 0000 0000

Bit descriptions

Figure B-108: ext_trclaimclr bit assignments

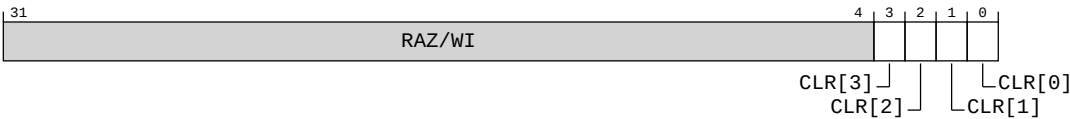


Table B-218: TRCCLAIMCLR bit descriptions

Bits	Name	Description	Reset
[31:4]	RAZ/WI	Reserved	RAZ/WI

Bits	Name	Description	Reset
[3]	CLR[3]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0
[2]	CLR[2]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0
[1]	CLR[1]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0
[0]	CLR[0]	Claim Tag Clear. Indicates the current status of Claim Tag bit <m>, and is used to clear Claim Tag bit <m> to 0. 0b0 On a read: Claim Tag bit <m> is not set. On a write: Ignored. 0b1 On a read: Claim Tag bit <m> is set. On a write: Clear Claim tag bit <m> to 0.	0b0

Accessibility

Component	Offset	Instance	Range
ETE	0xFA4	TRCCCLAIMCLR	None

This interface is accessible as follows:

When OSLockStatus() || !IsTraceCorePowered()
ERROR

Otherwise
RW

B.6.20 TRCDEVARCH, Device Architecture Register

Provides discovery information for the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

External register TRCDEVARCH bits [31:0] are architecturally mapped to AArch64 System register [A.11.20 TRCDEVARCH, Device Architecture Register](#) on page 530 bits [31:0].

Attributes

Width
32

Component
ETE

Register offset
0xFBC

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3
							0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-109: ext_trcdevarch bit assignments

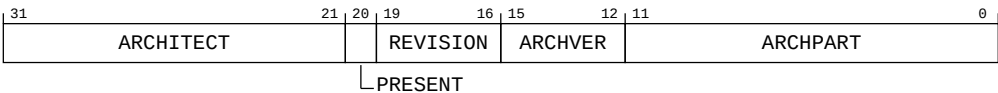


Table B-220: TRCDEVARCH bit descriptions

Bits	Name	Description	Reset
[31:21]	ARCHITECT	Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code. 0b01000111011 JEP106 continuation code 0x4, ID code 0x3B. Arm Limited. Other values are defined by the JEDEC JEP106 standard. This field reads as 0x23B.	11 {x}
[20]	PRESENT	DEVARCH Present. Defines that the DEVARCH register is present. 0b1 Device Architecture information present.	x
[19:16]	REVISION	Revision. Defines the architecture revision of the component. 0b0000 ETEv1.0, FEAT_ETE. 0b0001 ETEv1.1, FEAT_ETEv1p1. 0b0010 ETEv1.2, FEAT_ETEv1p2.	xxxx
[15:12]	ARCHVER	Architecture Version. Defines the architecture version of the component. 0b0101 ETEv1. ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12]. This field reads as 0x5.	xxxx
[11:0]	ARCHPART	Architecture Part. Defines the architecture of the component. 0b101000010011 Arm PE trace architecture. ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0]. This field reads as 0xA13.	12 {x}

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFBC	TRCDEVARCH	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise
RO

B.6.21 TRCDEVID2, Device Configuration Register 2

Provides discovery information for the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width
32

Component
ETE

Register offset
0xFC0

Access type
See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-110: ext_trcdevid2 bit assignments

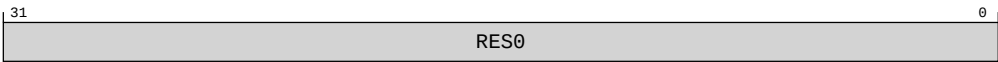


Table B-222: TRCDEVID2 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFC0	TRCDEVID2	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.22 TRCDEVID1, Device Configuration Register 1

Provides discovery information for the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFC4

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-111: ext_trcdevid1 bit assignments

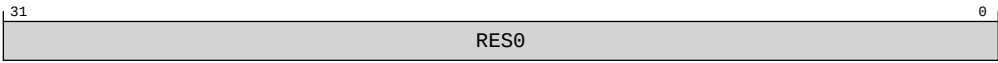


Table B-224: TRCDEVID1 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFC4	TRCDEVID1	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.23 TRCDEVID, Device Configuration Register

Provides discovery information for the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

External register TRCDEVID bits [31:0] are architecturally mapped to AArch64 System register [A.11.17 TRCDEVID, Device Configuration Register](#) on page 522 bits [31:0].

Attributes

Width

32

Component

ETE

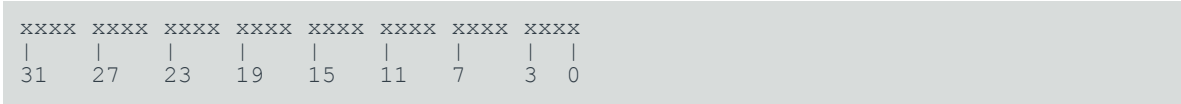
Register offset

0xFC8

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-112: ext_trcdevid bit assignments

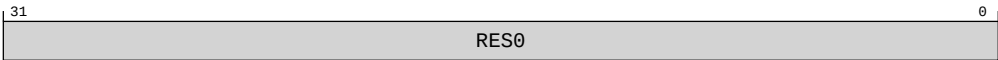


Table B-226: TRCDEVID bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFC8	TRCDEVID	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.24 TRCDEVTYPE, Device Type Register

Provides discovery information for the component. If the part number field is not recognized, a debugger can report the information that is provided by TRCDEVTYPE about the component instead.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFCC

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-113: ext_trcdevtype bit assignments



Table B-228: TRCDEVTYPE bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SUB	Component sub-type. 0b0001 When MAJOR == 0x3 (Trace source): Associated with a PE. This field reads as 0x1.	0b0001
[3:0]	MAJOR	Component major type. 0b0011 Trace source. Other values are defined by the CoreSight Architecture. This field reads as 0x3.	0b0011

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFCC	TRCDEVTYPE	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.25 TRCPIDR4, Peripheral Identification Register 4

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFD0

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0100
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-114: ext_trcpidr4 bit assignments

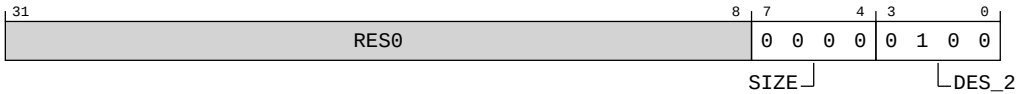


Table B-230: TRCPIDR4 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SIZE	Size of the component. The distance from the start of the address space used by this component to the end of the component identification registers. A value of 0b0000 means one of the following is true: - The component uses a single 4KB block. - The component uses an IMPLEMENTATION DEFINED number of 4KB blocks. Any other value means the component occupies $2^{\text{TRCPIDR4.SIZE}}$ 4KB blocks. 0b0000 The component uses a single 4KB block	0b0000
[3:0]	DES_2	Designer, JEP106 continuation code. This is the JEDEC-assigned JEP106 bank identifier for the designer of the component, minus 1. The code identifies the designer of the component, which might not be the same as the implementer of the device containing the component. To obtain a number, or to see the assignment of these codes, contact JEDEC http://www.jedec.org. 0b0100 Arm Limited	0b0100

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFD0	TRCPIDR4	None

This interface is accessible as follows:

When **!IsTraceCorePowered()**

ERROR

Otherwise

RO

B.6.26 TRCPIDR5, Peripheral Identification Register 5

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

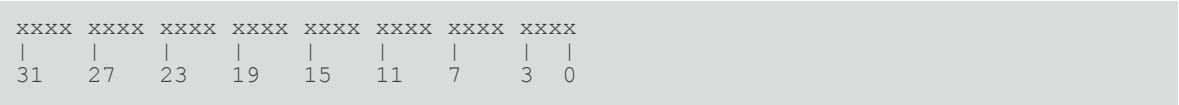
Register offset

0xFD4

Access type

See bit descriptions

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-115: ext_trcpidr5 bit assignments



Table B-232: TRCPIDR5 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFD4	TRCPIDR5	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.27 TRCPIDR6, Peripheral Identification Register 6

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFD8

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-116: ext_trcpidr6 bit assignments

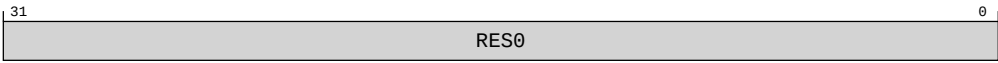


Table B-234: TRCPIDR6 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFD8	TRCPIDR6	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.28 TRCPIDR7, Peripheral Identification Register 7

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

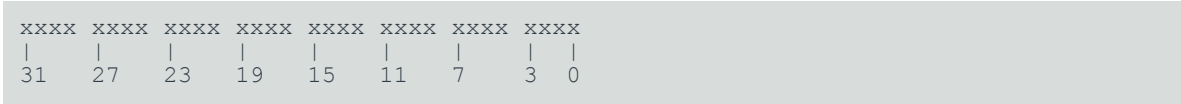
Register offset

0xFDC

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-117: ext_trcpidr7 bit assignments

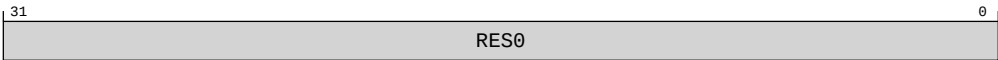


Table B-236: TRCPIDR7 bit descriptions

Bits	Name	Description	Reset
[31:0]	RES0	Reserved	RES0

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFDC	TRCPIDR7	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.29 TRCPIDR0, Peripheral Identification Register 0

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFE0

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-118: ext_trcpidr0 bit assignments



Table B-238: TRCPIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PART_0	Part number, bits [7:0]. The part number is selected by the designer of the component, and is stored in ext-TRCPIDR1.PART_1 and TRCPIDR0.PART_0. 0b10001110 Neoverse N3	0x8E

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFE0	TRCPIDR0	None

This interface is accessible as follows:

When `!IsTraceCorePowered()`
ERROR

Otherwise
RO

B.6.30 TRCPIDR1, Peripheral Identification Register 1

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width
32

Component
ETE

Register offset
0xFE4

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1011	1101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-119: ext_trcpidr1 bit assignments



Table B-240: TRCPIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	DES_0	Designer, JEP106 identification code, bits [3:0]. TRCPIDR1.DES_0 and ext-TRCPIDR2.DES_1 together form the JEDEC-assigned JEP106 identification code for the designer of the component. The parity bit in the JEP106 identification code is not included. The code identifies the designer of the component, which might not be not the same as the implementer of the device containing the component. To obtain a number, or to see the assignment of these codes, contact JEDEC http://www.jedec.org . 0b1011 Arm Limited	0b1011
[3:0]	PART_1	Part number, bits [11:8]. The part number is selected by the designer of the component, and is stored in TRCPIDR1.PART_1 and ext-TRCPIDR0.PART_0. 0b1101 Neoverse N3	0b1101

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFE4	TRCPIDR1	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.31 TRCPIDR2, Peripheral Identification Register 2

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFE8

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	1011
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-120: ext_trcpidr2 bit assignments

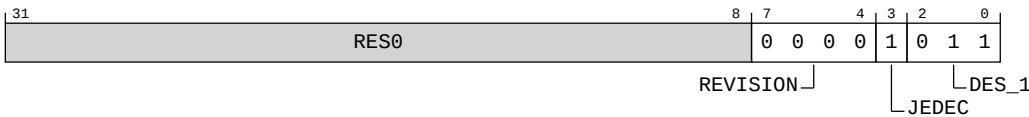


Table B-242: TRCPIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVISION	Component major revision. TRCPIDR2.REVISION and ext-TRCPIDR3.REVAND together form the revision number of the component, with TRCPIDR2.REVISION being the most significant part and ext-TRCPIDR3.REVAND the least significant part. When a component is changed, TRCPIDR2.REVISION or ext-TRCPIDR3.REVAND are increased to ensure that software can differentiate the different revisions of the component. ext-TRCPIDR3.REVAND should be set to 0b0000 when TRCPIDR2.REVISION is increased. 0b0000 rOp1	0b0000
[3]	JEDEC	JEDEC-assigned JEP106 implementer code is used. 0b1	0b1
[2:0]	DES_1	Designer, JEP106 identification code, bits [6:4]. ext-TRCPIDR1.DES_0 and TRCPIDR2.DES_1 together form the JEDEC-assigned JEP106 identification code for the designer of the component. The parity bit in the JEP106 identification code is not included. The code identifies the designer of the component, which might not be not the same as the implementer of the device containing the component. To obtain a number, or to see the assignment of these codes, contact JEDEC http://www.jedec.org . 0b011 Arm Limited	0b011

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFE8	TRCPIDR2	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.32 TRCPIDR3, Peripheral Identification Register 3

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFEC

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-121: ext_trcpidr3 bit assignments

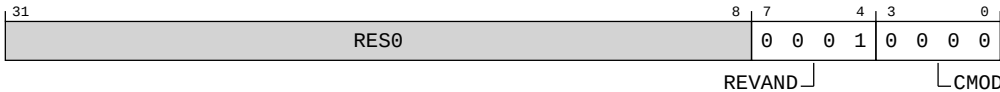


Table B-244: TRCPIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVAND	Component minor revision. ext-TRCPIDR2.REVISION and TRCPIDR3.REVAND together form the revision number of the component, with ext-TRCPIDR2.REVISION being the most significant part and TRCPIDR3.REVAND the least significant part. When a component is changed, ext-TRCPIDR2.REVISION or TRCPIDR3.REVAND are increased to ensure that software can differentiate the different revisions of the component. TRCPIDR3.REVAND should be set to 0b0000 when ext-TRCPIDR2.REVISION is increased. 0b0001 rOp1	0b0001
[3:0]	CMOD	Customer Modified. Indicates the component has been modified. A value of 0b0000 means the component is not modified from the original design. Any other value means the component has been modified in an IMPLEMENTATION DEFINED way. 0b0000	0b0000

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFEC	TRCPIDR3	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.6.33 TRCCIDR0, Component Identification Register 0

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

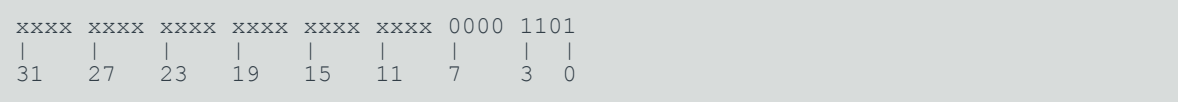
Register offset

0xFF0

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-122: ext_trccidr0 bit assignments

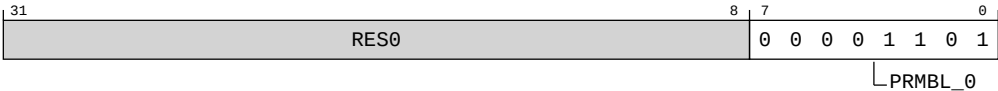


Table B-246: TRCCIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_0	Component identification preamble, segment 0. 0b00001101	0x0D

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFF0	TRCCIDR0	None

This interface is accessible as follows:

When `!IsTraceCorePowered()`
ERROR

Otherwise
RO

B.6.34 TRCCIDR1, Component Identification Register 1

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width
32

Component
ETE

Register offset
0xFF4

Access type
See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1001	0000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-123: ext_trccidr1 bit assignments

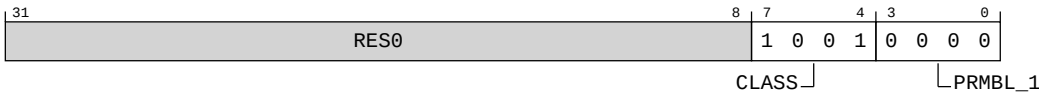


Table B-248: TRCCIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	CLASS	Component class. 0b1001 CoreSight peripheral. Other values are defined by the CoreSight Architecture. This field reads as 0x9.	0b1001
[3:0]	PRMBL_1	Component identification preamble, segment 1. 0b0000	0b0000

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFF4	TRCCIDR1	None

This interface is accessible as follows:

When **!IsTraceCorePowered()**

ERROR

Otherwise

RO

B.6.35 TRCCIDR2, Component Identification Register 2

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

Register offset

0xFF8

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-124: ext_trccidr2 bit assignments



Table B-250: TRCCIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_2	Component identification preamble, segment 2. 0b00000101	0x05

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFF8	TRCCIDR2	None

This interface is accessible as follows:

When **!IsTraceCorePowered()**

ERROR

Otherwise

RO

B.6.36 TRCCIDR3, Component Identification Register 3

Provides discovery information about the component.

For additional information, see the CoreSight Architecture Specification.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ETE

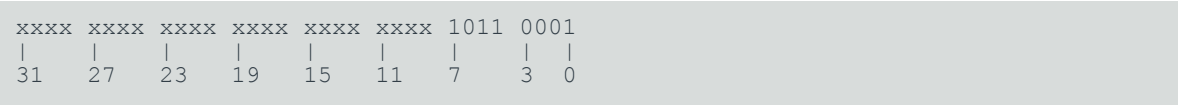
Register offset

0xFFC

Access type

See bit descriptions

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-125: ext_trccidr3 bit assignments



Table B-252: TRCCIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_3	Component identification preamble, segment 3. 0b10110001	0xB1

Accessibility

External debugger accesses to this register are unaffected by the OS Lock.

Component	Offset	Instance	Range
ETE	0xFFC	TRCCIDR3	None

This interface is accessible as follows:

When !IsTraceCorePowered()

ERROR

Otherwise

RO

B.7 External RAS registers summary

The summary table provides an overview of all memory-mapped RAS registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-254: RAS registers summary

Offset	Name	Reset	Width	Description
0x0	ERROFR	See individual bit resets.	64-bit	Error Record Feature Register
0x8	ERROCTL	See individual bit resets.	64-bit	Error Record Control Register
0x10	ERROSTATUS	See individual bit resets.	64-bit	Error Record Primary Status Register
0x18	ERROADDR	See individual bit resets.	64-bit	Error Record Address Register
0x20	ERROMISCO	See individual bit resets.	64-bit	Error Record Miscellaneous Register 0
0x28	ERROMISC1	See individual bit resets.	64-bit	Error Record Miscellaneous Register 1
0x30	ERROMISC2	See individual bit resets.	64-bit	Error Record Miscellaneous Register 2
0x38	ERROMISC3	See individual bit resets.	64-bit	Error Record Miscellaneous Register 3
0x800	ERROPFGF	See individual bit resets.	64-bit	Pseudo-fault Generation Feature Register
0x808	ERROPFGCTL	See individual bit resets.	64-bit	Pseudo-fault Generation Control Register
0x810	ERROPFGCDN	See individual bit resets.	64-bit	Pseudo-fault Generation Countdown Register
0xE00	ERRGSR	See individual bit resets.	64-bit	Error Group Status Register
0xE10	ERRIIDR	See individual bit resets.	32-bit	Implementation Identification Register
0xFA8	ERRDEVAFF	See individual bit resets.	64-bit	Device Affinity Register
0xFBC	ERRDEVARCH	See individual bit resets.	32-bit	Device Architecture Register
0xFC8	ERRDEVID	See individual bit resets.	32-bit	Device Configuration Register
0xFD0	ERRPIDR4	See individual bit resets.	32-bit	Peripheral Identification Register 4
0xFE0	ERRPIDR0	See individual bit resets.	32-bit	Peripheral Identification Register 0
0xFE4	ERRPIDR1	See individual bit resets.	32-bit	Peripheral Identification Register 1
0xFE8	ERRPIDR2	See individual bit resets.	32-bit	Peripheral Identification Register 2
0xFEC	ERRPIDR3	See individual bit resets.	32-bit	Peripheral Identification Register 3
0xFF0	ERRCIDR0	See individual bit resets.	32-bit	Component Identification Register 0
0xFF4	ERRCIDR1	See individual bit resets.	32-bit	Component Identification Register 1
0xFF8	ERRCIDR2	See individual bit resets.	32-bit	Component Identification Register 2
0xFFC	ERRCIDR3	See individual bit resets.	32-bit	Component Identification Register 3

B.7.1 ERROFR, Error Record Feature Register

Defines whether <n> is the first record owned by a node:

- If <n> is the first error record owned by a node, then ERR<n>FR.ED is not 0b00.
- If <n> is not the first error record owned by a node, then ERR<n>FR.ED is 0b00.

If <n> is the first record owned by the node, defines which of the common architecturally-defined features are implemented by the node and, of the implemented features, which are software programmable.

Configurations

This register is available in all configurations.

Attributes

Width

64

Component

RAS

Register offset

0x0

Access type

RO

Reset value

xxxx	xxxx	x101	0001	xxxx	xxxx	xxxx	xxxx	1xxx	xx00	0001	0000	1010	1001	1010	xx10
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-126: ext_err0fr bit assignments

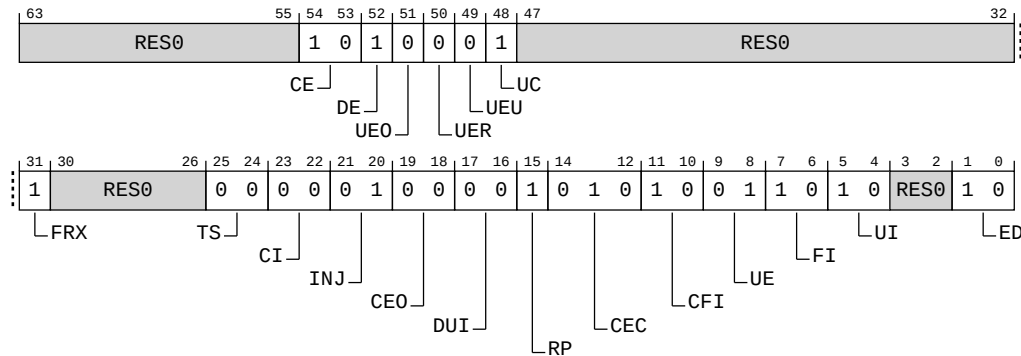


Table B-255: ERROFR bit descriptions

Bits	Name	Description	Reset
[63:55]	RES0	Reserved	RES0
[54:53]	CE	Corrected Error recording. Describes the types of Corrected errors the node can record, if any. 0b10 Records only non-specific Corrected errors. That is, Corrected errors recorded by setting ext-ERROSTATUS.CE to 0b10.	0b10
[52]	DE	Deferred Error recording. Describes whether the node supports recording Deferred errors. 0b1 Records Deferred errors.	0b1
[51]	UEO	Latent or Restartable Error recording. Describes whether the node supports recording Latent or Restartable errors. 0b0 Does not record Latent or Restartable errors.	0b0
[50]	UER	Signaled or Recoverable Error recording. Describes whether the node supports recording Signaled or Recoverable errors. 0b0 Does not record Signaled or Recoverable errors.	0b0
[49]	UEU	Unrecoverable Error recording. Describes whether the node supports recording Unrecoverable errors. 0b0 Does not record Unrecoverable errors.	0b0
[48]	UC	Uncontainable Error recording. Describes whether the node supports recording Uncontainable errors. 0b1 Records Uncontainable errors.	0b1
[47:32]	RES0	Reserved	RES0
[31]	FRX	Feature Register extension. Defines whether ext-ERROFR[63:48] are architecturally defined. 0b1 ext-ERROFR[63:48] are defined by the architecture.	0b1
[30:26]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[25:24]	TS	Timestamp Extension. Indicates whether, for each error record <m> owned by this node, ERR<m>MISC3 is used as the timestamp register, and, if it is, the timebase used by the timestamp. 0b00 Does not support a timestamp register. All other values are reserved.	0b00
[23:22]	CI	Critical error interrupt. Indicates whether the critical error interrupt and associated controls are implemented by the node. 0b00 Does not support the critical error interrupt. ext-ERROCTL.R.CI is RES0. All other values are reserved.	0b00
[21:20]	INJ	Fault Injection Extension. Indicates whether the Common Fault Injection Model Extension is implemented by the node. 0b01 Supports the Common Fault Injection Model Extension. See ext-ERROPFGF for more information. All other values are reserved.	0b01
[19:18]	CEO	Corrected Error overwrite. Indicates the behavior of the node when a second or subsequent Corrected error is recorded and a first Corrected error has previously been recorded by an error record <m> owned by the node. 0b00 Keeps the previous error syndrome. All other values are reserved. The second or subsequent Corrected error is counted by the Corrected error counter, regardless of the value of this field. If counting the error causes unsigned overflow of the counter, then ERR<m>STATUS.OF is set to 1. This means that, if no other error is subsequently recorded that overwrites the syndrome: • If ERROFR.CEO is 0b00, the error record holds the syndrome for the first recorded Corrected error. • If ERROFR.CEO is 0b01, the error record holds the syndrome for the most recently recorded Corrected error before the counter overflows.	0b00
[17:16]	DUI	Error recovery interrupt for deferred errors control. Indicates whether the enabling and disabling of error recovery interrupts on deferred errors is supported by the node. 0b00 Does not support the enabling and disabling of error recovery interrupts on deferred errors. ext-ERROCTL.R.DUI is RES0. All other values are reserved.	0b00
[15]	RP	Repeat counter. Indicates whether the node implements a second Corrected error counter in ERR<m>MISC0 for each error record <m> owned by the node that can record countable errors. 0b1 Implements a first (repeat) counter and a second (other) counter in ERR<m>MISC0 for each error record <m> owned by the node that can record countable errors. The repeat counter is the same size as the primary error counter.	0b1

Bits	Name	Description	Reset
[14:12]	CEC	Corrected Error Counter. Indicates whether the node implements the standard Corrected error counter mechanisms in ERR<m>MISC0 for each error record <m> owned by the node that can record countable errors. 0b010 Implements an 8-bit Corrected error counter in ERR<m>MISC0[39:32] for each error record <m> owned by the node that can record countable errors. All other values are reserved. Note: Implementations might include other error counter models, or might include the standard model and not indicate this in ERROFR.	0b010
[11:10]	CFI	Fault handling interrupt for corrected errors control. Indicates whether the enabling and disabling of fault handling interrupts on corrected errors is supported by the node. 0b10 Enabling and disabling of fault handling interrupts on corrected errors is supported and controllable using ext-ERROCTL.R.CFI. All other values are reserved.	0b10
[9:8]	UE	In-band error reponse (External Abort). Indicates whether the in-band error response and associated controls are implemented by the node. 0b01 In-band error response is supported and always enabled. ext-ERROCTL.R.UE is RES0.	0b01
[7:6]	FI	Fault handling interrupt. Indicates whether the fault handling interrupt and associated controls are implemented by the node. 0b10 Fault handling interrupt is supported and controllable using ext-ERROCTL.R.FI.	0b10
[5:4]	UI	Error recovery interrupt for uncorrected errors. Indicates whether the error handling interrupt and associated controls are implemented by the node. 0b10 Error handling interrupt is supported and controllable using ext-ERROCTL.R.UI.	0b10
[3:2]	RES0	Reserved	RES0
[1:0]	ED	Error reporting and logging. Indicates error record <n> is the first record owned the node, and whether the node implements the controls for enabling and disabling error reporting and logging. 0b10 Error reporting and logging is controllable using ext-ERROCTL.R.ED. All other values are reserved.	0b10

Accessibility

Component	Offset	Instance	Range
RAS	0x0	ERROFR	None

This interface is accessible as follows:

RO

B.7.2 ERROCTLR, Error Record Control Register

The error control register contains enable bits for the node that writes to this record:

- Enabling error detection and correction.
- Enabling the critical error, error recovery, and fault handling interrupts.
- Enabling in-band error response for uncorrected errors.

For each bit, if the node does not support the feature, then the bit is **RES0**. The definition of each record is IMPLEMENTATION DEFINED.

Configurations

ext-ERR<n>FR describes the features implemented by the node.

Attributes

Width

64

Component

RAS

Register offset

0x8

Access type

RW

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-127: ext_err0ctlr bit assignments

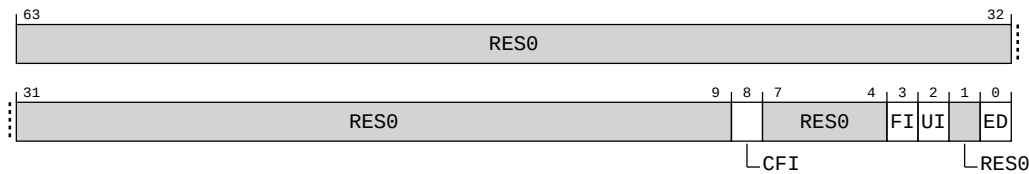


Table B-257: ERROCTLR bit descriptions

Bits	Name	Description	Reset
[63:9]	RES0	Reserved	RES0
[8]	CFI	Fault handling interrupt for Corrected errors enable. When ext-ERROFR.CFI == 0b10, this control applies to errors arising from both reads and writes. When enabled: - If the node implements Corrected error counters, then the fault handling interrupt is generated when a counter overflows and the overflow bit for the counter is set to 1. For more information, see ext-ERROMISCO. - Otherwise, the fault handling interrupt is also generated for all errors recorded as Corrected error. 0b0 Fault handling interrupt not generated for Corrected errors. 0b1 Fault handling interrupt generated for Corrected errors. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error.	x
[7:4]	RES0	Reserved	RES0
[3]	FI	Fault handling interrupt enable. When ext-ERROFR.FI == 0b10, this control applies to errors arising from both reads and writes. When enabled: - The fault handling interrupt is generated for all errors recorded as either Deferred error or Uncorrected error. - If the fault handling interrupt for Corrected errors control is not implemented: - If the node implements Corrected error counters, then the fault handling interrupt is also generated when a counter overflows and the overflow bit for the counter is set to 1. - Otherwise, the fault handling interrupt is also generated for all errors recorded as Corrected error. 0b0 Fault handling interrupt disabled. 0b1 Fault handling interrupt enabled. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error.	x

Bits	Name	Description	Reset
[2]	UI	Uncorrected error recovery interrupt enable. When ext-ERR0FR.UI == 0b10, this control applies to errors arising from both reads and writes. When enabled, the error recovery interrupt is generated for all errors recorded as Uncorrected error. 0b0 Error recovery interrupt disabled. 0b1 Error recovery interrupt enabled. The interrupt is generated even if the error syndrome is discarded because the error record already records a higher priority error.	x
[1]	RES0	Reserved	RES0
[0]	ED	Error reporting and logging enable. When disabled, the node behaves as if error detection and correction are disabled, and no errors are recorded or signaled by the node. 0b0 Error reporting disabled. 0b1 Error reporting enabled. This field is set to 0 on Cold reset	x

Accessibility

Component	Offset	Instance	Range
RAS	0x8	ERR0CTLR	None

This interface is accessible as follows:

RW

B.7.3 ERR0STATUS, Error Record Primary Status Register

Contains status information for error record <n>, including:

- Whether any error has been detected (valid).
- Whether any detected error was not corrected, and returned to a Requester.
- Whether any detected error was not corrected and deferred.
- Whether an error record has been discarded because additional errors have been detected before the first error was handled by software (overflow).
- Whether any error has been reported.
- Whether the other error record registers contain valid information.
- Whether the error was reported because poison data was detected or because a corrupt value was detected by an error detection code.

- A primary error code.
- An **IMPLEMENTATION DEFINED** extended error code.

Within this register:

- ERR<n>STATUS.{AV, V, MV} are valid bits that define whether error record <n> registers are valid.
- ERR<n>STATUS.{UE, OF, CE, DE, UET} encode the types of error or errors recorded.
- ERR<n>STATUS.{CI, ER, PN, IERR, SERR} are syndrome fields.

Configurations

ERROFR describes the features implemented by the node

Attributes

Width

64

Component

RAS

Register offset

0x10

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	00xx	x0xx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-128: ext_err0status bit assignments

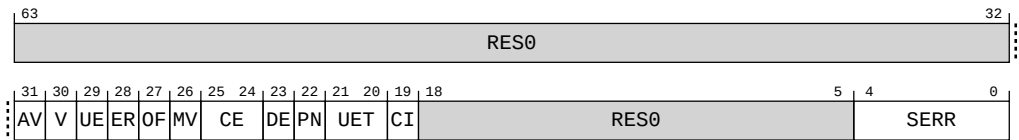


Table B-259: ERR0STATUS bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31]	AV	Address Valid. 0b0 ext-ERR<n>ADDR not valid. 0b1 ext-ERR<n>ADDR contains an address associated with the highest priority error recorded by this record. Access to this field is: W1C	0b0
[30]	V	Status Register Valid. 0b0 ERR<n>STATUS not valid. 0b1 ERR<n>STATUS valid. At least one error has been recorded. Access to this field is: W1C	0b0
[29]	UE	Uncorrected Error. 0b0 No errors have been detected, or all detected errors have been either corrected or deferred. 0b1 At least one detected error was not corrected and not deferred. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When ext-ERR0STATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x

Bits	Name	Description	Reset
[28]	ER	Error Reported. 0b0 No in-band error response (External Abort) signaled to the Requester making the access or other transaction. 0b1 An in-band error response was signaled by the component to the Requester making the access or other transaction. This can be because any of the following are true: - The applicable one of the ERR<q>CTLR.{WUE, RUE, UE} fields is implemented and was 1 when an error was detected and not corrected. - The applicable one of the ERR<q>CTLR.{WUE, RUE, UE} fields is not implemented and the component always reports errors. Note: An in-band error response signaled by the component might be masked and not generate any exception. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When ext-ERROSTATUS.UE == '0' && ext-ERROSTATUS.DE == '0' && this field can be set to 0b1 by a Deferred error Access to this field is: UNKNOWN/WI When ext-ERROSTATUS.UE == '0' && this field is never set to 0b1 by a Deferred error Access to this field is: UNKNOWN/WI When ext-ERROSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x

Bits	Name	Description	Reset
[27]	OF	Overflow. Indicates that multiple errors have been detected. This field is set to 1 when one of the following occurs: - A Corrected error counter is implemented, an error is counted, and the counter overflows. - ERR<n>STATUS.V was previously 1, a Corrected error counter is not implemented, and a Corrected error is recorded. - ERR<n>STATUS.V was previously 1, and a type of error other than a Corrected error is recorded. Otherwise, this field is unchanged when an error is recorded. If a Corrected error counter is implemented: - A direct write that modifies the counter overflow flag indirectly might set this field to an UNKNOWN value. - A direct write to this field that clears this field to zero might indirectly set the counter overflow flag to an UNKNOWN value. 0b0 Since this field was last cleared to zero, no error syndrome has been discarded and, if a Corrected error counter is implemented, it has not overflowed. 0b1 Since this field was last cleared to zero, at least one error syndrome has been discarded or, if a Corrected error counter is implemented, it might have overflowed. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When ext-ERROSTATUS.V == '0' Access to this field is: UNKNOWN/W1 Otherwise Access to this field is: W1C	x
[26]	MV	Miscellaneous Registers Valid. 0b0 ERR0MISC<m> not valid. 0b1 The contents of the ERR0MISC<m> registers contains additional information for an error recorded by this record. Access to this field is: W1C	0b0

Bits	Name	Description	Reset
[25:24]	CE	Corrected Error. 0b00 No errors were corrected. 0b10 At least one error was corrected. When clearing ERR0STATUS.V to 0, if this field is nonzero, then Arm recommends that software write ones to this field to clear this field to zero. When ext-ERR0STATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	xx
[23]	DE	Deferred Error. 0b0 No errors were deferred. 0b1 At least one error was not corrected and deferred. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When ext-ERR0STATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x
[22]	PN	Poison. 0b0 Uncorrected error or Deferred error recorded because a corrupt value was detected, for example, by an error detection code (EDC), or Corrected error recorded. 0b1 Uncorrected error or Deferred error recorded because a poison value was detected. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When ext-ERR0STATUS.V == '0' (ext-ERR0STATUS.DE == '0' && ext-ERR0STATUS.UE == '0') Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x

Bits	Name	Description	Reset
[21:20]	UET	Uncorrected Error Type. Describes the state of the component after detecting or consuming an Uncorrected error. 0b00 Uncorrected error, Uncontainable error (UC). 0b01 Uncorrected error, Unrecoverable error (UEU). 0b10 Uncorrected error, Latent or Restartable error (UEO). 0b11 Uncorrected error, Signaled or Recoverable error (UER). Note: Software might use the information in the error record registers to determine what recovery is necessary. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write ones to this field to clear this field to zero. When ext-ERROSTATUS.V == '0' ext-ERROSTATUS.UE == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	xx
[19]	CI	Critical Error. Indicates whether a critical error condition has been recorded. 0b0 No critical error condition. 0b1 Critical error condition. When clearing ERR<n>STATUS.V to 0, if this field is nonzero, then Arm recommends that software write 1 to this field to clear this field to zero. When ext-ERROSTATUS.V == '0' Access to this field is: UNKNOWN/WI Otherwise Access to this field is: W1C	x
[18:5]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[4:0]	SERR	Architecturally-defined primary error code. The primary error code might be used by a fault handling agent to triage an error without requiring device-specific code. For example, to count and threshold corrected errors in software, or generate a short log entry. 0b00000 No error. 0b00010 Data value from (non-associative) internal memory. For example, ECC from on-chip SRAM or buffer. 0b00110 Data value from associative memory. For example, ECC error on cache data. 0b00111 Address/control value from associative memory. For example, ECC error on cache tag. 0b01000 Data value from a TLB. For example, ECC error on TLB data. 0b01001 Address/control value from a TLB. For example, ECC error on TLB tag 0b10010 Error response from Completer of access. For example, error response from cache write-back. 0b11000 Deferred error from Requester passed through. For example, poisoned data received from the Requester of an access and deferred to the Completer. All other values are reserved. This field is not valid and reads UNKNOWN if ERROSTATUS.V == 0b0.	5{x}

Access

ERR<n>STATUS.{AV, V, UE, ER, OF, MV, CE, DE, PN, UET, CI} are write-one-to-clear (W1C) fields, meaning writes of zero are ignored, and a write of one or all-ones to the field clears the field to zero. ERR<n>STATUS.{IERR, SERR} are read/write (RW) fields, although the set of implemented valid values is **IMPLEMENTATION DEFINED**. See also ext-ERR<n>PFGF.SYN.

After reading ERR<n>STATUS, software must clear the valid fields in the register to allow new errors to be recorded. However, between reading the register and clearing the valid fields, a new error might have overwritten the register. To prevent this error being lost by software, the register prevents updates to fields that might have been updated by a new error.

When RAS System Architecture v1.0 is implemented:

- Writes to ERR<n>STATUS.{UE, DE, CE} are ignored if ERR<n>STATUS.OF is 1 and is not being cleared to 0.
- Writes to ERR<n>STATUS.V are ignored if any of ERR<n>STATUS.{UE, DE, CE} are nonzero and are not being cleared to zero.
- Writes to ERR<n>STATUS.{AV, MV} and the ERR<n>STATUS.{ER, PN, UET, IERR, SERR} syndrome fields are ignored if the highest priority nonzero error status field is not being cleared to zero. The error status fields in priority order from highest to lowest, are ERR<n>STATUS.UE, ERR<n>STATUS.DE, and ERR<n>STATUS.CE.

When RAS System Architecture v1.1 is implemented, a write to the register is ignored if all of:

- Any of ERR<n>STATUS.{V, UE, OF, CE, DE} are nonzero before the write.
- The write does not clear the nonzero ERR<n>STATUS.{V, UE, OF, CE, DE} fields to zero by writing ones to the applicable field or fields.

Some of the fields in ERR<n>STATUS are also defined as **UNKNOWN** where certain combinations of ERR<n>STATUS.{V, DE, UE} are zero. The rules for writes to ERR<n>STATUS allow a node to implement such a field as a fixed read-only value.

For example, when RAS System Architecture v1.1 is implemented, a write to ERR<n>STATUS when ERR<n>STATUS.V is 1 results in either ERR<n>STATUS.V field being cleared to zero, or ERR<n>STATUS.V not changing. Since all fields in ERR<n>STATUS, other than ERR<n>STATUS.{AV, V, MV}, usually read as **UNKNOWN** values when ERR<n>STATUS.V is zero, this means those fields can be implemented as read-only if applicable.

To ensure correct and portable operation, when software is clearing the valid fields in the register to allow new errors to be recorded, Arm recommends that software:

- Read ERR<n>STATUS and determine which fields need to be cleared to zero.
- Write ones to all the W1C fields that are nonzero in the read value.
- Write zero to all the W1C fields that are zero in the read value.
- Write zero to all the RW fields.

Otherwise, these fields might not have the correct value when a new fault is recorded.

An exception is when the node supports writing to these fields as part of fault injection. See also ext-ERR<n>PFGF.SYN.

Accessibility

ERR<n>STATUS.{AV, V, UE, ER, OF, MV, CE, DE, PN, UET, CI} are write-one-to-clear (W1C) fields, meaning writes of zero are ignored, and a write of one or all-ones to the field clears the field to zero. ERR<n>STATUS.{IERR, SERR} are read/write (RW) fields, although the set of implemented valid values is IMPLEMENTATION DEFINED. See also ext-ERR<n>PFGF.SYN.

After reading ERR<n>STATUS, software must clear the valid fields in the register to allow new errors to be recorded. However, between reading the register and clearing the valid fields, a new error might have overwritten the register. To prevent this error being lost by software, the register prevents updates to fields that might have been updated by a new error.

When RAS System Architecture v1.0 is implemented:

- Writes to ERR<n>STATUS.{UE, DE, CE} are ignored if ERR<n>STATUS.OF is 1 and is not being cleared to 0.
- Writes to ERR<n>STATUS.V are ignored if any of ERR<n>STATUS.{UE, DE, CE} are nonzero and are not being cleared to zero.
- Writes to ERR<n>STATUS.{AV, MV} and the ERR<n>STATUS.{ER, PN, UET, IERR, SERR} syndrome fields are ignored if the highest priority nonzero error status field is not being cleared

to zero. The error status fields in priority order from highest to lowest, are ERR<n>STATUS.UE, ERR<n>STATUS.DE, and ERR<n>STATUS.CE.

When RAS System Architecture v1.1 is implemented, a write to the register is ignored if all of:

- Any of ERR<n>STATUS.{V, UE, OF, CE, DE} are nonzero before the write.
- The write does not clear the nonzero ERR<n>STATUS.{V, UE, OF, CE, DE} fields to zero by writing ones to the applicable field or fields.

Some of the fields in ERR<n>STATUS are also defined as UNKNOWN where certain combinations of ERR<n>STATUS.{V, DE, UE} are zero. The rules for writes to ERR<n>STATUS allow a node to implement such a field as a fixed read-only value.

For example, when RAS System Architecture v1.1 is implemented, a write to ERR<n>STATUS when ERR<n>STATUS.V is 1 results in either ERR<n>STATUS.V field being cleared to zero, or ERR<n>STATUS.V not changing. Since all fields in ERR<n>STATUS, other than ERR<n>STATUS.{AV, V, MV}, usually read as UNKNOWN values when ERR<n>STATUS.V is zero, this means those fields can be implemented as read-only if applicable.

To ensure correct and portable operation, when software is clearing the valid fields in the register to allow new errors to be recorded, Arm recommends that software:

- Read ERR<n>STATUS and determine which fields need to be cleared to zero.
- Write ones to all the W1C fields that are nonzero in the read value.
- Write zero to all the W1C fields that are zero in the read value.
- Write zero to all the RW fields.

Otherwise, these fields might not have the correct value when a new fault is recorded.

An exception is when the node supports writing to these fields as part of fault injection. See also ext-ERR<n>PFGF.SYN.

Component	Offset	Instance	Range
RAS	0x10	ERR0STATUS	None

This interface is accessible as follows:

- When ext-ERR0STATUS.V != '0' && ERR<n>STATUS.V is not being cleared to 0b0 in the same write**
RO
- When ext-ERR0STATUS.UE != '0' && ERR<n>STATUS.UE is not being cleared to 0b0 in the same write**
RO
- When ext-ERR0STATUS.OF != '0' && ERR<n>STATUS.OF is not being cleared to 0b0 in the same write**
RO

When `ext-ERR0STATUS.CE != '00' && ERR<n>STATUS.CE` is not being cleared to `0b00` in the same write

RO

When `ext-ERR0STATUS.DE != '0' && ERR<n>STATUS.DE` is not being cleared to `0b0` in the same write

RO

Otherwise

RW

B.7.4 ERR0ADDR, Error Record Address Register

If an address is associated with a detected error, then it is written to `ERR0ADDR` when the error is recorded

Configurations

`ERR<q>FR` describes the features implemented by the node that owns error record `<n>`. `<q>` is the index of the first error record owned by the same node as error record `<n>`. If the node owns a single record, then `q = n`.

Attributes

Width

64

Component

RAS

Register offset

0x18

Access type

See bit descriptions

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Where the reset reads `xxxx`, see individual bits.

Bit descriptions

Figure B-129: ext_err0addr bit assignments

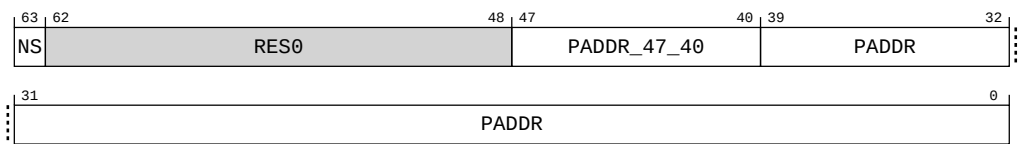


Table B-261: ERR0ADDR bit descriptions

Bits	Name	Description	Reset
[63]	NS	Non-secure attribute. 0b0 The address is Secure. 0b1 The address is Non-secure.	x
[62:48]	RES0	Reserved	RES0
[47:40]	PADDR_47_40	Physical Address. Address of the recorded location.	8 {x}
[39:0]	PADDR	Physical Address. Address of the recorded location.	40 {x}

Accessibility

Component	Offset	Instance	Range
RAS	0x18	ERR0ADDR	None

This interface is accessible as follows:

When ext-ERR<q>PFGF.AV == '0' && ext-ERR0STATUS.AV == '1'
RO

Otherwise
RW

B.7.5 ERR0MISC0, Error Record Miscellaneous Register 0

IMPLEMENTATION DEFINED error syndrome register. The miscellaneous syndrome registers might contain:

- Information to locate where the error was detected.
- If the error was detected within a FRU, the identity of the FRU.
- A Corrected error counter or counters.
- Other state information not present in the corresponding status and address registers.

If the node that owns error record <n> implements architecturally-defined Corrected error counters (ERR<q>FR.CEC != 0b000), and error record <n> can record countable errors, then ERR<n>MISCO implements the architecturally-defined Corrected error counter or counters.

Configurations

ERROFR describes the features implemented by the node

Attributes

Width

64

Component

RAS

Register offset

0x20

Access type

Read

R

Write

W

Reset value

xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3
															0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-130: ext_err0misc0 bit assignments

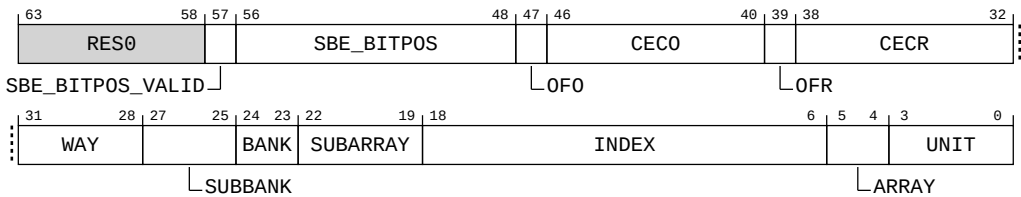


Table B-263: ERRORMISC0 bit descriptions

Bits	Name	Description	Reset
[63:58]	RES0	Reserved	RES0
[57]	SBE_BITPOS_VALID	Bit position of a corrected error from a RAM protected by ECC Valid	x
[56:48]	SBE_BITPOS	Bit position of a corrected error from a RAM protected by ECC	9 {x}
[47]	OFO	Sticky overflow bit, other. Set to 1 when ERRORMISC0.CECO is incremented and wraps through zero. 0b0 Other counter has not overflowed. 0b1 Other counter has overflowed. A direct write that modifies this bit might indirectly set ERR0STATUS.OF to an UNKNOWN value and a direct write to ERR0STATUS.OF that clears it to zero might indirectly set this bit to an UNKNOWN value. Unaffected by Warm reset.	0b0
[46:40]	CECO	Corrected error count, other. Incremented for each countable error that is not accounted for by incrementing ERRORMISC0.CECR. Unaffected by Warm reset.	0b0000000
[39]	OFR	Sticky overflow bit, repeat. Set to 1 when ERRORMISC0.CECR is incremented and wraps through zero. 0b0 Repeat counter has not overflowed. 0b1 Repeat counter has overflowed. A direct write that modifies this bit might indirectly set ERR0STATUS.OF to an UNKNOWN value and a direct write to ERR0STATUS.OF that clears it to zero might indirectly set this bit to an UNKNOWN value. Unaffected by Warm reset.	0b0
[38:32]	CECR	Corrected error count, repeat. Incremented for the first countable error, which also records other syndrome for the error, and subsequently for each countable error that matches the recorded other syndrome.	0b0000000

Bits	Name	Description	Reset
[31:28]	WAY	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L1 Data Cache] - Indicates which Tag RAM way or data RAM way detected the error. Upper 2 bits are unused. [L2 TLB] - Indicates which RAM detected an error. The possible values are 0 (RAM 1) to 9 (RAM 10). [L1 Instruction Cache] - Indicates which way detected the error. [L2 Tag Cache] - Indicates which way detected the error. Upper 1 bit unused. [L2 Data Cache] - Indicates which way detected the error. Unaffected by Warm reset.	0b0000
[27:25]	SUBBANK	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L1 Instruction Cache] Indicates which subbank detected the error, valid for Instruction Data Cache. For Tag errors this field is zero. Unaffected by Warm reset.	0b000
[24:23]	BANK	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Cache] - Indicates which L2 bank detected the error. Upper 1 bit is unused. [L1 Instruction Cache] - Indicates which bank detected the error, valid for Instruction Cache. Upper 1 bit is unused Unaffected by Warm reset.	0b00

Bits	Name	Description	Reset
[22:19]	SUBARRAY	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Cache] - Indicates which L2 data doubleword detected the error. Upper 1 bit is unused. [L1 Data Cache] - Indicates for L1 Data RAM which word had the error detected. For L1 Tag RAMs which bank had the error (0b0000: bank0 , 0b0001: bank1) [L2 TLB] - Indicates for L2 TLB RAM which word had the error detected. Upper 3 bits are unused. Unaffected by Warm reset.	0b0000
[18:6]	INDEX	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Tag Cache] - Indicates which index detected the error. Bits[n:6] are the index, n varies with the cache size. [L2 Data Cache] - Indicates which index detected the error. Bits[n:6] are the index, n varies with the cache size. [L1 Data Cache] - Indicates which index detected the error. Upper bits of the index are unused depending on the cache size [L2 TLB] - Index of TLB RAM. Upper 4 bits are unused. [L1 Instruction Cache] - Indicates which index detected the error. Upper bits of the index are unused depending on the cache size. Unaffected by Warm reset.	0b00000000000000

Bits	Name	Description	Reset
[5:4]	ARRAY	The encoding is dependent on the unit from which the error being recorded was detected. The possible values are: [L2 Cache] Indicates which array detected the error. The possible values are: 0b00 L2 Tag RAM. 0b01 L2 Data RAM. 0b10 Transaction Queue RAM. [L1 Data Cache] Indicates which array detected the error. The possible values are: 00 LS Tag RAM 0. 01 LS Tag RAM 1. 10 LS Data RAM. 11 LS Tag RAM 2. [L1 Instruction Cache] Indicates which array that detected the error, Data Array has higher priority. The possible values are: 0b00 Tag. 0b01 Data. Unaffected by Warm reset.	0b00
[3:0]	UNIT	Indicates the unit which detected the error. The possible values are: 0b0001 L1 Instruction Cache. 0b0010 L2 TLB. 0b0100 L1 Data Cache. 0b1000 L2 Cache.	0b0000

Access

Reads from ERR<n>MISCO return an **IMPLEMENTATION DEFINED** value and writes have **IMPLEMENTATION DEFINED** behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



Note

These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Accessibility

Reads from ERR<n>MISC0 return an IMPLEMENTATION DEFINED value and writes have IMPLEMENTATION DEFINED behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



Note

These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Component	Offset	Instance	Range
RAS	0x20	ERR0MISC0	None

This interface is accessible as follows:

RW

B.7.6 ERR0MISC1, Error Record Miscellaneous Register 1

IMPLEMENTATION DEFINED error syndrome register. The miscellaneous syndrome registers might contain:

- Information to locate where the error was detected.
- If the error was detected within a FRU, the identity of the FRU.
- A Corrected error counter or counters.

- Other state information not present in the corresponding status and address registers.

Configurations

ERROFR describes the features implemented by the node

Attributes

Width

64

Component

RAS

Register offset

0x28

Access type

Read

R

Write

W

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-131: ext_err0misc1 bit assignments

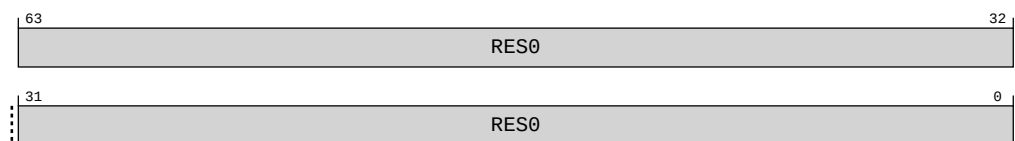


Table B-265: ERR0MISC1 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

Reads from ERR<n>MISC1 return an **IMPLEMENTATION DEFINED** value and writes have **IMPLEMENTATION DEFINED** behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Accessibility

Reads from ERR<n>MISC1 return an **IMPLEMENTATION DEFINED** value and writes have **IMPLEMENTATION DEFINED** behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Component	Offset	Instance	Range
RAS	0x28	ERR0MISC1	None

This interface is accessible as follows:

RW

B.7.7 ERRORMISC2, Error Record Miscellaneous Register 2

IMPLEMENTATION DEFINED error syndrome register. The miscellaneous syndrome registers might contain:

- Information to locate where the error was detected.
- If the error was detected within a FRU, the identity of the FRU.
- A Corrected error counter or counters.
- Other state information not present in the corresponding status and address registers.

Configurations

ERRROFR describes the features implemented by the node

Attributes

Width

64

Component

RAS

Register offset

0x30

Access type

Read

R

Write

W

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-132: ext_err0misc2 bit assignments

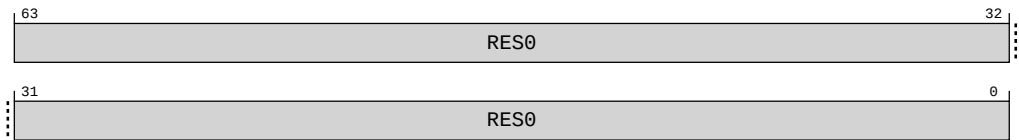


Table B-267: ERRORMISC2 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

Reads from ERR<n>MISC2 return an **IMPLEMENTATION DEFINED** value and writes have **IMPLEMENTATION DEFINED** behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Accessibility

Reads from ERR<n>MISC2 return an **IMPLEMENTATION DEFINED** value and writes have **IMPLEMENTATION DEFINED** behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.

- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Component	Offset	Instance	Range
RAS	0x30	ERR0MISC2	None

This interface is accessible as follows:

RW

B.7.8 ERR0MISC3, Error Record Miscellaneous Register 3

IMPLEMENTATION DEFINED error syndrome register. The miscellaneous syndrome registers might contain:

- Information to locate where the error was detected.
- If the error was detected within a FRU, the identity of the FRU.
- A Corrected error counter or counters.
- Other state information not present in the corresponding status and address registers.

If the node that owns error record n supports the RAS Timestamp Extension (ERR<q>FR.TS != 0b00), then ERR<n>MISC3 contains the timestamp value for error record n when the error was detected. Otherwise the contents of ERR<n>MISC3 are **IMPLEMENTATION DEFINED**.

Configurations

ERR0FR describes the features implemented by the node

Attributes

Width

64

Component

RAS

Register offset

0x38

Access type

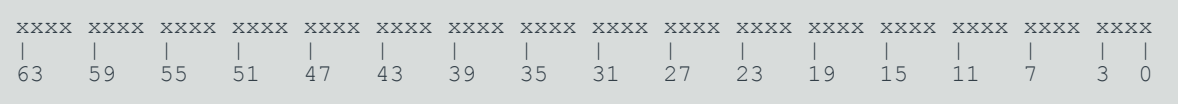
Read

R

Write

W

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-133: ext_err0misc3 bit assignments

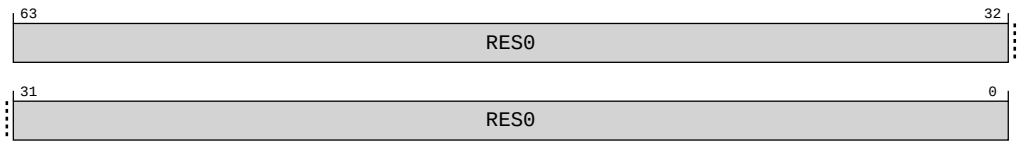


Table B-269: ERR0MISC3 bit descriptions

Bits	Name	Description	Reset
[63:0]	RES0	Reserved	RES0

Access

Reads from ERR<n>MISC3 return an **IMPLEMENTATION DEFINED** value and writes have **IMPLEMENTATION DEFINED** behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Accessibility

Reads from ERR<n>MISC3 return an IMPLEMENTATION DEFINED value and writes have IMPLEMENTATION DEFINED behavior.

If the Common Fault Injection Mechanism is implemented by the node that owns this error record, and ERR<q>PFGF.MV is 1, then some parts of this register are read/write when ext-ERR<n>STATUS.MV is 1. See ext-ERR<n>PFGF.MV for more information.

For other parts of this register, or if the Common Fault Injection Mechanism is not implemented, then Arm recommends that:

- Miscellaneous syndrome for multiple errors, such as a corrected error counter, is read/write.
- When ext-ERR<n>STATUS.MV is 1, the miscellaneous syndrome specific to the most recently recorded error ignores writes.



These recommendations allow a counter to be reset in the presence of a persistent error, while preventing specific information, such as that identifying a FRU, from being lost if an error is detected while the previous error is being logged.

Component	Offset	Instance	Range
RAS	0x38	ERR0MISC3	None

This interface is accessible as follows:

RW

B.7.9 ERR0PFGF, Pseudo-fault Generation Feature Register

Defines which common architecturally-defined fault generation features are implemented.

Configurations

ext-ERR<n>FR describes the features implemented by the node.

Attributes

Width

64

Component

RAS

Register offset

0x800

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-134: ext_err0pfgf bit assignments

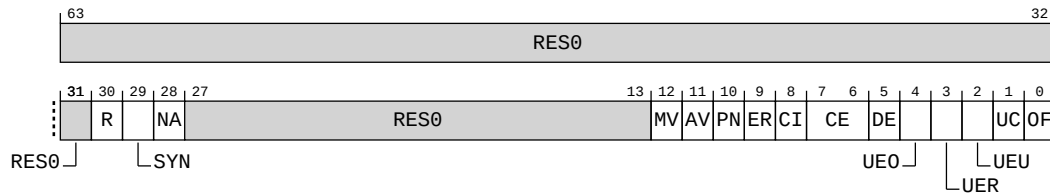


Table B-271: ERR0PFGF bit descriptions

Bits	Name	Description	Reset
[63:31]	RES0	Reserved	RES0
[30]	R	Restartable. Support for Error Generation Counter restart mode. 0b1 Error Generation Counter restart mode is implemented and is controlled by ext-ERR<n>PFGCTL.R. ext-ERR<n>PFGCTL.R is a read/write field.	x
[29]	SYN	Syndrome. Fault syndrome injection. 0b1 When an injected error is recorded, the node does not update the ext-ERROSTATUS.{IERR, SERR} fields. ext-ERROSTATUS.{IERR, SERR} are writable when ext-ERROSTATUS.V is 0	x
[28]	NA	No access required. Defines whether this component fakes detection of the error on an access to the component or spontaneously in the fault injection state. 0b1 The component fakes detection of the error spontaneously in the fault injection state.	x
[27:13]	RES0	Reserved	RES0
[12]	MV	Miscellaneous syndrome. Additional syndrome injection. Defines whether software can control all or part of the syndrome recorded in the ERR<n>MISC<m> registers when an injected error is recorded. 0b1 When an injected error is recorded, the node does not update ext-ERR0MISCO and ext-ERROSTATUS.MV is set to ext-ERR0PFGCTL.MV. ext-ERR0PFGCTL.MV is a read/write field.	x

Bits	Name	Description	Reset
[11]	AV	Address syndrome. Address syndrome injection. 0b1 When an injected error is recorded, the node does not update ext-ERR0ADDR and ext-ERR0STATUS.AV is set to ext-ERR0PFGCTL.AV. ext-ERR0PFGCTL.AV is a read/write field.	x
[10]	PN	Poison flag. Describes how the fault generation feature of the node sets the ext-ERR<n>STATUS.PN status flag. 0b0 When an injected error is recorded the node sets ext-ERR0STATUS.PN to 0. ext-ERR0PFGCTL.PN is RES0 .	x
[9]	ER	Error Reported flag. Describes how the fault generation feature of the node sets the ext-ERR<n>STATUS.ER status flag. 0b1 When an injected error is recorded, ext-ERR0STATUS.ER is set to ext-ERR0PFGCTL.ER. This behavior replaces the architecture-defined rules for setting the ER bit. ext-ERR0PFGCTL.ER is a read/write field.	x
[8]	CI	Critical Error flag. Describes how the fault generation feature of the node sets the ext-ERR<n>STATUS.CI status flag. 0b0 When an injected error is recorded, the node sets ext-ERR0STATUS.CI to 0. ext-ERR0PFGCTL.CI is RES0 .	x
[7:6]	CE	Corrected Error generation. Describes the types of Corrected error that the fault generation feature of the node can generate. 0b01 The fault generation feature of the node allows generation of a non-specific Corrected error, that is, a Corrected error that is recorded by setting ext-ERR<n>STATUS.CE to 0b10. ext-ERR<n>PFGCTL.CE is a read/write field. The values 0b10 and 0b11 in ext-ERR<n>PFGCTL.CE are reserved.	xx
[5]	DE	Deferred Error generation. Describes whether the fault generation feature of the node can generate Deferred errors. 0b1 The fault generation feature of the node allows generation of Deferred errors. ext-ERR<n>PFGCTL.DE is a read/write field.	x
[4]	UEO	Latent or Restartable Error generation. Describes whether the fault generation feature of the node can generate Latent or Restartable errors. 0b0 The fault generation feature of the node does not generate Latent or Restartable errors. ext-ERR<n>PFGCTL.UEO is RES0 .	x
[3]	UER	Signaled or Recoverable Error generation. Describes whether the fault generation feature of the node can generate Signaled or Recoverable errors. 0b0 The fault generation feature of the node does not generate Signaled or Recoverable errors. ext-ERR<n>PFGCTL.UER is RES0 .	x
[2]	UEU	Unrecoverable Error generation. Describes whether the fault generation feature of the node can generate Unrecoverable errors. 0b0 The fault generation feature of the node does not generate Unrecoverable errors. ext-ERR<n>PFGCTL.UEU is RES0 .	x

Bits	Name	Description	Reset
[1]	UC	Uncontainable Error generation. Describes whether the fault generation feature of the node can generate Uncontainable errors. 0b1 The fault generation feature of the node allows generation of Uncontainable errors. ext-ERR<n>PFGCTL.UC is a read/write field.	x
[0]	OF	Overflow flag. Describes how the fault generation feature of the node sets the ext-ERR<n>STATUS.OF status flag. 0b1 When an injected error is recorded, ext-ERR<n>STATUS.OF is set to ext-ERR<n>PFGCTL.OF. This behavior replaces the architecture-defined rules for setting the OF bit. ext-ERR<n>PFGCTL.OF is a read/write field.	x

Accessibility

Component	Offset	Instance	Range
RAS	0x800	ERR0PFGF	None

This interface is accessible as follows:

RO

B.7.10 ERR0PFGCTL, Pseudo-fault Generation Control Register

Enables controlled fault generation.

Configurations

ext-ERR<n>FR describes the features implemented by the node.

Attributes

Width

64

Component

RAS

Register offset

0x808

Access type

RW

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0xxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3	0

**Note**

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-135: ext_err0pfgctl bit assignments

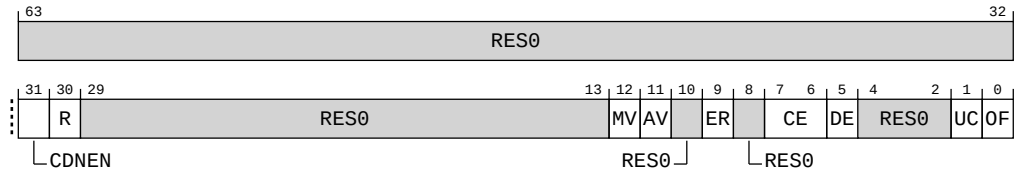


Table B-273: ERR0PFGCTL bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31]	CDNEN	Countdown Enable. Controls transfers of the value held in ext-ERR<n>PFGCDN to the Error Generation Counter and enables this counter. 0b0 The Error Generation Counter is disabled. 0b1 The Error Generation Counter is enabled. On a write of 1 to this field, the Error Generation Counter is set to ext-ERR<n>PFGCDN.CDN.	0b0
[30]	R	Restart. Controls whether the Error Generation Counter restarts or stops counting on reaching zero. 0b0 On reaching zero, the Error Generation Counter will stop counting. 0b1 On reaching zero, the Error Generation Counter is set to ext-ERR<n>PFGCDN.CDN.	x
[29:13]	RES0	Reserved	RES0
[12]	MV	Miscellaneous syndrome. The value written to ext-ERR<n>STATUS.MV when an injected error is recorded. 0b0 ext-ERR<n>STATUS.MV is set to 0 when an injected error is recorded. 0b1 ext-ERR<n>STATUS.MV is set to 1 when an injected error is recorded.	x
[11]	AV	Address syndrome. The value written to ext-ERR<n>STATUS.AV when an injected error is recorded. 0b0 ext-ERR<n>STATUS.AV is set to 0 when an injected error is recorded. 0b1 ext-ERR<n>STATUS.AV is set to 1 when an injected error is recorded.	x
[10]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[9]	ER	Error Reported flag. The value written to ext-ERR<n>STATUS.ER when an injected error is recorded. 0b0 ext-ERR<n>STATUS.ER is set to 0 when an injected error is recorded. 0b1 ext-ERR<n>STATUS.ER is set to 1 when an injected error is recorded.	x
[8]	RES0	Reserved	RES0
[7:6]	CE	Corrected Error generation enable. Controls the type of injected Corrected error generated by the fault injection feature of the node. 0b00 An injected Corrected error will not be generated by the fault injection feature of the node. 0b01 An injected non-specific Corrected error is generated in the fault injection state. ext-ERR<n>STATUS.CE is set to 0b10 when the injected error is recorded. 0b10 An injected transient Corrected error is generated in the fault injection state. ext-ERR<n>STATUS.CE is set to 0b01 when the injected error is recorded. 0b11 An injected persistent Corrected error is generated in the fault injection state. ext-ERR<n>STATUS.CE is set to 0b11 when the injected error is recorded. The set of permitted values for this field is defined by ext-ERR0PFGF.CE. The node enters the fault injection state when the Error Generation Counter decrements to zero.	xx
[5]	DE	Deferred Error generation enable. Controls whether an injected Deferred error is generated by the fault injection feature of the node. 0b0 An injected Deferred error will not be generated by the fault generation feature of the node. 0b1 An injected Deferred error is generated in the fault injection state. The node enters the fault injection state when the Error Generation Counter decrements to zero	x
[4:2]	RES0	Reserved	RES0
[1]	UC	Uncontainable Error generation enable. Controls whether an injected Uncontainable error is generated by the fault injection feature of the node. 0b0 An injected Uncontainable error will not be generated by the fault generation feature of the node. 0b1 An injected Uncontainable error is generated in the fault injection state. The node enters the fault injection state when the Error Generation Counter decrements to zero	x
[0]	OF	Overflow flag. The value written to ext-ERR<n>STATUS.OF when an injected error is recorded. 0b0 ext-ERR<n>STATUS.OF is set to 0 when an injected error is recorded. 0b1 ext-ERR<n>STATUS.OF is set to 1 when an injected error is recorded.	x

Accessibility

Component	Offset	Instance	Range
RAS	0x808	ERR0PFGCTL	None

This interface is accessible as follows:

RW

B.7.11 ERR0PFGCDN, Pseudo-fault Generation Countdown Register

Generates one of the errors enabled in the corresponding ext-ERR<n>PFGCTL register.

Configurations

ext-ERR<n>FR describes the features implemented by the node.

Attributes

Width

64

Component

RAS

Register offset

0x810

Access type

RW

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-136: ext_err0pfgcdn bit assignments

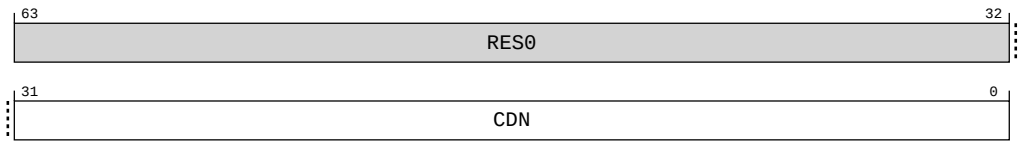


Table B-275: ERR0PFGCDN bit descriptions

Bits	Name	Description	Reset
[63:32]	RES0	Reserved	RES0
[31:0]	CDN	Countdown value. This field is copied to Error Generation Counter when either Software writes ERR0PFGCTL.CDNEN with 1 or The Error Generation Counter decrements to zero and ERR0PFGCTL.R == 1. Note: The current Error Generation Counter value is not visible to software.	32 {x}

Accessibility

Component	Offset	Instance	Range
RAS	0x810	ERR0PFGCDN	None

This interface is accessible as follows:

RW

B.7.12 ERRGSR, Error Group Status Register

Shows the status for the records in the group.

Configurations

ERRGSR is implemented only as part of a memory-mapped group of error records.

This manual describes a group of error records accessed via a standard 4KB memory-mapped peripheral. For a 4KB peripheral, up to 24 error records can be accessed if the Common Fault Injection Model is implemented, and up to 56 otherwise.

Attributes

Width

64

Component

RAS

Register offset

0xE00

Access type

RO

Reset value

xxxx	xxxx	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	000x
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-137: ext_errgsr bit assignments

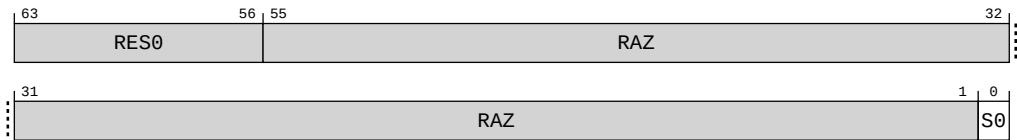


Table B-277: ERRGSR bit descriptions

Bits	Name	Description	Reset
[63:56]	RES0	Reserved	RES0
[55:1]	RAZ	Reserved	RAZ
[0]	S0	The status for error record <m>. A read-only copy of ERR<m>STATUS.V. 0b0 No error. 0b1 One or more errors.	x

Accessibility

Component	Offset	Instance	Range
RAS	0xE00	ERRGSR	None

This interface is accessible as follows:

RO

B.7.13 ERRIDR, Implementation Identification Register

Defines the implementer of the component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

RAS

Register offset

0xE10

Access type

RO

Reset value

1101 1000 1110 0000 0001 0100 0011 1011

Bit descriptions

Figure B-138: ext_erridr bit assignments

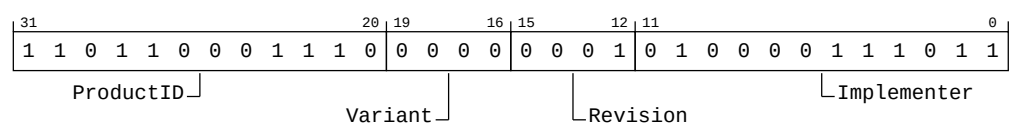


Table B-279: ERRIDR bit descriptions

Bits	Name	Description	Reset
[31:20]	ProductID	Part number, bits [11:0]. The part number is selected by the designer of the component. 0b110110001110 Neoverse N3	0xD8E
[19:16]	Variant	Component major revision. This field distinguishes product variants or major revisions of the product. 0b0000 rOp1	0b0000
[15:12]	Revision	Component minor revision. This field distinguishes minor revisions of the product. 0b0001 rOp1	0b0001
[11:0]	Implementer	Contains the JEP106 code of the company that implemented the RAS component. For an Arm implementation, this field has the value 0x43B. 0b010000111011 Arm Limited	0x43B

Accessibility

This interface is accessible as follows:

RO

B.7.14 ERRDEVAFF, Device Affinity Register

For a group of error records that has affinity with a single PE or a group of PEs, ERRDEVAFF is a copy of AArch64-MPIDR_EL1 or part of AArch64-MPIDR_EL1:

- If the group of error records has affinity with a single PE, the affinity level is 0, ERRDEVAFF reads the same value as AArch64-MPIDR_EL1, and ERRDEVAFF.FOV reads-as-one to indicate affinity level 0.
- If the group of error records has affinity with a group of PEs, the affinity level is 1, 2, or 3, parts of ERRDEVAFF reads the same value as parts of AArch64-MPIDR_EL1, and the rest of ERRDEVAFF indicates the level.

For example, if the group of PEs is a subset of the PEs at affinity level 1 then all of the following are true:

- All the PEs in the group have the same values in AArch64-MPIDR_EL1.{Aff3,Aff2}, and these values are equal to ERRDEVAFF.{Aff3,Aff2}.
- ERRDEVAFF.Aff1 is nonzero and not 0x80, and ERRDEVAFF.{Aff0,FOV} read-as-zero, to indicate at least affinity level 1. The subset of PEs at level 1 that the group of error records has affinity with is indicated by the least-significant set bit in ERRDEVAFF.Aff1. In this example, if ERRDEVAFF.Aff1[2:0] is 0b100, then the group of error records has affinity with the up-to 8 PEs that have AArch64-MPIDR_EL1.Aff1[7:3] == ERRDEVAFF.Aff1[7:3].

If RAS System Architecture v1.1 is not implemented, ERRDEVAFF can only describe a group of error records that is affine with a single PE or all the PEs at an affinity level.

Configurations

ERRDEVAFF is implemented only as part of a memory-mapped group of error records.

Attributes

Width

64

Component

RAS

Register offset

0xFA8

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1xxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	xxxx
63	59	55	51	47	43	39	35	31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-139: ext_errdevaff bit assignments

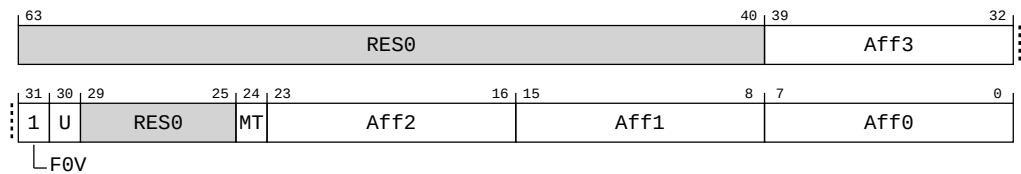


Table B-280: ERRDEVAFF bit descriptions

Bits	Name	Description	Reset
[63:40]	RES0	Reserved	RES0
[39:32]	Aff3	PE affinity level 3. The AArch64-MPIDR_EL1.Aff3 field, viewed from the highest Exception level of the associated PE or PEs.	8 { x }
[31]	FOV	Indicates that the ERRDEVAFF.Aff0 field is valid. 0b1 ERRDEVAFF.Aff0 is valid, and the PE affinity is at level 0.	0b1
[30]	U	Uniprocessor. The AArch64-MPIDR_EL1.U field, viewed from the highest Exception level of the associated PE.	x
[29:25]	RES0	Reserved	RES0
[24]	MT	Multithreaded. The AArch64-MPIDR_EL1.MT field, viewed from the highest Exception level of the associated PE.	x
[23:16]	Aff2	PE affinity level 2. The AArch64-MPIDR_EL1.Aff2 field, viewed from the highest Exception level of the associated PE or PEs.	8 { x }
[15:8]	Aff1	PE affinity level 1. The AArch64-MPIDR_EL1.Aff1 field, viewed from the highest Exception level of the associated PE or PEs.	8 { x }
[7:0]	Aff0	PE affinity level 0. The AArch64-MPIDR_EL1.Aff0 field, viewed from the highest Exception level of the associated PE.	8 { x }

Accessibility

Component	Offset	Instance	Range
RAS	0xFA8	ERRDEVAFF	None

This interface is accessible as follows:

RO

B.7.15 ERRDEVARCH, Device Architecture Register

Provides discovery information for the component.

Configurations

ERRDEVARCH is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

Register offset

0xFBC

Access type

RO

Reset value

0100 0111 0111 0001 0000 1010 0000 0000

Bit descriptions

Figure B-140: ext_errdevarch bit assignments

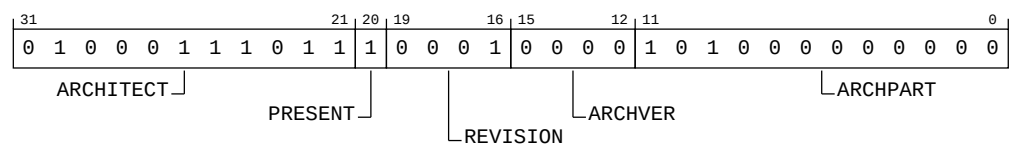


Table B-282: ERRDEVARCH bit descriptions

Bits	Name	Description	Reset
[31:21]	ARCHITECT	Architect. Defines the architect of the component. Bits [31:28] are the JEP106 continuation code (JEP106 bank ID, minus 1) and bits [27:21] are the JEP106 ID code. 0b01000111011 JEP106 continuation code 0x4, ID code 0x3B. Arm Limited. Other values are defined by the JEDEC JEP106 standard. This field reads as 0x23B.	0b01000111011
[20]	PRESENT	DEVARCH Present. Defines that the DEVARCH register is present. 0b1 Device Architecture information present.	0b1

Bits	Name	Description	Reset
[19:16]	REVISION	Revision. Defines the architecture revision of the component. 0b0001 RAS System Architecture v1.1. As 0b0000 and also: • Simplifies ext-ERR<n>STATUS. • Adds support for additional ERR<n>MISC<m> registers. • Adds support for the optional RAS Timestamp Extension. • Adds support for the optional Common Fault Injection Model Extension.	0b0001
[15:12]	ARCHVER	Architecture Version. Defines the architecture version of the component. 0b0000 RAS System Architecture v1. All other values are reserved. ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHVER is ARCHID[15:12]. This field reads as 0b0000.	0b0000
[11:0]	ARCHPART	Architecture Part. Defines the architecture of the component. 0b101000000000 RAS System Architecture. ARCHVER and ARCHPART are also defined as a single field, ARCHID, so that ARCHPART is ARCHID[11:0]. This field reads as 0xA00.	0xA00

Accessibility

Component	Offset	Instance	Range
RAS	0xFBC	ERRDEVARCH	None

This interface is accessible as follows:

RO

B.7.16 ERRDEVID, Device Configuration Register

Provides discovery information for the component.

Configurations

ERRDEVID is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

Register offset

0xFC8

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	0000	0000	0000	0001
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-141: ext_errdevid bit assignments

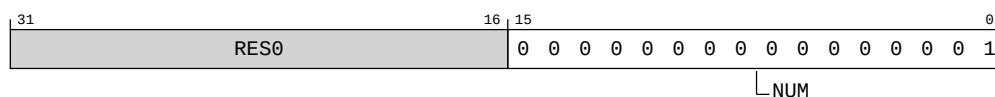


Table B-284: ERRDEVID bit descriptions

Bits	Name	Description	Reset
[31:16]	RES0	Reserved	RES0
[15:0]	NUM	Highest numbered index of the error records in this group, plus one. Each implemented record is owned by a node. A node might own multiple records. 0b0000000000000001 One record present.	0x0001

Accessibility

Component	Offset	Instance	Range
RAS	0xFC8	ERRDEVID	None

This interface is accessible as follows:

RO

B.7.17 ERRPIDR4, Peripheral Identification Register 4

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRPIDR4 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

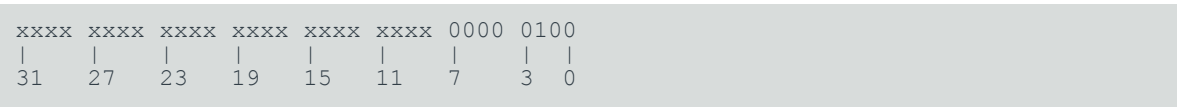
Register offset

0xFD0

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-142: ext_errpidr4 bit assignments

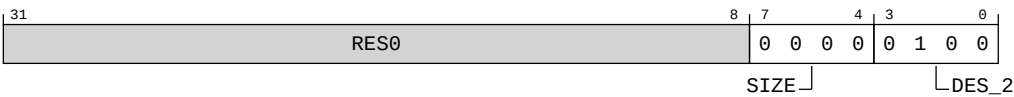


Table B-286: ERRPIDR4 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:4]	SIZE	Size of the component. The distance from the start of the address space used by this component to the end of the component identification registers. A value of 0b0000 means one of the following is true: - The component uses a single 4KB block. - The component uses an IMPLEMENTATION DEFINED number of 4KB blocks. Any other value means the component occupies $2^{\text{ERRPIDR4.SIZE}}$ 4KB blocks. 0b0000 The component uses a single 4KB block	0b0000
[3:0]	DES_2	Designer, JEP106 continuation code. This is the JEDEC-assigned JEP106 bank identifier for the designer of the component, minus 1. The code identifies the designer of the component, which might not be the same as the implementer of the device containing the component. To obtain a number, or to see the assignment of these codes, contact JEDEC http://www.jedec.org. 0b0100 Arm Limited	0b0100

Accessibility

Component	Offset	Instance	Range
RAS	0xFD0	ERRPIDR4	None

This interface is accessible as follows:

RO

B.7.18 ERRPIDR0, Peripheral Identification Register 0

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRPIDR0 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

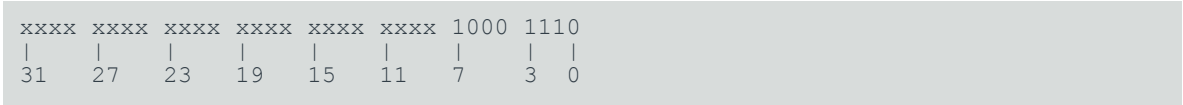
RAS

Register offset

0xFE0

Access type
RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-143: ext_errpidr0 bit assignments



Table B-288: ERRPIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PART_0	Part number, bits [7:0]. The part number is selected by the designer of the component. The designer chooses whether to use a 12-bit or a 16-bit part number: - If a 12-bit part number is used, it is stored in ext-ERRPIDR1.PART_1 and ERRPIDR0.PART_0. There are 8 bits, ext-ERRPIDR2.REVISION and ext-ERRPIDR3.REVAND, available to define the revision of the component. - If a 16-bit part number is used, it is stored in ext-ERRPIDR2.PART_2, ext-ERRPIDR1.PART_1 and ERRPIDR0.PART_0. There are 4 bits, ext-ERRPIDR3.REVISION, available to define the revision of the component. 0b10001110 Neoverse N3	0x8E

Accessibility

Component	Offset	Instance	Range
RAS	0xFE0	ERRPIDR0	None

This interface is accessible as follows:

RO

B.7.19 ERRPIDR1, Peripheral Identification Register 1

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRPIDR1 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

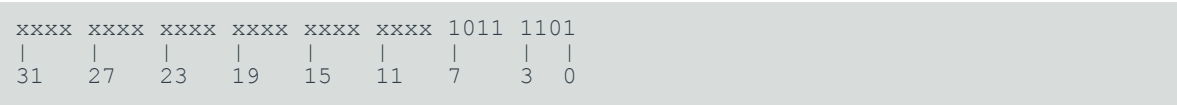
Register offset

0xFE4

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-144: ext_errpidr1 bit assignments

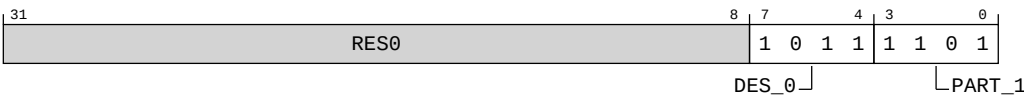


Table B-290: ERRPIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:4]	DES_0	Designer, JEP106 identification code, bits [3:0]. ERRPIDR1.DES_0 and ext-ERRPIDR2.DES_1 together form the JEDEC-assigned JEP106 identification code for the designer of the component. The parity bit in the JEP106 identification code is not included. The code identifies the designer of the component, which might not be not the same as the implementer of the device containing the component. To obtain a number, or to see the assignment of these codes, contact JEDEC http://www.jedec.org. 0b1011 Arm Limited	0b1011
[3:0]	PART_1	Part number, bits [11:8]. The part number is selected by the designer of the component. The designer chooses whether to use a 12-bit or a 16-bit part number: - If a 12-bit part number is used, it is stored in ERRPIDR1.PART_1 and ext-ERRPIDR0.PART_0. There are 8 bits, ext-ERRPIDR2.REVISION and ext-ERRPIDR3.REVAND, available to define the revision of the component. - If a 16-bit part number is used, it is stored in ext-ERRPIDR2.PART_2, ERRPIDR1.PART_1 and ext-ERRPIDR0.PART_0. There are 4 bits, ext-ERRPIDR3.REVISION, available to define the revision of the component. 0b1101 Neoverse N3	0b1101

Accessibility

Component	Offset	Instance	Range
RAS	0xFE4	ERRPIDR1	None

This interface is accessible as follows:

RO

B.7.20 ERRPIDR2, Peripheral Identification Register 2

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRPIDR2 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

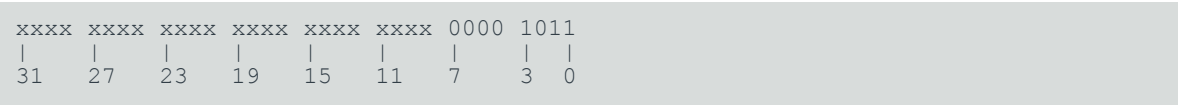
Register offset

0xFE8

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-145: ext_errpidr2 bit assignments

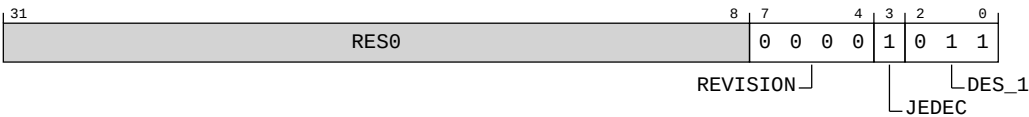


Table B-292: ERRPIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVISION	Component major revision. ERRPIDR2.REVISION and ext-ERRPIDR3.REVAND together form the revision number of the component, with ERRPIDR2.REVISION being the most significant part and ext-ERRPIDR3.REVAND the least significant part. When a component is changed, ERRPIDR2.REVISION or ext-ERRPIDR3.REVAND are increased to ensure that software can differentiate the different revisions of the component. ext-ERRPIDR3.REVAND should be set to 0b0000 when ERRPIDR2.REVISION is increased. 0b0000	0b0000
[3]	JEDEC	JEDEC-assigned JEP106 implementer code is used. 0b1	0b1
[2:0]	DES_1	Designer, JEP106 identification code, bits [6:4]. ext-ERRPIDR1.DES_0 and ERRPIDR2.DES_1 together form the JEDEC-assigned JEP106 identification code for the designer of the component. The parity bit in the JEP106 identification code is not included. The code identifies the designer of the component, which might not be not the same as the implementer of the device containing the component. To obtain a number, or to see the assignment of these codes, contact JEDEC http://www.jedec.org . 0b011 Arm Limited	0b011

Accessibility

Component	Offset	Instance	Range
RAS	0xFE8	ERRPIDR2	None

This interface is accessible as follows:

RO

B.7.21 ERRPIDR3, Peripheral Identification Register 3

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRPIDR3 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

Register offset

0xFEC

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0001	0000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-146: ext_errpidr3 bit assignments



Table B-294: ERRPIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVAND	Component minor revision. ext-ERRPIDR2.REVISION and ERRPIDR3.REVAND together form the revision number of the component, with ext-ERRPIDR2.REVISION being the most significant part and ERRPIDR3.REVAND the least significant part. When a component is changed, ext-ERRPIDR2.REVISION or ERRPIDR3.REVAND are increased to ensure that software can differentiate the different revisions of the component. ERRPIDR3.REVAND should be set to 0b0000 when ext-ERRPIDR2.REVISION is increased. 0b0001 r0p1	0b0001
[3:0]	CMOD	Customer Modified. Indicates the component has been modified. A value of 0b0000 means the component is not modified from the original design. Any other value means the component has been modified in an IMPLEMENTATION DEFINED way. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
RAS	0xFEC	ERRPIDR3	None

This interface is accessible as follows:

RO

B.7.22 ERRCIDR0, Component Identification Register 0

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRCIDR0 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

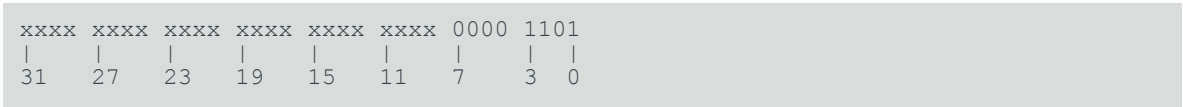
RAS

Register offset

0xFF0

Access type
RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-147: ext_errcidr0 bit assignments

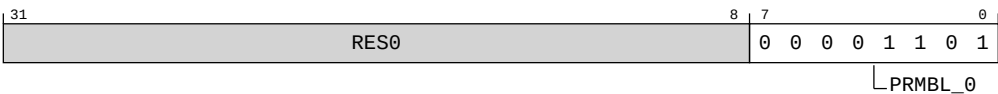


Table B-296: ERRCIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_0	Component identification preamble, segment 0. 0b00001101	0x0D

Accessibility

Component	Offset	Instance	Range
RAS	0xFF0	ERRCIDR0	None

This interface is accessible as follows:

RO

B.7.23 ERRCIDR1, Component Identification Register 1

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRCIDR1 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

Register offset

0xFF4

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-148: ext_errcidr1 bit assignments



Table B-298: ERRCIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	CLASS	Component class. 0b1111 Generic peripheral with IMPLEMENTATION DEFINED register layout. Other values are defined by the CoreSight Architecture. This field reads as 0xF.	0b1111
[3:0]	PRMBL_1	Component identification preamble, segment 1. 0b0000	0b0000

Accessibility

Component	Offset	Instance	Range
RAS	0xFF4	ERRCIDR1	None

This interface is accessible as follows:

RO

B.7.24 ERRCIDR2, Component Identification Register 2

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRCIDR2 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

Register offset

0xFF8

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0101
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-149: ext_errcidr2 bit assignments



Table B-300: ERRCIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_2	Component identification preamble, segment 2. 0b00000101	0x05

Accessibility

Component	Offset	Instance	Range
RAS	0xFF8	ERRCIDR2	None

This interface is accessible as follows:

RO

B.7.25 ERRCIDR3, Component Identification Register 3

Provides discovery information about the component.

For more information, see *About the Peripheral identification scheme* in the [Arm® Architecture Reference Manual for A-profile architecture](#).

Configurations

ERRCIDR3 is implemented only as part of a memory-mapped group of error records.

Attributes

Width

32

Component

RAS

Register offset

0xFFC

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1011	0001
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-150: ext_errcidr3 bit assignments

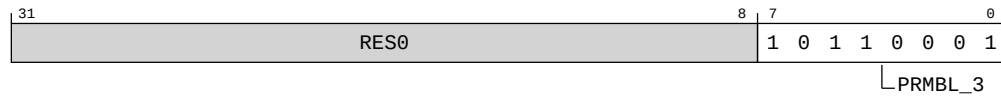


Table B-302: ERRCIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_3	Component identification preamble, segment 3. 0b10110001	0xB1

Accessibility

Component	Offset	Instance	Range
RAS	0xFFC	ERRCIDR3	None

This interface is accessible as follows:

RO

B.8 External ROM table registers summary

The summary table provides an overview of all memory-mapped ROM table registers in the core. For more information about a register, click the register name in the table.

If a register does not have a link in the summary table, you can find more information about this Architecturally defined register in the [Arm® Architecture Reference Manual for A-profile architecture](#).

For registers without a listed reset value, refer to the individual field resets documented on the register description pages or to the [Arm® Architecture Reference Manual for A-profile architecture](#).

Table B-304: ROM table registers summary

Offset	Name	Reset	Width	Description
0x0	ROMENTRY0	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0x4	ROMENTRY1	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0x8	ROMENTRY2	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0xC	ROMENTRY3	See individual bit resets.	32-bit	Class 0x9 ROM Table Entries
0xFB8	AUTHSTATUS	See individual bit resets.	32-bit	Authentication Status Register
0xFBC	DEVARCH	See individual bit resets.	32-bit	Device Architecture Register
0xFD0	PIDR4	See individual bit resets.	32-bit	Peripheral Identification Register 4
0xFE0	PIDR0	See individual bit resets.	32-bit	Peripheral Identification Register 0

Offset	Name	Reset	Width	Description
0xFE4	PIDR1	See individual bit resets.	32-bit	Peripheral Identification Register 1
0xFE8	PIDR2	See individual bit resets.	32-bit	Peripheral Identification Register 2
0xFEC	PIDR3	See individual bit resets.	32-bit	Peripheral Identification Register 3
0xFF0	CIDR0	See individual bit resets.	32-bit	Component Identification Register 0
0xFF4	CIDR1	See individual bit resets.	32-bit	Component Identification Register 1
0xFF8	CIDR2	See individual bit resets.	32-bit	Component Identification Register 2
0xFFC	CIDR3	See individual bit resets.	32-bit	Component Identification Register 3

B.8.1 ROMENTRY0, Class 0x9 ROM Table Entries

A Class 0x9 ROM Table contains up to 512 ROM Table entries. Each entry that is present, ext-ROMENTRY<n>, provides the address offset of the address space of one CoreSight component, component 0, along with information about its power domain.

The series of ROM Table entries starts at the base address of the Class 0x9 ROM Table:

- The first entry, entry 0, has offset 0x000.
- ext-ROMENTRY<n> has the offset 0x000 + 0x4, where $0 \leq n \leq 511$.
- If the number of components, 0, is lower than the maximum supported number, 512, the offsets of the ROM Table entries are in the following range:
 - ROM Table entries representing components have offsets from 0x000 to (0-1)×4.
 - The ext-ROMENTRY<n> at offset 0x4, which has a PRESENT field with the value 0b00, indicates the end of the ROM Table.
- If the number of components is equal to the maximum supported number, the ROM Table entries have offsets from 0x000 to 0x7FC. If a ROM Table entry is present at offset 0x7FC, its PRESENT field must have a value of either 0b00 or 0b11, and it must be interpreted as the final entry of the ROM Table, even if its PRESENT field has the value 0b11.

A component that requires differentiation between external and internal accesses may provide two views, one for internal and one for external accesses. For these components, Arm strongly recommends that ROM Tables provide a pointer only to the external view.

Configurations

If ext-DEVID.FORMAT has the value 0x0, ext-ROMENTRY<n> are 512 32-bit registers.

If ext-DEVID.FORMAT has the value 0x1, ext-ROMENTRY<n> are 256 64-bit registers.

Attributes

Width

32

Component

ROM table

Register offset

0x0

Access type

Read

R

Write

RESERVED

Reset value

0000	0000	0000	0001	0000	xxx0	0000	x011
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-151: ext_romentry0 bit assignments

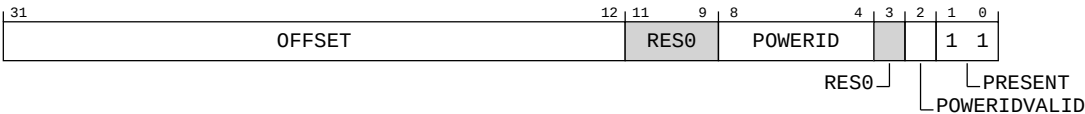


Table B-305: ROMENTRY0 bit descriptions

Bits	Name	Description	Reset
[31:12]	OFFSET	The component address, relative to the base address of this ROM Table. The component address is calculated using the following equation: Component Address = ROM Table Base Address + (OFFSET << 12). If a component occupies more than a single 4KB block, OFFSET points to the 4KB block which contains the Peripheral ID and Component ID registers for the component. Negative values of OFFSET are permitted, using two's complement. 0b000000000000000010000 Core Debug	0x00010
[11:9]	RES0	Reserved	RES0
[8:4]	POWERID	The power domain ID of the component. This field supports up to 32 power domains using values 0x00 to 0x1F.	0b00000
[3]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[2]	POWERIDVALID	Indicates if the Power domain ID field contains a Power domain ID. 0b0 A power domain ID is not provided.	0b0
[1:0]	PRESENT	Indicates whether an entry is present at this location in the ROM Table. 0b11 The ROM Entry is present.	0b11

Accessibility

A ROM Table does not have to use power domain IDs. If none of the ROM Table entries provides a power domain ID, all the components that pointed to by the ROM Table are in the same power domain as the ROM Table.

This interface is accessible as follows:

RO

B.8.2 ROMENTRY1, Class 0x9 ROM Table Entries

A Class 0x9 ROM Table contains up to 512 ROM Table entries. Each entry that is present, ext-ROMENTRY<n>, provides the address offset of the address space of one CoreSight component, component 1, along with information about its power domain.

The series of ROM Table entries starts at the base address of the Class 0x9 ROM Table:

- The first entry, entry 0, has offset 0x000.
- ext-ROMENTRY<n> has the offset $0x000 + 1 \times 4$, where $0 \leq n \leq 511$.
- If the number of components, 1, is lower than the maximum supported number, 512, the offsets of the ROM Table entries are in the following range:
 - ROM Table entries representing components have offsets from 0x000 to $(1-1) \times 4$.
 - The ext-ROMENTRY<n> at offset 1×4 , which has a PRESENT field with the value 0b00, indicates the end of the ROM Table.
- If the number of components is equal to the maximum supported number, the ROM Table entries have offsets from 0x000 to 0x7FC. If a ROM Table entry is present at offset 0x7FC, its PRESENT field must have a value of either 0b00 or 0b11, and it must be interpreted as the final entry of the ROM Table, even if its PRESENT field has the value 0b11.

A component that requires differentiation between external and internal accesses may provide two views, one for internal and one for external accesses. For these components, Arm strongly recommends that ROM Tables provide a pointer only to the external view.

Configurations

If ext-DEVID.FORMAT has the value 0x0, ext-ROMENTRY<n> are 512 32-bit registers.

If ext-DEVID.FORMAT has the value 0x1, ext-ROMENTRY<n> are 256 64-bit registers.

Attributes

Width

32

Component

ROM table

Register offset

0x4

Access type

Read

R

Write

RESERVED

Reset value

0000	0000	0000	0010	0000	xxx0	0000	x011
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-152: ext_romentry1 bit assignments

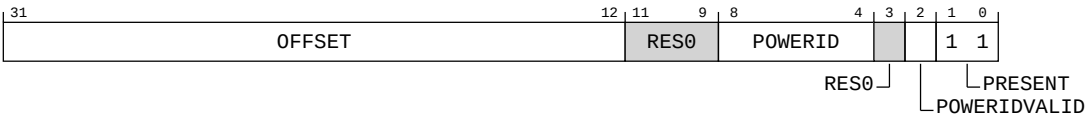


Table B-306: ROMENTRY1 bit descriptions

Bits	Name	Description	Reset
[31:12]	OFFSET	The component address, relative to the base address of this ROM Table. The component address is calculated using the following equation: Component Address = ROM Table Base Address + (OFFSET << 12). If a component occupies more than a single 4KB block, OFFSET points to the 4KB block which contains the Peripheral ID and Component ID registers for the component. Negative values of OFFSET are permitted, using two's complement. 0b000000000000000100000 Core PMU	0x00020

Bits	Name	Description	Reset
[11:9]	RES0	Reserved	RES0
[8:4]	POWERID	The power domain ID of the component. This field supports up to 32 power domains using values 0x00 to 0x1F.	0b00000
[3]	RES0	Reserved	RES0
[2]	POWERIDVALID	Indicates if the Power domain ID field contains a Power domain ID. 0b0 A power domain ID is not provided.	0b0
[1:0]	PRESENT	Indicates whether an entry is present at this location in the ROM Table. 0b11 The ROM Entry is present.	0b11

Accessibility

A ROM Table does not have to use power domain IDs. If none of the ROM Table entries provides a power domain ID, all the components that pointed to by the ROM Table are in the same power domain as the ROM Table.

This interface is accessible as follows:

RO

B.8.3 ROMENTRY2, Class 0x9 ROM Table Entries

A Class 0x9 ROM Table contains up to 512 ROM Table entries. Each entry that is present, ext-ROMENTRY<n>, provides the address offset of the address space of one CoreSight component, component 2, along with information about its power domain.

The series of ROM Table entries starts at the base address of the Class 0x9 ROM Table:

- The first entry, entry 0, has offset 0x000.
- ext-ROMENTRY<n> has the offset $0x000 + 2 \times 4$, where $0 \leq n \leq 511$.
- If the number of components, 2, is lower than the maximum supported number, 512, the offsets of the ROM Table entries are in the following range:
 - ROM Table entries representing components have offsets from 0x000 to $(2-1) \times 4$.
 - The ext-ROMENTRY<n> at offset 2×4 , which has a PRESENT field with the value 0b00, indicates the end of the ROM Table.
- If the number of components is equal to the maximum supported number, the ROM Table entries have offsets from 0x000 to 0x7FC. If a ROM Table entry is present at offset 0x7FC, its PRESENT field must have a value of either 0b00 or 0b11, and it must be interpreted as the final entry of the ROM Table, even if its PRESENT field has the value 0b11.

A component that requires differentiation between external and internal accesses may provide two views, one for internal and one for external accesses. For these components, Arm strongly recommends that ROM Tables provide a pointer only to the external view.

Configurations

If ext-DEVID.FORMAT has the value 0x0, ext-ROMENTRY<n> are 512 32-bit registers.

If ext-DEVID.FORMAT has the value 0x1, ext-ROMENTRY<n> are 256 64-bit registers.

Attributes

Width

32

Component

ROM table

Register offset

0x8

Access type

Read

R

Write

RESERVED

Reset value

0000	0000	0000	0011	0000	xxx0	0000	x011
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-153: ext_romentry2 bit assignments

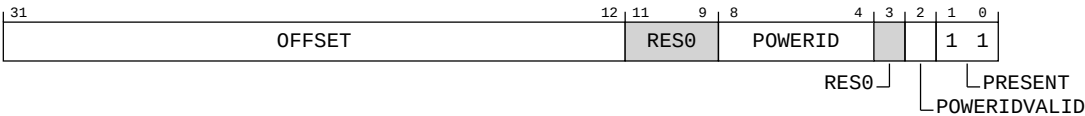


Table B-307: ROMENTRY2 bit descriptions

Bits	Name	Description	Reset
[31:12]	OFFSET	The component address, relative to the base address of this ROM Table. The component address is calculated using the following equation: $\text{Component Address} = \text{ROM Table Base Address} + (\text{OFFSET} \ll 12).$ If a component occupies more than a single 4KB block, OFFSET points to the 4KB block which contains the Peripheral ID and Component ID registers for the component. Negative values of OFFSET are permitted, using two's complement. 0b0000000000000000110000 Core trace unit	0x00030
[11:9]	RES0	Reserved	RES0
[8:4]	POWERID	The power domain ID of the component. This field supports up to 32 power domains using values 0x00 to 0x1F.	0b00000
[3]	RES0	Reserved	RES0
[2]	POWERIDVALID	Indicates if the Power domain ID field contains a Power domain ID. 0b0 A power domain ID is not provided.	0b0
[1:0]	PRESENT	Indicates whether an entry is present at this location in the ROM Table. 0b11 The ROM Entry is present.	0b11

Accessibility

A ROM Table does not have to use power domain IDs. If none of the ROM Table entries provides a power domain ID, all the components that pointed to by the ROM Table are in the same power domain as the ROM Table.

This interface is accessible as follows:

RO

B.8.4 ROMENTRY3, Class 0x9 ROM Table Entries

A Class 0x9 ROM Table contains up to 512 ROM Table entries. Each entry that is present, ext-ROMENTRY<n>, provides the address offset of the address space of one CoreSight component, component 3, along with information about its power domain.

The series of ROM Table entries starts at the base address of the Class 0x9 ROM Table:

- The first entry, entry 0, has offset 0x000.
- ext-ROMENTRY<n> has the offset $0x000 + 3 \times 4$, where $0 \leq n \leq 511$.
- If the number of components, 3, is lower than the maximum supported number, 512, the offsets of the ROM Table entries are in the following range:
 - ROM Table entries representing components have offsets from 0x000 to $(3-1) \times 4$.

- The ext-ROMENTRY<n> at offset 3×4, which has a PRESENT field with the value 0b00, indicates the end of the ROM Table.
- If the number of components is equal to the maximum supported number, the ROM Table entries have offsets from 0x000 to 0x7FC. If a ROM Table entry is present at offset 0x7FC, its PRESENT field must have a value of either 0b00 or 0b11, and it must be interpreted as the final entry of the ROM Table, even if its PRESENT field has the value 0b11.

A component that requires differentiation between external and internal accesses may provide two views, one for internal and one for external accesses. For these components, Arm strongly recommends that ROM Tables provide a pointer only to the external view.

Configurations

If ext-DEVID.FORMAT has the value 0x0, ext-ROMENTRY<n> are 512 32-bit registers.
If ext-DEVID.FORMAT has the value 0x1, ext-ROMENTRY<n> are 256 64-bit registers.

Attributes

Width

32

Component

ROM table

Register offset

0xC

Access type

Read

R

Write

RESERVED

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxx0	0000	x0xx
31	27	23	19	15	11	7	3 0



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-154: ext_romentry3 bit assignments

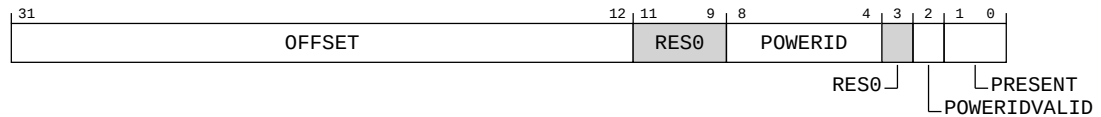


Table B-308: ROMENTRY3 bit descriptions

Bits	Name	Description	Reset
[31:12]	OFFSET	The component address, relative to the base address of this ROM Table. The component address is calculated using the following equation: $\text{Component Address} = \text{ROM Table Base Address} + (\text{OFFSET} \ll 12).$ If a component occupies more than a single 4KB block, OFFSET points to the 4KB block which contains the Peripheral ID and Component ID registers for the component. Negative values of OFFSET are permitted, using two's complement. 0b000000000000000000000000 ROM Entry is not present. 0b000000000000000001000000 Core ELA	20 {x}
[11:9]	RES0	Reserved	RES0
[8:4]	POWERID	The power domain ID of the component. This field supports up to 32 power domains using values 0x00 to 0x1F.	0b00000
[3]	RES0	Reserved	RES0
[2]	POWERIDVALID	Indicates if the Power domain ID field contains a Power domain ID. 0b0 A power domain ID is not provided.	0b0
[1:0]	PRESENT	Indicates whether an entry is present at this location in the ROM Table. 0b00 The ROM entry is not present, and this ext-ROMENTRY3 is the final entry in the ROM Table. If PRESENT has this value, all other fields in this ext-ROMENTRY3 must be zero. 0b11 The ROM Entry is present.	The reset values can be the following: 0b00, 0b11, respective to the value.

Accessibility

A ROM Table does not have to use power domain IDs. If none of the ROM Table entries provides a power domain ID, all the components that pointed to by the ROM Table are in the same power domain as the ROM Table.

This interface is accessible as follows:

RO

B.8.5 DEVARCH, Device Architecture Register

Identifies the architect and architecture of a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

Register offset

0xFBC

Access type

RO

Reset value

0100 0111 0111 0000 0000 1010 1111 0111

Bit descriptions

Figure B-155: ext_devarch bit assignments

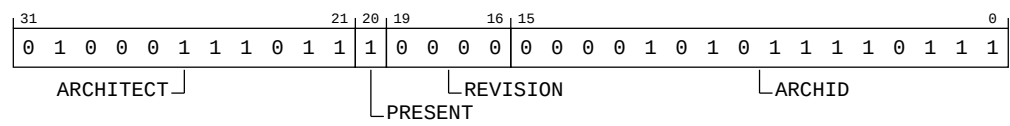


Table B-309: DEVARCH bit descriptions

Bits	Name	Description	Reset
[31:21]	ARCHITECT	Architect. 0b01000111011 JEP106 continuation code 0x4, ID code 0x3B. Arm Limited.	0b01000111011
[20]	PRESENT	Present. 0b1 DEVARCH information present.	0b1
[19:16]	REVISION	Revision. 0b0000 Revision 0.	0b0000

Bits	Name	Description	Reset
[15:0]	ARCHID	Architecture ID. 0b0000101011110111 ROM Table v0. The debug tool must inspect ext-DEVTYPE and ext-DEVID to determine further information about the ROM Table.	0x0AF7

Accessibility

This interface is accessible as follows:

RO

B.8.6 PIDR4, Peripheral Identification Register 4

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

Register offset

0xFD0

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	0000	0100
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-156: ext_pidr4 bit assignments

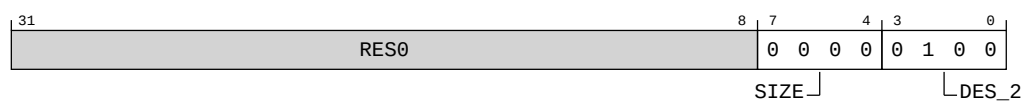


Table B-310: PIDR4 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	SIZE	Size of the component. RAZ . Log ₂ of the number of 4KB pages from the start of the component to the end of the component ID registers. 0b0000 A ROM Table occupies a single 4KB block of memory.	0b0000
[3:0]	DES_2	Designer, JEP106 continuation code, least significant nibble. For Arm Limited, this field is 0b0100. 0b0100 Arm Limited	0b0100

Accessibility

This interface is accessible as follows:

RO

B.8.7 PIDR0, Peripheral Identification Register 0

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

Register offset

0xFE0

Access type

RO

Reset value



312723191511730



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-157: ext_pidr0 bit assignments



Table B-311: PIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PART_0	Part number, least significant byte. 0b10001110 Neoverse N3	0x8E

Accessibility

This interface is accessible as follows:

RO

B.8.8 PIDR1, Peripheral Identification Register 1

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

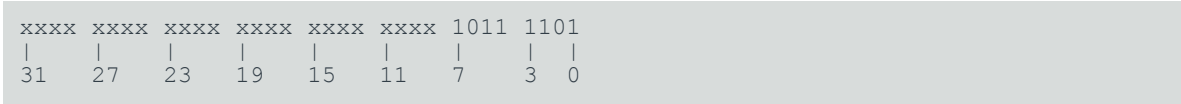
Register offset

0xFE4

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-158: ext_pidr1 bit assignments



Table B-312: PIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	DES_0	Designer, least significant nibble of JEP106 ID code. For Arm Limited, this field is 0b1011. 0b1011 Arm Limited	0b1011
[3:0]	PART_1	Part number, most significant nibble. 0b1101 Neoverse N3	0b1101

Accessibility

This interface is accessible as follows:

RO

B.8.9 PIDR2, Peripheral Identification Register 2

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

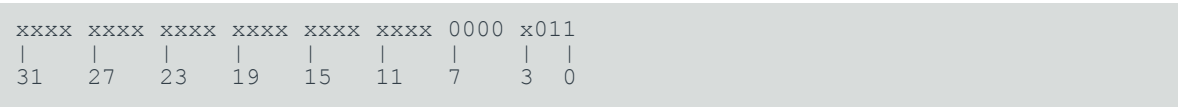
Register offset

0xFE8

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-159: ext_pidr2 bit assignments

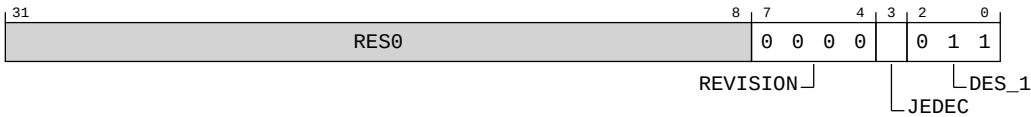


Table B-313: PIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVISION	Part major revision. Parts can also use this field to extend Part number to 16-bits. 0b0000 rOp1	0b0000
[3]	JEDEC	RAO . Indicates a JEP106 identity code is used.	x
[2:0]	DES_1	Designer, most significant bits of JEP106 ID code. For Arm Limited, this field is 0b011. 0b011 Arm Limited	0b011

Accessibility

This interface is accessible as follows:

RO

B.8.10 PIDR3, Peripheral Identification Register 3

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

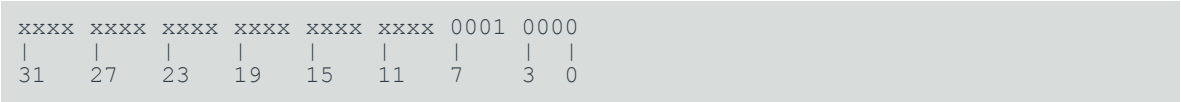
Register offset

0xFEC

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-160: ext_pidr3 bit assignments

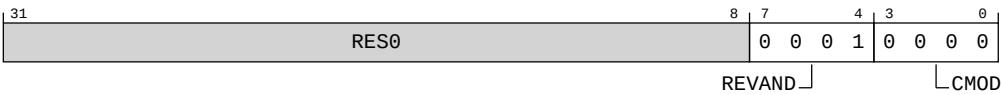


Table B-314: PIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	REVAND	Part minor revision. Parts using ext-PIDR2.REVISION as an extension to the Part number must use this field as a major revision number. 0b0001 rOp1	0b0001
[3:0]	CMOD	Customer modified. Indicates someone other than the Designer has modified the component. 0b0000	0b0000

Accessibility

This interface is accessible as follows:

RO

B.8.11 CIDR0, Component Identification Register 0

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

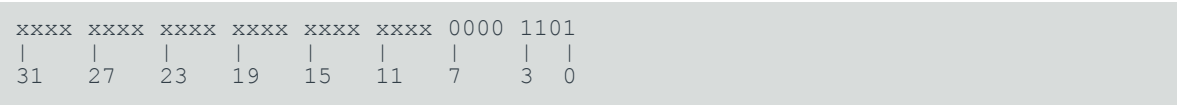
Register offset

0xFF0

Access type

RO

Reset value



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-161: ext_cidr0 bit assignments

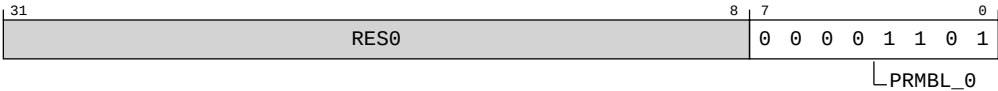


Table B-315: CIDR0 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0

Bits	Name	Description	Reset
[7:0]	PRMBL_0	CoreSight component identification preamble. 0b00001101 CoreSight component identification preamble.	0x0D

Accessibility

Component	Offset	Instance	Range
ROM table	0xFF0	CIDR0	None

This interface is accessible as follows:

RO

B.8.12 CIDR1, Component Identification Register 1

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

Register offset

0xFF4

Access type

RO

Reset value

xxxx	xxxx	xxxx	xxxx	xxxx	xxxx	1001	0000
31	27	23	19	15	11	7	3 0



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-162: ext_cidr1 bit assignments

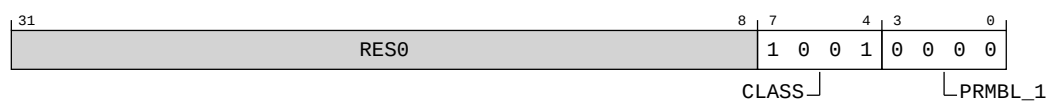


Table B-317: CIDR1 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:4]	CLASS	CoreSight component class. 0b1001 CoreSight component.	0b1001
[3:0]	PRMBL_1	CoreSight component identification preamble. 0b0000 CoreSight component identification preamble.	0b0000

Accessibility

This interface is accessible as follows:

RO

B.8.13 CIDR2, Component Identification Register 2

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

Register offset

0xFF8

Access type

RO

Reset value



312723191511730



Note

Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-163: ext_cidr2 bit assignments

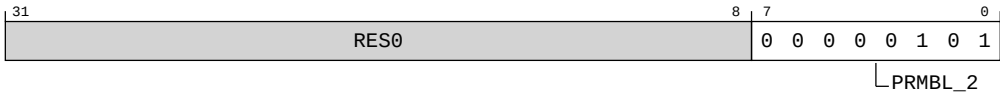


Table B-318: CIDR2 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_2	CoreSight component identification preamble. 0b000000101 CoreSight component identification preamble.	0x05

Accessibility

This interface is accessible as follows:

RO

B.8.14 CIDR3, Component Identification Register 3

Provide information to identify a CoreSight component.

Configurations

This register is available in all configurations.

Attributes

Width

32

Component

ROM table

Register offset

0xFFC

Access type

RO

Reset value



Where the reset reads xxxx, see individual bits.

Bit descriptions

Figure B-164: ext_cidr3 bit assignments



Table B-319: CIDR3 bit descriptions

Bits	Name	Description	Reset
[31:8]	RES0	Reserved	RES0
[7:0]	PRMBL_3	CoreSight component identification preamble. 0b10110001 CoreSight component identification preamble.	0xB1

Accessibility

This interface is accessible as follows:

RO

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant

export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in the Arm documents.

Product status

All products and Services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is Final, that is for a developed product.

Product revision status

This product is rOp1, which indicates the revision status of the product described in this manual, where:

- r (value)**Identifies the major revision of the product, for example, r1.
- p (value)**Identifies the minor revision or modification status of the product, for example, p2.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0001-03	25 February 2025	Non-Confidential	First release for rOp1
0000-02	21 February 2024	Non-Confidential	Second early access release for rOp0
0000-01	29 September 2023	Confidential	First early access release for rOp0

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in [Document release information](#) on page 897.

Table 2: Issue 0000-01

Change	Location
First Confidential early access release for r0p0	-

Table 3: Differences between issue 0000-01 and issue 0000-02

Change	Location
Second Non-Confidential early access release for r0p0	-
Editorial changes	Throughout document
Updated product name	Throughout document
Added information on tables content	1.4 Supported standards and specifications on page 20
Updated description	4.4.6 Warm reset mode on page 44
Added information to the description of the sw_force_pcsr_en field	A.1.12 IMP_CPUECTLR2_EL1, CPU Extended Control Register on page 198
Updated register description	A.2.2 IMP_CPUSYNCASSISTCR_EL1, Synchronization Assist Configuration Register on page 320
Modified value and description of the OF field	B.7.9 ERR0PFGF, Pseudo-fault Generation Feature Register on page 846

Table 4: Differences between issue 0000-02 and issue 0001-03

Change	Location
First Non-Confidential release for r0p1	-
Editorial changes	Throughout document
Inserted some content in a Note and added a Caution	4.4.6 Warm reset mode on page 44
Updated Transient hint description	8.2 Support for memory types on page 68
Values updated to consider r0p1 evolution	17.6 CoreSight component identification on page 104
Values updated to consider r0p1 evolution	17.7 CTI register identification values on page 105
Updated descriptions for the following events: 0x000D, 0x0010, 0x0015, 0x001B	18.1 Performance monitors events on page 110
Values updated in some registers to consider r0p1 evolution	A. AArch64 registers on page 160
Updated TR field description	A.1.40 IMP_MMU_DATA0_EL3, RAMINDEX TLB Data register 0 on page 269
Updated fields range and description	A.1.41 IMP_MMU_DATA1_EL3, RAMINDEX TLB Data register 1 on page 273
Updated fields range and description	A.1.42 IMP_MMU_DATA2_EL3, RAMINDEX TLB Data register 2 on page 279
Updated fields range to increase PA field size and update description	A.1.43 IMP_DSIDE_DATA0_EL3, RAMINDEX L1D Data register 0 on page 283
Updated description of the MPMM_EN field	A.2.4 IMP_CPUMPMCR_EL3, Global MPMM Configuration Register on page 325
Values updated in some registers to consider r0p1 evolution	B. External registers on page 609
Updated description of the MPMM_EN field	B.1.2 CPUMPMCR, Global MPMM Configuration Register on page 611
Updated description of the PC field	B.4.1 PMPCCSR, Snapshot Program Counter Sample Register on page 652

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
<i>italic</i>	Citations.
bold	Terms in descriptive lists, where appropriate.
monospace	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
monospace <u>underline</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<and>	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>
SMALL CAPITALS	Terms that have specific technical meanings as defined in the <i>Arm® Glossary</i> . For example, IMPLEMENTATION DEFINED , IMPLEMENTATION SPECIFIC , UNKNOWN , and UNPREDICTABLE .



We recommend the following. If you do not follow these recommendations your system might not work.



Your system requires the following. If you do not follow these requirements your system will not work.



You are at risk of causing permanent damage to your system or your equipment, or of harming yourself.



This information is important and needs your attention.



This information might help you perform a task in an easier, better, or faster way.



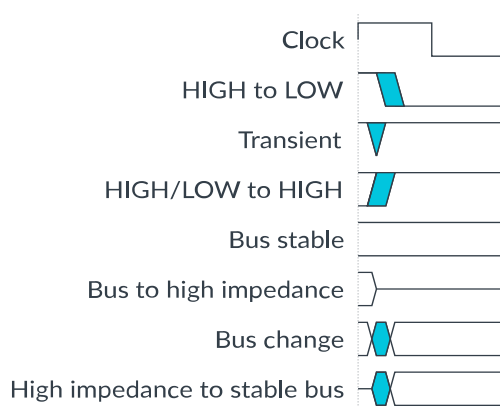
This information reminds you of something important relating to the current content.

Timing diagrams

The following figure explains the components used in timing diagrams. Variations, when they occur, have clear labels. You must not assume any timing information that is not explicit in the diagrams.

Shaded bus and signal areas are undefined, so the bus or signal can assume any value within the shaded area at that time. The actual level is unimportant and does not affect normal operation.

Figure 1: Key to timing diagram conventions



Signals

The signal conventions are:

Signal level

The level of an asserted signal depends on whether the signal is active-HIGH or active-LOW. Asserted means:

- HIGH for active-HIGH signals.
- LOW for active-LOW signals.

Lowercase n

At the start or end of a signal name, n denotes an active-LOW signal.

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Access to Arm documents depends on their confidentiality:

- Non-Confidential documents are available at developer.arm.com/documentation. Each document link in the following tables goes to the online version of the document.
- Confidential documents are available to licensees only through the product package.

Arm product resources	Document ID	Confidentiality
<i>Arm® Neoverse™ N3 Core Configuration and Integration Manual</i>	107998	Confidential
<i>Arm® Neoverse™ N3 Core Cryptographic Extension Technical Reference Manual</i>	107999	Non-Confidential
<i>Arm® Neoverse™ N3 Core (MP195) Release Note</i>	110281	Confidential
<i>Arm® DynamIQ™ Shared Unit-120 Configuration and Integration Manual</i>	102548	Confidential
<i>Arm® DynamIQ™ Shared Unit-120 Technical Reference Manual</i>	102547	Non-Confidential

Arm architecture and specifications	Document ID	Confidentiality
<i>AMBA® CHI Architecture Specification</i>	IHI 0050	Non-Confidential
<i>Arm® Architecture Reference Manual for A-profile architecture</i>	DDI 0487	Non-Confidential
<i>Arm® Memory System Resource Partitioning and Monitoring (MPAM) System Component Specification</i>	IHI 0099	Non-Confidential
<i>Arm® CoreSight™ Architecture Specification v3.0</i>	IHI 0029	Non-Confidential
<i>Arm® CoreSight™ ELA-600 Embedded Logic Analyzer Technical Reference Manual</i>	101088	Non-Confidential
<i>Arm® Generic Interrupt Controller Architecture Specification, GIC architecture version 3 and version 4</i>	IHI 0069	Non-Confidential
<i>Arm® Reliability, Availability, and Serviceability (RAS) System Architecture</i>	IHI 0100	Non-Confidential